Epidemiología: Nivel Avanzado

Instituto Nacional de Epidemiología Dr. Juan H. Jara

¡Les damos la bienvenida al curso!

Es un gusto recibirles en esta nueva edición del curso, diseñado para profundizar en el análisis de datos provenientes de distintos diseños de estudio mediante el uso de modelos estadísticos.

A lo largo del curso, abordaremos herramientas analíticas avanzadas que nos permitirán modelar adecuadamente la relación entre exposiciones y desenlaces en estudios transversales, de casos y controles, de cohortes y experimentales. Nos enfocaremos en la aplicación práctica de estos modelos, en la interpretación de resultados y en los supuestos que los sustentan, promoviendo una mirada crítica sobre su uso en la investigación epidemiológica.

Esperamos que este espacio sea una experiencia formativa enriquecedora y un nuevo impulso en su desarrollo como investigadoras e investigadores en salud.

El equipo docente

Christian Ballejo

Andrea Silva

Tamara Ricardo

Ramiro Dana Smith

Julia Maxwell

⚠ **Declaración de uso de IA generativa**

Se utilizaron herramientas de inteligencia artificial generativa (ChatGPT, versión GPT-5.3) para la revisión gramatical y ortográfica del manuscrito. El contenido final fue evaluado, validado y editado manualmente por los/as autores/as, quienes asumen la responsabilidad por su integridad.

Citar como

Ballejo, C., Silva, A., Ricardo, T., & Gauto, M. A. (2026). Epidemiología: Nivel Avanzado (Version v1.0.0). Zenodo. https://doi.org/10.5281/zenodo.23669104

Licencia

Este trabajo está distribuido bajo la licencia **Creative Commons Attribution 4.0 International (CC BY 4.0)**.

Parte I.

Unidad 1

1. Introducción a R

Figura 1.1.: Artwork por @allison_horst

> ⚠ Estos materiales, que forman parte del **Módulo 1 del Curso de Epidemiología Avanzada**, no pretenden cubrir exhaustivamente todos los temas relevantes de un curso de lenguaje R. Su objetivo es introducir únicamente aquellos aspectos más importantes y necesarios para su aplicación en los módulos siguientes.

1.1. ¿Qué es R?

El sitio oficial **r-project.org** define a R como *"un entorno de software libre para gráficos y computación estadística. Se compila y se ejecuta en una amplia variedad de plataformas UNIX, Windows y MacOS."*

Profundizando en su descripción, podemos decir que R es un *lenguaje de programación interpretado, orientado a objetos, multiplataforma y de código abierto (open source) aplicado al manejo y análisis de datos estadísticos.*

A continuación, detallamos cada una de sus características:

R es un lenguaje de programación estadístico

Si bien posee un entorno y se puede utilizar como calculadora avanzada o para simulaciones, R es fundamentalmente un lenguaje de programación. Presenta una estructura y reglas de sintaxis propias, así como gran variedad de funciones desarrolladas con fines estadísticos.

R es un lenguaje orientado a objetos

R implementa conceptos de la programación orientada a objetos, lo cual le permite ofrecer simpleza y flexibilidad en el manejo de datos. En R, todo con lo que trabajamos —variables, funciones, datos, resultados— son objetos que pueden ser modificados o combinados con otros objetos.

R es un lenguaje interpretado

R no requiere compilación previa. Los scripts se ejecutan directamente mediante el intérprete del lenguaje, que devuelve resultados de forma inmediata.

R es un lenguaje multiplataforma

R puede instalarse y utilizarse en sistemas operativos Linux, Windows y MacOS. En todos ellos funciona de la misma manera, lo que garantiza que los scripts pueden correr en cualquier plataforma sin necesidad de modificaciones.

R es software libre y de código abierto

R se distribuye gratuitamente bajo licencia GNU - GPL (*General Public License*), lo que otorga a los usuarios la libertad de usar, estudiar, compartir y modificar el software. Esto ha favorecido el crecimiento de una comunidad global activa y colaborativa.

1.2. Breve historia del lenguaje

R tiene su origen en el lenguaje **S**, desarrollado en los años 70 en los laboratorios Bell de AT&T (actualmente Lucent Technologies). Posteriormente, S dio lugar a una versión comercial llamada **S-Plus**, distribuida por Insightful Corporation.

En 1995, los profesores de estadística **Ross Ihaka** y **Robert Gentleman**, de la Universidad de Auckland (Nueva Zelanda) iniciaron el **“Proyecto R”**, con la intención de desarrollar un programa estadístico inspirado en el lenguaje S pero de dominio público.

Aunque R es considerado un dialecto de S, existen diferencias importantes en el diseño de ambos lenguajes.

El software está desarrollado principalmente en lenguaje C++, con algunas rutinas en Fortran. El nombre “R” hace referencia a las iniciales de sus creadores: Ross y Robert. Actualmente, R es mantenido por un grupo internacional de desarrolladores voluntarios conocido como el **Core Development Team**.

1.3. Scripts de R

Un script es un archivo de texto plano que contiene una secuencia de instrucciones para ser ejecutadas por el intérprete de R.

El término *script* puede traducirse como guión, archivo de órdenes, archivo de procesamiento por lotes o archivo de sintaxis.

Puede crearse con cualquier editor de texto o con entornos especializados, y permite ser leído, modificado, guardado y ejecutado de forma completa o línea por línea.

Una de sus principales ventajas es su **reutilización**: los scripts pueden adaptarse fácilmente a distintos análisis o contextos, lo que facilita la replicabilidad del trabajo.

1.4. Características generales del lenguaje

R posee una **sintaxis textual precisa**. Como en otros lenguajes de programación, la escritura debe ser exacta: distingue entre mayúsculas y minúsculas (*case sensitive*), y cada línea escrita en la consola comienza con el símbolo > (*prompt*[1]).

1.5. Documentación del código

La documentación es una tarea fundamental en cualquier lenguaje de programación, ya que nos permite entender el propósito del script, facilita su mantenimiento y posibilita su reutilización tanto por quien lo creó como por otras personas.

En R, la documentación de los scripts se realiza a través de **comentarios**, indicados con el símbolo #. Todo lo que sigue a ese símbolo es ignorado por el intérprete cuando se ejecute el código:

```
# esto es una línea de comentario y el intérprete no la ejecuta
```

Así que a la hora de documentar es preferible abusar de estos comentarios que no utilizarlos.

1.6. Funciones

Las órdenes elementales en R se denominan comandos o **funciones**. Algunas se conocen como “integradas” ya que están incluidas en el núcleo del lenguaje (R `base`) y otras provienen de **paquetes** adicionales.

La mayoría de las ofrecidas en los paquetes o librerías están elaboradas con R, dado que todos los usuarios podemos crear nuevas funciones con el mismo lenguaje.

Toda función tiene un nombre y puede recibir argumentos (obligatorios u opcionales), que se colocan entre **paréntesis y separados por comas**. Incluso algunas funciones que no tienen asociado argumentos necesitan, en su sintaxis, a los paréntesis `()`.

Siempre una función devuelve un resultado, un valor o realiza una acción:

---

[1] Símbolo que aparece en la pantalla de la computadora indicando que el sistema está esperando información del usuario o que el sistema está listo para recibir instrucciones del usuario.

```
nombre_de_la_función(arg_1, arg_2, arg_n)
```

Como el interprete de R no permite errores en la sintaxis de las expresiones, debemos atender a los siguientes puntos a la hora de escribirlas:

La sintaxis habitual de una función es la siguiente:

```
funcion(arg1, arg2, arg3, ...)
```

Los títulos de los argumentos pueden escribirse y mediante un igual agregar el valor correspondiente:

```
funcion(arg1 = 32, arg2 = 5, arg3 = 65, ...)
```

Se puede omitir el título del argumento y escribir directamente el valor, pero en este caso, hay que respetar el orden definido por la función:

```
funcion(32, 5, 65, ...)
```

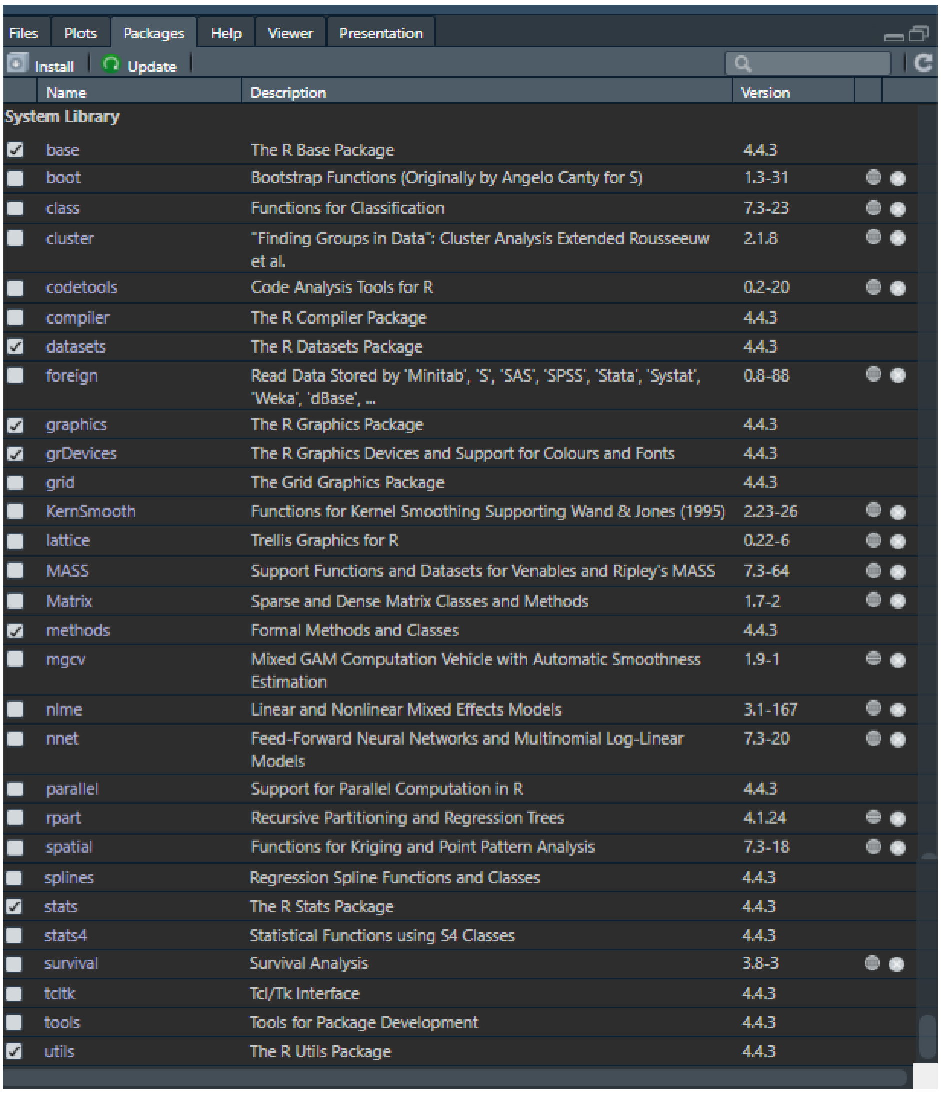
Los valores numéricos, lógicos, especiales y objetos van escritos en forma directa y cuando escribimos caracteres (texto) van necesariamente encerrados entre comillas:

```
funcion(arg1 = 3, arg2 = NA, arg3 = TRUE, arg4 = "less", arg5 = x, ...)
```

1.7. Paquetes o librerías

Los paquetes, también conocidos como *librerías*, son conjuntos de funciones, datos y documentación organizados organizadas en torno a una temática específica, que permiten ampliar las capacidades del sistema base de R.

Al instalar R, se incorpora un núcleo básico que queda activo automáticamente en cada sesión (`base`, `datasets`, `graphics`, `grDevices`, `methods`, `stats` y `utils`). Junto a este núcleo, se incluye un conjunto de paquetes recomendados que forman parte de la distribución oficial y pueden activarse manualmente cuando se los necesita. No obstante, la verdadera potencia de R reside en la posibilidad de incorporar nuevos paquetes desarrollados por la comunidad, lo que permite extender continuamente sus funcionalidades.



	Name	Description	Version
System Library			
☑	base	The R Base Package	4.4.3
☐	boot	Bootstrap Functions (Originally by Angelo Canty for S)	1.3-31
☐	class	Functions for Classification	7.3-23
☐	cluster	"Finding Groups in Data": Cluster Analysis Extended Rousseeuw et al.	2.1.8
☐	codetools	Code Analysis Tools for R	0.2-20
☐	compiler	The R Compiler Package	4.4.3
☑	datasets	The R Datasets Package	4.4.3
☐	foreign	Read Data Stored by 'Minitab', 'S', 'SAS', 'SPSS', 'Stata', 'Systat', 'Weka', 'dBase', ...	0.8-88
☑	graphics	The R Graphics Package	4.4.3
☑	grDevices	The R Graphics Devices and Support for Colours and Fonts	4.4.3
☐	grid	The Grid Graphics Package	4.4.3
☐	KernSmooth	Functions for Kernel Smoothing Supporting Wand & Jones (1995)	2.23-26
☐	lattice	Trellis Graphics for R	0.22-6
☐	MASS	Support Functions and Datasets for Venables and Ripley's MASS	7.3-64
☐	Matrix	Sparse and Dense Matrix Classes and Methods	1.7-2
☑	methods	Formal Methods and Classes	4.4.3
☐	mgcv	Mixed GAM Computation Vehicle with Automatic Smoothness Estimation	1.9-1
☐	nlme	Linear and Nonlinear Mixed Effects Models	3.1-167
☐	nnet	Feed-Forward Neural Networks and Multinomial Log-Linear Models	7.3-20
☐	parallel	Support for Parallel Computation in R	4.4.3
☐	rpart	Recursive Partitioning and Regression Trees	4.1.24
☐	spatial	Functions for Kriging and Point Pattern Analysis	7.3-18
☐	splines	Regression Spline Functions and Classes	4.4.3
☑	stats	The R Stats Package	4.4.3
☐	stats4	Statistical Functions using S4 Classes	4.4.3
☐	survival	Survival Analysis	3.8-3
☐	tcltk	Tcl/Tk Interface	4.4.3
☐	tools	Tools for Package Development	4.4.3
☑	utils	The R Utils Package	4.4.3

En la actualidad, existen más de 17.000 paquetes disponibles para una amplia variedad de aplicaciones, número que crece mes a mes gracias a la naturaleza *open source* del lenguaje. Cualquier persona puede desarrollar y compartir sus propios paquetes, aunque no todos llegan a publicarse en el repositorio oficial CRAN (Comprehensive R Archive Network), que actúa como principal fuente de distribución.

Los paquetes pueden descargarse directamente desde CRAN o instalarse a partir de archivos comprimidos locales (como `.zip` o `.tar.gz`). Una vez instalados, se los puede activar en cualquier análisis.

1.7.1. Dependencias

En muchos casos, al utilizar un paquete, este necesita de otros para funcionar correctamente. Esta relación se conoce como **dependencia**, y es una característica central en el ecosistema de R.

La mayoría de las funciones incluidas en los paquetes están desarrolladas en el propio lenguaje R y, durante su construcción, es habitual que recurran a funciones ya existentes en otros paquetes. Por ejemplo, una función nueva puede hacer uso de una función auxiliar que no pertenece al sistema base, sino a otro paquete externo.

Cuando intentamos ejecutar una función que depende de otras no disponibles en nuestra instalación, R no podrá encontrar esas funciones y nos devolverá un mensaje de error indicando que no reconoce el nombre solicitado. Este tipo de errores suele alertar sobre funciones “desconocidas” o “no encontradas”, lo que indica que falta instalar o activar alguno de los paquetes requeridos.

Para evitar estos inconvenientes, R intenta resolver automáticamente las dependencias cuando instalamos un paquete desde el repositorio oficial CRAN. Si el paquete necesita otros para funcionar, el sistema detecta esta relación y se encarga de instalar también aquellos paquetes auxiliares. De todos modos, si alguno no se instala correctamente o no se activa en la sesión, será necesario hacerlo de forma manual.

1.8. Errores y advertencias

El lenguaje R es muy preciso en su sintaxis. Cometer errores al escribir funciones u objetos es común durante el aprendizaje, y el intérprete de R devuelve mensajes de error cuando detecta algo incorrecto.

Uno de los aspectos clave a tener en cuenta es que R **distingue entre mayúsculas y minúsculas** (*case sensitive*), por lo que no es lo mismo escribir `a` que `A`.

Existen tres grupos de mensajes de error:

- Errores de sintaxis
- Error de objeto no encontrado
- Otros errores

1.8.1. Errores de sintaxis

Ocurre cuando R no puede interpretar una línea de código porque algo está mal escrito. Los errores de sintaxis más frecuentes incluyen:

- Paréntesis, comas, comillas, corchetes o llaves mal colocados o ausentes.
- Argumentos mal separados o incorrectamente definidos.

Por ejemplo la función `rep()` repite valores una cantidad de veces. Tiene dos argumentos, `x` donde se coloca el valor a repetir y `times` donde se define la cantidad de veces:

```r
rep(x = 3, times = 4) # repetimos 4 veces 3 con rep()
```

```
[1] 3 3 3 3
```

Si nos olvidamos de cerrar el paréntesis:

```r
rep(x = 3, times = 4
```

```
Error in parse(text = input): <text>:2:0: unexpected end of input
1: rep(x = 3, times = 4
   ^
```

Si omitimos la coma entre argumentos:

```r
rep(x = 3 times = 4)
```

```
Error in parse(text = input): <text>:1:11: unexpected symbol
1: rep(x = 3 times
              ^
```

Si usamos un nombre de argumento no válido:

```r
rep(y = 3, times = 4)
```

```
Error in `rep()`:
! attempt to replicate an object of type 'symbol'
```

Si escribimos mal el nombre de la función:

```r
rop(x = 3, times = 4)
```

```
Error in `rop()`:
! could not find function "rop"
```

Este último error se asemeja a un error de objeto no encontrado, aunque tiene origen en un problema de sintaxis.

Los mensajes de error en general y sobre todo al principio pueden parecer extraños y difíciles de entender, pero con un poco de práctica podemos inferir donde está el problema.

1.8.2. Error de objeto no encontrado

Este tipo de error se produce cuando R no reconoce un objeto utilizado en el código. Las causas más frecuentes incluyen:

- El nombre del objeto está mal escrito (por ejemplo, errores de sintaxis o uso incorrecto de mayúsculas/minúsculas).
- El objeto pertenece a un paquete o archivo que no fue cargado.
- Faltan comillas al declarar caracteres.
- Otras causas similares.

Volvamos al ejemplo anterior, ahora repitiendo un valor tipo `character`:

```r
rep(x = "A", times = 4) # repetimos 4 veces "A" con rep()
```

```
[1] "A" "A" "A" "A"
```

Si olvidamos las comillas:

```
rep(x = A, times = 4) # repetimos 4 veces A con rep()
```

```
Error:
! object 'A' not found
```

1.8.3. Otros errores

Son aquellos que se generan por causas distintas a errores de sintaxis o de objeto no encontrado. Por ejemplo:

```
"x" + 10
```

```
Error in `"x" + 10`:
! non-numeric argument to binary operator
```

El código anterior genera un mensaje de error porque intenta sumar objetos de tipos incompatibles entre sí.

Veamos otro ejemplo:

```
t.test(1)
```

```
Error in `t.test.default()`:
! not enough 'x' observations
```

En este caso, el error se debe a que no hay suficientes observaciones para realizar una prueba t de Student.

Otro caso frecuente ocurre cuando se intenta acceder a una posición inexistente en un vector:

```
x <- c(1, 2, 3)
x[[5]]
```

```
Error in `x[[5]]`:
! subscript out of bounds
```

Este mensaje de error indica que el subíndice está fuera de los límites del objeto (`subscript out of bounds`).

1.8.4. Advertencias

Las advertencia no son tan serias como un error, o al menos no lo parece, ya que permiten el código se ejecute igual. Pero puede ocurrir que ignorar una advertencia llegue a ser algo muy serio, si esto implica que la salida de la función es equivocada.

Ignorar advertencias puede llevar a conclusiones erróneas, por lo que es recomendable prestarles atención y entender su causa.

Por ejemplo:

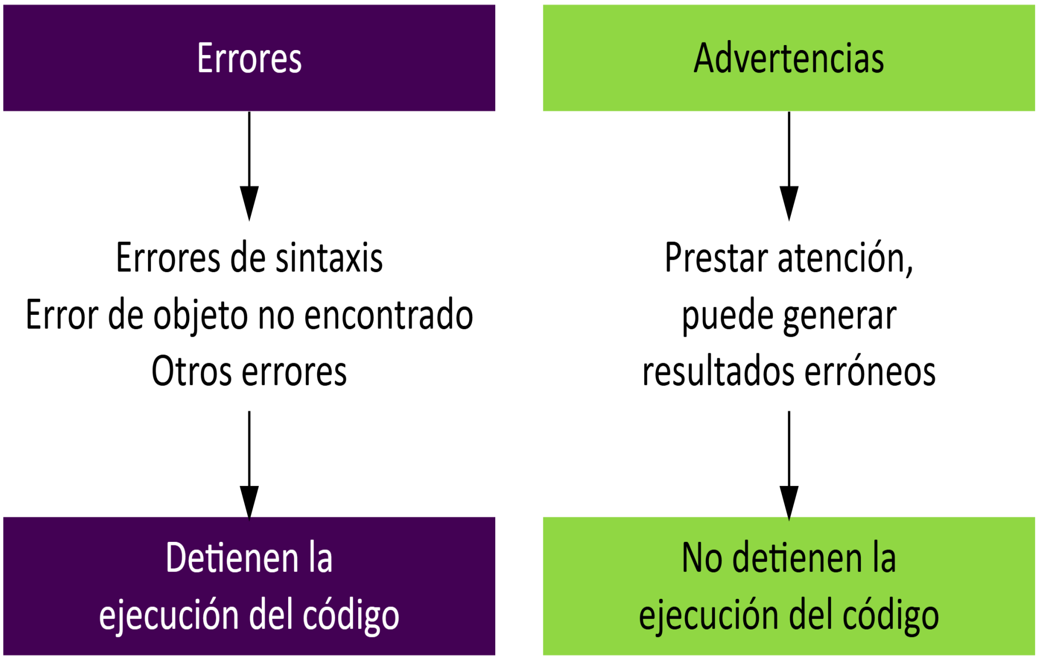
```
log(-1)
```

```
Warning in log(-1): NaNs produced
```

```
[1] NaN
```

la función genera una advertencia porque el logaritmo de un número negativo no está definido en los reales, y R devuelve un valor `NaN` (*Not a Number*).

1.8.5. Resumiendo...


Errores
Advertencias
Errores de sintaxis
Error de objeto no encontrado
Otros errores
Prestar atención,
puede generar
resultados erróneos
Detienen la
ejecución del código
No detienen la
ejecución del código

1.9. Creación de objetos

Todas las declaraciones donde se crean objetos usan el símbolo de asignación `<-`:

```
nombre_objeto <- valor
```

Veámoslo en un ejemplo:

```
a <- 1
```

En este caso asignamos el valor 1 al objeto `a`. El objeto `a` es un vector de una posición (un solo valor). Si llamamos al objeto, el intérprete devuelve el valor asignado previamente:

```
a
```

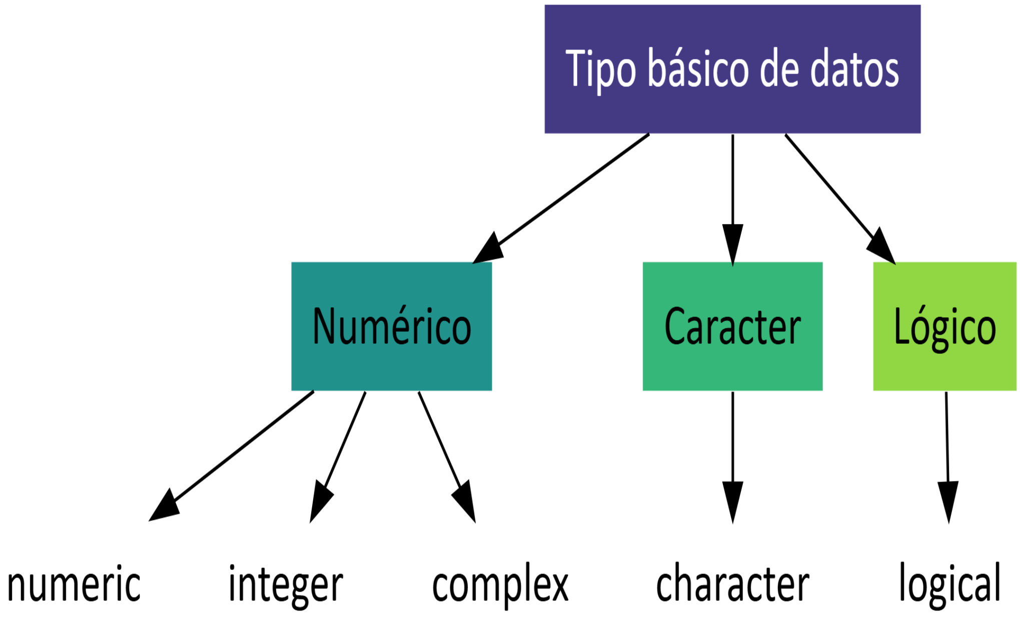
```
[1] 1
```

Observemos que, además de devolvernos el valor, aparece delante un número entre corchetes `[1]`. Este número es la ubicación o **índice** del comienzo del objeto. En este caso, como el vector tiene una sola posición, indica que el primer valor mostrado empieza en la posición 1.

1.10. Estructuras de datos

Los objetos contenedores de datos más simples pertenecen a cinco clases atómicas, que son:

- `integer` (números enteros)
- `numeric` (números reales)
- `complex` (números complejos)
- `character` (cadenas de caracteres)
- `logical` (valores lógicos: `TRUE` o `FALSE`)


Tipo básico de datos
Numérico
Caracter
Lógico
numeric
integer
complex
character
logical

Sin embargo, cada una de estas clases de datos no se encuentran de manera aislada, sino encapsuladas dentro de la clase de objeto más básica de R, a la que se denomina **vector**.

En RStudio, el sistema de colores ayuda a distinguir los tipos de valores:

```
# Número
1111
```

```
[1] 1111
```

```
# Caracter
"esto es una cadena de texto"
```

```
[1] "esto es una cadena de texto"
```

```
# Lógico
TRUE
```

```
[1] TRUE
```

```
# Objeto
a <- 1
```

1.10.1. Vectores

Un vector es un conjunto de valores (números o símbolos), del mismo tipo ordenados de la forma (elemento 1, elemento 2, … , elemento n), donde n es la longitud o tamaño del vector.

Los atributos principales son:

- **Tipo**: puede ser `integer`, `numeric`, `character`, `complex` o `logical`.
- **Longitud**: cantidad de elementos que contiene el objeto,

El vector más simple contiene un solo dato, por ejemplo un vector de longitud 1 y tipo `numeric`:

```
vec1 <- 1
vec1
```

```
[1] 1
```

Otro vector más grande por ejemplo podría ser `(1, 5, 2)`. En este caso también es del tipo `numeric` pero tiene una longitud de 3 elementos:

```
vec2 <- c(1, 5, 2)
vec2
```

```
[1] 1 5 2
```

Para concatenar elementos usamos la función `c()`, dentro de la cual van los valores separados por comas. El orden de los elementos importa, en nuestro ejemplo la primera posición la ocupa el 1, la segunda el 5 y la tercera el 2. Si tuvieramos otro vector `(5, 1, 2)`, no sería lo mismo porque los valores están ordenados de forma diferente.

Para conocer la longitud del vector usamos:

```
length(vec2)
```

```
[1] 3
```

Nos informa que `vec2` tiene 3 elementos.

Para conocer el tipo de dato ejecutamos:

```
class(vec2)
```

```
[1] "numeric"
```

Podemos ver que los datos almacenados en este segundo ejemplo cumplen con la definición en lo que respecta al tipo de dato, ya que cada elemento es del mismo tipo (`numeric`).

Veamos un ejemplo de asignación de otro tipo de dato atómico, como es el `character`:

```
vec3 <- "Hola"
vec3
```

```
[1] "Hola"
```

Siempre que escribamos contenido de tipo `character` debemos hacerlo entre comillas. En este caso generamos el vector `vec3` con el contenido `"Hola"`, que, a pesar de ser una palabra compuesta de varios caracteres, dentro del vector `vec3` esta ocupa una sola posición:

```
length(vec3)
```

```
[1] 1
```

Respecto al tipo de dato si usamos la función `class()` tendremos:

```
class(vec3)
```

```
[1] "character"
```

1.10.2. Factores

Un factor es un objeto especialmente diseñado para contener datos categóricos y se asocia particularmente con las *variables cualitativas*.

En su estructura interna está compuesto por dos vectores:

- Un vector de índices enteros.
- Un vector de categorías (niveles) de tipo `character`, a los que hace referencia el primer vector.

Existen de dos tipos: factores nominales y factores ordinales. En el caso del segundo se establece un orden en los niveles.

Normalmente, obtenemos un tipo `factor` de convertir un vector u otro tipo de objeto con caracteres, pero para mostrar un ejemplo lo realizamos con la función `factor()`:

```
factor1 <- factor(
  x = c("a", "b", "a", "c", "b", "a"),
  levels = c("a", "b", "c")
)
factor1
```

```
[1] a b a c b a
Levels: a b c
```

Creamos el objeto `factor1` con 6 elementos caracteres y tres niveles sin orden.

Además de la practicidad de trabajar con factores, muchas funciones de R requieren que las variables categóricas estén en formato factor para funcionar correctamente.

1.10.3. *Dataframe*

Un *dataframe* es un objeto diseñado para contener conjuntos de datos y representa una estructura bidimensional similar a una tabla, con filas y columnas. Cada columna puede almacenar elementos de distintos tipos (por ejemplo, numéricos, de texto o lógicos), siempre que todos tengan la misma longitud.

Las columnas suelen tener nombres únicos, lo que permite referenciarlas fácilmente como si fueran variables individuales dentro del conjunto de datos.

Este es el tipo de objeto que se utiliza habitualmente para almacenar información importada desde archivos externos (por ejemplo, archivos de texto separados por comas o planillas de Excel), y con el que más frecuentemente trabajamos en los análisis.

Desde el punto de vista estructural, un *dataframe* está compuesto por una serie de vectores de igual longitud dispuestos verticalmente, uno al lado del otro, conformando las columnas de la tabla.

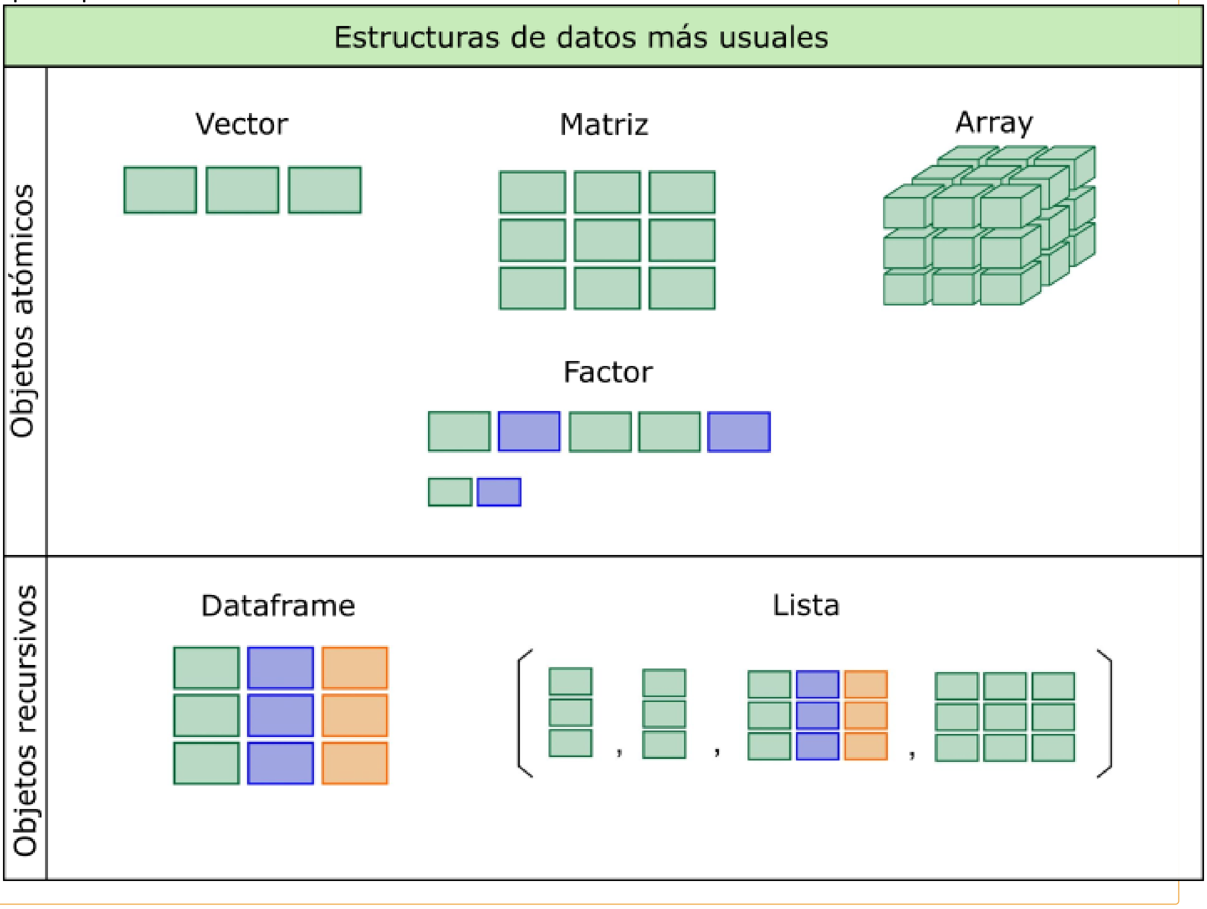
Veamos un ejemplo:

```
# Historia clínica
HC <- c("F324", "G21", "G34", "F231")

# Edad
edad <- c(34, 32, 34, 54)

# Sexo
sexo <- c("M", "V", "V", "M")

# dataframe
df1 <- data.frame(HC, edad, sexo)

df1
```

```
    HC edad sexo
1 F324   34    M
2  G21   32    V
3  G34   34    V
4 F231   54    M
```

Creamos tres vectores con datos de supuestos individuos, su historia clinica, la edad y el sexo. Luego mediante la función `data.frame()` "unimos" esos vectores en forma vertical para formar un *dataframe* de 3 variables y 4 observaciones.

⚠ Existen otras estructuras de datos que aparecen en la siguiente figura. Las más habituales en nuestro trabajo son los vectores y los *dataframes*. Los factores serán necesarios cuando tengamos que especificar distintos órdenes de niveles.


Estructuras de datos más usuales
Objetos atómicos
Vector
Matriz
Array
Factor
Objetos recursivos
Dataframe
Lista

1.11. Operadores

Además de funciones, el lenguaje R cuenta con operadores de uso relativamente intuitivo, que permiten realizar operaciones de diferentes tipos con los objetos que contienen datos.

1.11.1. Operadores aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
^	Potencia
%%	Módulo

Operador	Descripción
%/ %	División de enteros

Los operadores aritméticos se utilizan como si el lenguaje fuese una calculadora:

```
# Suma
2 + 5
```

```
[1] 7
```

```
# Resta
3 - 2
```

```
[1] 1
```

```
# Multiplicación
9 * 3
```

```
[1] 27
```

```
# División
10 / 2
```

```
[1] 5
```

```
# Potencia
5^2
```

```
[1] 25
```

También se pueden hacer operaciones con los objetos que almacenan valores numéricos:

```
a <- 3

b <- 6

(a + b) * b
```

```
[1] 54
```

Y funciona con objetos con más de un elemento, aplicando aritmética vectorial, donde las operaciones se realizan elemento a elemento:

```
a <- c(1, 2, 3)

a * 3
```

```
[1] 3 6 9
```

O bien, con operaciones entre los objetos, donde se realiza entre los elementos de la misma posición:

```
a <- c(1, 2, 3)

a * a
```

```
[1] 1 4 9
```

1.11.2. Operadores relacionales

Operador	Descripción
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que

Habitualmente estos operadores se utilizan asiduamente en expresiones para indicar relaciones entre valores.

Podemos ver su funcionamiento en el ejemplo siguiente:

```
a <- c(3, 8, 2)

a == c(3, 4, 5)
```

```
[1]  TRUE FALSE FALSE
```

El lenguaje evalúa las comparaciones que hace el operador relacional igual (en este caso) y en aquellos valores que coinciden devuelve `TRUE` y en los que no hay coincidencia devuelve `FALSE`.

Lo mismo sucede con los otros operadores relacionales:

```
a <- c(4, 8, 10)

a > 8
```

```
[1] FALSE FALSE  TRUE
```

```
a < 8
```

```
[1]  TRUE FALSE FALSE
```

```
a != 8
```

```
[1]  TRUE FALSE  TRUE
```

1.11.3. Operadores lógicos

Operador	Descripción
!	NOT
&	AND booleano
&&	AND booleano para vectores de longitud 1
|	OR booleano
||	OR booleano para vectores de longitud 1

Cuando queremos conectar algunas de las expresiones relacionales hacemos uso de estos operadores lógicos típicos (AND, OR, NOT).

Para ejemplificar podemos hacer:

```
a <- c(1:8)

a
```

```
[1] 1 2 3 4 5 6 7 8
```

```
(a > 3) & (a < 7)
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE FALSE
```

En el caso anterior usamos el operador & como conector AND de dos expresiones relacionales donde el lenguaje devuelve `TRUE` en el rango mayor a 3 y menor a 7 (valores 4,5 y 6).

1.12. Valores especiales

Existen algunos valores especiales para datos con expresiones reservadas en R, entre ellos encontramos los valores `NA`, `NaN`, `Inf` y `NULL`.

Operador	Significado	Descripción
NA	Not available	Es la forma de expresar a los valores perdidos o faltantes (missing values)

Operador	Significado	Descripción
NaN	Not a number	Utilizado para resultados de operaciones que devuelven error numérico
Inf	Infinity	Valor infinito (positivo)
-Inf	Infinity	Valor infinito (negativo)
NULL	Null	Valor nulo

El más relevante de estos valores especiales es el `NA` que sirve para indicar que no hay valor en esa posición o elemento de un objeto.

1.13. Secuencias regulares

Además de concatenar elementos con la función `c()`, existen tres formas comunes de generar secuencias regulares.

La primera es mediante un **operador secuencial** (`:`), que genera una secuencia de enteros entre dos valores, ya sea en forma ascendente o descendente:

```
1:10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
10:1
```

```
 [1] 10  9  8  7  6  5  4  3  2  1
```

La segunda forma es mediante la función `seq()`, cuyos argumentos principales son `from`, `to` y `by`. Esta función permite mayor flexibilidad:

```
seq(from = 1, to = 20, by = 2)
```

```
 [1]  1  3  5  7  9 11 13 15 17 19
```

El ejemplo anterior genera una secuencia de números que comienza en 1 y llega hasta 20, avanzando de dos en dos.

Algunos otros ejemplos de la misma función:

```
seq(from = 0.1, to = 0.9, by = 0.1)
```

```
[1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

```
seq(from = -5, to = 5, by = 1)
```

```
 [1] -5 -4 -3 -2 -1  0  1  2  3  4  5
```

```
seq(from = 300, to = 0, by = -50)
```

```
[1] 300 250 200 150 100  50   0
```

La tercera opción es la función `rep()`, que permite duplicar valores. Su forma más básica es `rep(x, times = n)`, donde se repite el valor `x` la cantidad de veces indicada por `n`.

Algunos ejemplos:

```
rep(x = 2, times = 5)
```

```
[1] 2 2 2 2 2
```

```
# combinada con el operador :
rep(1:4, 5)
```

```
 [1] 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4
```

```
# combinada con la función c()
rep(c(4.5, 6.8, 7.2), 2)
```

```
[1] 4.5 6.8 7.2 4.5 6.8 7.2
```

1.14. Índices

R implementa una forma eficiente y flexible de acceder selectivamente a elementos de un objeto basado en una indexación interna.

Para acceder a un índice se utiliza la notación de corchetes. Por ejemplo, si tenemos un vector `x` con varios elementos y queremos acceder al segundo, usamos `x[2]`.

Esta forma de indexar los elementos de los distintos objetos nos permite realizar muchas operaciones desde el simple llamado, hasta seleccionar, eliminar o modificar valores.

Veamos algunos ejemplos:

```
# generamos un vector x con 5 letras
x <- c("a", "b", "c", "d", "e")

# llamamos a la primer posición y nos devuelve su valor
x[1]
```

```
[1] "a"
```

```
# llamamos a la tercer posición y nos devuelve su valor
x[3]
```

```
[1] "c"
```

```
# llamamos a las posiciones 1 y 3 juntas mediante c()
x[c(1, 3)]
```

```
[1] "a" "c"
```

```
# llamamos a las posiciones menos la 1 y la 4
x[-c(1, 4)]
```

```
[1] "b" "c" "e"
```

```
# creamos otro vector y con los valores 1,2 y 5
y <- c(1, 2, 5)

# utilizamos el vector y como índice
x[y]
```

```
[1] "a" "b" "e"
```

```
# asignamos el valor "h" a la posición 2 del vector x
x[2] <- "h"

x
```

```
[1] "a" "h" "c" "d" "e"
```

```
# creamos el vector z eliminando la posición 5 de x
z <- x[-5]

z
```

```
[1] "a" "h" "c" "d"
```

Estos ejemplos muestran las múltiples posibilidades que ofrece el uso de índices para manipular objetos en R, lo que hace al lenguaje muy poderoso y versátil.

Cuando se aplican índices a estructuras bidimensionales, como matrices o *dataframes*, la notación general es:

```
nombre[índice de fila, índice de columna]
```

1.15. Gestión de factores

Al presentar las distintas estructuras de datos mencionamos que los factores suelen generarse a partir de vectores u otros objetos de tipo carácter.

Para entender cómo funcionan, partamos de un vector con datos categóricos:

```
respuesta <- c("Si", "No", "No", "Si", "Si", "Si", "No")
```

Creamos un vector llamado `respuesta` con 7 elementos de tipo carácter, en el que se repiten las categorías `"Si"` y `"No"`.

Podemos confirmar que se trata de un vector y que su contenido es de tipo carácter:

```
# preguntamos si sexo es un vector
is.vector(respuesta)
```

```
[1] TRUE
```

```
# visualizamos el tipo de dato de sexo
class(respuesta)
```

```
[1] "character"
```

Para crear un factor a partir de este vector debemos utilizar la función `factor()`:

```
respuesta <- factor(respuesta)

respuesta
```

```
[1] Si No No Si Si Si No
Levels: No Si
```

En la salida observamos los siete elementos y, debajo, una línea con los niveles del factor, indicados por `Levels`. Estos niveles son identificados automáticamente.

Podemos verificar cómo cambió el tipo de objeto:

```
# preguntamos si respuesta es un vector
is.vector(respuesta)
```

```
[1] FALSE
```

```
# preguntamos si respuesta es un factor
is.factor(respuesta)
```

```
[1] TRUE
```

```
# visualizamos el tipo de dato de respuesta
class(respuesta)
```

```
[1] "factor"
```

Aunque visualmente vemos palabras, internamente R trata al factor como un tipo especial basado en números. ¿Por qué sucede esto? Veámoslo con más detalle:

```
str(respuesta)
```

```
 Factor w/ 2 levels "No","Si": 2 1 1 2 2 2 1
```

La función `str()` devuelve la estructura interna del objeto. En este caso, muestra que `respuesta` tiene dos niveles y que cada elemento del vector es representado por un número (1 o 2), donde `1` corresponde a `"No"` y `2` a `"Si"`.

Esto significa que la estructura de los factores está compuesta por dos vectores: uno numérico que funciona como índice de enteros, que sustituye al vector de caracteres original, y el otro es un vector de caracteres, que contiene los niveles o categorías, a los que hace referencia el primer vector.

Para ver solo los niveles o categorías del factor podemos usar:

```
levels(respuesta)
```

```
[1] "No" "Si"
```

En los factores nominales donde no importa el orden, la función `factor()` implementa el orden alfabético para determinar a qué índice numérico pertenece cada categoría. Es claro en el ejemplo que a `No` le asigna el `1` y a `Si` el `2`.

Pero si nos encontramos frente a una variable cualitativa ordinal vamos a necesitar indicarle a la función cual es el orden de las categorías.

Vamos al siguiente ejemplo:

```
salud <- c(4, 3, 1, 3, 2, 2, 3, 3, 1)
salud
```

```
[1] 4 3 1 3 2 2 3 3 1
```

Tenemos en el vector salud algunos códigos numéricos que representan nivel de salud de personas registradas por una encuesta donde 1 significa mala salud, 2 regular, 3 buena y 4 muy buena.

Procedemos a crear el factor `nivsalud` a partir de este vector:

```
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena    
[8] Buena     Mala     
Levels: Mala Regular Buena Muy buena
```

Aquí utilizamos dos argumentos adicionales:

- `labels`: define las etiquetas que queremos mostrar para cada categoría.
- `levels`: indica el orden de los niveles originales (en este caso, 1 a 4).

Aquí es necesario definir estos argumentos porque, a diferencia del factor `respuesta`, el vector `salud` contiene números en lugar de las palabras correspondientes a las categorías.

Hasta aquí hemos creado un factor pero si miramos sus niveles no encontraremos señales que sigan un orden específico:

```
levels(nivsalud)
```

```
[1] "Mala"      "Regular"   "Buena"     "Muy buena"
```

Sin embargo, aún no hemos definido un **orden** explícito entre las categorías. Para hacerlo, agregamos el argumento `ordered = TRUE`:

```
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4,
  ordered = TRUE
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena    
[8] Buena     Mala     
Levels: Mala < Regular < Buena < Muy buena
```

Al incorporar este argumento, estamos diciendo que los niveles tienen orden natural. Esto puede observarse en la salida donde se muestra que `"Mala"` < `"Regular"` < `"Buena"` < `"Muy buena"`.

1.16. Gestión de *dataframes*

En R, la estructura utilizada para almacenar datos provenientes de fuentes externas (como archivos .csv, planillas de cálculo, bases SQL, etc.) es el *dataframe*.

A modo de ejemplo, vamos a construir un *dataframe* llamado `datos` con 4 variables y 5 observaciones. Las variables serán:

- `id`: identificador numérico entero y correlativo.
- `edad`: edad en años.
- `sexo`: codificado como `"V"` para varón y `"M"` para mujer.
- `trabaja`: variable lógica, donde `TRUE` (`T`) representa que la persona trabaja y `FALSE` (`F`) que no lo hace.

```
# construimos el vector id
id <- 1:5

# construimos el vector edad
edad <- c(23, 43, 12, 65, 37)

# construimos el vector sexo
sexo <- c("V", "M", "M", "V", "M")

# construimos el vector trabaja
trabaja <- c(T, T, F, F, T)

# construimos el dataframe datos
datos <- data.frame(id, edad, sexo, trabaja)
```

Aunque para el ejemplo construimos un *dataframe* manualmente, lo habitual en la práctica es leer archivos externos que se importan directamente como *dataframes*.

Algunas de las funciones generales que podemos aplicar son:

```
# pedimos el número de columnas (variables)
ncol(datos)
```

```
[1] 4
```

```
# pedimos el número de filas (registros u observaciones)
nrow(datos)
```

```
[1] 5
```

```
# pedimos las dimensiones del dataframe (observaciones,variables)
dim(datos)
```

```
[1] 5 4
```

También podemos visualizar como se compone el objeto `datos` aplicando `str()` que devuelve la estructura interna de cualquier objeto en R.

```
str(datos)
```

```
'data.frame':	5 obs. of  4 variables:
 $ id     : int  1 2 3 4 5
 $ edad   : num  23 43 12 65 37
 $ sexo   : chr  "V" "M" "M" "V" ...
 $ trabaja: logi  TRUE TRUE FALSE FALSE TRUE
```

Esta salida nos informa que:

- `datos` es un *dataframe* con 5 observaciones y 4 variables.
- `id` es un entero (`int`).
- `edad` es numérica (`num`).
- `sexo` es de tipo carácter (`chr`).
- `trabaja` es lógica (`logi`).

Además, `str()` muestra los primeros valores de cada variable, que en este caso son todos, ya que el *dataframe* tiene solo 5 registros.

Cuando necesitemos llamar al contenido de alguna columna o variable del *dataframe*, utilizamos la siguiente notación:

```
nombre_del_dataframe$nombre_de_la_variable
```

Por ejemplo, si queremos mostrar el contenido de la variable `sexo` del objeto `datos` hacemos:

```
datos$sexo
```

```
[1] "V" "M" "M" "V" "M"
```

Esto devuelve todos los valores de la variable `sexo`.

También podemos acceder a los elementos del *dataframe* usando **indexación por filas y columnas**:

```
# pedimos la tercer variable (sexo)
datos[, 3]
```

```
[1] "V" "M" "M" "V" "M"
```

Observemos que las dos salidas son idénticas, dado que muestran todas las observaciones de la variable `sexo`, aunque la solicitud sea de manera diferente.

Algunos otros ejemplos de uso de índices:

```
# observación 1 de la variable 2
datos[1, 2]
```

```
[1] 23
```

```
# observación 4 de todas las variables
datos[4, ]
```

```
  id edad sexo trabaja
4  4   65    V   FALSE
```

```
# observación 1,2 y 3 de la variable 3
datos[1:3, 3]
```

```
[1] "V" "M" "M"
```

```
# observación 5 de las variables 1 y 4
datos[5, c(1, 4)]
```

```
  id trabaja
5  5    TRUE
```

Por último, vamos podemos mostrar y gestionar los nombres de las variables con la función `names()`:

```
names(datos)
```

```
[1] "id"      "edad"    "sexo"    "trabaja"
```

1.17. Fórmulas

En R, las fórmulas se utilizan principalmente para describir modelos estadísticos, y las vamos a emplear a lo largo de toda la cursada.

Las fórmulas se escriben utilizando operadores como `~`, `+`, `*`, entre otros. Se usan para especificar modelos como regresiones, ANOVA, pruebas de hipótesis y, en algunos casos, también para generar ciertos tipos de gráficos.

Formula genérica

Aquí la formula está dada por el operador `~`, a la izquierda está la variable de respuesta o dependiente, a la derecha la o las variables explicativas o independientes. El esquema general es el siguiente:

```
variable_dependiente ~ variable_independiente
```

El símbolo `~` (llamado virgulilla) puede interpretarse como *"es modelada por"* o *"en función de"*.

Por ejemplo, en una regresión lineal se utiliza la función base `lm()` y el argumento principal de esta función es una formula:

```
regresion_lineal <- lm(variable_respuesta ~ variable_explicativa, data = datos)
```

Además de la fórmula, la función `lm()` incluye el argumento `data =`, que es fundamental: allí se le indica a R el *dataframe* en el que debe buscar las variables involucradas en el modelo.

La siguiente tabla muestra los usos de los los operadores más comunes dentro de una formula:

Operador	Ejemplo	Descripción
+	+x	Incluye la variable x
-	-x	Excluye la variable x
:	x : z	Incluye la interacción de la variable x con z
*	x * z	Incluye ambas variables y la interacción entre ellas

2. Introducción a RStudio

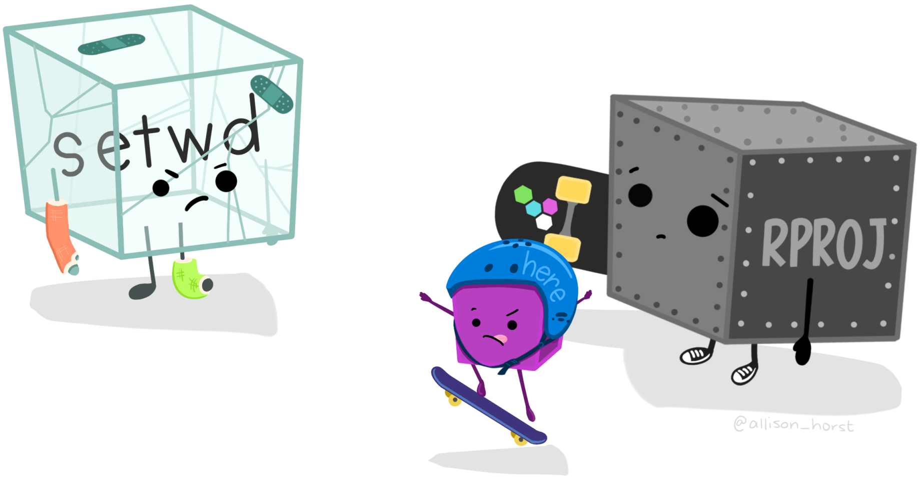


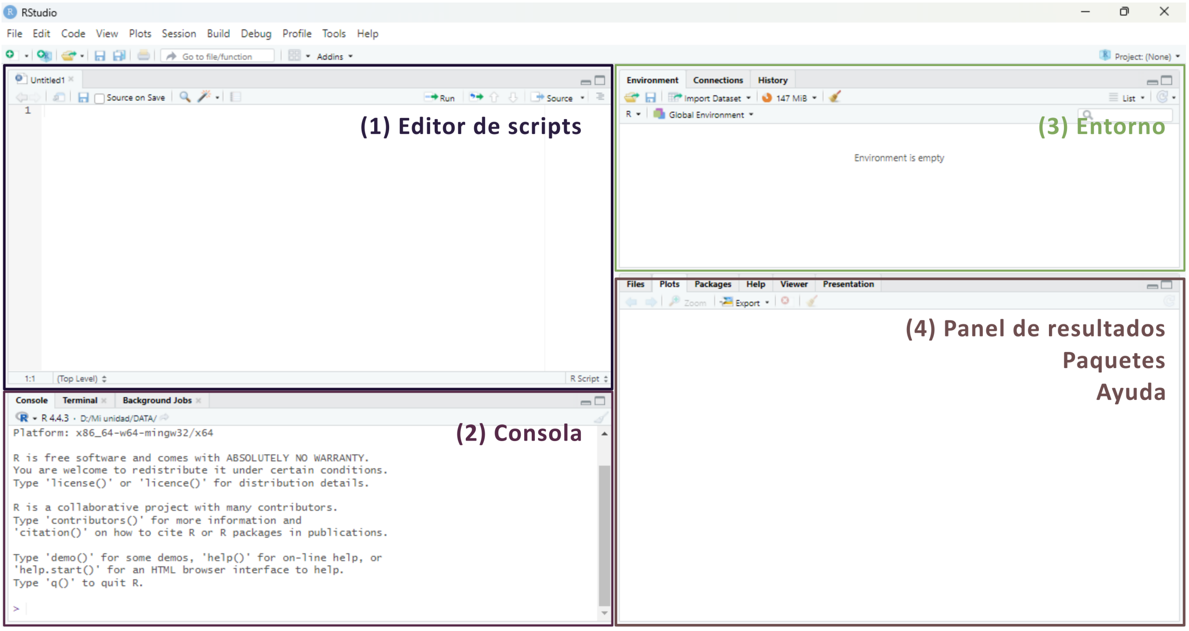
Figura 2.1.: Artwork por @allison_horst

2.1. Primeros pasos y generalidades

RStudio Desktop (2025, Posit Software) es un **entorno de desarrollo integrado** (*IDE*, por sus siglas en inglés) diseñado específicamente para trabajar con el lenguaje R.

Es una herramienta **multiplataforma y de código abierto** que facilita la programación, el análisis de datos y la elaboración de informes científicos. Ofrece una integración fluida con otros componentes del ecosistema de R, como R Markdown, Quarto, control de versiones (por ejemplo, Git) y la gestión de proyectos.

RStudio presenta una interfaz unificada compuesta por distintos paneles, lo que permite organizar el trabajo de forma clara y eficiente:


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Project: (None)
Untitled1
Source on Save
Run
Source
(1) Editor de scripts
1:1
(Top Level)
R Script
Environment
Connections
History
Import Dataset
147 MiB
List
R
Global Environment
(3) Entorno
Environment is empty
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
(4) Panel de resultados
Paquetes
Ayuda
Console
Terminal
Background Jobs
R 4.4.3 · D:/Mi unidad/DATA/
(2) Consola

```
Platform: x86_64-w64-mingw32/x64

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```

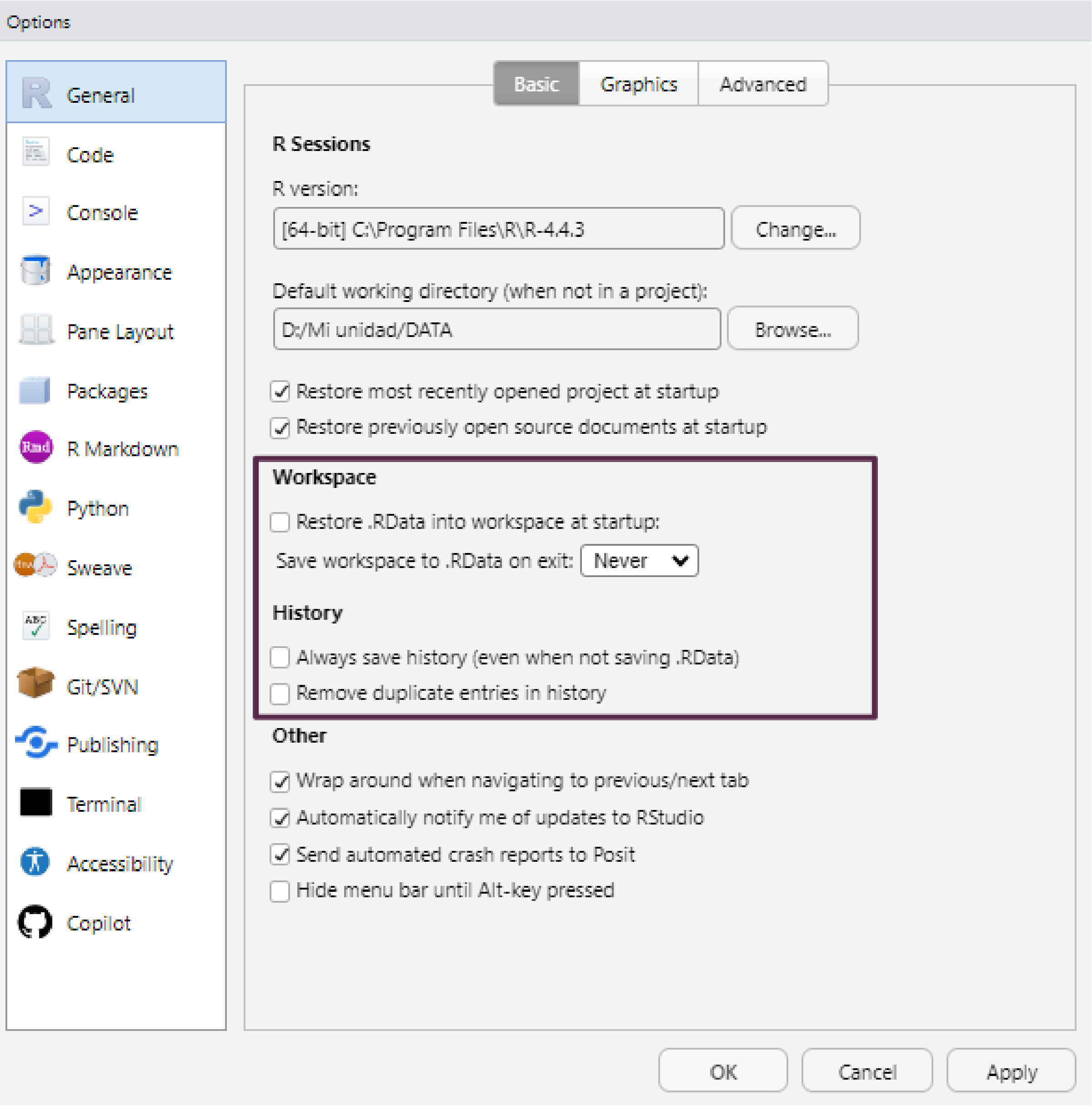
1. **Editor de scripts** (*Source*): permite crear y editar scripts de R, así como documentos R Markdown o Quarto.

2. **Consola de R** (*Console*): muestra la salida del código ejecutado y ejecuta código de R de forma inmediata.

3. **Entorno** (*Environment*): muestra los objetos creados durante la sesión, como vectores, *dataframes* o funciones.

4. **Panel de resultados** (*Output*): presenta los gráficos, tablas, visualizaciones en HTML y también incluye un explorador de archivos, visor de paquetes instalados y panel de ayuda.

Para garantizar la reproducibilidad de los resultados, es recomendable evitar el guardado automático del historial de objetos entre sesiones, ya que puede generar confusión. Para desactivar esta opción, ir a `Tools > Global Options` y desmarcar las casillas de las opciones **Workspace** y **History** como se muestra en la siguiente imagen:



Options
General
Code
Console
Appearance
Pane Layout
Packages
R Markdown
Python
Sweave
Spelling
Git/SVN
Publishing
Terminal
Accessibility
Copilot
Basic
Graphics
Advanced
Change...
Browse...
OK
Cancel
Apply

R Sessions

R version:
[64-bit] C:\Program Files\R\R-4.4.3

Default working directory (when not in a project):
D:/Mi unidad/DATA

☑ Restore most recently opened project at startup
☑ Restore previously open source documents at startup

Workspace

☐ Restore .RData into workspace at startup:
Save workspace to .RData on exit: Never

History

☐ Always save history (even when not saving .RData)
☐ Remove duplicate entries in history

Other

☑ Wrap around when navigating to previous/next tab
☑ Automatically notify me of updates to RStudio
☑ Send automated crash reports to Posit
☐ Hide menu bar until Alt-key pressed

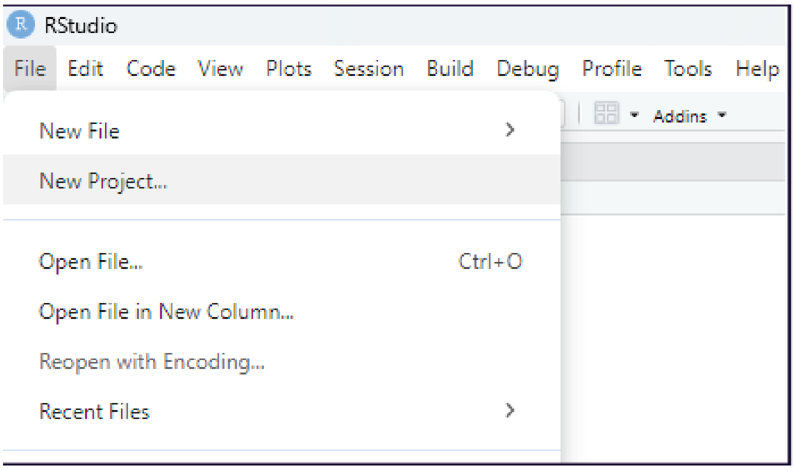
2.2. Proyectos de RStudio

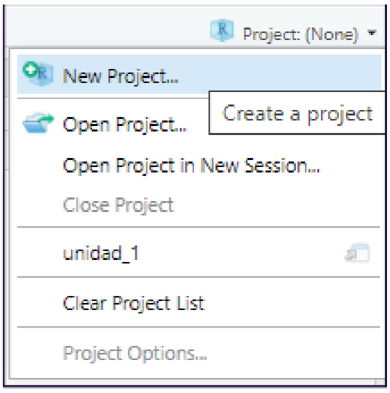
Los proyectos de RStudio permiten organizar de forma estructurada todo el material asociado a un análisis: scripts, informes, bases de datos, imágenes, etc. Cada proyecto se vincula a una carpeta específica del sistema de archivos, y RStudio la utiliza como directorio de trabajo por defecto. Esta carpeta puede estar ubicada en cualquier parte del sistema de almacenamiento que deseemos (disco rígido, pendrive, disco externo, etc).

Trabajar con proyectos facilita la importación de datos y evita errores relacionados con rutas relativas o absolutas.

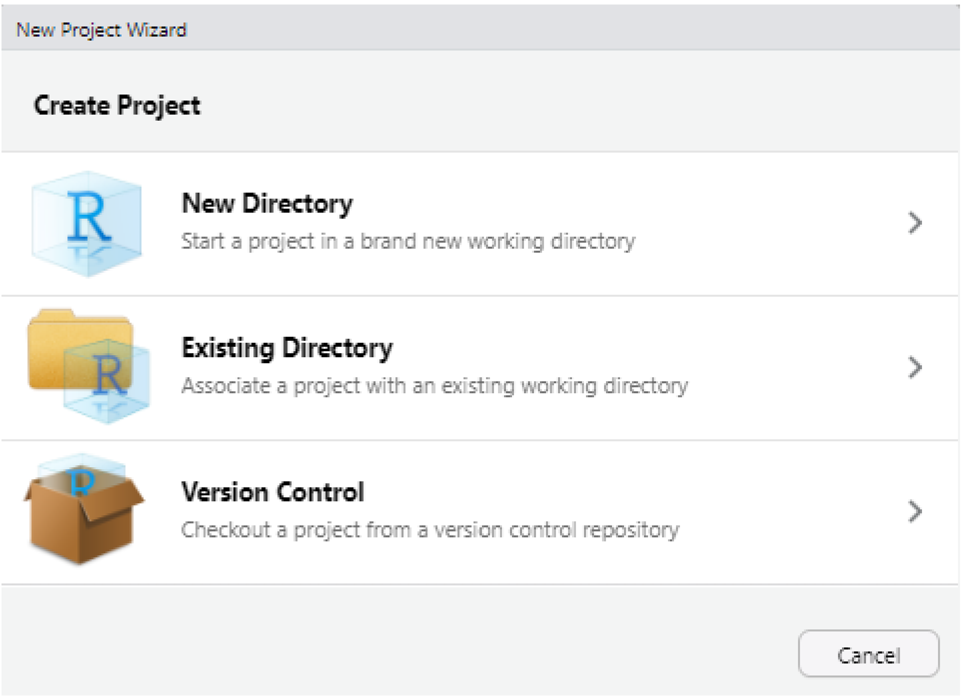
2.2.1. Crear un proyecto

Para crear un nuevo proyecto, se puede utilizar el menú `File > New Project...`. También accedemos a generar un proyecto nuevo a partir de pulsar el acceso directo `New Project...` ubicado en la esquina superior derecha de la interfaz:


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Addins
New File
New Project...
Open File... Ctrl+O
Open File in New Column...
Reopen with Encoding...
Recent Files

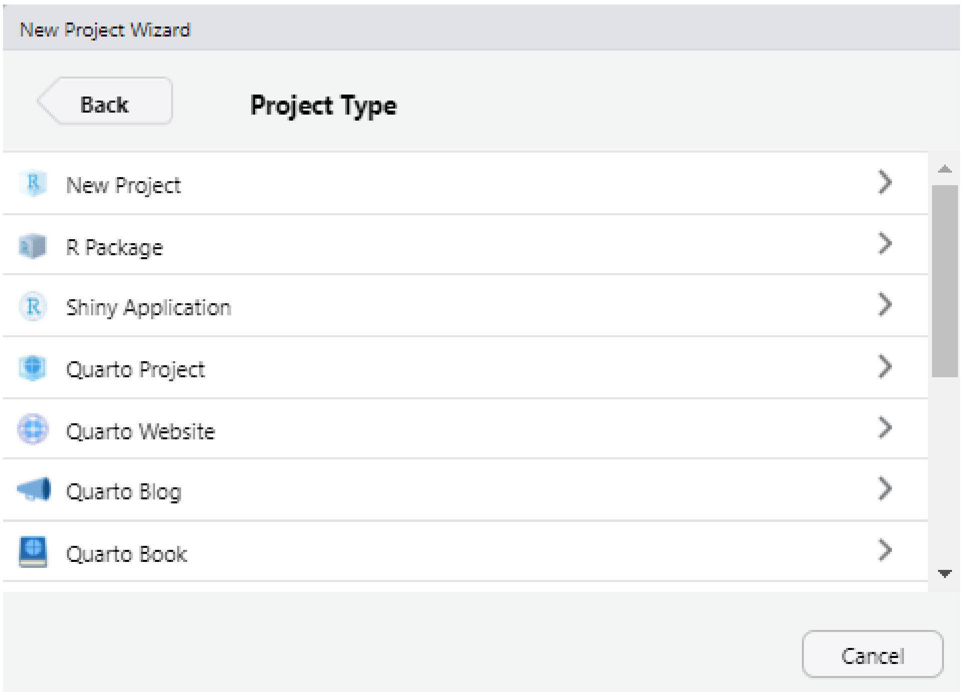

Project: (None)
New Project...
Open Project...
Create a project
Open Project in New Session...
Close Project
unidad_1
Clear Project List
Project Options...

En cualquiera de los dos casos aparecerá un cuadro de diálogo que presenta tres opciones para crear el **nuevo proyecto** de RStudio:


New Project Wizard
Create Project
New Directory
Start a project in a brand new working directory
Existing Directory
Associate a project with an existing working directory
Version Control
Checkout a project from a version control repository
Cancel

- **New Directory**: crea una nueva carpeta para el proyecto a la que deberemos asignarle un nombre, es la opción más habitual. Todos los archivos de configuración aparecerán asociados a esta nueva carpeta.
- **Existing Directory**: vincula el proyecto a una *carpeta ya existente* que contenga archivos previos con los que deseamos trabajar.
- **Version Control**: permite clonar un repositorio (Git o SVN). Esta opción no se utilizará durante el curso.

Una vez que creamos un nuevo proyecto con la opción **New Directory**, aparecerá una pantalla con una lista de tipos de proyectos que se pueden crear en RStudio:

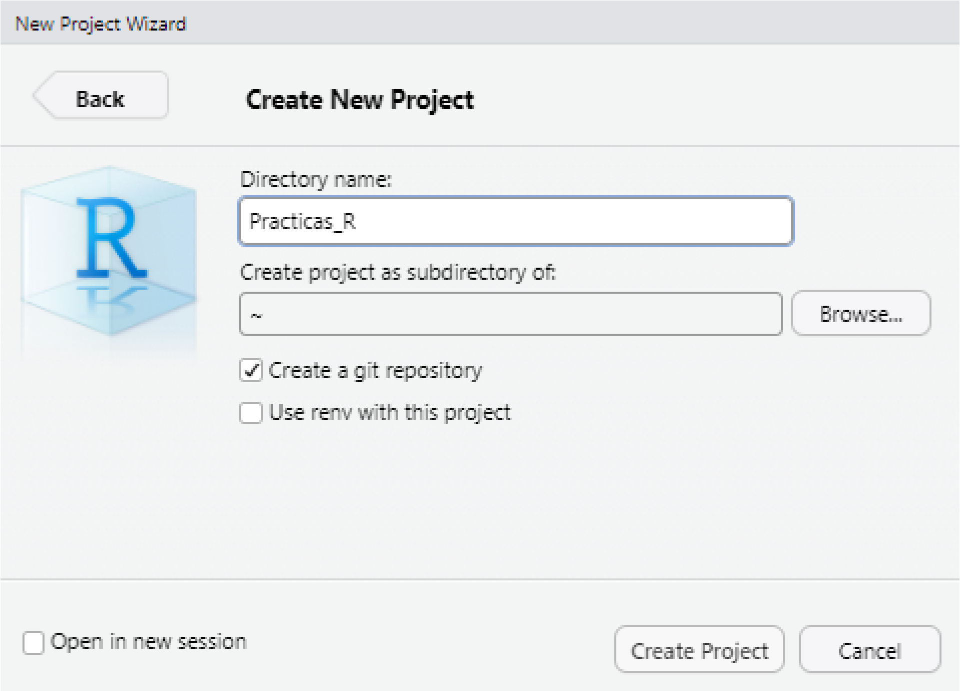

New Project Wizard
Back
Project Type
New Project
R Package
Shiny Application
Quarto Project
Quarto Website
Quarto Blog
Quarto Book
Cancel

Durante este curso utilizaremos siempre la **primera opción** (**New Project**). Las demás opciones están pensadas para usos más específicos, como el desarrollo de paquetes de R, sitios web o documentos con Quarto o R Markdown.

Al seleccionar **New Project**, se abrirá una nueva ventana con los siguientes campos:

- **Directory name:** aquí debemos escribir el nombre del nuevo directorio (carpeta) que también será el nombre del proyecto. Por ejemplo, podemos llamarlo `Practicas_R`.

- **Create project as subdirectory of:** este campo permite definir en qué ubicación del sistema de archivos se guardará el proyecto. Podemos hacer clic en el botón **Browse...** para abrir el explorador de archivos y seleccionar la carpeta contenedora. En nuestro ejemplo, lo ubicaremos dentro de **Mis Documentos**.



Directory name: Practicas_R

Create project as subdirectory of: ~

☑ Create a git repository

☐ Use renv with this project

☐ Open in new session

Una vez completados estos campos, hacemos clic en **Create Project**.

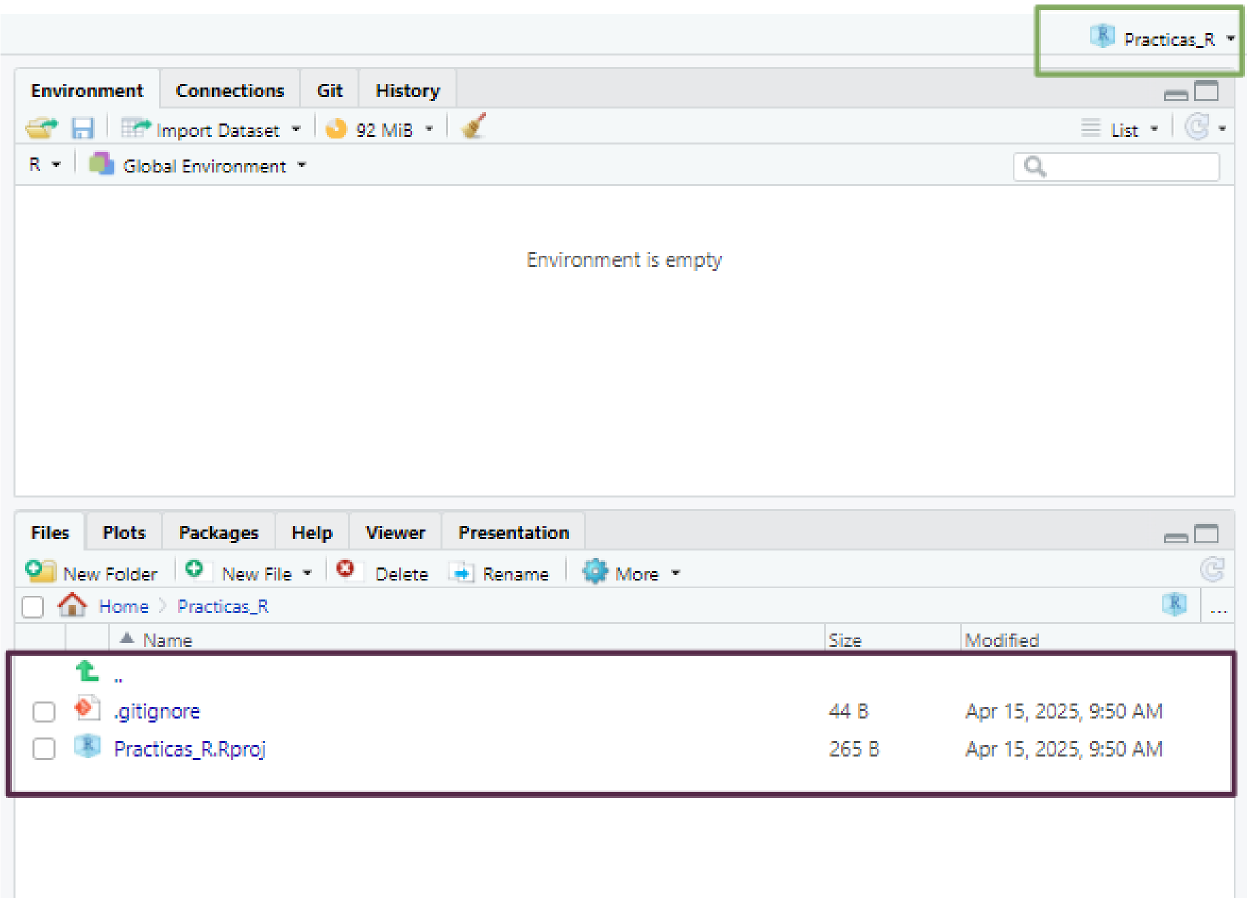
Los proyectos en RStudio tienen entornos independientes, lo que significa que si cerramos un proyecto o cambiamos a otro, la configuración de cada uno se mantendrá inalterable sin interferir con los demás.

Esto incluye los scripts abiertos, el directorio de trabajo, las pestañas que dejamos activas y otros elementos del entorno que puedan ser necesarios para continuar con un análisis. Este sistema permite mantener organizados los distintos trabajos que llevamos adelante.

Echemos un vistazo a lo que RStudio realizó:

- En primer lugar el panel **Files** (pantalla inferior derecha) apunta a la nueva carpeta **Practicas_R** y dentro de ella vemos un nuevo archivo el nombre del proyecto y la extensión `.Rproj`. Este archivo contiene todas las configuraciones para su proyecto.

- El otro cambio se observa en la parte superior derecha, que muestra el nombre del proyecto. Si hacemos click en él, se desplegará el menú de proyectos. Desde aquí se puede abrir y cerrar proyectos, navegar rápidamente a proyectos que se han abierto recientemente y configurar las opciones de RStudio para cada uno de ellos.


Practicas_R
Environment
Connections
Git
History
Import Dataset
92 MiB
List
R
Global Environment
Environment is empty
Files
Plots
Packages
Help
Viewer
Presentation
New Folder
New File
Delete
Rename
More
Home
Practicas_R
Name
Size
Modified
..
.gitignore
44 B
Apr 15, 2025, 9:50 AM
Practicas_R.Rproj
265 B
Apr 15, 2025, 9:50 AM

2.2.2. Abrir un proyecto existente

Cuando el proyecto ya existe, sea porque lo creamos nosotros o porque alguien nos pasó una carpeta con un proyecto de RStudio creado vamos a visualizar dentro de esa carpeta un archivo con extensión `.Rproj`.

La forma más veloz para abrir el proyecto es ejecutar este archivo (debería abrir una sesión de RStudio con el proyecto activo). La otra forma es desde el menú superior derecho de RStudio en la opción **Open project...** y luego buscando en nuestro directorio el mismo archivo `.Rproj`.

Nota

El menú de proyectos del área superior derecha va guardando como elementos recientes los proyectos que se van abriendo y también es una forma rápida de acceder a ellos pulsando sobre estos atajos.

2.3. Scripts en RStudio

Como vimos **anteriormente**, un script es un archivo de código que contiene una secuencia de instrucciones escritas en R. Estos scripts pueden ser reutilizados, modificados y compartidos, lo que los convierte en una herramienta fundamental para garantizar la reproducibilidad del trabajo.

2.3.1. Crear un nuevo script

Tenemos varias formas de crear un script nuevo:

- Desde el menú superior: `File > New File > R Script`
- Con el atajo de teclado: `Ctrl + Shift + N`
- Desde la barra de herramientas: presionando el ícono ⊕

2.3.2. Ejecutar scripts

La forma habitual de ejecutar el contenido de un script es línea por línea, usando alguna de las siguientes opciones:

- Presionando el botón Run del editor de código de RStudio
- Mediante el atajo de teclado `Ctrl + Enter`

Para ejecutar una línea, simplemente ubicamos el cursor en cualquier parte de ella y presionamos el comando correspondiente. Luego de ejecutarse, el cursor avanzará automáticamente a la siguiente línea de código.

Mientras ejecutamos cada línea debemos ir observando la salida en la consola (panel inferior izquierdo) y también los cambios que se dan en el panel *Environment* (panel superior derecho) donde aparecerán los objetos que vayamos creando y manipulando.

2.3.3. Edición de scripts

Modificar o agregar líneas al script puede hacerse directamente en el editor. Cada vez que realizamos un cambio, es necesario **volver a ejecutar la línea** o bloque modificado para que los cambios se reflejen en el entorno de trabajo.

Podemos probar y modificar tantas veces como sea necesario. Sin embargo, debemos tener presente que cada manipulación en los objetos se mantiene hasta que se vuelvan a cambiar y a veces, cuando los objetos están vinculados con otras líneas de código posteriores tenemos que tener cuidado que se mantenga la consistencia del script.

Por ejemplo: si definimos un vector numérico para realizar cálculos matemáticos, pero luego lo sobrescribimos con un valor de tipo caracter, los cálculos posteriores producirán un error y RStudio nos informará de esto en la consola.

Por eso, es clave observar el contenido de los objetos en el panel **Environment**, lo que nos ayuda a evitar errores y operaciones incoherentes.

2.3.4. Guardado de scripts

Cualquier agregado o modificación que hayamos realizado al script y nos interese mantener nos obligará a guardar el archivo de código editado.

Existen distintas formas de guardar un script:

- Desde el menú superior: `File > Save`
- Con el atajo de teclado: `Ctrl + S`
- Presionando el ícono del disquete azul □

Si en cambio quisiera guardarlo como otro archivo para mantener el script original, podemos guardarlo con diferente nombre o en otra ubicación mediante `File > Save As...`

2.3.5. Abrir scripts existentes

Los scripts que construyamos o nos compartan siempre tendrán extensión `.R` y, por lo general, se encontrarán dentro de un proyecto.

Para abrir estos archivos `.R` podemos:

- Desde el menú superior: `File > Open file...`
- Con el atajo de teclado: `Ctrl + O`
- Haciendo click sobre el archivo desde el panel *Files*
- Presionando el botón de la carpeta amarilla □

Esto abrirá el script en una nueva pestaña dentro del editor de código.

2.3.6. ¿Cómo trabajaremos en este curso?

En general, utilizaremos scripts dentro de **proyectos de RStudio**. La secuencia recomendada será:

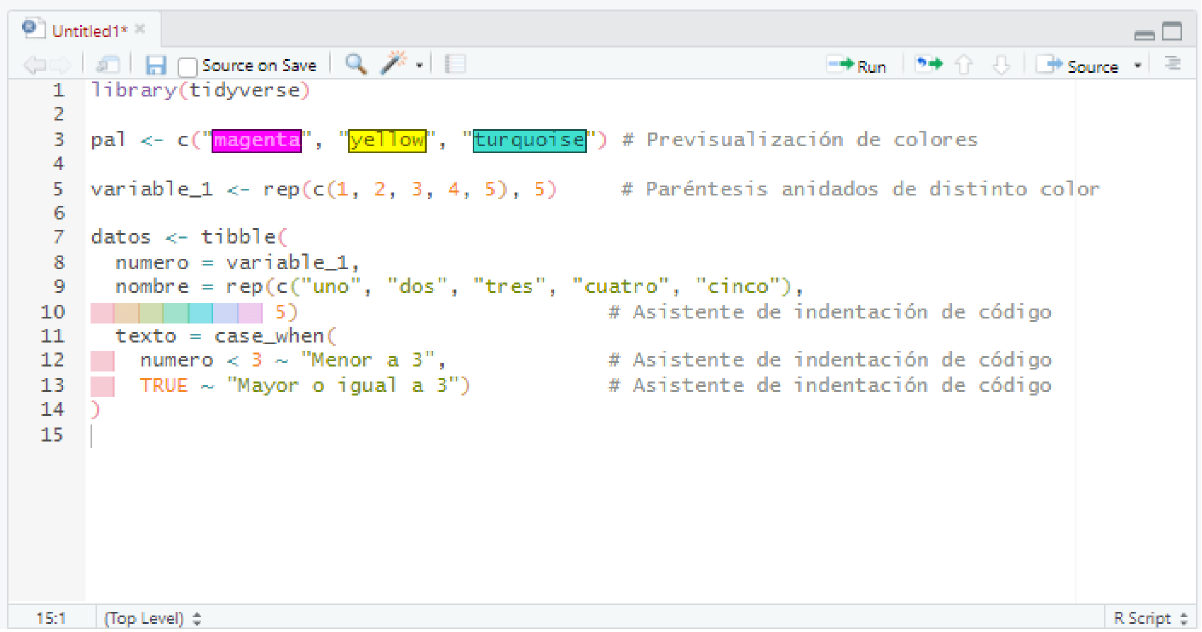
1. **Descargar** desde el Aula Virtual un archivo comprimido conteniendo la carpeta, el proyecto, los scripts y archivos de datos.
2. **Descomprimir** el archivo en la ubicación que deseamos (recomendamos crear una carpeta destinada al curso).
3. Abrir la carpeta y ejecutar el archivo de proyecto `.Rproj`.
4. Una vez abierto RStudio con el proyecto activo, ubicamos los scripts desde el panel **Files**.
5. **Ejecutar** cada línea del script, leyendo la documentación del código y observando la salida en la consola y los cambios en el entorno

2.4. Herramientas de RStudio

Algunas de las herramientas fundamentales de RStudio son el asistente de código, la ayuda en línea y el historial de comandos.

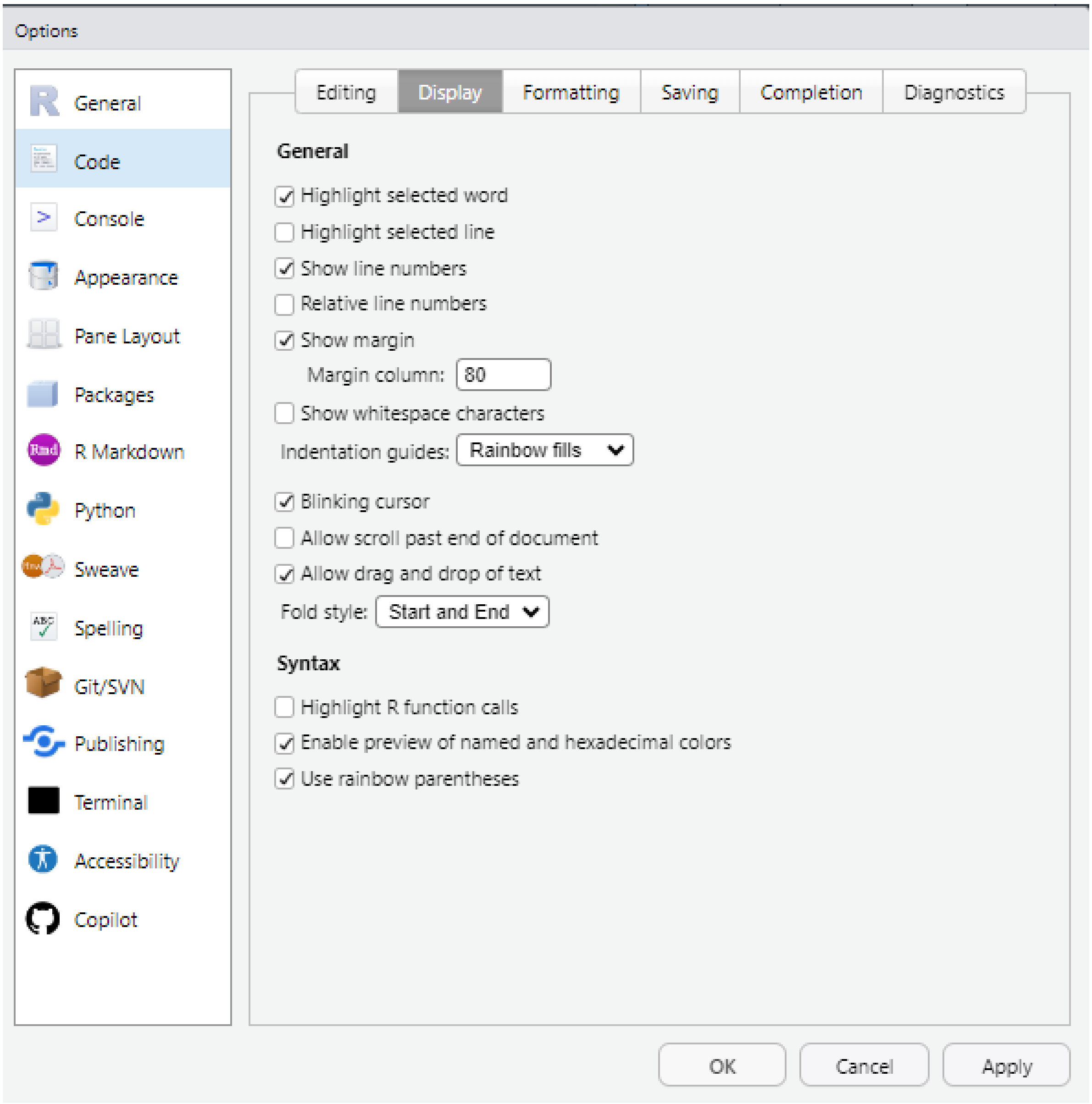
2.4.1. Asistente de código

Al escribir en el editor o la consola, la tecla `Tab` activa el autocompletado de funciones, nombres de objetos y argumentos, agilizando la escritura y reduciendo errores de sintaxis. En versiones recientes, el asistente también permite la previsualización de colores en los gráficos, resaltar los paréntesis de cierre en funciones anidadas con distintos colores y gestionar automáticamente la indentación del código.



```r
library(tidyverse)

pal <- c("magenta", "yellow", "turquoise") # Previsualización de colores

variable_1 <- rep(c(1, 2, 3, 4, 5), 5)     # Paréntesis anidados de distinto color

datos <- tibble(
  numero = variable_1,
  nombre = rep(c("uno", "dos", "tres", "cuatro", "cinco"),
               5)                         # Asistente de indentación de código
  texto = case_when(
    numero < 3 ~ "Menor a 3",             # Asistente de indentación de código
    TRUE ~ "Mayor o igual a 3")           # Asistente de indentación de código
)

```

Muchas de estas opciones se pueden configurar desde el menú `Code` y desde `Tools > Global Options > Code`(pestañas **Editing** y **Display**).


Options
General
Code
Console
Appearance
Pane Layout
Packages
R Markdown
Python
Sweave
Spelling
Git/SVN
Publishing
Terminal
Accessibility
Copilot
Editing
Display
Formatting
Saving
Completion
Diagnostics
OK
Cancel
Apply

General

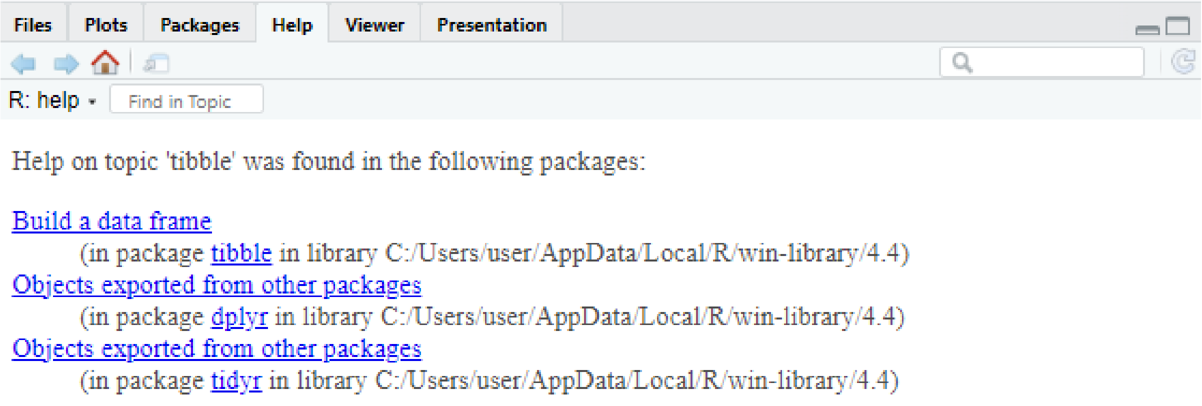
☑ Highlight selected word
☐ Highlight selected line
☑ Show line numbers
☐ Relative line numbers
☑ Show margin
Margin column: 80
☐ Show whitespace characters
Indentation guides: Rainbow fills
☑ Blinking cursor
☐ Allow scroll past end of document
☑ Allow drag and drop of text
Fold style: Start and End

Syntax

☐ Highlight R function calls
☑ Enable preview of named and hexadecimal colors
☑ Use rainbow parentheses

2.4.2. Ayuda en línea

Al posicionar el cursor sobre el nombre de una función en el editor y presionar `F1`, se accede directamente a la documentación correspondiente en el panel **Help** (habitualmente ubicado en la esquina inferior derecha).

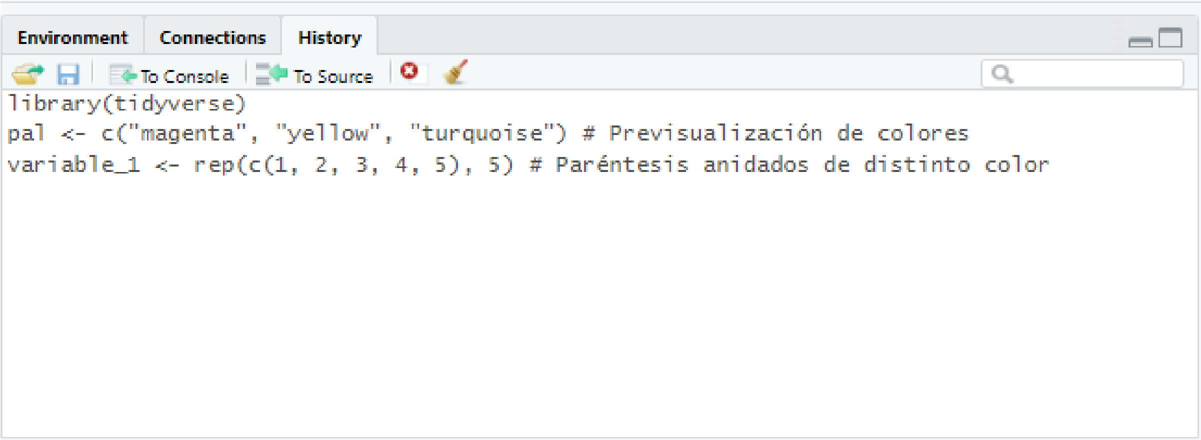


Help on topic 'tibble' was found in the following packages:

Build a data frame
(in package tibble in library C:/Users/user/AppData/Local/R/win-library/4.4)
Objects exported from other packages
(in package dplyr in library C:/Users/user/AppData/Local/R/win-library/4.4)
Objects exported from other packages
(in package tidyr in library C:/Users/user/AppData/Local/R/win-library/4.4)

2.4.3. Historial de comandos

En la consola, al usar las teclas de flecha arriba/abajo, se puede navegar por los comandos ejecutados durante la sesión actual. Además, el panel **History** (parte superior derecha) almacena los comandos de todas las sesiones previas, permitiendo reutilizarlos con un clic en **To Console** (`Enter`) o **To Source** (`Shift + Enter`), según se desee insertarlos en la consola o en el script activo.



```
library(tidyverse)
pal <- c("magenta", "yellow", "turquoise") # Previsualización de colores
variable_1 <- rep(c(1, 2, 3, 4, 5), 5) # Paréntesis anidados de distinto color
```

2.5. Atajos de teclado (Windows)

Menú	Descripción
Archivo (File)	
Ctrl + Shift + N	Crea un nuevo script
Ctrl + O	Abre un script guardado
Ctrl + S	Guarda el script activo
Ctrl + W	Cierra el script activo
Ctrl + Q	Sale del programa RStudio
Edición (Edit)	
Ctrl + F	Abre la ventana de búsqueda (para buscar palabras dentro de un script)
Ctrl + L	Limpia la consola
Código (Code)	

Menú	Descripción
Ctrl + Enter	Ejecuta la línea de código donde está situado el cursor
Ctrl + Alt + R	Ejecuta todo el código del script activo
Ctrl + Shift + N	Inserta nueva sección de código
Ctrl + Shift + R	Inserta nueva sección de comentarios de texto
Sesión (Session)	
Ctrl + Shift + H	Abre la ventana para establecer directorio de trabajo
Ctrl + Shift + F10	Reinicia la sesión de R
Herramientas (Tools)	
Alt + Shift + K	Abre la lista de ayuda de atajos de teclado

2.6. Paquetes

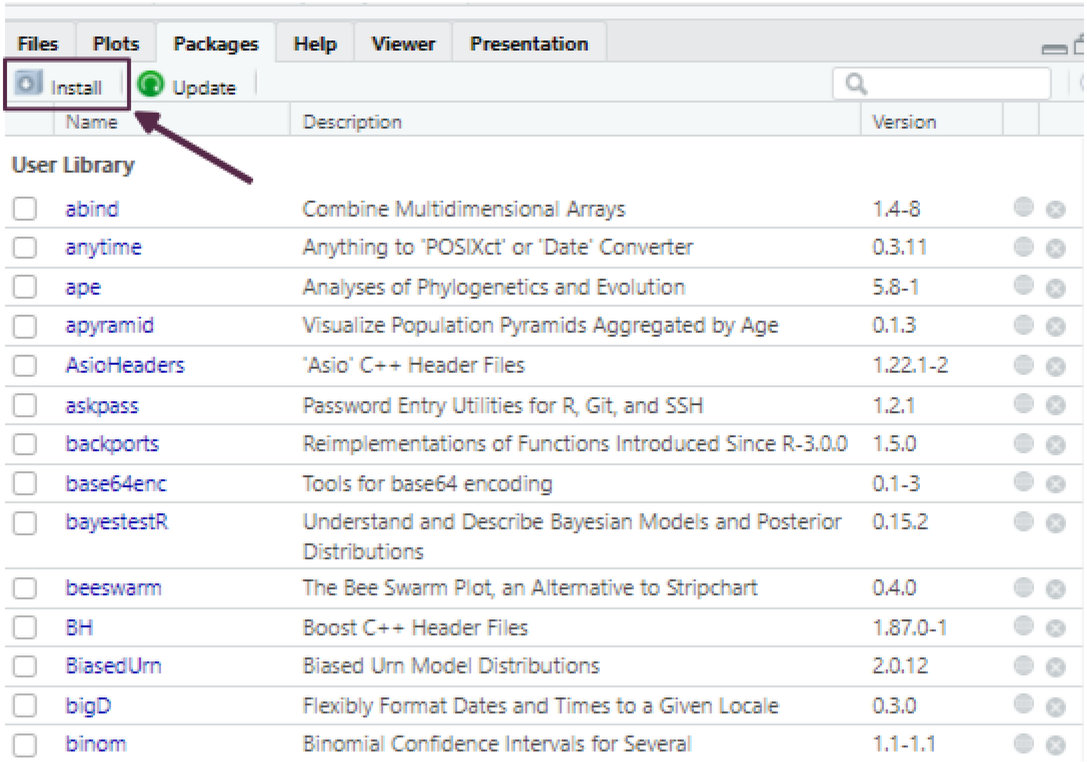
Existen dos formas principales de descargar paquetes: directamente desde RStudio o desde el **sitio web de CRAN**, descargándolos como archivos comprimidos. Si el equipo cuenta con conexión a Internet, lo más práctico es realizar la descarga directamente desde RStudio. En cambio, si no se dispone de acceso permanente a la red, es posible descargar los paquetes desde otro equipo y luego transferirlos como archivos `.zip` o `.tar.gz` al equipo donde se encuentra instalado R.

Cuando accedemos al sitio web de CRAN y buscamos un paquete específico, encontraremos información útil como una breve descripción del paquete, el número de versión, la fecha de publicación, el nombre del autor, documentación asociada y enlaces de descarga específicos para cada sistema operativo.

Dado que la mayoría de las computadoras hoy en día cuentan con acceso a Internet, en este curso nos enfocaremos en la instalación y activación de paquetes directamente desde RStudio.

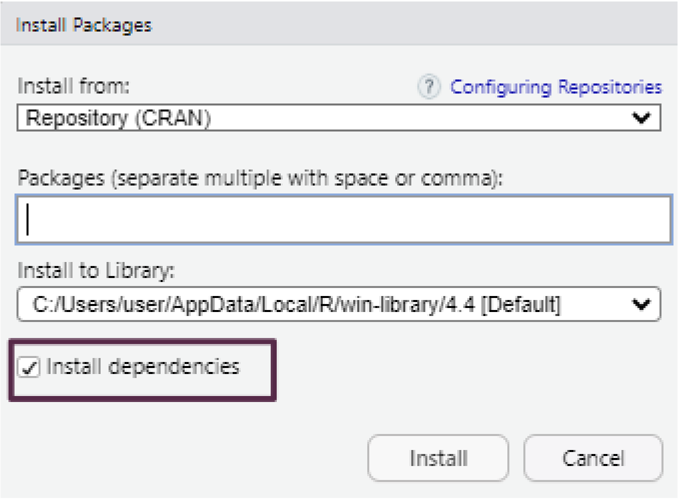
Dentro del entorno de RStudio, la gestión de paquetes se realiza desde la pestaña *Packages*, ubicada en el panel inferior derecho. Esta interfaz facilita tareas como la instalación, actualización y activación de paquetes, y cada acción que realizamos desde la interfaz gráfica se traduce internamente en la ejecución de funciones del lenguaje R que pueden observarse en la consola.

Para instalar un paquete nuevo, simplemente pulsamos el botón *Install* dentro de la pestaña *Packages*, lo cual abrirá una ventana emergente.



	Name	Description	Version
User Library			
☐	abind	Combine Multidimensional Arrays	1.4-8
☐	anytime	Anything to 'POSIXct' or 'Date' Converter	0.3.11
☐	ape	Analyses of Phylogenetics and Evolution	5.8-1
☐	apyramid	Visualize Population Pyramids Aggregated by Age	0.1.3
☐	AsioHeaders	'Asio' C++ Header Files	1.22.1-2
☐	askpass	Password Entry Utilities for R, Git, and SSH	1.2.1
☐	backports	Reimplementations of Functions Introduced Since R-3.0.0	1.5.0
☐	base64enc	Tools for base64 encoding	0.1-3
☐	bayestestR	Understand and Describe Bayesian Models and Posterior Distributions	0.15.2
☐	beeswarm	The Bee Swarm Plot, an Alternative to Stripchart	0.4.0
☐	BH	Boost C++ Header Files	1.87.0-1
☐	BiasedUrn	Biased Urn Model Distributions	2.0.12
☐	bigD	Flexibly Format Dates and Times to a Given Locale	0.3.0
☐	binom	Binomial Confidence Intervals for Several	1.1-1.1

Allí podemos escribir el nombre del paquete que deseamos instalar. Para asegurarnos de que también se descarguen e instalen automáticamente los paquetes de los que depende, es importante tildar la opción *Install dependencies*.



Install from: Repository (CRAN)

Packages (separate multiple with space or comma):

Install to Library: C:/Users/user/AppData/Local/R/win-library/4.4 [Default]

☑ Install dependencies

Como alternativa, también podemos realizar la instalación desde el editor de scripts mediante el siguiente comando:

```r
install.packages("nombre_del_paquete", dependencies = TRUE)
```

> Para facilitar la instalación de los paquetes requeridos durante el curso, recomendamos descargar el archivo **“Paquetes y datos para el curso”**, descomprimirlo y ejecutar el script que se encuentra en su interior.

2.7. Lectura de archivos de datos

R permite importar tablas de datos desde diversos formatos, tanto utilizando funciones de **R base** como funciones provistas por paquetes específicos.

El formato más común es el **texto plano** (ASCII), donde los valores están organizados en columnas separadas por caracteres delimitadores. Los separadores más habituales incluyen:

- Coma (`,`)
- Punto y coma (`;`)
- Tabulación (`\t`)
- Barra vertical (`|`)

Estos archivos suelen tener una **cabecera** (*header*) en la primera fila con los nombres de las variables, y cada columna debe contener datos del mismo tipo (números, texto, lógicos, etc.).

Para importar correctamente un archivo es importante conocer su estructura:

- Si incluye o no cabecera.
- Qué carácter se usa como separador.
- El tipo de codificación (UTF-8, Latin1, etc.).

Dado que son archivos de texto, pueden visualizarse con editores simples como el Bloc de Notas o desde RStudio, lo que facilita su inspección previa.

Para cargar los datos desde un archivo de texto plano usamos el código:

```r
datos <- read.xxx("mis_datos.xxx")
```

(Se debe reemplazar `read.xxx()` por la función correspondiente: `read.table()`, `read.csv()`, `read_delim()`, `read_excel()`, etc., según la extensión del archivo).

R también permite cargar bases de datos incluidas en paquetes instalados mediante:

```r
data(nombre_datos)

datos <- nombre_datos
```

2.8. Buenas prácticas

Adoptar buenas prácticas desde el inicio mejora la reproducibilidad, facilita el trabajo colaborativo y reduce errores. Algunas recomendaciones clave son:

- Trabajar **siempre** dentro de un **proyecto de RStudio** (`.Rproj`). Esto permite organizar los archivos, mantener rutas relativas consistentes y acceder a funcionalidades específicas como control de versiones o panel de archivos integrados.
- Incluir al comienzo de cada script las líneas de **activación de paquetes** necesarios, utilizando la función `library()`.

- **Cargar los datos** una vez activados los paquetes, para garantizar que todas las funciones requeridas estén disponibles.
- Documentar el código mediante **comentarios** iniciados con #. Esto permite entender qué hace cada bloque de código, facilitando futuras modificaciones o revisiones.
- **Usar espacios e indentación adecuada** para mejorar la legibilidad. Esto es especialmente importante en estructuras anidadas (como condicionales, bucles o funciones).

3. Introducción a `tidyverse`

readr
dplyr
%>%
Ceci n'est pas un pipe.
tidyverse
ggplot2
readxl
TIBBLE

3.1. Introducción

Tidyverse es el nombre que recibe el conjunto de paquetes desarrollados y/o promovidos por **Hadley Wickham** (jefe científico en Posit/RStudio) y su equipo, orientado al trabajo de ciencia de datos con R (Wickham et al. 2019). Estos paquetes están diseñados para integrarse de manera coherente, compartiendo una misma filosofía de diseño conocida como ***The tidy tools manifesto***.

Los cuatro principios básicos sobre los que se construye *tidyverse* son:

- Reutilización de estructuras de datos

- Resolución de problemas complejos combinando varias piezas sencillas

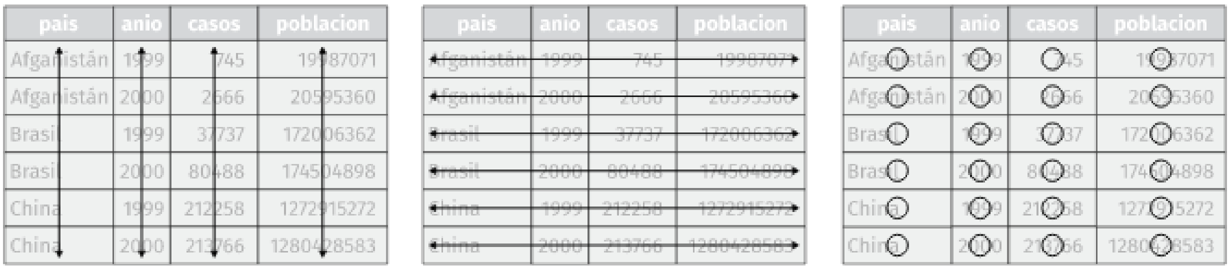
- Uso de programación funcional

- Diseño orientado a las personas

Los paquetes incluidos cubren todas las etapas del análisis de datos dentro de R: importación y ordenamiento de los datos (*tidy data*), transformación, visualización, modelado y la posterior comunicación de resultados.

La palabra *tidy* se traduce como “ordenado”, y hace referencia a una estructura específica que deben cumplir los datos:

- Cada ***variable*** es una *columna* de la tabla de datos.
- Cada ***observación*** es una *fila* de la tabla de datos.
- Cada ***tabla*** responde a una *unidad de observación o* análisis.



pais	anio	casos	poblacion
Afganistán	1999	745	19987071
Afganistán	2000	2666	20595360
Brasil	1999	37737	172006362
Brasil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

variables observaciones valores

Además de los paquetes principales, al instalar `tidyverse` se incluyen otros que permiten trabajar con fechas, cadenas de caracteres o factores, también siguiendo los mismos principios.

Uno de los objetivos de los desarrolladores fue dotar a la sintaxis de estos paquetes de una **gramática** clara: funciones cuyos nombres y argumentos permiten construir "*frases*" que sean semánticamente comprensibles. Un ejemplo de esto se ve en el paquete `dplyr`, donde la mayoría de las funciones son verbos en inglés como `filter()`, `mutate()`, `summarise()`, lo que facilita su lectura y comprensión.

El paquete `tidyverse` puede instalarse desde el repositorio oficial CRAN mediante el menú *Packages* de RStudio, o ejecutando el siguiente código:

```r
install.packages("tidyverse")
```

Una vez instalado, se activa mediante:

```r
library(tidyverse)
```

Al activarlo, se muestra un mensaje con la versión instalada, la lista de paquetes que se cargan automáticamente y posibles conflictos de nombres entre funciones. Esto es habitual cuando se utilizan múltiples paquetes, ya que algunas funciones pueden llamarse igual. Por ejemplo, la función `filter()` existe tanto en el paquete `stats` como en `dplyr`. Al cargar `tidyverse`, R avisa de esta superposición:

```
dplyr::filter() masks stats::filter()
```

Cuando necesitamos asegurarnos de que estamos usando la función de un paquete específico, se recomienda usar la notación `::`, por ejemplo:

```r
# Función filter() del paquete stats
stats::filter()

# Función filter() del paquete tidyverse
dplyr::filter()
```

Una estrategia útil cuando trabajamos con varios paquetes es cargar `tidyverse` al final de la lista de paquetes, para que sus funciones sobrescriban las de otros paquetes si fuese necesario:

```r
library(stats)
library(tidyverse)
```

Los paquetes que se instalan con la versión actual de `tidyverse` pueden consultarse ejecutando:

```
tidyverse_packages()
```

```
 [1] "broom"         "conflicted"    "cli"           "dbplyr"       
 [5] "dplyr"         "dtplyr"        "forcats"       "ggplot2"      
 [9] "googledrive"   "googlesheets4" "haven"         "hms"          
[13] "httr"          "jsonlite"      "lubridate"     "magrittr"     
[17] "modelr"        "pillar"        "purrr"         "ragg"         
[21] "readr"         "readxl"        "reprex"        "rlang"        
[25] "rstudioapi"    "rvest"         "stringr"       "tibble"       
[29] "tidyr"         "xml2"          "tidyverse"    
```

Además, existen muchos otros paquetes que siguen la misma filosofía pero no están incluidos por defecto. En esos casos, deben instalarse y activarse individualmente.

⚠ Para profundizar el uso de `tidyverse`, se recomienda consultar:
Sitio oficial: https://www.tidyverse.org/
Libro *R para Ciencia de Datos*: https://es.r4ds.hadley.nz/
Nueva versión (*r4ds 2e*): https://r4ds.hadley.nz/
EpiRhandbook en español: https://www.epirhandbook.com/es/index.es.html

TIBBLE
www.rstudio.com

3.2. *Dataframes* con `tibble`

Uno de los paquetes que forman parte del núcleo básico de *tidyverse* es `tibble` (Müller y Wickham 2026), que introduce una versión moderna del objeto `data.frame`. Todas las funciones que generan tablas de datos en *tidyverse* devuelven objetos *tibble* (`tbl_df`), los cuales son más eficientes y amigables para el flujo de trabajo.

Las principales ventajas de trabajar con `tibble` son:

- Impresión en consola más legible y controlada (muestran un número limitado de filas y columnas).
- No cambian automáticamente el tipo de datos.
- Permiten nombres de columnas con espacios o caracteres especiales si se encierran entre comillas invertidas ` (aunque **no se recomienda**).

Para crear un objeto *tibble* manualmente usamos el siguiente código:

```
datos <- tibble(
  nombre = c("Ana", "Luis", "María"),
  edad = c(34, 28, 45),
  altura = c(1.65, 1.80, 1.70)
)
```

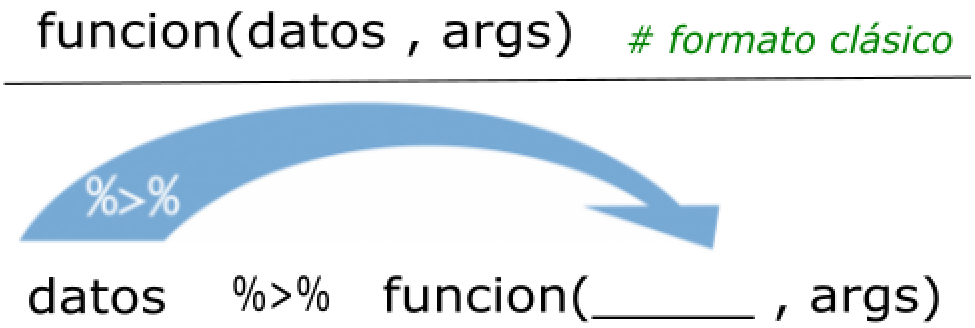
%>%
Ceci n'est pas un pipe.
www.tidyverse.org

3.3. Tuberías con `magrittr`

Una de las incorporaciones más útiles y transversales del ecosistema *tidyverse* es el uso de "*tuberías*" o *pipe operators*. Una tubería conecta un bloque de código con otro, permitiendo encadenar operaciones de manera legible. El operador %> %, proveniente del paquete `magrittr` (Bache y Wickham 2026), transforma llamadas de funciones anidadas (con múltiples paréntesis) en una secuencia de pasos más simple de leer y escribir.

A partir de la versión 4.1.0 de R, también se incorporó una **tubería nativa** (|>), con un comportamiento muy similar. Ambas opciones son válidas y su uso es prácticamente equivalente.

Este enfoque refleja el principio de que *cada función representa un paso en una secuencia lógica de transformación de datos*. La forma de trabajar se puede ver en el siguiente esquema general:



A continuación, mostramos un ejemplo comparativo de cómo cambia la sintaxis usando el dataset incorporado en R `mtcars`, que contiene datos sobre autos:

```
head(sqrt(mtcars))
```

```
                       mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

En la línea de código anterior estamos pidiendo mostrar la cabecera (6 primeras observaciones de la tabla de datos) de la raíz cuadrada de los valores de la tabla `mtcars`, en formato del lenguaje clásico (anidado).

Ahora activamos `magrittr` y ejecutamos la línea anterior en formato tubería:

```
# Activa paquete
library(magrittr)

# Formato tubería
mtcars %>%
  sqrt() %>%
  head()
```

```
                       mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

Podemos hacer lo mismo con la tubería nativa de R sin activar ningún paquete (revisar que esté activada desde `Tools > Global Options`):

```
# Tubería nativa
mtcars |>
  sqrt() |>
  head()
```

```
                       mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs  am     gear     carb
Mazda RX4         4.057093  0   1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0   1 2.000000 2.000000
Datsun 710        4.313931  1   1 2.000000 1.000000
Hornet 4 Drive    4.409082  1   0 1.732051 1.000000
Hornet Sportabout 4.125530  0   0 1.732051 1.414214
Valiant           4.496665  1   0 1.732051 1.000000
```

Las tuberías le dan mucha mas claridad al código separándolo en partes, como si fuesen oraciones de un párrafo.

3.4. Archivos de texto plano con `readr`

El paquete `readr` (Wickham, Hester, et al. 2026) contiene funciones similares a las de la familia `read.table()` de R base, pero desarrollados bajo el ecosistema `tidyverse`.

Los archivos de texto plano (ASCII u otras codificaciones) son universalmente utilizados por la mayoría de los gestores de bases de datos y planillas de cálculo. Generalmente se encuentran con extensiones `.txt` o `.csv` (por *comma-separated values*) y son el tipo de archivo de datos más habitual en R.

Estos datos planos tienen dos características principales:

- La cabecera (en inglés *header*).
- El carácter o símbolo separador que indica la separación de columnas: pueden estar separadas por comas, punto y coma, tabulación, etc.

La presencia o no de una cabecera se maneja con los argumentos `col_names` y `skip`:

- `col_names = TRUE` indica que la primera fila contiene los nombres de las columnas (cabecera).
- `col_names = FALSE` indica que no hay cabecera y las columnas se nombran automáticamente (`X1`, `X2`, etc).
- `skip = 0` (valor por defecto) lee los datos desde la primera fila, pero si hay encabezados complejos (por ejemplo, títulos y subtítulos), se puede indicar cuántas filas deben omitirse. Ejemplo: `skip = 5` omite las primeras 5 filas del archivo.

Otro aspecto a considerar es el carácter **separador** utilizado para indicar la separación entre columnas. Los separadores más comunes son:

- coma (`,`)
- punto y coma (`;`)
- tabulación (`TAB`)
- espacio (`" "`)
- barra vertical o *pipe* (`|`)

Funciones de lectura

Algunas de las funciones del paquete asumen un separador particular. Por caso `read_csv()` lee separados por coma y `read_tsv()` separado por tabulaciones, pero la función `read_delim()` permite que definamos el separador a través del argumento `delim`.

En forma detallada el paquete `readr` soporta siete formatos de archivo a partir de siete funciones:

- `read_csv()`: archivos separados por comas (CSV).
- `read_tsv()`: archivos separados por tabulaciones.
- `read_delim()`: archivos separados con delimitadores generales.
- `read_fwf()`: archivos con columnas de ancho fijo.
- `read_table()`: archivos formato tabla con columnas separadas por espacios.
- `read_log()`: archivos log web.

En comparación con las funciones R `base`, las funciones de `readr`:

- Usan un esquema de nombres consistente de parámetros.
- Son más rápidas.
- Analizan eficientemente los formatos de datos comunes (especialmente fechas y horas).
- Muestra una barra de progreso para archivos grandes.
- Vienen incluidas dentro de `tidyverse` pero también pueden usarse de forma independiente:

```r
library(readr)
```

A modo de ejemplo, leeremos un archivo sin cabecera separado por comas bajo el nombre `datos`:

```
datos <- read_csv("datos/ejemplo-datos.csv", col_names = F)
datos
```

```
# A tibble: 4 x 5
     X1 X2       X3       X4    X5
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Esteban  M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

Leemos el mismo archivo con cabecera y separado por punto y comas, bajo el nombre `info`:

```
info <- read_csv2("datos/ejemplo-datos-header.csv", col_names = T)
info
```

```
# A tibble: 4 x 5
   Iden Apellido Nombre   Sexo  FNac
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Laura    M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

En estos ejemplos:

- `read_csv()` espera comas como separador
- `read_csv2()` espera punto y coma como separador

Al leer un archivo, `readr` intenta adivinar automáticamente el tipo de dato de cada columna (*parse*). Si no hay cabecera, los nombres de columna serán `X1`, `X2`, etc.

Podemos inspeccionar la estructura de un *dataframe* con `glimpse()`:

```
glimpse(info)
```

```
Rows: 4
Columns: 5
$ Iden     <dbl> 9, 26, 35, 48
$ Apellido <chr> "Leone", "Garcia", "Salamone", "Gonzalez"
$ Nombre   <chr> "Fernando", "Laura", "Nicolas", "Viviana"
$ Sexo     <chr> "M", "M", "M", "F"
$ FNac     <date> 1958-12-24, 1954-01-21, 1993-06-27, 1965-06-21
```

Los tipos de datos posibles son:

- `character` (<chr>)
- `integer`, `double` o `numeric` (<int>, <dbl>)

- `logical` (`<lgl>`)
- `date`, `datetime`, etc.

Por ejemplo, columnas con enteros pueden aparecer como `<dbl>` si se interpretan como `double`, y las fechas como `<date>`.

Agregamos unos argumentos más y ejemplificamos la sintaxis con `read_delim()` para leer un archivo con cabecera compleja (la tabla comienza en la fila 9) separado por caracteres | (*pipes*).

```
read_delim(
  "ejemplo-datos-header-skip.txt",
  col_names = T,
  skip = 8,
  delim = "|"
)
```

> **Importante:** No olvides asignar la lectura a un nombre para guardar el *dataframe* dentro del entorno de trabajo (por ejemplo: `datos <-`).

Funciones de escritura

El paquete también incluye funciones para escribir archivos de texto plano, con formatos espejo de las funciones de lectura más comunes:

- `write_csv()`: escribe archivos separados por comas
- `write_csv2()`: escribe archivos separados por punto y comas
- `write_tsv()`: escribe archivos separados por tabulaciones
- `write_delim()`: escribe archivos separados con delimitadores definidos por el usuario

Los argumentos son generales y para el caso del último más extensos, dado que hay que definir cual es el separador que deseamos en el archivo. Podemos consultarlos con el siguiente código:

```
args(write_delim)
```

```
function (x, file, delim = " ", na = "NA", append = FALSE, col_names = !append,
    quote = c("needed", "all", "none"), escape = c("double",
        "backslash", "none"), eol = "\n", num_threads = readr_threads(),
    progress = show_progress())
NULL
```

Por ejemplo para exportar un conjunto de datos en texto plano al que denominaremos “`ejemplo.csv`“ con separador **punto y coma** y **cabecera incluida** podemos hacer:

```
write_delim(x = datos, file = "ejemplo.csv", delim = ";")
```

o más sencillo, usando la función específica `write_csv2()`:

```
write_csv2(datos, "ejemplo.csv") # define cabecera y separador ;
```

3.5. Lectura de hojas de cálculo con `readxl`

Uno de los formatos más comunes para almacenar datos son las hojas de cálculo, en particular las creadas con Microsoft Excel. El paquete `readxl` (Wickham y Bryan 2026), parte del ecosistema `tidyverse`, permite leer este tipo de archivos.

`readxl` es compatible con hojas de cálculo de Excel 97-2003, con extensión `.xls`, y con versiones más recientes, con extensión `.xlsx`.

Una primera función útil es `excel_sheets()`, que permite conocer y listar los nombres de las hojas contenidas en un archivo Excel (también llamado libro o *workbook*).

Por ejemplo, supongamos que tenemos un archivo denominado “`datos.xlsx`“ y queremos saber por cuantas hojas está compuesto y que nombre tienen:

```r
library(readxl) # hay que activarlo independientemente de tidyverse

excel_sheets("datos/datos.xlsx")
```

```
[1] "diabetes"   "vigilancia" "mortalidad"
```

Esto devuelve, por ejemplo, tres hojas: `"diabetes"`, `"vigilancia"` y `"mortalidad"`.

Para leer una de estas hojas utilizamos la función `read_excel()`, cuyos argumentos principales son:

```r
args(read_excel)
```

```
function (path, sheet = NULL, range = NULL, col_names = TRUE,
    col_types = NULL, na = "", trim_ws = TRUE, skip = 0, n_max = Inf,
    guess_max = min(1000, n_max), progress = readxl_progress(),
    .name_repair = "unique")
NULL
```

Entre los más relevantes encontramos:

- `path`: nombre del archivo y su ubicación (entre comillas)
- `sheet`: nombre de la hoja o su número de orden

- `col_names`: si es `TRUE`, toma la primera fila como nombres de las columnas
- `skip`: permite saltear un número determinado de filas antes de comenzar la lectura

Al ejecutar `read_excel()`, internamente se utiliza la función `excel_format()` para detectar si el archivo es `.xls` o `.xlsx`, y luego se aplica la función específica para cada caso: `read_xls()` o `read_xlsx()`. Estas funciones también pueden usarse directamente si se desea.

Supongamos ahora que queremos leer la hoja llamada `"diabetes"`:

```
diabetes <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "diabetes",
  col_names = T
)

# mostramos las 6 primeras observaciones
head(diabetes)
```

```
# A tibble: 6 x 8
    A1C  hba1 GLUCB   SOG Tol_Glucosa    DM    SM  HOMA
  <dbl> <dbl> <dbl> <dbl> <chr>       <dbl> <dbl> <dbl>
1  6.17   7.9   101   122 IFG             0     1  4.04
2  5.58   7.2   103   100 IFG             0     0  5.03
3  5.38   7.1   103    90 IFG             0     1  2.92
4  5.38   6.6   109    96 IFG             0     1  4.79
5  5.19   6.3   107    69 IFG             0     1  3.06
6  4.89   6      NA   117 IFG             0     0  5.77
```

Observemos que en los argumentos escribimos el nombre del archivo que se encuentra en nuestro proyecto y por lo tanto en nuestra carpeta activa, el nombre de la hoja y nos aseguramos que la primer fila representa a la cabecera de la tabla (sus nombres de variables).

Como `readxl` forma parte del ecosistema `tidyverse` el formato de salida es un *tibble*. En este caso de 23 observaciones por 8 variables.

Ahora leamos la segunda hoja de nombre `"vigilancia"`:

```
vigilancia <- read_excel(path = "datos/datos.xlsx", sheet = 2, col_names = F)

# mostramos las 6 primeras observaciones
head(vigilancia)
```

```
# A tibble: 6 x 9
   ...1 ...2        ...3      ...4  ...5  ...6  ...7 ...8  ...9
  <dbl> <chr>      <dbl>     <dbl> <dbl> <dbl> <dbl> <chr> <chr>
1   875 09/28/2015  2015 544080000     1    31     1 F     VIGILANCIA EN SALUD ~
2   875 42317       2015 544080000     1    35     1 F     VIGILANCIA EN SALUD ~
3   875 42317       2015 544080000     1    47     1 F     VIGILANCIA EN SALUD ~
4   307 09/26/2015  2015 544005273     1    23     1 M     VIGILANCIA INTEGRADA~
5   307 09/24/2015  2015 544005273     1    19     1 M     VIGILANCIA INTEGRADA~
6   875 09/28/2015  2015 544080000     1    63     1 F     VIGILANCIA EN SALUD ~
```

En este caso, en lugar del nombre de la hoja usamos un 2 que es su ubicación y especificamos `col_names = FALSE` porque el conjunto de datos no tiene cabecera. `readxl` asignará nombres genéricos como `...1`, `...2`, etc.

Finalmente leamos la última hoja disponible del archivo:

```
mortalidad <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "mortalidad",
  col_names = T,
  skip = 1
)

# mostramos las 6 primeras observaciones
head(mortalidad)
```

```
# A tibble: 5 x 10
  grupo_edad grupo.I.1.1 grupo.II.1.1 grupo.III.1.1 grupo.I.2.1 grupo.II.2.1
  <chr>            <dbl>        <dbl>         <dbl>       <dbl>        <dbl>
1 30-44               41          202           222         539         1438
2 45-59               99         1071           181         759         6210
3 60-69              114         1782           119         985         9238
4 70-79              221         2336           119        1571        12369
5 80+                362         2492            81        2523        14642
# i 4 more variables: grupo.III.2.1 <dbl>, grupo.I.3.1 <dbl>,
#   grupo.II.3.1 <dbl>, grupo.III.3.1 <dbl>
```

Lo novedoso de esta lectura es el argumento `skip = 1` que debimos incorporar dado que, en este caso, la hoja de Excel comienza con una línea de título que no pertenece al conjunto de datos. También que el argumento `sheet` permite el nombre de la hoja elegida entre comillas.

Además de los argumentos generales de `read_xl()`, podemos mencionar estos otros:

- `n_max`: número máximo de filas a leer.
- `range`: rango de celdas a importar (como en Excel, por ejemplo `"B3:D87"`).
- `col_types`: define el tipo de datos de cada columna. Valores posibles: `"numeric"`, `"logical"`, `"text"`, `"date"`, `"skip"` (no leer la columna), `"guess"` (modo predeterminado: la función decide automáticamente el tipo).
- `na`: carácter o vector de caracteres que se deben interpretar como valores perdidos (`NA`). Por defecto, las celdas vacías se interpretan así.

⚠ En los siguientes capítulos se abordarán dos componentes fundamentales del ecosistema `tidyverse`: `dplyr`, para la manipulación de datos y la construcción de tablas de resumen, y `ggplot2`, para la elaboración de gráficos estadísticos.

4. Gestión de datos con `dplyr`

4.1. Introducción

El paquete `dplyr` (Wickham, François, et al. 2026) fue desarrollado por *Hadley Wickham* como una versión optimizada del paquete `plyr` (Wickham 2011).

Su principal contribución es ofrecer una *gramática* para la manipulación de datos, basada en funciones que actúan como verbos, lo que facilita la lectura y comprensión del código.

Las funciones clave del paquete permiten realizar las siguientes acciones (verbos):

- `select()`: selecciona un conjunto de columnas (variables)
- `rename()`: renombra variables en un conjunto de datos
- `filter()`: selecciona un conjunto de filas (observaciones) según una o varias condiciones lógicas
- `arrange()`: reordena las filas de un conjunto de datos
- `mutate()`: añade nuevas variables/columnas o transforma variables existentes
- `summarise()`/ `summarize()`: genera resúmenes estadísticos de diferentes variables en el conjunto de datos
- `group_by()`: agrupa las observaciones en función de una o más variables, lo que permite realizar operaciones por grupo
- `count()`: contabiliza valores que se repiten, generando una tabla de frecuencias

Además, al ser parte del ecosistema *tidyverse*, `dplyr` integra al operador *pipe* (%>%) formando una única secuencia de procesamiento o *pipeline*.

4.1.1. Argumentos comunes

Todas las funciones, básicamente, tienen en común una serie de argumentos.

1. El primer argumento es el nombre del conjunto de datos (objeto donde esta nuestra tabla de datos).

2. Los otros argumentos describen que hacer con el conjunto de datos especificado en el primer argumento, podemos referirnos a las columnas en el objeto directamente sin utilizar el operador `$`, es decir sólo con el nombre de la columna/variable.
3. El valor de retorno es un nuevo conjunto de datos.
4. Los conjuntos de datos deben estar bien organizados/estructurados, es decir debe existir una observación por columna y, cada columna representar una variable, medida o característica de esa observación. Es decir, debe cumplir con *tidy data*.

4.1.2. Activación del paquete

`dplyr` está incluído en el núcleo base de `tidyverse`, por lo que se encuentra disponible si tenemos activado a este último.

También se puede activar en forma independiente:

```
library(dplyr)
```

4.1.3. Conjunto de datos para ejemplo

Para visualizar y comprender el funcionamiento de estos “verbos” de manipulación, resulta muy útil contar con ejemplos concretos. Por eso, en esta unidad trabajaremos con un conjunto de datos que nos permitirá practicar el uso de las funciones del paquete.

> Recuerden que pueden **descargar los datos** utilizados en los ejemplos del curso y descomprimirlos en la carpeta donde tengan guardado su proyecto de RStudio.

Uno de los archivos incluidos, “`noti-vih.csv`”, contiene registros de notificaciones de VIH por jurisdicción en Argentina correspondientes a los años 2015 y 2016.

```
# asignamos la lectura a datos
datos <- read_csv("datos/noti-vih.csv")

# mostramos las 6 primeras observaciones
head(datos)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 CABA          2015   901  3054237
4 CABA          2016   427  3050000
5 Catamarca     2015    69   396552
6 Catamarca     2016    51   401575
```

4.2. Función `select()`

La función `select()` permite elegir columnas específicas de un conjunto de datos, devolviendo una versión "recortada por columnas" del mismo.

A continuación, exploramos algunas formas útiles de seleccionar variables:

Seleccionar todas las variables excepto `pob`:

```
datos |>
  select(-pob)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Otra forma para el mismo resultado anterior (mediante el operador rango `:`):

```
datos |>
  select(jurisdiccion:casos)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Seleccionar solamente las variables `jurisdiccion` y `casos`:

```
datos |>
  select(jurisdiccion, casos)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Lo mismo que el ejemplo anterior, pero usando la posición de las columnas:

```
datos |>
  select(1, 3)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Mover la variable año al inicio y mantener todas las demás:

```
datos |>
  select("año", everything())
```

```
# A tibble: 48 x 4
     año jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Cambiar el nombre de una de la variable año y mover al inicio:

```
datos |>
  select(anio = año, everything())
```

```
# A tibble: 48 x 4
    anio jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Otros posibles argumentos son:

- `starts_with()`: selecciona todas las columnas que comiencen con el patrón indicado.
- `ends_with()`: selecciona todas las columnas que terminen con el patrón indicado.
- `contains()`: selecciona las columnas que posean el patrón indicado.
- `matches()`: similar a `contains()`, pero permite poner una expresión regular.
- `all_of()`: selecciona las variables pasadas en un vector (todos los nombres deben estar presentes o devuelve un error).
- `any_of()`: idem anterior excepto que no se genera ningún error para los nombres que no existen.
- `num_range()`: selecciona variables con nombre combinados con caracteres y números (ejemplo: `num_range("x", 1:3)` selecciona las variables `x1`, `x2` y `x3`.
- `where()`: aplica una función a todas las variables y selecciona aquellas para las cuales la función regresa TRUE (por ejemplo: `is.numeric()` para seleccionar todas las variables numéricas).

4.3. Función `rename()`

La función `rename()` puede considerarse una extensión de `select()`. Si bien `select()` también permite renombrar variables, no resulta muy útil para este fin, ya que descarta todas las variables que no se mencionan explícitamente.

En cambio, `rename()` permite cambiar el nombre de una o más variables sin eliminar las demás. Solo se modifican los nombres indicados, y el resto del conjunto de datos permanece sin cambios.

Ejemplo: renombrar la variable `pob` como `población`:

```r
datos |>
  rename("población" = pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos población
   <chr>        <dbl> <dbl>     <dbl>
 1 Buenos Aires  2015  1513  16626374
 2 Buenos Aires  2016   957  16789474
 3 CABA          2015   901   3054237
 4 CABA          2016   427   3050000
 5 Catamarca     2015    69    396552
 6 Catamarca     2016    51    401575
 7 Chaco         2015    15   1153846
 8 Chaco         2016     9   1125000
 9 Chubut        2015   110    567010
10 Chubut        2016    89    577922
# i 38 more rows
```

Se puede renombrar una serie de columnas usando `rename_with()`, por ejemplo para agregar el prefijo `"n_"` a las variables de conteo:

```r
datos |>
  rename_with(
    .cols = c(casos, pob), # Columnas a renombrar
    .fn = ~ paste0("n_", .x) # Función o vector para renombrar columnas
  )
```

```
# A tibble: 48 x 4
   jurisdiccion   año n_casos    n_pob
   <chr>        <dbl>   <dbl>    <dbl>
 1 Buenos Aires  2015    1513 16626374
 2 Buenos Aires  2016     957 16789474
 3 CABA          2015     901  3054237
 4 CABA          2016     427  3050000
 5 Catamarca     2015      69   396552
 6 Catamarca     2016      51   401575
 7 Chaco         2015      15  1153846
 8 Chaco         2016       9  1125000
 9 Chubut        2015     110   567010
```

```
10 Chubut         2016      89   577922
# i 38 more rows
```

4.4. Función `filter()`

La función `filter()` permite seleccionar filas de un conjunto de datos, produciendo un subconjunto de observaciones.

Veamos un ejemplo sencillo con nuestros datos:

```
datos |>
  filter(jurisdiccion == "Tucuman")
```

```
# A tibble: 2 x 4
  jurisdiccion   año casos     pob
  <chr>        <dbl> <dbl>   <dbl>
1 Tucuman       2015   258 1592593
2 Tucuman       2016   246 1618421
```

Utiliza los mismos operadores de comparación propios del lenguaje R:

Operador	Descripción
<	Menor que
>	Menor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que
%in %	Es parte de
is.na()	Es un valor ausente
!is.na()	No es un valor ausente

Lo mismo con los operadores lógicos que se utilizan como conectores entre las expresiones:

Operador	Descripción
&	AND booleano
|	OR booleano
xor()	OR exclusivo
!	NOT
any()	cualquier TRUE
all()	todos TRUE

Cuando usamos múltiples argumentos separados por coma dentro de `filter()`, estas se combinan con un operador **AND** implícito, es decir, cada expresión debe ser verdadera para que la fila sea incluida en la salida.

Por ejemplo, filtramos las observaciones que cumplan que `casos` sea mayor a 100 **y** que `pob` sea menor a 1.000.000:

```
datos |>
  filter(casos > 100, pob < 1000000)
```

```
# A tibble: 7 x 4
  jurisdiccion   año casos    pob
  <chr>        <dbl> <dbl>  <dbl>
1 Chubut        2015   110 567010
2 Jujuy         2015   160 727273
3 Jujuy         2016   133 734807
4 Neuquen       2015   109 619318
5 Neuquen       2016   101 627329
6 Rio Negro     2015   112 700000
7 Rio Negro     2016   105 709459
```

Para combinar condiciones dentro de una misma variable usamos el operador OR (|) o, de forma más práctica, `%in%`:

```
# Con OR
datos |>
  filter(jurisdiccion == "Buenos Aires" | jurisdiccion == "La Pampa")
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

```
# Con %in%
datos |>
  filter(jurisdiccion %in% c("Buenos Aires", "La Pampa"))
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

En el siguiente ejemplo, filtramos observaciones del año 2016 con más de 200 casos. El uso de & es equivalente al uso de coma:

```
datos |>
  filter(año == "2016" & casos > 200)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2016   957 16789474
2 CABA          2016   427  3050000
3 Cordoba       2016   368  3607843
4 Mendoza       2016   254  1909774
5 Salta         2016   230  1352941
6 Tucuman       2016   246  1618421
```

Por último, podemos usar `xor()` para seleccionar observaciones que cumplan **solo una** de las condiciones, pero no ambas. Por ejemplo, el siguiente filtro selecciona registros donde el año sea 2016 ó los casos sean mayores a 200, pero no ambos al mismo tiempo (es decir que no se den ambos en `TRUE`):

```
datos |>
  filter(xor(año == "2016", casos > 200))
```

```
# A tibble: 25 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 CABA          2015   901  3054237
 3 Catamarca     2016    51   401575
 4 Chaco         2016     9  1125000
 5 Chubut        2016    89   577922
 6 Cordoba       2015   468  3572519
 7 Corrientes    2016    99  1076087
 8 Entre Rios    2016   109  1329268
 9 Formosa       2016    60   582524
10 Jujuy         2016   133   734807
# i 15 more rows
```

En las últimas versiones de `dplyr` se añadió la función `filter_out()`, que equivale a usar el operador `!` para negar una condición. Por ejemplo, si quisiéramos excluir todos los datos de la provincia de Mendoza:

```
datos |>
  filter_out(jurisdiccion == "Mendoza")
```

```
# A tibble: 46 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
 9 Chubut          2015   110   567010
10 Chubut          2016    89   577922
# i 36 more rows
```

4.5. Función `arrange()`

La función `arrange()` se utiliza para ordenar las filas de un conjunto de datos de acuerdo a una o varias columnas/variables. Por defecto, el ordenamiento es ascendente alfanumérico.

Ordenamos la tabla por la variable `pob` (forma ascendente predeterminada):

```
datos |>
  arrange(pob)
```

```
# A tibble: 48 x 4
   jurisdiccion       año casos    pob
   <chr>            <dbl> <dbl>  <dbl>
 1 Tierra del Fuego  2015    36 152542
 2 Tierra del Fuego  2016    34 156682
 3 Santa Cruz        2015    65 320197
 4 Santa Cruz        2016    59 329609
 5 La Pampa          2015    57 343373
 6 La Pampa          2016    67 345361
 7 La Rioja          2015    41 369369
 8 La Rioja          2016     6 375000
 9 Catamarca         2015    69 396552
10 Catamarca         2016    51 401575
# i 38 more rows
```

Para ordenar en forma descendente podemos utilizar `desc()` dentro de los argumentos de `arrange()`:

```
datos |>
  arrange(desc(pob))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Que equivale a:

```
datos |>
  arrange(-pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Podemos combinar ordenamientos. Por ejemplo, en forma alfabética ascendente para `jusrisdiccion` y luego numérica descendente para `casos`:

```
datos |>
  arrange(jurisdiccion, desc(casos))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
 9 Chubut        2015   110   567010
10 Chubut        2016    89   577922
# i 38 more rows
```

4.6. Función `mutate()`

Esta función nos permite transformar variables dentro de un conjunto de datos. A menudo tendremos la necesidad de modificar variables existentes o crear nuevas variables a partir de las ya disponibles. La función `mutate()` nos ofrece una forma clara y eficiente de realizar este tipo de operaciones.

Por ejemplo, podríamos querer calcular tasas crudas para cada jurisdicción y año, en función del número de casos y de la población total:

```
datos |>
  mutate(tasa = casos / pob * 100000)
```

```
# A tibble: 48 x 5
   jurisdiccion   año casos      pob  tasa
   <chr>        <dbl> <dbl>    <dbl> <dbl>
 1 Buenos Aires  2015  1513 16626374  9.10
 2 Buenos Aires  2016   957 16789474  5.70
 3 CABA          2015   901  3054237 29.5
 4 CABA          2016   427  3050000 14
 5 Catamarca     2015    69   396552 17.4
 6 Catamarca     2016    51   401575 12.7
 7 Chaco         2015    15  1153846  1.30
 8 Chaco         2016     9  1125000  0.8
 9 Chubut        2015   110   567010 19.4
10 Chubut        2016    89   577922 15.4
# i 38 more rows
```

En este caso, `mutate()` calcula la tasa cruda por 100.000 habitantes e incorpora una nueva variable (`tasa`) con los resultados correspondientes a cada observación.

También se pueden construir múltiples variables en la misma expresión, solamente separadas por comas:

```
datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

```
# A tibble: 48 x 6
   jurisdiccion   año casos      pob tasaxcien_mil tasaxdiez_mil
   <chr>        <dbl> <dbl>    <dbl>         <dbl>         <dbl>
 1 Buenos Aires  2015  1513 16626374          9.10         0.910
 2 Buenos Aires  2016   957 16789474          5.70         0.570
 3 CABA          2015   901  3054237         29.5          2.95
 4 CABA          2016   427  3050000         14            1.4
 5 Catamarca     2015    69   396552         17.4          1.74
 6 Catamarca     2016    51   401575         12.7          1.27
 7 Chaco         2015    15  1153846          1.30         0.130
 8 Chaco         2016     9  1125000          0.8          0.08
 9 Chubut        2015   110   567010         19.4          1.94
10 Chubut        2016    89   577922         15.4          1.54
# i 38 more rows
```

Si deseamos que estas nuevas variables se incorporen de forma permanente al conjunto de datos (y no solo se muestren en la consola), debemos utilizar el operador de asignación `<-`:

```
datos <- datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

Un aspecto fundamental es que las funciones utilizadas dentro de `mutate()` deben estar *vectorizadas*: deben aceptar un vector de entrada y devolver otro vector del mismo tamaño como salida.

Existen muchas funciones que se pueden utilizar dentro de `mutate()`. A continuación se presentan algunas útiles:

- **Operadores aritméticos**: +, −, ∗, /, ^.
- **Aritmética modular**: `%/%` (división entera) y `%%` (resto), donde `x == y * (x %/% y) + (x %% y)`.
- **Funciones matemáticas**: `log()`, `log2()`, `log10()`, `exp()`, `sqrt()`, `abs()`, entre otras.
- **Valores acumulados**: `cumsum()`, `cumprod()`, `cummin()`, `cummax()` y `cummean()`.
- **Clasificación o *ranking***: funciones como `min_rank()` permiten asignar rangos (1º, 2º, etc.). Por defecto, los valores más pequeños reciben rangos más bajos. Si se desea invertir el orden, puede utilizarse `desc(x)`.

En versiones recientes de `dplyr`, `mutate()` incorpora argumentos que facilitan el trabajo con datos:

- `.by`: permite aplicar transformaciones por grupos de manera **temporal**, sin necesidad de usar `group_by()`
- `.before` / `.after`: permiten ubicar la nueva variable antes o después de otra columna del conjunto de datos

Finalmente, si se utiliza en `mutate()` el mismo nombre de una variable que ya existe en la tabla, dicha variable será sobrescrita (por ejemplo, al cambiarle el tipo de `character` a `factor`). Si se desea crear una variable nueva, se debe utilizar un nombre que no esté previamente en el conjunto de datos.

4.7. Funciones `group_by()` y `summarise()`

Estas dos funciones suelen utilizan en conjunto para calcular **resúmenes estadísticos por grupos**. En el caso de `summarise()` / `summarize()`, podemos reducir un conjunto de datos a uno o más valores resumen.

Por ejemplo:

```
datos |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 1 x 2
  promedio_casos casos_totales
           <dbl>         <dbl>
1           192.          9211
```

Así, se obtiene una tabla con una sola fila que contiene el promedio y el total de casos en todo el conjunto de datos.

La función `group_by()` permite definir grupos dentro del conjunto de datos según una o más variables. A partir de ese momento, las operaciones siguientes se aplican **de forma independiente dentro de cada grupo**.

Por ejemplo, si queremos calcular los mismos indicadores por año:

```
datos |>
  group_by(año) |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 2 x 3
    año promedio_casos casos_totales
  <dbl>          <dbl>         <dbl>
1  2015           224.          5369
2  2016           160.          3842
```

El resultado es una tabla con una fila por cada año, mostrando los valores correspondientes a cada grupo.

También es posible agrupar por más de una variable. Por ejemplo, calcular una tasa por jurisdicción y año:

```
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  )
```

```
# A tibble: 48 x 3
# Groups:   jurisdiccion [24]
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

Después de usar `group_by()`, los datos permanecen agrupados. Si se desea continuar trabajando sin esa estructura, se puede utilizar la función `ungroup()` o el argumento `.groups = "drop"` dentro de `summarise()`:

```
# Descartar grupos con ungroup()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  ) |>
  ungroup()
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5 
 4 CABA          2016 14   
 5 Catamarca     2015 17.4 
 6 Catamarca     2016 12.7 
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8 
 9 Chubut        2015 19.4 
10 Chubut        2016 15.4 
# i 38 more rows
```

```
# Descartar grupos dentro de summarise()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000,
    .groups = "drop"
  )
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5 
 4 CABA          2016 14   
 5 Catamarca     2015 17.4 
 6 Catamarca     2016 12.7 
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8 
 9 Chubut        2015 19.4 
10 Chubut        2016 15.4 
# i 38 more rows
```

4.7.1. Funciones de resumen más frecuentes

Algunas de las funciones del R `base` que se pueden utilizar dentro de los argumentos de esta función son:

- `min()`: mínimo
- `max()`: máximo
- `mean()`: media
- `median()`: mediana
- `var()`: varianza
- `sd()`: desvío
- `sum()`: sumatoria

Otras funciones que se pueden incorporar las provee el mismo paquete `dplyr`, por ejemplo:

- `first()`: primer valor en el vector.
- `last()`: último valor en el vector.
- `n()`: número de valores en el vector.
- `n_distinct()`: números de valores distintos en el vector.

4.7.2. Combinaciones

En los ejemplos anteriores vimos cómo se van integrando algunas de las funciones mediante el uso del operador de tubería %>% o `|>`. La idea detrás de esta "*gramática de los datos*" que propone el paquete `dplyr` es poder encadenar acciones de forma legible y lógica, construyendo oraciones más complejas paso a paso.

Veamos un ejemplo que integra muchas de las funciones vistas hasta ahora:

> Obtener una nueva tabla con las tasas crudas de casos notificados de VIH, por año y jurisdicción, **mayores a 20 por 100.000 habitantes**, ordenadas de **mayor a menor**.

```
datos |> # siempre partimos de los datos
  group_by(año, jurisdiccion) |> # agrupamos
  summarise(tasa = casos / pob * 100000) |> # resumimos
  filter(tasa > 20) |> # filtramos
  arrange(desc(tasa)) # ordenamos
```

```
# A tibble: 5 x 3
# Groups:   año [2]
    año jurisdiccion      tasa
  <dbl> <chr>            <dbl>
1  2015 CABA              29.5
2  2015 Tierra del Fuego  23.6
3  2015 Jujuy             22.0
4  2016 Tierra del Fuego  21.7
5  2015 Santa Cruz        20.3
```

Una buena práctica para construir este tipo de código es escribir cada paso de la operación **en una línea separada**. Esto no solo mejora la legibilidad, sino que facilita la identificación de errores o la modificación de pasos específicos.

Este ejemplo muestra claramente el poder y la claridad que se logran al combinar funciones como `group_by()`, `summarise()`, `filter()` y `arrange()` en una misma operación fluida y coherente.

4.8. Función `count()`

Esta función permite contar rápidamente los valores únicos de una o más variables en un conjunto de datos. Es especialmente útil para crear tablas de frecuencias absolutas, que posteriormente pueden ser usadas para calcular frecuencias relativas.

Tiene un par de argumentos opcionales:

- `name`: Define el nombre de la columna que contendrá el conteo. Por defecto, esta columna se llama `n`.
- `sort`: Ordena la tabla de frecuencias de mayor a menor (por defecto, no realiza ninguna ordenación).
- `wt`: Permite incorporar una variable que funcione como ponderación (o factor de expansión) para el cálculo de la frecuencia.

Un ejemplo básico de su uso es contar las observaciones por cada valor único de la variable `jurisdiccion` en el conjunto de datos:

```
datos |>
  count(jurisdiccion)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <int>
 1 Buenos Aires     2
 2 CABA             2
 3 Catamarca        2
 4 Chaco            2
 5 Chubut           2
 6 Cordoba          2
 7 Corrientes       2
 8 Entre Rios       2
 9 Formosa          2
10 Jujuy            2
# i 14 more rows
```

Usando el argumento `wt` obtenemos el conteo de casos por jurisdicción:

```
datos |>
  count(jurisdiccion, wt = casos)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

Lo cual es equivalente a:

```
datos |>
  group_by(jurisdiccion) |>
  summarise(
    n = sum(casos, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

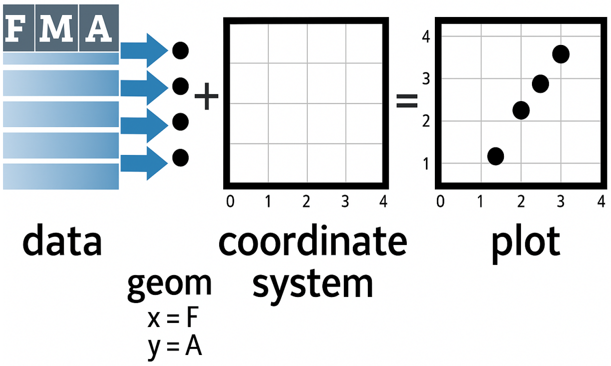
5. Gráficos estadísticos con `ggplot2`

5.1. Introducción

El paquete `ggplot2` (Wickham 2016) es parte del ecosistema `tidyverse` y se autodefine como una librería para *"crear elegantes visualizaciones de datos utilizando una gramática de gráficos"*. Proporciona una forma intuitiva de construir gráficos basada en **The Grammar of Graphics** a través de un sistema basado en tres componentes básicos:

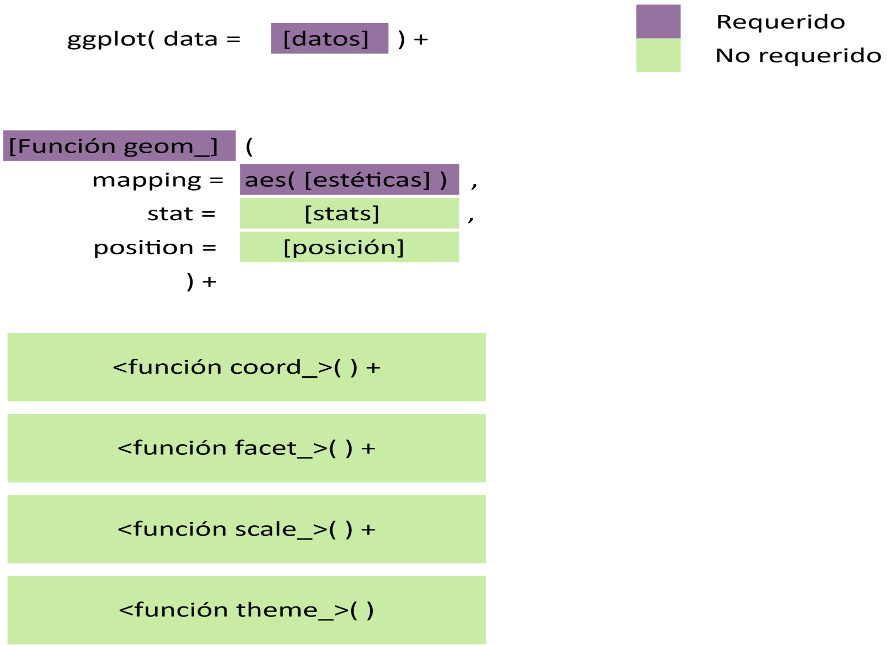
- datos
- coordenadas
- objetos geométricos

La estructura para construir un gráfico es la siguiente:



5.2. Anatomía de gráficos con `ggplot2`

La construcción de gráficos en `ggplot2` se organiza a partir de una gramática que puede entenderse a través de sus componentes fundamentales:



- `data`: el conjunto de datos que vamos a graficar, debe contener toda la información necesaria.
- `aes()`: el mapeo estético (*aesthetic mapping*) es donde se declaran las variables que se representarán en el gráfico (por ejemplo, los ejes X e Y o cómo se asignarán los colores).
- `geoms`: representaciones gráficas de los datos (puntos, líneas, barras, cajas, etc.). Son las “geometrías” que dibujan el gráfico.
- `stats`: transformaciones estadísticas sobre los datos (medias, regresiones, suavizados, etc.), que permiten resumir la información.
- `scales`: controlan cómo se representan los datos (ejes, colores, leyendas).
- `coordinate systems`: sistema de coordenadas utilizado para representar el gráfico en un plano bidimensional.
- `facets`:permiten dividir los datos en subconjuntos y generar gráficos en paneles (viñetas).
- `theme`: controla la apariencia general de los elementos no asociados a los datos (fondos, tipografías, bordes, etc.).

5.2.1. Ejemplo con datos

Antes de mostrar como usar estos componentes, cargaremos la base de datos “`facultad.csv`”, que contiene datos ficticios sobre ingresantes a una facultad (por ejemplo, sexo, edad, talla y peso). Este conjunto se utilizará en los ejemplos:

```r
# Cargar datos
facultad <- read_csv("datos/facultad.csv")

# Mostramos las 6 primeras observaciones
head(facultad)
```

```
# A tibble: 6 x 18
     hc sexo   edad ant_diabetes ant_tbc ant_cancer ant_obesidad ant_ecv ant_ht
  <dbl> <chr> <dbl> <chr>        <chr>   <chr>      <chr>        <chr>   <chr>
1 26880 M        17 NO           NO      NO         SI           NO      SI
2 26775 M        18 SI           NO      NO         NO           NO      NO
3 26877 M        18 SI           NO      SI         NO           NO      SI
4 26776 M        18 NO           NO      NO         SI           SI      NO
5 26718 M        18 NO           NO      NO         NO           NO      SI
6 26738 M        18 NO           NO      NO         NO           NO      SI
# i 9 more variables: ant_col <chr>, fuma <chr>, edadini <dbl>, cantidad <dbl>,
#   col <dbl>, peso <dbl>, talla <dbl>, sist <dbl>, diast <dbl>
```

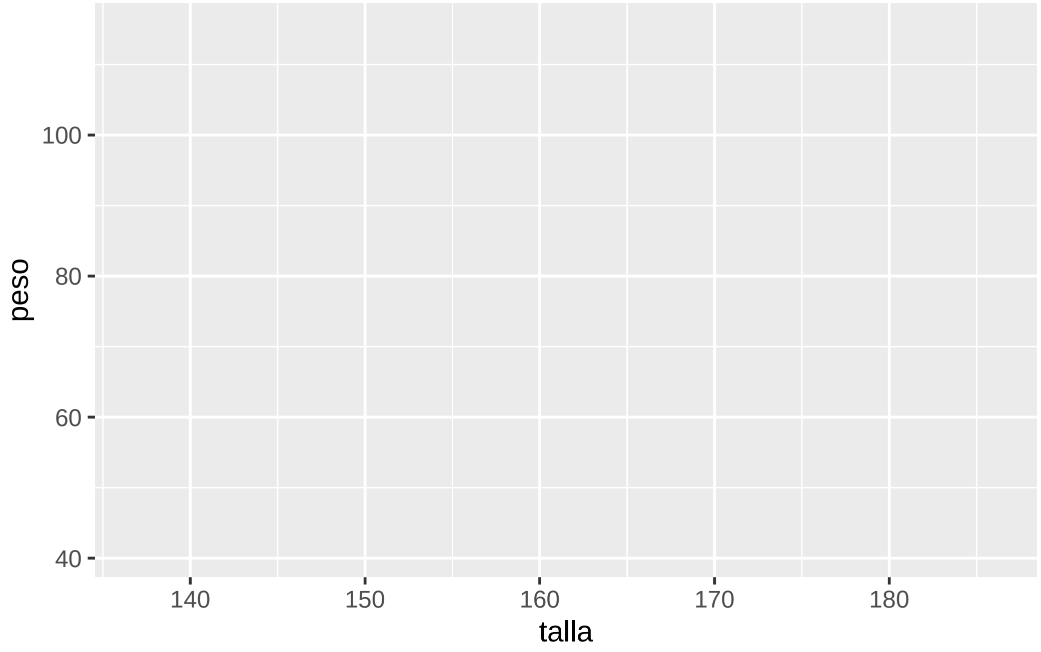
5.2.2. Mapeo estético

La función básica para construir un gráfico en `ggplot2` es `ggplot()`, que crea una estructura inicial (vacía) a la que le irán agregando capas. Anteriormente dijimos que la función `aes()` define el contenido estético del gráfico, es decir, indica cómo se representan visualmente las variables (posición, color, tamaño, forma, etc.).

Es importante notar que: - Si aes() se incluye dentro de ggplot(), el mapeo se aplica a todas las capas. - Si se incluye dentro de una geometría, se aplica solo a esa capa. - aes() genera automáticamente leyendas cuando corresponde.

Veamos cómo funciona y cuáles son sus implicancias:

```
facultad |>
  # solo la capa estética aes()
  ggplot(aes(x = talla, y = peso))
```



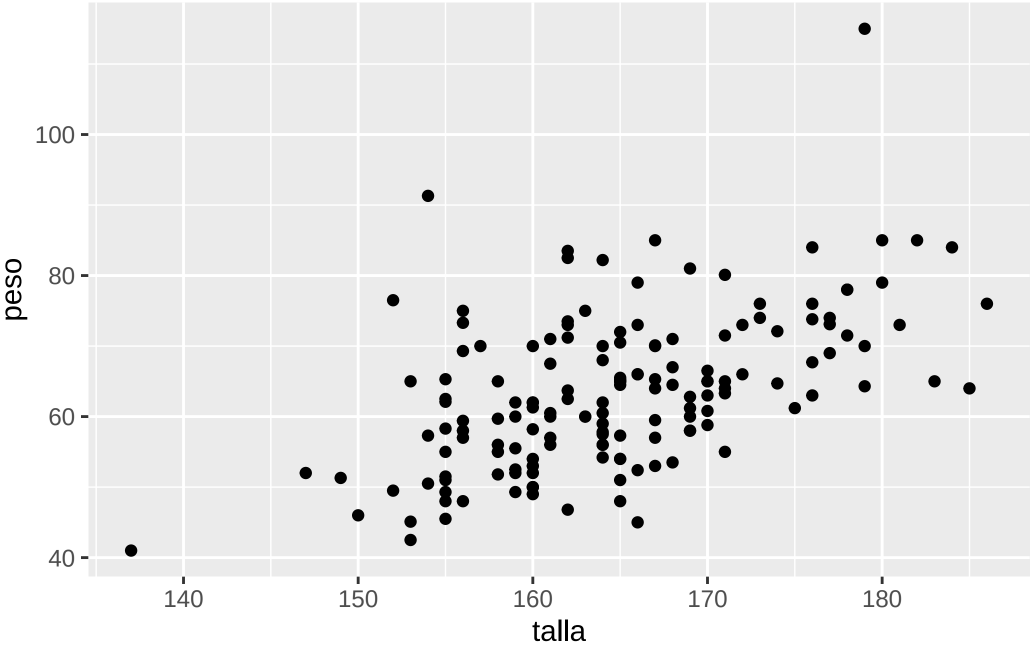
Este código genera un gráfico vacío que contiene únicamente los ejes definidos (`peso` y `talla`), pero aún no muestra los datos.

5.2.3. Capas geométricas

Para visualizar los datos, es necesario agregar capas geométricas mediante funciones `geom_`. Estas representan los objetos gráficos que se dibujan en el gráfico.

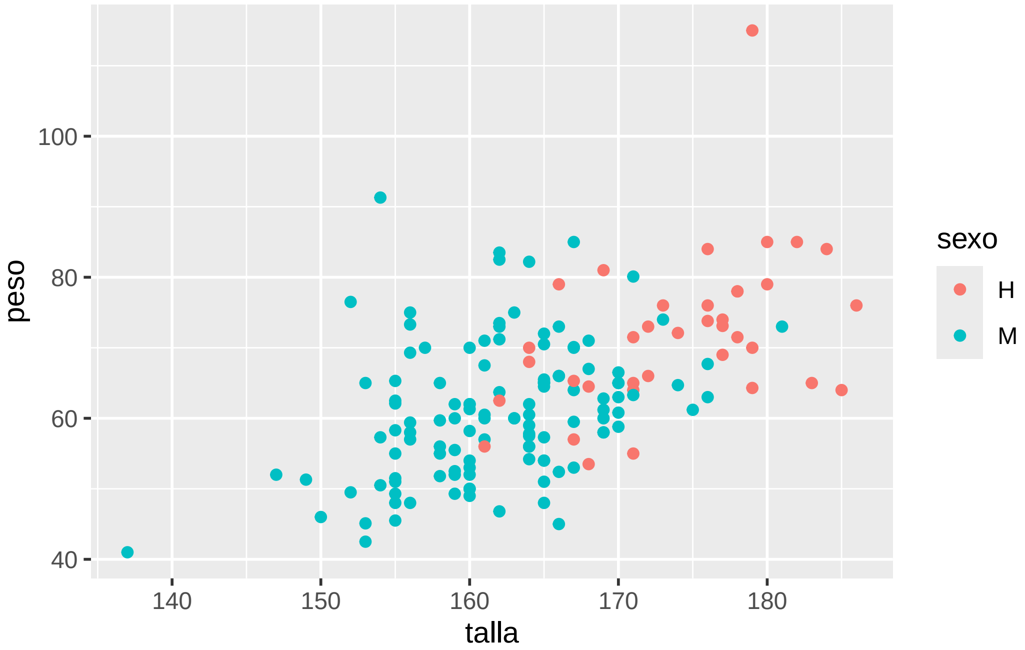
Las capas se agregan utilizando el operador +:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

  # agregamos la capa geométrica de puntos
  geom_point()
```



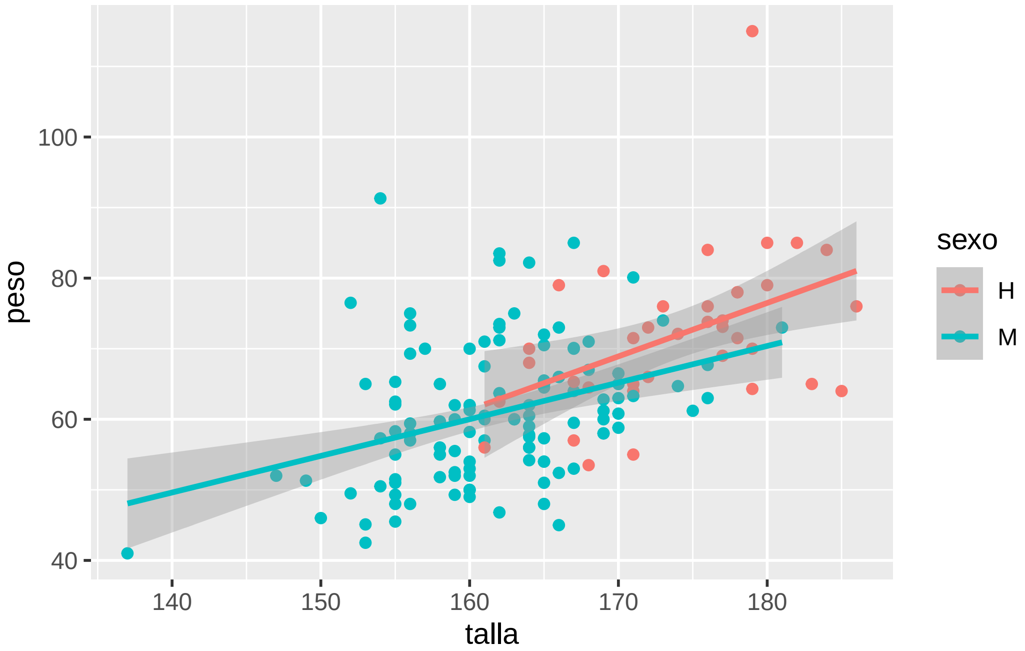
Podemos diferenciar los puntos según `sexo` incorporando esa variable al mapeo estético:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point()
```



También es posible superponer capas. Por ejemplo, agregar rectas de regresión:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



La función `geom_smooth()` permite aplicar distintos métodos de ajuste. En este caso, `"lm"` indica una regresión lineal, incluyendo su intervalo de confianza.

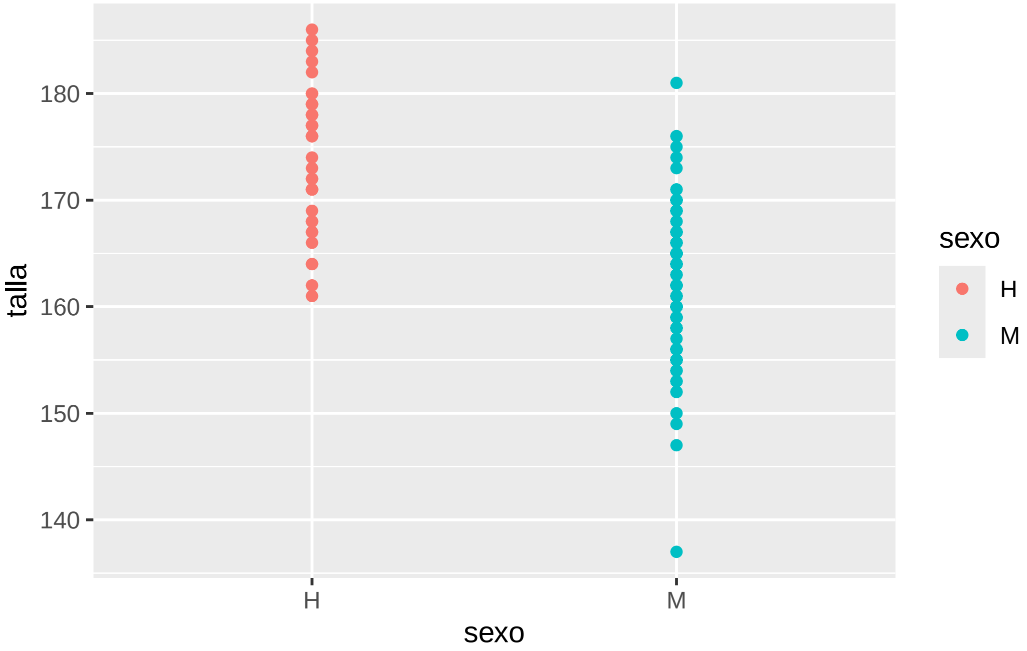
Algunas de las capas geométricas más utilizadas son:

- Histogramas: `geom_histogram()`
- Gráfico de caja y bigotes (*boxplot*): `geom_boxplot()`
- Gráfico de barras: `geom_bar()`
- Gráfico de columnas: `geom_col()`
- Gráfico de puntos (*scatter plot*): `geom_point()` o `geom_jitter()`
- Gráfico de líneas: `geom_line()` o `geom_path()`
- Gráfico de violín: `geom_violin()`
- Tendencias: `geom_smooth()`
- Curvas de densidad: `geom_density()`

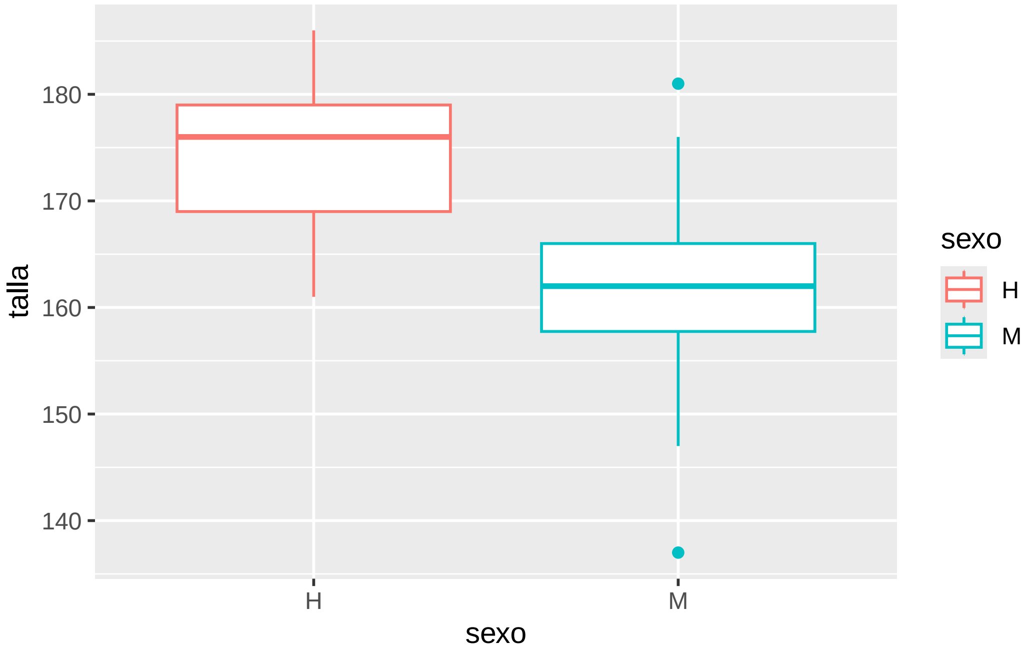
Todas estas funciones pueden aplicarse sobre los mismos datos, y su uso depende del objetivo del análisis. Las capas se grafican secuencialmente, de modo que la última geometría en añadirse aparece adelante.

A continuación, algunos ejemplos para visualizar la relación entre `sexo` y `talla` utilizando distintas capas geométricas:

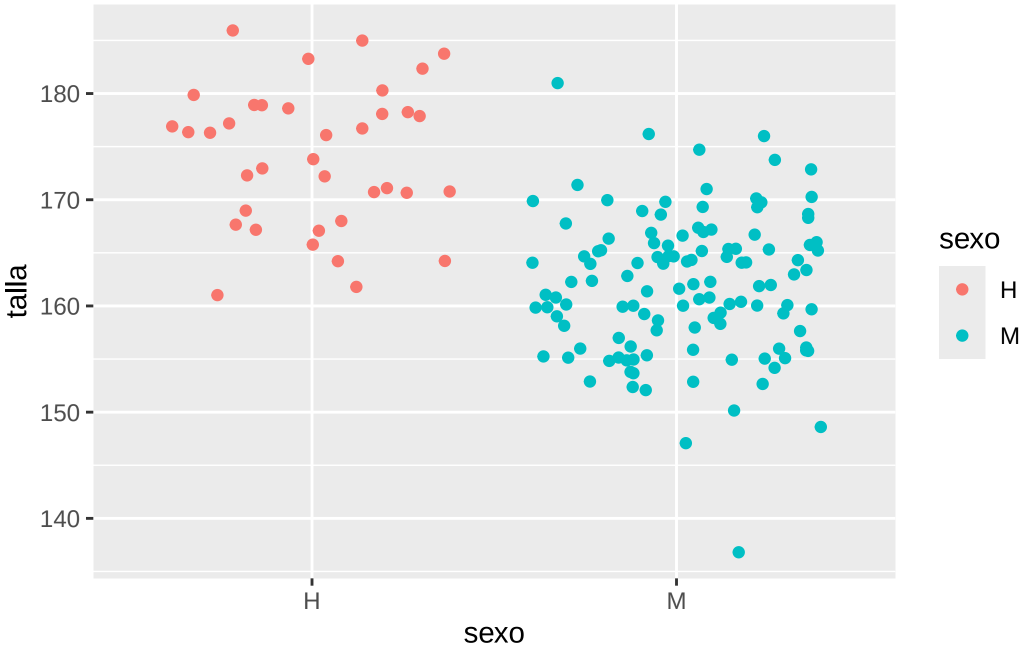
```r
# Gráfico de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de puntos
  geom_point()
```



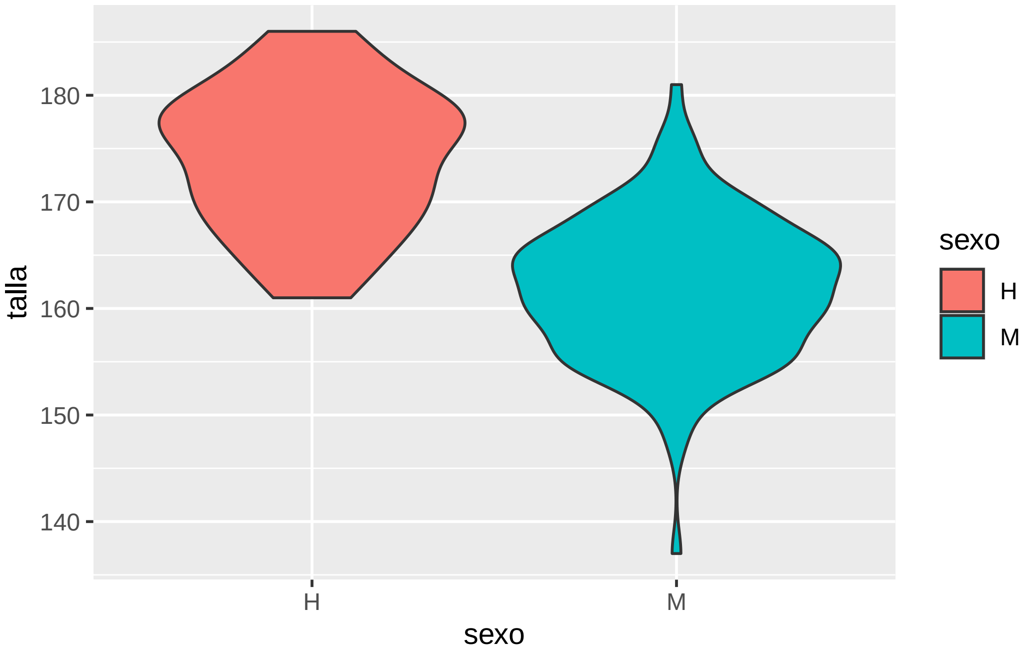
```
# Boxplot
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de boxplot
  geom_boxplot()
```



```
# Entramado de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica jitter (entramado de puntos)
  geom_jitter()
```



```
# Gráfico de violín
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +
  # capa geométrica de violin
  geom_violin()
```



En el último caso se utiliza `fill` en lugar de `color`. Mientras `color` controla el contorno, `fill` define el relleno de las geometrías.

5.2.4. Estéticas estáticas y dinámicas

En `ggplot2`, las estéticas son propiedades visuales asociadas a los datos en las capas geométricas. No incluyen elementos como títulos o etiquetas.

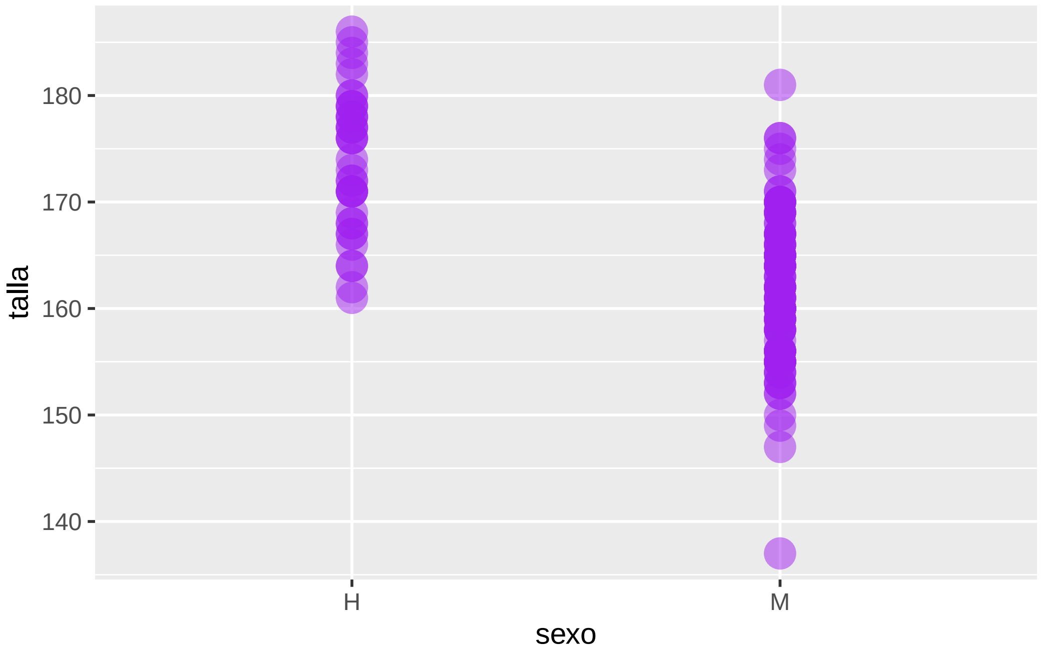
Algunas estéticas frecuentes:

- `color` / `colour`
- `fill`
- `alpha`
- `size`
- `linewidth` / `lwd`
- `linetype`/ `lty`

Valores estáticos

Si queremos que todas las observaciones tengan una misma estética, colocamos los argumentos por fuera de `aes()`:

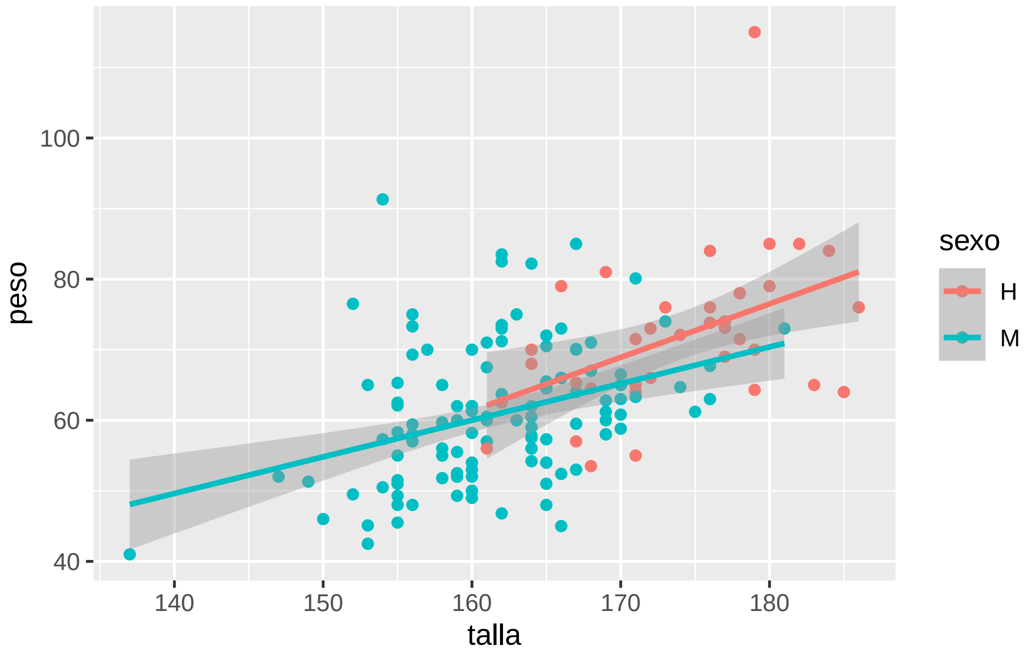
```
facultad |>
  ggplot(aes(x = sexo, y = talla)) +
  # capa geométrica de puntos
  geom_point(
    color = "purple", # Color de los puntos
    size = 5, # Tamaño de los puntos
    alpha = .5 # Transparencia de los puntos
  )
```



Valores dinámicos

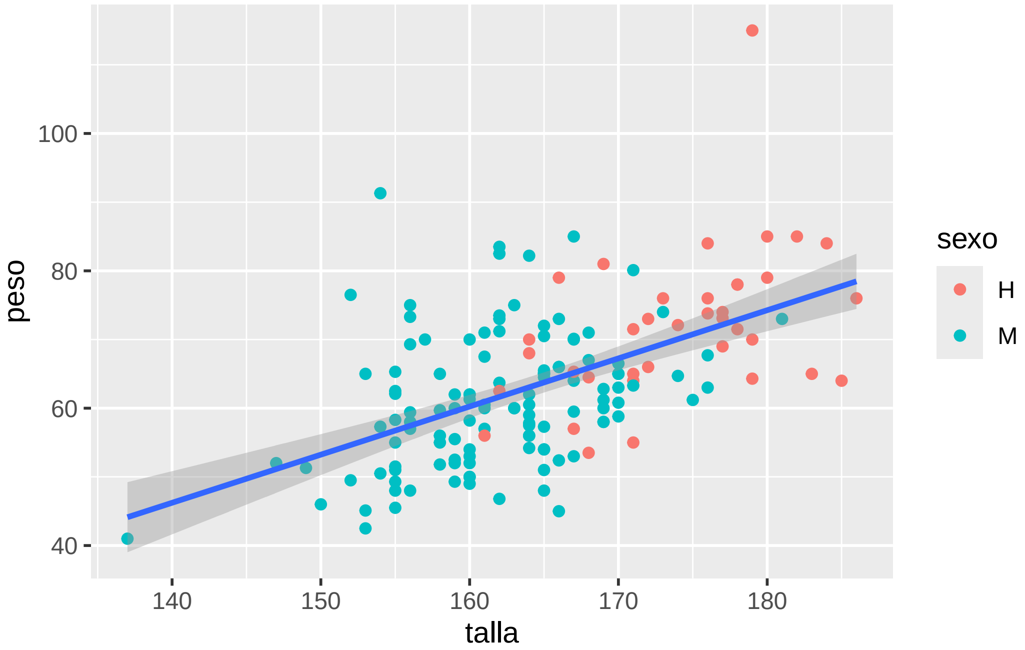
Se definen dentro de `aes()` y aplican a todas las capas:

```
facultad |>
  # Mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



En este otro ejemplo, el color por sexo solo se aplica a los puntos de `geom_point()`, mientras que la regresión se calcula sobre el conjunto completo:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

  # agregamos la capa geométrica de puntos coloreada por sexo
  geom_point(aes(color = sexo)) +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



Este comportamiento permite una gran flexibilidad al construir gráficos, definiendo qué capas responden a cada mapeo estético.

5.3. Personalización de escalas

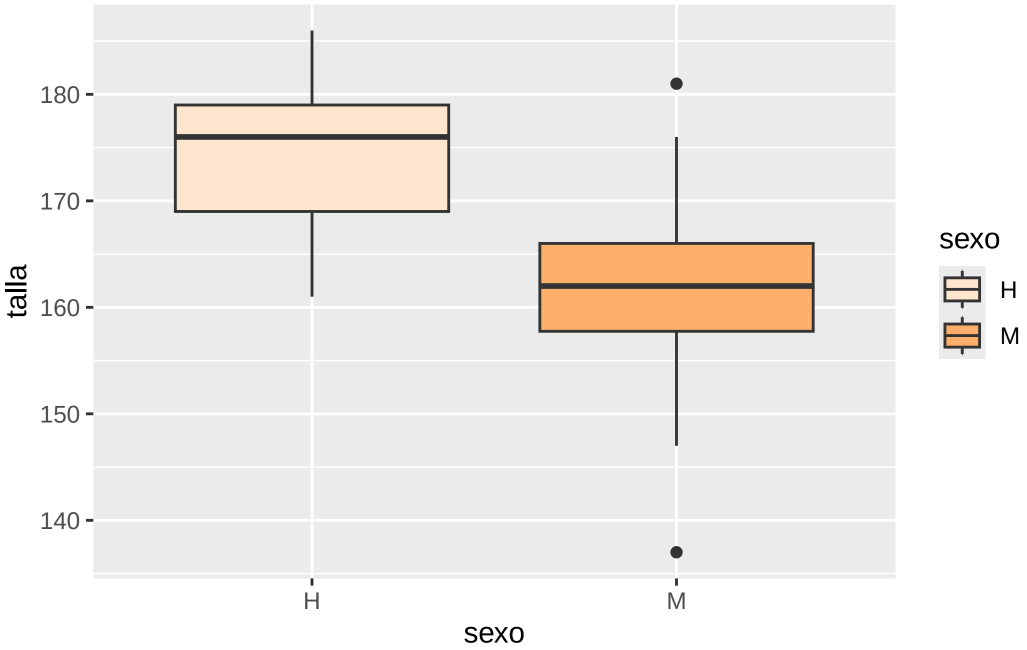
El sistema de escalas en `ggplot2` permite ajustar distintos aspectos visuales de un gráfico, como colores de contorno y relleno, ejes, tamaños y tipos de línea, entre otros.

Todas las funciones asociadas a escalas comienzan con el prefijo `scale_`, seguido del atributo que se desea modificar (por ejemplo, `scale_fill_` para el color de relleno).

5.3.1. Escalas de colores

A continuación, se presentan algunos ejemplos utilizando el conjunto de datos `facultad`:

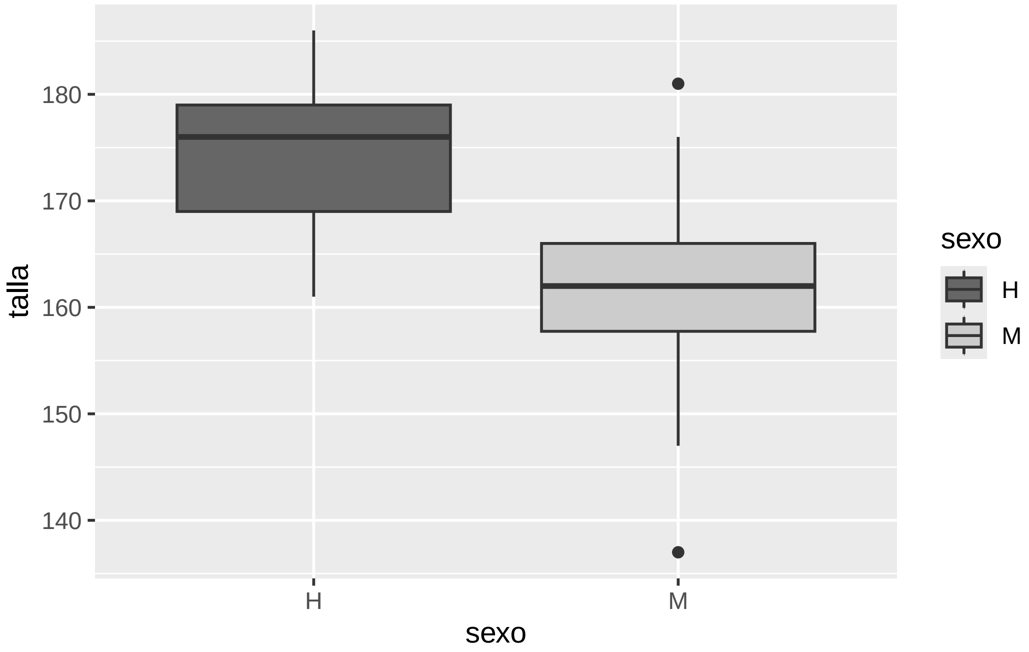
```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta de naranjas
  scale_fill_brewer(palette = "Oranges")
```



En este ejemplo, se aplica la función `scale_fill_brewer()`, que utiliza una paleta de colores (en este caso, `"Oranges"`) y se vincula con el argumento `fill` definido en `aes()`, determinando el color de relleno del *boxplot*.

Otra alternativa es utilizar una escala de grises mediante `scale_fill_grey()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8)
```

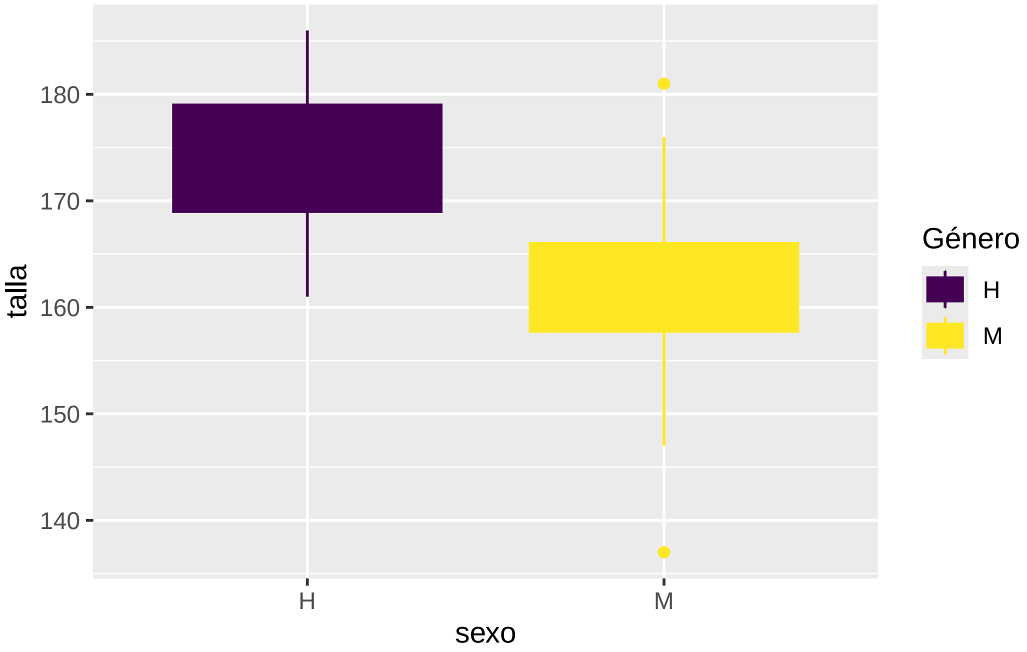


> Se recomienda trabajar con paletas de colores accesibles para personas con daltonismo y otras dificultades en la percepción visual, como las incluidas en las dependencias `RColorBrewer` y `viridisLite`, así como en paquetes específicos con paletas *colorblind-friendly*, como `scico` (Pedersen y Crameri 2023) y `cols4all` (Tennekes y Puts 2023).

En versiones recientes de `ggplot2`, varias funciones de escala permiten utilizar el argumento `aesthetics`, que facilita aplicar una misma escala a múltiples estéticas simultáneamente. Esto resulta especialmente útil cuando queremos usar una misma paleta de colores tanto para el contorno (`color`) como para el relleno (`fill`).

Retomando el ejemplo anterior:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo, color = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala viridis
  scale_fill_viridis_d(
    name = "Género",
    aesthetics = c("fill", "color")
  )
```



En este caso, la paleta `"viridis"` se aplica simultáneamente al relleno y al contorno de las cajas, evitando inconsistencias visuales entre ambas estéticas.

Sin `aesthetics` los colores de relleno y de línea deberían especificarse como:

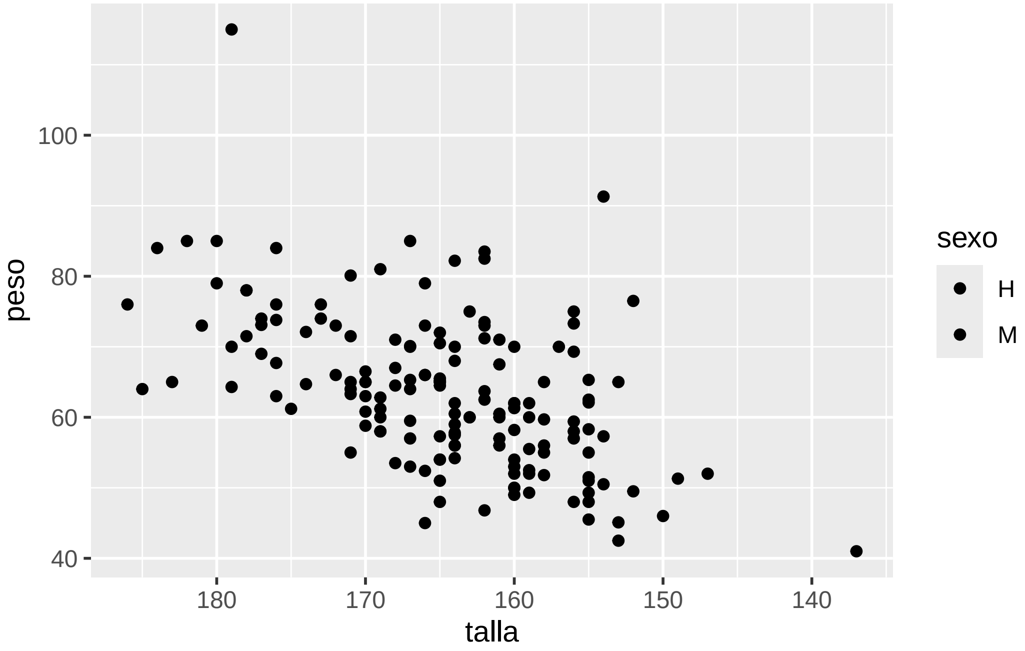
```
# paleta en escala viridis
scale_fill_viridis_d(name = "Género") +
  scale_color_viridis_d()
```

Lo que duplica código y puede desincronizar los cambios en una paleta.

5.3.2. Escalas de ejes

También es posible modificar las escalas de los ejes. Por ejemplo, invertir el eje X con `scale_x_reverse()`:

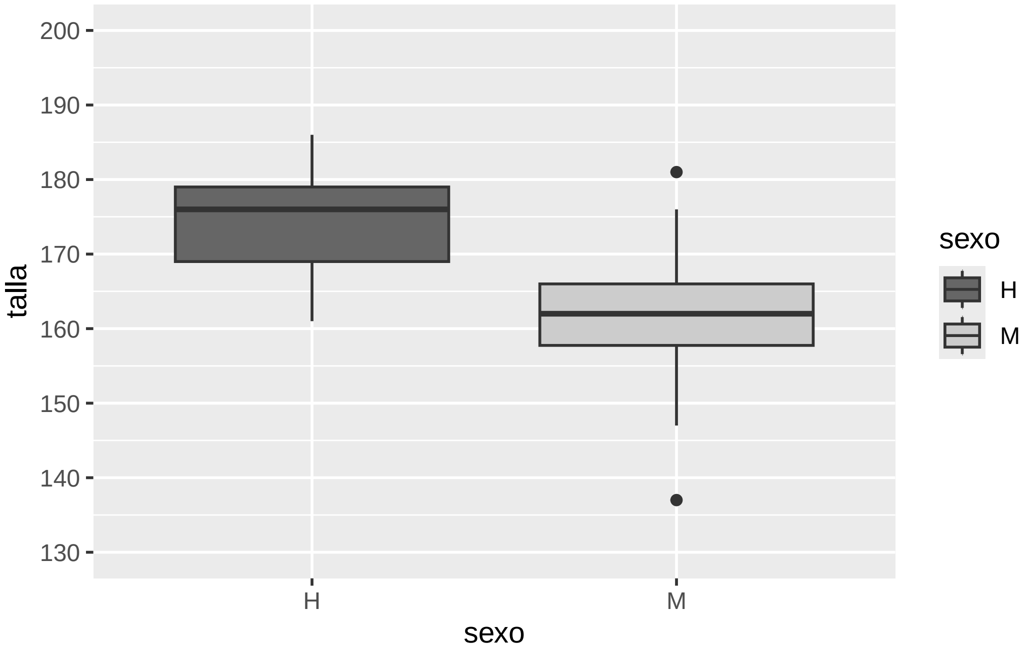
```
facultad |>
  ggplot(aes(x = talla, y = peso, fill = sexo)) +

  # capa geométrica de puntos
  geom_point() +

  # invierte el eje x
  scale_x_reverse()
```



La inclusión de `scale_x_reverse()` provoca que la variable `talla` se muestre en orden descendente (de mayor a menor).

Por último, se pueden personalizar los cortes del eje Y mediante `scale_y_continuous()`. En el siguiente ejemplo, se define que el eje Y vaya de 130 a 200 cm, con marcas cada 10 unidades:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8) +

  # puntos de corte del eje Y
  scale_y_continuous(
    limits = c(130, 200), # límites
    breaks = seq(130, 200, 10)
  ) # nro de etiquetas
```



5.4. Transformaciones estadísticas

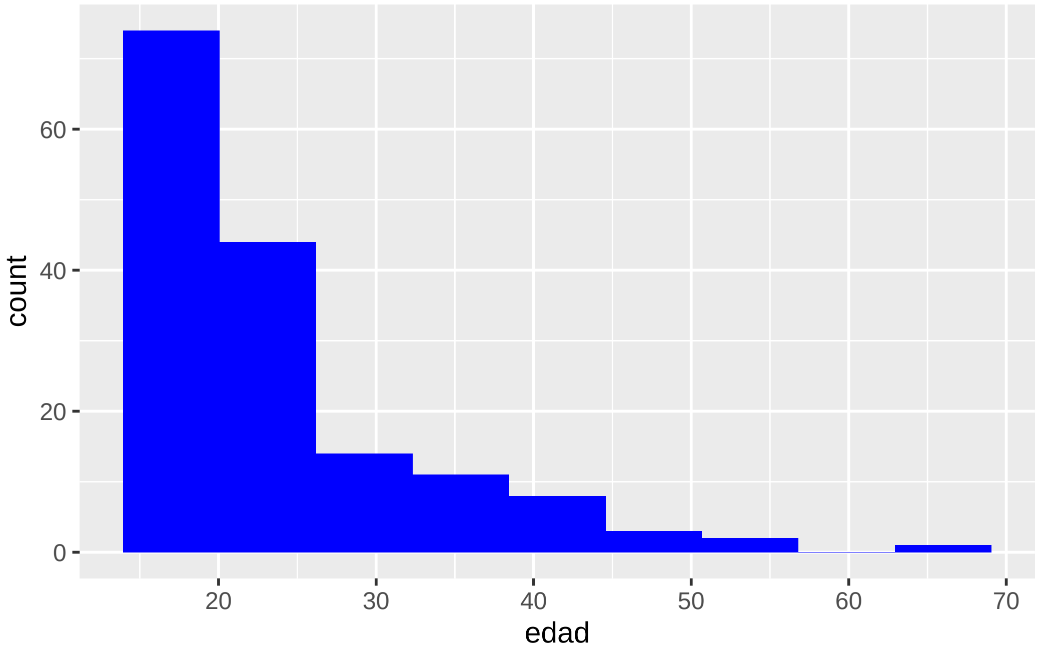
En `ggplot2`, algunos gráficos no requieren transformaciones estadísticas explícitas, como los gráficos de dispersión. Sin embargo, otros —como histogramas, *boxplots* o curvas de tendencia— incorporan **transformaciones estadísticas implícitas** que pueden modificarse o personalizarse.

Estas transformaciones pueden:

- estar integradas dentro de las funciones geométricas (`geom_`)
- agregarse como capas independientes mediante funciones `stat_`

Por ejemplo, en un histograma la transformación estadística consiste en agrupar los datos en intervalos (clases) y contar las observaciones en cada uno. Esta transformación está incluida en `geom_histogram()`, pero puede parametrizarse:

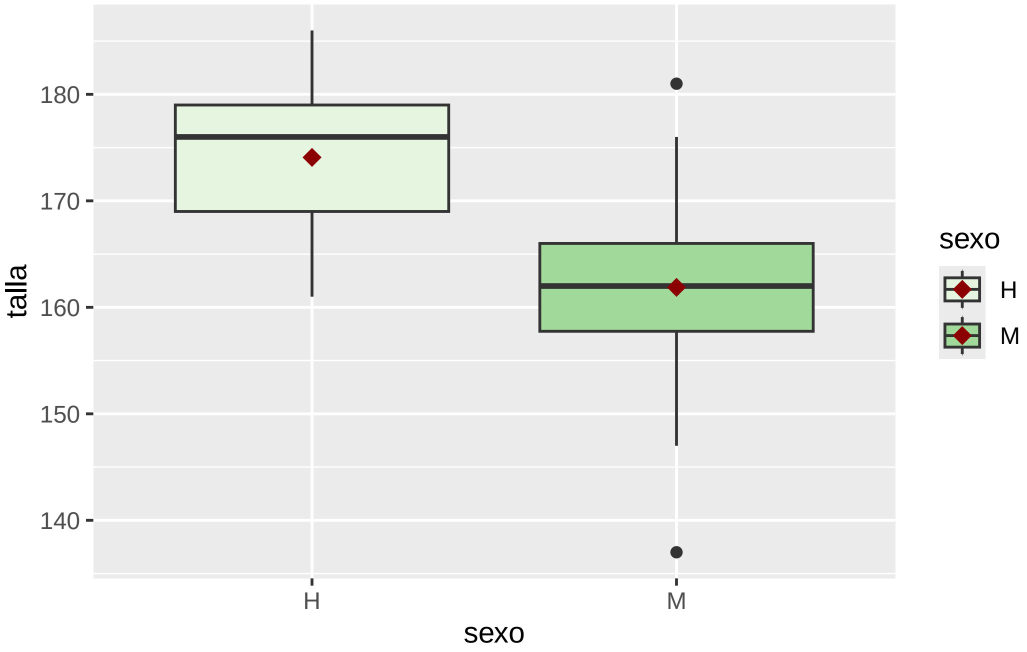
```
facultad |>
  ggplot(aes(edad)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$edad), fill = "Blue")
```



En este caso, se utiliza la regla de Sturges —a través de `nclass.Sturges()`— para determinar automáticamente la cantidad de clases de la variable `edad`.

También es posible agregar transformaciones estadísticas como capas adicionales. Por ejemplo, si queremos mostrar la media de `talla` para cada grupo de `sexo` sobre un *boxplot*, podemos utilizar `stat_summary()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta de tonos verdes
  scale_fill_brewer(palette = "Greens") +

  # añade capa summary
  stat_summary(
    fun = mean,
    color = "darkred",
    geom = "point",
    shape = 18,
    size = 3
  )
```



La función `stat_summary()` permite calcular estadísticas (como `mean`, `median`, `sd`, entre otras) y representarlas mediante una geometría. En este caso, la media de `talla` se muestra como un punto rojo oscuro (`color = "darkred"`), con forma romboidal (`shape = 18`) y tamaño destacado (`size = 3`).

5.5. Facetado

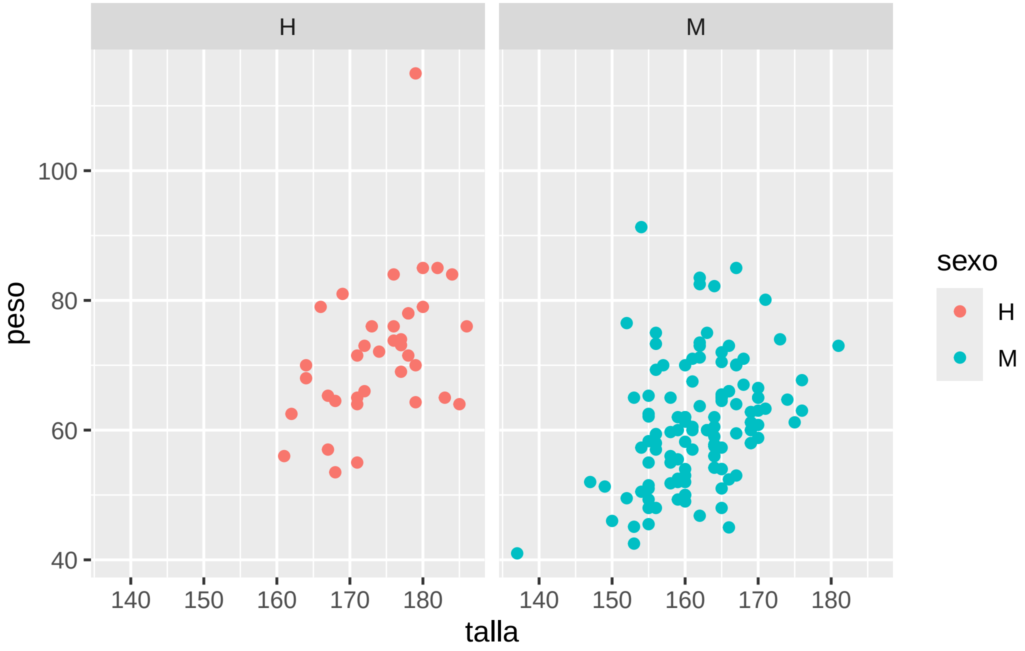
El **facetado** permite dividir un gráfico en múltiples paneles (o viñetas) según los niveles de una o más variables categóricas. Esto resulta especialmente útil para comparar patrones entre grupos sin sobrecargar un único gráfico.

En `ggplot2` existen dos funciones principales para implementar esta técnica:

- `facet_wrap()`: divide los datos según una única variable categórica, generando paneles dispuestos automáticamente en filas y columnas.
- `facet_grid()`: permite crear una matriz de paneles a partir de dos variables categóricas, una definida en las filas y otra en las columnas.

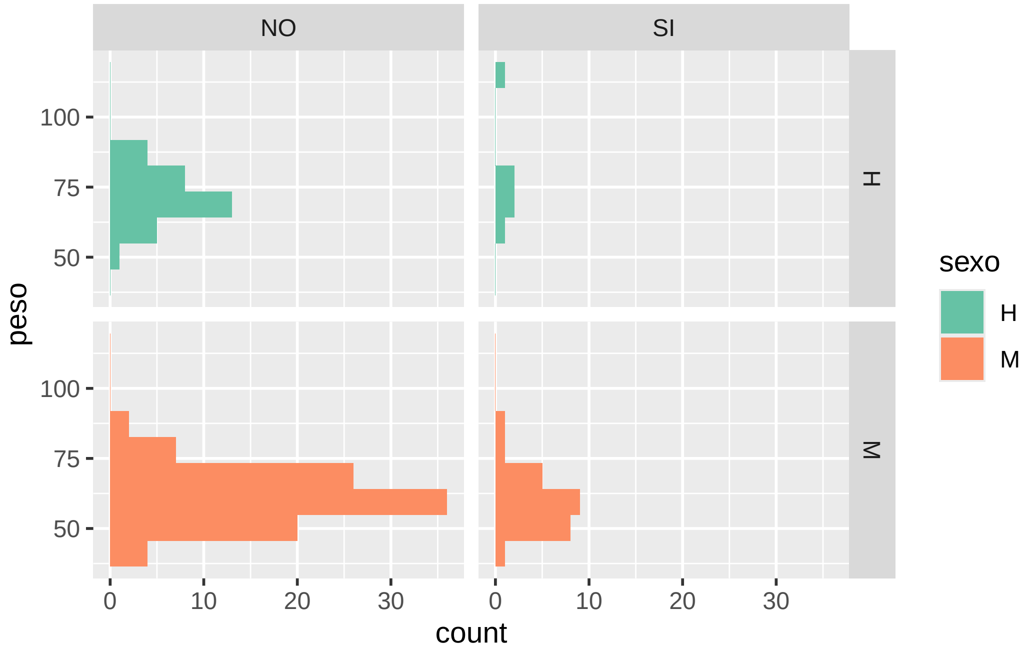
Retomemos el gráfico de dispersión entre `talla` y `peso`, y generemos paneles separados para cada nivel de la variable `sexo` usando `facet_wrap()`:

```
facultad |>

  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # capa geométrica de puntos
  geom_point() +

  # separa en paneles por sexo
  facet_wrap(~sexo)
```



Ahora utilizamos `facet_grid()` para crear una matriz de histogramas a partir del cruce de las variables `sexo` y `fuma`. En cada panel se representa la distribución de `peso`:

```
facultad |>
  ggplot(aes(y = peso, fill = sexo)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$peso)) +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # separa en paneles por sexo y fuma
  facet_grid(sexo ~ fuma)
```



Como se observa en estos ejemplos, el facetado se integra naturalmente con otros componentes de la gramática de gráficos: mapeo estético (`aes()`), capas geométricas (`geom_`) y escalas (`scale_`).

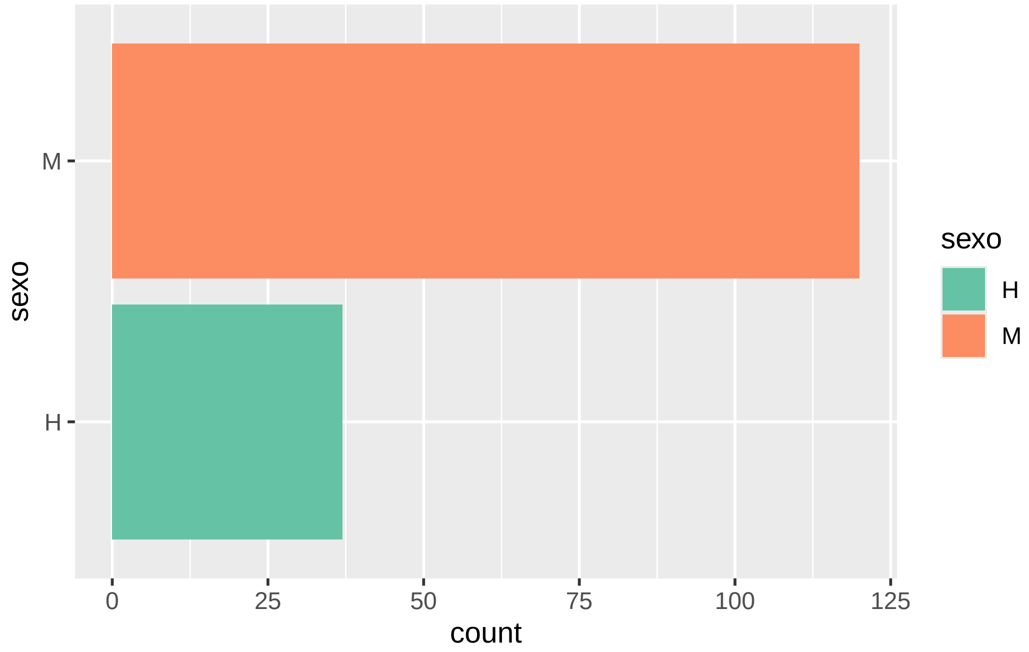
El número de combinaciones posibles es muy amplio, dada la flexibilidad de `ggplot2`. Sin embargo, el objetivo de este material es comprender los principios fundamentales sobre los que se construye esta *gramática de los gráficos*.

5.6. Sistema de coordenadas

En algunos casos, puede ser útil modificar el sistema de coordenadas predeterminado de un gráfico. Mientras que las funciones `scale_` controlan **cómo se representan los datos**, las funciones de coordenadas (`coord_`) modifican **cómo se visualiza el espacio del gráfico**.

En `ggplot2` una de las transformaciones más comunes es invertir la orientación de los ejes mediante `coord_flip()`, lo que resulta útil, por ejemplo, para presentar gráficos de barras en forma horizontal:

```
facultad |>

  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip()
```

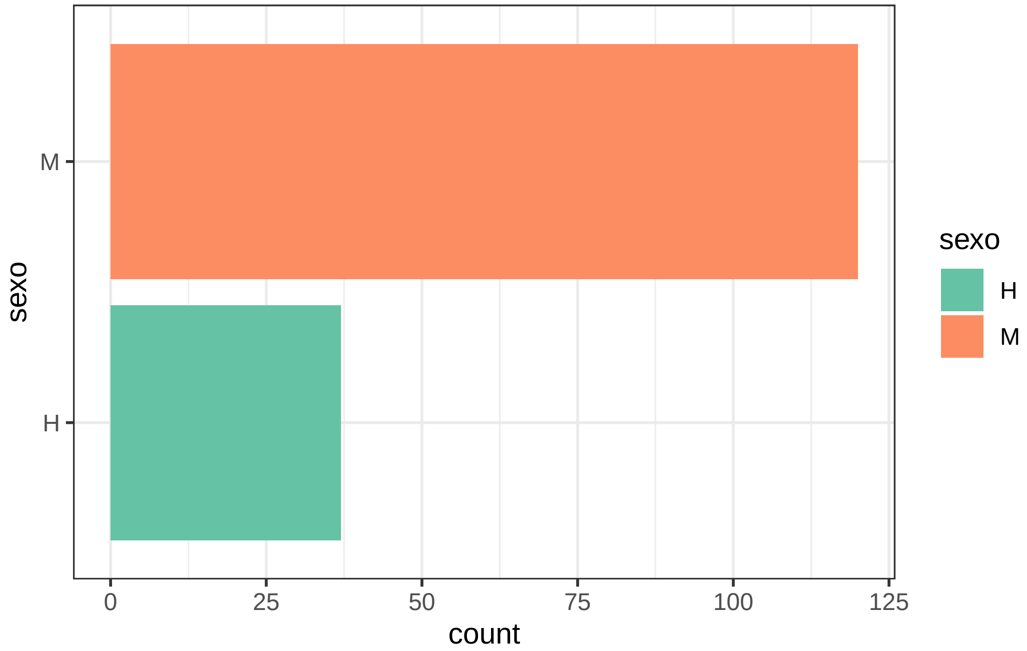


5.7. Temas

`ggplot2` incluye un conjunto de temas gráficos predefinidos que permiten modificar el aspecto general del gráfico. El tema por defecto es `theme_gray()`, pero puede cambiarse agregando una capa `theme_*()` dentro del gráfico.

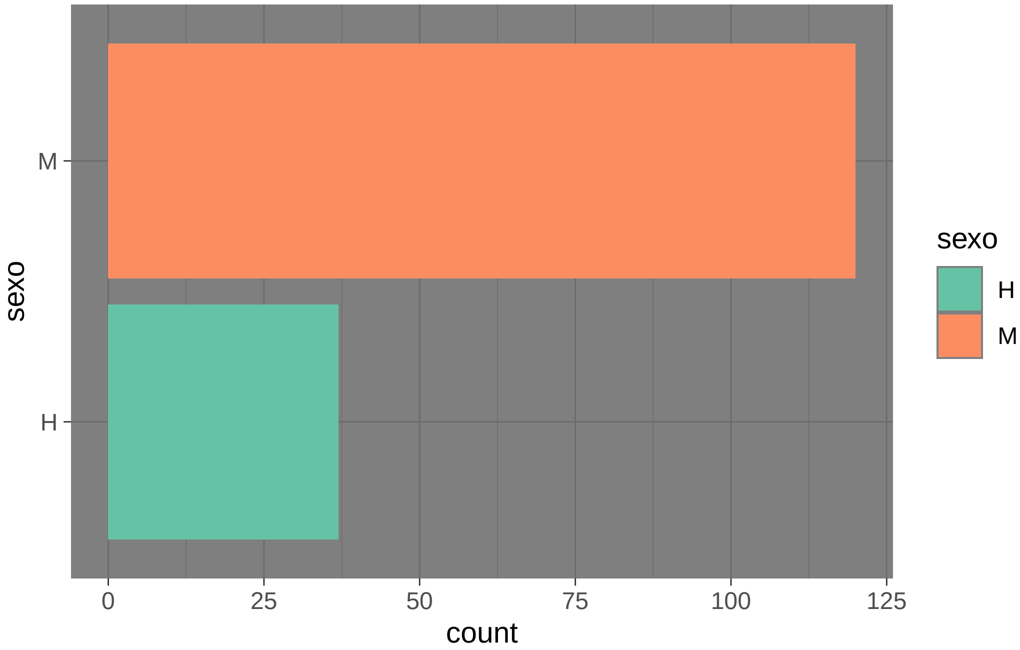
Repetimos el gráfico anterior, esta vez utilizando el tema blanco y negro (`theme_bw()`):

```
facultad |>

  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip() +

  # tema en blanco y negro
  theme_bw()
```

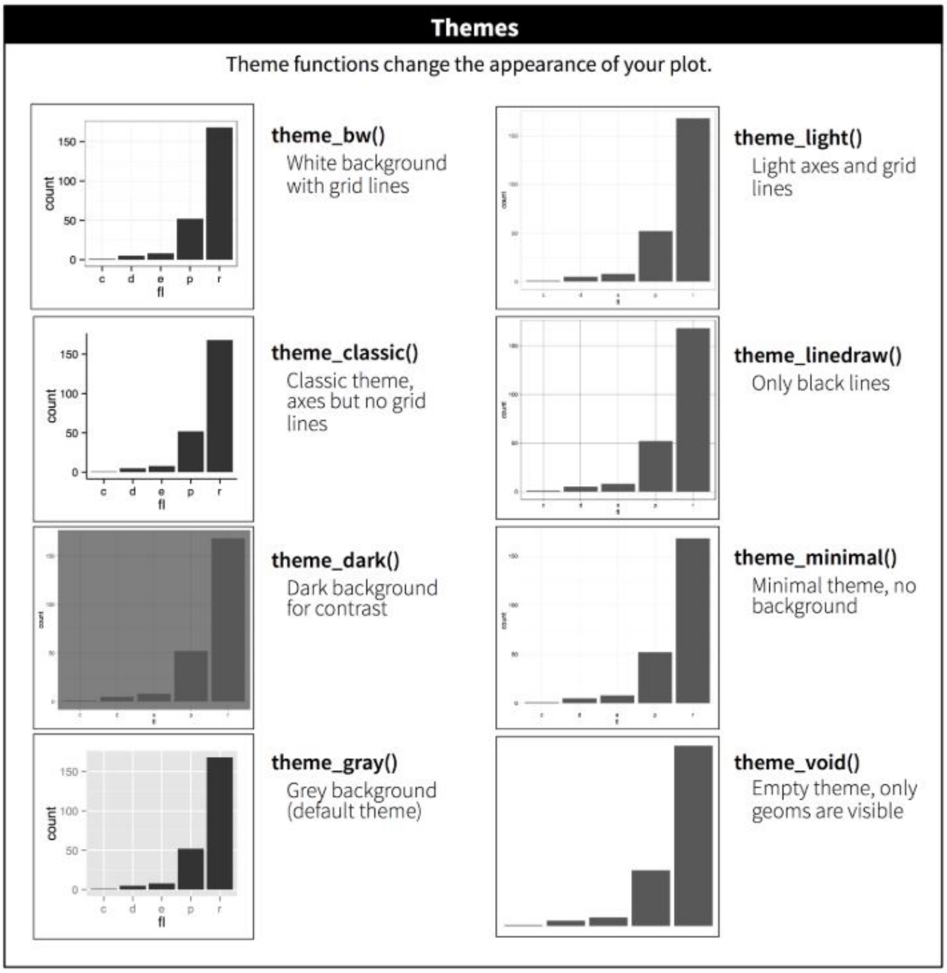


También podemos aplicar un tema de fondo oscuro con `theme_dark()`:

```
facultad |>

  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip() +

  # tema con fondo oscuro
  theme_dark()
```



A continuación se muestra un cuadro con los principales temas disponibles en `ggplot2` y sus características visuales:



Themes

Theme functions change the appearance of your plot.

theme_bw()
White background with grid lines

theme_light()
Light axes and grid lines

theme_classic()
Classic theme, axes but no grid lines

theme_linedraw()
Only black lines

theme_dark()
Dark background for contrast

theme_minimal()
Minimal theme, no background

theme_gray()
Grey background (default theme)

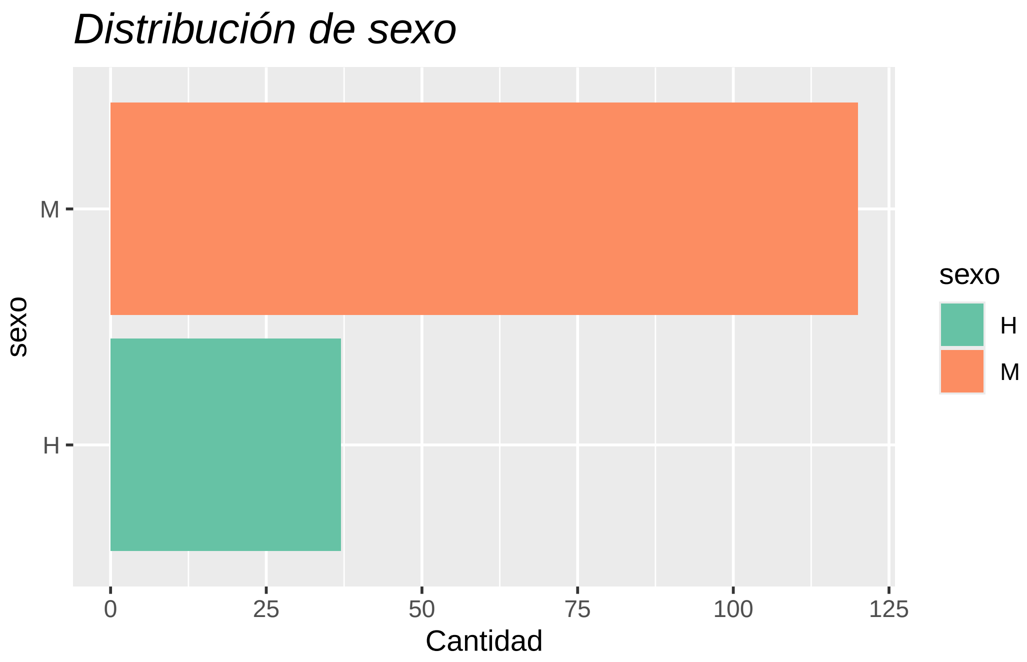
theme_void()
Empty theme, only geoms are visible

Además del aspecto general del gráfico, es posible agregar títulos, subtítulos y etiquetas de ejes con la función `labs()`. Por ejemplo:

```
labs(
  x = "Etiqueta X",
  y = "Etiqueta Y",
  title = "Título del gráfico",
  subtitle = "Subtítulo del gráfico"
)
```

También se pueden ajustar detalles del texto, como tipo de fuente o tamaño, mediante la función `theme()`:

```
facultad |>
  ggplot(aes(sexo, fill = sexo)) +
  # capa geométrica barras
  geom_bar() +
  # paleta de colores
  scale_fill_brewer(palette = "Set2") +
  # invierte disposición de ejes
  coord_flip() +
  # cambia etiquetas de los ejes
  labs(y = "Cantidad", title = "Distribución de sexo") +
  # modifica el estilo de fuente del título
  theme(plot.title = element_text(face = "italic", size = 16))
```



5.8. Paquete `esquisse`

El paquete`esquisse` (Meyer y Perrier 2025) es una extensión de `ggplot2` que permite crear gráficos de manera interactiva, mediante una interfaz gráfica intuitiva basada en el sistema de arrastrar y soltar (*drag & drop*).

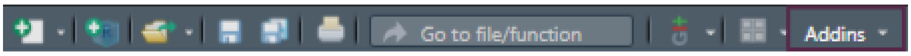
Con `esquisse` es posible:

- Explorar visualmente los datos según su tipo.
- Asignar variables a diferentes estéticas del gráfico (ejes, color, tamaño, etc.).
- Exportar los resultados en distintos formatos.
- Recuperar el código R que genera el gráfico, facilitando su reproducción y modificación.

El paquete se instala mediante el menú **Packages** de RStudio o ejecutando:

```
install.packages("esquisse")
```

Luego se puede acceder a la aplicación por medio del acceso Addins:



o ejecutando en consola:

```
esquisser()
```

También se puede agregar el nombre de la tabla de datos dentro de los paréntesis

```
esquisser(datos)
```

Para más detalles, se puede consultar la viñeta del paquete disponible en CRAN o en su repositorio de GitHub.

5.9. Extensiones de `ggplot2`

Una de las grandes fortalezas de `ggplot2` es su ecosistema de extensiones. Hasta el momento, existen más de 158 **paquetes** desarrollados para ampliar sus funcionalidades, muchos de ellos diseñados para facilitar tareas específicas o agregar nuevas capas, geometrías, temas, escalas o herramientas interactivas.

Algunas de las extensiones que utilizaremos durante el curso son:

- `patchwork` (Pedersen 2025): permite combinar múltiples gráficos de `ggplot2` en una misma figura de forma sencilla y controlada.
- `GGally` (Schloerke et al. 2025): extiende `ggplot2` con funciones para crear matrices de gráficos (como `ggpairs()`), útiles para análisis exploratorio multivariado.
- `ggpubr` (Kassambara 2026a): facilita la creación de gráficos listos para publicar, con funciones simplificadas para agregar títulos, etiquetas, comparaciones estadísticas y anotaciones.
- `see` (Lüdecke et al. 2021): ofrece temas, paletas de colores y escalas compatibles con personas con daltonismo, además de geometrías personalizadas.
- `survminer`(Kassambara et al. 2026): diseñado para la visualización de análisis de supervivencia (como curvas de Kaplan-Meier) utilizando `ggplot2`.
- `ggfortify`(Horikoshi y Tang 2018): permite graficar objetos estadísticos como modelos lineales, PCA o series temporales sin necesidad de escribir código `ggplot2` desde cero.

Estas extensiones permiten ampliar la flexibilidad y expresividad gráfica de `ggplot2` sin perder la estructura gramatical que lo caracteriza.

6. Análisis exploratorio de datos

janitor
Skimr
dlookr
tidyverse

6.1. Introducción

El análisis exploratorio de datos (conocido como **EDA**, su sigla en inglés) es un enfoque de análisis fundamental para resumir y visualizar las características importantes de un conjunto de datos.

John Tukey, estadístico estadounidense, fue uno de los principales impulsores de este enfoque. En 1977 publicó el libro *Exploratory Data Analysis*, donde, entre otras contribuciones, introdujo el gráfico *boxplot* (diagrama de caja y bigotes).

En términos sencillos, antes de avanzar hacia el análisis formal o la construcción de modelos estadísticos, resulta esencial explorar, conocer y describir las variables presentes en nuestra tabla de datos.

Entre los principales objetivos perseguidos por EDA se encuentran:

- Conocer la estructura de la tabla de datos y sus tipos de variable.
- Detectar observaciones incompletas (valores *missing* o `NA`).
- Explorar la distribución de las variables de interés a partir de:
    - Estadísticos descriptivos
    - Representaciones gráficas
- Detectar valores atípicos (*outliers*).

Aclaración

En este documento utilizaremos funciones del lenguaje R basadas en la filosofía *tidyverse*, junto con otros paquetes diseñados para tareas específicas. Esto no implica que no se puedan emplear funciones del R base; sin embargo, el ecosistema `tidyverse` facilita la comprensión y legibilidad del código.

Presentaremos estas diferentes funciones de distintos paquetes que pueden servir en cada etapa de un EDA. Los paquetes con los que trabajaremos son:

- `tidyverse`
- `skimr`
- `dlookr`
- `janitor`

⚠ **Nota:** Algunos paquetes, como `dlookr`, pueden generar falsos positivos en la detección del antivirus durante el proceso de instalación. Sugerimos desactivar momentáneamente el antivirus para evitar inconvenientes.

Una vez instalados, podemos activar los paquetes con el siguiente código:

```
# Carga de paquetes
library(skimr)
library(janitor)
library(dlookr)
library(tidyverse)
```

Se recomienda cargar `tidyverse` al final de la lista para evitar conflictos con funciones que puedan solaparse entre paquetes.

> Es importante destacar que no existe un único camino y/o función para realizar un análisis exploratorio. Esta selección de herramientas puede adaptarse según las preferencias y necesidades de cada usuario. Por lo tanto, quienes ya tengan familiaridad con otras funciones o paquetes pueden continuar utilizándolos sin inconvenientes.

Para ilustrar los pasos del análisis exploratorio, utilizaremos un archivo con datos ficticios llamado "`datos2.txt`", que contiene variables de distintos tipos.

6.2. Conocer la estructura de la tabla de datos y sus tipos de variable

El primer paso en la exploración de un conjunto de datos es conocer su estructura y tamaño:

- El **tamaño** se refiere a la cantidad de observaciones (filas) y de variables (columnas).
- La **estructura** incluye cómo están organizadas las variables, qué tipo de datos contiene cada una y qué categorías o valores pueden tomar.

Comenzaremos por cargar los datos de ejemplo con la función `read_csv2()` de `tidyverse`:

```
datos <- read_csv2("datos/datos2.txt")
```

Una vez cargados los datos, la función `glimpse()` permite obtener una visión general de la tabla:

```
glimpse(datos)
```

```
Rows: 74
Columns: 7
$ id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,~
$ sexo    <chr> "M", "M", "M", "M", "M", "M", "M", "M", NA, "F", "F", "M", "F"~
$ edad    <dbl> 76, 68, 50, 49, 51, 68, 70, 64, 60, 57, 83, 76, 27, 34, 17, 45~
$ peso    <dbl> 71.0, 71.0, 79.0, 71.0, 87.0, 75.0, 80.0, 83.0, 69.0, 73.0, 60~
$ talla   <dbl> 167.0, 164.0, 164.0, 164.0, 167.5, 170.0, 166.0, 160.0, 160.0,~
$ trabaja <lgl> FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, NA, TRUE, TRUE, TRUE, ~
$ fecha   <chr> "20/10/2020", "20/10/2020", "20/10/2020", "05/11/2020", "05/11~
```

Esta función nos informa que la tabla contiene, por ejemplo, 74 observaciones y 7 variables, mostrando el tipo de dato de cada una y los primeros valores que aparecen.

Entre los tipos de datos más comunes que podemos encontrar se incluyen:

- `int` (*integer*): números enteros.
- `dbl` (*double*): números reales.
- `lgl` (*logical*): valores lógicos (`TRUE`, `FALSE`).
- `chr` (*character*): texto o cadenas de caracteres.
- `Date`: fechas.
- `fct` (*factor*): variables categóricas con niveles.
- `dttm` (*date-time*): fechas y horas.

Esta primera revisión de la estructura suele complementarse con el **diccionario de datos**, un recurso fundamental que describe el significado, tipo, unidad y codificación de cada variable. Este diccionario puede acompañar tanto a bases de datos generadas en investigaciones propias (fuentes primarias) como a datos provenientes de fuentes secundarias.

Es importante tener en cuenta que el tipo de dato en R no siempre coincide con la naturaleza estadística de la variable. Por ejemplo:

- Una variable codificada como `dbl` puede representar una medida cuantitativa continua, como la edad o el peso.
- Pero también puede representar una variable cualitativa codificada con números. Por ejemplo, si una variable que registra respuestas “Sí” y “No” fue codificada como `1` y `0`, su tipo de dato será numérico (`dbl` o `int`), aunque conceptualmente sea una variable categórica.

Por esta razón, además de inspeccionar el tipo de datos en R, es importante revisar el significado y el uso previsto de cada variable dentro del contexto del análisis.

6.3. Detectar observaciones incompletas

Los valores perdidos o faltantes (conocidos como *missing* en inglés), representados en R por el valor especial `NA`, constituyen un desafío importante en el análisis de datos. Su presencia puede afectar la calidad del análisis y condicionar las decisiones estadísticas posteriores.

Existen numerosos enfoques para el tratamiento de valores faltantes, incluyendo técnicas de imputación y modelado específico. Sin embargo, en este curso nos enfocaremos exclusivamente en cómo **detectar**, **contabilizar** y, en algunos casos, **excluir** valores faltantes utilizando funciones del lenguaje R.

Una forma sencilla de detectar valores faltantes es mediante la función `count()` del paquete `dplyr`. Al aplicarla a una variable, la salida incluye una fila adicional que informa cuántos valores `NA` hay:

```
datos |>
  count(trabaja)
```

```
# A tibble: 3 x 2
  trabaja     n
  <lgl>   <int>
1 FALSE      26
2 TRUE       39
3 NA          9
```

Una alternativa más completa es la función `find_na()` del paquete `dlookr` (Ryu 2025):

```
find_na(datos, rate = T)
```

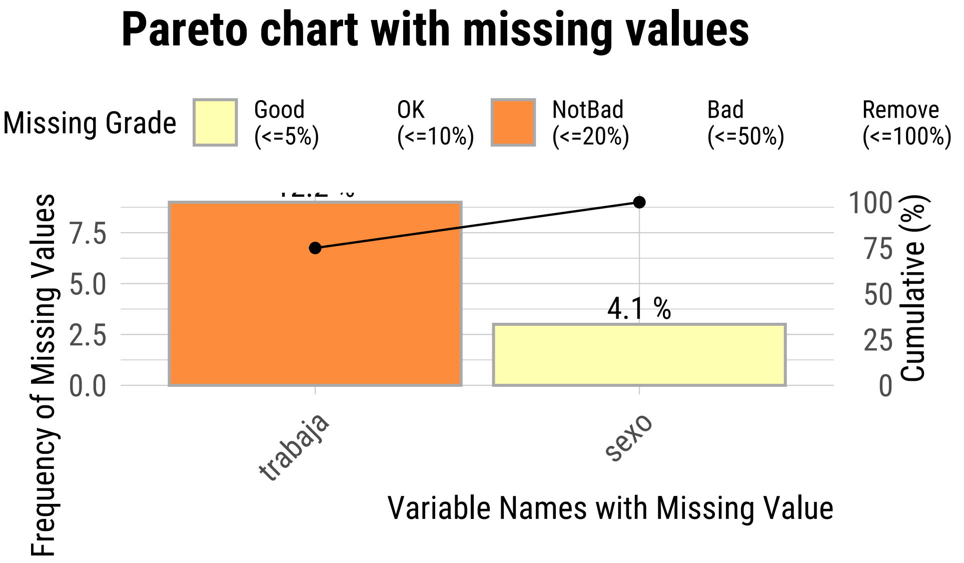
```
     id    sexo    edad    peso   talla trabaja   fecha 
  0.000   4.054   0.000   0.000   0.000  12.162   0.000 
```

Esta función se puede aplicar al conjunto de datos completo y devuelve, para cada variable, la cantidad y el porcentaje de valores `NA`. Por ejemplo, podríamos observar que la variable `sexo` tiene alrededor de un 4 % de valores faltantes, y la variable `trabaja`, algo más del 12 %.

Estos porcentajes pueden ayudarnos a decidir si una variable debe incluirse en un análisis o si es conveniente excluir ciertas observaciones con datos incompletos, siempre que los `NA` sean el resultado de una ausencia real de información.

El mismo paquete trae una función gráfica llamada `plot_na_pareto()`, que genera un gráfico de barras ordenados por frecuencia de valores faltantes:

```
plot_na_pareto(datos, only_na = T)
```



Finalmente, para un diagnóstico más integral de la calidad de las variables, puede utilizarse la función `diagnose()`:

```
diagnose(datos)
```

```
# A tibble: 7 x 6
  variables types     missing_count missing_percent unique_count unique_rate
  <chr>     <chr>             <int>           <dbl>        <int>       <dbl>
1 id        numeric               0            0              74      1
2 sexo      character             3            4.05            3      0.0405
3 edad      numeric               0            0              45      0.608
4 peso      numeric               0            0              56      0.757
5 talla     numeric               0            0              38      0.514
6 trabaja   logical               9           12.2             3      0.0405
7 fecha     character             0            0              11      0.149
```

Esta función ofrece un resumen detallado que incluye el tipo de variable, la cantidad de valores faltantes, la proporción de valores únicos, entre otros indicadores de utilidad para la exploración inicial.

6.4. Conocer la distribución de las variables de interés

6.4.1. Resumir variables cuantitativas

La instalación básica de R tiene incorporadas múltiples funciones estadísticas que permiten calcular medidas resumen para variables cuantitativas. Estas funciones pueden integrarse a la función `summarise()` de `tidyverse`.

Medidas de tendencia central

Las medidas de tendencia central forman parte del grupo de medidas de posición o localización, pero su objetivo principal es resumir la información en torno a un valor que representa el "centro" de la distribución. Es decir, un valor respecto al cual tienden a agruparse los demás valores.

Podemos obtener la media y la mediana de nuestros datos con el siguiente código:

```
datos |>
  summarise(
    # Media
    media = mean(edad),
    # Mediana
    mediana = median(edad)
  )
```

```
# A tibble: 1 x 2
  media mediana
  <dbl>   <dbl>
1  48.1    52.5
```

En cambio, R base no incluye una función específica para calcular la **moda**. Para obtenerla, debemos escribir una función propia o utilizar algún paquete adicional que la implemente (por ejemplo, `modeest::mlv()`).

Medidas de posición

Las medidas de posición dividen los datos en grupos con igual número de observaciones. Entre las más utilizadas se encuentran los **cuartiles** y **percentiles**.

La función `quantile()` del paquete base `stats` permite calcular cuartiles u otros percentiles. Por ejemplo, para calcular los cuartiles Q1 y Q3, indicamos en el argumento `probs` los valores 0.25 y 0.75:

```
datos |>
  summarise(
    # Primer cuartil
    cuartil1 = quantile(edad, probs = 0.25),
    # Tercer cuartil
    cuartil3 = quantile(edad, probs = 0.75)
  )
```

```
# A tibble: 1 x 2
  cuartil1 cuartil3
     <dbl>    <dbl>
1       28       64
```

Para obtener el mínimo y máximo de estos valores numéricos usamos el siguiente código:

```
datos |>
  summarise(
    # Mínimo
    minimo = min(edad),
    # Máximo
    maximo = max(edad)
  )
```

```
# A tibble: 1 x 2
  minimo maximo
   <dbl>  <dbl>
1     13     86
```

Medidas de dispersión

Las medidas de dispersión nos permiten conocer cuán dispersos o variables son los valores dentro del conjunto de datos.

Entre las más clásicas se encuentran la **varianza** y el **desvío estándar**, que se calculan fácilmente con las funciones `var()` y `sd()`:

```
datos |>
  summarise(
    # Varianza
    varianza = var(edad),
    # Desvío estándar
    desvio = sd(edad)
  )
```

```
# A tibble: 1 x 2
  varianza desvio
     <dbl>  <dbl>
1     405.   20.1
```

También puede ser útil calcular el **rango**, que se obtiene como la diferencia entre el valor máximo y el mínimo, y el **rango intercuartílico (RIC)**, mediante `IQR()`:

```
datos |>
  summarise(
    # Rango
    rango = max(edad) - min(edad),
    # Rango intercuartílico
    ric = IQR(edad)
  )
```

```
# A tibble: 1 x 2
  rango   ric
  <dbl> <dbl>
1    73    36
```

El paquete `dlookr` ofrece la función `describe()` para generar un resumen completo de las variables numéricas:

```
describe(datos, -id)
```

```
# A tibble: 3 x 26
  described_variables     n    na  mean    sd se_mean   IQR skewness kurtosis
  <chr>               <int> <int> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 edad                   74     0  48.1 20.1    2.34  36      -0.211  -1.11
2 peso                   74     0  72.5 13.2    1.54  17.7     0.145   0.215
3 talla                  74     0 165.   8.12   0.944  9.75    0.256   0.0363
# i 17 more variables: p00 <dbl>, p01 <dbl>, p05 <dbl>, p10 <dbl>, p20 <dbl>,
#   p25 <dbl>, p30 <dbl>, p40 <dbl>, p50 <dbl>, p60 <dbl>, p70 <dbl>,
#   p75 <dbl>, p80 <dbl>, p90 <dbl>, p95 <dbl>, p99 <dbl>, p100 <dbl>
```

Esta función puede aplicarse directamente sobre todo el conjunto de datos. Si bien selecciona automáticamente las variables numéricas, en este caso estamos excluyendo explícitamente la variable `id`, ya que un identificador no tiene interés estadístico.

El resumen que devuelve incluye:

- `na`: cantidad de observaciones con datos y con `NA`.
- `mean`: media aritmética.
- `sd`: desvío estándar de la media.
- `se_mean`: error estándar de la media.
- `IQR`: rango intercuartílico.
- Medidas de forma como la simetría (`skewness`) y la curtosis (`kurtosis`).
- Percentiles, incluyendo la mediana (`P50`) y los cuartiles (`P25` y `P75`).

6.4.2. Resumir variables cualitativas

Las variables cualitativas o categóricas pueden encontrarse en R bajo los tipos de dato `character` o `factor`. En ocasiones será necesario convertirlas a `factor`, ya que este tipo permite aplicar ciertos procedimientos específicos para variables categóricas.

Frecuencias

Podemos resumir individualmente variables cualitativas mediante las frecuencias absolutas y relativas de sus categorías. La función `count()` de `dplyr` nos muestra el conteo absoluto:

```
datos |>
  count(sexo)
```

```
# A tibble: 3 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
3 <NA>      3
```

En la salida se incluirán, además de las categorías presentes, las observaciones con valores faltantes (`NA`).

La inclusión o no de los valores faltantes dependerá del propósito del análisis. Para excluirlos, podemos utilizar `drop_na()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na()
```

```
# A tibble: 2 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
```

Para obtener frecuencias relativas en porcentaje:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes
  mutate(porc = 100 * n / sum(n))
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

Redondeamos el valor del porcentaje con `round()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes y redondea decimales
  mutate(
    porc = 100 * n / sum(n),
    porc = round(porc, digits = 2)
  )
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

El paquete `janitor` (Firke 2024) ofrece una alternativa más completa mediante la función `tabyl()`:

```
datos |>
  tabyl(sexo)
```

```
 sexo  n    percent valid_percent
    F 27 0.36486486     0.3802817
    M 44 0.59459459     0.6197183
 <NA>  3 0.04054054            NA
```

Esta función muestra tanto frecuencias absolutas como relativas, incluyendo y excluyendo los valores `NA` (porcentaje sobre el total de valores válidos).

Podemos mejorar la presentación combinando otras funciones del paquete:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo  n percent
     F 27  38.03%
     M 44  61.97%
 Total 71 100.00%
```

Tablas de contingencia

La forma más adecuada de describir la relación entre dos variables cualitativas es a través de una **tabla de contingencia**, en la cual:

- Las filas representan las categorías de una variable.
- Las columnas representan las categorías de otra variable.
- Las celdas muestran el número de observaciones correspondientes a cada combinación de categorías.

La función `tabyl()` también permite crear este tipo de tablas. A continuación, un ejemplo entre `sexo` y `trabaja` (aunque `trabaja` sea lógica, puede tratarse como categórica):

```
datos |>
  tabyl(sexo, trabaja)
```

```
 sexo FALSE TRUE NA_
    F     8   15   4
    M    17   22   5
 <NA>     1    2   0
```

Recordemos que el orden dentro de los paréntesis de la función es igual al de los índices, el primer argumento es la variable que aparecerá en las filas y el segundo la variable de las columnas. Por ese motivo, en la tabla de contingencia absoluta tenemos `sexo` en las filas y `trabaja` en las columnas.

Se puede mejorar la tabla excluyendo los valores `NA` y agregando totales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row")
```

```
  sexo FALSE TRUE
     F     8   15
     M    17   22
 Total    25   37
```

Para calcular frecuencias relativas porcentuales por columna usamos el siguiente código:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales
  adorn_totals(where = "row") |>
  # Añadimos porcentajes por columna
  adorn_percentages(denominator = "col") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo   FALSE    TRUE
     F  32.00%  40.54%
     M  68.00%  59.46%
 Total 100.00% 100.00%
```

Calculamos frecuencias relativas porcentuales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes por fila
  adorn_percentages(denominator = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE   Total
    F 34.78% 65.22% 100.00%
    M 43.59% 56.41% 100.00%
```

Cambiando el argumento `denominator` por `"all"` se calculan frecuencias relativas al total:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes al total
  adorn_percentages(denominator = "all") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE  Total
    F 12.90% 24.19% 37.10%
    M 27.42% 35.48% 62.90%
```

6.4.3. Explorar variables mediante gráficos

Uno de los aportes más importantes de John Tukey al análisis de datos es la incorporación de los gráficos como herramienta exploratoria. A través de representaciones visuales podemos detectar rápidamente patrones, anomalías, valores extremos, asimetrías o relaciones entre variables.

En R, los gráficos más útiles para explorar la distribución **univariada** de las variables son:

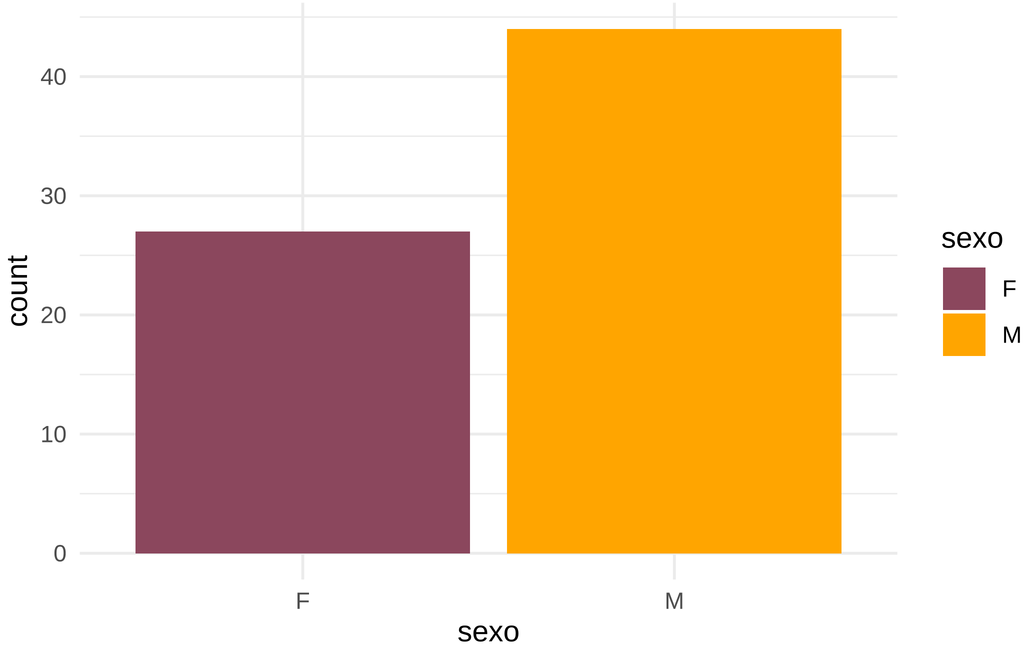
- Para variables **cualitativas**: gráficos de barras
- Para variables **cuantitativas**: histogramas, gráficos de densidad, boxplots y violin plots
- Cuando queremos explorar la relación entre **dos o más variables**, los tipos de gráficos más comunes incluyen:
    - Diagramas de dispersión (puntos)
    - Gráficos de líneas
    - Gráficos de mosaico para variables categóricas cruzadas

El lenguaje R soporta una serie de sistemas gráficos asociados a paquetes como `graphics`, `lattice`, `ggplot2`, etc. que sirven de base incluso para otros paquetes con funciones más específicas. Actualmente el estándar gráfico en R es `ggplot2`.

Barras (univariado)

El gráfico de barras permite visualizar la frecuencia de las categorías de una variable cualitativa:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Generamos histograma
  ggplot(aes(x = sexo, fill = sexo)) +
  geom_bar() +
  scale_fill_manual(values = c("palevioletred4", "orange")) +
  theme_minimal()
```

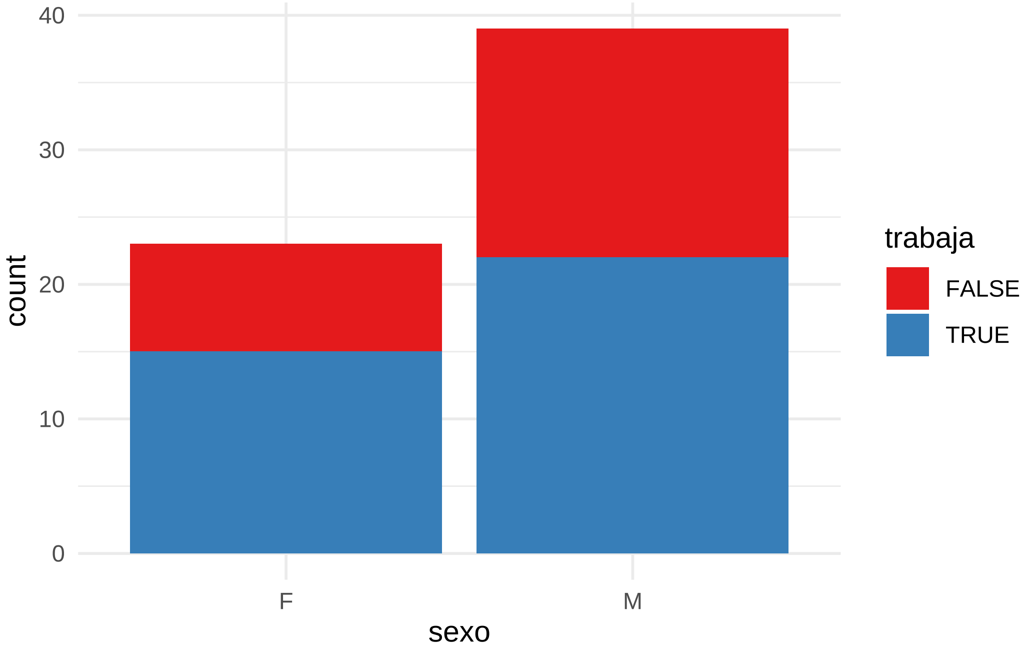


Barras (bivariado)

Cuando cruzamos dos variables categóricas, podemos representar la relación entre ambas modificando el argumento `position` de `geom_bar()`.

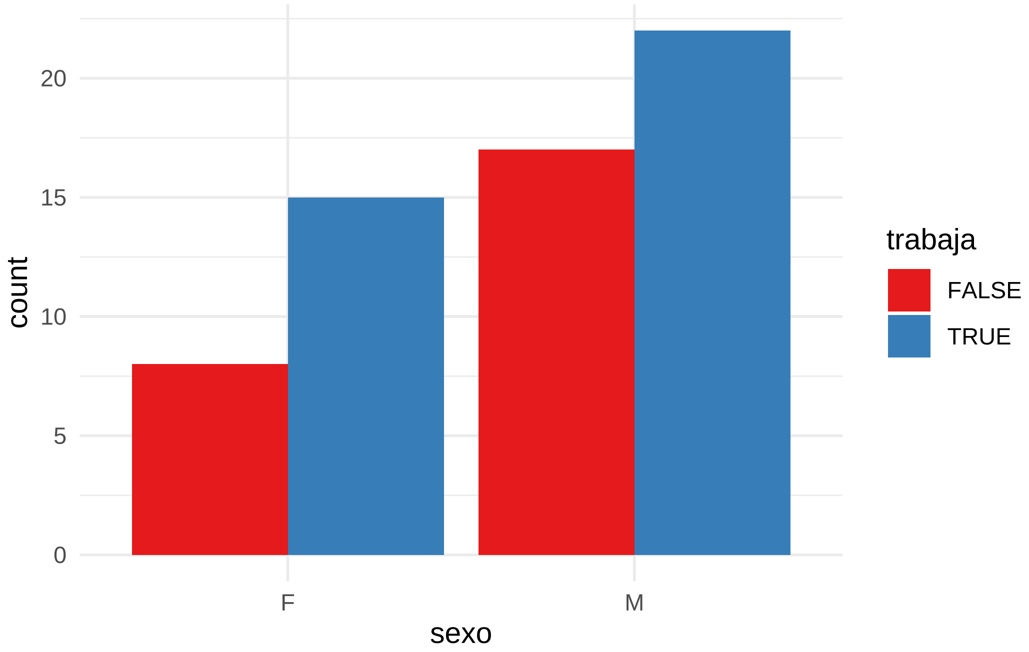
El argumento `position = "stack"` nos muestra los valores absolutos acumulados:

```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "stack") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



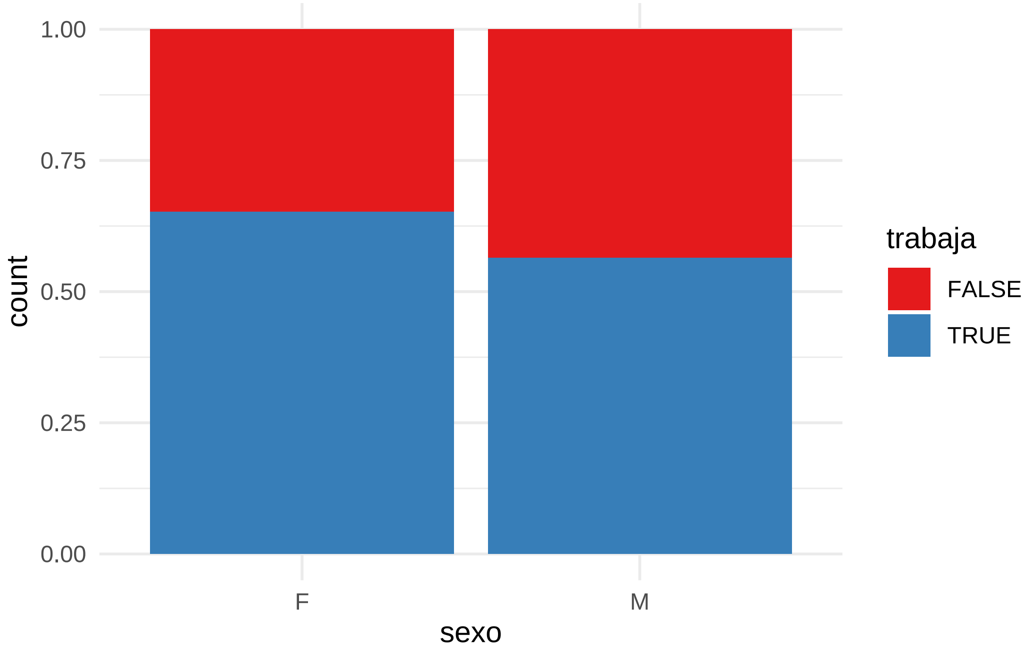
Por otro lado, el argumento `position = "dodge"` muestra las barras lado a lado, permitiendo comparar proporciones entre grupos:

```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "dodge") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Finalmente, `position = "fill"` convierte las alturas en proporciones sobre el total por grupo:

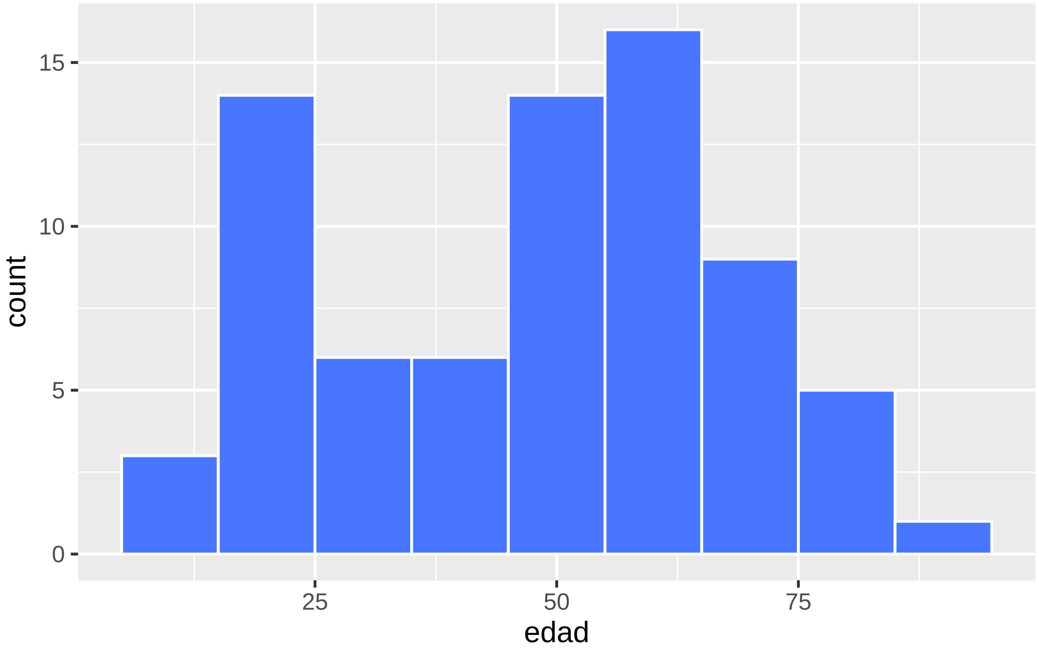
```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "fill") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Histograma

Representa la frecuencia de valores en intervalos definidos. Útil para observar la forma general de la distribución:

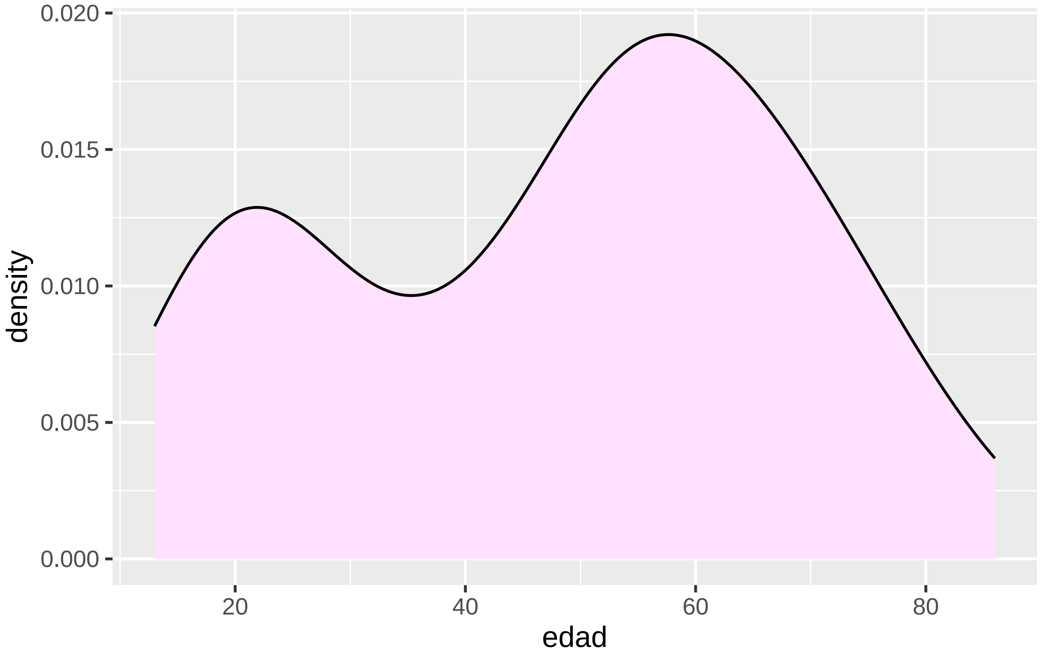
```r
datos |>
  # Genera histograma
  ggplot(aes(x = edad)) +
  geom_histogram(binwidth = 10, fill = "royalblue1", color = "white")
```



Densidad

Es una estimación suave de la distribución de frecuencias:

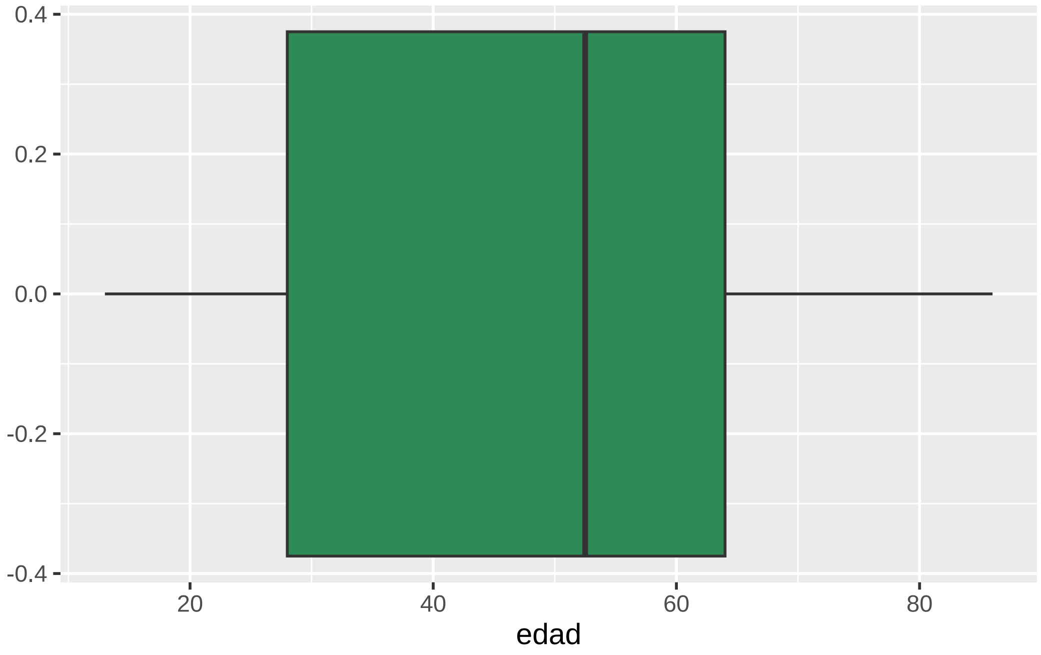
```r
datos |>
  ggplot(aes(x = edad)) +
  geom_density(fill = "thistle1")
```



Boxplot

Muestra el rango intercuartílico, la mediana y los valores atípicos. Ideal para detectar asimetrías y *outliers*:

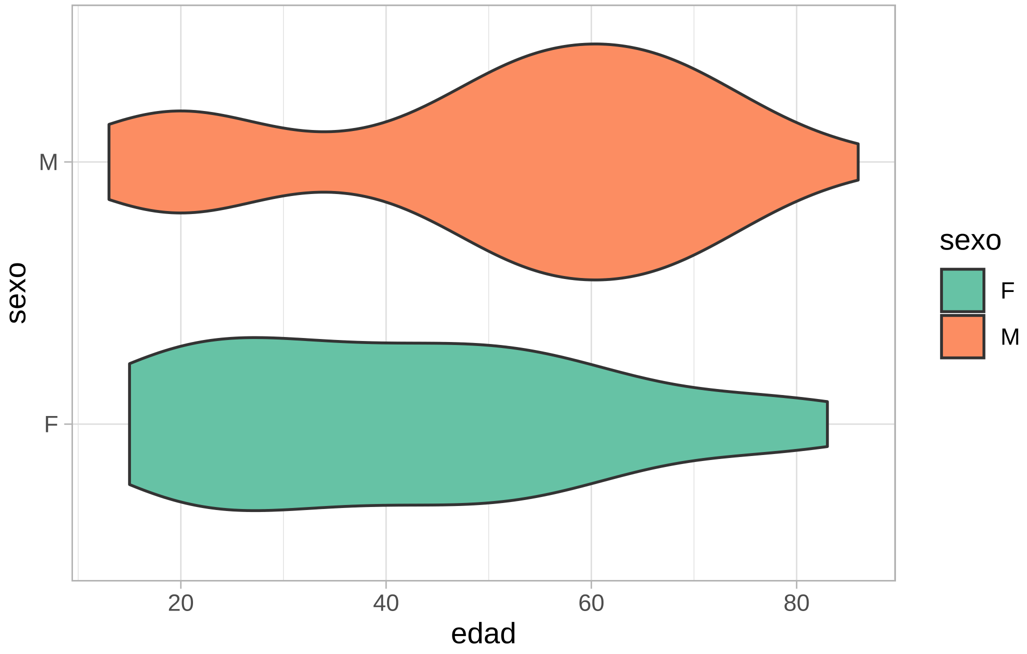
```
datos |>
  # Genera boxplot
  ggplot(aes(x = edad)) +
  geom_boxplot(fill = "seagreen4")
```



Violinplot

Combina el *boxplot* con una curva de densidad reflejada. Permite visualizar tanto la forma de la distribución como los cuantiles:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Genera violinplot
  ggplot(aes(x = edad, y = sexo, fill = sexo)) +
  geom_violin() +
  scale_fill_brewer(palette = "Set2") +
  theme_light()
```

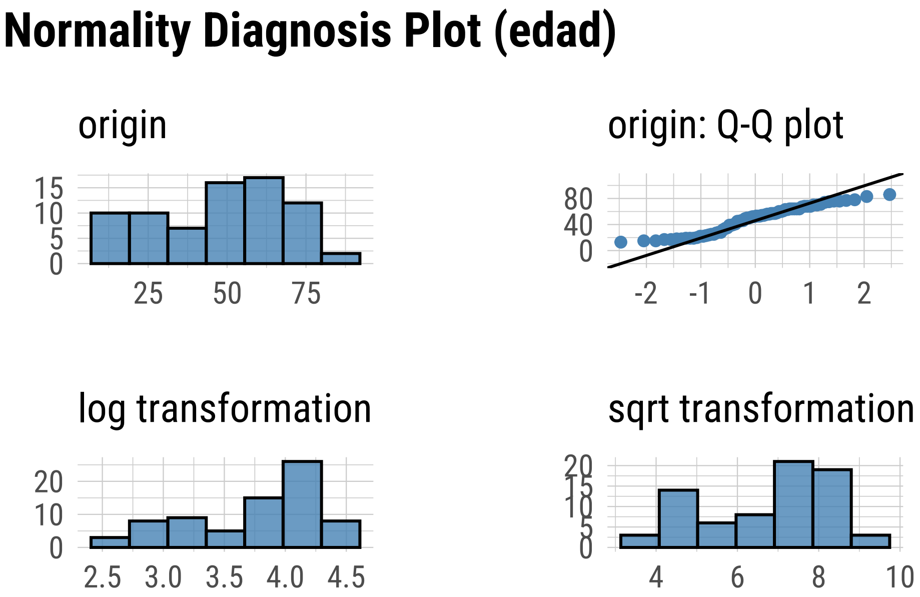


Q-Q Plot

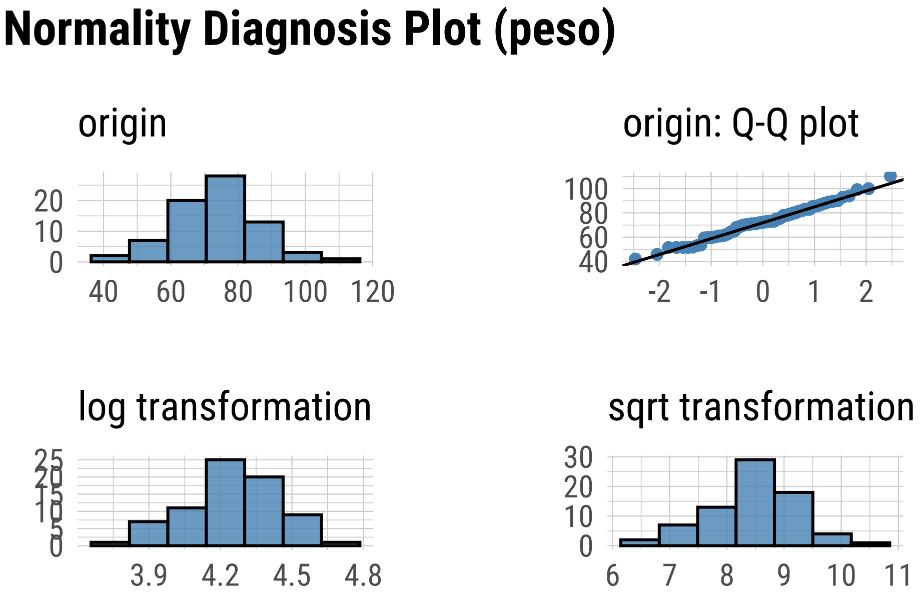
Los gráficos Q-Q (cuantil-cuantil) permiten evaluar visualmente si una variable sigue una distribución teórica, como la normal. Suelen usarse como método gráfico para analizar "normalidad", es decir cuanto se asemeja la distribución de la variable a la distribución normal o gaussiana.

La función `plot_normality()` de `dlookr` muestra un diagnóstico gráfico de normalidad de una variable usando histogramas y Q-Q plot. Además muestra otros histogramas con conversiones de datos (logarítmico y raíz cuadrada por defecto, pero también "Box-Cox" y otras):

```
# Sobre la variable edad
datos |>
  plot_normality(edad)
```



```r
# Sobre la variable peso
datos |>
  plot_normality(peso)
```



Podemos decir que la variable `peso` se ajusta mejor a una distribución normal, ya que los puntos del Q-Q plot se alinean más cercanamente a la diagonal teórica.

> **Nota**: Este análisis gráfico de normalidad suele complementarse con pruebas estadísticas específicas, que abordaremos en la próxima unidad.

6.5. Detección de valores atípicos

Un valor atípico (*outlier*) es una observación que se encuentra numéricamente alejada del resto de los datos. Su presencia puede tener diferentes causas, y su tratamiento dependerá del contexto:

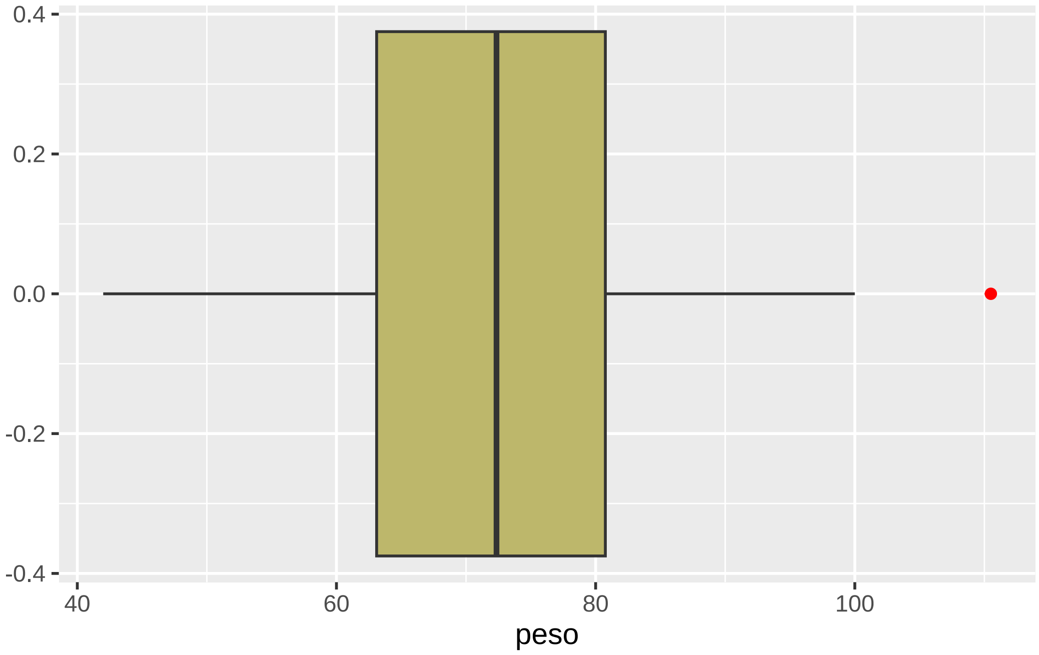
- **Errores de carga o procedimiento**: deben corregirse si se detectan.
- **Valores extremos plausibles**: pueden ser válidos, pero conviene evaluarlos en detalle.
- **Eventos extraordinarios o causas desconocidas**: si no se pueden justificar, suelen excluirse del análisis.

Estos valores pueden afectar sensiblemente ciertos estadísticos como la media, distorsionando su interpretación.

Una forma gráfica común de detectar valores atípicos es mediante los *boxplots*. Los puntos situados fuera de los "bigotes" representan posibles *outliers*.

A continuación, se presenta un ejemplo con la variable `peso`, donde se observa un valor extremo en el límite superior de la distribución (punto rojo):

```
datos |>
  ggplot(aes(x = peso)) +
  geom_boxplot(fill = "darkkhaki", outlier.color = "red")
```



Este valor coincide con el máximo observado:

```
max(datos$peso)
```

```
[1] 110.5
```

El paquete `dlookr` incluye la función `diagnose_outlier()` para la detección automatizada de valores atípicos en todas las variables numéricas de un conjunto de datos:

```
diagnose_outlier(datos)
```

```
# A tibble: 4 x 6
  variables outliers_cnt outliers_ratio outliers_mean with_mean without_mean
  <chr>            <int>          <dbl>         <dbl>     <dbl>        <dbl>
1 id                   0           0              NaN      37.5         37.5
2 edad                 0           0              NaN      48.1         48.1
3 peso                 1           1.35          110.      72.5         72.0
4 talla                0           0              NaN     165.         165.
```

Esta función devuelve una tabla que incluye, para cada variable: cantidad y proporción de *outliers* detectados, media de la variable incluyendo los *outliers*, media de la variable excluyendo los *outliers*. En función de estos dos estadísticos se puede comparar el efecto de los valores atípicos en la media.

El paquete `skimr` (Waring et al. 2026) permite obtener un resumen estadístico compacto y amigable de un conjunto de datos mediante la función `skim()`:

```
skim(datos)
```

Tabla 6.1.: Data summary

Name	datos
Number of rows	74
Number of columns	7
Column type frequency:	
character	2
logical	1
numeric	4
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
sexo	3	0.96	1	1	0	2	0
fecha	0	1.00	10	10	0	11	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
trabaja	9	0.88	0.6	TRU: 39, FAL: 26

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
id	0	1	37.50	21.51	1	19.25	37.50	55.75	74.0	⍰⍰⍰⍰⍰
edad	0	1	48.07	20.12	13	28.00	52.50	64.00	86.0	⍰⍰⍰⍰⍰
peso	0	1	72.49	13.23	42	63.10	72.35	80.75	110.5	⍰⍰⍰⍰⍰
talla	0	1	164.85	8.12	148	160.00	164.00	169.75	185.0	⍰⍰⍰⍰⍰

Además, puede integrarse fácilmente con la gramática `tidyverse`. Por ejemplo, podemos explorar estadísticas descriptivas de variables numéricas agrupadas por `sexo`:

```r
datos |>
  # Excluye NAs de sexo
  drop_na(sexo) |>
  # Agrupa por sexo
  group_by(sexo) |>
  # Solo variables numéricas - id
  select(where(is.numeric), -id) |>
  # Explora outliers
  skim()
```

Tabla 6.2.: Data summary

Name	select(...)
Number of rows	71
Number of columns	4
Column type frequency:	
numeric	3
Group variables	sexo

Variable type: numeric

skim_variable	sexo	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
edad	F	0	1	41.89	19.64	15.0	26.00	39.0	54.50	83.0	⍰⍰⍰⍰⍰
edad	M	0	1	51.59	19.56	13.0	40.75	55.5	64.75	86.0	⍰⍰⍰⍰⍰
peso	F	0	1	63.88	12.02	42.0	53.30	61.0	72.00	85.6	⍰⍰⍰⍰⍰
peso	M	0	1	77.97	11.49	51.8	71.00	77.2	83.75	110.5	⍰⍰⍰⍰⍰
talla	F	0	1	158.57	6.08	148.0	155.00	159.5	162.25	172.0	⍰⍰⍰⍰⍰
talla	M	0	1	168.85	6.58	158.0	164.00	167.0	173.00	185.0	⍰⍰⍰⍰⍰

En este ejemplo, mostramos resultados de variables numéricas menos de `id` agrupados por `sexo` (sin considerar valores `NA` en las categorías de sexo).

7. Hoja de Estilo del lenguaje R

7.1. Convenciones de estilo R

R es bastante indulgente con la forma en que escribimos código, a diferencia de otros lenguajes como Python, donde un espacio mal puesto puede arruinar el script. Sin embargo, adoptar una guía de estilo mejora la legibilidad, facilita la colaboración y reduce errores.

Las siguientes líneas de código producen el mismo resultado, pero no todas son igual de claras:

```
# Opción 1
mpg |>
  filter(cty > 10, class == "compact")

# Opción 2
mpg |> filter(cty > 10, class == "compact")

# Opción 3
mpg |>
  filter(cty > 10,
         class == "compact")

# Opción 4
mpg |> filter(cty>10, class=="compact")

# Opción 5
filter(mpg,cty>10,class=="compact")

# Opción 6
mpg |>
filter(cty > 10,
                        class == "compact")

# Opción 7
filter ( mpg,cty>10,     class=="compact" )
```

Las tres primeras versiones son más legibles. El resto, aunque válidas, son difíciles de seguir, especialmente en trabajos colaborativos o materiales docentes.

7.2. Guía de estilo de `tidyverse`

Para ayudar a mejorar la legibilidad y facilitar el compartir código con otros, el equipo de Tidyverse publicó una guía concisa con ejemplos claros de buenas y malas formas de escribir código, nombres de variables, sangría, líneas largas, y más:

▯ **https://style.tidyverse.org/**

Además, RStudio incluye herramientas para aplicar estas convenciones automáticamente. Por ejemplo, seleccionando el código y presionando `Ctrl + i` (Windows) se puede reidentar el texto. No siempre es perfecto, pero es realmente útil para lograr la sangría correcta sin tener que presionar manualmente espacio muchas veces.

7.2.1. Espaciado

Colocar espacios después de las comas:

▯ **Correcto**

```
filter(mpg, cty > 10)
```

▯ **Incorrecto**

```
filter(mpg , cty > 10)

filter(mpg ,cty > 10)

filter(mpg,cty > 10)
```

Colocar espacios después de comas y alrededor de operadores (+, −, >, =, etc.) mejora la legibilidad. También se deben evitar espacios innecesarios dentro de paréntesis:

▯ **Correcto**

```
filter(mpg, cty > 10)
```

▯ **Incorrecto**

```
filter(mpg, cty>10)

filter(mpg, cty> 10)

filter(mpg, cty >10)
```

No colocar espacios alrededor de paréntesis que sean parte de funciones:

▯ **Correcto**

```
filter(mpg, cty > 10)
```

▯ **Incorrecto**

```
filter (mpg, cty > 10)

filter ( mpg, cty > 10)

filter( mpg, cty > 10 )
```

7.2.2. Líneas largas

En general, es una buena práctica no tener líneas de código muy largas. Se recomienda limitar las líneas a **80 caracteres**. Para visualizarlo, en RStudio vamos a `Tools > Global Options > Code > Display`y seleccionamos la casilla **Show margin.**

Se sugiere agregar saltos de línea dentro de las líneas de código más largas, los mismos deben colocarse luego de las comas y los argumentos se deben alinear dentro de la función:

⍰ Correcto

```
filter(mpg, cty > 10, class == "compact")


filter(mpg, cty > 10,
       class == "compact")


filter(mpg,
       cty > 10,
       class == "compact")

filter(mpg,
       cty > 10,
       class %in% c("compact", "pickup", "midsize", "subcompact",
                    "suv", "2seater", "minivan"))
```

⍰ Incorrecto

```
filter(mpg, cty > 10, class %in% c("compact", "pickup", "midsize", "subcompact", "suv", "2sea
```

7.2.3. Tuberías y capas `ggplot2`

Colocar cada paso de la tubería (%>% ó |>) en una línea separada, con sangría de dos espacios debajo del operador:

⍰ Correcto

```
mpg |>
  filter(cty > 10) |>
  group_by(class) |>
  summarize(avg_hwy = mean(hwy))
```

⍰ Incorrecto

```
# Mal
mpg |> filter(cty > 10) |> group_by(class) |>
  summarize(avg_hwy = mean(hwy))

# Muy mal
mpg |> filter(cty > 10) |> group_by(class) |> summarize(avg_hwy = mean(hwy))

# Tan mal que no funciona
mpg |>
  filter(cty > 10)
  |> group_by(class)
  |> summarize(avg_hwy = mean(hwy))
```

Lo mismo aplica para las capas de gráficos de `ggplot2`, usando el conector + al final de la línea y debajo sangría de dos espacios:

⍰ Correcto

```
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()
```

⍰ Incorrecto

```
# Mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() + geom_smooth() +
  theme_bw()

# Muy mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) + geom_point() + geom_smooth() + theme_bw

# Tan mal que no funciona
ggplot(mpg, aes(x = cty, y = hwy, color = class))
  + geom_point()
  + geom_smooth()
  + theme_bw()
```

7.2.4. Comentarios

Los comentarios deben comenzar con # seguido de un espacio:

⍰ Correcto

```
# Bien
```

⍰ Incorrecto

```
#Mal

      #Mal
```

Si el comentario es corto, se puede incluir en la misma línea, separado por al menos dos espacios para mejorar la legibilidad:

```
mpg |>
  filter(cty > 10) |>  # filtro filas donde cty es 10 o más
  group_by(class) |>  # estratifica por class
  summarize(avg_hwy = mean(hwy))  # resume la media de hwy por cada grupo
```

Se puede agregar espacios adicionales para alinear los comentarios en línea, si lo deseamos:

```
mpg |>
  filter(cty > 10) |>             # filtro filas donde cty es 10 o más
  group_by(class) |>              # estratifica por class
  summarize(avg_hwy = mean(hwy))  # resume la media de hwy por cada grupo
```

Si el comentario es muy largo, podemos dividirlo en varias líneas. RStudio incluye una herramienta útil para comentarios largos: `Code > Reflow Comment` los ajusta automáticamente al ancho deseado.

7.2.5. Nombres de columnas

Se recomienda utilizar nombres de columnas en un formato consistente y compatible con el enfoque del ecosistema `tidyverse`. Esto facilita la manipulación de datos y reduce errores en el código.

En particular, se sugiere evitar:

- Nombres excesivamente largos
- Uso de mayúsculas
- Signos de puntuación, acentos y caracteres especiales
- Espacios en blanco

En su lugar, es recomendable utilizar:

- Nombres breves pero descriptivos
- Letras minúsculas

palabras separadas por guiones bajos (_) o puntos (`.`)

⍰ Correcto

```
names(mpg)
```

```
 [1] "manufacturer" "model"        "displ"        "year"         "cyl"
 [6] "trans"        "drv"          "cty"          "hwy"          "fl"
[11] "class"
```

⍰ Incorrecto

En bases de datos existentes:

```r
names(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

Al crear un nuevo dataset:

```r
datos <- tibble(
  Jurisdicción = c(rep("Buenos Aires", 10), rep("CABA", 10)),
  CasosTotales = sample(100, 20),
  POBLACIÓN = c(rep(16626374, 10), rep(3054237, 10)),
)

names(datos)
```

```
[1] "Jurisdicción" "CasosTotales" "POBLACIÓN"
```

Al crear una nueva variable o una tabla resumen:

```r
# Tan mal que da error
datos  |>
summarise(tasa-incidencia(100k) = CasosTotales/POBLACIÓN * 100000)
```

```
Error in parse(text = input): <text>:3:30: unexpected symbol
2: datos  |>
3: summarise(tasa-incidencia(100k
                                ^
```

En caso de que nuestros datos tengan nombres de columnas no compatibles con `tidyverse`, se recomienda usar la función `clean_names()` del paquete `janitor` para estandarizarlas:

```r
library(janitor)

iris |>
  clean_names() |>
  head(10)
```

```
   sepal_length sepal_width petal_length petal_width species
1           5.1         3.5          1.4         0.2  setosa
2           4.9         3.0          1.4         0.2  setosa
3           4.7         3.2          1.3         0.2  setosa
4           4.6         3.1          1.5         0.2  setosa
5           5.0         3.6          1.4         0.2  setosa
6           5.4         3.9          1.7         0.4  setosa
7           4.6         3.4          1.4         0.3  setosa
8           5.0         3.4          1.5         0.2  setosa
9           4.4         2.9          1.4         0.2  setosa
10          4.9         3.1          1.5         0.1  setosa
```

```
datos <- datos |>
  clean_names()

names(datos)
```

```
[1] "jurisdiccion"  "casos_totales" "poblacion"
```

⚠ **Importante**

Las versiones recientes de RStudio y Positron (el nuevo entorno de desarrollo de Posit) incorporan soporte para **Air**, un formateador de código de alto rendimiento desarrollado en Rust. Esta herramienta aplica de manera automática los estándares de estilo recomendados por el ecosistema de `tidyverse`, lo que optimiza la legibilidad y simplifica el mantenimiento de los scripts.
Para habilitar esta función en RStudio hay que entrar a:
`Tools > Global Options > Code > Formatter`
Y habilitar las opciones correspondientes.

Parte II.

Unidad 2

8. Estudios epidemiológicos

Un estudio epidemiológico es un proceso que se desarrolla en varias fases: desde la definición del **problema de investigación**, la selección de la estrategia o **diseño del estudio**, hasta la planificación de las **actividades**. En esencia, el diseño constituye la estrategia metodológica elegida para alcanzar los objetivos planteados.

8.1. Clasificación de estudios epidemiológicos

Existen distintos criterios para clasificar los estudios epidemiológicos. La siguiente tabla (Tabla 32.1) resume esta clasificación, que probablemente ya hayan abordado en cursos previos:

Tabla 8.1.: Clasificación de estudios epidemiológicos.

Poblacionales	Individuales	Observacionales	Experimentales
Análisis de situación	Serie de casos	Estudios de casos y controles	Ensayos clínicos
Estudios Ecológicos	Reporte de casos	Estudios de cohortes	Ensayos comunitarios y/o de campo
	Estudios transversales		

Toda investigación debe partir de una revisión actualizada del conocimiento sobre el tema a abordar. Esto implica delimitar el problema de investigación de forma precisa, definiendo objetivos, propósito, justificación e hipótesis. Para ello, es fundamental conocer las características y ventajas de cada tipo de estudio epidemiológico, evaluar su adecuación a los objetivos planteados y considerar los recursos disponibles en términos de tiempo, personal y presupuesto.

8.1.1. Clasificación GRADE

Existen propuestas que jerarquizan la calidad de la evidencia según el tipo de diseño, bajo el supuesto de que algunos están más expuestos a sesgos que otros. Esta jerarquización permite orientar la toma de decisiones clínicas y de salud pública.

El análisis riguroso de la evidencia facilita establecer grados de recomendación para intervenciones diagnósticas, terapéuticas o preventivas. Uno de los sistemas más utilizados para clasificar la calidad de la evidencia y la fuerza de las recomendaciones es **GRADE** (*Grading of Recommendations Assessment, Development and Evaluation*), que permite sintetizar la calidad de la evidencia según el tipo de diseño:

- **Ensayos clínicos aleatorizados (EC):** calidad alta
- **Estudios observacionales:** calidad baja
- **Otras fuentes:** calidad muy baja

La siguiente tabla (Tabla 32.2) muestra la clasificación de GRADE, que combina calidad metodológica, balance entre beneficios y riesgos, y las implicancias de cada recomendación:

Tabla 8.2.: Clasificación GRADE de estudios epidemiológicos.

Grado	Recomendación	Calidad	Beneficio vs. Riesgo y cargas	Calidad metodológica que apoya la evidencia	Implicancias
1.A	Fuerte	Alta	Superan ampliamente los riesgos y cargas (o viceversa)	EC sin importantes limitaciones o evidencia abrumadora de estudios observacionales.	Puede aplicarse a la mayoría de los pacientes, en la mayoría de las circunstancias sin reservas.
1.B	Fuerte	Moderada	Superan ampliamente los riesgos y cargas (o viceversa)	EC con importantes limitaciones (resultados inconsistentes, defectos metodológicos indirectos o imprecisos) o pruebas excepcionalmente fuertes a partir de estudios observacionales.	Puede aplicarse a la mayoría de los pacientes, en la mayoría de las circunstancias sin reservas.
1.C	Fuerte	Baja o muy baja	Superan ampliamente los riesgos y cargas (o viceversa)	Estudios observacionales o series de casos.	Puede cambiar cuando se disponga de mayor evidencia de calidad.
2.A	Débil	Alta	Estrechamente equilibrados	EC sin importantes limitaciones o evidencia abrumadora de estudios observacionales.	La mejor acción puede variar dependiendo de las circunstancias de los pacientes o de los valores de la sociedad.
2.B	Débil	Moderada	Estrechamente equilibrados	EC con importantes limitaciones (resultados inconsistentes, defectos metodológicos indirectos o imprecisos) o pruebas excepcionalmente fuertes a partir de estudios observacionales.	La mejor acción puede variar dependiendo de las circunstancias de los pacientes o de los valores de la sociedad.
2.C	Débil	Baja o muy baja	Incertidumbre en las estimaciones de beneficios, riesgos y carga	Estudios observacionales o series de casos.	Otras alternativas pueden ser igual de razonables.

8.2. Planificación y análisis de los estudios epidemiológicos

Una vez definidos los objetivos y el diseño, es fundamental planificar detalladamente el estudio: seleccionar la población, estimar el tamaño muestral, establecer criterios de inclusión y exclusión, definir las variables, las fuentes de datos y los métodos de recolección, procesamiento y análisis.

Mientras que la definición del problema y la elección del diseño ya han sido abordadas en instancias previas, en este curso nos centraremos en la **fase analítica**, haciendo especial énfasis en el tratamiento de los datos derivados de distintos tipos de estudios.

Dado que una gran proporción de la investigación en salud es observacional, comenzaremos por los análisis correspondientes a estudios transversales, de casos y controles, y de cohortes. En todos los casos, la comunicación de los resultados debe ser clara y estructurada, para facilitar la comprensión del proceso, desde la planificación hasta las conclusiones.

La formulación de recomendaciones en torno a la comunicación de la investigación puede contribuir a mejorar su calidad.

8.2.1. Declaración STROBE

La **Declaración STROBE** (*Strengthening the Reporting of Observational Studies in Epidemiology*) (Elm et al. 2008) es una guía internacional destinada a estandarizar y mejorar la calidad del reporte de estudios observacionales, incluyendo estudios transversales, de casos y controles, y de cohortes.

Se trata de una iniciativa colaborativa que reúne a epidemiólogos, metodólogos, estadísticos, investigadores y editores de revistas científicas, comprometidos con la realización y difusión de investigaciones observacionales rigurosas.

STROBE incluye una lista de **22 ítems**, organizados en secciones del manuscrito (título, resumen, introducción, métodos, resultados, discusión y otra información relevante), que especifican lo que debe informarse para asegurar la **transparencia** y **reproducibilidad** de los hallazgos. Esta guía también es una herramienta útil para investigadores, ya que indica claramente los elementos esenciales que deben estar presentes en un artículo científico basado en estudios observacionales. El respaldo de STROBE en revistas biomédicas es creciente, siendo requisito editorial en gran parte de ellas.

Puede descargarse desde el sitio oficial de STROBE:

⍰**Guía completa y *checklist***

⍰ **Versión en español**

La declaración STROBE ha sido adaptada para contextos específicos mediante extensiones temáticas:

STROME-ID

STROME-ID (*Strengthening the Reporting of Molecular Epidemiology for Infectious Diseases*), tiene como objetivo establecer recomendaciones que respalden un adecuado informe científico de estudios en epidemiología molecular de enfermedades infecciosas. Incentiva un reporte adecuado que considere amenazas específicas a la validez, como errores de clasificación, contaminación cruzada, o sesgos derivados de los métodos de laboratorio y análisis genético.

⍰**Descargar STROME-ID**

STROBE-VET

STROBE-VET (*STROBE - Veterinary Extension*), orienta el reporte de estudios observacionales en medicina veterinaria, salud pública, zoonosis y seguridad alimentaria. Facilita la evaluación crítica y reproducibilidad en investigaciones con poblaciones animales.

⍰ **Descargar STROBE-VET**

RECORD

RECORD (*Reporting of studies Conducted using Observational Routinely-collected health Data*) fue creada para estudios que utilizan datos rutinarios, como registros electrónicos de salud, bases de datos administrativas o registros poblacionales.

Descargar RECORD

8.2.2. Declaración CONSORT

En cuanto a estudios experimentales, la **Declaración CONSORT** (*Consolidated Standards of Reporting Trials*) (Schulz et al. 2010) es una guía internacional destinada a mejorar la calidad del reporte de los ensayos clínicos aleatorizados (ECA). Fue desarrollada en 1996 y revisada por primera vez en 2001, con actualizaciones posteriores (la última versión es de 2010).

Muchas revistas biomédicas la han adoptado como requisito editorial, lo que ha contribuido significativamente a la mejora en la calidad, transparencia y reproducibilidad de los informes sobre ECA.

CONSORT establece un conjunto mínimo de 25 ítems organizados en secciones del artículo científico (título, resumen, introducción, métodos, resultados, discusión y otros), complementados por un diagrama de flujo que muestra el número de participantes en cada fase del estudio (asignación, intervención, seguimiento y análisis). Esta herramienta ofrece un formato estándar para que los autores preparen informes completos y transparentes de los resultados de los ensayos, facilitando su evaluación e interpretación crítica.

Puede descargarse desde el sitio oficial de CONSORT:

https://www.consort-spirit.org/

> ⚠ En este curso nos centraremos en los aspectos metodológicos y en el análisis de datos según el diseño del estudio. Para ello, utilizaremos el **lenguaje R**, un entorno especializado en análisis estadístico que permitirá explorar diferentes enfoques adaptados a cada tipo de diseño epidemiológico. Partiremos de cada diseño y discutiremos posibles caminos de análisis, reconociendo que dichos caminos no siempre son únicos y que ciertos elementos del análisis pueden ser comunes a varios diseños.

9. Estudios de corte transversal

9.1. Introducción

Los estudios transversales se caracterizan por realizar mediciones en un **único momento** en el tiempo, sin llevar a cabo un seguimiento de los participantes. En este diseño, el investigador mide de forma simultánea la **exposición** y el **evento** en los sujetos seleccionados mediante criterios de inclusión y exclusión predefinidos.

Se trata de un diseño **observacional de base individual**, que puede tener un propósito **descriptivo**, **analítico**, o ambos. También se los conoce como estudios de prevalencia o encuestas transversales. Los estudios transversales entonces pueden dividirse en:

- **Descriptivos:** se enfocan en determinar la **frecuencia** y **distribución** de eventos de salud y enfermedad en un momento determinado, midiendo una o más variables o condiciones.
- **Analíticos:** se orientan a explorar **asociaciones** y generar hipótesis de investigación. Algunos autores ubican este diseño en el límite entre los estudios descriptivos y los analíticos.

En los estudios transversales, la medida de frecuencia más utilizada es la **prevalencia**. Para evaluar asociaciones se pueden calcular dos medidas principales:

- **Razón de prevalencia** (*prevalence ratio*)**:** que aproxima el riesgo relativo observado en estudios de cohortes.
- **Odds ratio de prevalencia** (*prevalence odds ratio*): que se utiliza como aproximación al *odds ratio* calculado en estudios de casos y controles.

A continuación se resumen las ventajas y desventajas de este diseño:

Ventajas	Desventajas
Eficientes para estudiar la prevalencia de enfermedades en la población	Problemas para definir y medir exposición
Se pueden estudiar varias exposiciones	Sesgos de selección
Son poco costosos y se pueden realizar en poco tiempo	Sesgos por casos prevalentes
Se puede estimar la prevalencia del evento	La relación causa efecto no siempre es verificable
	Sobrerrepresentación de enfermos con tiempos prolongados de sobrevida o con manifestaciones con mejor curso clínico
	Se puede presentar causalidad débil

Nota: Los aspectos metodológicos esenciales para el reporte de estudios transversales pueden consultarse en la **Declaración STROBE**, sección *Métodos*.

9.2. Análisis de estudios transversales

Al planificar un estudio transversal, es fundamental definir con precisión la **población de estudio**. La selección de los sujetos depende directamente de la pregunta de investigación y los objetivos planteados. Una identificación adecuada de la población garantiza la validez externa de los resultados y su potencial generalización.

En la práctica, estos estudios rara vez incluyen a todos los miembros de la población objetivo, sino que se trabaja con una **muestra**. Solo una muestra representativa, que refleje las características de la población base, permite desarrollar el componente analítico y exploratorio del diseño transversal, de modo que las conclusiones sean relevantes y aplicables.

Una vez obtenidos los datos, éstos pueden representarse gráficamente, resumirse mediante estadísticas descriptivas (como medidas de tendencia central y dispersión) e incluso utilizarse para modelar relaciones entre variables.

- Si el objetivo es **describir los resultados** en la muestra, las herramientas descriptivas pueden ser suficientes.
- Si se busca **extrapolar a la población general,** es necesario aplicar métodos de **inferencia estadística**, fundamentados en la teoría de la probabilidad.

10. Inferencia estadística

10.1. Introducción

La **estadística inferencial** es la rama de la estadística que permite formular conclusiones sobre una población a partir del análisis de una muestra. Se apoya en el cálculo de probabilidades, que proporciona el marco teórico para modelar fenómenos aleatorios y generalizar los resultados muestrales a toda la población. Dado que las inferencias basadas en muestras están sujetas a incertidumbre, es fundamental expresarlas siempre en términos **probabilísticos**.

Las dos actividades principales en este proceso son:

- **Estimación de parámetros**: consiste en calcular, a partir de los datos muestrales, valores que aproximen parámetros desconocidos de la población.
    - *Ejemplo: Estimar la prevalencia de una enfermedad X en la población.*
- **Pruebas de hipótesis**: implica evaluar con base estadística afirmaciones acerca de uno o más parámetros.

  Ejemplos:

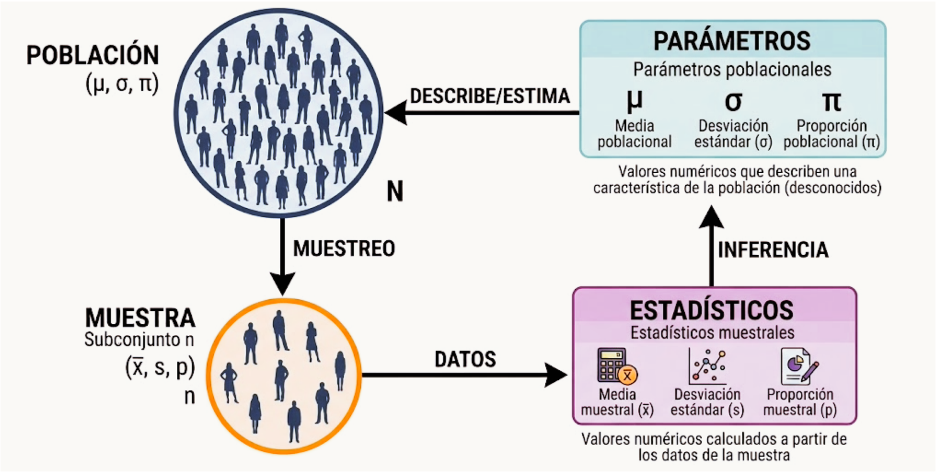
    - *¿La prevalencia de la enfermedad X en Argentina es menor que en Uruguay?*
    - *¿La prevalencia de la enfermedad X en Argentina en 2010 fue menor que en el año 2000?*

Al trabajar con muestras, entra en juego el concepto de **teoría del muestreo**, que, si bien no abordaremos en profundidad aquí, es clave para comprender cómo se relacionan los valores observados en la muestra con los de la población.

Esta teoría estudia la relación entre la distribución de una variable en la población y el comportamiento de dicha variable en muestras aleatorias extraídas de ella. A las medidas obtenidas a partir de la muestra se las denomina **estadísticos muestrales** o simplemente *estadísticos*, mientras que sus contrapartes en la población se denominan **parámetros**.

Por ejemplo, supongamos que queremos conocer el valor medio de colesterol total de la población de Mar del Plata y tomamos una muestra de tamaño n.

- La media poblacional del colesterol total se representa con la letra griega μ y corresponde al **parámetro**.
- La media muestral, que se obtiene a partir de los datos de la muestra, se representa como $\bar{x}$ y es un **estimador** o estadístico muestral.


POBLACIÓN
(μ, σ, π)
N
DESCRIBE/ESTIMA
PARÁMETROS
Parámetros poblacionales
μ
Media poblacional
σ
Desviación estándar (σ)
π
Proporción poblacional (π)
Valores numéricos que describen una característica de la población (desconocidos)
MUESTREO
INFERENCIA
MUESTRA
Subconjunto n
(x̄, s, p)
n
DATOS
ESTADÍSTICOS
Estadísticos muestrales
Media muestral (x̄)
Desviación estándar (s)
Proporción muestral (p)
Valores numéricos calculados a partir de los datos de la muestra

La **distribución muestral de un estadístico** es la distribución de todos los valores posibles que ese estadístico puede tomar al calcularse en muestras aleatorias del mismo tamaño extraídas de una misma población. Este concepto es central en la inferencia estadística, ya que permite cuantificar la incertidumbre asociada a las estimaciones.

Para comprender mejor qué es una distribución muestral y cómo se construye, pueden ver la siguiente videoclase:

⍰ **Distribuciones Muestrales**

Como construir una distribución muestral puede resultar muy laborioso cuando la población es grande, y directamente imposible si es infinita, se suelen utilizar aproximaciones basadas en la toma de un gran número de muestras aleatorias del mismo tamaño.

10.2. Intervalos de Confianza (IC)

Una forma eficaz de abordar la inferencia estadística es a través de los **intervalos de confianza (IC)**, ya que, aunque son procedimientos inferenciales, están estrechamente vinculados con la estadística descriptiva.

Supongamos que queremos estimar la **media de colesterol** de la población de Mar del Plata. Sería inviable medir el colesterol de cada habitante, por lo que optamos por tomar una muestra de, por ejemplo, 100, 200 o 300 individuos (más adelante veremos cómo determinar el tamaño adecuado de la muestra). Debemos recordar que diferentes muestras producirán en general medias diferentes. Existe, por tanto, un grado de incertidumbre asociado. Si hiciéramos una estimación puntual, obtendríamos un solo valor, pero sin información sobre su variabilidad. No sabríamos qué tan cerca o lejos está nuestra estimación ($\bar{x}$) de la verdadera media poblacional (μ).

El **intervalo de confianza** proporciona un rango de valores dentro del cual se espera que se encuentre el valor verdadero del parámetro poblacional, con un cierto nivel de confianza. A diferencia de la estimación puntual que proporciona un único valor numérico, el intervalo consta de dos valores entre los cuales se supone está contenido el parámetro estimado. Entonces, el intervalo de confianza puede expresarse como:

$$IC = estimador\ puntual \pm (coeficiente\ de\ confiabilidad) * (error\ estandar)$$

donde:

- **Estimador puntual**:

  - Para la media poblacional (μ), se toma la media muestral($\bar{x}$).
  - Para una proporción de la población (p), se toma la proporción muestral ($\widehat{p}$).

- **Coeficiente de confiabilidad:** Se relaciona con el nivel de confianza deseado (por ejemplo, 90 %, 95 % o 99 %), y se expresa como $1-\alpha$, es decir la probabilidad de que el parámetro se encuentre dentro del IC. Recordemos que el nivel de significancia (α) es la probabilidad de que el parámetro no se halle dentro del IC y es un valor generalmente pequeño (por ejemplo, 0.1, 0.05 o 0.01) expresado como probabilidad o porcentaje (por ejemplo, 10 %, 5 % o 1 %).
- **Error estándar (SE):** Representa la variabilidad de la distribución muestral. Por ejemplo, para la media el error estándar se calcula como la raíz cuadrada de la varianza de la distribución muestral:

$$SE = \frac{\sigma}{\sqrt{n}}$$

Donde σ es la desviación estándar poblacional y n el tamaño de la muestra. Si se estima un IC para una proporción, el error estándar es:

$$SE = \sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}}$$

El proceso se fundamenta en el Teorema del Límite Central (TCL), que establece que, para muestras suficientemente grandes, la distribución de $\bar{x}$ es aproximadamente normal, con media μ y varianza σ^2/n. Así, la variable tipificada:

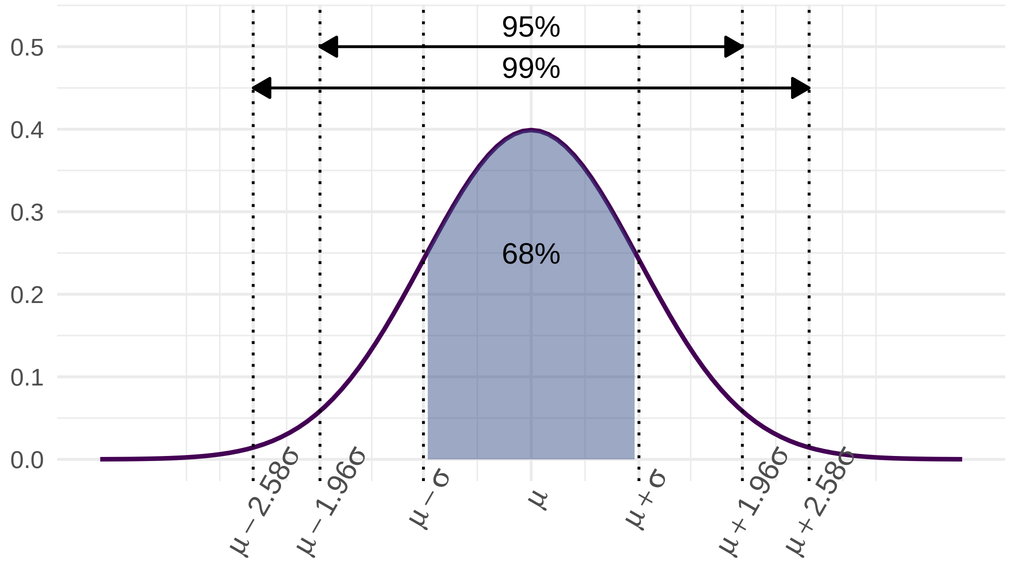
$$Z = \frac{\bar{x}-\mu}{\sigma}$$

sigue una distribución normal estándar (media 0 y desviación estándar 1), lo que permite calcular probabilidades y construir el IC.

En cualquier distribución normal:

- Entre $\mu \pm \sigma$ se encuentra el 68 % de los datos.
- Entre $\mu \pm 2\sigma$ se encuentra el 95 %.
- Entre $\mu \pm 3\sigma$ se encuentra el 99 %.

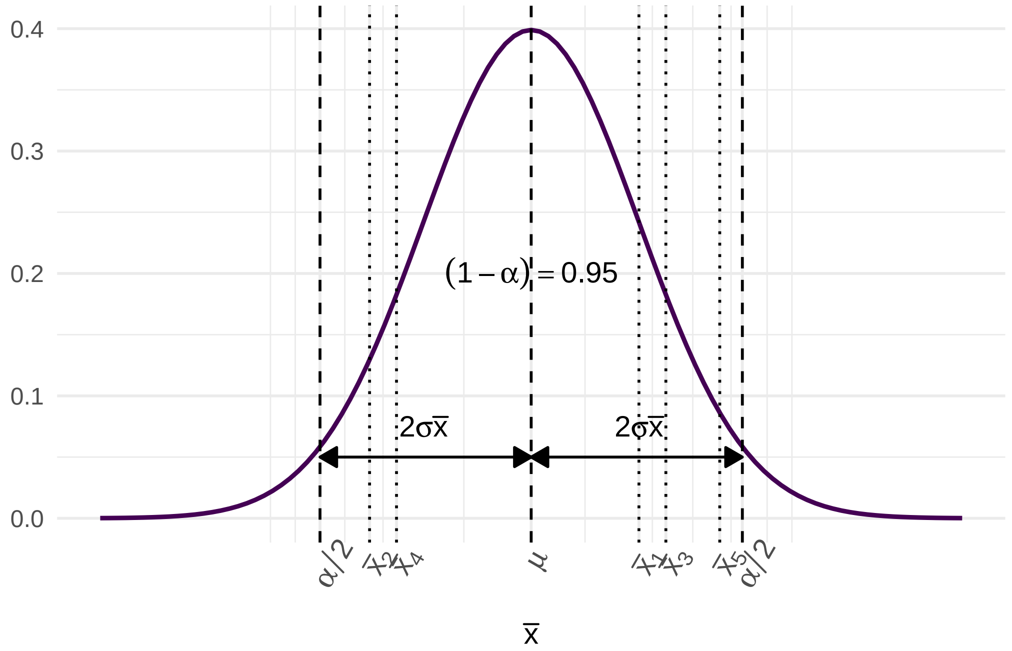
El siguiente gráfico ilustra lo explicado anteriormente:



Sabemos que, independientemente de la localización de los valores, aproximadamente el **95 %** de los valores posibles de $\bar{x}$ en la distribución muestral estarán a menos de dos desviaciones estándar de la media μ. Es decir, el intervalo $\mu \pm 2\sigma$ contendrá el **95 %** de los valores posibles de $\bar{x}$.

Supongamos que formamos intervalos a partir de todos los posibles valores de $\bar{x}$, calculados a partir de todas las muestras posibles de tamaño n tomadas de la población de interés. Esto generará una gran cantidad de intervalos de la forma $\mu \pm 2\sigma$, todos con la misma amplitud, centrados en torno a una μ desconocida.

Aproximadamente el **95 %** de estos intervalos tendrán sus centros dentro del intervalo $\mu \pm 2\sigma$. Cada uno de estos intervalos, que se encuentran dentro de $\mu \pm 2\sigma$, puede contener el valor verdadero de μ.



Finalmente, y basándonos en las propiedades de la distribución Normal, se puede deducir la expresión del IC:

$$P(Z_{\alpha/2} < Z_{1-\alpha/2)} = 1 - \alpha$$

Reemplazando Z:

$$P(Z_{\alpha/2} < \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}} < Z_{(1-\alpha/2)}) = 1 - \alpha$$

Reordenando, la expresión del IC para la media queda:

$$\bar{x} - Z_{\alpha/2}\sqrt{\frac{\sigma^2}{n}} < \mu < \bar{x} + Z_{\alpha/2}\frac{\sigma^2}{n}$$

10.2.1. ¿Cómo se interpreta un IC?

Si hubiésemos tomado múltiples muestras del mismo tamaño de la población, en al menos $100 * (1 - \alpha)\,\%$ de las ocasiones el intervalo calculado contendría el parámetro poblacional real. Es decir, un IC al 95 % implica que, a largo plazo, el 95 % de los intervalos obtenidos a partir de muestras repetidas incluirán el valor verdadero del parámetro.

El producto del coeficiente de confiabilidad y el error estándar se denomina **precisión de la estimación** y es el componente responsable de la amplitud del IC. Recordemos que la fórmula general para construir un intervalo de confianza era:

$$IC = estimador\, puntual \pm (coeficiente\, de\, confiabilidad) * (error\, estandar)$$

Para el caso de la media:

- **Aumento de la confiabilidad:** Si se incrementa el nivel de confianza, el coeficiente (por ejemplo, pasando de 1.96 a un valor mayor) aumenta, lo que a su vez incrementa la amplitud del IC.
- **Reducción del error estándar:** Si se fija la confiabilidad (por ejemplo, al 95 %), para disminuir la amplitud del IC es necesario reducir el error estándar. Dado que el error estándar de la media es:

$$SE = \frac{\sigma}{\sqrt{n}}$$

y considerando que σ es constante, la única forma de disminuir el error estándar es aumentando el tamaño muestral (n).

Surge entonces la pregunta: **¿qué tan grande debe ser n?**
La respuesta dependerá de σ, del nivel de significación (α) y de la amplitud deseada para el IC. La relación es:

$$Amplitud = Z\frac{\sigma}{\sqrt{n}} \Longrightarrow n = \frac{Z^2\sigma^2}{Amplitud^2} \quad (Z = 1.96\; si\; \alpha = 0.05)$$

(En la práctica, σ generalmente no se conoce, así que se usa su estimación muestral).

La expresión del error estándar varía según el parámetro a estimar. Hemos visto el caso de la media; si lo que se desea es calcular un IC para una proporción, recordemos que, para muestras grandes, la distribución de las proporciones de la muestra es aproximadamente normal de acuerdo con el TCL. En este caso:

- La media de la distribución es la proporción real p
- La varianza es $p(1-p)/n$, lo que nos lleva a que el error estándar es:

$$SE = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

y el IC para la proporción se expresa como:

$$\hat{p} - Z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} < p < \hat{p} + Z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Dado que un intervalo de confianza implica una declaración probabilística, su cálculo se fundamenta en las distribuciones muestrales de los estimadores y en el correspondiente error estándar. Aunque las fórmulas pueden parecer complejas, los paquetes estadísticos (como R) realizan estos cálculos automáticamente. Lo fundamental es comprender en qué depende la amplitud del IC (nivel de confianza, error estándar y tamaño muestral) y cómo cada uno de estos componentes influye en la precisión de la estimación.

Para profundizar y visualizar simulaciones sobre estos conceptos, pueden explorar recursos interactivos como:

⍰ **Viendo la teoría: Una introducción visual a probabilidad y estadística**

Además, existe un ejemplo práctico de cálculo de IC para la media utilizando la distribución t de Student en el siguiente enlace:

⍰ **Intervalos de confianza con R**

10.3. Test de hipótesis

Aunque los estudios de corte transversal no se diseñan originalmente con grupos de comparación, en aquellos de carácter más analítico es frecuente establecer comparaciones. Por ejemplo, pueden surgir preguntas como:

¿La prevalencia de la enfermedad es mayor en mujeres, en determinados grupos etarios o en una provincia específica?

Con esto en mente, revisaremos las herramientas que permiten comparar grupos mediante test o contrastes de hipótesis. El propósito de estos test es ofrecer al investigador una herramienta para tomar decisiones sobre la población a partir de la información obtenida en una muestra.

Antes de adentrarnos en la parte estadística, es importante distinguir entre dos tipos de hipótesis:

- **Hipótesis de investigación:** Es la conjetura o suposición que motiva la investigación.
- **Hipótesis estadística:** Es aquella que puede ser evaluada mediante técnicas estadísticas apropiadas.

En este texto nos centraremos en aclarar aspectos relativos a las hipótesis estadísticas, asumiendo que las hipótesis de investigación ya han sido discutidas previamente por los investigadores. Describiremos brevemente el razonamiento subyacente a estos test.

Los contrastes de hipótesis parten de una **hipótesis nula**, la cual afirma que los dos grupos comparados son iguales o, en otras palabras, que las diferencias observadas se deben únicamente al azar. Como ya se ha mencionado, la variabilidad intrínseca de cualquier muestra impide que la diferencia entre grupos sea exactamente cero.

El método estadístico nos permite cuantificar la diferencia entre grupos asumiendo que, si repitiésemos el experimento infinitas veces obtenemos todas las posibles muestras del tamaño indicado a partir de nuestras poblaciones (*distribución muestral*), las diferencias entre grupos “iguales” se distribuirían conforme a una curva teórica. Basándonos en las propiedades de esta distribución, podemos determinar un valor límite que comprende, por ejemplo, el 95 % o el 99 % de las diferencias esperadas. Si la diferencia observada entre las muestras supera este valor límite, se considera excesiva para ser atribuible al azar y, por tanto, se rechaza la hipótesis nula. Por el contrario, si la diferencia cae dentro del área del 95 %, se concluye que la diferencia encontrada podría atribuirse al azar, y no habría evidencia que nos permita rechazar la hipótesis nula. En estos casos decimos que los grupos “no son diferentes” pero no “son iguales”, ya que la variabilidad inherente impide probar una igualdad exacta.

Los contrastes de hipótesis se realizan generalmente bajo las siguientes condiciones:

- Se asume a priori que la ley de distribución de la población es conocida.
- Se extrae una muestra aleatoria de dicha población.

El conjunto de estas técnicas de inferencia se denomina **técnicas paramétricas**. Sin embargo, existen otros métodos, denominados **técnicas no paramétricas** o contrastes de distribuciones libres, que no requieren estimar parámetros ni suponer una ley de probabilidad específica para la población. Algunos de estos métodos serán desarrollados más adelante en este módulo.

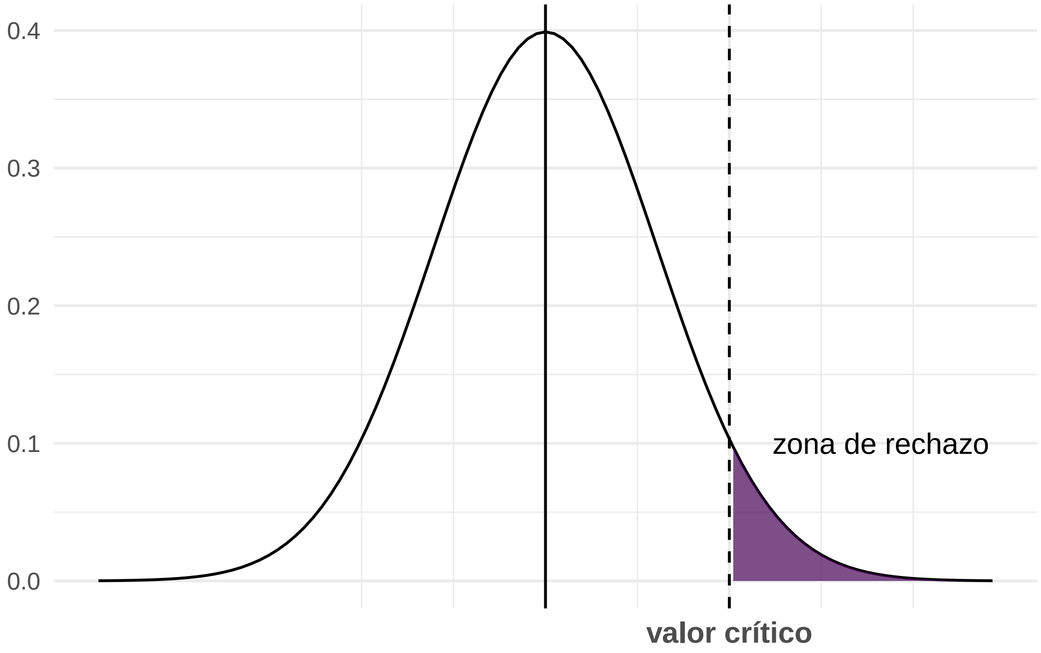
Es importante señalar que los contrastes de hipótesis se utilizan no solo en estudios transversales, sino con mayor frecuencia en diseños en los que *a priori* existen grupos de comparación, como en estudios de casos y controles, cohortes, ensayos clínicos, entre otros. Desarrollaremos aquí la teoría que los sustenta y los utilizaremos a lo largo de todo el curso.

10.3.1. Estructura del test de hipótesis

Los componentes básicos de cualquier test de contraste de hipótesis son:

- **Hipótesis nula** (H_0): Afirma que no existe diferencia entre los grupos que se comparan, es decir, que las variaciones observadas se deben únicamente al azar.
- **Hipótesis alternativa** (H_1): Es la conjetura o suposición que plantea el investigador, estableciendo que sí existe una diferencia entre los grupos. Generalmente es complementaria de la H_0.
- **Estadístico de prueba**: Es el valor calculado a partir de los datos muestrales que se utiliza para tomar la decisión sobre la H_0. Cada tipo de problema tiene un estadístico adecuado, cuya magnitud, al compararse con su distribución muestral (por ejemplo, la distribución normal estándar en el caso del estadístico Z), permite determinar si las diferencias observadas son atribuibles al azar.

- **Valor crítico o Región crítica**: La región crítica se establece en función del nivel de significación (α) y consiste en el conjunto de valores extremos del estadístico de prueba que, de ser alcanzados, llevarían a rechazar la H_0. Todos los posibles valores del estadístico se ubican en el eje horizontal de la gráfica de su distribución, y la región crítica delimita aquellos valores que son muy poco probables bajo la hipótesis nula.



La regla de decisión es la siguiente:

- Si el valor del estadístico de prueba calculado a partir de la muestra cae en la región crítica, se rechaza la H_0 y se concluye que las diferencias observadas son **estadísticamente significativas**.
- Si el valor no cae en la región crítica, no se rechaza la H_0; esto indica que las diferencias entre lo observado y lo esperado pueden explicarse por el azar. Es decir, **no son estadísticamente significativas**.
- **Nivel de significación** (α): Es la probabilidad de cometer un error tipo I, es decir, rechazar H_0 cuando es verdadera. Este valor, elegido por el investigador (comúnmente 5 % o 1 %), determina el límite entre la región de no rechazo y la región crítica.
- **Valor *p***: Es una medida de qué tan probable son los resultados de la muestra, considerando que H_0 sea verdadera.
  - Un valor p muy pequeño indica que es muy poco probable obtener los resultados observados para el estadístico muestral si H_0 fuese cierta, por lo que debemos rechazarla.
  - Esto significa que si el valor $p \leq \alpha$, es posible rechazar H_0; mientras que si $p > \alpha$, no es posible rechazar la hipótesis nula.

  Por ejemplo, si en un test de contraste de dos proporciones se obtiene un estadístico $Z = 3,034$ y un valor $p = 0.0024$, esto significa que la probabilidad de obtener un valor de Z de 3.034 o mayor, suponiendo que H_0 es cierta, es del 0.24 %. Dado que este valor es mucho menor que un nivel de significación del 5 % (o incluso del 1 %), se rechaza H_0 y se concluye que la diferencia observada es estadísticamente significativa. En otras palabras, si bajo un supuesto dado, la probabilidad de un suceso observado particular es excepcionalmente pequeña, concluimos que el supuesto probablemente es incorrecto (*regla del suceso infrecuente*).

10.3.2. Tipos de contrastes

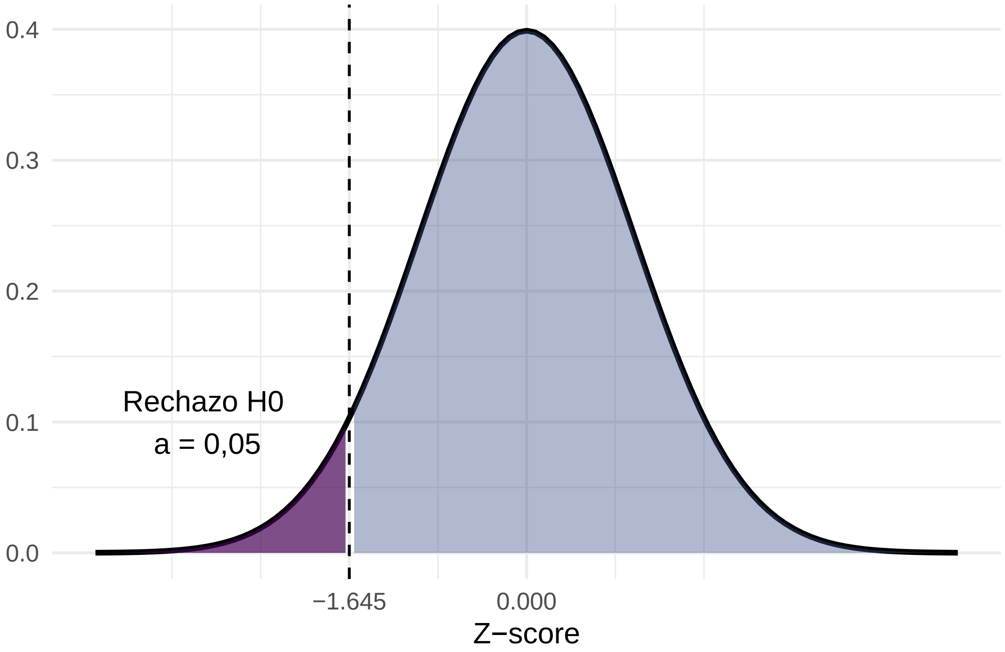
Los contrastes de hipótesis se clasifican según la forma de la hipótesis alternativa (H_1). Esta clasificación determina si la prueba es **unilateral** (de cola izquierda o derecha) o **bilateral** (de dos colas). Siendo las colas de la distribución las regiones extremas limitadas por los valores críticos.

Test de cola izquierda

La hipótesis alternativa plantea que la media del primer grupo es significativamente **menor** que la del segundo:

$$H_1 : \mu_1 < \mu_2$$

La región crítica se encuentra en el extremo izquierdo de la distribución. Todo el área crítica tiene un tamaño α con un valor crítico de $-1,645$.

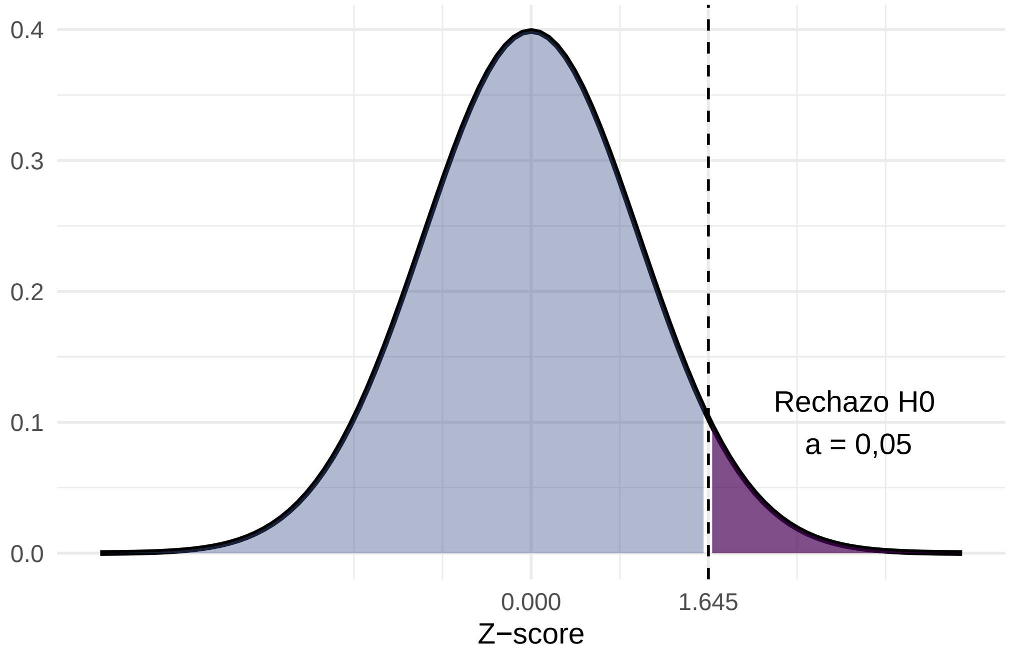


Test de cola derecha

La hipótesis alternativa establece que la media del primer grupo es significativamente **mayor** que la del segundo:

$$H_1 : \mu_1 > \mu_2$$

La región crítica se concentra en el extremo derecho de la distribución y toda el área crítica tiene un tamaño α con un valor crítico de $1,645$.

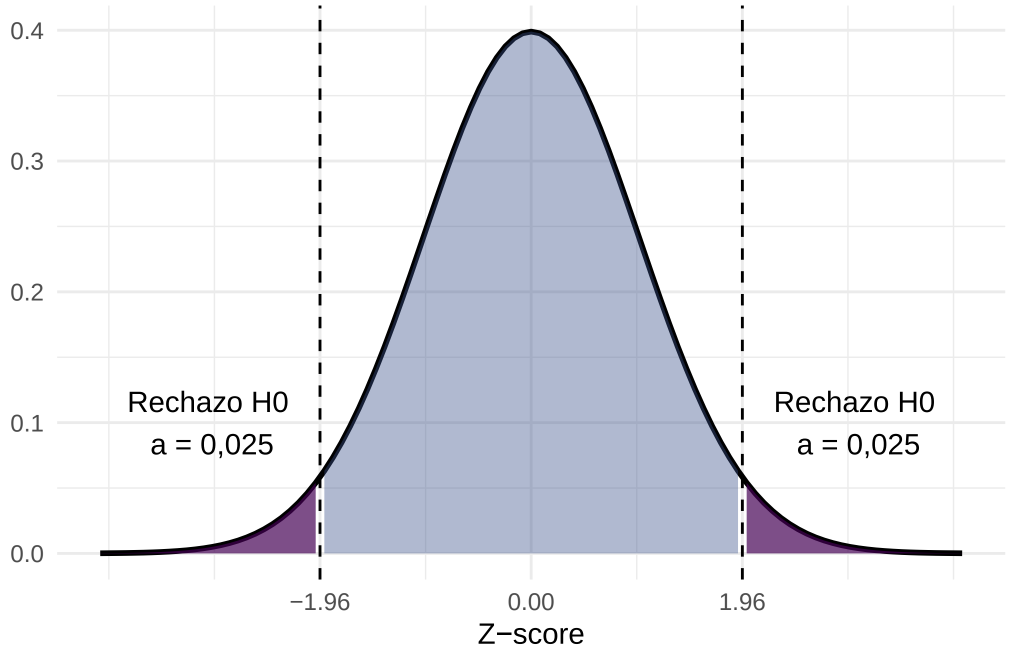


Pruebas bilaterales

La hipótesis alternativa afirma que existen diferencias entre los grupos, **sin especificar la dirección**:

$$H_1 : \mu_1 \neq \mu_2$$

La región crítica se divide entre ambos extremos de la distribución, con valores críticos de $\pm 1,96$. El nivel de significación total (α) se reparte en partes iguales entre las dos colas ($\alpha/2$ en cada una), lo que implica un 2,5 % de probabilidad en cada cola si H_0 es verdadera.



10.3.3. Errores

Al utilizar el razonamiento de los contrastes de hipótesis, existen dos tipos principales de errores que podemos cometer:

- **Error tipo I** (α): Ocurre cuando el investigador rechaza la hipótesis nula (H_0) siendo esta verdadera en la población y se concluye erróneamente que existe una diferencia cuando en realidad no la hay. Se suele eligir un valor pequeño de α (0.01, 0.05 y 0.10) para hacer que la probabilidad de rechazo de H_0 sea pequeña.
- **Error tipo II (**β): Ocurre cuando el investigador no rechaza la H_0 siendo esta falsa en la población, es decir, se falla en detectar una diferencia real. Generalmente β es mayor que α, pero su valor real se desconoce en la práctica.

Es importante notar que, una vez que se realiza el procedimiento de prueba, no es posible saber con certeza si se ha cometido alguno de estos errores, ya que se desconoce el verdadero estado de la realidad. Sin embargo, al fijar un α pequeño, se busca asegurar que, en caso de rechazar la H_0, la probabilidad de haber cometido un error Tipo I sea baja.

En resumen, al interpretar los resultados de un test de hipótesis:

- **Si se rechaza la** H_0: se asume que la probabilidad de haber cometido un error Tipo I es baja (debido al valor pequeño de α).
- **Si no se rechaza la** H_0: se desconoce el riesgo real de un error Tipo II, pero es importante tener en cuenta que, en muchas situaciones, este riesgo es mayor que el de un error Tipo I.

La tabla que se presenta a continuación resume las posibles situaciones a las que nos enfrentamos con los test de hipótesis:

	No rechazar H0	Rechazar H0
H0 es cierta	Correcto (1-α)	Error tipo I (α)
H0 es falsa	Error tipo II (β)	Correcto (1-β)

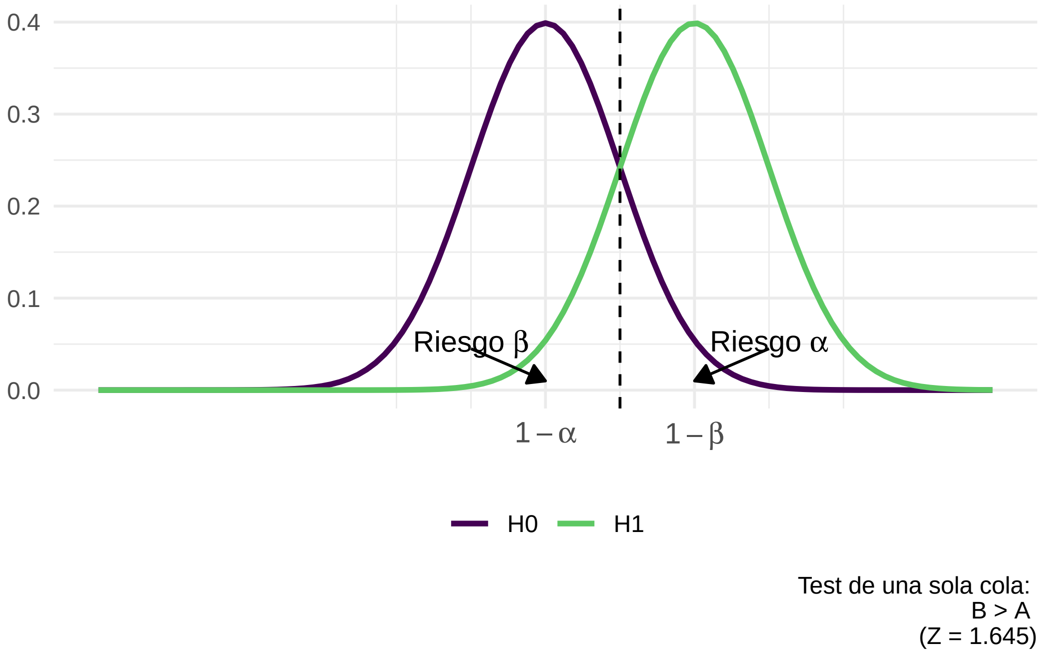
Para quienes están familiarizados con el ámbito del diagnóstico, existe una clara analogía entre los falsos positivos y falsos negativos en las pruebas diagnósticas y, respectivamente, el error Tipo I y el error Tipo II en los contrastes de hipótesis.

Hemos discutido el significado de α (error Tipo I); ahora veamos qué implica β. Recordemos que el error Tipo II es análogo a los falsos negativos de las pruebas diagnósticas: es la probabilidad de no detectar una diferencia cuando, en realidad, ésta existe. En otras palabras, β es la probabilidad de no rechazar la hipótesis nula (H_0) siendo esta falsa.

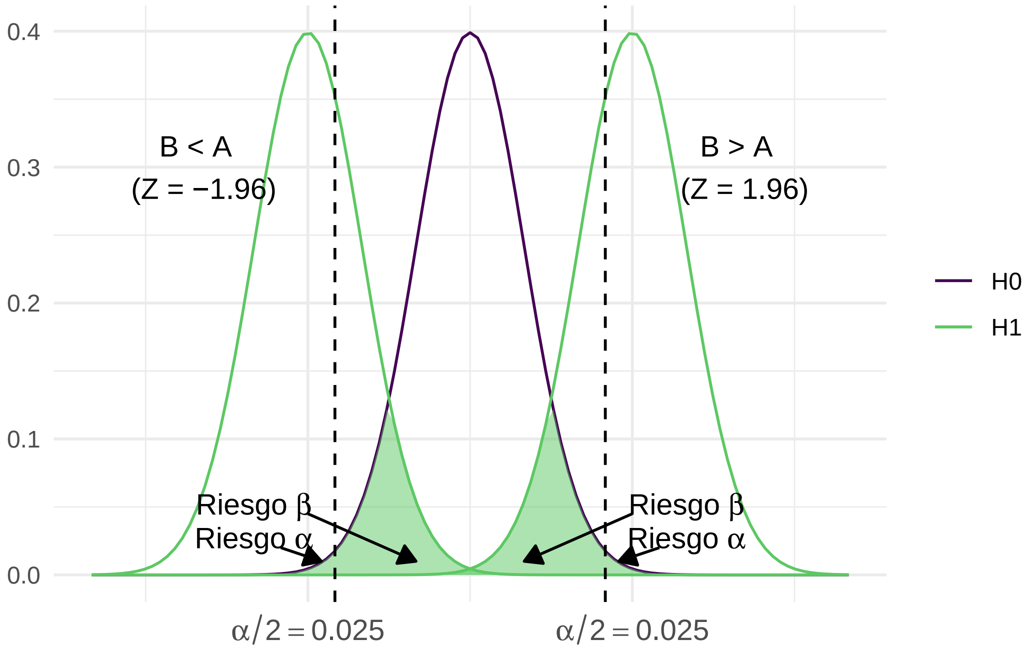
A diferencia de α que se fija en un único valor y es determinado por el investigador, β varía según el valor real del parámetro en estudio. Por ejemplo, si consideramos la hipótesis nula $H_0 : \mu_1 - \mu_2 = 0$, habrá un valor de β para cada posible diferencia entre μ_1 y μ_2 cuando el valor real no sea cero. La probabilidad de detectar correctamente una diferencia real - es decir, de obtener un resultado estadísticamente significativo cuando la diferencia existe- se denomina **potencia estadística** y se expresa como $1 - \beta$.

Los contrastes de hipótesis no son exclusivos de los estudios transversales; por el contrario, su uso es más común en estudios analíticos que involucran grupos de comparación, tales como estudios de casos y controles, cohortes o ensayos clínicos. Por ejemplo, en un ensayo clínico que evalúa dos tratamientos, la potencia estadística refleja la capacidad del estudio para identificar un efecto real del tratamiento.

Es deseable que la potencia del estudio sea lo mayor posible, ya que esto incrementa la probabilidad de detectar diferencias verdaderas. Sin embargo, no es posible minimizar ambos errores simultáneamente ya que al disminuir α (es decir, al ser más exigentes para rechazar la H_0), β tiende a aumentar, y viceversa. En los contrastes, la hipótesis privilegiada es H_0 que solo será rechazada cuando la evidencia de su falsedad supere el umbral del $1-\alpha$. Esto significa que, a menos que la evidencia en contra de H_0 sea muy significativa, se opta por no rechazarla. Lo ideal a la hora de definir un test es encontrar un compromiso entre β y α.



Mientras que para tests bilaterales:



Finalmente, podemos decir que la potencia estadística ofrece un segundo mecanismo de seguridad en un contraste de hipótesis. Es como contar con una protección adicional en la toma de decisiones: si solo dispusiéramos del nivel de significación (α), tendríamos menos garantías. Al incorporar la potencia ($1-\beta$) agregamos un segundo control. Por ello, en un contraste no basta con tener un valor *p* pequeño; también se necesita una potencia alta, que en la práctica suele fijarse en un 80 % (es decir, $1-\beta=0.8$).
Por otro lado, cuando existe una diferencia real —o un efecto real de una terapia, o una verdadera diferencia entre dos fármacos—, la **magnitud** de ese efecto influye en la facilidad para detectarlo. Los efectos grandes son más fáciles de identificar que los pequeños. Para estimar la potencia de una prueba, debemos especificar el **efecto mínimo** que valga la pena identificar.

La potencia de un test estadístico depende de tres factores que actúan de manera interrelacionada:

- El **riesgo de error** que se tolerará al rechazar la hipótesis de ausencia de efecto o diferencia (α).
- La **dimensión de la diferencia** que se desea identificar en relación con la variabilidad en las poblaciones.
- El **tamaño de la muestra**.

Del mismo modo que en un problema de estimación se necesita una idea de la magnitud a estimar y del error aceptable para definir el tamaño de la muestra, en un contraste de hipótesis se requiere conocer el **tamaño del efecto** que se quiere detectar. Así, el tamaño muestral se determina en función del nivel de confianza y de la potencia de la prueba, además de otros aspectos relacionados con el diseño y la prueba estadística elegida.

En epidemiología, una de las situaciones más frecuentes al diseñar un estudio es el cálculo del tamaño muestral para un **nivel de confianza del 95 %** y una **potencia del 80 %**, que, como se mencionó, es un nivel de potencia alto y que además permite manejar un α relativamente bajo. El lenguaje **R** cuenta con diversas funciones para **calcular y visualizar** la relación entre tamaño muestral, potencia, tamaño del efecto y nivel de confianza, facilitando el diseño de estudios con un adecuado balance entre la probabilidad de detectar diferencias reales y la de controlar errores estadísticos.

Por ejemplo, la función `pwr.t.test()` del paquete `pwr` (Champely 2020), calcula la potencia para pruebas t de Student de medias (para una muestra, dos muestras y muestras pareadas), basado en el tamaño de la muestra, el nivel de confianza y el tamaño de efecto:

```
pwr.t.test(n, d, sig.level = 0.05, power, type, alternative)
```

Donde:

- `n`: Número de observaciones para cada grupo.
- `d`: tamaño de efecto (d de Cohen) - diferencia (estandarizada) entre grupos.

  > *Nota:* La d de Cohen representa las desviaciones estándar que separan dos o más grupos. Por ejemplo: $d_{Cohen} = 0.5$ representa que la diferencia entre grupo experimental y muestral es de media desviación estándar. Cohen sugirió (provisoriamente) que 0.2 es un tamaño de efecto pequeño, 0.5 es mediano y 0.8 es grande,

- `sig.level`: nivel de significación (probabilidad del error de tipo I).
- `power`: potencia del test (1 menos la probabilidad del error tipo II).
- `type`: tipo de test (`"two.sample"`, `"one.sample"`, `"paired"`).
- `alternative`: palabra que especifica la hipotesis alternativa, debe ser `"two.sided"` (predeterminado), `"greater"` or `"less"`.

La función se ejecuta incorporando todos los argumentos obligatorios (`d`,`n`, `power` y `sig.level`) menos el que se quiere calcular. En ese caso se iguala a `NULL` o se omite.

Supongamos que queremos conocer el tamaño de la muestra para detectar diferencias en la media de la hemoglobina glicosilada entre dos grupos de pacientes con tratamientos de control de la diabetes distintos. Aceptamos un nivel de efecto convencional de una pequeña desviación ($d_{Cohen} = 0.2$), una potencia del 80 % y una significación habitual de 0,05.

Cargamos el paquete requerido:

```r
library(pwr)
```

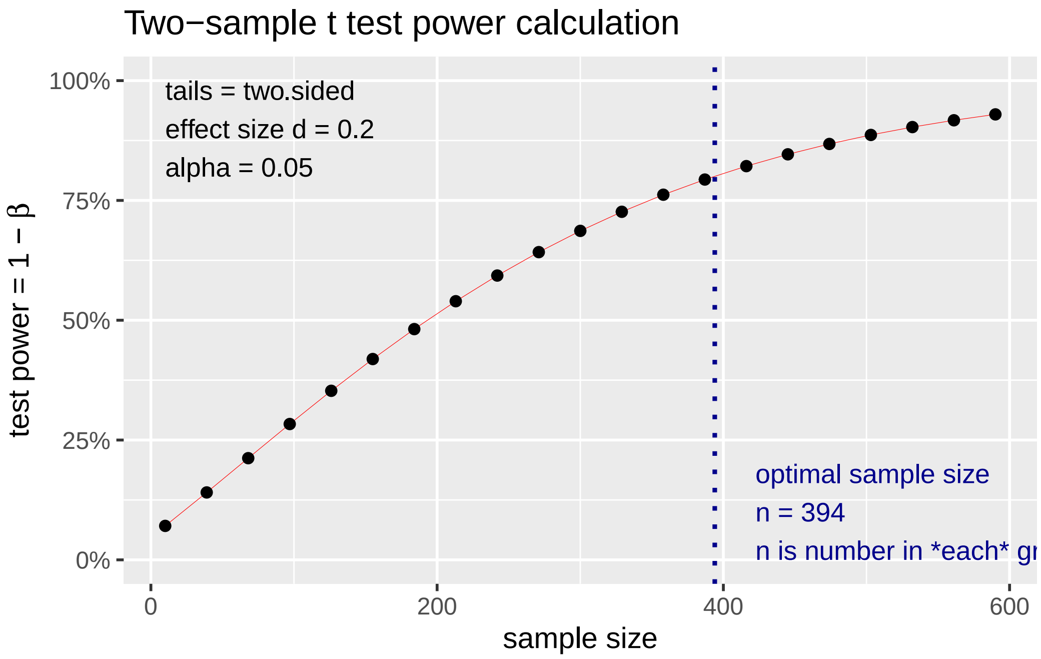
Calculamos el tamaño de muestra:

```r
pwr.t.test(
  d = 0.2,
  power = 0.8,
  sig.level = 0.05,
  type = "two.sample",
  alternative = "two.sided"
)
```

```
     Two-sample t test power calculation 

              n = 393.4057
              d = 0.2
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

También podemos graficar la salida:

```r
pwr.t.test(
  d = 0.2,
  power = 0.8,
  sig.level = 0.05,
  type = "two.sample",
  alternative = "two.sided"
) |>
  plot()
```



Observamos que, si por ejemplo tomásemos una muestra de 300 individuos por grupo, la potencia del estudio con ese tamaño de efecto y nivel de significación del 0,05 sería aproximadamente de 68 %.

10.3.4. ¿Qué test se debe aplicar en cada caso?

Hemos discutido la mecánica general de los test de hipótesis. Ahora, nos centraremos en orientarnos sobre qué test aplicar en cada situación. Aunque existe un desarrollo teórico detrás de cada caso, aquí nos quedaremos con ciertas reglas prácticas.

Para sistematizar los contrastes de hipótesis, es útil responder dos preguntas fundamentales:

- **¿Qué tipo de variable dependiente tengo?**
  La variable resultado puede ser cuantitativa, cualitativa, de tiempo hasta un evento, etc.
- **¿Qué se está comparando?**
  Esto se traduce en evaluar el tipo de experimento: ¿se comparan dos grupos o más? ¿Son muestras independientes o relacionadas (por ejemplo, medidas en el mismo grupo antes y después de una intervención)?

Variable resultado cuantitativa

Cuando la variable de interés es cuantitativa, la comparación de grupos generalmente se traduce en comparar las medias de dichos grupos. En este contexto, se deben responder una tercera pregunta y es si dicha variable se *distribuye normalmente*, porque si no fuera así, debemos recurrir a los contrastes *no paramétricos*.

> En la **siguiente sección** detallaremos las pruebas de normalidad y heterogeneidad de varianzas y su aplicación en R.

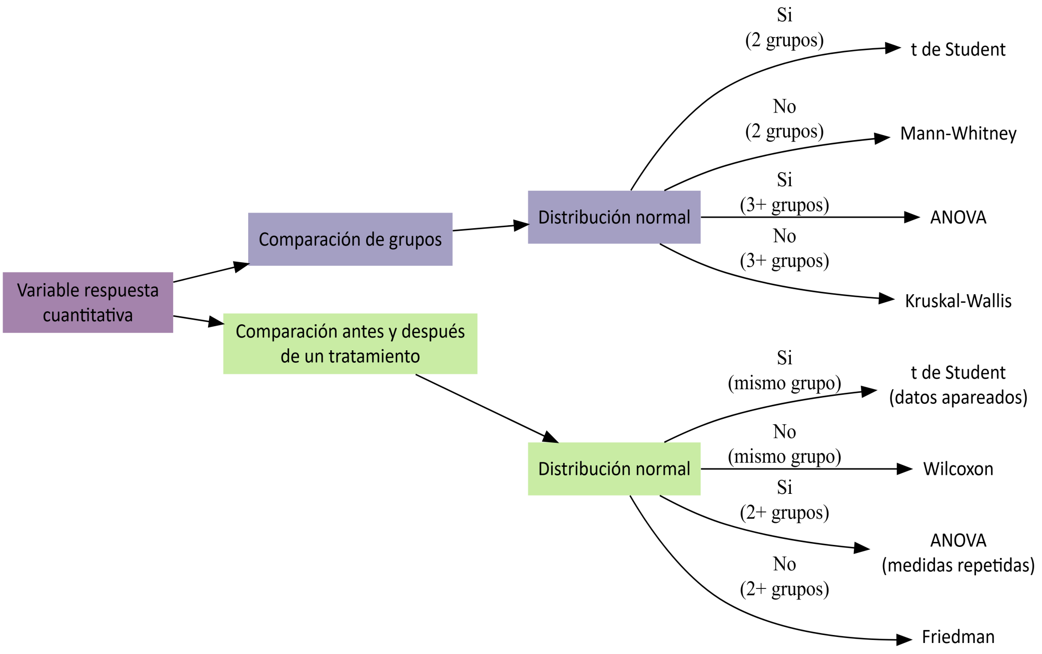
Por ejemplo, para comparar dos grupos se plantearía:

- $H_0 : \mu_1 = \mu_2$ (o su equivalente $\mu_1 - \mu_2 = 0$)

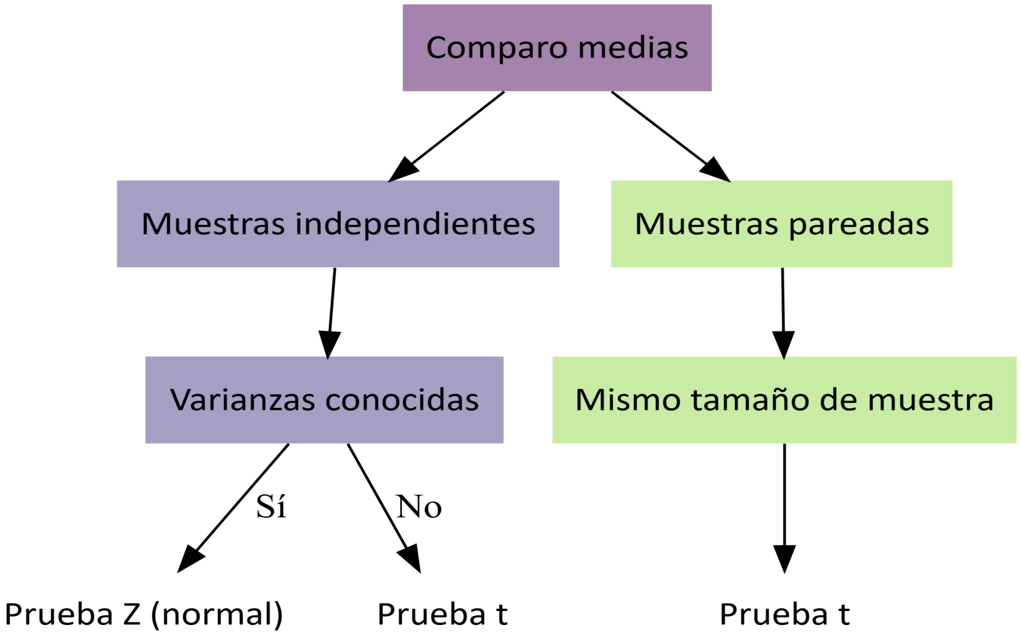
- $H_1 : \mu_1 \neq \mu_2$ (contraste bilateral) o bien $\mu_1 < \mu_2$ o $\mu_1 > \mu_2$ (contrastes unilaterales).

Se calcula el estadístico de prueba (por ejemplo, t de Student o Z, según el tamaño de la muestra y si se conoce la varianza poblacional), se determina la región de rechazo y, finalmente, si $p < \alpha$ rechazo H_0 y si $p > \alpha$ no rechazo H_0.

A continuación presentamos un esquema que sirve de guía para potenciales situaciones:


Variable respuesta cuantitativa
Comparación de grupos
Comparación antes y después de un tratamiento
Distribución normal
Si (2 grupos)
t de Student
No (2 grupos)
Mann-Whitney
Si (3+ grupos)
ANOVA
No (3+ grupos)
Kruskal-Wallis
Distribución normal
Si (mismo grupo)
t de Student (datos apareados)
No (mismo grupo)
Wilcoxon
Si (2+ grupos)
ANOVA (medidas repetidas)
No (2+ grupos)
Friedman

Hay algo más que los estadísticos nos “obligan” a considerar en el caso que compare medias independientes y se haya verificado la normalidad de la distribución, y se trata de considerar si las varianzas de las poblaciones que comparo son iguales o diferentes. El algoritmo de resolución es el siguiente:

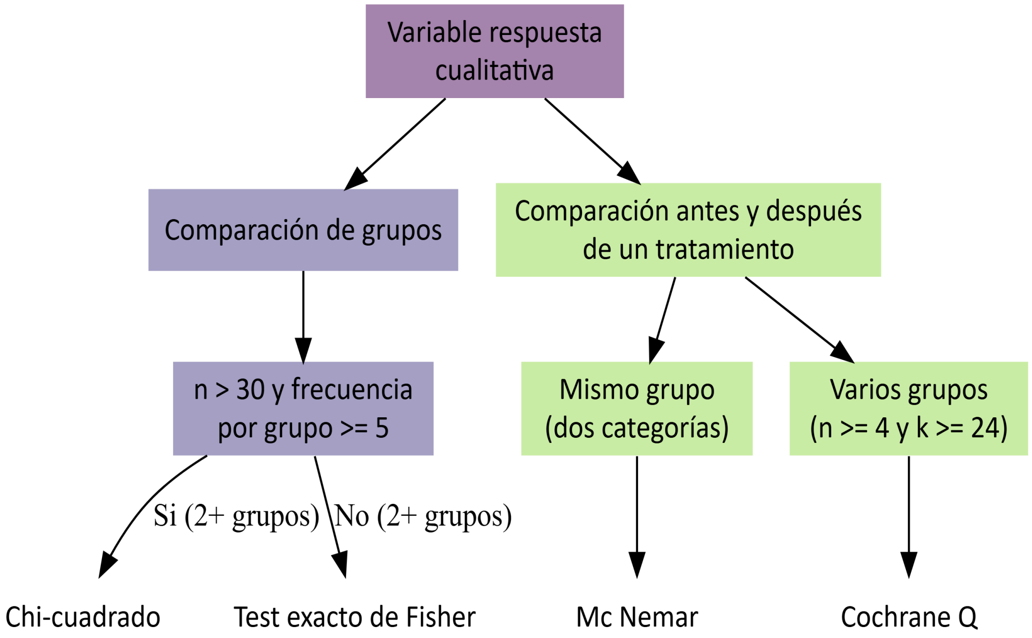

Comparo medias
Muestras independientes
Muestras pareadas
Varianzas conocidas
Mismo tamaño de muestra
Sí
No
Prueba Z (normal)
Prueba t
Prueba t

Cuando trabajamos con muestras independientes, a menudo nos encontramos en el escenario de *varianzas desconocidas*. Esto nos obliga a realizar, de manera previa, un test de homogeneidad de varianzas para determinar si se puede asumir que son iguales o si, por el contrario, difieren. La razón es que, aunque se utiliza el estadístico t de Student para comparar las medias, su cálculo y la forma en que se distribuye varían en función de si las varianzas de los grupos son iguales o no.

Variable resultado cualitativa

Cuando la variable dependiente es cualitativa (categórica), como por ejemplo Enfermo (Sí/No) o Expuesto (Sí/No), la comparación de grupos no se basa en medias, sino en **proporciones**. Para ello, se utilizan pruebas estadísticas específicas según el número de grupos y el diseño del estudio.

A continuación, presentamos una guía análoga a la anterior:


Variable respuesta cualitativa
Comparación de grupos
Comparación antes y después de un tratamiento
n > 30 y frecuencia por grupo >= 5
Mismo grupo (dos categorías)
Varios grupos (n >= 4 y k >= 24)
Si (2+ grupos)
No (2+ grupos)
Chi-cuadrado
Test exacto de Fisher
Mc Nemar
Cochrane Q

10.4. Ejemplo práctico en R

En el archivo “`bebe1.txt`” se encuentran datos de una muestra aleatoria de los pesos al nacer de 186 bebés cuyas madres no consumieron cocaína durante el embarazo. En el archivo “`bebe2.txt`” se encuentran datos de una muestra aleatoria de 190 bebés cuyas madres consumieron cocaína durante el embarazo. Nos interesa conocer el peso promedio al nacer de los bebés de la primer base de datos y, de acuerdo a estos resultados, saber si el consumo de cocaína afecta el peso que tiene un bebé al nacer.

Podemos resolver la primer parte manualmente:

$$IC = \bar{x} \pm Z_{1-\alpha/2}\frac{\sigma}{\sqrt{n}}$$

donde:

- media ($\bar{x}$): 3103 gramos
- desvío estándar (σ): 696

- número de observaciones (n): 186

$$IC = 3103 \pm 2,57\frac{696}{\sqrt{186}} = 3103 \pm 131,16$$

$$IC = 2971,85 - 3234,15$$

El peso medio al nacer de los bebés oscila entre 2971,85 y 3234,15 gramos ($\alpha = 0,99$).

Para resolver las preguntas en R, comenzaremos cargando los paquetes necesarios:

```
library(tidyverse)
library(dlookr)
```

Cargamos los datos:

```
datos1 <- read_csv2("datos/bebe1.txt")

datos2 <- read_csv2("datos/bebe2.txt")
```

La función `t.test()` del paquete `base` utiliza la distribución t *de Student*:

```
t.test(datos1, conf.level = 0.99)
```

```

	One Sample t-test

data:  datos1
t = 60.804, df = 185, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
99 percent confidence interval:
 2970.178 3235.822
sample estimates:
mean of x 
     3103 
```

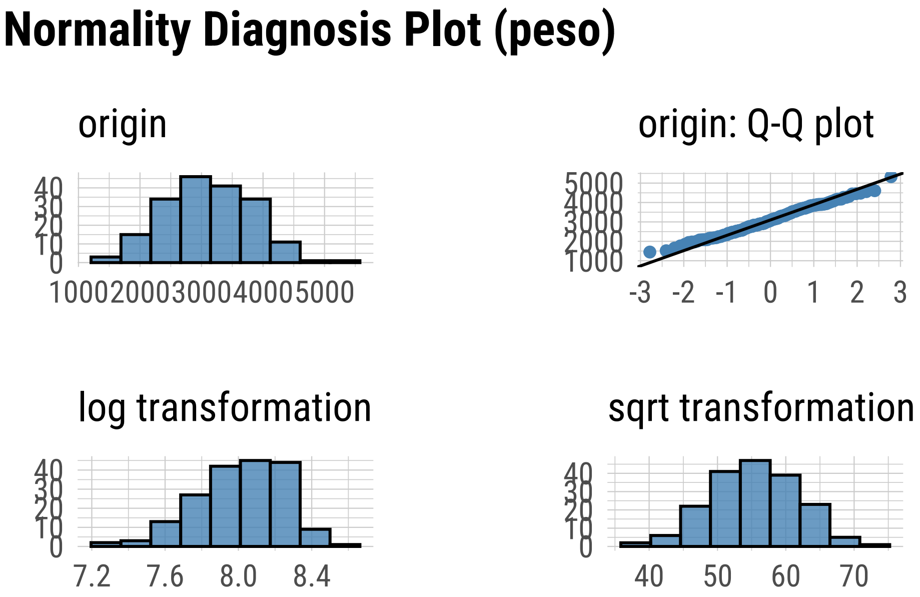
Los IC producidos son similares a los calculados manualmente.

Para comparar los datos de los bebés de madres que consumieron cocaína, debemos realizar un test de comparación de medias para dos poblaciones independientes. Para ello, comenzamos por chequear normalidad:

```
shapiro.test(datos1$peso)
```

```

	Shapiro-Wilk normality test

data:  datos1$peso
W = 0.99293, p-value = 0.5092
```

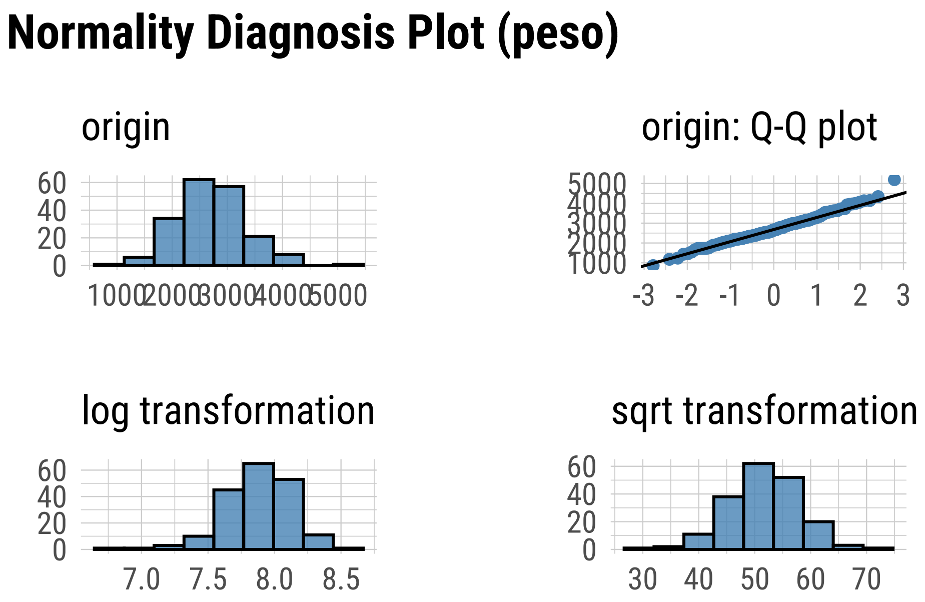
```r
datos1 |>
  plot_normality()
```



```r
shapiro.test(datos2$peso)
```

```

	Shapiro-Wilk normality test

data:  datos2$peso
W = 0.98951, p-value = 0.1774
```

```r
datos2 |>
  plot_normality()
```



Se cumple con el supuesto de normalidad, ahora procedemos a comparar varianzas usando la función `var.test()`:

```
var.test(datos1$peso, datos2$peso)
```

```

	F test to compare two variances

data:  datos1$peso and datos2$peso
F = 1.1644, num df = 185, denom df = 189, p-value = 0.2986
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.8736082 1.5526583
sample estimates:
ratio of variances 
          1.164392 
```

Como las varianzas son iguales, aplicamos un t test para comparar medias de dos poblaciones independientes con varianzas iguales planteando un contraste bilateral:

$$H_0 : \mu_1 = \mu_2$$
$$H_1 : \mu_1 \neq \mu_2$$

```
t.test(
       x = datos1$peso,
       y = datos2$peso,
       alternative = "two.sided",
       var.equal = TRUE
)
```

```
	Two Sample t-test

data:  datos1$peso and datos2$peso
t = 5.8252, df = 374, p-value = 1.231e-08
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 266.9644 539.0356
sample estimates:
mean of x mean of y 
     3103      2700 
```

O también podríamos plantear un contraste unilateral, si consideramos que nuestra hipótesis de trabajo es que el peso de los bebés de madres que no consumen cocaína es mayor que aquellos de madres que sí consumen.

$$H_0 : \mu_1 = \mu_2$$
$$H_1 : \mu_1 > \mu_2 \qquad \mu_1 - \mu_2 > 0$$

```r
t.test(
       x = datos1$peso,
       y = datos2$peso,
       alternative = "greater",
       var.equal = TRUE
)
```

```
	Two Sample t-test

data:  datos1$peso and datos2$peso
t = 5.8252, df = 374, p-value = 6.155e-09
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
 288.9222      Inf
sample estimates:
mean of x mean of y 
     3103      2700 
```

Concluimos que los pesos de los bebés hijos de madres que no consumen cocaína es significativamente mayor respecto de los hijos de madres que sí la consumen.

10.4.1. Inferencia sobre una población

En algunas situaciones, además de obtener intervalos de confianza para estimar parámetros como la media o la proporción poblacional, también puede ser de interés realizar contrastes de hipótesis sobre un valor específico de referencia. Este tipo de análisis se denomina **inferencia sobre una población** y permite evaluar si un parámetro poblacional es significativamente diferente de un valor dado.

Un caso común en epidemiología y medicina es el estudio de la eficacia de un tratamiento. Supongamos que queremos evaluar si un nuevo fármaco reduce la presión arterial en pacientes con un determinado síndrome. Se sabe que, en ausencia de tratamiento, la presión arterial media en estos pacientes es de **165 mmHg**. Para investigar el efecto del fármaco, se administra a un grupo de **15 pacientes** y se mide su presión arterial después del tratamiento.

Queremos evaluar si la presión arterial media en pacientes tratados con el nuevo fármaco es **significativamente menor** que la presión arterial histórica de 165 mmHg:

- $H_0 : \mu = 165\, mmHg$
- $H_1 : \mu < 165\, mmHg$

Este es un **contraste unilateral a la izquierda**, ya que buscamos evidencia de que el tratamiento reduce la presión arterial.

Cargamos los datos:

```
enfermos <- read_csv2("datos/enfermos.txt")
```

Calculamos la media y el desvío estándar:

```
enfermos |>
  summarise(media = mean(V1), desvio = sd(V1))
```

```
# A tibble: 1 x 2
  media desvio
  <dbl>  <dbl>
1  152.   22.0
```

Evaluamos normalidad:

```
shapiro.test(enfermos$V1)
```

```

	Shapiro-Wilk normality test

data:  enfermos$V1
W = 0.94967, p-value = 0.5192
```

Este grupo de enfermos tratados con el nuevo fármaco tienen una media de aproximadamente 151.9619928, con un desvío de 21.9788237. Además, los datos cumplen con el supuesto de normalidad.

Realizamos la prueba de hipótesis para evaluar si este valor es significativamente menor a 165:

```
t.test(x = enfermos$V1, alternative = "less", mu = 165, conf.level = 0.95)
```

```
    One Sample t-test

data:  enfermos$V1
t = -2.2975, df = 14, p-value = 0.01876
alternative hypothesis: true mean is less than 165
95 percent confidence interval:
     -Inf 161.9573
sample estimates:
mean of x
  151.962
```

Dado que $p < 0.05$, rechazamos la hipótesis nula y concluimos que, con un nivel de confianza del 95 %, el nuevo fármaco **reduce significativamente la presión arterial** en estos pacientes.

11. Análisis de normalidad y homocedasticidad

11.1. Introducción

Este material es una continuación del capítulo sobre **Análisis exploratorio de datos** de la **Unidad 1**. Recordemos que uno de los objetivos del EDA es conocer cómo se distribuyen los valores de las variables de interés.

Las características fundamentales a la hora de decidir si utilizaremos métodos paramétricos o no paramétricos para la inferencia estadística, es que los datos se ajusten a una distribución normal y conocer si tienen una dispersión homogénea o heterogénea. Ahora que conocemos el funcionamiento interno de los test de hipótesis, podemos usar **pruebas de bondad de ajuste** para evaluar los supuestos de normalidad y homocedasticidad.

11.2. Normalidad

Determinar que una distribución es aproximadamente normal nos permite decidirnos por test de comparaciones paramétricos.

Existen tres enfoques que debemos analizar simultáneamente:

- Métodos gráficos
- Métodos analíticos
- Pruebas de bondad de ajuste

11.2.1. Métodos gráficos

El gráfico por excelencia para evaluar normalidad es el Q-Q Plot que consiste en comparar los cuantiles de la distribución observada con los cuantiles teóricos de una distribución normal con la misma media y desviación estándar que los datos.

Cuanto más se aproximen los datos a una normal, más alineados están los puntos entorno a la recta.

En el lenguaje R hay varios paquetes que tienen funciones para construirlos:

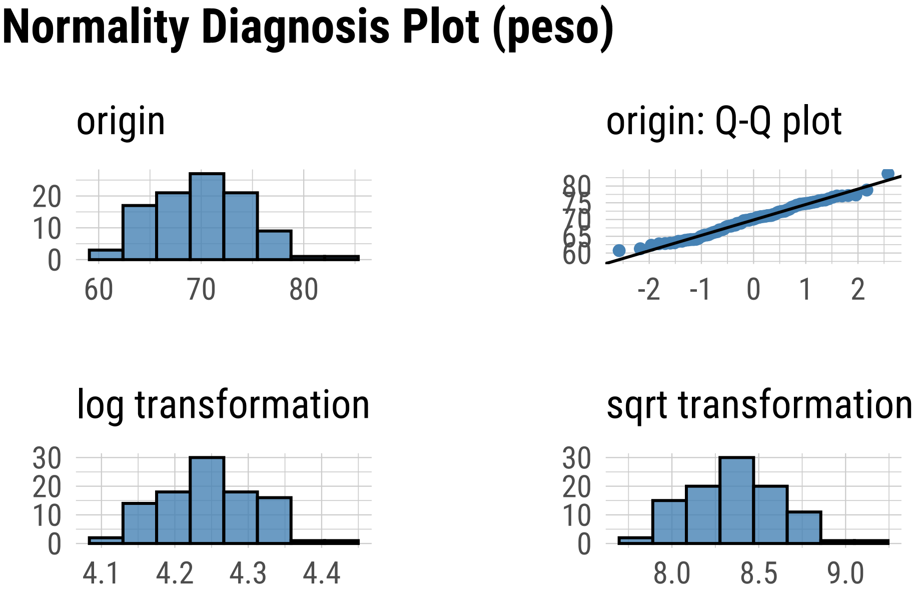
```
library(dlookr)
library(ggpubr)
library(moments)
library(nortest)
library(car)
library(tidyverse)
```

Cargamos datos de ejemplo:

```
datos <- read_csv2("datos/datos_normalidad.csv")
```

Evaluación gráfica de normalidad usando la función `plot_normality()` del paquete `dlookr`:

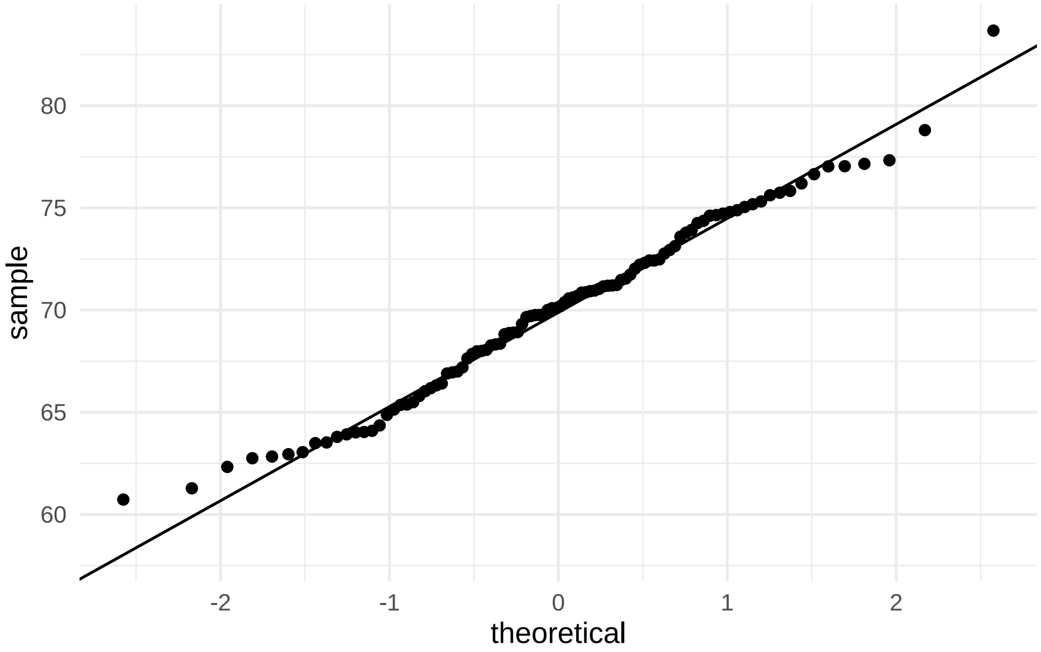
```
datos |>
  plot_normality(peso)
```



A simple vista observamos que los puntos de la variable `peso` *se ajustan bastante bien a la recta*.

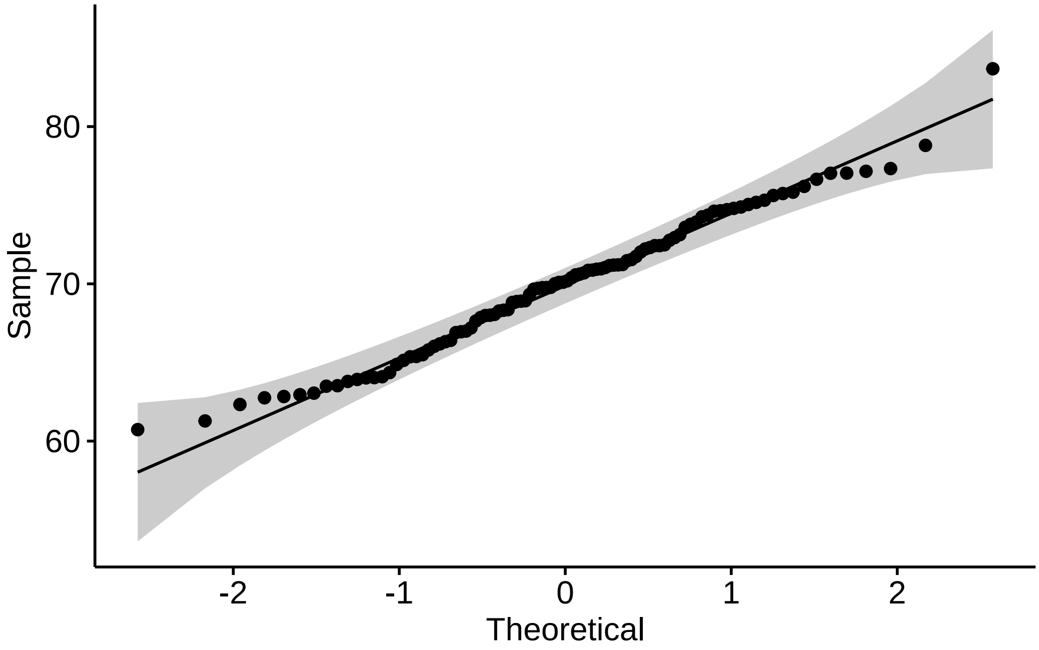
También podemos generar el qqplot usando las funciones `geom_qq_line()` y `geom_qq()` de `ggplot2`:

```
datos |>
  ggplot(mapping = aes(sample = peso)) +
  geom_qq_line() +
  geom_qq() +
  theme_minimal()
```



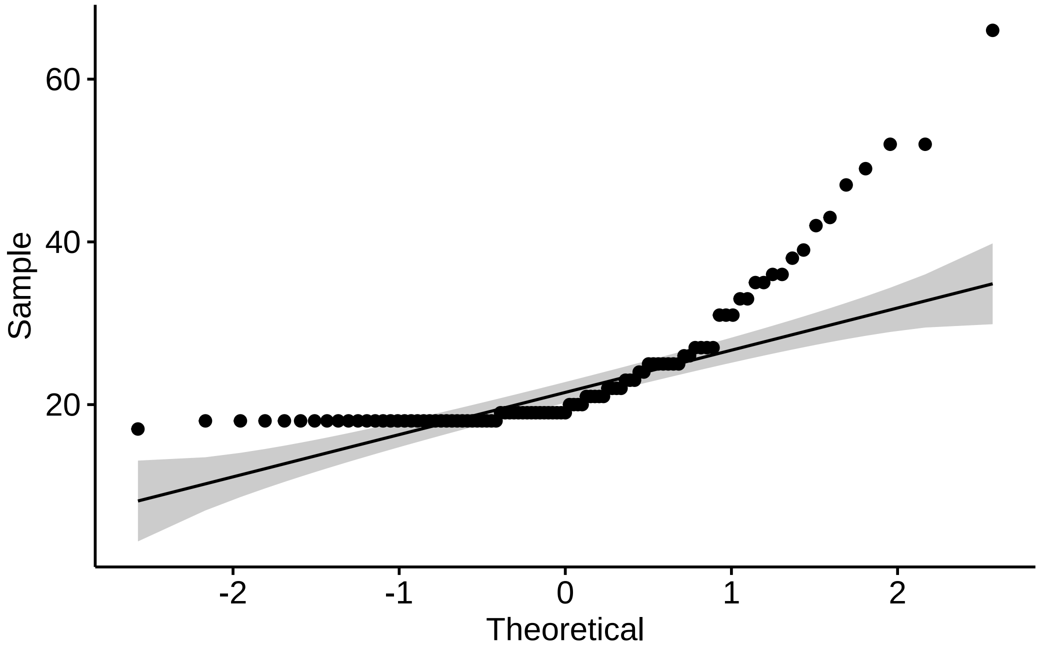
La función `ggqqplot()` del paquete `ggpubr` (Kassambara 2026a) nos permite agregar intervalos de confianza (zona gris alrededor de la recta) que nos orienta mejor sobre “donde caen” los puntos de la variable analizada:

```
datos$peso |>
  ggqqplot()
```



Un ejemplo donde la variable parece no cumplir con el supuesto de normalidad en estos datos de prueba es `edad`:

```
datos$edad |>
  ggqqplot()
```



11.2.2. Métodos analíticos

Medidas de forma

Existen dos medidas de forma útiles que podemos calcular mediante funciones de R.

- La curtosis (kurtosis)
- La asimetría (skewness)

La curtosis mide el grado de agudeza o achatamiento de una distribución con relación a la distribución normal.

- < 0 Distribución platicúrtica (apuntamiento negativo): baja concentración de valores
- > 0 Distribución leptocúrtica (apuntamiento positivo): gran concentración de valores
- $= 0$ Distribución mesocúrtica (apuntamiento normal): concentración como en la distribución normal.

El paquete `moments` (Komsta y Novomestky 2022) posee algunas funciones interesantes para analizar medidas de forma, como el estimador de Pearson para curtosis:

```
datos |>
  summarise(
    kurtosis_edad = kurtosis(edad, na.rm = T),
    kurtosis_peso = kurtosis(peso, na.rm = T)
  )
```

```
# A tibble: 1 x 2
  kurtosis_edad kurtosis_peso
          <dbl>         <dbl>
1          8.05          2.72
```

En los dos casos estamos frente a una *distribución leptocúrtica* pero de magnitudes bien diferentes. Muy alta en el caso de la variable `edad` (8,0) y mucho menor para la variable `peso` (2,7).

El índice de asimetría es un indicador que permite establecer el *grado de asimetría* que presenta una distribución. Los valores menores que 0 indican distribución asimétrica negativa; los mayores a 0: distribución asimétrica positiva y cuando sea 0, o muy próximo a 0, distribución simétrica:

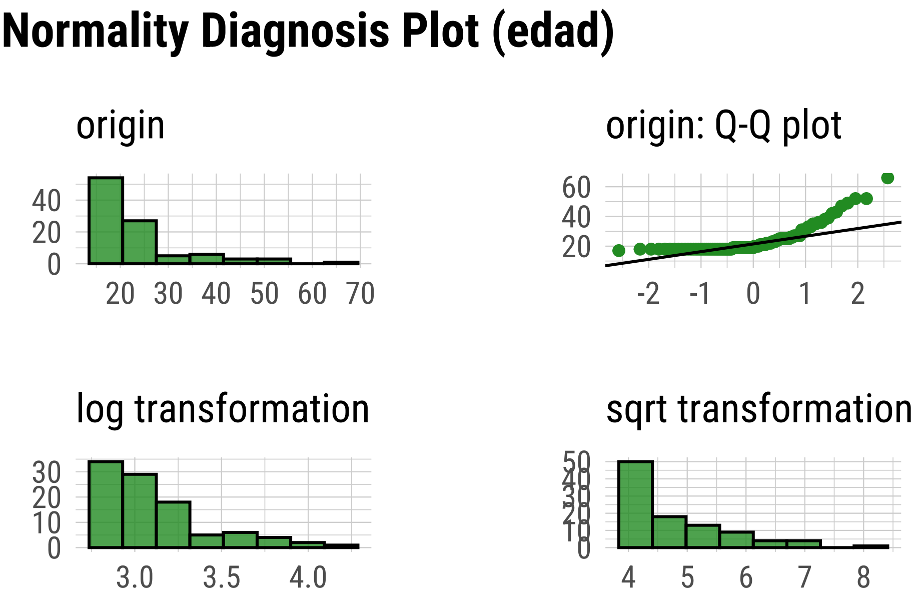
```
datos |>
  summarise(
    asimetria_edad = skewness(edad, na.rm = T),
    asimetria_peso = skewness(peso, na.rm = T)
  )
```

```
# A tibble: 1 x 2
  asimetria_edad asimetria_peso
           <dbl>          <dbl>
1           2.19          0.122
```

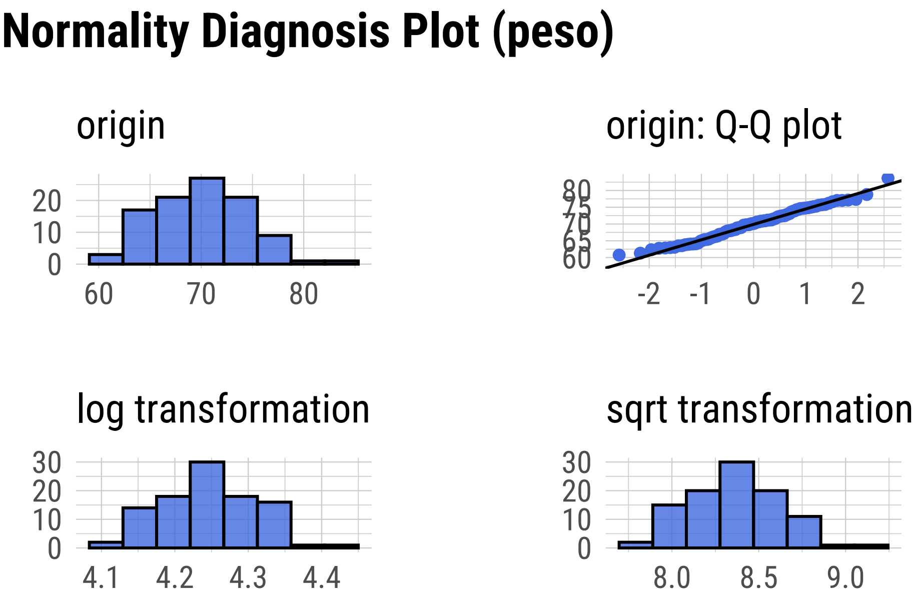
Los valores obtenidos con la función `skewness()` del paquete `moments` nos informan que la distribución de la edad tienen una asimetría positiva (2,2) y que los valores de peso se distribuyen bastante simétricos (0,1).

Estas características de las distribuciones también se pueden ver mediante histogramas o gráficos de densidad:

```
datos |>
  plot_normality(edad, col = "forestgreen")
```



```
datos |>
  plot_normality(peso, col = "royalblue")
```



Los histogramas que se acerquen a la clásica "campana de Gauss" tendrán curtosis y asimetrías alrededor del valor cero.

11.2.3. Pruebas de bondad de ajuste

Una prueba de bondad de ajuste permite testear la hipótesis de que una variable aleatoria sigue cierta distribución de probabilidad y se utiliza en situaciones donde se requiere comparar una distribución observada con una teórica o hipotética.

El mecanismo es idéntico a cualquier test de hipótesis salvo que aquí esperamos no descartar la hipótesis nula de igualdad, por lo que obtener valores *p* de probabilidad mayores a 0,05 es signo de que la distribución de la variable analizada se ajusta.

A continuación, presentaremos los test de hipótesis más utilizados para analizar normalidad.

Test de Shapiro-Wilk

Lleva el nombre de sus autores (Samuel Shapiro y Martin Wilk) y es usado preferentemente para muestras de hasta 50 observaciones.

La función se encuentra desarrollada en el paquete `stats` y se llama `shapiro.test()`:

```
shapiro.test(datos$edad)
```

```

	Shapiro-Wilk normality test

data:  datos$edad
W = 0.69517, p-value = 4.912e-13
```

```r
shapiro.test(datos$peso)
```

```

	Shapiro-Wilk normality test

data:  datos$peso
W = 0.98615, p-value = 0.383
```

Interpretación: Siendo la hipótesis nula que la población está distribuida normalmente, si el p-valor es menor a α (nivel de significancia, convencionalmente un 0,05) entonces la hipótesis nula es rechazada (se concluye que los datos no provienen de una distribución normal). Si el p-valor es mayor a α, se concluye que no se puede rechazar dicha hipótesis.

En función de esta interpretación (que es común a todos los test de hipótesis de normalidad), podemos decir que la distribución de la variable **edad** *no se ajusta a la normal* y no podemos rechazar que la distribución de la variable **peso** se ajuste.

Test de Kolmogorov-Smirnov

El test de **Kolmogorov-Smirnov** permite estudiar si una muestra procede de una población con una determinada distribución que no está limitado únicamente a la distribución normal.

El test asume que se conoce la media y varianza poblacional, lo que en la mayoría de los casos no es posible. Para resolver este problema, se realizó una modificación conocida como test Lilliefors.

Test de Lilliefors

El test de **Lilliefors** asume que la media y varianza son desconocidas y está especialmente desarrollado para contrastar la normalidad.

Es la alternativa al test de Shapiro-Wilk cuando el número de observaciones es mayor de 50.

La función `lillie.test()` del paquete `nortest` (Gross y Ligges 2015) permite aplicarlo:

```r
lillie.test(datos$edad)
```

```

	Lilliefors (Kolmogorov-Smirnov) normality test

data:  datos$edad
D = 0.24892, p-value < 2.2e-16
```

```r
lillie.test(datos$peso)
```

```

	Lilliefors (Kolmogorov-Smirnov) normality test

data:  datos$peso
D = 0.049534, p-value = 0.7905
```

Los resultados son coincidentes con los obtenidos anteriormente.

Test de D'agostino

Esta prueba se basa en las transformaciones de la curtosis y la asimetría de la muestra, y solo tiene poder frente a las alternativas de que la distribución sea sesgada.

El paquete `moments` la tiene implementada en `agostino.test()`:

```r
agostino.test(datos$edad)
```

```

	D'Agostino skewness test

data:  datos$edad
skew = 2.1947, z = 6.3465, p-value = 2.203e-10
alternative hypothesis: data have a skewness
```

```r
agostino.test(datos$peso)
```

```

	D'Agostino skewness test

data:  datos$peso
skew = 0.12160, z = 0.52683, p-value = 0.5983
alternative hypothesis: data have a skewness
```

Los resultados coinciden con la observación de asimetría que efectuamos con los métodos analíticos, confirmando que la variable `edad` *no se ajusta* a una curva simétrica y la variable `peso` si lo hace.

Cuando estos test se emplean con la finalidad de verificar las condiciones de métodos paramétricos es importante tener en cuenta que, al tratarse de valores probabilidad, cuanto mayor sea el tamaño de la muestra más poder estadístico tienen y más fácil es encontrar evidencias en contra de la hipótesis nula de normalidad.

Por otra parte, cuanto mayor sea el tamaño de la muestra, menos sensibles son los métodos paramétricos a la falta de normalidad. Por esta razón, es importante no basar las conclusiones únicamente en los resultados de los test, sino también considerar los otros métodos (gráfico y analítico) y no olvidar el tamaño de la muestra.

11.3. Homocedasticidad

La homogeneidad de varianzas es un supuesto que considera constante la varianza en los distintos grupos que queremos comparar.

Esta homogeneidad es condición necesaria antes de aplicar algunos test de hipótesis de comparaciones o bien para aplicar correcciones mediante los argumentos de las funciones de R.

Existen diferentes test de bondad de ajuste que permiten evaluar la distribución de la varianza. Todos ellos consideran como H_0 que la varianza es igual entre los grupos y como H_1 que no lo es.

La diferencia entre ellos es el estadístico de centralidad que utilizan:

- Media de la varianza: son los más potentes pero se aplican en distribuciones que se aproximan a la normal.
- Mediana de la varianza: son menos potentes pero consiguen mejores resultados en distribuciones asimétricas.

11.3.1. F-test

Este test es un contraste de la razón de varianzas, mediante el estadístico F que sigue una distribución F-Snedecor.

Se utiliza cuando las distribuciones se aproximan a la “normal” y en R base se la encuentra en la función `var.test()` que permite utilizar la sintaxis de fórmula:

```
variable_cuantitativa ~ variable_categórica_grupos
```

Por ejemplo:

```
var.test(formula = peso ~ sexo, data = datos)
```

```

    F test to compare two variances

data:  peso by sexo
F = 0.67914, num df = 50, denom df = 48, p-value = 0.1781
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.384712 1.194943
sample estimates:
ratio of variances 
         0.6791414 
```

Comparamos las varianzas de la variable peso entre el grupo de mujeres y hombres. El valor p del test indica que no podemos descartar la igualdad de varianzas entre los grupos (H_0) o lo que es lo mismo el test no encuentra diferencias significativas entre las varianzas de los dos grupos.

11.3.2. Test de Bartlett

Este test se puede utilizar como alternativa al F-test, sobre todo porque nos permite aplicarlo cuando tenemos más de 2 grupos de comparación. Al igual que el anterior es sensible a las desviaciones de la normalidad.

La función en R base es `bartlett.test()` y también se pueden usar argumentos tipo fórmula:

```
bartlett.test(formula = peso ~ sexo, data = datos)
```

```


	Bartlett test of homogeneity of variances

data:  peso by sexo
Bartlett's K-squared = 1.8082, df = 1, p-value = 0.1787
```

El resultado es coincidente con el mostrado por `var.test()`. No se encuentran diferencias significativas entres las varianzas de los pesos en los dos grupos (Mujer - Varon)

11.3.3. Test de Levene

El test de Levene sirve para comparar la varianza de 2 o más grupos pero además permite elegir distintos estadísticos de tendencia central. Por lo tanto, la podemos adaptar a distribuciones alejadas de la normalidad seleccionando por ejemplo la mediana.

La función `leveneTest()` se encuentra disponible en el paquete `car`. La vemos aplicada sobre `peso` para los diferentes grupos de `sexo` y utilizando la **media** como estadístico de centralidad, dado que la distribución de `peso` se aproxima a la normal.

```
leveneTest(y = peso ~ sexo, data = datos, center = "mean")
```

```
Levene's Test for Homogeneity of Variance (center = "mean")
      Df F value Pr(>F)
group  1       2 0.1605
      98               
```

La conclusión es la misma que la encontrada anteriormente.

Ahora vamos aplicarla sobre la variable `edad`, de la que habíamos descartado “normalidad”. Lo hacemos usando el argumento `center` con `"mean"` (media) y con `"median"` (mediana).

```
leveneTest(y = edad ~ sexo, data = datos, center = "mean")
```

```
Levene's Test for Homogeneity of Variance (center = "mean")
      Df F value Pr(>F)  
group  1  4.6784  0.033 *
      97                 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
leveneTest(y = edad ~ sexo, data = datos, center = "median")
```

```
Levene's Test for Homogeneity of Variance (center = "median")
      Df F value Pr(>F)
group  1  2.4413 0.1214
      97
```

Los resultados son diferentes. Mientras con el centrado en la media nos da un *p* valor significativo menor a 0,05 con el centrado en la mediana no nos permite descartar homocedasticidad.

Observamos aquí las distorsiones sobre la media y las formas paramétricas que devienen de distribuciones asimétricas y alejadas de la curva normal. El código correcto para este caso (variable `edad`) es usar el centrado en la mediana (`center = "median"`).

11.4. Ejemplo práctico en R

La contaminación por plásticos ha alcanzado incluso las zonas más remotas del océano, fragmentándose en partículas microscópicas conocidas como microplásticos (< 5 mm). Debido a su capacidad para filtrar grandes volúmenes de agua, los bivalvos como el mejillón (*Mytilus edulis*) actúan como bioindicadores de la salud ambiental y, al mismo tiempo, como una vía directa de transferencia de contaminantes hacia los seres humanos.

En este ejercicio, analizaremos datos basados en la investigación pionera de Van Cauwenberghe y Janssen (2014). Evaluaremos la carga promedio de microplásticos en muestras destinadas al consumo y estimaremos la magnitud de la proporción de bivalvos que superan los límites de seguridad.

Los valores de concentración de microplásticos (medidos en partículas/g) hallados en estos bivalvos estudiados se encuentran almacenadas en el archivo **bivalvos.txt** y se nos pide realizar los siguientes items:

a. Lea el archivo y explore su contenido
b. Describa estadísticamente las mediciones
c. ¿Estos datos parecen tomados de una población de distribución normal?, ¿por qué?
d. ¿Cuáles son los intervalos de confianza de 90 y 95 % para la concentración media de microplasticos por el método frecuentista tradicional (fórmulas)?
e. Construya la variable **umbral_excedido** tomando el valor de corte hipotético en 0,5 partículas/g (si concentración es menor o igual a 0,5 el umbral_excedido valdrá “No” y si es mayor valdrá “Si”)
f. ¿Cuál es el intervalo de confianza del 95 % de la proporcion de bivalvos con riesgo potencial para el consumo humano?

Veamos como llevamos adelante estos pasos mediante el lenguaje R:

11.4.1. Lectura y exploración inicial

```
# Activamos paquetes necesarios y conocidos
library(tidyverse)
library(dlookr)

# Leemos la tabla de datos
bivalvos <- read_csv2("datos/bivalvos.txt")
```

Visualizamos la estructura de la tabla de datos

```
glimpse(bivalvos)
```

```
Rows: 50
Columns: 2
$ id            <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
$ concentracion <dbl> 0.37, 0.42, 0.68, 0.46, 0.47, 0.71, 0.52, 0.26, 0.35, 0.~
```

Tenemos una tabla con 50 observaciones (mediciones en bivalvos) y la variable de interés se llama concentracion (nos informan que está expresada como partículas/g de tejido húmedo)

11.4.2. Describa estadísticamente las mediciones

Con la función `describe()` de **dlookr** podemos obtener los estadísticos de esta variable cuantitativa continua.

```
bivalvos |>
  describe(concentracion)
```

```
# A tibble: 1 x 26
  described_variables     n    na  mean    sd se_mean   IQR skewness kurtosis
  <chr>                 <int> <int> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 concentracion            50     0 0.456 0.138  0.0196 0.188    0.197   -0.299
# i 17 more variables: p00 <dbl>, p01 <dbl>, p05 <dbl>, p10 <dbl>, p20 <dbl>,
#   p25 <dbl>, p30 <dbl>, p40 <dbl>, p50 <dbl>, p60 <dbl>, p70 <dbl>,
#   p75 <dbl>, p80 <dbl>, p90 <dbl>, p95 <dbl>, p99 <dbl>, p100 <dbl>
```

La media (0,4556 partículas/g) y la mediana (0,44 partículas/g) son cercanas. El desvío estandar es de 0,138 partículas/g.

11.4.3. Test de distribución normal

Aplicaremos el test de normalidad de Shapiro-Wilk (forma analítica)

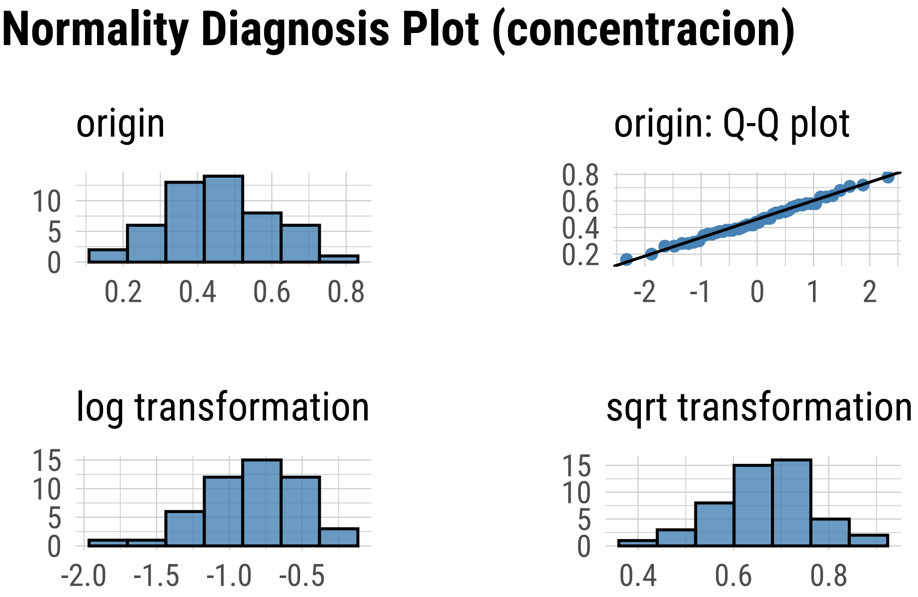
```
shapiro.test(bivalvos$concentracion)
```

```

	Shapiro-Wilk normality test

data:  bivalvos$concentracion
W = 0.989, p-value = 0.92
```

Un p-valor mayor a 0,05 habla de un ajuste a la curva normal (es su hipotesis nula de igualdad).

Complementamos visualizandolo graficamente con un Q-QPlot.

```
bivalvos |>
  plot_normality(concentracion)
```



Los puntos se desarrollan "alrededor" a la recta de normalidad.

Con estos dos elementos podemos confirmar que la distribución se aproxima a la "Normal" y por lo tanto es confiable hacer uso de métodos paramétricos.

Si bien con ayuda del Teorema del Límite Central (TCL) sabemos que si el tamaño de la muestra es suficientemente grande, la distribución de la media muestral (o del estadístico) tiende a una distribución normal, independientemente de la forma que tengan los datos originales. En este caso, la definición de "grande" depende de varios factores, pero generalmente se considera que 30 es un buen umbral.

En la práctica, cuando trabajamos con variables continuas:

- Muestras pequeñas ($n < 30$): Si podemos asumir que la población original es aproximadamente normal, utilizamos la distribución t de Student con $n-1$ grados de libertad. Esta distribución es más 'ancha' en las colas, lo que penaliza nuestra incertidumbre por tener pocos datos.

- Muestras grandes ($n \geq 30$): Por el TLC, la distribución del estadístico tiende a la normalidad (z). Sin embargo, dado que la distribución t converge a la z a medida que el tamaño de muestra aumenta, es una práctica estándar (y simplificadora) utilizar la distribución t en toda la escala.Por eso, funciones en R como `t.test()` se usan universalmente: para un n pequeño aplican la corrección necesaria, y para un n grande los resultados son virtualmente idénticos a los de una distribución z.

Es una buena oportunidad para mencionar que el número 30 es una convención histórica (una regla empírica). Actualmente, a veces con $n = 30$ y datos muy sesgados, el TLC aún no ha "actuado" lo suficiente y hay que recurrir a métodos no paramétricos donde obtenemos IC más robustos y honestos que confiar ciegamente en la distribución t".

11.4.4. Calculo de IC paramétrica

Estamos frente al calculo de IC de una muestra de n = 50 (mayor a 30 observaciones) y que cumple con el supuesto de normalidad, por lo tanto deciamos que podemos aplicar un método paramétrico basado en la distribución t de Student.

Podríamos hacer las operaciones y cálculos individuales utilizando las funciones que R tiene sobre esta distribución (familia TDist: `qt()`, etc), pero la idea de este ejercicio es mostrarles la forma operativa más sencilla de llevarlo a cabo.

Función t.test()

La función `t.test()`, incluída en el paquete stats de R base, sirve apara ejecutar test de hipótesis de medias de una y dos poblaciones (independientes o pareadas) pero además nos calcula automáticamente el intervalo de confianza requerido, basado en la distribución t de Student.

Para este ejemplo debemos utilizar como argumentos:

- la variable de interés que contiene los valores de la muestra
- el nivel de confianza con el que necesitamos el IC

```
t.test(x = bivalvos$concentracion, conf.level = 0.95)
```

```

	One Sample t-test

data:  bivalvos$concentracion
t = 23.296, df = 49, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 0.4162995 0.4949005
sample estimates:
mean of x 
   0.4556 
```

En la lista de resultados devuelta debemos observar los valores que figuran debajo de “95 percent confidence interval:”. En este caso, el resultado es de una media de 0,4556 IC95 %: 0,4163-0,4949 partículas/g

Si cambiamos el argumento conf.level a 0.90 obtendremos el IC 90 %:

```
t.test(x = bivalvos$concentracion, conf.level = 0.90)
```

```

	One Sample t-test

data:  bivalvos$concentracion
t = 23.296, df = 49, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
90 percent confidence interval:
 0.4228123 0.4883877
sample estimates:
mean of x 
   0.4556 
```

Este va de 0,4228 a 0,4884 partículas/g, lógicamente más estrecho que el anterior porque redujimos el intervalo de confianza.

La redacción correcta sería *“con un nivel de confianza del 90 %, se estima que el verdadero valor de la media poblacional de la concentración de microplásticos en bivalvos se encuentra entre 0,4228 y 0,4884 partículas/g.”*

Técnicamente las funciones estadísticas de R base devuelven generalmente resultados en formato lista y reciben su entrada con formato vector o bien sintaxis fórmula. Ese formato no cumple con ser ordenado como el dataframe. Dado que la filosofía de trabajo del tidyverse (y por lo tanto todos sus paquetes que pertenecen al ecosistema) tienen como premisa recibir y devolver un dataframe, lo que permite usar tambien las características tuberías, Kassambara (2026b) propuso el paquete **rstatix**.

La idea central del paquete es ofrecer funciones que hacen lo mismo que las de R base pero en lugar de devolver una lista de valores como resultado lo haga mediante un dataframe.

Veamos su uso:

```r
library(rstatix)

bivalvos |>
  t_test(concentracion ~ 1, conf.level = 0.95, detailed = T) |>
  select(estimate, conf.low, conf.high)
```

```
# A tibble: 1 x 3
  estimate conf.low conf.high
     <dbl>    <dbl>     <dbl>
1    0.456    0.416     0.495
```

11.4.5. Construcción de umbral_excedido

Debemos proceder ahora a construir la variable categórica **umbral_excedido** con valores Si y No en función del valor de corte hipotético de 0,5.

Podemos hacerlo simplemente con el condicional `if_else()`.

```r
bivalvos <- bivalvos |>
  mutate(umbral_excedido = if_else(concentracion > 0.5, "Si", "No"))

bivalvos |>
  count(umbral_excedido)
```

```
# A tibble: 2 x 2
  umbral_excedido     n
  <chr>           <int>
1 No                 31
2 Si                 19
```

Observamos que 19 de los 50 bivalvos de nuestra muestra representa un riesgo sanitario para los consumidores.

```
bivalvos |>
  count(umbral_excedido) |>
  mutate(porcentaje = n / sum(n) * 100)
```

```
# A tibble: 2 x 3
  umbral_excedido     n porcentaje
  <chr>           <int>      <dbl>
1 No                 31         62
2 Si                 19         38
```

Eso representa el 38 % de las observaciones que tenemos.

11.4.6. IC 95 % de la proporción

En este punto queremos construir el intervalo de confianza del 95 % que acompañe a la estimación puntual anterior.

Tenemos dos posibilidades que están implementadas en R base: la función `prop.test()` o `binom.test()`.

`prop.test()` utiliza la aproximación de la distribución Normal (puntuación z) a la distribución Binomial. Podríamos decir que es el análogo categórico del `t.test()`. Los elementos necesarios son: **x** número de éxitos y **n** que es tamaño de la muestra.

Se usa con muestras “grandes” ($n > 30$). Pero si en cambio el tamaño de la muestra es pequeño, la proporción es muy cercana a 0 o a 1, o quieres una precisión absoluta y no quieres depender de aproximaciones matemáticas ejecutamos `binom.test()`.

La función aplica la prueba exacta de Clopper-Pearson, donde en **x** definimos cuántos bivalvos superaron el umbral y en **n** el tamaño total de la muestra.

Ejecutamos las dos posibilidades:

```
prop.test(x = 19, n = 50)
```

```

	1-sample proportions test with continuity correction

data:  19 out of 50, null probability 0.5
X-squared = 2.42, df = 1, p-value = 0.1198
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.2499804 0.5283672
sample estimates:
   p 
0.38 
```

```
binom.test(x = 19, n = 50)
```

```
    Exact binomial test

data:  19 and 50
number of successes = 19, number of trials = 50, p-value = 0.1189
alternative hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.2465011 0.5282508
sample estimates:
probability of success
                  0.38
```

Los resultados son muy similares, dado que cumplimos con los supuestos necesarios para usar la aproximación del test de proporciones.

Entonces podemos decir con un nivel de confianza del 95 %, que estimamos que la proporción poblacional de bivalvos que exceden el límite de seguridad sanitaria se encuentra entre 0,25 y 0,53.

Las dos funciones estan representadas en **rstatix**, con `prop_test()` y `binom_test()` respectivamente.

Apredimos con este ejercicio que el lenguaje R siempre ofrece múltiples caminos, pero nuestro criterio es el que decide cuál usar y reportar.

12. Selección de la muestra

12.1. Introducción

Toda investigación epidemiológica requiere, en principio, la selección de una muestra representativa de la población objeto de estudio. Esto implica:

- Determinar el tamaño de la muestra.
- Definir cómo se seleccionarán los sujetos (es decir, las unidades de análisis).
- Calcular la incertidumbre que introduce este proceso de muestreo en nuestras estimaciones.

El tamaño de la muestra se establece en la fase de diseño del estudio, habitualmente cuando se redacta el protocolo de investigación. Es importante considerar que este tamaño es, en cierto sentido, orientador, ya que su cálculo depende de factores que en ocasiones pueden ser subjetivos o estar limitados por la información previa disponible. Además, las exigencias varían según el objetivo del estudio. Por ejemplo:

- En un estudio transversal, donde se estima una prevalencia, se puede aceptar una precisión menor.
- En un estudio experimental, donde se compara la eficacia de un fármaco frente a un placebo, el tamaño de la muestra debe ser suficiente para detectar una “diferencia significativa” entre los grupos.

Cuando el objetivo de la investigación incluye la estimación de un intervalo de confianza (IC) para alguna variable, se busca que dicho IC tenga alta confiabilidad y sea lo más estrecho posible.

En el caso de los estudios de corte transversal, que se orientan a estimar una proporción, se formulan preguntas como:
— ¿Cuál es la prevalencia, proporción o porcentaje del fenómeno en estudio?

Al tomar una muestra para estimar lo que sucede en la realidad, debemos aceptar que la medición obtenida reflejará la situación poblacional, pero siempre con cierto grado de variabilidad (o error). La **precisión** es ese grado de variabilidad aceptable y se expresa a través de los intervalos de confianza, que definen los límites inferior y superior dentro de los cuales se espera encontrar el verdadero valor del parámetro.

12.2. Cálculo del tamaño muestral

Como se mencionó anteriormente, se requiere conocer algunos datos para el cálculo del tamaño de muestra requerido, entre los que destacan:

- **La proporción esperada del evento:** Aunque resulte paradójico, ya que es el parámetro que se desea estimar, el investigador suele contar con información previa que permite delimitar sus posibles valores. Si no fuera el caso, se utiliza el peor escenario, es decir, una proporción del 50 %, ya que este valor maximiza el producto $p(1-p)$ y, por ende, exige el mayor tamaño muestral. De esta forma, muchas veces se recurre a esto para tener al menos una idea del “límite superior” que estaríamos manejando.

- **El nivel de confianza deseado:** Habitualmente se emplea un 95 % (lo que equivale a un α de 0.05). Este valor indica la probabilidad de que el intervalo calculado contenga el parámetro poblacional verdadero. A mayor nivel de confianza (es decir, menor α), mayor es el coeficiente de confiabilidad y, por ende, mayor el tamaño muestral requerido.
- **La precisión (δ) deseada:** Se refiere a la amplitud aceptable del intervalo de confianza. Cuanta más precisión se exija (es decir, un intervalo más estrecho), mayor será el tamaño de la muestra necesario.

La fórmula para el cálculo es la siguiente:

$$N = \frac{p.q.(Z\alpha)^2}{\delta^2}$$

Donde: - N es el tamaño muestra requerido. - p es la proporción esperada de sujetos portadores del evento. - $q = 1 - p$ es la proporción complementaria (sujetos que no tienen la variable en estudio). - δ es la precisión o magnitud de error aceptable. - $Z\alpha$ es el valor crítico de la distribución normal asociado al nivel de significancia. Se obtiene de tablas de distribución normal de probabilidades y habitualmente se utiliza un valor α de 0.05, al que le corresponde un valor Z de 1.96

Cuando se conoce el tamaño de la población, es necesario aplicar una corrección por población finita:

$$N = \frac{n^1}{1 + \frac{n^1}{Poblacin}}$$

donde n^1 es el N calculado con la ecuación anterior.

Es fundamental entender que el tamaño de la muestra dependerá de p de α y de la precisión deseada, más que de la complejidad de la fórmula en sí. Existen numerosos softwares que calculan tamaños de muestra. Algunos están especializados en esta tarea (por ejemplo, **EpiDat**, **EpiInfo**, **OpenEp**i), mientras que otros lo incluyen como una opción dentro de un paquete más amplio de análisis. En el lenguaje R, funciones como `power.prop.test()` (para comparación de proporciones) y `power.t.test()` (para pruebas de medias basadas en la distribución t de Student) están incluidas en el paquete `stats`. Otros paquetes útiles incluyen `samplingbook` (Manitz et al. 2021), `samplesize`, `pwr` (Champely 2020) y `epiR` (Stevenson y Sergeant 2026).

A continuación, se presenta un ejemplo práctico utilizando la función `sample.size.prop()` del paquete `samplingbook`. Supongamos que queremos estimar la prevalencia de diabetes en una ciudad de 600,000 habitantes, donde estudios previos sugieren una prevalencia cercana al 10.2 %. El objetivo es determinar el tamaño de la muestra en un muestreo aleatorio simple.

Comenzamos cargando el paquete requerido:

```r
library(samplingbook)
```

Calculamos el tamaño muestral en base a los siguientes argumentos:

- `e`: número positivo que especifica la precisión que es la mitad del ancho del intervalo de confianza
- `p:` proporción esperada de eventos con dominio entre los valores 0 y 1.
- `N`: número entero positivo para el tamaño de la población. El valor predeterminado es `Inf`, lo que significa que los cálculos se realizan sin *corrección de población finita*.
- `level`: nivel de confianza para los intervalos de confianza. El valor predeterminado es 95 %.

```
sample.size.prop(e = 0.02, P = 0.102, N = 600000, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
With finite population correction: N=6e+05, precision e=0.02 and expected proportion P=0.102

Sample size needed: 879
```

El resultado, tras aplicar la corrección por población finita, indica que se requieren 879 sujetos en la muestra. Por simplicidad, es habitual considerar infinita a la población objetivo, ya que hacerlo así minimiza los riesgos estadísticos:

```
sample.size.prop(e = 0.02, P = 0.102, N = Inf, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
Without finite population correction: N=Inf, precision e=0.02 and expected proportion P=0.102

Sample size needed: 880
```

Si se considera una población infinita, el tamaño muestral calculado es de 880 personas, apenas una unidad más, lo que demuestra que el ajuste por población finita en este caso es mínimo. En cambio, si no se conoce o no se confía en el valor de prevalencia y se opta por el peor escenario ($p = 0.5$):

```
sample.size.prop(e = 0.02, P = 0.5, N = 600000, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
With finite population correction: N=6e+05, precision e=0.02 and expected proportion P=0.5

Sample size needed: 2392
```

En este caso, el tamaño muestral aumenta considerablemente hasta 2392 unidades, lo que refleja un enfoque más conservador.

Además del cálculo del tamaño muestral, es esencial planificar la forma en que se seleccionarán los individuos. En estudios de corte transversal se debe realizar un muestreo probabilístico, ya que la técnica de muestreo influirá en los factores de expansión necesarios para generalizar los resultados a la población. Aunque el proceso de muestreo no se detalla en este curso, es vital considerarlo para garantizar la validez externa de la investigación.

12.3. Conceptos generales sobre muestreo

Un muestreo se considera **probabilístico** cuando se cumplen dos condiciones fundamentales:

- Es posible conocer de antemano la probabilidad de selección que tiene cada elemento de la población.
- Esta probabilidad es mayor que cero para todos los elementos.

Solo con un muestreo probabilístico se puede medir el grado de precisión de las estimaciones realizadas, ya que se conocen las probabilidades de inclusión. Algunos de los métodos más usados son: Muestreo aleatorio simple, muestreo sistemático en fases, Muestreo aleatorio estratificado, muestreo por conglomerados, etc.

12.3.1. Muestreo aleatorio simple (MAS)

En el MAS, cada elemento de la población tiene la misma probabilidad de ser seleccionado (es decir, es un método **equiprobabilístico**). La probabilidad de inclusión se expresa como:

$$P = \frac{n}{N}$$

donde:

- n es el tamaño de la muestra,
- N es el tamaño de la población.

Este método requiere contar con un listado completo de la población y se utiliza principalmente en poblaciones cerradas (por ejemplo, escuelas, cárceles, hospitales o muestras de laboratorio), donde los elementos pueden identificarse y numerarse fácilmente.

12.3.2. Muestreo sistemático (MS)

El muestreo sistemático es un procedimiento alternativo al MAS, adecuado para la selección de muestras equiprobabilísticas de poblaciones organizadas según algún orden conocido. Sus ventajas incluyen no requerir necesariamente un listado completo de todas las unidades, lo que resulta útil, por ejemplo, para seleccionar a los usuarios que llegan a un servicio hospitalario durante un período determinado.

El procedimiento consiste en:

1. Asignar a cada elemento de la población un número del 1 a N.
2. Calcular el intervalo de selección:

$$k = \frac{N}{n}$$

3. Seleccionar aleatoriamente un número r entre 1 y k.
4. Elegir los elementos que ocupan las posiciones: $r, r+k, r+2k, ...$, es decir, la secuencia $r, r+jk$ con $j = 0, 1, 2, ...$.

La probabilidad de ser seleccionado es igual que en el MAS

$$P = \frac{n}{N}$$

12.3.3. Muestreo aleatorio estratificado (MAE)

Este método se emplea cuando la población puede dividirse en subgrupos o **estratos** que difieren en las características de interés (por ejemplo, edad, sexo o nivel socioeconómico). La idea es asegurar que cada estrato esté representado en la muestra. El proceso habitual consiste en:

- Elaborar listas separadas para cada estrato.
- Seleccionar una submuestra de cada uno de ellos.

La probabilidad de selección en cada estrato es:

$$P\, estrato = \frac{n\, estrato}{N\, estrato}$$

Si se aplica un muestreo proporcional, el tamaño de la muestra en cada estrato se calcula como:

$$n\, estrato = n\frac{N\, estrato}{N}$$

El objetivo es obtener una muestra cuya variabilidad interna se asemeje a la de la población, logrando que los estratos sean internamente homogéneos y heterogéneos entre sí.

12.3.4. Muestreo por conglomerados monoetápico

El muestreo por conglomerados es ideal cuando la población es difícil de delimitar o está muy dispersa. Se divide la población en **conglomerados** (o grupos) que actúan como unidades de primera etapa (UPE) y no deben solaparse.

La población se divide en grupos (por ejemplo, áreas geográficas, instituciones o servicios). Se selecciona una muestra de n conglomerados y se incluyen **todos** las unidades de análisis de los conglomerados seleccionados. El tamaño del conglomerado se refiere al número de unidades elementales que contiene, pudiendo ser de igual o distinto tamaño. Los conglomerados deberían ser lo más heterogéneos dentro de ellos y lo más homogéneos entre ellos.

12.3.5. Muestreo por conglomerados bietápico

Se utiliza cuando existe gran variabilidad en el tamaño de los conglomerados o la población es muy grande. En este caso, después de dividir la población en conglomerados y seleccionar algunos de ellos como UPE, se realiza una subselección (muestreo secundario) de los elementos dentro de los conglomerados elegidos.

Existen más posibilidades, que no abordaremos aquí, pero sí nos detendremos en discutir qué implicaciones tiene el muestreo en el proceso de estimación.

	Población	Muestra
Tamaño total	966	120
Número de conglomerados	21	10

12.3.6. Implicaciones en la Estimación y Ponderación

En métodos equiprobabilísticos (como el MAS o el MS), cada elemento tiene la misma probabilidad de selección. Sin embargo, en diseños muestrales más complejos (como el muestreo estratificado o por conglomerados), las probabilidades de selección pueden variar entre individuos.

En estos casos, la probabilidad total de selección de un individuo es el producto de las probabilidades de selección en cada etapa del muestreo. Una vez conocida esta probabilidad, se calcula el peso (o **factor de expansión**) como su inverso. Este peso se interpreta como el número de individuos en la población que representa cada individuo de la muestra.

> El factor de expansión se interpreta como *la cantidad de personas en la población, que representa una persona en la muestra*.

El factor de expansión se utiliza para extrapolar los resultados obtenidos en la muestra a la población total. Por ejemplo, para estimar un total poblacional para una variable, se pondera el valor medido en cada individuo por su factor de expansión y se suman dichos valores.

12.4. Ejemplo práctico

Supongamos que queremos estimar la prevalencia de sobrepeso/obesidad en estudiantes de enseñanza media de una localidad. Se tiene la siguiente información:

- **Población total de alumnos:** 966
- **Número de escuelas (conglomerados):** 21
- **Tamaño de la muestra global (ya calculado):** 120 alumnos

Si se realizara un **muestreo aleatorio simple (MAS)**, se necesitaría disponer del listado completo de los 966 alumnos y, mediante un sorteo, seleccionar 120 individuos. En ese caso, la probabilidad de selección para cada alumno sería:

$$P = \frac{n}{N} = \frac{120}{966} = 0,124$$

Supongamos ahora que decide realizar un muestreo por conglomerados biétapico. Cada escuela se considera un conglomerado; se seleccionan 10 de las 21 escuelas por MAS y, dentro de cada escuela seleccionada, se elige una submuestra de estudiantes también mediante MAS. Se decide que el número de estudiantes elegidos en cada conglomerado sea igual para todos.

Los datos globales quedan de la siguiente forma:

La decisión de cuántos conglomerados forman parte de la muestra, es una decisión del investigador: a veces puede ser necesario fijar un número, a veces puede ser más conveniente fijar el tamaño de la muestra en cada conglomerado. Por otra parte, puede que decidamos que el número de sujetos seleccionados en cada conglomerado sea igual para todos; que sea una fracción fija o algo a definir.

Conglomerado	Tamaño	Muestra
5.1	48	12
1.3	28	12
2.3	64	12
2.2	43	12
3.1	37	12
5.3	86	12
1.4	37	12
3.3	56	12
2.4	48	12
1.5	33	12
Total	480	120

Dentro de los conglomerados seleccionados, se obtiene la siguiente distribución (en este ejemplo, la suma de los tamaños de los conglomerados seleccionados es 480 y en cada uno se eligen 12 alumnos):

12.4.1. Cálculo de las Probabilidades de Selección y Ponderaciones

Para que un individuo sea seleccionado en este diseño, debe ocurrir primero que su conglomerado sea escogido y, posteriormente, que él sea seleccionado dentro de ese conglomerado. Si asumimos que:

> *"Dos sucesos son independientes si la probabilidad de que ocurran ambos simultáneamente es igual al producto de las probabilidades de que ocurra cada uno de ellos"*.

La probabilidad total de selección es el producto de:

1. La probabilidad de que el conglomerado sea seleccionado:

$$P_{conglomerado} = \frac{10}{21}$$

2. La probabilidad de que un individuo sea seleccionado dentro de su conglomerado:

$$P_{dentro} = \frac{N^{\circ}\ alumnos\ en\ la\ muestra\ del\ conglomerado}{Tamao\ del\ conglomerado}$$

Para los individuos del conglomerado 5.1; la probabilidad de selección será:

$$P_{5.1} = \frac{10}{21} * \frac{12}{48} \approx 0,1190 \quad (11.9\,\%)$$

La ponderación o **factor de expansión** se obtiene como el inverso de la probabilidad de selección:

$$w = \frac{1}{P}$$

Es decir:

Conglomerado	Probabilidad	Ponderación
5.1	11.90	8.40
1.3	20.41	4.90
2.3	8.93	11.20
2.2	13.29	7.53
3.1	15.44	6.48
5.3	6.65	15.05
1.4	15.44	6.48
3.3	10.20	9.80
2.4	11.90	8.40
1.5	17.32	5.78

$$w_{5.1} = \frac{1}{1.190} \approx 8.4$$

Esto significa que cada alumno seleccionado del conglomerado 5.1 representa aproximadamente a 8 alumnos de la población.

Si se realizan los cálculos para cada uno de los conglomerados, se obtiene la siguiente tabla:

Cada individuo seleccionado en el estudio tiene una probabilidad de inclusión que depende tanto de la probabilidad de que su escuela (conglomerado) sea seleccionada como de la probabilidad de que él sea elegido dentro de su escuela. La ponderación resultante (el inverso de esta probabilidad) se utiliza para "expandir" el resultado del estudio, de manera que cada alumno en la muestra represente a un número determinado de alumnos en la población total.

Por ejemplo, en el conglomerado 5.1, cada alumno seleccionado representa aproximadamente a 8 alumnos de la población. De este modo, al calcular la prevalencia de sobrepeso/obesidad en la muestra, se aplican estos factores de expansión para obtener estimaciones que sean representativas de los 966 alumnos de la localidad.

Este proceso es fundamental en estudios basados en muestreos complejos, ya que permite corregir posibles diferencias en las probabilidades de selección y garantizar que las estimaciones sean válidas y precisas para toda la población.

13. Funciones condicionales en R

13.1. Introducción

Continuamos incorporando funciones útiles para el manejo de datos, enfocándonos en la creación de nuevas variables y en la transformación de variables cuantitativas a variables categóricas, ya sean nominales u ordinales.

Algunas de estas operaciones serán necesarias para alcanzar los objetivos del trabajo práctico grupal de esta unidad, y seguramente también se utilizarán en las unidades siguientes.

Como vimos dentro del universo `tidyverse`, para crear nuevas variables a partir de cálculos utilizamos la función-verbo `mutate()`:

```
datos <- datos |>
  mutate(variable_nueva = funcion(var1))
```

Frecuentemente necesitaremos agrupar, resumir o **discretizar variables continuas**, dividiendo su rango en categorías que pueden ser **dicotómicas** o **politómicas**, según los objetivos del análisis.

13.2. Condicional simple: `if_else()`

Para obtener salidas dicotómicas, podemos usar la función condicional `if_else()` del paquete `dplyr`. Esta función es una versión simplificada y segura del clásico condicional `if` que existe en la mayoría de los lenguajes de programación, así como en hojas de cálculo como *Microsoft Excel®* y *Google Sheets®*, entre otras.

Supongamos que tenemos un dataframe llamado `datos`, con una variable cuantitativa denominada `var1`:

```
# Cargar tidyverse
library(tidyverse)

# Datos ejemplo
set.seed(123)

datos <- tibble(
  var1 = sample(1:90, size = 50, replace = TRUE)
)
```

Queremos crear una variable cualitativa dentro de `datos`, llamada `variable_nueva`, cuyos valores estén dados por el cumplimiento de alguna condición de `var1`. Por ejemplo, si los valores de `var1` son mayores a 10, entonces `variable_nueva`, tomará el valor `"mayor a 10"`, en caso contrario, tomará el valor `"menor o igual a 10"`:

```
# Crear variable_nueva
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "mayor a 10",
      false = "menor o igual a 10"
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
     variable_nueva  n percent
         mayor a 10 44    0.88
 menor o igual a 10  6    0.12
```

La función `if_else()` tiene tres argumentos obligatorios:

- `condition`: la condición a evaluar.
- `true`: valor que tomará la nueva variable si se cumple la condición.
- `false`: valor que tomará la nueva variable si no se cumple la condición.

Este proceso se conoce como **dicotomización** de una variable, ya que el resultado posible consta de solo dos categorías.

Los valores de salida pueden ser de distintos tipos (carácter, numérico o lógico). Sin embargo, cuando discretizamos una variable cuantitativa, habitualmente generamos una variable cualitativa de tipo **ordinal**, con categorías expresadas como texto (tipo `character`).

Ahora bien, al ser ordinal estas categorías de la `variable_nueva` deben "ordenarse" en la forma de los valores de la variable, pero el lenguaje R no sabe con que estamos trabajando y respeta siempre el ordenamiento alfanumérico. Por lo tanto, en este ejemplo las categorías se van a estar ordenando al reves del orden numérico natural (de menor a mayor).

La categoría `"mayor a 10"` se ordena alfabéticamente antes de `"menor o igual a 10"`, porque luego del empate de las letras `m`, le siguen la `a` en el primer caso y la `e` en el segundo.

Para ordenar estas categorías debemos transformar la variable de `character` a `factor`. Esto se puede hacer en un solo paso dentro del `mutate()`:

```
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "mayor a 10",
      false = "menor o igual a 10"
    ),

    variable_nueva = factor(
      variable_nueva,
      levels = c("menor o igual a 10", "mayor a 10")
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
     variable_nueva  n percent
 menor o igual a 10  6    0.12
         mayor a 10 44    0.88
```

Otra forma más artesanal, igualmente válido, es "forzar" el ordenamiento con las categorías así:

```
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "2.mayor a 10",
      false = "1.menor o igual a 10"
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
       variable_nueva  n percent
 1.menor o igual a 10  6    0.12
         2.mayor a 10 44    0.88
```

Aquí agregamos números iniciales a las etiquetas de las categorías para darle el orden que deseamos, sin necesidad de convertir a factor.

13.2.1. Función `cut_interval()`

El ecosistema `tidyverse` ofrece la función `cut_interval()` para la creación de intervalos regulares.

Es una adaptación de la función `cut()` de R base para *tidy data* y sus argumentos son similares. Volviendo al ejemplo anterior:

```
datos <- datos |>
  mutate(
    grupo_var = cut_interval(
      x = var1,
      length = 10,
      right = TRUE,
      ordered_result = TRUE
    )
  )
```

```
# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var n percent
    [0,10] 6    0.12
   (10,20] 4    0.08
   (20,30] 7    0.14
   (30,40] 6    0.12
   (40,50] 6    0.12
   (50,60] 5    0.10
   (60,70] 4    0.08
   (70,80] 7    0.14
   (80,90] 5    0.10
```

Los argumentos función son:

- `x`: conjunto de datos numéricos de entrada (obligatorio).
- `length`: longitud de cada intervalo regular (obligatorio).
- `n`: cantidad de intervalos a crear (obligatorio, omitir si se definió `length`).
- `right`: indica si los intervalos son cerrados a la derecha o viceversa (valor por defecto `TRUE`, intervalos cerrados a la derecha).
- `labels`: etiquetas de los intervalos automáticas o numéricas (por defecto intervalos matemáticos).
- `ordered_result`: determina si el resultado es un factor ordenado. Por defecto vale `FALSE` (la salida es tipo carácter).

No es necesario definir los argumentos opcionales siempre y cuando los valores por defecto sirvan para nuestra tarea.

13.3. Función `case_when()`

Cuando las condiciones no son simples, es decir, el resultado no es dicotómico y además los intervalos son irregulares, utilizamos la función `case_when()` que es una vectorización de la función `if_else()`.

Supongamos que queremos agrupar la variable cuantitativa de números enteros (`var1`) en tres grupos irregulares:

```
datos <- datos |>
  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3"
    )
  )
```

```
# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 23    0.46
   Grupo 3 15    0.30
```

Existe una condición por cada grupo creado, como si fuese un `if_else()` donde el valor declarado siempre es el verdadero. Se utilizan operadores de comparación como mayor (>), menor (<) y/o igual (==) y conectores lógicos como `&` (AND), | (NOT) e `%in%`. En cada línea va una virgulilla similar a la usada en la sintaxis formula (~) y luego la etiqueta que tomarán las observaciones que cumplan con esa condición en la nueva variable (`grupo_var`).

Esta evaluación es secuencial y su funcionamiento provoca que el usuario del lenguaje tenga el control de lo que esta sucediendo, por lo que cualquier mala definición de las condiciones puede provocar resultados incorrectos.

Si incorporamos el argumento `.default` podemos indicar que valor toma si no se cumple ninguna de las condiciones anteriores.

Por ejemplo, podríamos tener algún valor perdido (`NA`) en `var1` y queremos que la variable `grupo_var` etiquete esos valores perdidos como `"Sin dato"`.

```
datos <- datos |>
  # Generar valores NA
  mutate(var1 = na_if(var1, 50)) |>

  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3",
      .default = "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Otra forma de definir los valores que no cumplen con ninguna condición es usando `TRUE ~ valor`:

```
datos <- datos |>
  # Generar valores NA
  mutate(var1 = na_if(var1, 50)) |>

  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3",
      TRUE ~ "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Las salidas son de tipo ***carácter*** (`chr`) y debemos manejar el ordenamiento de las etiquetas como vimos anteriormente, por medio de factores o comenzando con caracteres ordenados alfabéticamente.

Para simplificar el trabajo de estos intervalos de clase irregulares y no provocar errores en la confección de las condiciones, `tidyverse` tiene a la función `between()`.

13.3.1. Intervalos: función `between()`

La función `between()` básicamente opera como un atajo para condiciones de intervalos. Define dentro de los argumentos los límites inferior y superior de un intervalo y se utiliza dentro de una función de condición tipo `if_else()` o `case_when()`.

Aplicado sobre el ejemplo anterior se vería así:

```
datos <- datos |>
  mutate(
    grupo_var = case_when(
      between(var1, 0, 24) ~ "Grupo 1",
      between(var1, 25, 64) ~ "Grupo 2",
      between(var1, 65, Inf) ~ "Grupo 3",
      .default = "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Los valores declarados como límites quedan incluidos siempre dentro del intervalo (son cerrados ambos). También podemos utilizar valores reservados como `Inf` o `-Inf` cuando desconocemos con que valor máximo o mínimo nos vamos a encontrar en la variable cuantitativa original.

13.3.2. Intervalos: función `age_categories()`

El paquete `epikit` (Spina et al. 2025), contiene herramientas útiles para trabajar con datos en salud públicas. La función `age_categories()`, transforma una variable numérica discreta en grupos de edad. Volviendo al ejemplo anterior, supongamos que `var1` contiene edades de pacientes:

```r
# Cargar paquete
library(epikit)

datos <- datos |>
  mutate(
    edad_cat = age_categories(
      x = var1,
      lower = 1,
      upper = 80,
      by = 10,
      separator = " a ",
      above.char = " y más"
    )
  )

# Explora edad_cat
tabyl(datos, edad_cat)
```

```
 edad_cat n percent valid_percent
   1 a 10 6    0.12    0.12244898
  11 a 20 4    0.08    0.08163265
  21 a 30 7    0.14    0.14285714
  31 a 40 6    0.12    0.12244898
  41 a 50 5    0.10    0.10204082
  51 a 60 5    0.10    0.10204082
  61 a 70 4    0.08    0.08163265
  71 a 79 7    0.14    0.14285714
 80 y más 5    0.10    0.10204082
     <NA> 1    0.02            NA
```

Los argumentos principales de la función son:

- `x`: vector numérico (obligatorio).

- `lower`: límite inferior de edades (por defecto `0`).
- `upper`: límite superior de edades.
- `by`: intervalo de años para los grupos.
- `breakers`: alternativo a `by`, para definir manualmente los grupos.
- `separator`: carácter que separa el rango de edades (por defecto `"-"`).
- `ceiling`: define si incluir el valor más alto en los grupos (por defecto `FALSE`).
- `above.char`: el valor que queremos que aparezca para las edades por fuera del grupo de edad más alto (por defecto +). Solo funciona si `ceiling = FALSE`.

13.4. Ejemplo práctico en R

Tomemos un caso clásico como la variable edad medida en años, variable que generalmente tenemos en toda tabla de datos vinculada a personas. Trabajaremos con la base de datos “`edad.txt`” que contiene 106 observaciones de pacientes:

```r
# Cargar dataset
datos <- read_csv2("datos/edad.txt")

# Explorar datos
glimpse(datos)
```

```
Rows: 106
Columns: 3
$ id_mujer         <dbl> 115971, 1689710, 114710, 3058900, 3223547, 3234027, 3~
$ fecha_nacimiento <chr> "30/05/1967", "23/04/1991", "06/11/1962", "09/03/1964~
$ fecha_test       <chr> "17/01/2022", "01/02/2022", "16/03/2022", "23/02/2022~
```

A continuación, vamos a transformar las variables cuyo nombre contenga “`fecha_`” a formato fecha para poder calcular edades:

```r
datos <- datos |>
  # Transformar a formato fecha
  mutate(across(.cols = starts_with("fecha_"), .fns = ~ dmy(.x))) |>
  # Calcular edad al momento del testeo
  mutate(
    edad = round(
      as.duration(fecha_test - fecha_nacimiento) / dyears(1)
    )
  )
```

Una primera posibilidad es dicotomizar la edad usando el valor de la mediana:

```r
datos |>
  summarise(mediana = median(edad))
```

```
# A tibble: 1 x 1
  mediana
    <dbl>
1      56
```

Aplicando el valor de la mediana dentro de un `if_else()` podríamos hacer:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "mayor a la mediana",
      false = "menor o igual a la mediana"
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                    n
  <chr>                      <int>
1 mayor a la mediana            52
2 menor o igual a la mediana    54
```

Observamos en el conteo que `grupo_edad1` se construyó adecuadamente pero el orden de los niveles no es correcto si queremos que siga el ordenamiento natural de edad (de menor a mayor).

Una de las formas que vimos es convertir a factor:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "mayor a la mediana",
      false = "menor o igual a la mediana"
    ),
    grupo_edad1 = factor(
      grupo_edad1,
      levels = c("menor o igual a la mediana", "mayor a la mediana")
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                    n
  <fct>                      <int>
1 menor o igual a la mediana    54
2 mayor a la mediana            52
```

Vemos que en el conteo el formato de la variable ya no es `chr` sino `fct` y el orden de las etiquetas siguen la forma *"menor a mayor"*.

Otra forma es:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "2.mayor a la mediana",
      false = "1.menor o igual a la mediana"
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                      n
  <chr>                        <int>
1 1.menor o igual a la mediana    54
2 2.mayor a la mediana            52
```

Si en cambio necesitamos que los grupos sean mas de dos y que estos intervalos de clase sean regulares, podemos usar `cut_interval()`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <fct>       <int>
1 [0,10]          3
2 (10,20]         3
3 (20,30]         2
4 (30,40]         3
5 (40,50]        13
6 (50,60]        52
7 (60,70]        27
8 (70,80]         3
```

La salida muestra ocho grupos etarios con etiquetas ordenadas con notación matemática, donde un corchete indica que el límite del intervalo es cerrado, es decir contiene el valor y un paréntesis es abierto y no lo hace.Así es que el primer grupo va de 0 a 10 años y el segundo de 11 a 20.

Estos sucede así porque en forma predeterminada el argumento `right = TRUE`. Veamos que pasa si lo cambiamos a `FALSE`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, right = F))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <fct>       <int>
1 [0,10)          3
2 [10,20)         3
3 [20,30)         2
4 [30,40)         3
5 [40,50)        10
6 [50,60)        48
7 [60,70)        32
8 [70,80]         5
```

En esta salida el primer grupo va de 0 a 9 y el segundo de 10 a 19.

Hasta ahora la variable `grupo_edad2` es de tipo carácter, pero si deseamos que la salida sea factor podemos incorporar el argumento `ordered_result = TRUE`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, ordered_result = T))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <ord>       <int>
1 [0,10]          3
2 (10,20]         3
3 (20,30]         2
4 (30,40]         3
5 (40,50]        13
6 (50,60]        52
7 (60,70]        27
8 (70,80]         3
```

Construimos así una variable factor ordenada `<ord>`.

Por último, con el argumento `labels= FALSE` hacemos que las etiquetas de los ocho grupos sean numéricas:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, labels = F))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
        <int> <int>
1           1     3
2           2     3
3           3     2
4           4     3
5           5    13
6           6    52
7           7    27
8           8     3
```

Otro ejemplo, podría ser aplicando `case_when()` donde discretizamos la edad en cuatro grupos irregulares, forzando sus etiquetas para lograr el orden adecuado:

```
datos <- datos |>
  mutate(
    grupo_edad3 = case_when(
      edad < 13 ~ "1.Niño",
      edad > 12 & edad < 26 ~ "2.Adolescente",
      edad > 25 & edad < 65 ~ "3.Adulto_joven",
      edad > 64 ~ "4.Adulto_mayor"
    )
  )

# Explorar grupo_edad3
datos |>
  count(grupo_edad3)
```

```
# A tibble: 4 x 2
  grupo_edad3        n
  <chr>          <int>
1 1.Niño             3
2 2.Adolescente      5
3 3.Adulto_joven    86
4 4.Adulto_mayor    12
```

Si no hubiésemos etiquetado con los números por delante el orden alfabético hacía que Niño fuese a parar al final del conteo.

De la misma forma pero más sencillo y controlado es:

```
datos <- datos |>
  mutate(
    grupo_edad3 = case_when(
      between(edad, 0, 12) ~ "1.Niño",
      between(edad, 13, 25) ~ "2.Adolescente",
      between(edad, 26, 64) ~ "3.Adulto_joven",
      between(edad, 65, Inf) ~ "4.Adulto_mayor"
    )
  )

# Explorar grupo_edad3
datos |>
  count(grupo_edad3)
```

```
# A tibble: 4 x 2
  grupo_edad3        n
  <chr>          <int>
1 1.Niño             3
2 2.Adolescente      5
3 3.Adulto_joven    86
4 4.Adulto_mayor    12
```

⚠ Estas funciones condicionales que tratamos en este documento no se limitan a la tarea de construir agrupamientos de variables cuantitativas sino que sirven para cualquier situación donde a partir de una o más condiciones se produzcan una o más valores como respuesta.

Finalmente veamos como quedarían los grupos de edad con la función `age_categories()`:

```
datos <- datos |>
  mutate(
    grupo_edad4 = age_categories(
      x = edad,
      breakers = c(0, 12, 25, 65),
      above.char = " y más"
    )
  )

# Explorar grupo_edad4
datos |>
  count(grupo_edad4)
```

```
# A tibble: 4 x 2
  grupo_edad4     n
  <fct>       <int>
```

```
1 0-11            3
2 12-24           4
3 25-64          87
4 65 y más       12
```

En este caso se genera una variable de tipo factor con los grupos de edad ordenados de mayor a menor.

14. Intervalos de confianza en muestras complejas

14.1. Introducción

En los estudios epidemiológicos de corte transversal, es común el uso de encuestas poblacionales para estimar la frecuencia de determinadas condiciones de salud o la presencia de factores de riesgo en la población.

Sin embargo, la implementación de estas encuestas enfrenta restricciones prácticas que hacen inviable o poco conveniente el muestreo aleatorio simple (MAS). En su lugar, se recurre a otras alternativas de muestreo, como la estratificación, la selección en etapas, la formación de conglomerados o el empleo de probabilidades de selección desiguales.

Los diseños muestrales que incorporan combinaciones de estas estrategias se denominan **muestreos complejos**, por contraste con el MAS, en el que las unidades muestrales se seleccionan independientemente unas de otras y todas tienen igual probabilidad de selección y distribución.

El análisis de los datos obtenidos mediante diseños muestrales complejos presenta desafíos adicionales debido a la posible correlación entre observaciones dentro de un mismo conglomerado. Esta correlación impide asumir independencia de las observaciones, condición necesaria para sostener el supuesto de normalidad. Si no se considera el efecto del diseño muestral al realizar estimaciones e intervalos de confianza (IC), los resultados pueden ser erróneos. Aspectos clave como el **efecto del diseño**, los **dominios de estimación** y los **factores de expansión** deben ser incorporados en el análisis para obtener inferencias válidas.

14.1.1. Algunas definiciones necesarias

Estratos

Subpoblaciones “naturales” que, *a priori*, son homogéneos en su interior pero heterogéneos entre sí. El diseño de la encuesta se hace de modo que se garantiza cubrir adecuadamente todos los estratos de interés. Algunas variables de estratos habituales son: sexo, edad (como grupo etario), nivel de educación, urbano/rural, etc.

Conglomerados

Unidades definidas dentro de cada estrato (si es muestreo polietápico), cuyo tamaño es conocido. Lo ideal es que la población dentro de un conglomerado sea lo más heterogénea posible, pero debe haber homogeneidad entre los conglomerados. Cada grupo debe ser una representación a pequeña escala de la población total. Los grupos deben ser mutuamente excluyentes y colectivamente exhaustivos.

Probabilidad de inclusión

Es la probabilidad que cada unidad tiene de estar incluida en la muestra. En los muestreos complejos polietápicos se obtiene, en general, multiplicando las probabilidades asociadas al estrato, a cada **Unidad Primaria de Muestreo** (UPM o PSU en inglés) dentro del estrato, y a la unidad de segunda etapa dentro cada UPM.

Dominios de estimación

Subconjuntos de la población objetivo cuyos elementos pueden ser identificados en el marco muestral sin ambigüedad, y en los que se permite desagregar los resultados de la encuesta. Es aconsejable respetar estos dominios de estimación y no realizar inferencia de parámetros de interés para otros dominios no previstos que conlleva estimaciones inválidas.

Factor de expansión

Es el valor asociado a cada unidad elegible y que responde a la muestra, que se construye a partir de la inversa de la probabilidad de inclusión de cada unidad o peso muestral inicial. Puede incluir distintos tipos de ajustes, para disminuir en lo posible los errores de cobertura y de no respuesta que afectan a la encuesta, y ser tratados por un proceso de calibración que lleva en general a ganar eficiencia y precisión en las estimaciones. Los factores de expansión finales son los que se emplean tanto para generar todas las estimaciones de una encuesta, como en los cálculos del error muestral al determinar la precisión alcanzada.

Efecto de diseño (DEFF por sus siglas en inglés)

Mide la pérdida en precisión al utilizar un diseño muestral complejo en lugar de un diseño aleatorio simple, por ejemplo, un efecto de diseño de 1,5 indica que la varianza del diseño complejo es 1,5 veces más grande que la varianza de un diseño aleatorio simple, en otras palabras se dio un aumento en la varianza de un 50 %.

14.2. Paquetes survey y srvyr

El paquete más conocido y utilizado del lenguaje R para trabajar con datos provenientes de muestreos complejos es `survey` (Lumley 2010) . Desarrollado por *Thomas Lumley* que a su vez es autor del libro *Complex Surveys - A Guide to Analysis Using R (2010)*.

Las funciones que lo integran utilizan en sus argumentos la sintaxis fórmula preferentemente y no son compatibles con el universo “ordenado” de `tidyverse`. Con el objetivo de hacer compatible estas funciones crearon un paquete “*wrapper*” (envoltorio) llamado `srvyr` (Freedman Ellis y Schneider 2026), que incorpora sintaxis *tidy*, haciendo uso de tuberías y funciones tales como `group_by()`, `summarise()`, entre otras.

El primer paso para trabajar con las funciones de `srvyr` es crear un objeto con la información relacionada con el diseño muestral. Esta tarea se realiza con la función `as_survey_design()` que tiene estos argumentos principales:

```
datos |>
  as_survey_design(
    ids = ...,
    strata = ...,
    variables = ...,
    fpc = ...,
    nest = F,
    weights = ...
  )
```

donde:

- `ids`: Variables que especifican identificadores de conglomerados desde el nivel más grande hasta el nivel más pequeño (dejar el argumento vacío, `NULL`, 1 o 0 indica que no hay conglomerados).
- `strata`: Variables que identifican a los estratos. Si no hay estratos, se ignora esta especificación.

Usualmente, se declara a partir de una variable. Por ejemplo, si la variable `estrato` define los estratos, sería,

```
datos |>
  as_survey_design(..., strata = estrato, ...)
```

- `variables`: Variables que se incluirán en el diseño. El valor predeterminado es todas las variables de datos.
- `fpc`: Variables con el factor de corrección por población finita.
- `nest:` Si es `TRUE`, re-etiqueta los conglomerados considerando anidamiento dentro de los estratos. Necesario activar cuando las etiquetas de las categorías de los conglomerados en los distintos estratos se llaman igual.
- **`weights`**: Variables de ponderación de cada observación (inverso a la probabilidad).

Los argumentos mínimos que debemos definir dependerá de la estructura muestral de la base de datos que estemos analizando y de las variables que tengamos a disposición. Habitualmente se tiene referencia de estratos, conglomerados y ponderaciones. También podemos encontrarnos con situaciones donde están definidos los tamaños poblacionales (variable `fpc`).

14.3. Ejemplo práctico en R

Para ejemplificar vamos a tomar a la **Encuesta Mundial de Salud Escolar** (EMSE) en su tercera edición que en la Argentina se realizó en 2018. Fue llevada a cabo por el Ministerio de Salud y Desarrollo Social de la Nación, contó con la colaboración de los Ministerios de Educación Nacional y Provinciales, la OPS/OMS Argentina, OPS/OMS Washington y el CDC.

El diseño de muestreo tuvo dos etapas (selección de escuelas y luego de divisiones al azar) para producir una muestra representativa de alumnos de 1º a 5º año de educación media a nivel nacional (8º EGB a 3º polimodal en el caso de la provincia de Buenos Aires) y provincial. Se relevaron 523 escuelas en todo el país y se encuestaron 57.095 alumnos de los cuales se analizaron 56.981 cuestionarios correspondientes a las edades de 13 a 17 años, con una tasa de respuesta global de 63 %. Se puede acceder a sus datos abiertos en EMSE 2018

Activamos los paquetes necesarios:

```
library(srvyr)
library(tidyverse)
```

Leemos el archivo de datos “`EMSE.txt`” con:

```
datos <- read_csv2("datos/EMSE.txt")

# dimensiones de la tabla de datos
glimpse(datos)
```

```
Rows: 56,981
Columns: 14
$ record      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
$ q2          <dbl> 2, 2, 1, 1, 2, 1, 2, 2, 1, 2, 1, 2, 2, 2, 2, 2, 1, 1, 2, 1,~
$ texto_q2    <chr> "Femenino", "Femenino", "Masculino", "Masculino", "Femenino~
$ q3          <dbl> 2, 2, 4, 2, 2, 2, 4, 4, 4, 2, 2, 4, 2, 4, 4, 4, 4, 2, 2, 2,~
$ texto_q3    <chr> "9no grado nivel Primario/Polimodal o 2do año nivel Secunda~
$ q4          <dbl> NA, NA, 1.67, 1.73, NA, 1.72, NA, NA, NA, NA, 1.64, NA, NA,~
$ q5          <dbl> NA, NA, 56, 70, NA, 48, NA, NA, NA, NA, 45, NA, NA, 40, 52,~
$ q6          <dbl> 2, 1, 1, 1, 1, 1, 3, 1, 4, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ texto_q6    <chr> "Rara vez", "Nunca", "Nunca", "Nunca", "Nunca", "Nunca", "A~
$ qn24        <dbl> 1, 2, NA, 2, 2, 2, 2, 2, 2, 1, 2, 1, 2, NA, 1, 2, 2, 2, 2, ~
$ texto_qn24 <chr> "Si", "No", "Dato perdido", "No", "No", "No", "No", "No", "~
$ psu         <dbl> 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19,~
$ stratum     <dbl> 201801010, 201801010, 201801010, 201801010, 201801010, 2018~
$ weight      <dbl> 457.09, 457.09, 261.44, 550.90, 457.09, 550.90, 274.43, 274~
```

El dataframe importado consta de *14 variables y 56.981 observaciones*.

Calculamos el total de la muestra expandida:

```
datos |>
  summarise(total_expandido = sum(weight))
```

```
# A tibble: 1 x 1
  total_expandido
            <dbl>
1         2637546
```

Las variables relevantes para el diseño muestral que el documento de usuario define son:

- `psu`: Unidades primarias de muestreo (conglomerados)
- `stratum`: Estratos del muestreo
- `weight`: Ponderación

Además, nuestras variables de interés son:

- `record`: Nro. de registro
- `q2`: Sexo
- `texto_q2`: Sexo categorizado
- `q3`: Grado / año en el que se encuentra el estudiante
- `texto_q3`: Grado / año en el que se encuentra el estudiante categorizado
- `q4`: Estatura sin zapatos (medido en metros)
- `q5`: Peso sin zapatos (medido en kilogramos)
- `q6`: Pregunta sobre hambre
- `texto_q6`: Pregunta sobre hambre categorizada
- `qn24`: Pregunta sobre idea suicida
- `texto_qn24`: Pregunta sobre idea suicida categorizada

Para facilitar el análisis, vamos a renombrar nuestras variables de interés usando la función `rename()` de `tidyverse`:

```
datos <- datos |>
  rename(
    sexo = texto_q2,
    grado = texto_q3,
    estatura_m = q4,
    peso_kg = q5,
    hambre = texto_q6,
    idea_suicida = texto_qn24
  )
```

A partir de las variables cuantitativas `estatura_m` y `peso_kg` vamos a construir una nueva variable llamada `imc` (Índice de Masa Corporal) :

```
datos <- datos |>
  # IMC: peso sobre talla al cuadrado
  mutate(imc = peso_kg / (estatura_m)^2)
```

Ahora podemos comenzar a definir el objeto de diseño muestral:

```
d <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = stratum, # estratos
    weights = weight, # ponderación
    nest = T
  ) # anidación
```

Una vez que tenemos creado el objeto ***"survey.design"***, que en nuestro ejemplo se denomina `d`, podemos avanzar en el calculo de las estimaciones.

Si bien podemos manipular datos dentro de este formato de diseño utilizando algunas de las funciones de `tidyverse` como `select()`, `mutate()`, `filter()` y `rename()`, lo conveniente es modificar previamente la estructura de la tabla de datos, ya sea para crear una nueva variable, como hicimos recién con IMC o cambiar una existente y luego generar el objeto con el diseño muestral para poder hacer uso de esos cambios en las estimaciones.

14.3.1. Estimaciones

> Todos los códigos para las estimaciones comenzaran a partir del objeto de diseño creado inicialmente (llamado `d` en nuestro ejemplo) y evitando utilizar el nombre de la tabla de datos leída que solo tiene los datos de la muestra sin especificar el diseño de muestreo.

En primer lugar tomaremos la variable `imc`, variable cuantitativa continua que construimos.

La función `survey_mean()` computa estimaciones de medias en diseños complejos usando la sintaxis de `tidyverse`.

Los siguientes son los argumentos comunes de la función:

- `x`: Variable o expresión o vacía.
- `na.rm`: Si es `TRUE`, omite los valores `NA` de la variable
- `vartype`: Obtiene la variabilidad como uno o más estimadores: error estándar (`"se"`, predeterminado), intervalo de confianza (`"ci"`), varianza (`"var"`) o coeficiente de variación (`"cv"`).
- `level`: Nivel de confianza (solo se usa si el anterior es `"ci"`)
- `deff`: Si es `TRUE`, calcula el efecto de diseño para la estimación

Aplicamos la función con todas las posibles salidas de varianza sobre la variable de `imc` dentro de `summarise()` de `tidyverse`:

```
d |>
  summarise(
    imc = survey_mean(
      x = imc,
      na.rm = T,
      vartype = c("se", "ci", "var", "cv"),
      level = 0.95
    )
  )
```

```
# A tibble: 1 x 6
    imc imc_se imc_low imc_upp imc_var  imc_cv
  <dbl>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1  22.1 0.0869    22.0    22.3 0.00755 0.00392
```

El resultado muestra una media de IMC de 22,1, un error estándar de 0,09, un intervalo de confianza al 95 % de (22,0-22,3), una varianza de 0,007 y un coeficiente de variación de 0,4 %.

Como nos informan que hubo no respuesta en la encuesta y las faltantes de información en altura y/o peso provoca observaciones perdidas en la variable IMC, vamos a ver cuantos de estos valores perdidos tenemos en la variable.

Si deseamos calcular totales, es decir a cuantos estudiantes representa estos valores perdidos podemos usar la función `survey_total()`:

```
d |>
  filter(is.na(imc)) %>% # filtramos los NA en IMC

  summarise(imc_na = survey_total(vartype = "ci"))
```

```
# A tibble: 1 x 3
   imc_na imc_na_low imc_na_upp
    <dbl>      <dbl>      <dbl>
1 976104.    878120.   1074088.
```

Hay aproximadamente 976.104 IC 95 % (87.820-1.074.088) estudiantes sin valor en la variable `imc` del total de la población de 2.637.546 (un 37 %).

Si en lugar de la media quisiéramos la mediana podemos cambiar la función y usar `survey_median()`:

```
d |>
  summarise(
    imc = survey_median(
      x = imc,
      na.rm = T,
      vartype = c("se", "ci", "var", "cv"),
      level = 0.95
    )
  )
```

```
# A tibble: 1 x 6
    imc imc_se imc_low imc_upp imc_var  imc_cv
  <dbl>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1  21.5 0.0772    21.4    21.7 0.00596 0.00359
```

Podemos estratificar estas estimaciones haciendo uso del `group_by()`, por ejemplo con la variable `sexo` del estudiante:

```
d |>
  group_by(sexo) |>
  summarise(media_imc = survey_mean(imc, vartype = "ci", na.rm = T))
```

```
# A tibble: 3 x 4
  sexo          media_imc media_imc_low media_imc_upp
  <chr>             <dbl>         <dbl>         <dbl>
1 Dato perdido        0             0             0
2 Femenino           22.0          21.8          22.2
3 Masculino          22.3          22.1          22.5
```

Como hay datos perdidos en la variable sexo nos aparece una categoría más en los grupos de los resultados. Al estar vacía en las estimaciones (evidentemente son “no respuestas” completas) podemos deshacernos de esa línea con un `filter()`:

```
d |>
  filter(sexo != "Dato perdido") |> # o filter(!is.na(q2))

  group_by(sexo) |>
  summarise(media_imc = survey_mean(imc, vartype = "ci", na.rm = T))
```

```
# A tibble: 2 x 4
  sexo      media_imc media_imc_low media_imc_upp
  <chr>         <dbl>         <dbl>         <dbl>
1 Femenino       22.0          21.8          22.2
2 Masculino      22.3          22.1          22.5
```

Esta tabla muestra las estimaciones de `imc` según sexo con sus intervalos de confianza.

Otra variable de la encuesta es la respuesta a la pregunta *“Durante los últimos 30 días ¿con qué frecuencia te quedaste con hambre porque no había suficiente comida en tu hogar?”*. Las categorías válidas fueron: Nunca, Rara vez, Algunas veces, Casi siempre, Siempre y Dato perdido si no hubo respuesta.

Para abordar estas variables cualitativas donde queremos obtener proporciones el paquete tiene la función `survey_prop()` que se utiliza declarando la variable de interés en un `group_by()` previo:

```
d |>
  group_by(hambre) %>% # variable para la estimación
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100) # usamos 100 para %
```

```
# A tibble: 6 x 4
  hambre        prop_hambre prop_hambre_low prop_hambre_upp
  <chr>               <dbl>           <dbl>           <dbl>
1 Algunas veces       9.34            8.49           10.3  
2 Casi siempre        1.36            1.18            1.56 
3 Dato perdido        0.804           0.678           0.952
4 Nunca              67.8            66.6            69.0  
5 Rara vez           20.1            19.2            20.9  
6 Siempre             0.627           0.487           0.807
```

El resultado muestra las categorías desordenadas en relación a lo que significa cada una respecto a la definición de la pregunta, es decir a la ordinalidad de la variable, aunque si está ordenada respecto a la forma en que lo hace el lenguaje R (alfabético).

Para darle el orden adecuado, deberíamos convertir la variable `hambre` a *factor* y declarar correctamente sus niveles. Conviene hacer esto previo a la creación del objeto de diseño pero el paquete también lo permite una vez creado:

```r
d <- d |>
  mutate(
    hambre = factor(
      hambre,
      c(
        "Nunca",
        "Rara vez",
        "Algunas veces",
        "Casi siempre",
        "Siempre",
        "Dato perdido"
      )
    )
  )
```

Repetimos el código anterior:

```r
d |>
  group_by(hambre) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 6 x 4
  hambre        prop_hambre prop_hambre_low prop_hambre_upp
  <fct>               <dbl>           <dbl>           <dbl>
1 Nunca              67.8            66.6            69.0  
2 Rara vez           20.1            19.2            20.9  
3 Algunas veces       9.34            8.49           10.3  
4 Casi siempre        1.36            1.18            1.56 
5 Siempre             0.627           0.487           0.807
6 Dato perdido        0.804           0.678           0.952
```

El ordenamiento de las categorías ahora es el correcto producto de la transformación a factor.

Para estratificar estas proporciones lo único que debemos hacer es incorporar la variable de agrupamiento dentro del `group_by()`:

```r
d |>
  group_by(sexo, hambre) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 18 x 5
# Groups:   sexo [3]
   sexo         hambre        prop_hambre prop_hambre_low prop_hambre_upp
   <chr>        <fct>               <dbl>           <dbl>           <dbl>
 1 Dato perdido Nunca              54.3            43.9            64.3
 2 Dato perdido Rara vez           23.5            17.3            31.2
 3 Dato perdido Algunas veces      14.2             8.18           23.5
 4 Dato perdido Casi siempre        1.07            0.459           2.49
 5 Dato perdido Siempre             1.18            0.589           2.36
 6 Dato perdido Dato perdido        5.74            3.45            9.41
 7 Femenino     Nunca              68.1            66.7            69.4
 8 Femenino     Rara vez           19.7            18.7            20.8
 9 Femenino     Algunas veces       9.78            8.96           10.7
10 Femenino     Casi siempre        1.18            0.933           1.49
11 Femenino     Siempre             0.549           0.389           0.775
12 Femenino     Dato perdido        0.673           0.530           0.854
13 Masculino    Nunca              67.8            66.2            69.4
14 Masculino    Rara vez           20.3            19.2            21.5
15 Masculino    Algunas veces       8.75            7.69            9.94
16 Masculino    Casi siempre        1.56            1.20            2.01
17 Masculino    Siempre             0.700           0.518           0.946
18 Masculino    Dato perdido        0.842           0.624           1.13
```

Este esquema de agrupamientos o estratificaciones se anidan en el orden en que se declaran dentro del `group_by()` siendo la última variable la de estimación. Por ende, los porcentajes también salen anidados, tomando como denominador para el calculo del 100 % al total de cada categoría de `sexo` en el ejemplo anterior.

También podemos hacer que para el calculo de los porcentajes se tome como denominador el total general de la siguiente forma:

```r
d |>
  group_by(interact(sexo, hambre)) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 18 x 5
   sexo         hambre        prop_hambre prop_hambre_low prop_hambre_upp
   <chr>        <fct>               <dbl>           <dbl>           <dbl>
 1 Dato perdido Nunca              0.539          0.429            0.677
 2 Dato perdido Rara vez           0.234          0.160            0.342
 3 Dato perdido Algunas veces      0.141          0.0755           0.263
 4 Dato perdido Casi siempre       0.0107        0.00464          0.0245
 5 Dato perdido Siempre            0.0117        0.00594          0.0232
 6 Dato perdido Dato perdido       0.0570        0.0328           0.0990
 7 Femenino     Nunca             35.0          33.7             36.4
 8 Femenino     Rara vez          10.2           9.58            10.8
 9 Femenino     Algunas veces      5.03          4.63             5.47
10 Femenino     Casi siempre       0.608         0.481            0.768
11 Femenino     Siempre            0.283         0.200            0.400
12 Femenino     Dato perdido       0.346         0.270            0.443
13 Masculino    Nunca             32.2          31.1             33.4
14 Masculino    Rara vez           9.66          9.11            10.2
15 Masculino    Algunas veces      4.16          3.63             4.76
16 Masculino    Casi siempre       0.741         0.569            0.965
17 Masculino    Siempre            0.333         0.246            0.450
18 Masculino    Dato perdido       0.400         0.295            0.543
```

Un argumento incluido dentro de las posibilidades de `survey_prop()` es el método para su calculo (`prop_method`). En este documento no vamos a profundizar en los métodos que la función ofrece, solo mencionaremos las posibilidades permitidas según la ayuda del paquete:

- `"likelihood"`: utiliza la distribución de chi-cuadrado escalada (Rao-Scott) para la log-likelihood de una distribución binomial.
- `"asin"`: usa la transformación estabilizadora de la varianza para la distribución binomial, la raíz cuadrada del arcoseno, y luego transforma el intervalo a la escala de probabilidad.
- `"beta"`: usa la función beta incompleta como en `binom.test`, con una tamaño de muestra efectivo basado en la varianza estimada de la proporción. (Korn y Graubard, 1998).
- `"mean"`: utiliza un intervalo tipo Wald en la escala de probabilidad, igual que `confint()`.
- `"logit"`: ajusta un modelo de regresión logística y calcula un intervalo tipo Wald en la escala logarítmica de probabilidades, que luego se transforma a la escala de probabilidad. Es el método predeterminado de la función.

Otra función útil es `survey_count()` que estima el conteo poblacional de las observaciones según categoría:

```r
d |>
  survey_count(hambre, vartype = "ci")
```

```
# A tibble: 6 x 4
  hambre              n   n_low   n_upp
  <fct>           <dbl>   <dbl>   <dbl>
1 Nunca         1788701. 1683670. 1893733.
2 Rara vez       529025.  498681.  559370.
3 Algunas veces  246227.  213940.  278513.
4 Casi siempre    35857.   30276.   41438.
5 Siempre         16542.   12251.   20832.
6 Dato perdido    21194.   16920.   25468.
```

Aquí lo usamos acompañando con el intervalo de confianza del 95 % (predeterminado).

Ahora tomaremos otra variable de la EMSE2018 llamada `idea_suicida` que refiere a la pregunta: *"Durante los últimos 12 meses, ¿alguna vez consideraste seriamente la posibilidad de intentar suicidarte?"*. Sus posibles respuestas son `"Si"`, `"No"` y el habitual `"Dato perdido"`. Estimaremos su proporción para toda la población pero también para una subpoblación en particular, por ejemplo para *"3er año/12vo grado nivel Polimodal o 5to año nivel Secundario"*.

Como es habitual que necesitemos obtener estimaciones en subgrupos o subpoblaciones determinados por categorías definidas de una variable, debemos tener cuidado al interpretar los elementos del muestro para que esas subpoblaciones hayan sido consideradas y por lo tanto estén incluídas como **dominio de estimación**.

Estos subgrupos requeridos pueden no coincidir con los estratos de la muestra compleja generando un inconveniente para las estimaciones, dado que las ponderaciones muestrales serían correctas pero la probabilidad de muestreo seguramente no, lo que produce estimaciones puntuales correctas con errores estándar incorrectos.

Las funciones del paquete `srvyr` maneja estos detalles sin ningún esfuerzo especial por parte del analista siempre y cuando utilice la muestra completa para definir el objeto de diseño de la encuesta pero no asegura que los resultados tengan la calidad necesaria.

Mostremos el código entonces:

```
d |>
  group_by(idea_suicida) |>

  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          2.16              1.90              2.45
2 No                   76.8              76.1              77.6
3 Si                   21.0              20.3              21.7
# i 1 more variable: prop_suicidio_cv <dbl>
```

Seleccionamos una subpoblación correspondiente a los estudiantes de "*3er año/12vo grado nivel Polimodal o 5to año nivel Secundario*" (código 5 de q3):

```
d |>
  filter(q3 == 5) |>

  group_by(idea_suicida) |>

  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          1.71              1.27              2.30
```

```
2 No                    79.3              77.6              81.0
3 Si                    18.9              17.5              20.4
# i 1 more variable: prop_suicidio_cv <dbl>
```

Un 21,0 % (95 % IC: 20,3-21,7 %) del total de estudiantes dicen haber considerado un intento de suicidio en el último año, mientras que un 18,9 % (95 % IC: 17,5-20,4 %) dice lo mismo en el grupo de "*3er año/12vo grado nivel Polimodal o 5to año nivel Secundario*".

Agregamos a los resultados algo importante, que va a garantizar la calidad de la estimaciones y evitar estar informando valores poco confiables para los cuales el muestreo quizás no esté preparado. Este es el **coeficiente de variación** (CV). Comparativamente vamos a encontrar valores más elevados de CV, en la medida que se agreguen filtros o agrupamientos anidados que reduzcan la muestra de la estimación.

Que pasa si filtramos y nos quedamos además solo con las mujeres que respondieron que se quedaron con hambre "algunas veces" en los últimos 30 días:

```r
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>

  group_by(idea_suicida) |>
  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          3.15             0.941              10.0
2 No                   63.5             53.7                72.3
3 Si                   33.4             25.6                42.2
# i 1 more variable: prop_suicidio_cv <dbl>
```

Los CV de las estimaciones siguen creciendo, por ejemplo en el caso de la respuesta **Si** a un 12,6 % en comparación de la población completa que era de 1,6 %. Este aumento tiene que ver sobre todo con el tamaño muestral que nos queda luego de tantos filtros:

```r
# Total muestra
d |>
  survey_count(idea_suicida)
```

```
# A tibble: 3 x 3
  idea_suicida        n   n_se
  <chr>           <dbl>  <dbl>
1 Dato perdido   56885.  4176.
2 No           2026726. 56584.
3 Si            553935. 21644.
```

```r
# Subgrupo seleccionado
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>
  survey_count(idea_suicida)
```

```
# A tibble: 3 x 3
  idea_suicida       n  n_se
  <chr>          <dbl> <dbl>
1 Dato perdido    600.  356.
2 No            12084. 1695.
3 Si             6348.  808.
```

Solo 6.347 estudiantes expandidos que respondieron que **Si** sobre los 553.934 totales que lo hicieron. Y cuantos respecto a la muestra sin expandir?

Podemos verlo así:

```
# Total muestra
d |>
  group_by(idea_suicida) |>

  summarise(n = unweighted(n()))
```

```
# A tibble: 3 x 2
  idea_suicida     n
  <chr>        <int>
1 Dato perdido  1353
2 No           43666
3 Si           11962
```

```
# Subgrupo seleccionado
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>

  group_by(idea_suicida) |>

  summarise(n = unweighted(n()))
```

```
# A tibble: 3 x 2
  idea_suicida     n
  <chr>        <int>
1 Dato perdido     9
2 No             287
3 Si             194
```

194 estudiantes de la muestra sin expandir respondieron que **Si** sobre 11.962 sin aplicar los filtros.

14.4. Errores estándar según efecto del diseño

En este punto vamos a comparar estimaciones en función de construir objetos de diseños muestrales diferentes sobre la misma base de datos.

Diseño complejo y pesos (igual al ejemplo anterior):

```
d_complejo <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = stratum, # estratos
    weights = weight, # ponderación
    nest = T
  ) # anidación
```

Diseño sin pesos ni estratos:

```
d_simple <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = NULL, # estratos
    weights = NULL, # ponderación
    nest = T
  ) # anidación
```

Diseño con pesos y sin estratos:

```
d_ponde <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = NULL, # estratos
    weights = weight, # ponderación
    nest = T
  ) # anidación
```

Tenemos tres diseños distintos: un diseño complejo basado en conglomerados, estratos y pesos, un diseño sólo con pesos y un diseño simple sin estratos ni ponderación. Sabemos que el primero es el diseño “real” con el que se llevó a cabo la recolección de los datos.

Ahora volvamos a la variable `imc` para estimar su media en cada diseño:

```
d_complejo |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.13994      21.96900      22.31089  0.003923691
```

```
d_simple |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.23472      22.16037      22.30906  0.001669130
```

```
d_ponde |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.13994      21.98614      22.29375  0.003467962
```

La estimación puntual (`imc` = 22,14) es la misma para los análisis que usan el diseño complejo y los que usan sólo pesos. A su vez es diferente si las estimaciones se llevan a cabo sin considerar el factor de expansión (muestreo simple - `imc` = 22,23). Los intervalos de confianza estimados son diferentes para todos los casos porque la varianza se vincula con la estructura del muestreo (estratos y conglomerados). Si no se utilizan pesos en las estimaciones, los estimadores no serán representativos de la población muestreada.

Si solo se utilizan pesos sin considerar el diseño complejo, en general, se infravalorará la dispersión de los estimadores, llevando a intervalos de confianza excesivamente estrechos y a niveles de significación reales mayores que los nominales (como si se tratase de un MAS). Basta comparar el resultado correcto IMC 22,14 (95 % IC: 21,97-22,31) frente al producido con el diseño simple IMC 22,23 (95 % IC: 22,16-22,30).

Como vemos, el lenguaje R permite que construyamos el objeto de diseño de la muestra compleja con la forma que indiquemos pero esto no siempre es correcto. Debemos conocer previamente la forma en que se diseño el muestreo y como se expresa en las variables de la tabla de datos para hacerlo adecuadamente. **Es fundamental leer detenidamente los documentos de utilización de los datos de toda encuesta donde se hayan recolectado datos mediante muestreos complejos**.

14.4.1. Criterios de calidad en las estimaciones

Todas las estimaciones elaboradas a partir de datos obtenidos por encuestas poblacionales con muestreos complejos están sujetas al error muestral, lo que hace necesario evaluar su validez estadística mediante diversos indicadores de precisión y confiabilidad.

Veamos algunos de estos indicadores de calidad:

Coeficiente de variación

Es el principal indicador de calidad de una estimación. Esta medida configura un acercamiento al error de muestreo que permite verificar si la inferencia es válida. Se caracteriza por ser proporcional a la amplitud del intervalo de confianza, que provee una versión estandarizada y relativa de la precisión alrededor de la estimación puntual.

Un umbral aproximado de CV mayor a 20 %-30 % puede asumirse como un valor de referencia útil a nivel regional para señalar una cifra de poco confiable, *Gutiérrez y otros (2020)*. Para calcular el coeficiente de variación de las estimaciones en estas estimaciones basta con incluirlo dentro del argumento `vartype`.

Tamaño de muestra

Este criterio se encuentra generalmente relacionado al anterior y es relevante a la hora de decidir la calidad de la estimación. La cobertura de los intervalos de confianza y la distribución de los estimadores dependen de que, tanto el tamaño de la subpoblación como su tamaño de muestra asociado, no sean pequeños. Con este espíritu, *Gutiérrez y otros (2020)* proponen que todas las estimaciones basadas en un tamaño de muestra expandida menor a 100 unidades deberían ser marcadas como no confiables.

Conteo de casos no ponderado

Cuando la incidencia de un fenómeno es muy baja y el diseño de la encuesta no lo tuvo en cuenta, entonces es posible que las estimaciones asociadas a tamaños, totales y proporciones sobre este fenómeno no sean confiables. Por ejemplo, *National Research Council (2015)* plantea que si el número de casos no ponderados es menor a 50 unidades entonces la estimación no sería publicable.

Parte III.

Unidad 3

15. Estudios ecológicos

15.1. Introducción

Por definición, un **estudio ecológico** es aquel en el cual las unidades de análisis son poblaciones o agrupamientos de individuos, no los individuos propiamente dichos. Estos conglomerados pueden estar definidos en un contexto espacial (ciudad, provincia, país, región, etc), institucional (hospitales, escuelas, etc), social, temporal, etc. La característica principal de este tipo de diseños es que se cuenta con información sobre la exposición y el evento para el conglomerado en su totalidad, pero se desconoce la información a escala individual.

En este tipo de estudios se asigna la misma exposición (generalmente una exposición promedio) a todo el conglomerado. Algo similar ocurre con el evento considerado, tenemos un número de eventos correspondiente al conglomerado, pero no se sabe si los individuos expuestos son los que tienen el evento.

Repasemos las principales ventajas y desventajas de este diseño:

Ventajas	Desventajas
Se pueden estudiar grandes grupos poblacionales	No se tiene información del individuo,por lo que no se puede ajustar por diferencias a nivel individual
Relativamente fáciles de realizar	No se puede saber quién sí está expuesto o quién sí desarrolló el evento de interés
Aumenta el poder estadístico	No se tiene información sobre factores de confusión y no se puede corregir por esto
Aumenta la variabilidad en exposición	
Se puede utilizar información de estadísticas vitales	

Recordemos también que puede incurrirse en la llamada *falacia ecológica*:

> ⚠ **Falacia ecológica (*ecological fallacy*)**
>
> Sesgo que puede aparecer al observar una asociación a partir de un estudio ecológico pero que no representa una asociación causal a nivel individual.

En un estudio ecológico, existen distintos niveles de medición de las variables de grupo:

- **Medidas agregadas**: Se trata de resumir las observaciones sobre individuos en cada grupo, expresándolas como medias o proporciones (ejemplo la proporción de usuarios de cinturón de seguridad).
- **Mediciones ambientales**: Son características generalmente del lugar en el que los miembros del grupo viven o trabajan (ejemplo: niveles de contaminación ambiental). Cada medición ambiental tiene su análogo a nivel individual y la exposición individual puede variar dentro del grupo.

- **Mediciones globales**: Son características de grupos, organizaciones o áreas para las que no existe una analogía con el nivel individual (ejemplo: densidad de población).

Esta distinción es importante, porque para las dos primeras siempre existe el recurso de la medición individual, en tanto que en las mediciones globales, el diseño ecológico es la única alternativa viable.

15.2. Clasificación de estudios ecológicos

- **Exploratorios**: En los estudios exploratorios se comparan las tasas de enfermedad entre muchas regiones durante un mismo periodo, o se compara la frecuencia de la enfermedad a través del tiempo en una misma región. Se podría decir que se buscan patrones regionales o temporales, que pudieran dar origen a alguna hipótesis.

  ⍰ Ocaña-Riola, R., et al. (2016). Geographical and Temporal Variations in Female Breast Cancer Mortality in the Municipalities of Andalusia (Southern Spain). International journal of environmental research and public health, 13(11), 1162.

- **Estudios de grupos múltiples**: Este es el tipo de estudio ecológico más común. Se evalúa la asociación entre los niveles de exposición promedio y la frecuencia de la enfermedad entre varios grupos. La fuente de datos suele ser las estadísticas de morbilidad y mortalidad rutinarias.

  ⍰ Vinikoor-Imler, L. C., et al. (2011). An ecologic analysis of county-level PM2.5 concentrations and lung cancer incidence and mortality. International journal of environmental research and public health, 8(6), 1865–1871.

- **Estudios de series temporales**: Se comparan las variaciones temporales de los niveles de exposición con otra serie de tiempo que refleja los cambios en la frecuencia de la enfermedad en la población de un área geográfica determinada.

  ⍰ Ballester, F., et al. (2001). Air pollution and emergency hospital admissions for cardiovascular diseases in Valencia, Spain.Journal of Epidemiology & Community Health, 55, 57-65.

- **Estudios mixtos**: En esta categoría se incluyen los estudios de series de tiempo combinadas con la evaluación de grupos múltiples.

  ⍰ Schneider, M. C., et al. (2017). Leptospirosis in Latin America: Exploring the first set of regional data. Revista Panamericana de Salud Pública, 41, 1.

Algunos autores presentan la perspectiva que mostramos en la siguiente tabla, para una comprensión más rápida de las distintas posibilidades de los diseños ecológicos:

Tipo	Diseño	Marco temporal
Transversal	A lo largo de diferentes comunidades	Mismo período
Series temporales	Dentro de la misma comunidad	A lo largo del tiempo
Descriptivo	A lo largo de diferentes comunidades o dentro de la misma comunidad	Un punto en el tiempo o a lo largo del tiempo

15.3. Análisis de estudios ecológicos

En un **análisis ecológico completo**, todas las variables son medidas ecológicas (exposición o exposiciones, enfermedad u otras variables incluidas), ya que la unidad de análisis es el grupo. Ello implica que desconocemos la distribución conjunta de cualquier combinación de variables a nivel individual.

En un **análisis ecológico parcial** de tres o más variables puede tenerse información de la distribución conjunta de alguna de las variables en cada grupo. Por ejemplo, en el estudio de incidencia de cáncer se conocen la edad y el sexo de los casos (información individual) pero la exposición derivada de la residencia en un área concreta (municipio) es información ecológica.

El **análisis multinivel** es un tipo especial de modelización que combina el análisis efectuado a dos o más niveles. Ejemplo: la modelización de la incidencia de cáncer incluyendo el sexo y la edad como variables explicativas, además de variables socio-demográficas.

Como podemos apreciar, tenemos muchas posibilidades para contemplar. En este capítulo, nos centraremos en aquellos estudios ecológicos completos donde el objetivo principal sea encontrar una relación entre la exposición y la enfermedad, es decir un ***estudio de grupos múltiples***.

La manera usual de evaluación de la asociación en estudios de grupos múltiples es mediante modelos lineales de regresión, de hecho algunos autores se refieren a estos estudios como "Estudios de correlación". Dependiendo del diseño y la distribución de los datos se pueden emplear otros modelos no lineales o no aditivos. Como las tasas de morbilidad y mortalidad en las regiones geográficas que se comparan comúnmente son eventos raros o que ocurren a bajas frecuencias, éstos semejan una distribución Poisson; así que la regresión de Poisson también puede ser usada.

En este capítulo abordaremos a modo de introducción el **modelo lineal general**, la **covarianza y correlación**(03_covarianza_correlacion.qmd) para luego entender los modelos de **regresión lineal simple, análisis de la varianza** y **regresión lineal múltiple**, ampliamente usados en el análisis de los estudios ecológicos, aunque no exclusivos de ellos.

16. Introducción al modelado estadístico

16.1. Introducción

Uno de los propósitos fundamentales del modelado estadístico es explicar la realidad de la manera más “simple” posible, centrándose en su esencia y omitiendo elementos cuya variabilidad podría introducir ruido en la interpretación de los fenómenos. Existen eventos en la naturaleza que pueden ser explicados con exactitud si se cuenta con los datos adecuados. Por ejemplo, el cálculo del volumen de un cubo o la predicción de la trayectoria de un objeto en caída libre bajo condiciones ideales. Los modelos que explican estos fenómenos se denominan **modelos determinísticos**.

Sin embargo, cuando se estudian fenómenos en sistemas complejos, la situación se vuelve más desafiante debido a la influencia de factores externos que impiden explicar completamente la variable dependiente a partir de otras variables. Para abordar esta incertidumbre, se emplean modelos que incorporan un componente de error, conocidos como **modelos probabilísticos**. Estos modelos constituyen la base de la estadística inferencial y se utilizan en el análisis de regresión para describir y predecir relaciones entre variables.

16.1.1. Modelos determinísticos

En los modelos determinísticos, se asume una relación exacta entre las variables. Esto significa que tanto los parámetros como las variables son conocidos sin error, y el modelo genera siempre el mismo resultado para un conjunto de datos dado. Sin embargo, estos modelos no consideran la variabilidad causada por el azar o la incertidumbre.

Un ejemplo clásico de modelos determinísticos son los **modelos SIR** (Susceptibles, Infectados, Recuperados), empleados para estudiar la propagación de enfermedades en poblaciones cerradas. Estos modelos asumen tasas de transmisión y recuperación constantes, ignorando la variabilidad individual y otros factores externos.

16.1.2. Modelos probabilísticos o de regresión

Los modelos probabilísticos asumen que los fenómenos observados no son completamente predecibles, ya que ciertas variables están sujetas al azar. Estos modelos no generan un único resultado, sino una **distribución de resultados** expresada en términos de probabilidades.

Un modelo de regresión describe la relación entre la variable dependiente (Y) y una o más variables independientes (X), incorporando un **componente sistemático** (función del modelo) y un **componente aleatorio** (residuo o error residual).

Matemáticamente, esto se representa como:

$$Y = \underbrace{\beta_0 + \beta_1 X_1}_{Componente\ sistemtico} + \underbrace{\epsilon}_{Componente\ aleatorio}$$

La función puede ser lineal o no lineal según la naturaleza y distribución de la variable dependiente. El componente aleatorio es esa parte de la variación de Y que no puede ser totalmente explicada por la variación de la/s variable/s independiente/s. En algunos casos el error tendrá valor positivo y en otros tendrá valor negativo. El promedio del error es igual a 0.

Una vez establecida esta función estaremos en condiciones de:

- Comprender cómo se comporta la variable respuesta Y en función de la/s variable/s independientes (X).
- Estimar o predecir el valor de Y para determinados valores de X.
- Calcular intervalos de confianza para estas estimaciones o predicciones.

Los modelos probabilísticos pueden clasificarse en:

- **Paramétricos**: Asumen que los datos provienen de una distribución específica con un conjunto fijo de parámetros (por ejemplo, media, varianza).
- **No paramétricos**: Se utilizan cuando la distribución subyacente de los datos es desconocida o no sigue un patrón estándar.

16.2. Modelo lineal general

El modelo lineal general (MLG) describe la relación lineal entre una variable respuesta numérica con distribución normal (Y) y una o más variables independientes. Se expresa matemáticamente como:

$$Y_i = X_i\beta + \epsilon_i$$

Donde:

- Y_i: Vector de valores de la variable dependiente (también llamada variable respuesta o resultado), que representa el efecto observado de un proceso o interacción causal.
- X_i: Matriz de diseño que contiene las variables independientes (variables explicativas o predictores), las cuales representan la influencia medida sobre la variable dependiente.
- β: Vector de coeficientes (parámetros), que cuantifican la magnitud y dirección de la influencia de cada predictor.
- ϵ_i: Error aleatorio, que refleja la variabilidad no explicada por las variables independientes. Este componente puede deberse a diferencias individuales, errores de medición o calibración, y puede controlarse aumentando el tamaño muestral.

Casos específicos del MLG

- **Regresión lineal simple**: Examina la relación entre Y y una variable independiente numérica.
- **Análisis de la varianza (ANOVA)**: Evalúa las diferencias en la media de Y respecto a los niveles de una variable categórica.
- **Regresión lineal múltiple**: Explora la relación entre Y y dos o más variables independientes, que pueden ser numéricas y/o categóricas.

⚠ La principal fuente del “error” se debe a la variabilidad entre individuos propia de la naturaleza (error aleatorio). Puede haber otras fuentes de error, incluso no detectadas y que deben ser tenidas en cuenta, como pueden ser errores en la medición, calibración o incluso por una mala elección del método.

16.2.1. Supuestos del MLG:

Para garantizar la validez del modelo, deben cumplirse los siguientes supuestos:

1. **Linealidad:** Relación lineal entre la variable dependiente e independientes.
2. **Independencia:** Las observaciones deben ser independientes.
3. **Homoscedasticidad:** La varianza de los errores debe ser constante.
4. **Normalidad de los errores:** Los errores deben seguir una distribución normal.

Si alguno de estos supuestos se viola, podría ser necesario aplicar transformaciones a los datos, emplear técnicas alternativas o ajustar el modelo según corresponda.

16.3. Construcción del MLG en R

Para ajustar un MLG en R, se utiliza la función `lm()` cuyas letras vienen de *linear models* (modelos lineales), ingresando los argumentos el formato fórmula. Esto significa especificar primero el nombre de la variable dependiente y luego la variable independiente.

La estructura sintáctica es:

```
lm(variable_dependiente ~ variable_independiente)
```

La función muestra resultados básicos, tales como la relación entre las variables que son parte del modelo y los coeficientes. Habitualmente `lm()` suele asignarse a un objeto de regresión para que contenga todos los resultados producto del ajuste:

```
modelo <- lm(variable_dependiente ~ variable_independiente, data = datos)
```

Todos los ajustes de modelos lineales que produce la función `lm()` tienen la forma de una lista de 12 componentes. Sus componentes pueden ser llamados en resúmenes más completos, usando la función:

```
summary(modelo)
```

La salida de esta función incluye:

- `Call`: Fórmula del modelo.
- `Residuals`: Distribución de los residuos (mínimo, máximo, mediana y percentiles 25-75).
- `Coefficients`: Incluye el intercepto y la pendiente, junto con errores estándar, valores *t* y *p*-valores asociados.

- `Residual standard error`: Error estándar de los residuos con sus grados de libertad.
- `Multiple R-squared`: Coeficiente de determinación R^2.
- `Adjusted R-squared`: Coeficiente R^2 ajustado.
- `F-statistic`: Estadístico F y su *p*-valor asociado.

Además, la salida de `summary(modelo)` utiliza asteriscos (`*`) para indicar la significación de los coeficientes. Debajo de la tabla de coeficientes se encuentra la referencia del significado de los códigos, que van desde el 0 hasta el 1 como posible resultado del valor de probabilidad, y donde:

Código	Rango
***	0 a 0,001
**	0,001 a 0,01
*	0,01 a 0,05
.	0,05 a 0,1
	0,1 a 1

Los elementos de la lista también pueden llamarse de forma independiente, siendo los más importantes:

- `coefficients`: Vector con los coeficientes del modelo, pueden visualizarse usando alguno de los siguientes comandos:

  ```
  modelo$coefficients

  coef(modelo)
  ```

- `residuals`: Los residuos o residuales para cada valor que surgen de la diferencia entre los valores predictivos calculados por el modelo y los valores reales. Se visualizan desde el objeto resultado de la regresión:

  ```
  modelo$residuals
  ```

  O bien usando el comando:

  ```
  resid(modelo)
  ```

- `fitted.values`: Los valores calculados por el modelo en base a los datos existentes en la variable independiente. Los encontramos en:

  ```
  modelo$fitted.values
  ```

  O usando la función:

  ```
  fitted(modelo)
  ```

- Otra función interesante para el análisis del objeto de regresión es `confint()` que calcula los intervalos de confianza de los coeficientes o parámetros del modelo de regresión. Para este modelo la línea de ejecución de la función es:

  ```
  confint(modelo)
  ```

  Si agregamos la función `round()` podemos redondear los valores con la cantidad de decimales que necesitemos.

```
round(confint(mode lo2) )
```

En forma predeterminada los IC se calculan al 95 %, pero mediante el argumento `level` podemos modificarlo, por ejemplo al 99 %:

```
confint(modelo, level = 0.99)
```

17. Covarianza y correlación

17.1. Introducción

Una parte fundamental del análisis epidemiológico es detectar relación entre dos variables numéricas. Este análisis permite identificar patrones, tendencias y posibles asociaciones. Por ejemplo:

- Presión sanguínea y edad
- Estatura y peso
- Concentración de un medicamento y frecuencia cardíaca

Establecer estas relaciones facilita la identificación de factores de riesgo y/o la planificación de intervenciones. Para ello, se emplean dos herramientas estadísticas clave: la **covarianza** y la **correlación**.

- **Covarianza**: Mide si dos variables aumentan o disminuyen simultáneamente.
- **Correlación**: Evalúa la dirección e intensidad de la relación, lo que permite interpretar y comparar con mayor claridad.

17.2. Covarianza

La **covarianza** es una medida estadística que indica el grado de variabilidad conjunta de dos variables cuantitativas X e Y. Matemáticamente se define como:

$$S_{XY} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

donde:

- x_i e y_i son los valores individuales de cada variable.
- $\bar{x}_i$ e $\bar{y}_i$ son las medias de las variables x e y.
- n es el número de pares de datos.

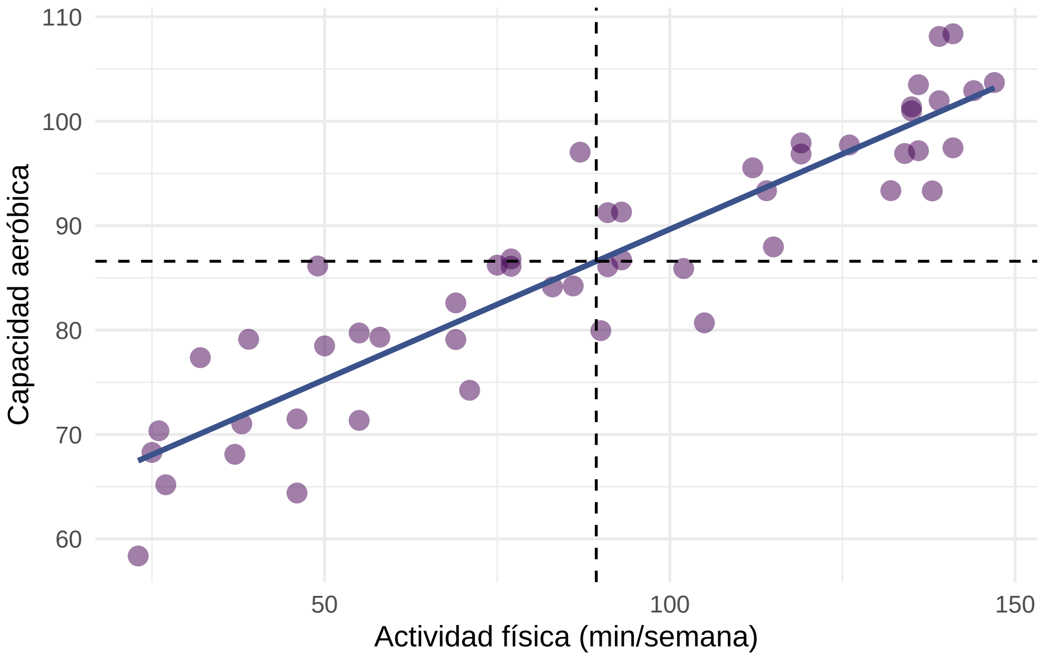
17.2.1. Representación gráfica

La covarianza suele representarse gráficamente usando **diagramas de dispersión** (*scatterplots*), que muestran la dirección de la asociación y permiten detectar valores extremos (*outliers*). Cuando examinamos un diagrama de dispersión, debemos observar si existe un patrón general en los puntos graficados y señalar su dirección. Es decir, mientras una variable se incrementa, ¿la otra parece aumentar o disminuir? Además, debemos observar si hay datos distantes, que son puntos que se ubican muy lejos de todos los demás.

Consideremos la nube de puntos formadas por las n parejas de datos (x_i, y_i). El centro de gravedad de esta nube de puntos es ($\bar{x}$, $\bar{y}$). Trasladamos los ejes XY al nuevo centro de coordenadas. Los puntos que se encuentran en el **primer y tercer cuadrante contribuyen positivamente** al valor de S_{XY}, y los que se encuentran en el **segundo y el cuarto lo hacen negativamente**.

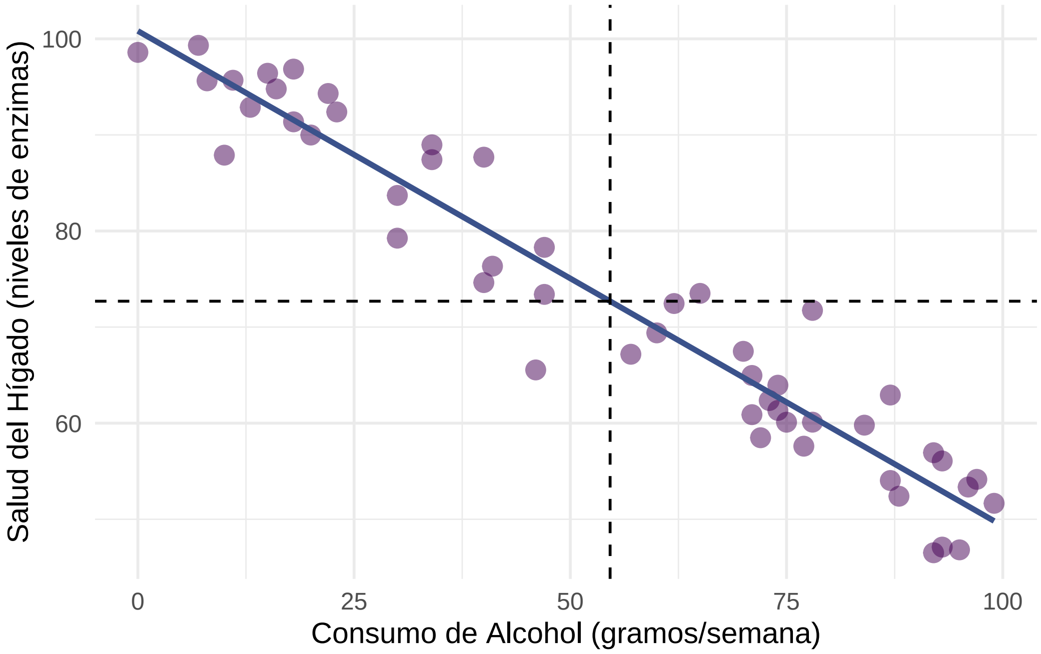
Covarianza positiva

El siguiente gráfico muestra una relación positiva entre minutos de actividad física semanal (X) y capacidad aeróbica (Y). Podemos observar que, cuando crece X también crece Y y casi todos los puntos pertenecen a los cuadrantes primero y tercero ($S_{XY} > 0$).



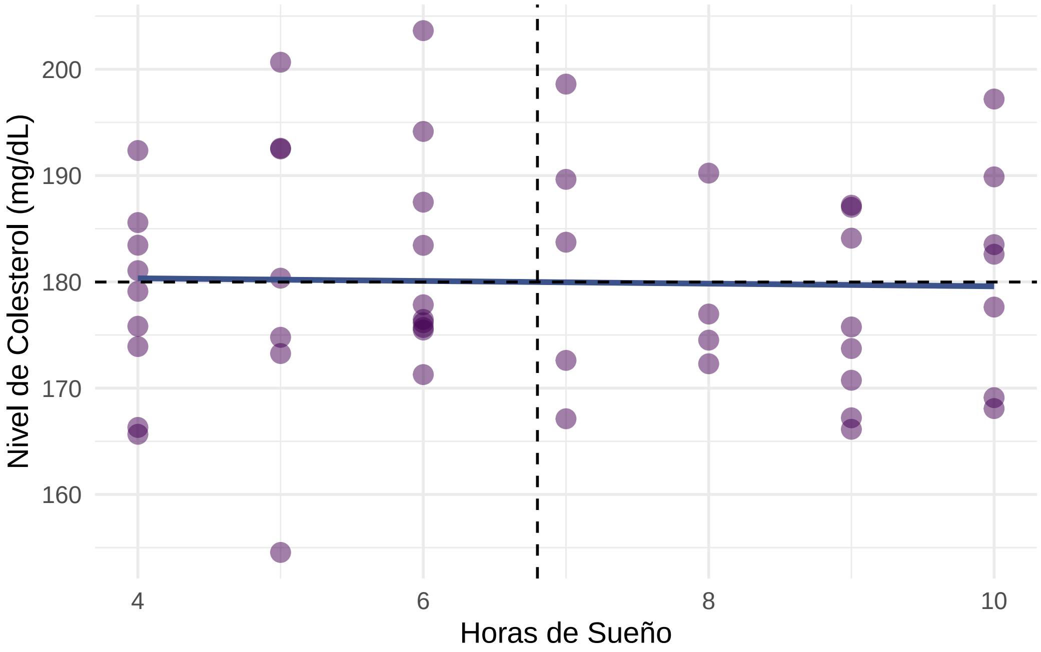
Covarianza negativa

En este gráfico, a medida que aumenta el consumo de alcohol semanal (X), disminuyen los niveles de enzimas hepáticas (Y). Casi todos los puntos pertenecen a los cuadrantes segundo y cuarto ($S_{XY} < 0$).



Covarianza cercana a cero

En el diagrama de dispersión se puede observar que los cambios en las horas de sueño (X) no tienen relación con el nivel de colesterol en sangre (Y). No se observa un patrón, sino que los puntos se reparten de modo más o menos homogéneo entre los cuadrantes ($S_{XY} \simeq 0$).



17.2.2. Limitaciones de la covarianza

La covarianza está afectada por las unidades en las que se miden las variables, lo que puede dificultar la interpretación de su magnitud. Para resolver este problema, es necesario utilizar una medida que no esté afectada por las unidades de medida de las variables: la **correlación.**

17.3. Correlación

La correlación trata de establecer la relación o dependencia que existe entre las dos variables que intervienen en una distribución bidimensional. Es decir, determinar si los cambios en una de las variables influyen en los cambios de la otra. En caso de que suceda, diremos que las variables están correlacionadas o que hay correlación entre ellas.

La correlación, como cuantificación del grado de relación que hay entre dos variables, es un valor entre -1 y +1, pasando, por el cero. Hay, por lo tanto, correlaciones positivas y negativas. El **signo** es, pues, el primer elemento básico a tener en cuenta.

- **Correlación positiva**: existe relación directa; ambas variables aumentan o disminuyen simultáneamente. El coeficiente de correlación tiene valores entre 0 y 1.
- **Correlación negativa**: existe relación inversa; una variable aumenta mientras la otra disminuye. El coeficiente de correlación tiene valores entre -1 y 0.
- **Correlación cercana a cero**: no hay relación lineal, aunque podría existir una relación no lineal.

Lo segundo a tener en cuenta en la correlación es la **magnitud**. Y esto lo marca el valor absoluto de la correlación. En la magnitud se valora la correlación sin el signo, valorando la magnitud del número puro. Esto significa que cuanto más cerca estemos de los extremos del intervalo de valores posibles: -1 y +1, más correlación tenemos.

La correlación más usada para variables cuantitativas es la **correlación de Pearson**. Es especialmente apropiada cuando la distribución de las variables es la normal. Si no se cumple la normalidad o si las variables son ordinales es más apropiado usar la **correlación de Spearman** o la **correlación de Kendall**.

17.3.1. Correlación de Pearson

Mide la relación lineal entre dos variables, eliminando la influencia de las unidades de medida. Es una medida adimensional, obtenida al estandarizar la covarianza entre dos variables X e Y:

$$r = \frac{Cov_{XY}}{S_x S_y}$$

donde:

- Cov_{XY} es la covarianza entre las variables X e Y.
- S_x y S_y son las desviaciones estándar de las variables X e Y.

El coeficiente de correlación r entonces se interpreta como:

- $0 < r \leq 1$: correlación positiva; ambas variables aumentan o disminuyen simultáneamente.

- $-1 \leq r > 0$: correlación negativa; una variable aumenta mientras la otra disminuye.

- $r \approx 0$: no hay relación lineal, aunque podría existir una relación no lineal.

¿A partir de qué valores de r se considera que hay *"buena correlación"*? La respuesta no es simple. Hay que tener en cuenta la presencia de observaciones anómalas y si la varianza se mantiene homogénea. En reglas generales se acepta que:

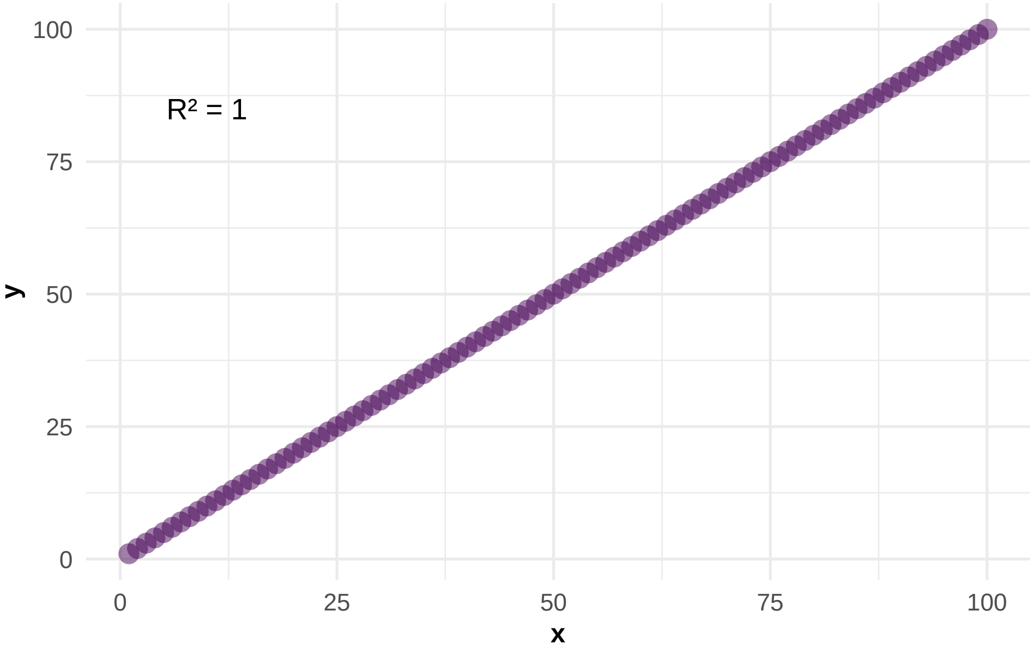
- $r \approx \pm 1$: correlación perfecta.

- $r \geq \pm 0.7$: correlación fuerte.

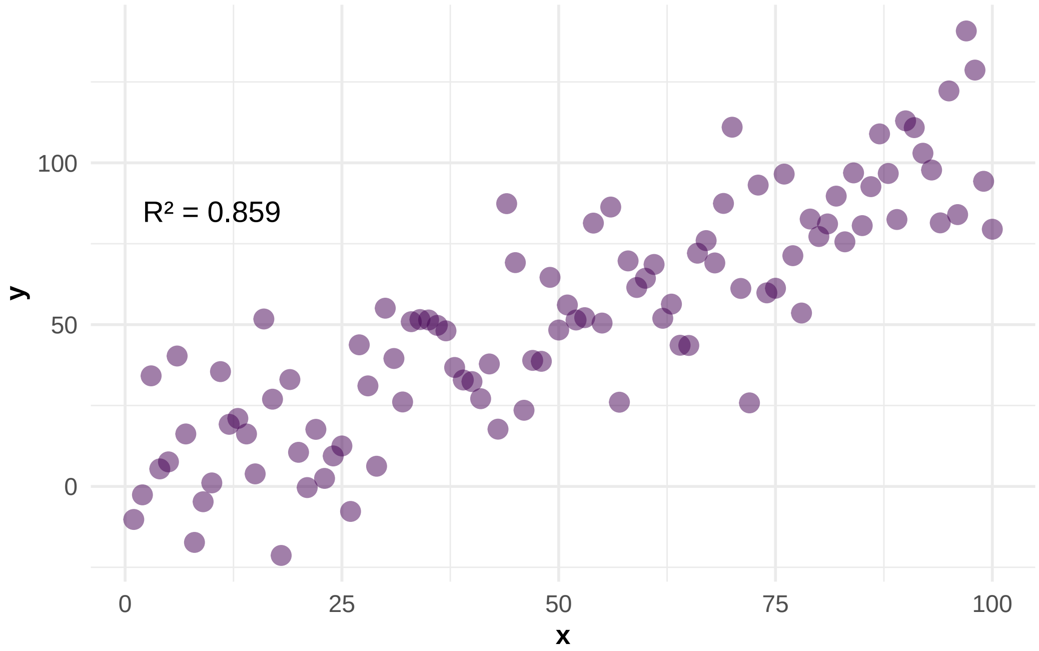
- $\pm 0.4 \leq r < \pm 0.7$: correlación moderada.

- $r \leq \pm 0.4$: correlación débil.

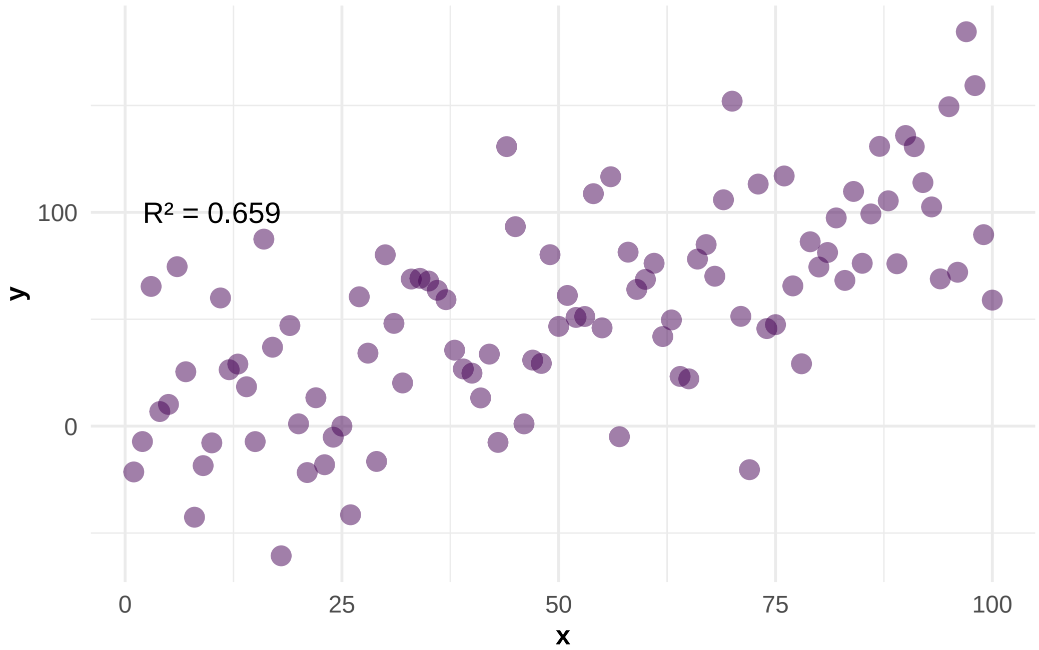
Correlación positiva perfecta entre X e Y



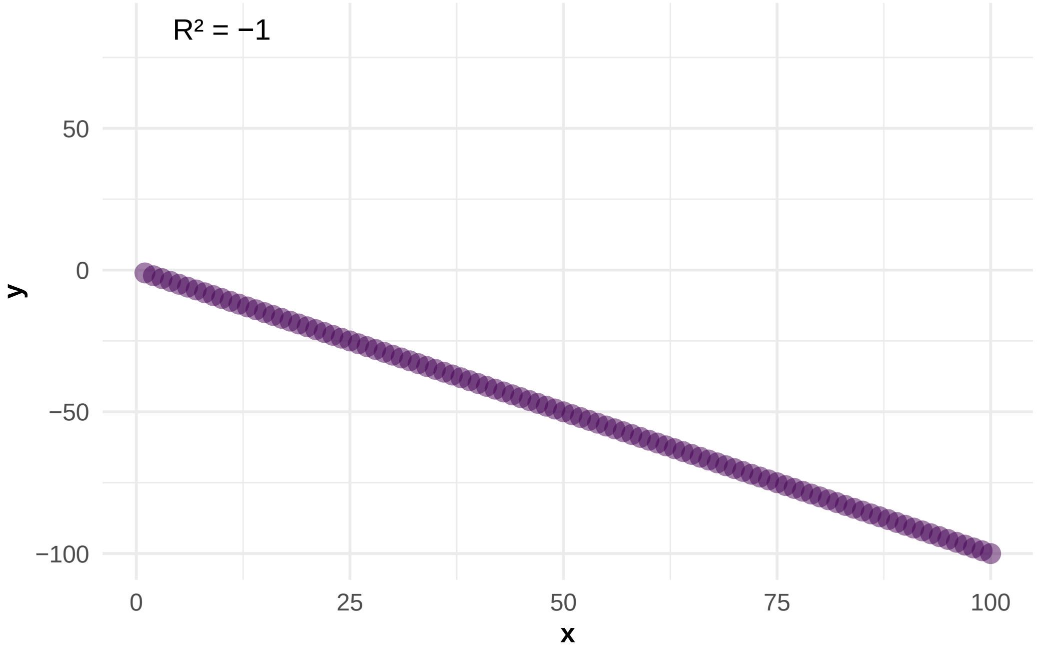
Correlación positiva fuerte entre X e Y



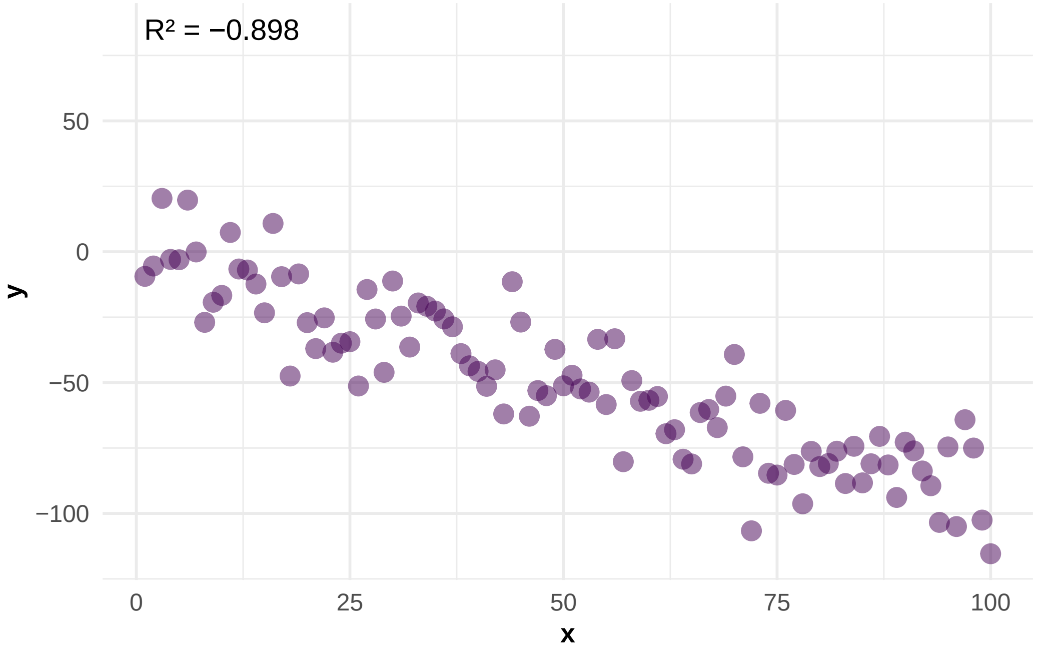
Correlación positiva moderada entre X e Y



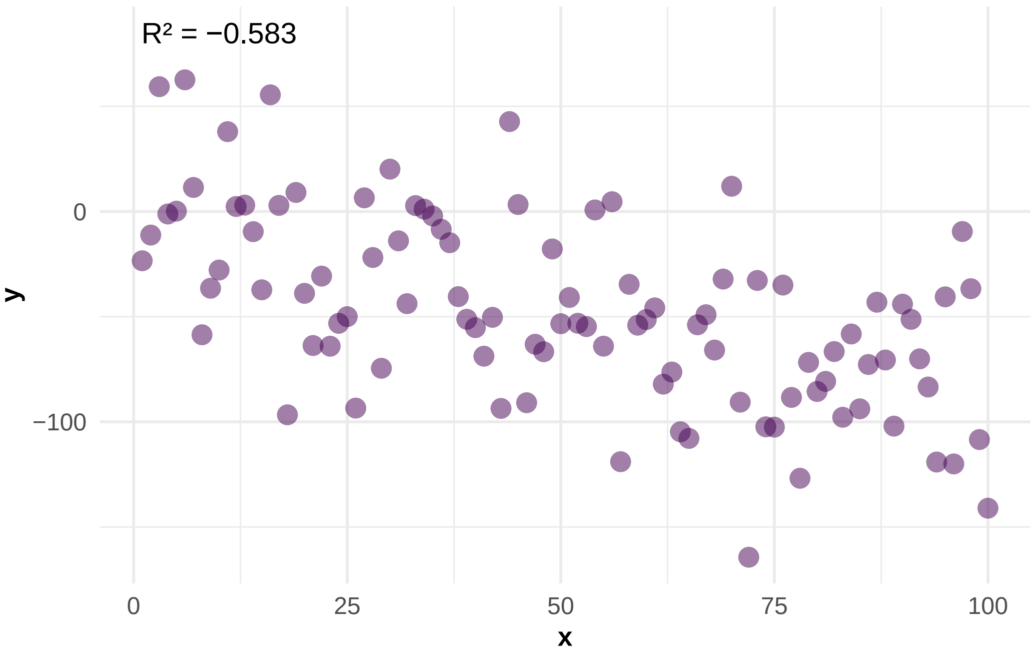
Correlación negativa perfecta entre X e Y



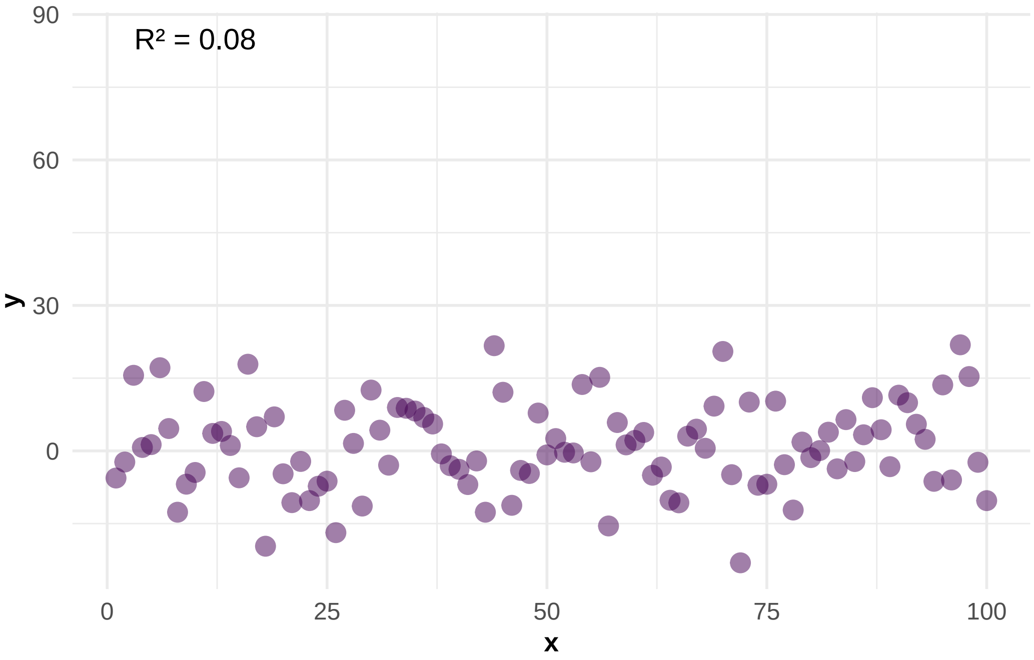
Correlación negativa fuerte entre X e Y



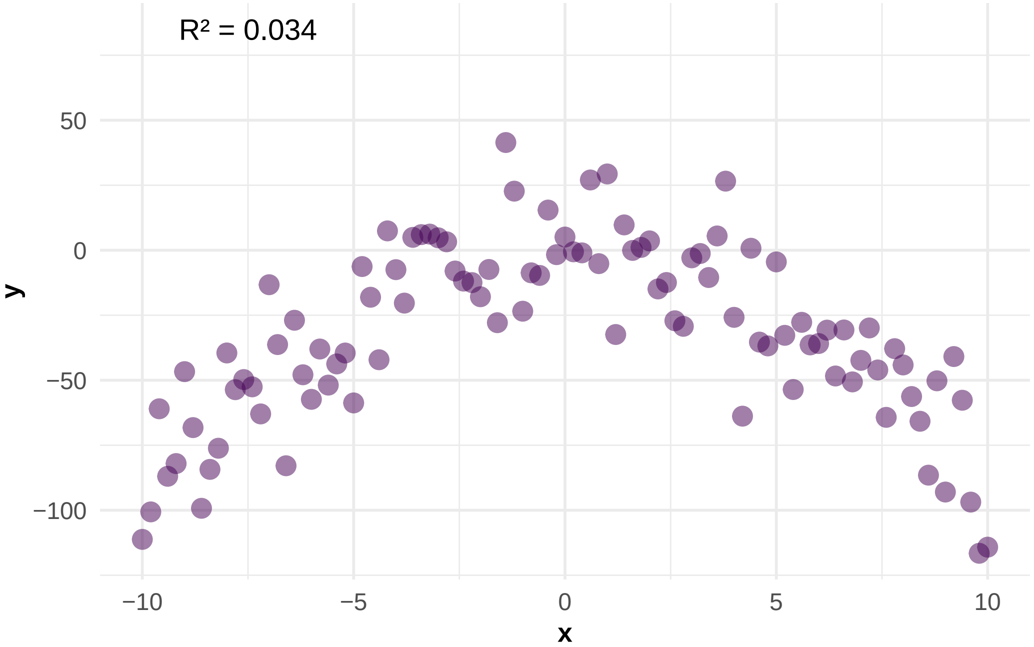
Correlación negativa moderada entre X e Y



X e Y no correlacionadas



Relación no lineal entre X e Y



Correlogramas

Los correlogramas son gráficos que muestra la relación entre cada par de variables numéricas en un conjunto de datos. Se utiliza para analizar la correlación entre variables, identificar patrones y controlar la aleatoriedad de los datos. Existen diversos paquetes en R que permiten generar correlogramas, como `correlation`, que forma parte del ecosistema de paquetes `easystats` (Lüdecke et al. 2022), o `GGally`(Schloerke et al. 2025) que usa como base el paquete `ggplot2`.

17.3.2. Correlación de Spearman

Mide la correlación entre dos variables basada en los **rangos** (orden) de los valores. Se utiliza cuando los datos **no presentan una relación lineal** o tienen una relación monótona. Matemáticamente se expresa como:

$$\rho = \frac{1 - 6\sum d_i^2}{n(n^2 - 1)}$$

donde:

- d_i es la diferencia en los rangos de cada par de observaciones.
- n es el número de observaciones.

17.3.3. Correlación de Kendall

Mide la relación ordinal entre dos variables numéricas en base a la **concordancia** y **discordancia** entre pares. Se utiliza cuando hay datos con valores repetidos o la muestra es pequeña. Es más robusta frente a datos con valores atípicos en comparación con Spearman. Matemáticamente se expresa como:

$$\tau = \frac{n_c - n_d}{n(n-1)/2}$$

donde:

- n_c y n_d son los pares concordantes o discordantes.

17.4. Ejemplo práctico en R

Tomemos como ejemplo la base `cancer_USA.txt` que contiene información sobre la tasa de mortalidad por cáncer cada 100.000 habitantes de distintos condados de la Costa Este de Estados Unidos.

Comenzaremos por cargar los paquetes necesarios para el análisis:

```r
# Gráficos de correlación
library(easystats)

library(GGally)

# Manejo de datos
library(tidyverse)
```

Cargamos los datos y exploramos su estructura:

```r
# Carga de datos
datos <- read_csv2("datos/cancer_USA.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 213
Columns: 10
$ condado            <chr> "Belknap County", "Carroll County", "Cheshire Count~
$ estado             <chr> "New Hampshire", "New Hampshire", "New Hampshire", ~
$ tasa_mortalidad    <dbl> 182.6, 168.8, 162.8, 181.6, 155.0, 163.2, 173.1, 17~
$ mediana_edad       <dbl> 46.1, 50.3, 42.0, 48.1, 41.9, 40.1, 42.6, 43.5, 37.~
$ mediana_edad_cat   <chr> "45+ años", "45+ años", "36-39 años", "45+ años", "~
$ mediana_ingresos   <dbl> 59831, 57556, 56008, 42491, 56353, 71233, 62429, 79~
$ pct_pobreza        <dbl> 10.0, 10.7, 11.8, 14.9, 11.6, 8.7, 9.5, 6.1, 11.8, ~
$ pct_salud_publica  <dbl> 16.8, 13.4, 12.7, 22.5, 14.0, 12.7, 13.1, 9.0, 12.4~
$ pct_sec_incompleta <dbl> 15.4, 14.8, 6.1, 13.2, 6.6, 12.9, 10.9, 13.5, 6.1, ~
$ pct_desempleo      <dbl> 5.3, 5.8, 6.1, 6.9, 5.0, 5.9, 5.2, 5.6, 6.5, 5.8, 1~
```

La base de datos tiene 213 observaciones y 10 variables. La variable dependiente es `tasa_mortalidad` y hay 3 variables independientes categóricas y 6 variables independientes numéricas o continuas:

- `tasa_mortalidad`: Tasa de mortalidad por cáncer a nivel de condado (datos agregados).
- `pct_salud_publica`: Porcentaje de personas con cobertura exclusiva de salud pública en el condado.
- `pct_pobreza`: Porcentaje de personas por debajo de la línea de pobreza en el condado.
- `pct_sec_incompleta`: Porcentaje de personas con estudios secundarios incompletos en el condado.
- `mediana_edad`: Mediana de la edad de fallecimiento por cáncer en el condado.
- `mediana_edad_cat`: Versión categorizada de la mediana de edad al fallecer por cáncer en el condado.
- `condado`: Condado de residencia.
- `estado`: Estado de residencia.

La función `cor()` estima la correlación entre dos variables. El método predeterminado devuelve la correlación de Pearson y puede modificarse el argumento `method` para obtener la correlación de Kendall o Spearman:

```r
cor(datos$tasa_mortalidad, datos$pct_pobreza, method = "pearson")
```

```
[1] 0.3301532
```

La función `cor.test()` determina si la prueba de correlación de Pearson calculada es significativa mediante el estadístico t de Student:

```r
cor.test(datos$tasa_mortalidad, datos$pct_pobreza)
```

```

	Pearson's product-moment correlation

data:  datos$tasa_mortalidad and datos$pct_pobreza
t = 5.0806, df = 211, p-value = 8.266e-07
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.2048117 0.4448414
sample estimates:
      cor 
0.3301532 
```

Los resultados de la función son:

- El valor del estadístico t
- El valor de p para el estadístico
- El valor de la correlación de Pearson
- Los intervalos de confianza para la correlación

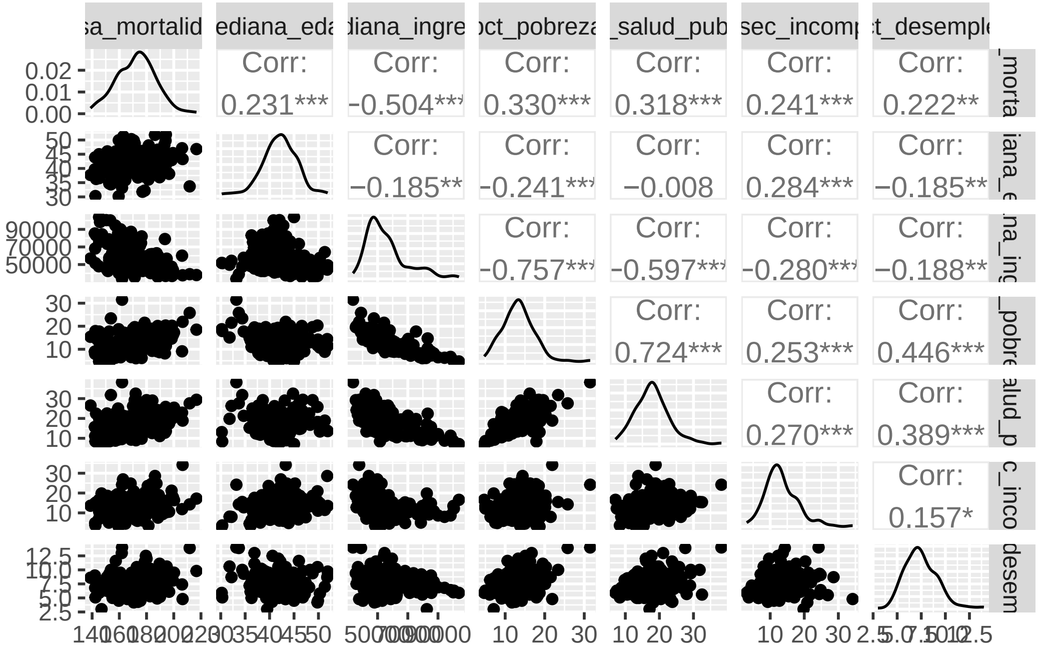
Los argumentos predeterminados para la función `cor.test()` son:

- `alternative = "two.sided"` - indica la hipotesis alternativa, también puede ser `"greater"` para asociación positiva, `"less"` para asociación negativa.
- `conf.level = 0.95` - determina el nivel de confianza (se puede modificar).
- `method = "pearson"` - especifíca el tipo de test de correlación. También permite `"kendall"` o `"spearman"`.

17.4.1. Gráficos de correlación

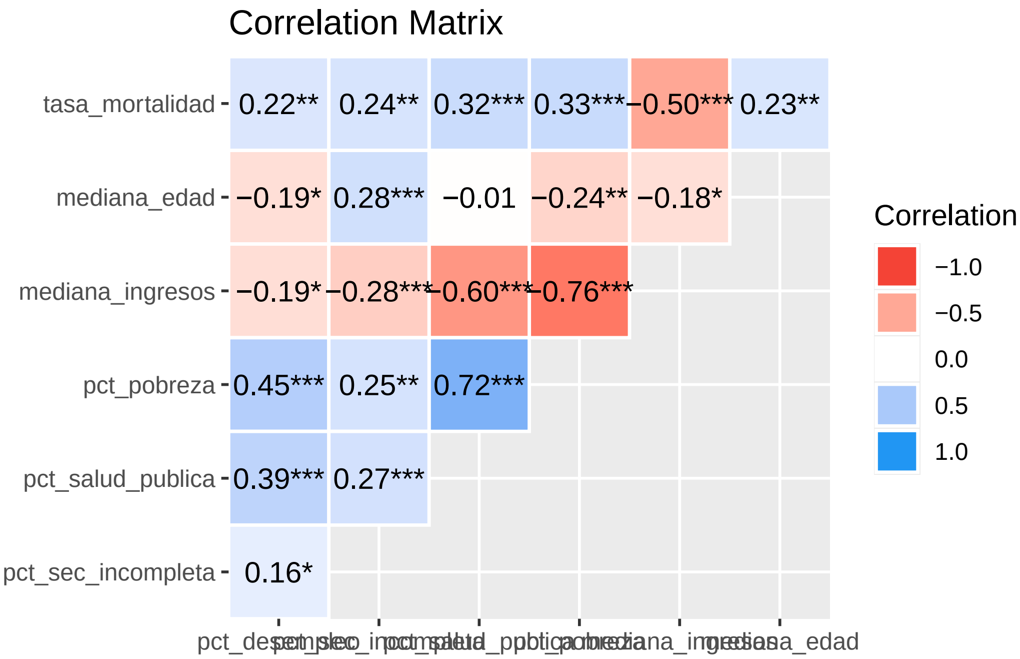
Podemos generar diagramas de dispersión para cada par de variables numéricas usando la función `ggpairs()` del paquete `GGally` (Schloerke et al. 2025):

```r
datos |>
  # selecciono soloamente las variables numéricas
  select(where(is.numeric)) |>

  # diagramas de dispersión
  ggpairs()
```



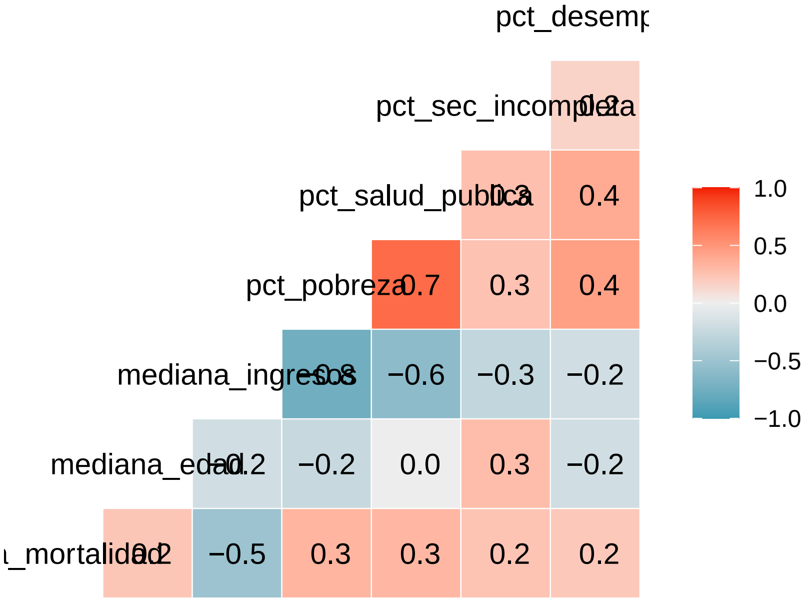
Podemos generar los correlogramas con el paquete `correlation` usando el siguiente código:

```r
datos |>
  correlation() |>
  summary() |>
  plot()
```



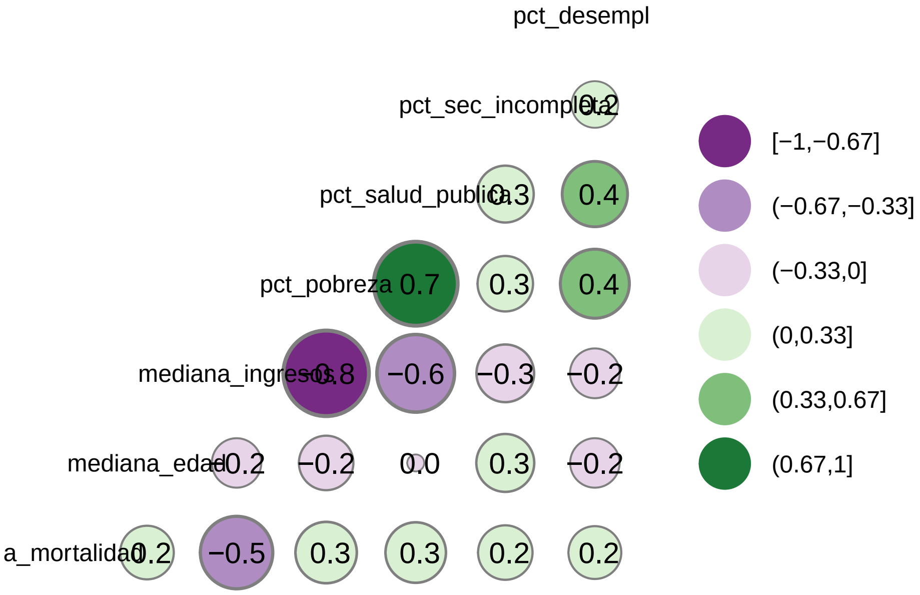
El comando `ggcorr()` del paquete `GGally` también nos permite generar correlogramas:

```
datos |>
  # selecciono soloamente las variables numéricas
  select(where(is.numeric)) |>

  # correlograma
  ggcorr(label = T)
```



Podemos hacerlos más atractivos visualmente modificando los argumentos `nbreaks`, `geom`, `size`, `max_size` y `palette`:

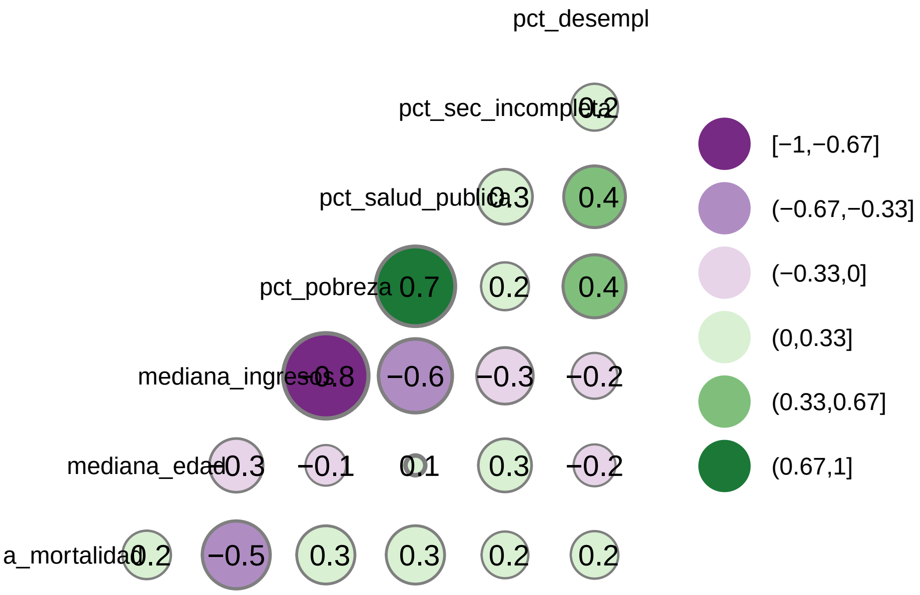
```r
datos |>
  # selecciono soloamente las variables numéricas
  select(where(is.numeric)) |>

  # correlograma
  ggcorr(
    label = T, # coeficientes
    size = 3, # tamaño de texto
    geom = "circle", # forma del gráfico
    max_size = 15, # tamaño máximo de las formas
    nbreaks = 6, # número de categorías de color
    palette = "PRGn"
  ) # color de las formas
```



Observamos que la variable dependiente (`tasa_mortalidad`) tiene correlación negativa moderada con `mediana_ingresos`, también que existe una correlación negativa fuerte entre `pct_pobreza` y `mediana_ingresos` y moderada con `pct_salud_publica`. Las correlaciones entre las demás variables son débiles o nulas.

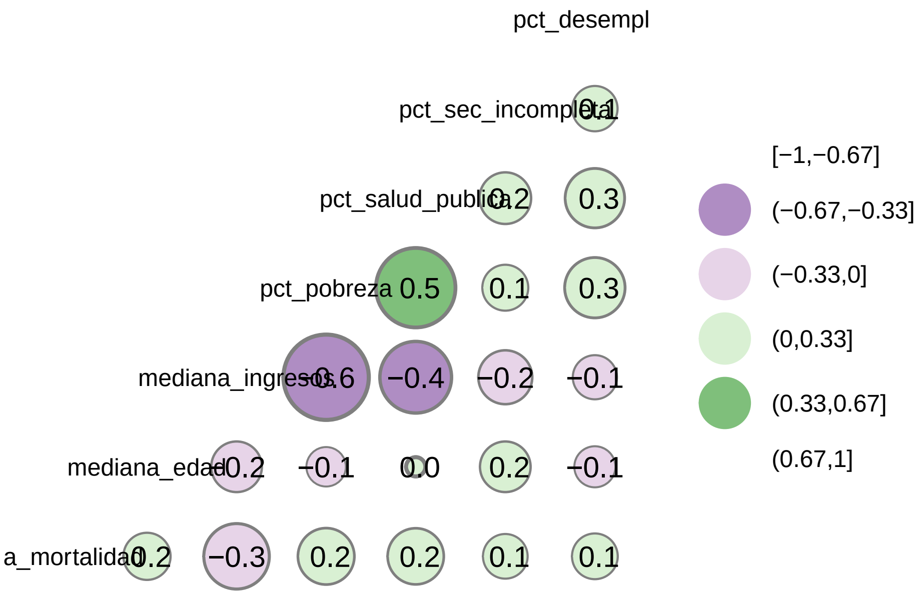
Para visualizar el correlograma con el coeficiente de Spearman, añadimos el argumento `method = c("pairwise", "spearman")` al código del ejemplo anterior:

```r
datos |>
  # selecciono soloamente las variables numéricas
  select(where(is.numeric)) |>

  # correlograma
  ggcorr(
    label = T, # coeficientes
    size = 3, # tamaño de texto
    geom = "circle", # forma del gráfico
    max_size = 15, # tamaño máximo de las formas
    nbreaks = 6, # número de categorías de color
    palette = "PRGn", # color de las formas
    method = c("pairwise", "spearman")
  ) # coeficiente de Spearman
```



De igual modo, si queremos aplicar el coeficiente de Spearman, añadimos el argumento `method = c("pairwise", "kendall")`:

```
datos |>
  # selecciono soloamente las variables numéricas
  select(where(is.numeric)) |>

  # correlograma
  ggcorr(
    label = T, # coeficientes
    size = 3, # tamaño de texto
    geom = "circle", # forma del gráfico
    max_size = 15, # tamaño máximo de las formas
    nbreaks = 6, # número de categorías de color
    palette = "PRGn", # color de las formas
    method = c("pairwise", "kendall")
  ) # coeficiente de Kendall
```



Para explicar la forma de esta correlación e incluso predecir los valores que puede alcanzar una variable dependiente numérica en función de una variable independiente también numérica podemos utilizar la **regresión lineal simple**.

18. Regresión lineal simple

18.1. Introducción

Para entender el modelo de regresión lineal simple, volvamos al ejemplo del peso en niñas menores de 6 meses de la **unidad anterior**, donde habíamos trazado una recta que nos ilustraba la relación lineal del peso en función de la edad.

Según el modelo estadístico para la función lineal de Y según X:

$$Y(X) = \beta_0 + \beta_1 + X_1$$

Hemos ajustado un modelo cuyos parámetros son:

$$\hat{y} = b_0 + b_1 X_1$$

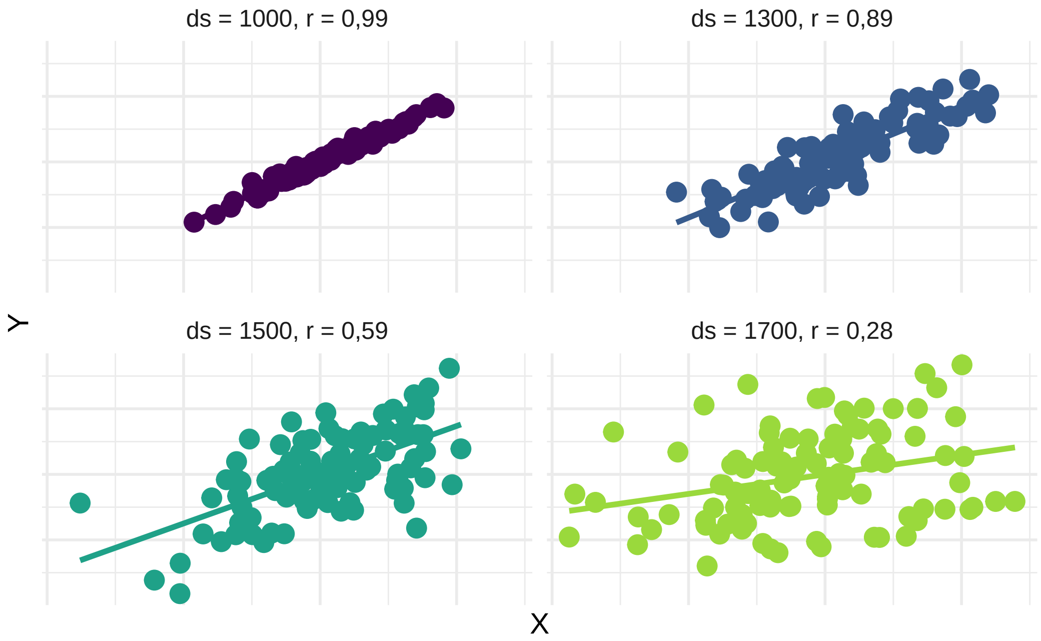
donde b_1 nos está indicando cuánto se modifica $\hat{y}$ por cada unidad de aumento de X_1.

$$\hat{y} = 2468,9 + 100,7\ edad\ (en\ semanas)$$

Se interpreta que por cada semana este grupo de lactantes ha aumentado en promedio 100 gramos, o que cada 1 mes (4 semanas) aumentan una media de 400 gramos.

Podemos observar que hay una relación lineal y que esta relación no es perfecta. Existe cierta dispersión entre los puntos sugiriendo que alguna variación en el peso no se asocia con un incremento de la edad (por ejemplo dos lactantes de 15 semanas. Tienen la misma edad y 1.600 grs de diferencia. Cabría preguntarse si esas niñas que se "alejan" tanto de la recta de regresión no tienen algún antecedente distinto del resto). Más adelante veremos cómo se interpretan esas diferentes distancias entre las observaciones y la recta de regresión.

Ahora vamos a concentrarnos en la relación entre la varianza de la muestra, a través del desvío estándar ($ds = \sqrt{varianza}$) y la magnitud de la asociación. Se muestran cuatro ejemplos en los cuáles se fue aumentando progresivamente el desvío estándar de los datos. Observen cómo a medida que aumenta la variabilidad entre los individuos va disminuyendo el coeficiente de correlación y el coeficiente b_1 (pendiente de la recta)



Podemos observar que cuanto mayor es la varianza en una muestra:

- Mayor es la variabilidad de y en torno a la recta de regresión
- Mayor es la imprecisión asociada a la estimativa de los parámetros de regresión

18.2. Presupuestos del modelo

Cuando planeamos realizar un análisis de regresión con un conjunto de datos es necesario saber que para que podamos plantearlo adecuadamente deben cumplirse ciertas condiciones, que llamaremos Presupuestos del modelo:

1. Independencia: los valores de y deben ser independientes unos de otros
2. Linealidad: la relación entre x e y debe ser una función lineal
3. Homocedasticidad: la varianza de y debe mantenerse constante para los distintos valores de x
4. Normalidad: y debe tener una distribución normal

¿Cómo se obtiene la recta de regresión? ¿Cómo se calculan los coeficientes de la regresión?

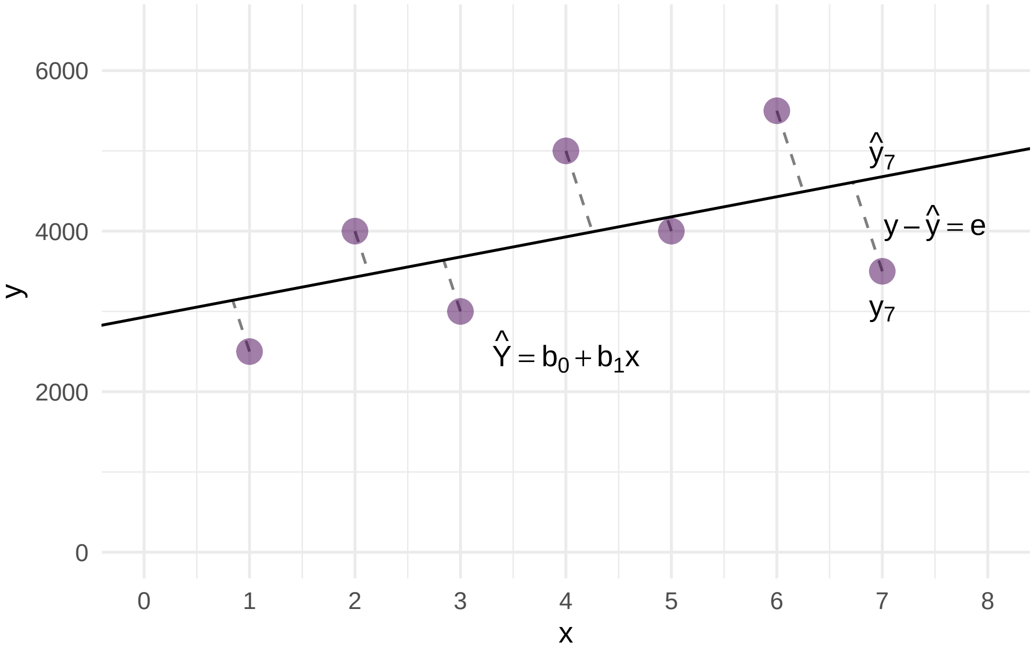
En el ejemplo del peso según edad en niñas menores de 6 meses, la idea es encontrar una función lineal (que gráficamente es una recta) que aplicada a los valores de x nos permita aproximar los valores de y. La ecuación de la recta que describe la relación entre x e y:

$$\hat{y} = b_0 + b_1 x$$

Por muy bueno que sea el modelo de regresión y e $\hat{y}$ rara vez coincidirán.

Entonces podríamos pensar que la mejor recta que permita predecir (o aproximar) los valores de y en función de x es aquella que minimice estos errores residuales (que algunos serán en más y otros serán en menos).

Gráficamente:



donde:

- $\hat{y}$ es la ecuación de la "mejor" recta que puede trazarse entre estos puntos
- b_0 es la ordenada al origen o constante, también llamada alfa. Es el punto donde la recta de regresión corta al eje de ordenadas.
- b_1 es la pendiente de la recta (más adelante veremos cuál es la interpretación de estos coeficientes).

Consideremos qué pasa en el caso de la niña 7. Veamos las distancias para este punto.

- y_7 es el valor "real" del peso de la niña 7.
- $\hat{y}_7$ es el valor estimado de y que obtendremos a través de la regresión.
- $y – \hat{y} = e$ (residuo o error residual) es el desvío de y del valor ajustado $\hat{y}$ en la ecuación de la regresión estimada.

Para poder operar con el valor de estos errores (ya que algunos tendrán valor positivo y otros valor negativo) se los eleva al cuadrado. Esta técnica se denomina **"método de los mínimos cuadrados"** y consiste en adoptar como estimativas de los parámetros de la regresión (o sea los coeficientes b_0 y b_1 y por ende la recta de regresión) los valores que minimizan la suma de los cuadrados de los residuos o error (**SCE**) para todas las observaciones de y. Lo podemos expresar así:

$$SCE = \sum \hat{e}^2 = \sum (y - \hat{y})^2$$

Sabíamos que:

$$\hat{y} = b_0 + b_1 x$$

Entonces si reemplazamos:

$$SCE = \sum_{i=1}^{i=n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{i=n} (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_1))^2$$

Es posible obtener los estimadores β_1 y β_0.

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}$$

Otra fórmula para β_1:

$$\hat{\beta}_1 = \frac{\sum x_i y_i - \frac{\sum x_i \sum y_i}{n}}{\sum x_i^2 - \frac{(\sum x_i)^2}{n}}$$

$$\hat{\beta}_0 = \bar{y} - \beta_1 \bar{x}$$

El método de los mínimos cuadrados fue creado por *Johann Carl Friedrich Gauss* (1777-1855). Tiene además la ventaja que el promedio de los errores residuales = 0 y que para cada estimación, la varianza del error es mínima.

Test de hipótesis para β_1 e Intervalo de Confianza 95 %

Como siempre que trabajamos con una muestra, será necesario aplicar los procesos de inferencia. Es por eso que los softwares ofrecen un test de hipótesis para el coeficiente.

La hipótesis nula podría entenderse como que x no logra explicar la variación de y (entonces la pendiente de la recta sería nula)

$$H_0 : \beta_1 = 0$$

Al calcular el modelo de regresión, todos los *software* estiman el coeficiente y el error estándar del mismo (SE) y testean el coeficiente.

Bondad de ajuste

Hasta ahora hemos aprendido a explicar la variación de y según la variación de x mediante un modelo en donde las desviaciones entre el valor observado ("real") y el estimado ("modelo de regresión") son las menores posibles.

Ahora debemos saber, según los datos que tenemos, cuán bueno es el modelo que ajustamos (qué capacidad tiene de explicar la variabilidad de y o, si lo quiero utilizar para realizar una predicción, cuánto se alejará mi valor estimado del verdadero, "real" valor de y). Esta evaluación la realizaremos mediante la descomposición de la varianza del modelo. Por definición la varianza o variabilidad total es la sumatoria de la diferencia entre cada valor de y con el promedio de y (elevado al cuadrado ya que hay valores negativos y positivos que si los sumamos se anularían).

La variabilidad total del modelo es la suma entre la variabilidad que logró explicar la regresión y la variabilidad residual.

$$\sum (y_i - \bar{y})^2 = \sum (\hat{y} - \bar{y})^2 + \sum (y_i - \hat{y}_i)^2$$

$$\frac{Suma\ de\ cuadrados}{totales\ (SCT)}\quad \frac{Suma\ de\ cuadrados}{de\ la\ regresion\ (SCR)}\quad \frac{Suma\ de\ cuadrados}{residuales\ (SCE)}$$

Cuanto mayor sea la variabilidad que logre explicar la regresión en relación a los residuos, tanto mejor será el modelo. Este es el fundamento para el cálculo del coeficiente de determinación (R^2)

$$R^2 = \frac{SCR}{SCT} = 1 - \frac{SCE}{SCT}$$

R^2 expresa la proporción de la variación total que logra explicar el modelo de regresión. Su valor oscila entre 0 y 1, es una cantidad adimensional.

- Cuando el ajuste es bueno R^2 será cercano a 1, cuando el ajuste es malo R^2 será cercano a 0.
- En la Regresión lineal simple el coeficiente de determinación (R^2) es igual al r de Pearson elevado al cuadrado.

Para visualizar simulaciones al respecto pueden visitar el siguiente enlace:

Viendo la teoría. Una introducción visual a probabilidad y estadística

18.3. Ejemplo práctico en R

Para llevar a cabo el análisis en R y presentar las funciones y paquetes que nos pueden ayudar en la tarea vamos a trabajar con el set de datos `cancer_USA.txt` que utilizamos para el ejemplo de covarianza y correlación.

Comenzaremos por cargar los paquetes necesarios para el análisis:

```r
# Tablas de coeficientes
library(gtsummary)

# Chequeo de supuestos
library(easystats)
library(lmtest)
library(nortest)

# Paletas aptas daltonismo
library(scico)

# Manejo de datos
library(tidyverse)
```

Cargamos los datos y revisamos la estructura de la tabla:

```r
# Carga datos
datos <- read_csv2("datos/cancer_USA.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 213
Columns: 10
$ condado            <chr> "Belknap County", "Carroll County", "Cheshire Count~
$ estado             <chr> "New Hampshire", "New Hampshire", "New Hampshire", ~
$ tasa_mortalidad    <dbl> 182.6, 168.8, 162.8, 181.6, 155.0, 163.2, 173.1, 17~
$ mediana_edad       <dbl> 46.1, 50.3, 42.0, 48.1, 41.9, 40.1, 42.6, 43.5, 37.~
$ mediana_edad_cat   <chr> "45+ años", "45+ años", "36-39 años", "45+ años", "~
$ mediana_ingresos   <dbl> 59831, 57556, 56008, 42491, 56353, 71233, 62429, 79~
$ pct_pobreza        <dbl> 10.0, 10.7, 11.8, 14.9, 11.6, 8.7, 9.5, 6.1, 11.8, ~
$ pct_salud_publica  <dbl> 16.8, 13.4, 12.7, 22.5, 14.0, 12.7, 13.1, 9.0, 12.4~
$ pct_sec_incompleta <dbl> 15.4, 14.8, 6.1, 13.2, 6.6, 12.9, 10.9, 13.5, 6.1, ~
$ pct_desempleo      <dbl> 5.3, 5.8, 6.1, 6.9, 5.0, 5.9, 5.2, 5.6, 6.5, 5.8, 1~
```

Recordemos que la base de datos tiene 213 observaciones y 10 variables y la variable dependiente es `tasa_mortalidad`. Para ejemplificar los pasos de una regresión lineal simple, evaluaremos la asociación entre la variable dependiente y `mediana_ingresos`.

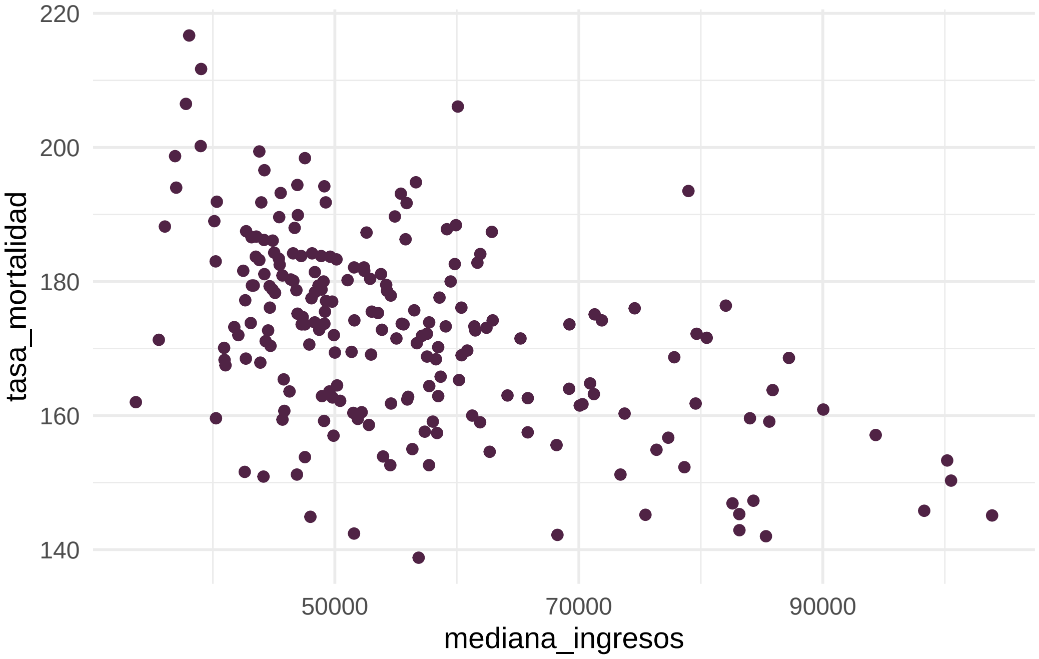
18.3.1. Presupuestos

Para que un modelo lineal sea válido, debe cumplir con cuatro supuestos fundamentales: **independencia, linealidad, homocedasticidad y normalidad**. Aunque la verificación rigurosa de estos criterios suele realizarse a partir del análisis de residuos tras ajustar el modelo, es recomendable realizar una evaluación preliminar de los datos para identificar posibles problemas desde el inicio.

La **independencia** (o ausencia de autocorrelación) puede determinarse, en gran medida, a partir del conocimiento sobre la fuente de los datos y su método de recolección. Sin embargo, siempre es recomendable verificarla en los residuales del modelo.

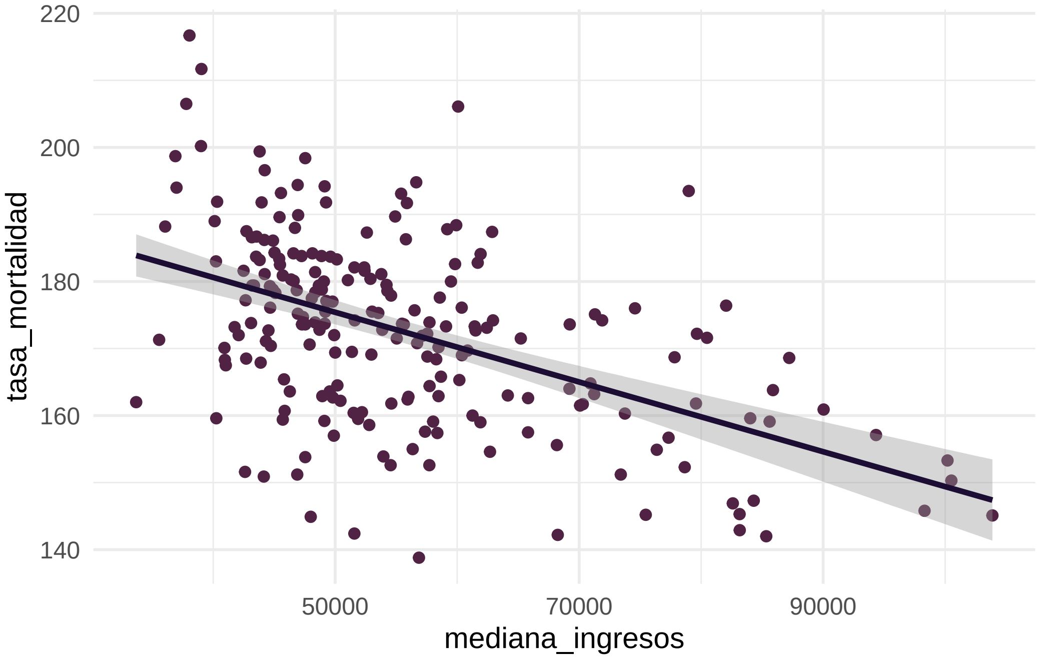
La **linealidad** se refiere a la relación entre las variables `tasa_mortalidad` y `mediana_ingresos`. Para comprobarla, se puede utilizar un diagrama de dispersión:

```r
datos |>

  ggplot(mapping = aes(x = mediana_ingresos, y = tasa_mortalidad)) +

  # gráfico de dispersión
  geom_point(color = "#502345") +

  # tema
  theme_minimal()
```



Observamos en el gráfico una clara relación inversa entre las variables, dado que los condados donde las personas tienen mayores ingresos la mortalidad por cáncer es menor y viceversa. Podemos dibujar la recta de regresión lineal sobre el diagrama de dispersión adicionando una capa más al gráfico mediante `geom_smooth()` e indicando `method = "lm"` como método. Además de la recta se puede ver el intervalo de confianza (zona gris alrededor de ella).

```r
datos |>
  ggplot(mapping = aes(x = mediana_ingresos, y = tasa_mortalidad)) +

  # diagrama de dispersión
  geom_point(color = "#502345") +

  # añade línea de regresión
  geom_smooth(method = "lm", color = "#1B0D33") +

  # cambia color de fondo
  theme_minimal()
```



Usaremos la función `cor()` para estimar la correlación entre las dos variables:

```
cor(datos$mediana_ingresos, datos$tasa_mortalidad, method = "pearson")
```

```
[1] -0.5041398
```

El valor es negativo, lo que confirma lo observado en la nube de puntos anterior. Para poder descartar que esta **correlación negativa** se debe al azar, debemos calcular su significancia:

```
cor.test(datos$mediana_ingresos, datos$tasa_mortalidad)
```

```

	Pearson's product-moment correlation

data:  datos$mediana_ingresos and datos$tasa_mortalidad
t = -8.4795, df = 211, p-value = 3.938e-15
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.5980408 -0.3965857
sample estimates:
       cor
-0.5041398
```

En este ejemplo $p < 0,05$, por lo tanto la correlación es significativa.

La **homocedasticidad** implica que la varianza de los residuos se mantiene aproximadamente constante a lo largo de los valores de la variable independiente. Se puede evaluar gráficamente mediante la dispersión de los residuos respecto a los valores ajustados o a través de contrastes de hipótesis, como el **test de Breusch-Pagan**.

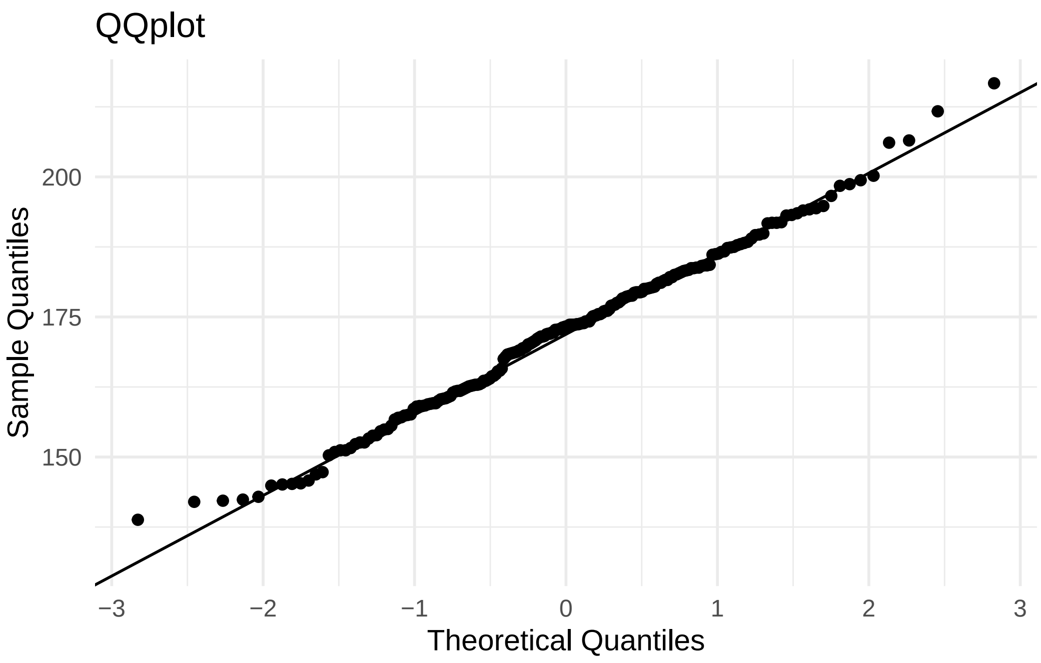
Finalmente, la **normalidad** de los residuos se puede evaluar mediante el **test de Lilliefors**, disponible en el paquete `nortest` (Gross y Ligges 2015):

```
lillie.test(datos$tasa_mortalidad)
```

```

	Lilliefors (Kolmogorov-Smirnov) normality test

data:  datos$tasa_mortalidad
D = 0.042152, p-value = 0.4707
```

También se puede verificar gráficamente con un **QQ plot**:

```
datos |>
  ggplot(mapping = aes(sample = tasa_mortalidad)) +

  # añade qqplot
  stat_qq() +
  stat_qq_line() +

  # cambia nombres de los ejes X e Y
  labs(title = "QQplot", x = "Theoretical Quantiles", y = "Sample Quantiles") +

  # modifico color de fondo
  theme_minimal()
```



Tanto el test de hipótesis como los gráficos de cuantiles nos informan que las distribuciones de la variable dependiente cumple con el criterio de "normalidad".

18.3.2. Ajuste del modelo

Como vimos **anteriormente**, la función `lm()` del paquete `stats` nos permite ajustar modelos de regresión lineal:

```
lm(tasa_mortalidad ~ mediana_ingresos, data = datos)
```

```


Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos, data = datos)

Coefficients:
     (Intercept)  mediana_ingresos
       201.41284          -0.00052
```

En este caso, `Intercept` representa el valor de `mediana_ingresos` cuando `tasa_mortalidad` vale cero (ordenada al origen) y el coeficiente de `mediana_ingresos` representa la pendiente de la recta.

Estos resultados obtenidos y aplicados en la fórmula del modelo simple quedarían así:

$$\text{tasamortalidad} = \alpha + \beta_1(\text{medianaingresos}) + \epsilon$$

$$\text{tasamortalidad} = 201.4128 + -0.0005 * \text{medianaingresos} + \epsilon$$

Guardamos el modelo como un objeto:

```
modelo <- lm(tasa_mortalidad ~ mediana_ingresos, data = datos)
```

Podemos acceder a la salida del modelo mediante la función `summary()`:

```
summary(modelo)
```

```


Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max
-33.040  -8.210   1.093   7.637  35.932

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)       2.014e+02  3.521e+00  57.200  < 2e-16 ***
mediana_ingresos -5.200e-04  6.133e-05  -8.479 3.94e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12.32 on 211 degrees of freedom
Multiple R-squared:  0.2542,    Adjusted R-squared:  0.2506
F-statistic:  71.9 on 1 and 211 DF,  p-value: 3.938e-15
```

La asociación entre la tasa de mortalidad por cáncer y la mediana de ingresos es estadísticamente significativa ($p < 0.001$).

Todos los ajustes de modelos lineales que produce la función `lm()` tienen la forma de una lista.La manera de conocer su clase es `class()` y su estructura mediante `str()`

```
class(modelo)
```

```
[1] "lm"
```

```
str(modelo)
```

```
List of 12
 $ coefficients : Named num [1:2] 201.41284 -0.00052
  ..- attr(*, "names")= chr [1:2] "(Intercept)" "mediana_ingresos"
 $ residuals    : Named num [1:213] 12.3 -2.68 -9.49 2.28 -17.11 ...
  ..- attr(*, "names")= chr [1:213] "1" "2" "3" "4" ...
 $ effects      : Named num [1:213] -2516.47 104.46 -10.27 1.26 -17.89 ...
  ..- attr(*, "names")= chr [1:213] "(Intercept)" "mediana_ingresos" "" "" ...
 $ rank         : int 2
 $ fitted.values: Named num [1:213] 170 171 172 179 172 ...
  ..- attr(*, "names")= chr [1:213] "1" "2" "3" "4" ...
 $ assign       : int [1:2] 0 1
 $ qr           :List of 5
  ..$ qr   : num [1:213, 1:2] -14.5945 0.0685 0.0685 0.0685 0.0685 ...
  .. ..- attr(*, "dimnames")=List of 2
  .. .. ..$ : chr [1:213] "1" "2" "3" "4" ...
  .. .. ..$ : chr [1:2] "(Intercept)" "mediana_ingresos"
  .. ..- attr(*, "assign")= int [1:2] 0 1
  ..$ qraux: num [1:2] 1.07 1.01
  ..$ pivot: int [1:2] 1 2
  ..$ tol  : num 1e-07
  ..$ rank : int 2
  ..- attr(*, "class")= chr "qr"
 $ df.residual  : int 211
 $ xlevels      : Named list()
 $ call         : language lm(formula = tasa_mortalidad ~ mediana_ingresos, data = datos)
 $ terms        :Classes 'terms', 'formula'  language tasa_mortalidad ~ mediana_ingresos
  .. ..- attr(*, "variables")= language list(tasa_mortalidad, mediana_ingresos)
  .. ..- attr(*, "factors")= int [1:2, 1] 0 1
  .. .. ..- attr(*, "dimnames")=List of 2
  .. .. .. ..$ : chr [1:2] "tasa_mortalidad" "mediana_ingresos"
  .. .. .. ..$ : chr "mediana_ingresos"
  .. ..- attr(*, "term.labels")= chr "mediana_ingresos"
  .. ..- attr(*, "order")= int 1
  .. ..- attr(*, "intercept")= int 1
  .. ..- attr(*, "response")= int 1
  .. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
  .. ..- attr(*, "predvars")= language list(tasa_mortalidad, mediana_ingresos)
  .. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
```

Characteristic	Beta	95 % CI	p-value
(Intercept)	201	194, 208	<0.001
mediana_ingresos	0.00	0.00, 0.00	<0.001

Abbreviation: CI = Confidence Interval

```
  .. .. ..- attr(*, "names")= chr [1:2] "tasa_mortalidad" "mediana_ingresos"
 $ model        :'data.frame':  213 obs. of  2 variables:
  ..$ tasa_mortalidad : num [1:213] 183 169 163 182 155 ...
  ..$ mediana_ingresos: num [1:213] 59831 57556 56008 42491 56353 ...
  ..- attr(*, "terms")=Classes 'terms', 'formula'  language tasa_mortalidad ~ mediana_ingres
  .. .. ..- attr(*, "variables")= language list(tasa_mortalidad, mediana_ingresos)
  .. .. ..- attr(*, "factors")= int [1:2, 1] 0 1
  .. .. .. ..- attr(*, "dimnames")=List of 2
  .. .. .. .. ..$ : chr [1:2] "tasa_mortalidad" "mediana_ingresos"
  .. .. .. .. ..$ : chr "mediana_ingresos"
  .. .. ..- attr(*, "term.labels")= chr "mediana_ingresos"
  .. .. ..- attr(*, "order")= int 1
  .. .. ..- attr(*, "intercept")= int 1
  .. .. ..- attr(*, "response")= int 1
  .. .. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
  .. .. ..- attr(*, "predvars")= language list(tasa_mortalidad, mediana_ingresos)
  .. .. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
  .. .. .. ..- attr(*, "names")= chr [1:2] "tasa_mortalidad" "mediana_ingresos"
 - attr(*, "class")= chr "lm"
```

La función `tbl_regression()` del paquete `gtsummary` (Sjoberg et al. 2021), nos permite generar una tabla con los coeficientes del modelo. El nivel de confianza puede ajustarse con el argumento `conf.level` y el argumento `intercept` muestra u oculta el intercepto:

```r
tbl_regression(modelo, intercept = T, conf.level = .95)
```

18.3.3. Residuales

El **residuo** o **residual** de una estimación se define como la diferencia entre el valor observado y el valor predicho por el modelo de regresión. Son fundamentales para evaluar la bondad de ajuste y verificar los supuestos básicos de los modelos lineales. Para resumir el conjunto de residuales, se pueden emplear dos enfoques:

1. La **sumatoria del valor absoluto** de cada residual.
2. La **sumatoria del cuadrado** de cada residual (**RSS,** *Residual Sum of Squares*), basada en el método de los mínimos cuadrados, amplifica las desviaciones extremas y permite minimizar errores en la estimación de parámetros.

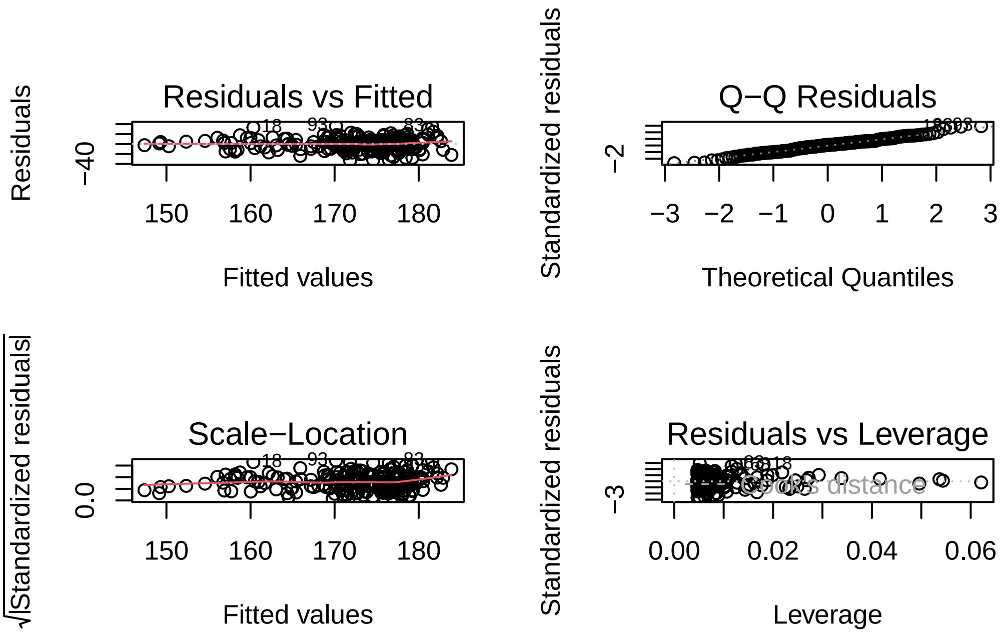
Los residuales se almacenan en el objeto de regresión y pueden visualizarse mediante:

```
resid(modelo)
```

Cuanto mayor sea la **sumatoria de los cuadrados de los residuales**, menor será la precisión con la que el modelo predice el valor de la variable dependiente a partir de la variable predictora.

Un análisis gráfico resulta muy útil para evaluar los supuestos del modelo. Aplicando la función `plot()` al objeto de regresión:

```
par(mfrow = c(2, 2))
plot(modelo)
```



Esta función genera cuatro gráficos automáticos:

- `Residuals vs Fitted`: Permite evaluar la **linealidad**. La línea roja debería ser lo más horizontal posible y sin curvaturas pronunciadas. Si hay curvatura, podría indicar la necesidad de un término no lineal (cuadrático, logarítmico, etc.) o la omisión de una variable importante en el modelo.
- `Normal Q-Q`: Evalúa la **normalidad** de los residuos. Los puntos deberían ajustarse a la diagonal. Desviaciones importantes sugieren un incumplimiento del supuesto de normalidad.
- `Scale-Location`: Indica si los residuos se distribuyen uniformemente a lo largo del rango de los predictores, verificando la **homocedasticidad**. Una línea aproximadamente horizontal con puntos dispersos de manera aleatoria es una buena señal.
- `Residuals vs Leverage`: Ayuda a identificar **valores influyentes**. Un punto extremo no necesariamente es influyente, pero aquellos fuera de las líneas rojas punteadas (altas puntuaciones de distancia de Cook) pueden estar afectando considerablemente el modelo.

Otra opción para evaluar gráficamente los supuestos del modelo es la función `check_model()` del paquete `performance`, el cual forma parte del ecosistema `easystats` (Lüdecke et al. 2022):

```
check_model(modelo)
```

Podemos elegir que gráficos visualizar usando el argumento `check`:

```
check_model(
  modelo,
  check = c("normality", "qq", "linearity", "homogeneity", "outliers")
)
```

Para evaluar la linealidad se puede aplicar el test RESET de Ramsey, mediante la función `resettest()` del paquete `lmtest` (Zeileis y Hothorn 2002):

```
resettest(modelo)
```

```

	RESET test

data:  modelo
RESET = 0.82653, df1 = 2, df2 = 209, p-value = 0.439
```

La hipótesis nula indica que las variables se relacionan de forma lineal; un $p < 0,05$ sugiere lo contrario. En nuestro modelo, el valor *p* de 0.439 respalda la suposición de linealidad.

Otro supuesto fundamental es que los residuales deben distribuirse de forma normal (con media 0). Además del QQ-plot, se puede aplicar el test de Lilliefors:

```
lillie.test(modelo$residuals)
```

```

	Lilliefors (Kolmogorov-Smirnov) normality test

data:  modelo$residuals
D = 0.05804, p-value = 0.07821
```

Con un *p*-valor de 0.078 se confirma la normalidad.

El test de Breusch-Pagan, implementado en `lmtest` y `performance`, evalúa si la varianza de los residuos es constante. Parte de la hipótesis nula de **homocedasticidad** o varianza constante en las perturbaciones y la enfrenta a la alternativa de varianza variable, por lo que es válido decir que cumple con el supuesto de homocedasticidad si el valor $p > 0,05$:

```
# paquete lmtest
bptest(modelo)
```

```

	studentized Breusch-Pagan test

data:  modelo
BP = 2.5078, df = 1, p-value = 0.1133
```

```
# paquete performance
check_heteroscedasticity(modelo)
```

```
OK: Error variance appears to be homoscedastic (p = 0.087).
```

Incluso cuando el modelo cumple todos los supuestos, es importante identificar observaciones atípicas o de alto *leverage*, ya que podrían condicionar el ajuste. Los valores atípicos u *outliers* son observaciones que no se ajustan bien al modelo, con residuos excesivamente grandes. Por otro lado, los puntos con alto *leverage* tienen valores extremos en los predictores, lo que los hace potencialmente influyentes.

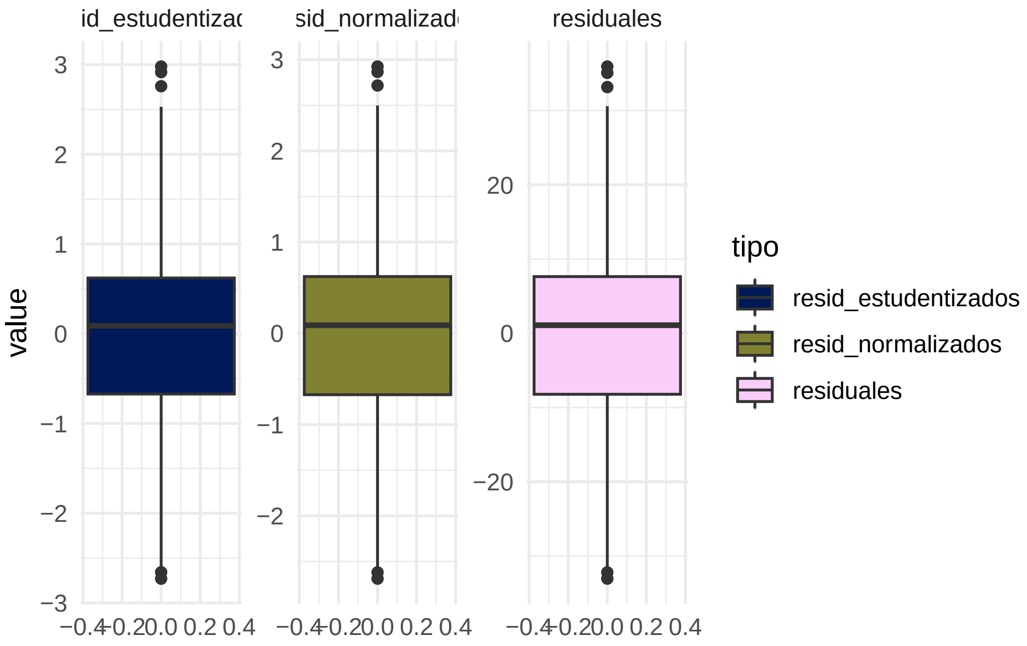
Para ello se pueden analizar los residuales estandarizados y los residuales estudentizados. Los **residuales estandarizados** se obtienen normalizando los residuales por su desviación estándar. Esta aproximación permite identificar valores atípicos que se alejan más de ± 3 desviaciones estándar. Sin embargo, si un *outlier* influye lo suficiente en el modelo como para atraer la línea de regresión, su residual podría ser pequeño y pasar desapercibido. Una alternativa más robusta es utilizar los **residuales estudentizados**. Se trata de un proceso iterativo en el que se va excluyendo cada vez una observación i distinta y se reajusta el modelo con las $n-1$ restantes. En cada proceso de exclusión y reajuste se calcula la diferencia (d_i) entre el valor predicho para i habiendo y sin haber excluido esa observación. Finalmente, se normalizan las diferencias d_i . Aquellos valores d_i cuyo residual estudentizado supera ± 3 suelen considerarse significativos.

Estos dos procesos sobre los residuales se pueden calcular en R mediante las funciones `rstandar()` y `rstudent()`:

```
# Creación del dataset de residuales
residuales <- tibble(
  residuales = residuals(modelo),
  resid_normalizados = rstandard(modelo),
  resid_estudentizados = rstudent(modelo)
)
```

Podemos visualizar la comparación de estos residuales mediante un *boxplot* usando *facets*:

```
residuales |>

  # Pasa a formato long
  pivot_longer(cols = everything(), names_to = "tipo") |>

  # Genera gráfico
  ggplot(mapping = aes(y = value, fill = tipo)) +

  geom_boxplot() +

  scale_fill_scico_d() +

  facet_wrap(~tipo, scales = "free_y") + # subdivide en facets

  theme_minimal()
```



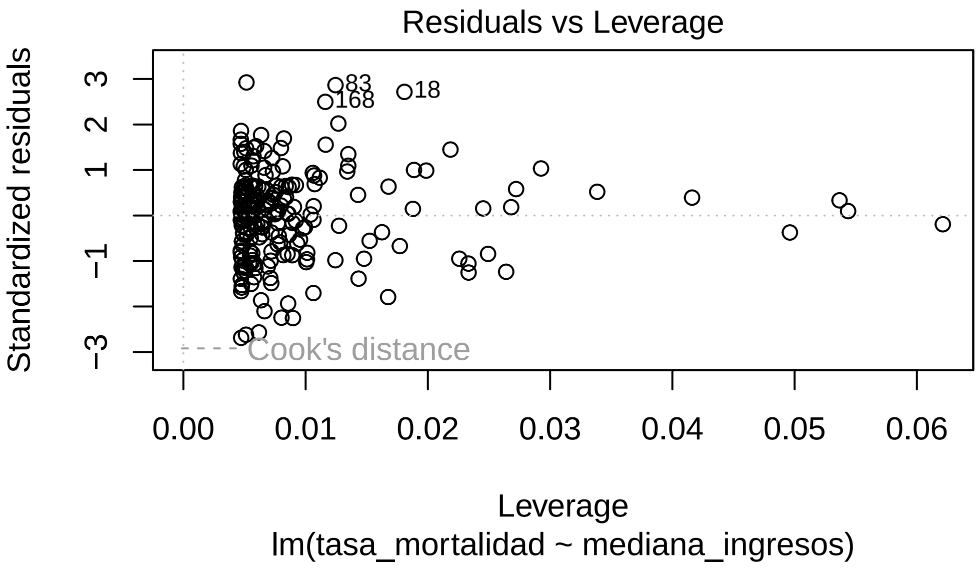
Si encontramos residuales con valores absolutos superiores a 3 en el gráfico de residuales estudentizados, es recomendable investigarlos más a fondo. Podemos identificarlos con:

```r
table(rstudent(modelo) > 3)
```

```
FALSE 
  213 
```

Para evaluar la influencia de observaciones, se utiliza la **distancia de Cook**, que combina la magnitud del residual y el *leverage*. Valores superiores a 1 se consideran generalmente influyentes. Esto se puede visualizar mediante:

```r
plot(modelo, which = 5)
```



Al observar el gráfico, se debe prestar especial atención a los puntos situados en las esquinas superior e inferior, es decir, aquellos que aparecen fuera de las líneas discontinuas rojas. Estos puntos suelen presentar altas puntuaciones en la distancia de Cook, lo que indica que pueden influir significativamente en los resultados de la regresión.

Además, el paquete `performance` incluye la función `check_outliers()` para detectar valores atípicos según su distancia de Cook:

```
check_outliers(modelo)
```

```
OK: No outliers detected.
- Based on the following method and threshold: cook (0.7).
- For variable: (Whole model)
```

En el ejemplo analizado, no se identificaron valores influyentes evidentes. Sin embargo, si se detecta alguna observación fuera de estos límites, es recomendable estudiarla de forma individual para determinar, por ejemplo, si su elevada influencia se debe a un error de registro. En tal caso podremos eliminar la observación (o corregirla) y analizar los casos restantes. Pero si el dato es correcto, quizás sea diferente de las otras observaciones y encontrar las causas de este fenómeno puede llegar a ser la parte más interesante del análisis. Por supuesto que todo esto dependerá del contexto del problema que uno esta estudiando.

18.3.4. Bondad de ajuste del modelo

Una vez ajustado un modelo, es necesario verificar su eficiencia, ya que, aun siendo la línea que mejor se ajusta a las observaciones entre todas las posibles, el modelo puede resultar inadecuado. Entre las medidas más utilizadas para evaluar la calidad del ajuste se encuentran el error estándar de los residuales, el test F y el coeficiente de determinación R^2. Estos valores aparecen al final de la salida de la función `summary()`, donde se pueden identificar el RSE (*Residual Standard Error*), el R^2 (*Multiple R-squared*) y el R^2 ajustado (*Adjusted R-squared*).

Podemos acceder al valor de R^2 del modelo usando la función `r2()` del paquete `performance`:

```
r2(modelo)
```

```
# R2 for Linear Regression
       R2: 0.254
  adj. R2: 0.251
```

El R^2 oscila entre 0 y 1, de modo que valores cercanos a 1 indican un buen ajuste del modelo lineal a los datos. Por otro lado, el R^2 ajustado penaliza la inclusión de variables independientes poco relevantes en la explicación de la variable dependiente, razón por la cual su valor es menor o igual al R^2. En nuestro ejemplo los valores de R^2 obtenidos nos indican que el modelo lineal simple no se ajusta demasiado bien a los datos. Esto significa que solamente un 25.1 % de la variación en la tasa de mortalidad por cáncer se explica únicamente por la mediana de ingresos como variable explicativa; el restante 74.9 % no se explica, lo que sugiere que agregar otras variables independientes podría mejorar el ajuste y, por ende, la confiabilidad de las predicciones.

La salida de `summary()` también incluye un estadístico F de Snedecor y su p-valor correspondiente, que se utilizan en el contraste ómnibus para evaluar, de forma global, la idoneidad del modelo. En nuestro ejemplo, $p > 0,05$, por lo que, al 95 % de confianza, se rechaza la hipótesis nula y se concluye que el modelo lineal es adecuado para el conjunto de datos. Cabe mencionar que, cuando el modelo de regresión tiene una única variable explicativa, este contraste es equivalente al contraste del parámetro β_1.

Otra forma de verificar de manera independiente la significación del modelo es mediante la función `anova()`, que plantea el contraste de la regresión mediante el **análisis de la varianza**:

```
anova(modelo)
```

```
Analysis of Variance Table

Response: tasa_mortalidad
                  Df Sum Sq Mean Sq F value    Pr(>F)
mediana_ingresos   1  10912 10911.8  71.901 3.938e-15 ***
Residuals        211  32021   151.8
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

La tabla de análisis de varianza muestra resultados coherentes con el bloque final de `summary(modelo)`, presentando un valor F de 71.9 y un p-valor significativo.

18.3.5. Interpretación de resultados

Se realizó una regresión lineal simple para analizar la relación entre la mediana de ingresos y la tasa de mortalidad por cáncer en 174 condados de la Costa Este de Estados Unidos. El diagrama de dispersión mostró una relación lineal negativa moderada entre ambas variables, lo que se confirmó con un coeficiente de correlación de Pearson de
-0.5. Además, el análisis de regresión reveló que la relación es estadísticamente significativa (t = -8.48, *p* = 0).

El coeficiente de pendiente para la mediana de ingresos fue de
0, lo que indica que, por cada unidad de incremento en la mediana de ingresos, la tasa de mortalidad por

cáncer disminuye aproximadamente en 0,05 puntos porcentuales. El R^2 del modelo muestra que el 25.1 % de la variación en la tasa de mortalidad se explica únicamente con esta variable, lo que sugiere la existencia de otros factores que también influyen en la mortalidad por cáncer.

Finalmente, la gráfica de dispersión de los valores pronosticados estandarizados frente a los residuos estandarizados evidenció que se cumplen los supuestos de homogeneidad de varianza y linealidad, y que los residuos se distribuyen de forma aproximadamente normal. Se cumplieron todos los presupuestos necesarios para validar la regresión y no se encontraron valores atípicos influyentes.

19. Análisis de la varianza (ANOVA)

19.1. Introducción

El análisis de varianza (ANOVA) es una extensión del modelo lineal general que se utiliza para comparar las medias de una variable dependiente continua (Y) entre diferentes niveles de una variable explicativa categórica (X), que debe tener al menos tres niveles.

La hipótesis nula (H_0) del test estadístico establece que las medias de la variable dependiente son iguales en todos los grupos, mientras que la hipótesis alternativa (H_1) plantea que al menos dos medias difieren significativamente:

- H0: $⍰_1 = ⍰_2 = … = ⍰_i $
- H_1: al menos una $⍰_i ≠ ⍰_j $

Por lo tanto, el ANOVA permite comparar múltiples medias, pero lo hace analizando la variabilidad entre y dentro de los grupos.

La variabilidad total se descompone en dos componentes:

- **Intervarianza (SSB):** Variabilidad entre los grupos.
- **Intravarianza (SSE):** Variabilidad dentro de los grupos.

El estadístico F del ANOVA, que sigue una distribución F de Fisher-Snedecor, compara estas dos fuentes de variabilidad:

$$F = \frac{SSB/(k-1)}{SSE/(n-k)}$$

donde:

- k: el número de grupos.
- n: número total de observaciones.

Si se cumple H0, el estadístico F tiende a 1, ya que las varianzas entre y dentro de los grupos son similares. Si las medias difieren significativamente, la intervarianza será mayor que la intravarianza, resultando en valores de F superiores a 1.

19.2. ANOVA de un factor o de una vía

El ANOVA de una vía puede considerare como una extensión de los *t-test* independientes para comparar más de dos grupos de un factor. En este contexto, factor se refiere a la variable categórica que define los grupos.

Para que los resultados del ANOVA sean válidos, deben cumplirse los siguientes supuestos:

- **Aleatoriedad:** Las observaciones deben ser aleatorias.
- **Independencia:** Las observaciones entre grupos deben ser independientes.
- **Variable dependiente:** Cuantitativa continua.
- **Variable explicativa:** Categórica con más de dos niveles.
- **Normalidad:** La distribución de la variable dependiente dentro de cada grupo debe ser normal.
- **Homocedasticidad:** Las varianzas de los grupos deben ser homogéneas.

19.3. Ejemplo práctico en R

Nuevamente vamos a trabajar con el conjunto de datos “`cancer_USA.txt`”, que contiene información sobre la tasa de mortalidad por cáncer (por 100.000 habitantes) en distintos condados de la Costa Este de Estados Unidos. Nuestro objetivo será analizar la relación entre la tasa de mortalidad (`tasa_mortalidad`) y el estado de residencia (`estado`).

Comenzaremos por cargar los paquetes necesarios:

```r
# tablas salida del modelo
library(gtsummary)

# chequeo de supuestos
library(easystats)
library(nortest)
library(pwr)

# comparación de medias
library(emmeans)

# análisis exploratorio
library(skimr)
library(dlookr)

# paletas de colores aptas daltonismo
library(scico)

# manejo de datos
library(janitor)
library(tidyverse)
```

Cargamos y exploramos los datos:

```r
# Carga datos
datos <- read_csv2("datos/cancer_USA.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 213
Columns: 10
$ condado           <chr> "Belknap County", "Carroll County", "Cheshire Count~
$ estado            <chr> "New Hampshire", "New Hampshire", "New Hampshire", ~
$ tasa_mortalidad   <dbl> 182.6, 168.8, 162.8, 181.6, 155.0, 163.2, 173.1, 17~
$ mediana_edad      <dbl> 46.1, 50.3, 42.0, 48.1, 41.9, 40.1, 42.6, 43.5, 37.~
$ mediana_edad_cat  <chr> "45+ años", "45+ años", "36-39 años", "45+ años", "~
$ mediana_ingresos  <dbl> 59831, 57556, 56008, 42491, 56353, 71233, 62429, 79~
$ pct_pobreza       <dbl> 10.0, 10.7, 11.8, 14.9, 11.6, 8.7, 9.5, 6.1, 11.8, ~
$ pct_salud_publica <dbl> 16.8, 13.4, 12.7, 22.5, 14.0, 12.7, 13.1, 9.0, 12.4~
$ pct_sec_incompleta <dbl> 15.4, 14.8, 6.1, 13.2, 6.6, 12.9, 10.9, 13.5, 6.1, ~
$ pct_desempleo     <dbl> 5.3, 5.8, 6.1, 6.9, 5.0, 5.9, 5.2, 5.6, 6.5, 5.8, 1~
```

La base de datos tiene observaciones y variables. La variable dependiente es `tasa_mortalidad`, mientras que `estado` es nuestra variable explicativa, categórica con 9 niveles.

Exploremos más en profundidad la estructura de los datos usando la función `skim()` del paquete `skimr`:

```r
# Resumen de los datos
skim(datos)
```

Tabla 19.1.: Data summary

Name	datos
Number of rows	213
Number of columns	10
Column type frequency:	
character	3
numeric	7
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
condado	0	1	10	19	0	174	0
estado	0	1	5	13	0	9	0
mediana_edad_cat	0	1	8	10	0	4	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
tasa_mortalidad	0	1	172.43	14.23	138.8	162.2	173.3	181.6	216.7	⍰⍰⍰⍰⍰
mediana_edad	0	1	42.06	3.67	30.2	39.9	42.2	44.4	51.9	⍰⍰⍰⍰⍰
mediana_ingresos	0	1	55740.31	13795.54	33687.0	45860.0	52195.0	60387.0	103876.0	⍰⍰⍰⍰⍰
pct_pobreza	0	1	13.25	3.91	4.7	10.7	13.2	15.4	31.5	⍰⍰⍰⍰⍰
pct_salud_publica	0	1	17.86	4.91	7.1	14.6	17.7	20.7	38.2	⍰⍰⍰⍰⍰
pct_sec_incompleta	0	1	13.21	4.76	3.1	10.2	12.7	15.7	34.2	⍰⍰⍰⍰⍰
pct_desempleo	0	1	7.50	1.76	3.0	6.4	7.3	8.6	14.0	⍰⍰⍰⍰⍰

```r
# Frecuencia por estado
tabyl(datos$estado)
```

```
  datos$estado  n    percent
   Connecticut  8 0.03755869
         Maine 16 0.07511737
 Massachusetts 14 0.06572770
 New Hampshire 10 0.04694836
    New Jersey 21 0.09859155
      New York 62 0.29107981
  Pennsylvania 64 0.30046948
  Rhode Island  4 0.01877934
       Vermont 14 0.06572770
```

Observamos que la variable `estado` tiene 9 niveles y ningún valor ausente (`n_missing = 0`). La variable dependiente tiene valores entre 139 y 217 (casos/100 000 habitantes) y tampoco presenta valores `NA`.

Los estados con menos observaciones (Connecticut, New Hampshire y Rhode Island) serán excluidos para evitar sesgos en el análisis. También convertiremos la variable estado a un factor ordenado con el comando `fct_relevel()` del paquete `tidyverse`:

```r
# Filtramos estados con pocas observaciones y convertimos a factor
datos <- datos |>
  filter(!estado %in% c("New Hampshire", "Rhode Island", "Connecticut")) |>

  mutate(estado = fct_relevel(estado, "New York", after = 0))

# Verificamos con class() y levels()
class(datos$estado)
```

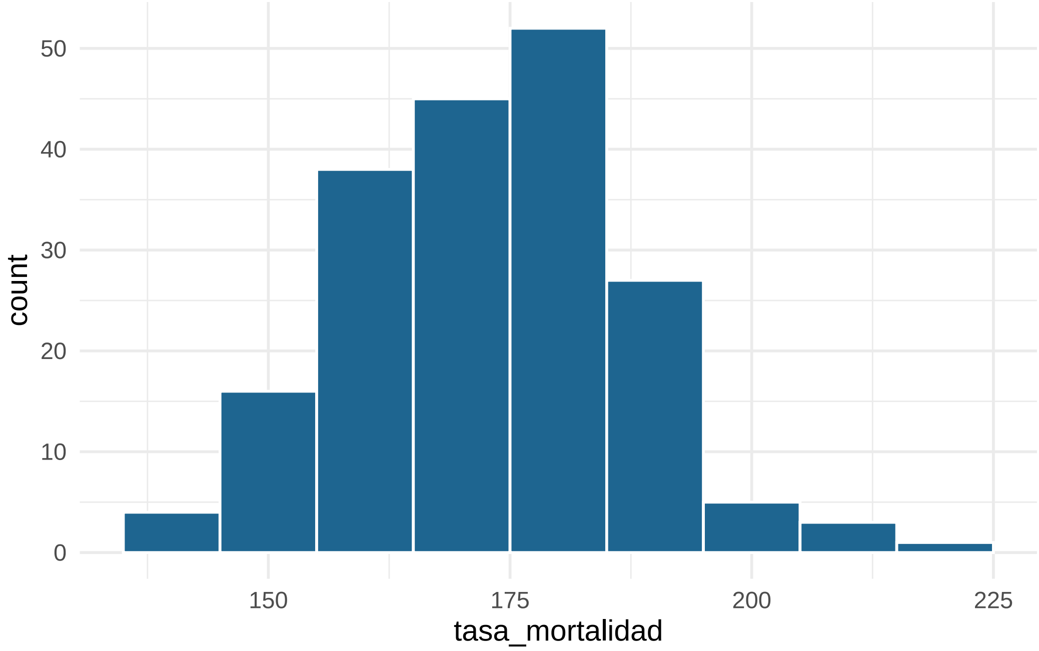
```
[1] "factor"
```

```r
levels(datos$estado)
```

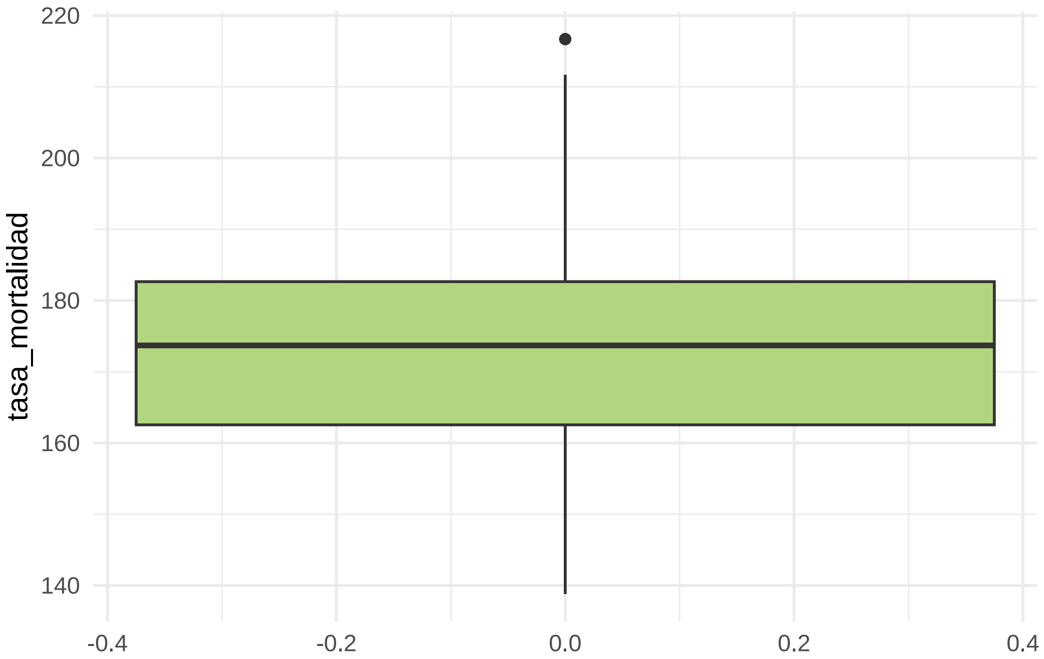
```
[1] "New York"      "Maine"         "Massachusetts" "New Jersey"
[5] "Pennsylvania"  "Vermont"
```

Graficamos la distribución de la variable dependiente mediante un histograma y un *boxplot*:

```r
# histograma
datos |>
  ggplot(mapping = aes(x = tasa_mortalidad)) +
  geom_histogram(binwidth = 10, color = "white", fill = "#1E6590") +
  theme_minimal() # tema claro
```



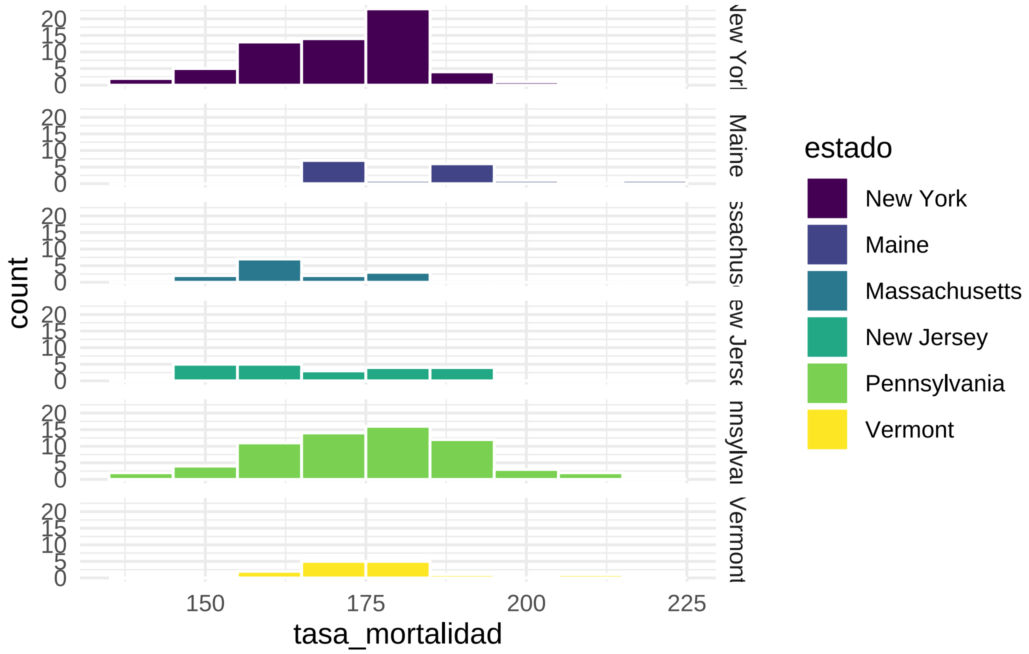
```r
# boxplot
datos |>
  ggplot(mapping = aes(y = tasa_mortalidad)) +
  geom_boxplot(fill = "#B2D680") +
  theme_minimal() # tema claro
```



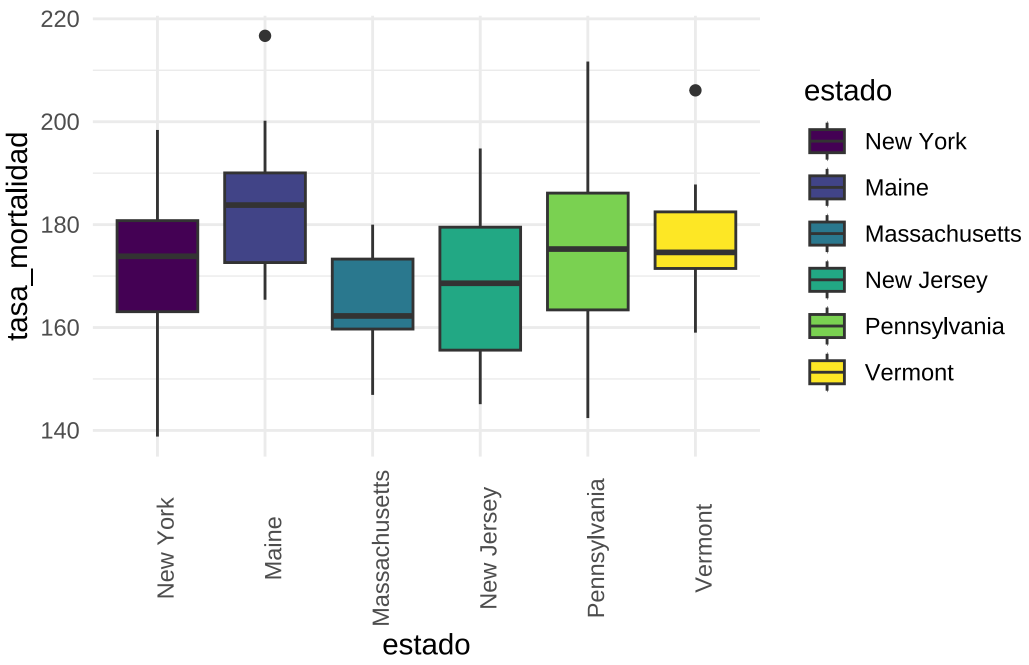
La distribución global es simétrica pareciera existir un valor atípico.

Ahora veamos como es la distribución de la tasa de mortalidad por estado.

```
# Histograma por grupos
datos |>
  ggplot(mapping = aes(x = tasa_mortalidad, fill = estado)) +
  geom_histogram(binwidth = 10, color = "white") +
  scale_fill_viridis_d() + # paleta colorblind-friendly
  facet_grid(estado ~ .) + # variable estado en filas
  theme_minimal() # tema claro
```



```
# Boxplot por grupos
datos |>
  ggplot(mapping = aes(x = estado, y = tasa_mortalidad, fill = estado)) +
  geom_boxplot() +
  scale_fill_viridis_d() + # paleta colorblind-friendly
  theme_minimal() + # tema claro
  theme(axis.text.x = element_text(angle = 90)) # gira etiquetas eje x
```



Observamos que las distribuciones en algunos grupos parecen simétricas y hay diferencias visuales entre las medianas. Esto plantea la pregunta de si dichas diferencias son estadísticamente significativas.

19.3.1. Análisis de supuestos

Si bien es más recomendable y habitual realizar el análisis de supuestos a partir de los residuales del modelo, podemos chequear que la variable dependiente cumpla con los supuestos de normalidad y homocedasticidad previo a realizar el análisis.

Normalidad

Usamos el test de Kolmogorov-Smirnov con corrección de Lilliefors para grupos con más de 50 observaciones con la función `lillie.test()` del paquete `nortest`. Como algunos grupos tienen menos de 50 observaciones, también podríamos utilizar el test de Shapiro-Wilk mediante la función `shapiro.test()` de R `base`:

```
datos |>
  filter(estado == "New York") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "New York"), tasa_mortalidad)
D = 0.11633, p-value = 0.03637
```

```
datos |>
  filter(estado == "Maine") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "Maine"), tasa_mortalidad)
D = 0.18801, p-value = 0.1364
```

```
datos |>
  filter(estado == "Massachusetts") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "Massachusetts"), tasa_mortalidad)
D = 0.1591, p-value = 0.4377
```

```
datos |>
  filter(estado == "New Jersey") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "New Jersey"), tasa_mortalidad)
D = 0.14095, p-value = 0.3387
```

```
datos |>
  filter(estado == "Pennsylvania") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

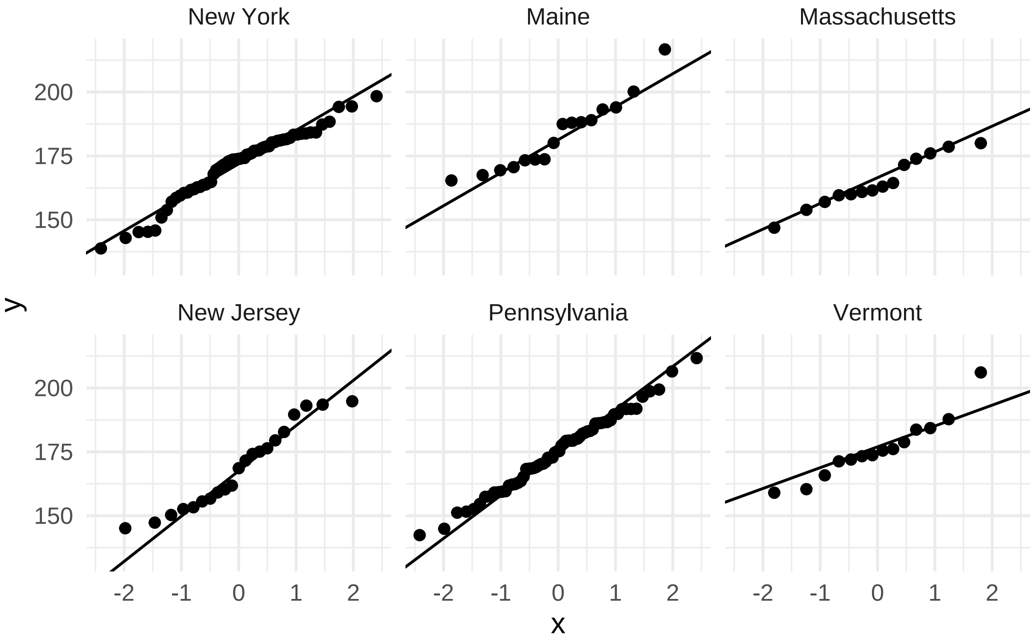
```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "Pennsylvania"), tasa_mortalidad)
D = 0.066685, p-value = 0.6839
```

```r
datos |>
  filter(estado == "Vermont") |>
  pull(tasa_mortalidad) |> # con pull() extraemos datos de tasa_mortalidad como vector
  lillie.test() # aplicamos test de bondad de ajuste
```

```
	Lilliefors (Kolmogorov-Smirnov) normality test

data:  pull(filter(datos, estado == "Vermont"), tasa_mortalidad)
D = 0.14856, p-value = 0.5497
```

Podemos graficar el comportamiento de la variable dependiente en cada grupo mediante Q-Q plots usando el paquete `ggplot2`:

```r
datos |>
  ggplot() +
  geom_qq(mapping = aes(sample = tasa_mortalidad)) +
  geom_qq_line(mapping = aes(sample = tasa_mortalidad)) +
  facet_wrap(~estado) +
  theme_minimal()
```



Homocedasticidad

Verificamos la homogeneidad de varianzas por grupo con el test de Bartlett:

```r
bartlett.test(
  tasa_mortalidad ~ estado, # utiliza sintaxis fórmula
  data = datos
)
```

```
	Bartlett test of homogeneity of variances

data:  tasa_mortalidad by estado
Bartlett's K-squared = 4.8185, df = 5, p-value = 0.4384
```

La variable respuesta cumple con el supuesto de homocedasticidad ($p = 0.438$).

19.3.2. Análisis de varianza

Si bien podemos realizar un test ANOVA mediante la función `aov()` de R `base`, en el contexto de este curso utilizaremos la función `lm()` del paquete `stats` que usamos anteriormente para ajustar el modelo de **regresión lineal simple:**

```
lm(tasa_mortalidad ~ estado, data = datos)
```

```

Call:
lm(formula = tasa_mortalidad ~ estado, data = datos)

Coefficients:
        (Intercept)           estadoMaine  estadoMassachusetts  
            171.640                11.510               -6.840  
   estadoNew Jersey   estadoPennsylvania        estadoVermont  
             -3.007                 3.419                4.631  
```

Siempre que trabajemos con modelos, conviene que asignemos los resultados a objetos que luego nos servirán para aplicar otras funciones. Como nuestra variable explicativa es categórica y nuestro principal interés es conocer si existen diferencias significativas entre grupos, no analizaremos los coeficientes del modelo con `summary()`, sino la significancia del test F mediante la función `anova()`.

```
# ANOVA
modelo <- lm(tasa_mortalidad ~ estado, data = datos)

# Salida del modelo
anova(modelo)
```

```
Analysis of Variance Table

Response: tasa_mortalidad
           Df Sum Sq Mean Sq F value   Pr(>F)   
estado      5   3513  702.67  3.7103 0.003179 **
Residuals 185  35036  189.38                    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Los resultados del test F nos muestran un $p = 0,003$, sugiriendo que al menos dos grupos son diferentes entre sí.

19.3.3. Comparaciones múltiples

Una vez que comprobamos que existen diferencias significativas entre grupos, nos interesa saber cuáles grupos son diferentes entre sí. Para ello, existen distintos algoritmos de comparaciones múltiples con sus respectivas correcciones, como el test de *Diferencia Honestamente Significativa de Tukey,* también llamado Tukey HSD o **test de Tukey**. Esta prueba se aplica para grupos equilibrados (mismo tamaño) y varianzas similares (homocedásticas). Es una prueba conservadora, dado que mantiene bajo el error de tipo I, sacrificando la capacidad de detectar diferencias existentes.

Si las varianzas son homocedásticas pero los grupos difieren en tamaño, podemos usar el test de Tukey si tenemos que comparar entre varios grupos, o la **corrección de Bonferroni** para grupos más reducidos.

Código	Rango
***	0 a 0,001
**	0,001 a 0,01
*	0,01 a 0,05
.	0,05 a 0,1
	0,1 a 1

El paquete `emmeans` (Lenth y Piaskowski 2026) es una herramienta muy versátil y poderosa para realizar comparaciones múltiples. Comenzaremos utilizando la función `emmeans()` con el argumento `specs = "estado"` para crear un objeto que contenga las medias marginales por grupo:

```
comp <- emmeans(modelo, specs = "estado")
```

Para realizar las comparaciones múltiples mediante test de Tukey usamos el comando `contrast()` con el argumento `method = "pairwise"`:

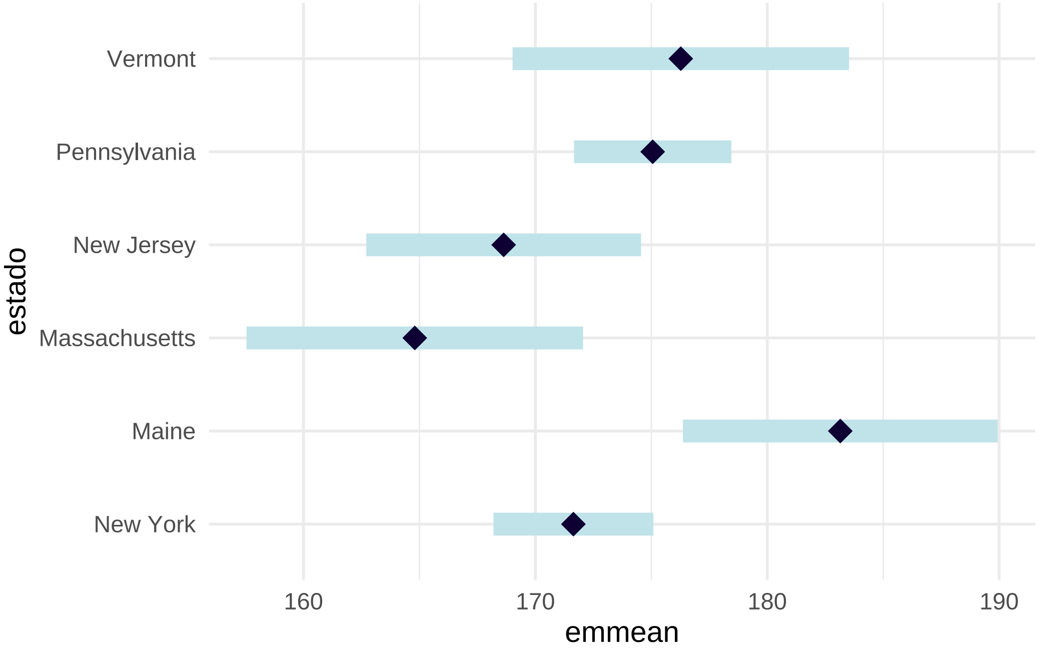
```
contrast(comp, method = "pairwise")
```

```
 contrast                     estimate   SE  df t.ratio p.value
 New York - Maine               -11.51 3.86 185  -2.983  0.0376
 New York - Massachusetts         6.84 4.07 185   1.680  0.5469
 New York - New Jersey            3.01 3.47 185   0.865  0.9542
 New York - Pennsylvania         -3.42 2.45 185  -1.394  0.7305
 New York - Vermont              -4.63 4.07 185  -1.137  0.8652
 Maine - Massachusetts           18.35 5.04 185   3.644  0.0046
 Maine - New Jersey              14.52 4.57 185   3.179  0.0211
 Maine - Pennsylvania             8.09 3.85 185   2.103  0.2901
 Maine - Vermont                  6.88 5.04 185   1.366  0.7474
 Massachusetts - New Jersey      -3.83 4.75 185  -0.807  0.9659
 Massachusetts - Pennsylvania   -10.26 4.06 185  -2.527  0.1217
 Massachusetts - Vermont        -11.47 5.20 185  -2.205  0.2402
 New Jersey - Pennsylvania       -6.43 3.46 185  -1.857  0.4322
 New Jersey - Vermont            -7.64 4.75 185  -1.609  0.5939
 Pennsylvania - Vermont          -1.21 4.06 185  -0.299  0.9997

P value adjustment: tukey method for comparing a family of 6 estimates
```

Los resultados del test de Tukey nos muestran diferencias significativas en la tasa de mortalidad por cáncer en el estado de Maine respecto de New York, Massachusetts y New Jersey.

Para mayor claridad, podemos mostrar las comparaciones múltiples mediante un gráfico:

```r
plot(comp) +
  theme_minimal() # tema claro
```



El rombo del medio representa la media marginal para cada grupo y el rectángulo azul, su intervalo de confianza, aquellos grupos en los que no se superponen los intervalos de confianza son estadísticamente diferentes entre sí.

19.3.4. Bondad de ajuste

El tamaño de efecto comúnmente utilizado para el caso de ANOVA es η^2 (*eta-cuadrado)*, que se calcula como el cociente de la suma de cuadrados del efecto sobre la suma de cuadrados total.

```r
anova(modelo)
```

```
Analysis of Variance Table

Response: tasa_mortalidad
           Df Sum Sq Mean Sq F value   Pr(>F)
estado      5   3513  702.67  3.7103 0.003179 **
Residuals 185  35036  189.38
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# calculo eta-cuadrado
eta2 <- 3513 / (3513 + 35036)

eta2
```

```
[1] 0.09113077
```

Podemos llegar al mismo valor usando la función `r2()` del paquete `performance`:

```r
r2(modelo)
```

```
# R2 for Linear Regression
       R2: 0.091
  adj. R2: 0.067
```

19.3.5. Potencia

Los test de potencia permiten determinar la probabilidad de encontrar diferencias significativas entre las medias para un determinado nivel de significancia indicando el tamaño de los grupos.

En base al eta-cuadrado obtenido anteriormente, calculamos el tamaño de efecto convencional mediante la función `cohen.ES()` del paquete `pwr` (Champely 2020):

```r
cohen.ES(test = "anov", size = "small")
```

```

     Conventional effect size from Cohen (1982) 

           test = anov
           size = small
    effect.size = 0.1
```

La función `pwr.anova.test()` del paquete `pwr` nos permite calcular la potencia en base al número de grupos, número de observaciones por grupo, tamaño de efecto y nivel de significancia (habitualmente 0,05).

Para grupos de igual tamaño usamos el siguiente código:

```r
pwr.anova.test(k = k, n = n, f = eta_cuadrado, sig.level = 0.05)
```

En nuestro ejemplo, donde los tamaños de grupo son diferentes, primero debemos calcular el tamaño de grupo efectivo:

```
# Número de grupos
k <- nlevels(datos$estado)

# Número de observaciones por grupo
n <- tabyl(datos$estado)$n

# Tamaño de grupo efectivo
neff <- sum(n)^2 / sum(n^2)
```

Calculamos la potencia:

```
pwr.anova.test(k = k, n = neff, f = eta2, sig.level = 0.05)
```

```

     Balanced one-way analysis of variance power calculation 

              k = 6
              n = 4.040425
              f = 0.09113077
      sig.level = 0.05
          power = 0.05670154

NOTE: n is number in each group
```

19.3.6. Análisis de residuales

Para poder dar por válidos los resultados del ANOVA es necesario verificar que se satisfacen las condiciones analizando los residuos del modelo. Para ello, usaremos el mismo procedimiento que vimos para la **regresión lineal simple**.

Método gráfico con paquete `performance`:

```
check_model(modelo, panel = FALSE)
```

Chequeo normalidad de residuales mediante test de Lilliefors:

```
lillie.test(modelo$residuals)
```

```

    Lilliefors (Kolmogorov-Smirnov) normality test

data:  modelo$residuals
D = 0.046833, p-value = 0.388
```

Chequeo homocedasticidad (test de Breusch-Pagan):

```
check_heteroscedasticity(modelo)
```

```
OK: Error variance appears to be homoscedastic (p = 0.597).
```

Chequeo presencia de outliers (distancia de Cook):

```
check_outliers(modelo)
```

```
OK: No outliers detected.
- Based on the following method and threshold: cook (0.5).
- For variable: (Whole model)
```

19.4. Conclusión

Se ha realizado un ANOVA de una vía para comparar la tasa de mortalidad por cáncer en seis estados de Estados Unidos. Se encontraron diferencias estadísticamente significativas entre las medias de los grupos (F = 3,71, p = 0.003). El *eta-cuadrado* fue de 0.091, lo que indica que el estado de residencia explica un 9.1 % de la varianza total en los valores de la tasa de mortalidad por cáncer. La potencia de la prueba estadística fue del 5.7 %.

Se realizó la prueba post hoc de Tukey HSD y se encontró que media de mortalidad para el estado de Maine fue estadísticamente superiror respecto de los estados de Massachusetts, New York y New Jersey (p < 0.05).

20. Rol de las variables

20.1. Introducción

Antes de comenzar a construir modelos de **regresión lineal múltiple**, es necesario repasar algunos conceptos fundamentales.

Los estudios epidemiológicos suelen partir de modelos teóricos conocidos, consensos científicos establecidos o hipotéticos, intentando responder a una pregunta de investigación. Las variables recolectadas se definen en la etapa de diseño del estudio, cada una cumpliendo un rol específico producto del modelo teórico asumido que dialoga con el estado del arte del tema investigado.Como mínimo, se identifican dos variables principales y excluyentes: la exposición (variable independiente) y el desenlace (variable dependiente). Una vez seleccionadas la variable dependiente e independiente, las demás variables medidas o no medidas son las que se denominan covariables.

En el marco de nuestros estudios, las covariables pueden asumir diversos roles, como confusoras, intermedias, colisionadoras, modificadoras de efecto, entre otros. Algunos de estos roles suelen estar definidos previamente por la literatura mientras que otros pueden surgir como hipotéticas o ser identificados durante el análisis.

Por ello, la/el epidemióloga/o debe definir un marco teórico previo y conocer qué alcance tiene el tipo de estudio desarrollado.

20.2. Causalidad vs asociación

Un error persistente en la práctica es suponer que los modelos de regresión (lineales, logísticos, etc) poseen la capacidad intrínseca de discernir causalidad. Sin embargo, estos modelos son *"agnósticos"*: procesan datos y devuelven coeficientes basados exclusivamente en correlaciones matemáticas, sin comprender la naturaleza biológica o social de las variables. Por ello, el epidemiólogo debe definir un marco teórico previo y conocer que alcance tiene el tipo de estudio desarrollado.

Partiendo de la premisa que asociación no implica causalidad, podemos razonar causalmente para decidir qué variables controlar o que ajustes hacer en el análisis, aun cuando al final concluyamos que los datos no permiten establecer causalidad con certeza. De hecho, si no razonamos causalmente sobre el rol de las variables, no podemos saber si la estimación obtenida del modelo como mera asociación es siquiera válida internamente.

Los estudios observacionales trabajan muchas veces con asociaciones y para aproximarnos a responder las preguntas de investigación que nos hacemos es necesario tener presente algunas cuestiones: criterios de causalidad (temporalidad, plausibilidad biológica, socioeconómica, etc), presencia de variables de confusión, sesgos y estructura causal subyacente.

Para formalizar estos razonamientos, se suelen utilizar los *Grafos Acíclicos Dirigidos* (DAG, por sus siglas en inglés), herramientas que permiten visualizar supuestos y detectar sesgos estructurales antes de avanzar con la metodología.

20.3. El rol de las variables

Los roles de confusora, mediadora, colisionadora o modificadora del efecto son propiedades que emergen de la estructura causal del fenómeno que se estudia (representada en el DAG) y no del diseño del estudio. El tabaquismo es un factor de riesgo con efecto causal sobre el cáncer, la edad es causa de tabaquismo y cáncer, la hospitalización es consecuencia de ambos: eso es cierto independientemente de si llegamos a esos datos a un diseño de cohorte, casos-controles o estudio transversal.

Lo que cambia según el diseño no es la estructura causal subyacente, sino dos cosas:

- Lo que podemos estimar (un OR en lugar de RR, prevalencia en lugar de incidencia, etc.)
- Los riesgos estructurales de sesgo que introduce el propio diseño

20.4. Los DAGs

Un *Grafo Acíclico Dirigido* (**DAG**) es un grafo donde los nodos representan variables y las flechas representan efectos causales directos. *"Acíclico"* significa que no hay ciclos: ninguna variable puede causarse a sí misma a través de una cadena de efectos.

20.4.1. Convenciones básicas

- $X \to Y$ se lee: *X tiene un efecto causal directo sobre Y*
- La ausencia de flechas entre dos variables significa que no existe efecto causal directo entre ellas (no implica que la asociación estadística sea cero)
- Los DAG son cualitativos, representan la presencia o ausencia de efectos, no su magnitud.

20.4.2. Caminos en un DAG

Un camino entre dos nodos es cualquier secuencia de flechas que los conecta, independientemente de la dirección. Existen tres tipos de estructuras básicas:

Estructura del DAG	Nombre
X –> M –> Y	Cadena (mediación)
X <– Z –> Y	Bifurcación (confusión)
X –> C <– Y	Colisionador

Usamos los DAG, a *priori*, como una representación gráfica de las suposiciones causales que como investigadores tenemos antes de analizar los datos. Nos permite visualizar qué variables debemos ajustar (controlar confusión) y cuales no debemos incluir para evitar sesgos, independientemente del diseño de estudio. Además, nos obliga a explicitar el modelo teórico subyacente y a formular hipótesis causales plausibles.

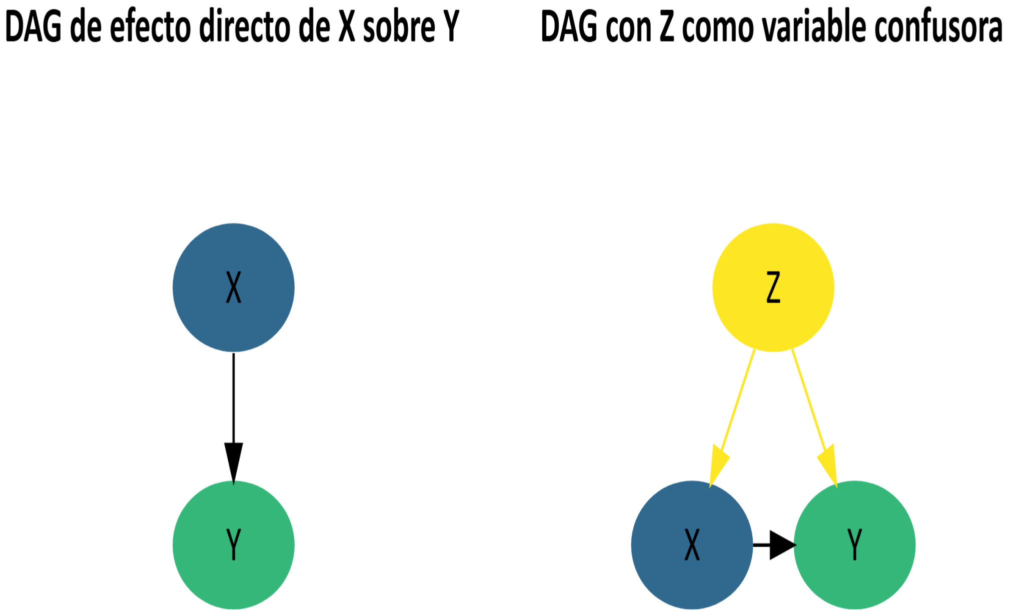
20.5. Confusión

Una variable de confusión es una variable que distorsiona la medida de asociación entre las variables principales del estudio (exposición-desenlace). En presencia de confusión, pueden observarse los siguientes escenarios:

- **Asociación espuria:** Efecto observado donde en realidad no existe.
- **Confusión positiva:** Exageración o atenuación de una asociación real.
- **Confusión negativa:** Inversión del sentido de una asociación real.

Según Gordis (2017), en un estudio que evalúa si la exposición (X) causa un resultado (Y), se dice que un tercer factor (Z) es una variable de confusión si cumple con los siguientes criterios:

- Z es una causa (o proxy de una causa) conocido para Y.
- Z está asociado con la exposición X, pero no es un resultado de X.
- Z no está en el camino causal entre X e Y (no es mediadora)



DAG de efecto directo de X sobre Y

DAG con Z como variable confusora

Para quienes necesiten profundizar, se recomienda leer el artículo:

▯ De Irala, J., et al. (2001). ¿Qué es una variable de confusión? *Medicina Clínica*, *117*(10), 377–385. https://doi.org/10.1016/S0025-7753(01)72121-5

20.5.1. Manejo de la confusión

Dentro de las estrategias para manejar la confusión, podemos pensar en dos momentos:

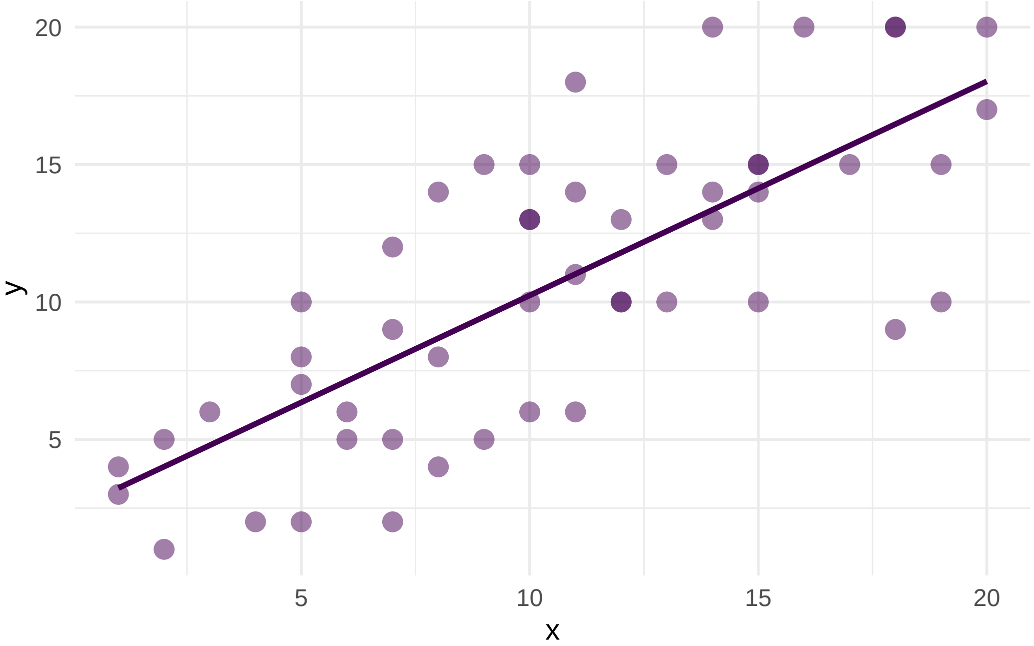
- A la hora de diseñar y llevar a cabo el estudio:
  - Emparejamiento individual.
  - Emparejamiento de grupo.
- Al momento de analizar los datos:

- Estratificación.
- Ajuste estadístico.

El ajuste estadístico, característico de los análisis multivariados, permite estimar el efecto específico de cada variable independiente sobre la dependiente, controlando por las demás variables.

Un criterio estadístico comúnmente aceptado es que un factor se considera confusor si el ajuste por dicha variable provoca un cambio de al menos el 10 % en la magnitud de la asociación. Por ejemplo, edad y sexo son variables frecuentemente confusoras en estudios epidemiológicos y generalmente son pocos los trabajos que no presentan datos ajustados por estas covariables.

En este curso utilizaremos la **regresión lineal múltiple** y los modelos lineales generalizados para manejar la confusión, ajustando por múltiples covariables. A continuación, se ejemplifica el fenómeno gráficamente con un diagrama de dispersión.



La recta de regresión muestra una correlación positiva entre los valores de las variables, X e Y, con una ecuación de la forma:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \epsilon$$

La salida en R de este modelo se vería de la siguiente forma:

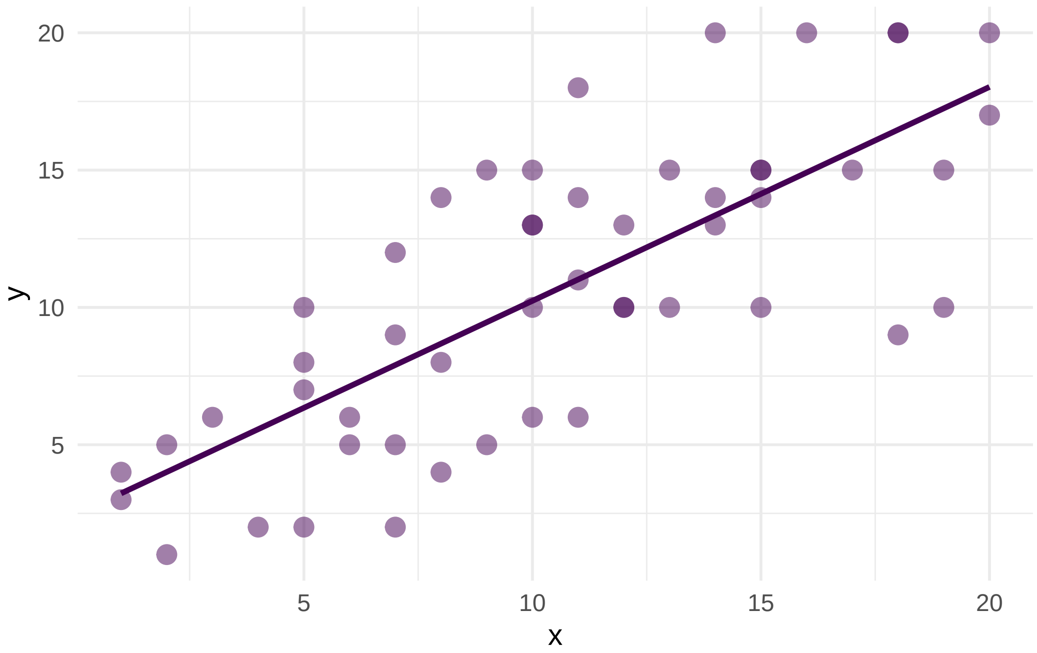
```
Call:
lm(formula = y ~ x, data = datos_conf)

Residuals:
    Min      1Q  Median      3Q     Max 
-7.4701 -2.4124 -0.0179  2.5926  6.9821 

Coefficients:
```

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.45017    1.13262   2.163   0.0354 *
x            0.77889    0.09642   8.078 1.45e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.559 on 49 degrees of freedom
Multiple R-squared:  0.5712,    Adjusted R-squared:  0.5624
F-statistic: 65.26 on 1 and 49 DF,  p-value: 1.45e-10
```

El intercepto (β_0) es 2,45 y la pendiente (β_1) es 0,78. El modelo explica el 56 % de la variabilidad de Y (R^2 = 0,56). En la ecuación podemos representarlo como:

$$\hat{y} = 2.45 + 0.78x_1 + \epsilon$$

Ahora incorporemos la covariable Z, con categorías **A** y **B**, que sospechamos tiene un rol de confusión en el modelo teórico (DAG).



El gráfico de dispersión muestra que hay una diferencia entre las rectas de regresión que se mantiene prácticamente constante (paralelas) en todo su desarrollo. Esa distancia medida en valores de Y es β_2:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon$$

Visto en resultados de consola:

```
Call:
lm(formula = y ~ x + z, data = datos_conf)
```

```
Residuals:
   Min     1Q Median     3Q    Max 
-7.958 -1.502 -0.225  1.840  5.652 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  0.65684    1.00484   0.654    0.516    
x            0.65260    0.08372   7.795 4.49e-10 ***
zB           4.55436    0.93258   4.884 1.20e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.939 on 48 degrees of freedom
Multiple R-squared:  0.7135,	Adjusted R-squared:  0.7016 
F-statistic: 59.77 on 2 and 48 DF,  p-value: 9.347e-14
```

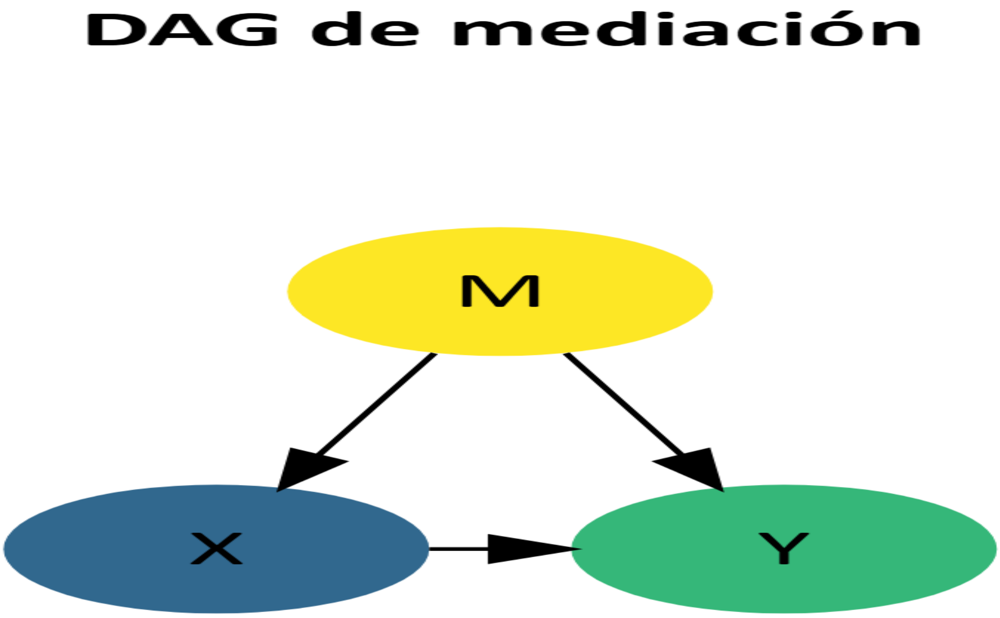
El coeficiente β_1 de la variable independiente principal (X) varió al incorporar la nueva variable (Z), pasando de 0,78 (cruda) a 0,65 (ajustada), es decir que disminuyó casi un 20 %. A la vez, la covariable tiene una relación significativa con la variable dependiente (Y) y el modelo aumenta el R^2 ajustado a 0,70.

Entonces podemos ver que la regresión multiple ajustó el efecto de X sobre Y, teniendo en cuenta el efecto confusor de Z que sospechabamos. El valor de Y ahora es 0,66 (β_0) + 0,65* el valor de x ($\beta_1 * x$) mientras Z es igual al nivel de referencia **A**, en cambio Y vale 0,66 (β_0) + 0,65* el valor de X ($\beta_1 * x$) + 4,55 (β_2) cuando Z es igual a **B**.

Nota: No toda variable asociada con X e Y es confusora. La estructura causal es la que define ese rol, que luego la matemática del modelo confirmará. Una variable puede ser un proxy de la confusora (que en la práctica es como tener a la variable confusora) o jugar otro rol, siempre dependiendo de la pregunta de investigación que nos hagamos.

20.6. Variable mediadora

Una variable M es mediadora si se encuentra en la cadena causal entre X e Y: X actúa a través de M para afectar a Y. Puede coexistir un efecto directo $X \to Y$.



El análisis de mediación descompone el efecto total en:

- **Efecto directo (ED)**: efecto de X sobre Y a través de todas las vías que no involucran M
- **Efecto indirecto (EI)**: efecto de X sobre Y que opera a través de M
- **Efecto total** = ED + EI

Si incluimos M como covariable en un modelo que busca estimar el efecto total de X sobre Y, bloquearemos el camino $X \to M \to Y$ y subestimaremos el efecto de la exposición.

Un ejemplo lo podemos observar analizando este estudio ecológico, donde se preguntan por el efecto del porcentaje de hogares con acceso a espacios verdes (por municipio) sobre la tasa de mortalidad por enfermedad cardiovascular (por 100.000 hab.)

Cuentan con otras variables que consideraron, entre las cuales, tienen la proporción de población con actividad física regular.

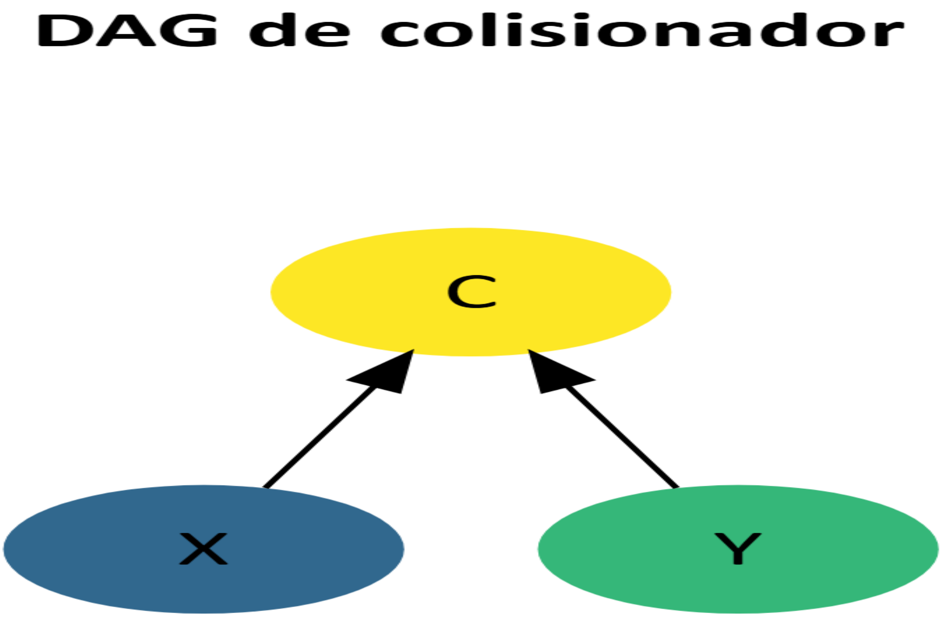
		mortalidad ev			mortalidad ev	
Predictors	Estimates	CI	p	Estimates	CI	p
(Intercept)	357.61	343.40 – 371.81	**<0.001**	367.97	356.27 – 379.67	**<0.001**
espverde	-2.20	-2.52 – -1.87	**<0.001**	-0.78	-1.21 – -0.34	**0.001**
actfis				-2.35	-2.92 – -1.78	**<0.001**
Observations	120			120		
R^2 / R^2 adjusted	0.602 / 0.599			0.745 / 0.741		

Llegamos a estos resultados, hipotetizando en nuestro modelo teórico que los espacios verdes reducen la mortalidad cardiovascular parcialmente a través de facilitar la actividad física, pero también por vías directas (reducción de estrés, calidad del aire, cohesión social). Razonar con este modelo teórico, explica porque el coeficiente de espacios verdes cambia al incluirla. Es decir, que el efecto total de espacios verdes pasa de -2,20 a -0,78, es decir de un efecto total a un efecto directo ante la presencia de actividad fisica.

Por lo tanto, si una variable es mediadora, no debe incluirse en el modelo cuando se busca el efecto total. Para estimar el efecto indirecto, se utilizan métodos de análisis de mediación (p. ej., producto de coeficientes con bootstrap o modelos de ecuaciones estructurales que exceden los alcances de este curso).

20.7. Variable colisionadora

Una variable C es colisionadora (o collider en inglés), si es una consecuencia común de X e Y (o de causas de X e Y).



A diferencia del confusor, el camino a través de un colisionador está naturalmente bloqueado. La asociación espuria se crea cuando condicionamos o controlamos por el colisionador: al incluirlo en el modelo, restringir la muestra, o seleccionar participantes basándonos en él, podemos generar una asociación artificial. Esto se llama sesgo de colisionador (usualmente conocido como *collider bias*, en inglés).

Es consecuencia generalmente de un sesgo de selección y se suele manifestar en estudios de base hospitalaria (sesgo de Berkson), en estudios longitudinales, si el abandono depende de la exposición y de covariables del desenlace (pérdida no aleatoria) y en estudios con sesgo de supervivencia (análisis de los supervivientes). Ver Holmberg y Andersen (2022)

El ejemplo clásico es el de Berkson: Imaginemos un hospital que atiende pacientes con dos enfermedades, enfermedad cardiovascular (CV) y diabetes. La hospitalización actúa como colisionador. Supongamos que en la población general la enfermedad CV y la diabetes son independientes (no hay relación causal entre ellas). Sin embargo, si restringimos nuestro análisis solo a pacientes hospitalizados, podríamos observar una asociación negativa entre las dos. Quienes tienen enfermedad CV parecen tener menos diabetes que el resto, y viceversa. Esta asociación es completamente artificial (un artefacto del diseño). ¿Por qué? Porque la hospitalización es consecuencia de tener al menos una de las dos condiciones. Al condicionar sobre “estar hospitalizado”, conectamos artificialmente las dos enfermedades.

Otro ejemplo, que podemos ver en los datos, es el de una investigación sobre el efecto del consumo de sodio (exposición) en la presión arterial sistólica (desenlace). La recolección de datos incluye la edad y la proteinuria (presencia de proteína en la orina).

		presion			presion			presion
Predictors	Estimates	CI	p	Estimates	CI	p	Estimates	CI
(Intercept)	119.42	117.22 – 121.62	**<0.001**	-0.31	-1.11 – 0.49	0.445	-0.09	-0.47 – 0.2
sodio	3.96	3.37 – 4.55	**<0.001**	1.04	0.98 – 1.10	**<0.001**	-0.90	-0.97 – -0.
edad				2.00	1.99 – 2.02	**<0.001**	0.42	0.36 – 0.4

proteinuria					0.40	0.38 – 0.41
Observations	1000		1000		1000	
R^2 / R^2 adjusted	0.150 / 0.149		0.991 / 0.991		0.998 / 0.998	

Vemos tres modelos, el primero es un modelo del efecto bruto del consumo de sodio sobre la presión arterial, el segundo incluye la edad y manifiesta un efecto confusor sobre la relación y el tercer modelo, al incluir proteinuria, manfiesta el efecto colisionador (ver que el coeficiente de sodio invierte su signo).

El DAG de este ejemplo, muestra que tanto el consumo elevado de sodio como la presión arterial alta son causas de la proteinuria, por lo que es un error controlar por esa variable. (Luque-Fernandez et al. (2019))

20.8. Interacción o modificación de efecto

MacMahon (1972) definió la interacción de la siguiente manera:

> ! *"Cuando la incidencia de la enfermedad en presencia de dos o más factores de riesgo difiere de la incidencia que sería previsible por sus efectos individuales"*

Esto puede manifestarse como:

- **Sinergismo (interacción positiva):** El efecto combinado es mayor que la suma de sus efectos individuales.
- **Antagonismo (interacción negativa):** El efecto combinado es menor.

La modificación del efecto ocurre cuando la magnitud de la asociación entre la exposición (X) y el resultado (Y) varía según los niveles de una tercera variable (I).

No es un sesgo —es una característica real del fenómeno— y no debe eliminarse sino *describirse* (es decir, incluirla en el modelo e informar en los resultados de forma estratificada)

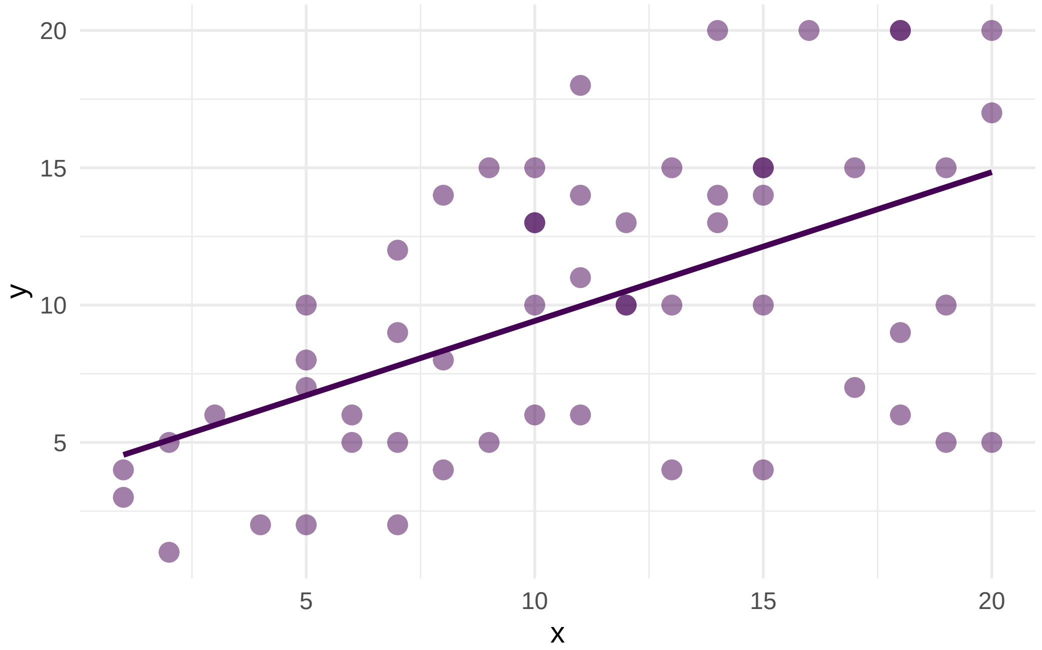
Para quienes tengan interés en profundizar el tema, pueden leer el artículo:

□ De Irala, J., et al. (2001). ¿Qué es una variable modificadora del efecto? *Medicina Clínica*, *117*(8), 297–302. https://doi.org/10.1016/S0025-7753(01)72092-1

Para identificar la interacción en regresión lineal múltiple, se incluyen términos de interacción ($X * I$), que representan una nueva variable con efectos multiplicativos. El término de interacción implica el exceso de la variabilidad de los datos que no puede ser explicada por la suma de las variables consideradas.

En cursos anteriores de Epidemiología se estudió que, frente a un modificador de efecto (I) lo más adecuado era presentar las medidas de asociación según los estratos formados por las categorías de la variable I (no estimar una medida ajustada para ambos estratos, como se hace en caso de variables confusoras).

Un ejemplo similar al recién mostrado para la confusión, pero para la interacción podría ser:



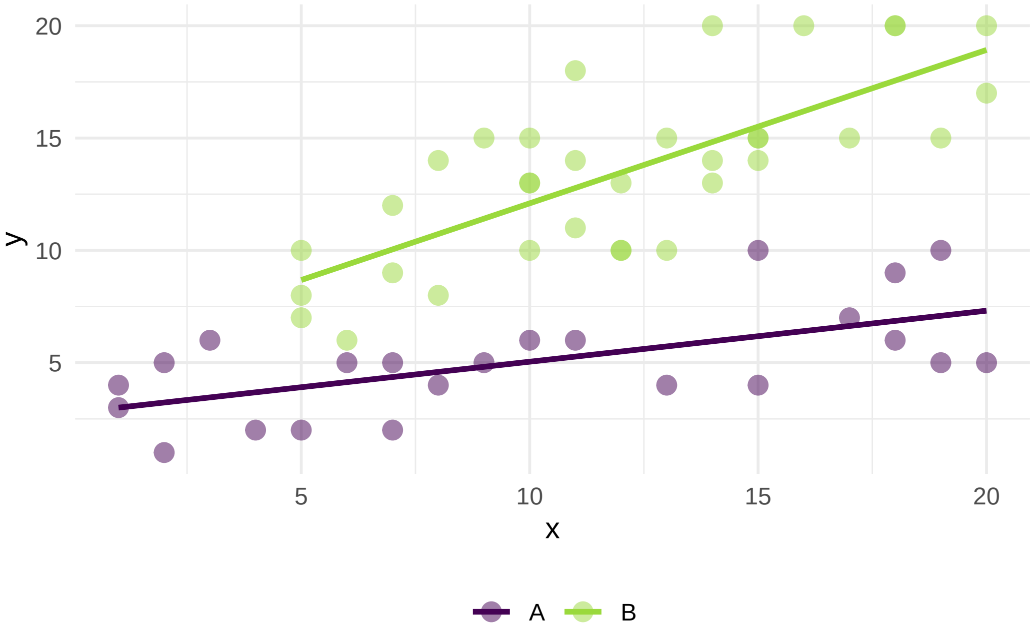
$$\hat{y} = \beta_0 + \beta_1 x_1 + \epsilon$$

Partimos de esta relación entre X e Y representada por la recta de la ecuación con los valores de la siguiente tabla:

```
Call:
lm(formula = y ~ x, data = datos_int)

Residuals:
    Min      1Q  Median      3Q     Max 
-9.8410 -3.8804  0.3709  3.2871  8.4102 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)   4.0035     1.4065   2.847   0.0062 ** 
x             0.5419     0.1132   4.789 1.31e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.554 on 55 degrees of freedom
Multiple R-squared:  0.2943,    Adjusted R-squared:  0.2814 
F-statistic: 22.93 on 1 and 55 DF,  p-value: 1.307e-05
```

El intercepto (β_0) es de 4,0 y el coeficiente β_1 (pendiente) significativo de 0,54.

Ahora incorporemos la covariable Z, con categorías **A** y **B**, que sospechamos tiene un rol de interacción.



El gráfico de dispersión muestra que hay una diferencia entre las rectas de regresión que tienen distintas pendientes según el valor de Z. Esa diferencia no es aditiva y pasa a ser multiplicativa y da lugar a la ecuación:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \epsilon$$

Visto en resultados de consola:

```
Call:
lm(formula = y ~ x * z, data = datos_int)

Residuals:
    Min      1Q  Median      3Q     Max 
-4.1427 -1.8739 -0.4598  1.3199  5.2230 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  2.77269    0.91587   3.027 0.003804 ** 
x            0.22708    0.07719   2.942 0.004830 ** 
zB           2.49329    1.51850   1.642 0.106522    
x:zB         0.45574    0.12213   3.732 0.000465 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.364 on 53 degrees of freedom
Multiple R-squared:  0.8168,	Adjusted R-squared:  0.8064 
F-statistic: 78.75 on 3 and 53 DF,  p-value: < 2.2e-16
```

El término de interacción es significativo, aunque la variable I por sí misma no lo sea, lo que implica que la significancia se encuentra asociada específicamente a uno de los niveles de I (en este caso, la categoría B). Dado que no es posible separar los niveles de I, esta variable debe permanecer en el modelo. Además, la inclusión del término de interacción mejora el R^2 ajustado del modelo, que pasa de 0,28 a 0,81. Esto demuestra que, cuando la interacción es significativa, la significancia estadística de los efectos simples de las variables X e I se vuelve habitualmente menos relevante.

El modelo ajustado para Y es el siguiente:

- Para $I = A$:

$$y = 2,77 + 0,23x$$

- Para $I = B$:

$$y = 2,77 + 0,23x + 2,49 + 0,46x$$

Esto muestra una interacción sinérgica entre la categoría B de la variable I y la variable X, reflejada en el efecto sobre Y. La pendiente de la recta para $I = B$ es mayor que para $I = A$, lo que indica que el efecto de X sobre Y varía según el nivel de I.

20.8.1. Limitaciones de los términos de interacción

Aunque el análisis de interacción en modelos lineales generales es una herramienta útil para explorar relaciones complejas entre variables, es importante ser consciente de sus limitaciones y asumirlas en el diseño, análisis e interpretación de los resultados. Entre ellas encontramos:

- **Linealidad y Simplicidad del Modelo:** El modelo lineal general asume relaciones lineales entre las variables, pero las interacciones a veces no muestran linealidad.
- **Sobreajuste en Modelos con Múltiples Interacciones:** La inclusión de muchos términos de interacción, puede sobreajustar el modelo y dificultar su interpretación y generalización.
- **Colinealidad:** La colinealidad entre las variables independientes puede ser exacerbada al incluir términos de interacción, lo que dificultaría la identificación de relaciones significativas.
- **Pérdida de Potencia Estadística:** Al incluir términos de interacción, el número de parámetros a estimar aumenta, reduciendo los grados de libertad y pudiendo disminuir la potencia estadística.
- **Dificultades en la Interpretación:** Los coeficientes de interacción pueden ser difíciles de interpretar, especialmente cuando las variables son continuas.
- **Heterogeneidad de Efectos:** Los términos de interacción pueden no capturar variaciones en los efectos entre subgrupos específicos.
- **Errores de Especificación del Modelo:** Si las interacciones relevantes no se incluyen o se modelan incorrectamente, el modelo puede estar mal especificado.
- **Sensibilidad a la Codificación de las Variables:** La codificación de variables categóricas puede influir en la interpretación de los coeficientes de interacción.

20.9. Resumen: ¿Qué ajustar y que no?

Variable	Estructura DAG	Ajuste	Si no ajustás	Si ajustas innecesariamente
Confusora	X <– Z –> Y	Sí	Sesgo de confusión	—
Mediadora	X –> M –> Y	Depende del objetivo	Se estima efecto total	Sesgo por sobre-ajuste; subestimación del efecto total
Colisionadora	X –> C <– Y	No	Sin problema (camino bloqueado)	Sesgo de colisionador; asociación espuria
Interacción	I –> (X –> Y)	Incluir como interaccón	Efecto promediado (heterogeneidad oculta)	—

20.10. Teoría vs algoritmo

Muchos algoritmos de selección automática de variables (como *stepwise regression*) incluyen toda variable que mejore *“matemáticamente”* el ajuste de un modelo o tiene un p-valor bajo. No distingue que rol juega esa variable, si se trata de un mediador o de un colisionador, de un confusor o es modificadora de efecto.

Ante las preguntas, ¿Por qué en estos casos el criterio estadístico puede fallar en identificar el modelo correcto? ¿Qué nos dice esto sobre la importancia de construir un DAG antes de tocar los datos?, la respuesta es que el modelo estadístico no sabe de epidemiología.

Según Almeida Filho y Barreto (2011), *“Es verdad que las técnicas utilizadas para la atribución de valores numéricos al grado de certeza de que las variables se encuentran asociadas son eminentemente estadísticas. Sin embargo, en el campo de la Epidemiología, la estadística no habla por sí sola. Le corresponde al epidemiólogo analizar los resultados obtenidos a la luz del conocimiento epidemiológico acumulado, el contexto en el cual el fenómeno analizado es parte y las características propias, cualitativas, asumidas por el fenómeno en su especificidad temporal y espacial. Es decir, más que significancia estadística, la interpretación de los datos debe buscar establecer la”significancia epidemiológica” de los hallazgos. Esa es la esencia del análisis epidemiológico.”*

21. Regresión lineal múltiple

21.1. Introducción

Los modelos de Regresión Lineal Múltiple (RLM) son herramientas estadísticas ampliamente utilizadas cuando la variable dependiente es continua y existen dos o más variables independientes, que pueden ser tanto continuas como categóricas. En el caso de las covariables categóricas, estas pueden ser dicotómicas, ordinales o tener múltiples niveles.

Al igual que la Regresión Lineal Simple (RLS), que permite estimar el efecto bruto de una variable independiente sobre la variable dependiente, la RLM nos permite conocer el efecto conjunto de dos o más variables independientes (X_1, X_2,...X_k) sobre la variable dependientea (Y). De esta manera, la RLM nos permite:

- **Analizar la dirección y fuerza de la asociación** entre la variable dependiente y las variables independientes.
- **Identificar las variables independientes importantes** en la predicción o explicación de la variable dependiente.
- **Describir la relación entre una o más variables independientes**, controlando la confusión.
- **Detectar interacciones**, es decir, cómo cambia la relación entre la variable dependiente y una variable independiente según el nivel de otra variable independiente.

El modelo estadístico de la RLS que expresa la relación entre X e Y es:

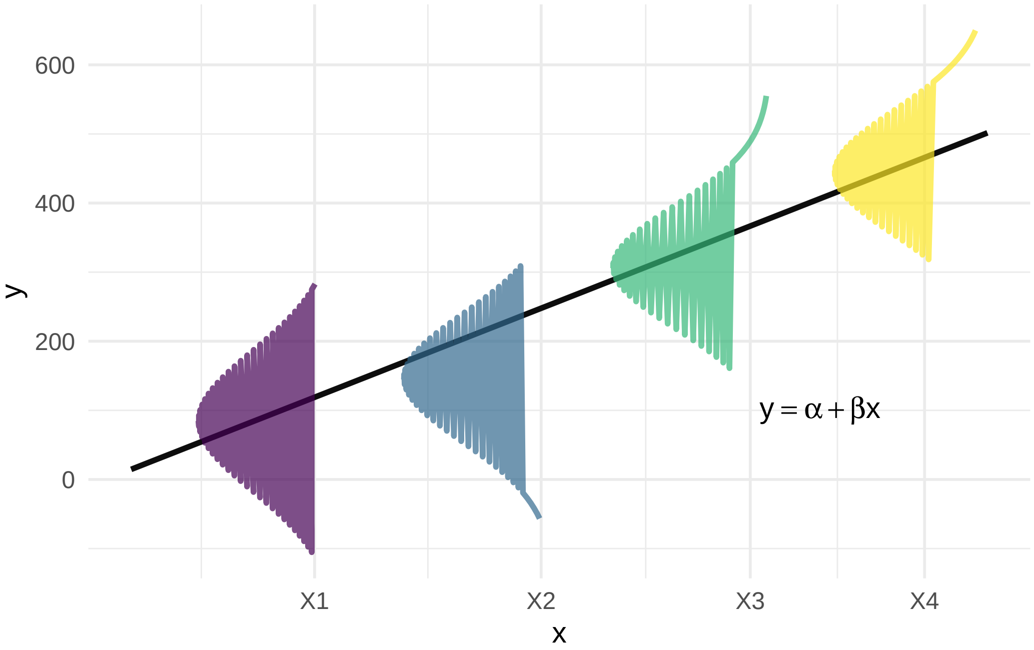
$$Y = \beta_0 + \beta_1 X_1$$

Este modelo se representa gráficamente como una recta de ajuste en un plano bidimensional (dos dimensiones).

Por otro lado, el modelo estadístico de la RLM es:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_k X_k$$

Donde β_0, β_1, β_2,...,β_k son los parámetros de la regresión. Para cada combinación de valores de X_1, X_2,...X_k existe una distribución Y cuya **media** es una función lineal de X_1, X_2,..., X_k.

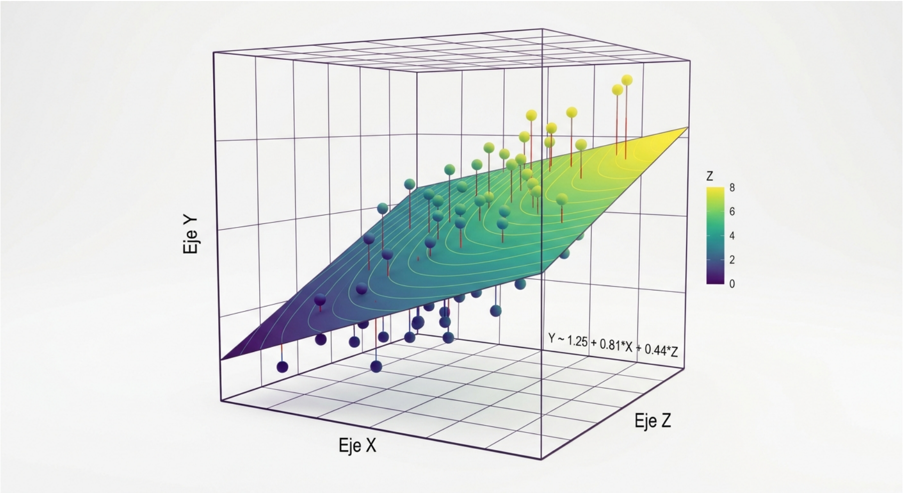


La representación gráfica de la recta de ajuste en la regresión lineal múltiple se realiza en un espacio de dimensión $K+1$, donde K es el número de variables independientes. En el caso de la regresión lineal simple (RLS), la relación se puede representar fácilmente en un plano bidimensional (2D). Sin embargo, a medida que aumentamos el número de variables independientes, la representación gráfica se vuelve más compleja, ya que requerimos más dimensiones, lo que dificulta visualizar el modelo en el espacio.

En el caso puntual que el modelo tuviera 2 variables independientes, la ecuación sería:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

En este caso, la relación podría representarse en un plano tridimensional (3D), donde los ejes corresponden a X_1, X_2 y la variable dependiente Y. En este plano, la superficie de ajuste es un plano que describe la relación entre las tres variables, y la orientación de dicho plano estará determinada por los coeficientes β_1 y β_2.



En forma similar a la RLS, la interpretación de cada parámetro β de la regresión es:

- β_0: es el valor esperado de Y cuando todas las otras variables son iguales a cero.
- β_1 es la pendiente a lo largo del eje X_1 y representa el cambio esperado en la respuesta por unidad de cambio en X_1 a valores constantes de X_2.
- β_2 es la pendiente a lo largo del eje X_2 y representa el cambio esperado en la respuesta por unidad de cambio en X_2 a valores constantes de X_1.

21.2. Presupuestos del modelo de RLM

21.2.1. Independencia

Las observaciones Y_i son independientes unas de otras: el efecto de X_1 sobre la respuesta media no depende de X_2 y viceversa, siempre y cuando no exista interacción. Cuando existe interacción entre X_1 e X_2 , el efecto de X_1 sobre la respuesta media de Y depende X_2 y viceversa (X_1 e X_2 no son independientes cuando existe interacción).

21.2.2. Linealidad

Para cada combinación de valores de las variables independientes (X_1, X_2,..., X_k) el valor medio de Y es función lineal de X_1, X_2,...,X_k.. La linealidad se define en relación a los coeficientes de la regresión, por lo tanto el modelo puede incluir términos cuadráticos e interacciones

- Modelo con interacción

$$Y = \beta_0 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2$$

- Modelo con términos cuadráticos

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2$$

21.2.3. Homocedasticidad

la varianza de Y para los distintos valores de X_1, X_2,...,X_k se mantiene constante.

21.2.4. Normalidad

Los valores de Y tienen una distribución normal según los valores de X_1, X_2, X_k , ésto nos permite realizar inferencias en relación a los parámetros del modelo.

Al igual que en la RLS la estimación de los parámetros de la regresión (coeficientes) se realiza mediante el **Método de los Mínimos Cuadrados (MMC)**. El mismo consiste en adoptar como estimativas de los parámetros de la regresión los valores que minimicen la suma de los cuadrados de los residuos.

$$\sum_{i=1}^{i=n} e_1^2 = \sum_{i=1}^{i=n} (Y_i - \hat{Y}_i)^2 = \sum_{i=n}^{i=n} (Y_i - (\hat{\beta}_0 + \hat{\beta}_1 X_1 + \cdots + \hat{\beta}_k X_k))^2$$

21.3. Interpretación del modelo

Comenzaremos aprendiendo cómo interpretar un modelo de regresión lineal múltiple (RLM) antes de abordar su construcción. Para ilustrarlo, observemos en detalle la salida de R para un modelo de RLM en el que modelamos la variable `V23`, en función de las variables `V10`, `V11`, `V12`, `V14`, `V15`, `V17`, `V18` y `V24`.

```
Call:
lm(formula = V23 ~ ., data = data)

Residuals:
     Min       1Q   Median       3Q      Max
-2.45958 -0.57136  0.05354  0.57868  2.25036

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.09504    0.09502   1.000   0.3199
V10         -0.05189    0.09725  -0.534   0.5950
V11         -0.11508    0.09711  -1.185   0.2391
V12         -0.08137    0.09151  -0.889   0.3763
V14         -0.16411    0.09738  -1.685   0.0954 .
V15         -0.03240    0.10283  -0.315   0.7534
V17         -0.04053    0.09040  -0.448   0.6550
V18          0.14425    0.09456   1.525   0.1306
V24         -0.04953    0.08940  -0.554   0.5809
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9081 on 91 degrees of freedom
Multiple R-squared:  0.09027,   Adjusted R-squared:  0.0103
F-statistic: 1.129 on 8 and 91 DF,  p-value: 0.3518
```

En la salida obtenida, encontramos varios términos clave para interpretar:

- `Estimate`: muestra los coeficientes estimados (β) para el intercepto (β_0) y para cada una de las variables explicativas (β_i).
- `Std. Error`: el error estándar de cada coeficiente estimado β.
- `t-value`: los valores del test t para cada coeficiente, que evalúan si los coeficientes son significativamente diferentes de cero.
- `Pr(>|t|)`: los valores p asociados a los test t para cada coeficiente.
- `Residual standard error`: el error estándar de los residuales.
- `Multiple R-squared`: el coeficiente de determinación R^2 múltiple, que indica qué proporción de la variabilidad de la variable dependiente es explicada por el modelo.
- `Adjusted R-squared`: el coeficiente de determinación ajustado, que penaliza por la inclusión de variables que no mejoran el modelo.
- `F-statistic`: el resultado del test F global, que evalúa la significancia del modelo en su conjunto, junto con su valor p.

21.3.1. Test F parcial

En un modelo de regresión, la estimación de los coeficientes se realiza partiendo de una muestra, lo que implica que diferentes muestras pueden generar distintos valores de los parámetros. El test F parcial evalúa la siguiente hipótesis:

$$H_0 = \beta_1 = \beta_2 = ...\beta_n = 0$$

$$H_1 = \exists \beta_i \neq 0$$

Donde H_0 indica que todos los coeficientes de las variables independientes son cero, y H_1 implica que al menos uno de los coeficientes es diferente de cero. Este test evalúa la contribución de cada variable al modelo, es decir, si la inclusión de esa variable mejora significativamente la explicación de la variabilidad de Y. En modelos de regresión lineales generalizados (GLM), el test de Wald cumple esta misma función, mientras que en los modelos de regresión lineales múltiplos, el test F parcial cumple el mismo rol.

21.3.2. Varianza residual

Al igual que en la **regresión lineal simple**y el **análisis de la varianza (ANOVA)**, podemos descomponer la variabilidad de la variable dependiente Y en dos componentes:

1. Variabilidad explicada por el modelo.
2. Variabilidad **no explicada** por el modelo, atribuida a factores aleatorios.

Matemáticamente, esto se expresa como:

$$Variabilidad\, total = Variabilidad\, regresin \;+\; Variabilidad\, residual$$

De manera equivalente, la suma de cuadrados totales (SCT) se descompone en la suma de cuadrados de la regresión (SCR) y la suma de cuadrados residuales (SCE):

$$\sum (y_i - \bar{y})^2 = \sum (\hat{y} - \bar{y})^2 + \sum (y_i - \hat{y}_i)^2$$

$$SCT = SCR + SCE$$

A continuación, mostramos cómo estos términos se distribuyen entre los grados de libertad y se obtienen los cuadrados medios correspondientes:

	Grados de libertad	CM
SCT	n-1	CMT = SCT/n-1
SCR	k-1	CMR = SCR/k-1
SCE	n-k-1	CME = SCE/n-k-1

21.3.3. Test F global

El test F global compara el modelo de regresión ajustado con el modelo nulo (sin variables independientes) y evalúa el efecto conjunto de todas las variables independientes. Su fórmula es:

$$F = CMR/CME$$
$$gl = (k-1, n-k-1)$$

21.3.4. Coeficiente de determinación

Al igual de lo que aprendimos en la RLS la bondad de ajuste del modelo de RLM se valora con el coeficiente de determinación (R^2), que nos dice qué proporción de la variabilidad de Y es explicada por los coeficientes de la regresión del modelo en estudio. Su valor se calcula como:

$$R^2 = \frac{SCT - SCE}{SCT} = \frac{SCR}{SCT}$$

Sin embargo, al agregar más variables al modelo, R^2 siempre mejora, incluso si la nueva variable no mejora sustancialmente el modelo. Este fenómeno se debe a que R^2 solo refleja la proporción de variabilidad explicada, pero no penaliza la inclusión de variables que no aportan al modelo.

Por esta razón, se utiliza el R^2 ajustado, que corrige el R^2 penalizando la inclusión de variables que no aportan significativamente:

$$R_a^2 = 1 - \left[\frac{n-1}{n-(k+1)}\right]\frac{SCE}{SCT} = 1 - \left[\frac{n-1}{n-(k+1)}\right](1-R^2)$$

Como el término $1 - R^2$ es constante y n es mayor que k, a medida que agregamos variables, el cociente entre paréntesis crece, lo que hace que la penalización por agregar variables no significativas también aumente. Esto hace que el R^2 ajustado sea una medida más fiable de la calidad del modelo.

> ⚠ Cuando tenemos que elegir el mejor modelo será necesario utilizar distintos criterios para compararlos y basar nuestra decisión en elegir el modelo que mejor explique la variación de Y con el menor número de variables independientes, el modelo más simple y efectivo, también llamado el modelo más **parsimonioso**.

21.4. Variables *dummy*

En los modelos de regresión, podemos incluir tanto variables cuantitativas como variables categóricas. Las variables categóricas pueden ser dicotómicas (por ejemplo, sexo: femenino/masculino; hábito de fumar: sí/no) o tener más de dos categorías (por ejemplo, grupo sanguíneo, religión, color de ojos).

Para modelar las variables categóricas, cada categoría se transforma en una variable dicotómica (binaria), con un número de categorías menos uno. Es decir, para una variable con n categorías, se crearán $n-1$ variables *dummy*, donde 1 indica la presencia de esa categoría y 0 indica su ausencia. En estos casos, se selecciona como grupo basal (o grupo de referencia) la categoría con el valor más bajo.

Ejemplo para variable independiente dicotómica

Supongamos que tenemos una variable cualitativa dicotómica, como "Hábito de Fumar", que toma los valores 1 (si fuma) y 0 (si no fuma). Si ajustamos un modelo con esta variable, la ecuación sería:

$$Y = \beta_0 + \beta_1 F_1$$

Donde:

- Y es la variable dependiente.
- β_0 es el intercepto (valor basal de Y cuando el individuo no fuma).
- β_1 es el coeficiente asociado al hábito de fumar, que indica el cambio en Y cuando el individuo fuma (es decir, la diferencia entre el grupo de fumadores y no fumadores).

Ejemplo para variable independiente con múltiples categorías

Ahora, consideremos una variable "Región", que tiene tres categorías: Noreste (NE), Norte (N) y Centro Oeste (CO). Para modelarla como variables *dummy*, la transformamos en dos variables: Re_1 y Re_2, ya que hay $n - 1 = 3 - 1 = 2$ categorías *dummy*.

La transformación de las categorías sería:

Región	RE1	RE2
NE (basal)	0	0
N	1	0
CO	0	1

En este caso:

- $Re_1 = 1$ si vive en el Norte, $Re_1 = 0$ si vive en otra región.
- $Re_2 = 1$ si vive en el Centro Oeste, $Re_2 = 0$ si vive en otra región.

Ahora imaginemos el modelo de regresión entre Y y la variable *"región"* (supongamos que Y es la Tasa de Mortalidad Infantil):

$$Y = \beta_0 + \beta_1 Re_1 + \beta_2 Re_2$$

Para interpretar el modelo:

- Si el individuo vive en la región Norte ($Re_1 = 1$ y $Re_2 = 0$), la ecuación se reduce a:

$$Y = \beta_0 + \beta_1 Re_1$$

- Si el individuo vive en la región Centro Oeste ($Re_1 = 0$ y $Re_2 = 1$), la ecuación sería:

$$Y = \beta_0 + \beta_2 Re_2$$

- Si el individuo vive en la región Noreste (grupo basal, $Re_1 = 0$ y $Re_2 = 0$), la ecuación se reduce a:

$$Y = \beta_0$$

En este contexto, β_1 y β_2 nos estarán indicando cuánto se modifica la TMI según consideremos la región Norte o Centro Oeste. El coeficiente β_0 es el valor basal medio de la TMI considerada en la región Noreste

Las variables *dummy* o indicadoras no tienen un significado por sí solas, ya que su interpretación depende de cómo se comparan con el grupo basal. Es importante que todas las categorías de la variable cualitativa estén representadas en el modelo, y la inclusión de estas variables debe ser contrastada en bloque, incluso si algunos de los tests F parciales para una categoría no son significativos.

Además, al agregar una variable *dummy* al modelo, esta incrementa los grados de libertad de la regresión en función de la cantidad de categorías de la variable. Por ejemplo, si una variable tiene n categorías, se agregarán $n - 1$ variables *dummy*, cada una con su respectivo grado de libertad.

Cuando trabajamos en R, el lenguaje se encarga de construir las variables *dummy* automáticamente siempre que su estructura sea **factor**. Los tipos de datos **factor** tienen una estructura de niveles donde el primero de ellos es el de referencia. Para modificar el nivel de referencia podemos utilizar las funciones `relevel()` de R `base` o `fct_relevel()` de `tidyverse`.

21.5. Estimación y predicción en modelos de regresión

La regresión es una herramienta clave en los análisis tanto explicativos como predictivos, y se utiliza de las siguientes maneras:

1. **Con fines explicativos**: Este uso tiene como objetivo obtener estimaciones precisas sobre variables de interés para realizar inferencias o cuantificar relaciones entre variables, controlando por otras. Aquí se busca entender cómo los cambios en las variables independientes afectan a la variable dependiente.
2. **Con fines predictivos**: La regresión también se utiliza para predecir el comportamiento de la variable dependiente (Y) basándose en los valores de las variables independientes (X_k) observados en la muestra. Sin embargo, la predicción solo debe hacerse si existe una correlación lineal adecuada y si el modelo ajustado es adecuado para los datos.

Consideraciones para la predicción:

- **No utilizar el modelo si no hay correlación lineal**: Si no existe una correlación lineal significativa entre Y y las X_k, la predicción no será válida. En estos casos, la mejor predicción es la media muestral de Y.
- **No extrapolar más allá de los valores muestrales conocidos**: Las predicciones fuera del rango de los datos observados pueden ser inexactas.
- **Datos actualizados**: La predicción debe basarse en datos recientes y relevantes.
- **No predecir para poblaciones distintas**: El modelo debe ser usado solo para hacer predicciones dentro de la población de la cual se obtuvieron los datos muestrales.
- **Relación no necesariamente causal para modelos predictivos**: Mientras que para modelos explicativos la relación entre X y Y debe ser causal (en términos temporales y lógicos), para la predicción basta con que exista una correlación.

21.6. Lineamientos la selección del modelo de regresión

Dada una variable dependiente Y y un conjunto de k variables independientes X_1, X_2, X_3....,X_k , nuestro interés es definir el mejor conjunto de p predictores ($p \leq k$) y el correspondiente modelo de regresión para describir la relación entre Y y las variables X.

Para ello, debemos seguir los siguientes pasos:

1. Especificar el conjunto de variables potencialmente predictivas/explicativas
2. Evaluar colinealidad
3. Especificar un criterio estadístico para la elección de las variables
4. Especificar una estrategia para seleccionar modelos
5. Conducir el análisis específico
6. Evaluar los presupuestos del modelo
7. Evaluar la confiabilidad del modelo escogido

21.6.1. Identificación de variables potencialmente predictivas/explicativas

El primer paso para construir un modelo de regresión es seleccionar un conjunto adecuado de variables predictivas. Esto implica identificar aquellas variables que mejor expliquen la variable dependiente sin que ninguna de ellas sea combinación lineal de las restantes.

El primer paso a seguir es identificar las relaciones entre la variable dependiente y las variables independientes de forma bivariada utilizando tests de correlación y gráficos de dispersión para las variables explicativas continuas y tests de asociación para las variables explicativas categóricas.

21.6.2. Colinealidad

Un problema frecuente en los modelos de RLM es el de la **multicolinealidad**, que ocurre cuando las variables independientes están relacionadas entre sí en forma lineal. Si bien no implica una violación de las hipótesis o presupuestos del modelo, puede ocasionar problemas en la inferencia, ya que:

- Aumenta las varianzas y covarianzas de los estimadores.
- Los errores de las estimaciones serán grandes.
- Tiende a producir estimadores con valores absolutos grandes.
- Los coeficientes de cada variable independiente difieren notablemente de los que se obtendrían por RLS.
- No se puede identificar de forma precisa el efecto individual de cada variable colineal sobre la variable respuesta.

A la hora de plantear modelos de RLM conviene estudiar previamente la existencia de casi-colinealidad (la colinealidad exacta no es necesario estudiarla previamente, ya que todos los algoritmos la detectan, de hecho no pueden acabar la estimación). Como medida de la misma hay varios estadísticos propuestos:

- Podemos examinar la matriz de correlación
- Realizar gráficos de dispersión entre las variables explicativas/predictivas
- Cálculo del factor de inflación de la varianza (VIF por sus siglas en inglés):

$$VIF = \frac{1}{1 - R_i^2}$$

Una regla empírica, citada por Kleinbaum et al. (1988), consiste en considerar que existen problemas de colinealidad si algún VIF es superior a 10. Por otro lado Kim (2019) considera que un VIF entre 5 y 10 indican la presencia de colinealidad. En caso de detectar colinealidad entre dos predictores, existen dos posibles soluciones:

- Excluir uno de los predictores problemáticos intentando conservar el que, a juicio del investigador, está influyendo realmente en la variable respuesta
- Combinar las variables colineales en un único predictor, aunque con el riesgo de perder su interpretación

21.6.3. Selección de variables

Generalmente incluiremos en el modelo aquellas variables que resultaron significativas en el análisis bivariado y otras que, aun cuando no resultaran significativas, decidamos mantener por cuestiones teóricas, porque se necesitan establecer predicciones para distintas categorías de dicha variable, etc.

El proceso de selección de variables puede realizarse en forma manual o automática, siendo este último desaconsejado por la mayoría de los autores, ya que en el proceso de ajuste de un modelo no sólo se involucran criterios estadísticos sino también conceptuales. Existen tres estrategias para realizar el proceso , basadas en el valor del test F parcial:

- **Método jerárquico o *forward:*** se basa en el criterio del investigador que introduce predictores determinados en un orden específico en relación al marco teórico. Comienza con un modelo nulo que solo contiene el intercepto (β_0) y agrega secuencialmente una variable a la vez, eligiendo la que proporciona el mayor beneficio en términos de ajuste del modelo. Este proceso continúa hasta que agregar más variables no mejore significativamente el ajuste del modelo.
- **Método de entrada forzada o *backward***: es el método inverso al anterior. Se introducen todos los predictores simultáneamente y, en cada paso, elimina la variable que tenga el menor impacto en el ajuste del modelo. Este proceso continúa hasta que eliminar más variables no mejore significativamente el ajuste del modelo. Permite evaluar cada variable en presencia de las otras.
- **Método paso a paso o *stepwise***: emplea criterios matemáticos para decidir qué predictores contribuyen significativamente al modelo y en qué orden se introducen. Se trata de una combinación de la selección *forward* y *backward.* Comienza con un modelo nulo, pero tras cada nueva incorporación se realiza un test de extracción de predictores no útiles como en el *backward.* Presenta la ventaja de que si a medida que se añaden predictores, alguno de los ya presentes deja de contribuir al modelo, se elimina.

21.6.4. Selección de modelos

Hasta ahora hemos desarrollado algunos criterios que se pueden utilizar para comparar modelos como R^2, R^2 ajustado y F global. Como hemos mencionado anteriormente el uso de R^2 como único criterio de selección tiene varias desventajas: tiende a sobreestimar, al adicionar variables siempre aumenta, por lo que si fuera el único criterio elegiría modelos con el mayor número de variables, no tiene en consideración la relación entre parámetros y tamaño muestral.

Para modelos anidados, podemos realizar una comparación entre ambos modelos mediante un ANOVA. Existen otros criterios como el Criterio de Información de Akaike (**AIC**), el Criterio de Información Bayesiano (**BIC**), etc. Tanto el BIC como el AIC, son funciones del logaritmo de la verosimilitud y un término de penalidad basado en el número de parámetros del modelo.

Recuerden que frente a p variables independientes existen 2^p posibles modelos. No necesariamente el modelo con mayor número de variables es el mejor. Debemos priorizar siempre el principio de parsimonia (el modelo más simple que mejor explique). El tamaño de la muestra también es importante, algunos autores recomiendan que el número de observaciones sea como mínimo entre 10 y 20 veces el número de predictores del modelo.

21.7. Validación y diagnóstico del modelo

Utilizaremos el análisis de los residuales para realizar los contrastes *a posteriori* de las hipótesis del modelo. Recordemos que los residuos o residuales se definen como la diferencia entre el valor observado y el valor predicho por el modelo.

$$y - \hat{y} = e\,(residuo\, o\, error\, residual)$$

El planteamiento habitual es considerar que, como dijimos inicialmente, los valores de Y tienen una distribución normal según los valores de X_1, X_2,... X_k., entonces, los residuos también tendrán una distribución normal. Los residuos tienen unidades de medida y, por tanto no se puede determinar si es grande o pequeño a simple vista. Para solucionar este problema se define el residuo estandarizado como el cociente entre el residuo y su desvío estandar. Se considera que un residuo tiene un valor alto, y por lo tanto puede influir negativamente en el análisis, si su residuo estandarizado es mayor a 3 en valor absoluto. También se trabaja con los residuos tipificados o con los residuos estudentizados.

21.7.1. Normalidad

El análisis de normalidad de los residuos lo realizaremos gráficamente (Histograma y gráfico de probabilidad normal) y analíticamente (Contraste de Kolmogorov-Smirnov) o similar.

21.7.2. Homocedasticidad y linealidad

La hipótesis de homocedasticidad establece que la variabilidad de los residuos es independiente de las variables explicativas. En general, la variabilidad de los residuos estará en función de las variables explicativas, pero como las variables explicativas están fuertemente correlacionadas con la variable dependiente, bastara con examinar el gráfico de valores pronosticados versus residuos (a veces residuos al cuadrado).

Comprobamos la hipótesis de homogeneidad de las varianzas gráficamente representando los residuos tipificados frente a los valores predichos por el modelo. El análisis de este gráfico puede revelar una posible violación de la hipótesis de homocedasticidad, por ejemplo si detectamos que el tamaño de los residuos aumenta o disminuye de forma sistemática para algunos valores ajustados de la variable Y, si observamos que el gráfico muestra forma de embudo. Si por el contrario dicho gráfico no muestra patrón alguno, entonces no podemos rechazar la hipótesis de igualdad de varianzas.

21.7.3. Valores de influencia (*leverage*)

Se considera que una observación es influyente *a priori* si su inclusión en el análisis modifica sustancialmente el sentido del mismo. Una observación puede ser influyente si es un *outlier* respecto a alguna de las variables explicativas. Para detectar estos problemas se utiliza la medida de **Leverage**:

$$l(i) = \frac{1}{n}\left(1 + \frac{(x_i - \bar{x})^2}{S_x^2}\right)$$

Este estadístico mide la distancia de un punto a la media de la distribución. Valores cercanos a 2/n indican casos que pueden influir negativamente en la estimación del modelo introduciendo un fuerte sesgo en el valor de los estimadores.

21.7.4. Distancia de Cook

Es una medida de cómo influye la observación i-ésima sobre la estimación de β al ser retirada del conjunto de datos. Una distancia de Cook grande significa que una observación tiene un peso grande en la estimación de β. Son puntos influyentes las observaciones que presenten

$$D_i = \frac{4}{n-p-2}$$

21.7.5. Independencia de residuos

La hipótesis de independencia de los residuos la realizaremos mediante el contraste de *Durbin-Watson*.

21.8. Ejemplo práctico en R

En el documento **Introducción al modelado estadístico** vimos como generar en R una fórmula para un modelo lineal simple:

$$variable\ dependiente\ \sim variable\ independiente$$

Para generar fórmulas que contengan más de una variable independiente o predictora (necesario para que sea una regresión lineal múltiple) debemos agregarlas mediante el símbolo $+$

$$variable\ dependiente\ \sim var\_indepen\_1\ +\ var\_indepen\_2\ +\ ...\ +\ var\_indepen\_n$$

Si luego de la ~ incluímos **un punto** como notación de “todas”, estamos creando un *modelo saturado* con todas las variables incluidas dentro del *dataframe*:

```
lm(y ~ ., data = datos)
```

Esto es útil cuando tenemos muchas posibles variables explicativas y queremos conocer cuáles tienen una correlación significativa.

También se puede descartar alguna o algunas variables explicativas en particular basado en la misma estructura, mediante el símbolo —

```
lm(y ~ . - x2, data = datos)
```

En la línea anterior, incluimos dentro del modelo a todas las variables de `data` menos `x2`.

Usaremos para el ejemplo el dataset “`cancer_USA.txt`”, que contiene información sobre la tasa de mortalidad por cáncer cada 100.000 habitantes de distintos condados de Estados Unidos.

21.8.1. Lectura de datos y visualización de estructura

Para comenzar, cargamos los paquetes de R necesarios para el análisis:

```
# correlogramaas
library(GGally)

# chequeo de supuestos y análisis de residuales
library(easystats)
library(gvlma)
library(lmtest)
library(nortest)

# manejo de datos
library(tidyverse)
```

A continuación leemos los datos y realizamos una exploración rápida

```
# Carga datos
datos <- read_csv2("datos/cancer_USA.txt")

# Explora datos
glimpse(datos)
```

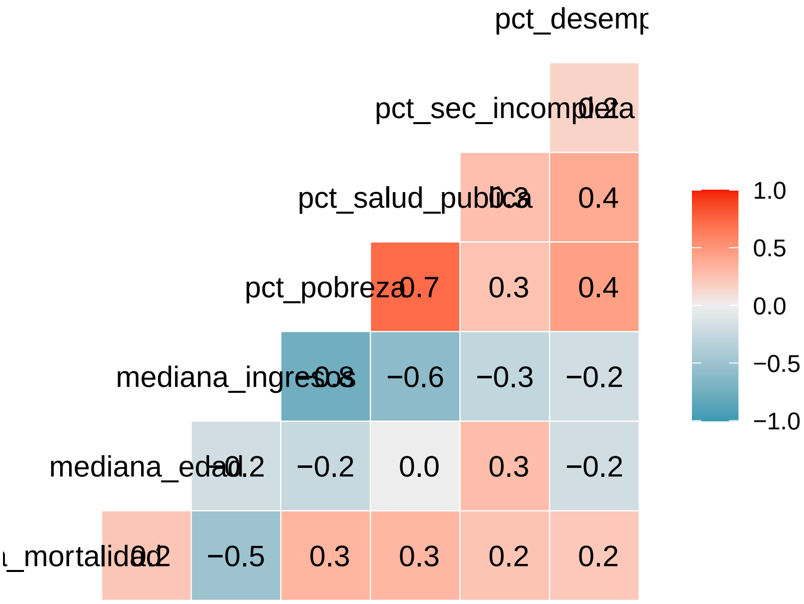
```
Rows: 213
Columns: 10
$ condado            <chr> "Belknap County", "Carroll County", "Cheshire Count~
$ estado             <chr> "New Hampshire", "New Hampshire", "New Hampshire", ~
$ tasa_mortalidad    <dbl> 182.6, 168.8, 162.8, 181.6, 155.0, 163.2, 173.1, 17~
$ mediana_edad       <dbl> 46.1, 50.3, 42.0, 48.1, 41.9, 40.1, 42.6, 43.5, 37.~
$ mediana_edad_cat   <chr> "45+ años", "45+ años", "36-39 años", "45+ años", "~
$ mediana_ingresos   <dbl> 59831, 57556, 56008, 42491, 56353, 71233, 62429, 79~
$ pct_pobreza        <dbl> 10.0, 10.7, 11.8, 14.9, 11.6, 8.7, 9.5, 6.1, 11.8, ~
$ pct_salud_publica  <dbl> 16.8, 13.4, 12.7, 22.5, 14.0, 12.7, 13.1, 9.0, 12.4~
$ pct_sec_incompleta <dbl> 15.4, 14.8, 6.1, 13.2, 6.6, 12.9, 10.9, 13.5, 6.1, ~
$ pct_desempleo      <dbl> 5.3, 5.8, 6.1, 6.9, 5.0, 5.9, 5.2, 5.6, 6.5, 5.8, 1~
```

La base de datos tiene 100 observaciones y 9 variables. La variable dependiente es `tasa_mortalidad` y hay 3 variables independientes categóricas y 6 variables independientes numéricas o continuas.

21.8.2. Análisis de variables potencialmente explicativas

Analizaremos la relación entre las variables explicativas numéricas y la variable dependiente mediante un correlograma con el paquete `GGally`:

```
datos |>
  # selecciono variables numéricas
  select(where(is.numeric)) |>

  # correlograma
  ggcorr(label = T)
```



Como habíamos observado en el capítulo sobre **covarianza y correlación**, la `tasa_mortalidad` tiene correlación negativa moderada con `mediana_ingresos`, también que existe una correlación negativa fuerte entre `pct_pobreza` y `mediana_ingresos` y moderada con `pct_salud_publica`.

21.8.3. Estrategia de construcción del modelo

Al momento de seleccionar las variables independientes que formarán parte del modelo, una de las herramientas más utilizadas es el Criterio de Información de Akaike (AIC) que ajusta mediante *máxima verosimilitud*. Al penalizar la complejidad excesiva, el AIC ayuda a prevenir el sobreajuste y favorece la inclusión de variables relevantes buscando el modelo equilibrado que describa adecuadamente la relación y tenga el mínimo AIC. Podemos consultar el AIC de un modelo en R mediante la función `AIC()`.

Como vimos anteriormente, la selección de variables se puede hacer mediante métodos *forward*, *backward* o *stepwise*. La función `step()` de R `base` permite encontrar de forma automática el mejor modelo basado en AIC utilizando cualquiera de las 3 variantes del método paso a paso. Por otro lado, la función `drop1()` de `R base` nos permite realizar un proceso *backward* manual, eligiendo que variable quitar del modelo en cada caso según su contribución al AIC. Siempre es oportuno aclarar que estos últimos métodos se basan en cálculos matemático/estadísticos que no tienen en cuenta criterios conceptuales epidemiológicos que surjan del marco teórico, por lo que exigen un control especial del analista.

el lenguaje R ofrece una variedad de funciones para abordar y facilitar la tarea de seleccionar el mejor modelo de regresión (más parsimonioso). Como existe correlación fuerte entre `mediana_ingresos`, `pct_pobreza` y `pct_salud_publica` sólo usaremos la que presente mayor correlación con `tasa_mortalidad`.

```r
# Modelo saturado
mod_full <- lm(
  tasa_mortalidad ~ mediana_ingresos +
    mediana_edad +
    pct_sec_incompleta +
    pct_desempleo +
    estado,
  data = datos
)

# Mediana ingresos
mod1 <- lm(tasa_mortalidad ~ mediana_ingresos, data = datos)

# Mediana edad
mod2 <- lm(tasa_mortalidad ~ mediana_edad, data = datos)

# Secundaria incompleta
mod3 <- lm(tasa_mortalidad ~ pct_sec_incompleta, data = datos)

# Desempleo
mod4 <- lm(tasa_mortalidad ~ pct_desempleo, data = datos)

# Estado
mod5 <- lm(tasa_mortalidad ~ estado, data = datos)
```

Mediante `summary()` observamos sus resultados:

```r
# Modelo saturado
summary(mod_full)
```

```

Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos + mediana_edad +
    pct_sec_incompleta + pct_desempleo + estado, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max
-31.475  -7.254   1.161   7.192  29.261

Coefficients:
```

```
                      Estimate Std. Error t value Pr(>|t|)    
(Intercept)          1.488e+02  1.487e+01  10.006  < 2e-16 ***
mediana_ingresos    -3.810e-04  8.165e-05  -4.666 5.62e-06 ***
mediana_edad         5.005e-01  2.588e-01   1.934  0.05453 .  
pct_sec_incompleta   1.652e-01  1.945e-01   0.849  0.39681    
pct_desempleo        1.643e+00  5.781e-01   2.842  0.00494 ** 
estadoMaine          1.462e+01  5.662e+00   2.582  0.01054 *  
estadoMassachusetts  6.389e+00  5.344e+00   1.196  0.23328    
estadoNew Hampshire  1.166e+01  5.963e+00   1.955  0.05198 .  
estadoNew Jersey     1.020e+01  5.010e+00   2.036  0.04302 *  
estadoNew York       7.909e+00  4.706e+00   1.680  0.09443 .  
estadoPennsylvania   9.905e+00  4.867e+00   2.035  0.04315 *  
estadoRhode Island   5.131e+00  7.359e+00   0.697  0.48646    
estadoVermont        1.325e+01  5.814e+00   2.278  0.02377 *  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.93 on 200 degrees of freedom
Multiple R-squared:  0.3373,	Adjusted R-squared:  0.2975 
F-statistic: 8.483 on 12 and 200 DF,  p-value: 6.303e-13
```

```r
# Modelos simples
summary(mod1)
```

```

Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max 
-33.040  -8.210   1.093   7.637  35.932 

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)    
(Intercept)       2.014e+02  3.521e+00  57.200  < 2e-16 ***
mediana_ingresos -5.200e-04  6.133e-05  -8.479 3.94e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12.32 on 211 degrees of freedom
Multiple R-squared:  0.2542,	Adjusted R-squared:  0.2506 
F-statistic:  71.9 on 1 and 211 DF,  p-value: 3.938e-15
```

```r
summary(mod2)
```

```

Call:
lm(formula = tasa_mortalidad ~ mediana_edad, data = datos)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-31.870  -9.631   0.512   9.539  46.770

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  134.7306    10.9761  12.275  < 2e-16 ***
mediana_edad   0.8961     0.2600   3.447 0.000684 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.88 on 211 degrees of freedom
Multiple R-squared:  0.05332,	Adjusted R-squared:  0.04883
F-statistic: 11.88 on 1 and 211 DF,  p-value: 0.0006835
```

```r
summary(mod3)
```

```

Call:
lm(formula = tasa_mortalidad ~ pct_sec_incompleta, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max
-33.903  -9.319   1.485   9.185  41.404

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)        162.9089     2.8037  58.106  < 2e-16 ***
pct_sec_incompleta   0.7202     0.1997   3.607 0.000387 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.84 on 211 degrees of freedom
Multiple R-squared:  0.05808,	Adjusted R-squared:  0.05362
F-statistic: 13.01 on 1 and 211 DF,  p-value: 0.0003867
```

```r
summary(mod4)
```

```

Call:
lm(formula = tasa_mortalidad ~ pct_desempleo, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max
-35.594  -8.948   0.508   9.029  40.157

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   158.9950     4.1741  38.091  < 2e-16 ***
```

```
pct_desempleo   1.7906     0.5418   3.305  0.00112 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.91 on 211 degrees of freedom
Multiple R-squared:  0.04922,	Adjusted R-squared:  0.04471 
F-statistic: 10.92 on 1 and 211 DF,  p-value: 0.001117
```

```r
summary(mod5)
```

```
Call:
lm(formula = tasa_mortalidad ~ estado, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max 
-32.840  -9.533   0.360   9.100  36.641 

Coefficients:
                    Estimate Std. Error t value Pr(>|t|)    
(Intercept)          157.713      4.785  32.960  < 2e-16 ***
estadoMaine           25.438      5.860   4.341 2.24e-05 ***
estadoMassachusetts    7.087      5.998   1.182 0.238741    
estadoNew Hampshire   13.167      6.420   2.051 0.041533 *  
estadoNew Jersey      10.921      5.623   1.942 0.053491 .  
estadoNew York        13.928      5.084   2.739 0.006700 ** 
estadoPennsylvania    17.347      5.075   3.418 0.000762 ***
estadoRhode Island     8.263      8.288   0.997 0.319968    
estadoVermont         18.559      5.998   3.094 0.002251 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.53 on 204 degrees of freedom
Multiple R-squared:  0.1297,	Adjusted R-squared:  0.09555 
F-statistic:   3.8 on 8 and 204 DF,  p-value: 0.0003514
```

En los resúmenes observamos que todos los coeficientes de las variables involucradas son significativas y que los valores mostrados tienen forma de lista, es decir no son tablas "ordenadas", por lo que muchas veces cuando trabajamos con numerosas variables, se hace difícil la comparación de resultados.

21.8.4. Comparación de modelos

El comando `compare_performance()` del paquete `performance` permite comparar de manera sencilla entre modelos anidados:

```r
compare_performance(mod_full, mod1, mod2, mod3, mod4, mod5, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name     | Model |  AIC (weights) |  BIC (weights) |    R2 | R2 (adj.) |   RMSE
-------------------------------------------------------------------------------
mod_full |    lm | 1675.0 (0.830) | 1722.1 (<.001) | 0.337 |     0.298 | 11.557
mod1     |    lm | 1678.2 (0.170) | 1688.3 (>.999) | 0.254 |     0.251 | 12.261
mod2     |    lm | 1729.0 (<.001) | 1739.1 (<.001) | 0.053 |     0.049 | 13.814
mod3     |    lm | 1727.9 (<.001) | 1738.0 (<.001) | 0.058 |     0.054 | 13.779
mod4     |    lm | 1729.9 (<.001) | 1740.0 (<.001) | 0.049 |     0.045 | 13.844
mod5     |    lm | 1725.1 (<.001) | 1758.7 (<.001) | 0.130 |     0.096 | 13.245
```

Las métricas comunes extraídas de los objetos de regresión, presentadas en una tabla, son:

- `AIC`: Criterio de información de Akaike
- `BIC`: Criterio de información bayesiano
- `R2`: Coeficiente de determinación R^2
- `R2 (adj.)`: Coeficiente de determinación ajustado
- `RMSE`: Error cuadrático medio

También podemos ordenar los modelos de mejor a peor con el argumento `rank = TRUE`:

```
compare_performance(
  mod_full,
  mod1,
  mod2,
  mod3,
  mod4,
  mod5,
  metrics = "common",
  rank = TRUE
)
```

```
# Comparison of Model Performance Indices

Name     | Model |    R2 | R2 (adj.) |   RMSE | AIC weights | BIC weights | Performance-
Score
----------------------------------------------------------------------------------------------
mod_full |    lm | 0.337 |     0.298 | 11.557 |       0.830 |    4.58e-08 |           80.00%
mod1     |    lm | 0.254 |     0.251 | 12.261 |       0.170 |       1.000 |           68.44%
mod5     |    lm | 0.130 |     0.096 | 13.245 |    1.12e-11 |    5.15e-16 |           14.85%
mod3     |    lm | 0.058 |     0.054 | 13.779 |    2.71e-12 |    1.60e-11 |            1.89%
mod2     |    lm | 0.053 |     0.049 | 13.814 |    1.59e-12 |    9.36e-12 |            0.87%
mod4     |    lm | 0.049 |     0.045 | 13.844 |    1.00e-12 |    5.90e-12 |        1.18e-
10%
```

Podemos ver que el modelo saturado tiene un menor AIC y *performance* global que los modelos univariados.

21.8.5. Colinealidad

Hay varias maneras de detectar la presencia de colinealidad entre variables independientes:

- Antes de ajustar el modelo, al comparar los coeficientes de correlación.
- Una vez ajustado el modelo, si tenemos un R^2 alto y muchas variables independientes que no son estadísticamente significativas.

Cuando hemos intuido que tenemos multicolinealidad y queremos comprobar, el paquete `performance` nos ofrece la función `check_collinearity()` que implementa el método de factor de inflación de la varianza (VIF):

```
check_collinearity(mod_full)
```

```
# Check for Multicollinearity

Low Correlation

              Term  VIF   VIF 95% CI adj. VIF Tolerance Tolerance 95% CI
   mediana_ingresos 1.89 [1.60, 2.31]     1.38      0.53     [0.43, 0.62]
       mediana_edad 1.34 [1.19, 1.63]     1.16      0.75     [0.61, 0.84]
 pct_sec_incompleta 1.28 [1.14, 1.56]     1.13      0.78     [0.64, 0.88]
      pct_desempleo 1.55 [1.34, 1.88]     1.24      0.65     [0.53, 0.75]
             estado 2.52 [2.09, 3.12]     1.59      0.40     [0.32, 0.48]
```

Los resultados del ejemplo muestra VIF de entre 1.28 y 2.52 (no hay colinealidad). Recordemos que el umbral de detección parte de valores cercanos a 5 y un $VIF \geq 10$ indicaría que el modelo de regresión lineal presenta un grado de multicolinealidad preocupante.

21.8.6. Análisis de residuales

Como vimos para la regresión lineal simple, podemos analizar gráficamente los residuales con la función `check_model()` del paquete `performance`:

```
check_model(mod_full)
```

Por otra parte, también tenemos las funciones de análisis para los supuestos (independencia, linealidad, normalidad, homocedasticidad).

- **Independencia:** El paquete `lmtest` permite evaluar independencia según estadístico de Durbin-Watson.

```
dwtest(mod_full)
```

```

	Durbin-Watson test

data:  mod_full
DW = 2.1769, p-value = 0.7772
alternative hypothesis: true autocorrelation is greater than 0
```

- **Linealidad:** El paquete `lmtest` implementa el *Ramsey's RESET* bajo la función `resettest()`

```
resettest(mod_full)
```

```

	RESET test

data:  mod_full
RESET = 0.66971, df1 = 2, df2 = 198, p-value = 0.513
```

- **Normalidad:** El paquete `nortest` permite chequear normalidad mediante test de Lilliefors.

```
lillie.test(mod_full$residuals)
```

```

	Lilliefors (Kolmogorov-Smirnov) normality test

data:  mod_full$residuals
D = 0.060065, p-value = 0.05954
```

- **Homocedasticidad:** El paquete `performance` permite chequear homocedasticidad con el test de Breush-Pagan.

```
check_heteroscedasticity(mod_full)
```

```
Warning: Heteroscedasticity (non-constant error variance) detected (p = 0.012).
```

Finalmente presentamos un paquete interesante para validar supuestos de modelos lineales denominado `gvlma`(Pena y Slate 2019). Su función, de mismo nombre `gvlma()`, implementa un procedimiento global sobre vector residual estandarizado para probar los cuatro supuestos del modelo lineal.

Si el procedimiento global indica una violación de al menos uno de los supuestos, los componentes se pueden utilizar para obtener información sobre qué supuestos se han violado.

```
gvlma(mod_full)
```

```

Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos + mediana_edad + 
    pct_sec_incompleta + pct_desempleo + estado, data = datos)

Coefficients:
        (Intercept)     mediana_ingresos         mediana_edad  
         148.784543            -0.000381             0.500486  
 pct_sec_incompleta        pct_desempleo          estadoMaine  
           0.165173             1.643031            14.617808  
estadoMassachusetts  estadoNew Hampshire     estadoNew Jersey  
           6.388818            11.657941            10.203316  
     estadoNew York   estadoPennsylvania   estadoRhode Island  
```

```
          7.908722             9.905042             5.131109
      estadoVermont
         13.245742


ASSESSMENT OF THE LINEAR MODEL ASSUMPTIONS
USING THE GLOBAL TEST ON 4 DEGREES-OF-FREEDOM:
Level of Significance =  0.05

Call:
 gvlma(x = mod_full)

                    Value p-value                Decision
Global Stat        2.3043  0.6800 Assumptions acceptable.
Skewness           0.9587  0.3275 Assumptions acceptable.
Kurtosis           0.0572  0.8110 Assumptions acceptable.
Link Function      0.4246  0.5147 Assumptions acceptable.
Heteroscedasticity 0.8639  0.3527 Assumptions acceptable.
```

21.8.7. Resumen del mejor modelo elegido

```
summary(mod_full)
```

```
Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos + mediana_edad +
    pct_sec_incompleta + pct_desempleo + estado, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max
-31.475  -7.254   1.161   7.192  29.261

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)          1.488e+02  1.487e+01  10.006  < 2e-16 ***
mediana_ingresos    -3.810e-04  8.165e-05  -4.666 5.62e-06 ***
mediana_edad         5.005e-01  2.588e-01   1.934  0.05453 .
pct_sec_incompleta   1.652e-01  1.945e-01   0.849  0.39681
pct_desempleo        1.643e+00  5.781e-01   2.842  0.00494 **
estadoMaine          1.462e+01  5.662e+00   2.582  0.01054 *
estadoMassachusetts  6.389e+00  5.344e+00   1.196  0.23328
estadoNew Hampshire  1.166e+01  5.963e+00   1.955  0.05198 .
estadoNew Jersey     1.020e+01  5.010e+00   2.036  0.04302 *
estadoNew York       7.909e+00  4.706e+00   1.680  0.09443 .
estadoPennsylvania   9.905e+00  4.867e+00   2.035  0.04315 *
estadoRhode Island   5.131e+00  7.359e+00   0.697  0.48646
estadoVermont        1.325e+01  5.814e+00   2.278  0.02377 *
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.93 on 200 degrees of freedom
Multiple R-squared:  0.3373,    Adjusted R-squared:  0.2975
F-statistic: 8.483 on 12 and 200 DF,  p-value: 6.303e-13
```

El modelo `mod_full` es capaz de explicar el 29,8 % de la variabilidad observada en la tasa de mortalidad por cáncer en estos condados ($R^2_{ajustado} = 0,2975$). El test F global muestra que es significativo ($p < 0,001$).

Los coeficientes se pueden interpretar como:

- **Mediana de ingresos**: Cada aumento de una unidad en la mediana de ingresos reduce la tasa de mortalidad en 0,0004 unidades.
- **Mediana de edad**: Cuando aumento una unidad en la mediana de edad incrementa la tasa de mortalidad en 0,5 unidades.
- **Porcentaje de población con secundaria incompleta**: Cada aumento de una unidad en este porcentaje incrementa la tasa de mortalidad en 0,17 unidades, pero esta relación no es significativa.
- **Porcentaje de desempleo**: Cada aumento de una unidad en el desempleo incrementa la tasa de mortalidad en 1.64 unidades.
- **Estados**: Las tasas de mortalidad varían entre los estados, con algunos efectos significativos, como en Maine, New Jersey, Pennsylvania y Vermont, todos con tasas de mortalidad más altas que el estado de referencia. Es decir, la recta de regresión para cada estado tendrá un intercepto diferente.

21.9. Funciones para modelado automático

El uso de la función `step()` se enmarca en los procedimientos automáticos y puede ser *forward*, *backward* o mixto. La sintaxis básica de la función es:

```r
step(object = modelo, direction = c("both", "backward", "forward"))
```

donde:

- `object`: es el modelo inicial de regresión lineal (nulo o saturado).
- `direction`: es la dirección que indicamos. Puede ser `"forward"`, `"backward"` o `"both"` (predeterminado, si no se define).

Generalmente conviene partir del modelo saturado y utilizar la dirección por defecto, donde se usan ambas direcciones.

```r
modelo_step <- step(mod_full, direction = "both")
```

```
Start:  AIC=1068.56
tasa_mortalidad ~ mediana_ingresos + mediana_edad + pct_sec_incompleta + 
    pct_desempleo + estado

                     Df Sum of Sq   RSS    AIC
```

```
- estado              8   1470.71 29922 1063.3
- pct_sec_incompleta  1    102.58 28554 1067.3
<none>                            28452 1068.6
- mediana_edad        1    532.09 28984 1070.5
- pct_desempleo       1   1149.28 29601 1075.0
- mediana_ingresos    1   3096.96 31549 1088.6

Step:  AIC=1063.3
tasa_mortalidad ~ mediana_ingresos + mediana_edad + pct_sec_incompleta + 
    pct_desempleo

                     Df Sum of Sq   RSS    AIC
- pct_sec_incompleta  1      79.5 30002 1061.9
<none>                            29922 1063.3
- mediana_edad        1    1033.9 30956 1068.5
- pct_desempleo       1    1036.6 30959 1068.5
+ estado              8    1470.7 28452 1068.6
- mediana_ingresos    1    6922.4 36845 1105.6

Step:  AIC=1061.86
tasa_mortalidad ~ mediana_ingresos + mediana_edad + pct_desempleo

                     Df Sum of Sq   RSS    AIC
<none>                            30002 1061.9
+ pct_sec_incompleta  1      79.5 29922 1063.3
+ estado              8    1447.7 28554 1067.3
- pct_desempleo       1    1176.8 31179 1068.1
- mediana_edad        1    1302.7 31305 1068.9
- mediana_ingresos    1    7528.4 37530 1107.5
```

Observamos que el proceso automático nos indica que debemos eliminar las variables explicativas `pct_sec_incompleta` y `estado`.

```
summary(modelo_step)
```

```


Call:
lm(formula = tasa_mortalidad ~ mediana_ingresos + mediana_edad + 
    pct_desempleo, data = datos)

Residuals:
    Min      1Q  Median      3Q     Max 
-31.570  -7.621   1.269   7.564  32.218 

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)    
(Intercept)       1.574e+02  1.271e+01  12.388  < 2e-16 ***
mediana_ingresos -4.517e-04  6.237e-05  -7.242 8.34e-12 ***
mediana_edad      7.065e-01  2.345e-01   3.013  0.00291 ** 
pct_desempleo     1.397e+00  4.881e-01   2.863  0.00462 ** 
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.98 on 209 degrees of freedom
Multiple R-squared:  0.3012,    Adjusted R-squared:  0.2912
F-statistic: 30.03 on 3 and 209 DF,  p-value: 3.488e-16
```

Si bien en el ejemplo mostramos como usar el método *stepwise* automático para la selección de variables, no se recomienda su uso ya que está basado únicamente en criterios matemáticos y no en la relevancia en el mundo real.

Además, cuenta con las siguientes limitaciones:

- Sensibilidad a la selección de variables
- Propensión al sobreajuste
- Violación de supuestos estadísticos
- Inclusión de variables irrelevantes

Para evitar estos inconvenientes, es recomendable usar métodos alternativos de selección de variables tales como:

- **Selección manual de variables**, según su relevancia teórica y conocimiento experto del tema.
- **Selección de variables por AIC o BIC** (Criterio de Información Bayesiano).
- **Regularización** por regresión LASSO (*Least Absolute Shrinkage and Selection Operator*), que penaliza los coeficientes de las variables menos importantes, favoreciendo modelos más simples y generalizables. Para conocer más sobre este método pueden entrar al siguiente enlace:

  ⍰ **Stopping stepwise**
- **Árboles de decisión** (*random forest* y *gradient boosting),* herramientas de aprendizaje automático que permiten manejar automáticamente la selección de variables con menor probabilidad de sobreajuste. Para conocer más sobre este método pueden entrar al siguiente enlace:

  ⍰ **Decision trees**

Estos dos últimos métodos escapan al alcance del curso. Sin embargo, invitamos a quienes tengan interés a explorar las múltiples posibilidades que ofrecen, en particular al momento de analizar datasets grandes y con múltiples variables explicativas.

Parte IV.

Unidad 4

22. Estudios de casos y controles

22.1. Introducción

Un estudio de casos y controles (CC) es un estudio analítico observacional que comienza con la identificación de dos grupos:

- Casos: individuos que presentan algún evento particular de salud.

- Controles: individuos que no presentan el evento.

El objetivo es comparar la frecuencia de exposición a un factor de interés entre ambos grupos. La hipótesis subyacente es que si la exposición es más frecuente en los casos que en los controles, podría tratarse de un **factor de riesgo**. En cambio, si la exposición es menos frecuente en los casos, podría actuar como un **factor protector**.

Los principales aspectos metodológicos a tener en cuenta para llevar adelante un estudio de casos y controles son:

- Definición precisa de la variable dependiente (evento de interés).
- Definición de las variables independientes (factores de exposición).
- Criterios de selección de los casos (fuente y definición).
- Criterios de selección de los controles (fuente y criterios de inclusión/exclusión).
- Análisis estadístico.

Es fundamental tener criterios precisos para la definición de caso. Los criterios pueden estar basados en resultados de pruebas de laboratorio o pueden estar basados en la historia clínica o en encuestas.

Según la definición de casos, se diferencian dos tipos importantes de estudios de casos y controles:

- **Estudios de casos y controles con casos incidentes**: Incluyen solo casos nuevos de la enfermedad durante el período de estudio.

- **Estudios de casos y controles con casos prevalentes**: Consideran todos los individuos que tienen la enfermedad en un momento determinado.

Algunos autores consideran los estudios de CC como una estrategia alternativa para estudiar una cohorte. Esto permite diferenciar dos enfoques:

- **Casos en cohortes de densidad de incidencia**: Los casos se reclutan conforme aparecen (incidentes). Este enfoque es útil cuando la exposición tiene períodos de latencia prolongados.
- **Casos en cohortes de incidencia acumulada**: Los casos se seleccionan al final del estudio, con un período de seguimiento previamente definido.

22.1.1. Selección de los controles

La fuente de los controles quedará definida en la medida en que se definan claramente los criterios de selección de casos, así como la población hipotética de origen. Algunos puntos a considerar son los siguientes:

- Los controles deben seleccionarse de la misma base poblacional (de la cohorte hipotética) de donde se originaron los casos. Si uno de estos controles desarrollara el evento en cuestión, pasaría a formar parte del grupo de los casos. Esto último implica que los controles estuvieron en riesgo de desarrollar el evento en forma simultánea a los casos.
- Los controles deben seleccionarse independientemente de su condición de expuestos o no expuestos.
- La probabilidad de selección para los controles debe ser proporcional al tiempo que el sujeto estuvo en riesgo de desarrollar el evento o enfermedad en estudio.
- Para minimizar confusión, se pueden emparejar controles con casos en función de variables confusoras.
- Los procedimientos para medir la exposición, deben ser los mismos, tanto en casos como en controles.

Las fuentes de controles pueden incluir población general, hospitales, vecindarios, registros de mortalidad, entre otros.

22.2. Clasificación de los estudios de casos y controles

En los últimos tiempos, se ha profundizado en los aspectos metodológicos de los estudios de casos y controles; y también se ha clarificado la estrecha relación que existe con los estudios de cohorte, lo que ha permitido el desarrollo de diferentes variantes. Describiremos brevemente algunas de ellas a continuación:

- **Estudios caso-cohorte**. En esta variante, la definición de casos y controles se encuentra anidada en una cohorte fija, se utiliza un enfoque de incidencia acumulada.

- **Estudios de casos y controles anidado o de grupo de riesgo**. Es similar al anterior, sólo que se trata de una cohorte dinámica, y un enfoque de densidad de incidencia.

- **Estudios caso-autocontrol**. Esta variante utiliza al mismo sujeto que se consideró como caso, como su propio control. Este tipo de estrategia se suele utilizar para exposiciones que son de corta duración y que cambian en el tiempo y con eventos que son fáciles de detectar.

- **Estudios de mortalidad proporcional**. Tanto los casos como los controles se obtienen de los registros de mortalidad poblacionales. *Rothman* dice que este tipo de controles son aceptables, sólo si la distribución de la exposición entre los grupos es similar a la que presenta la base poblacional.

- **Estudios de caso-caso**. En esta estrategia se compara la historia de exposición en subgrupos de casos. En el caso de enfermedades infecciosas, es posible conformar diversos subgrupos de enfermedad, partiendo de datos de vigilancia epidemiológica.

Quienes desean profundizar estos tópicos sobre el diseño de los CyC, pueden leer el capítulo correspondiente de *"Epidemiología. Diseño y análisis de estudios"* de Hernández-Ávila (2011). Comenzaremos ahora con la parte de análisis de estos diseños.

22.3. Análisis de estudios de casos y controles

En estudios de CC, la medida de asociación principal es el **odds ratio (OR)** de la exposición. Este se calcula comparando la frecuencia de exposición entre casos y controles:

$$OR = \frac{Odds\, exposicin\, casos}{Odds\, exposicin\, controles}$$

El modelo multivariado a utilizar para estudios de casos y controles no apareados es la **regresión logística**, que desarrollaremos a continuación. Por otro lado, los estudios de casos y controles apareados se analizan mediante modelos de **regresión logística condicional**, los cuales abordaremos brevemente al final de la unidad.

23. Regresión logística

23.1. Introducción

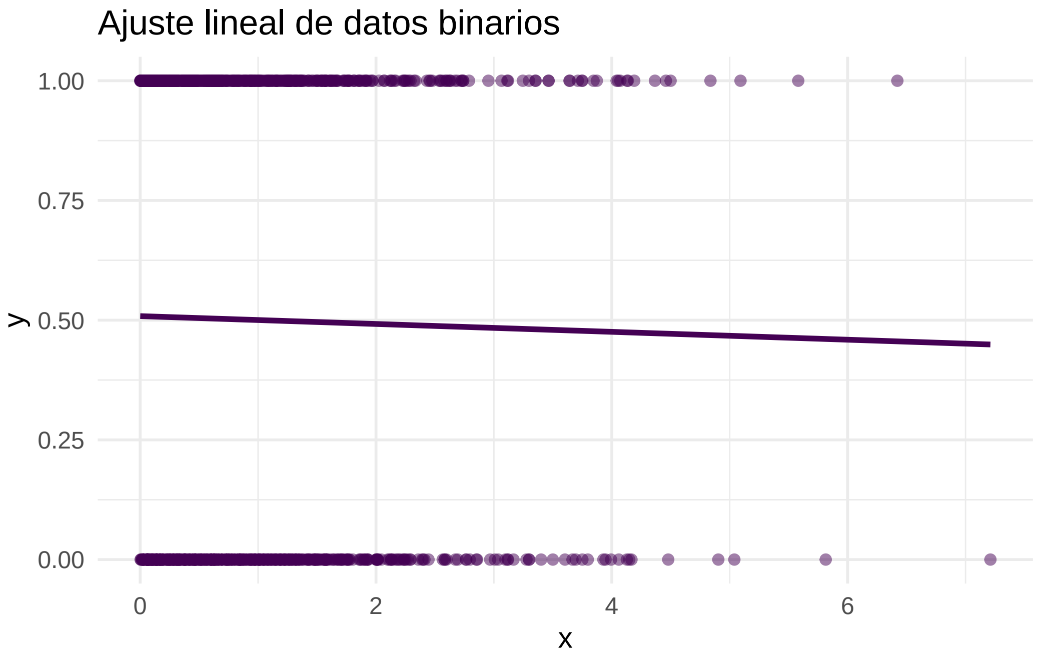
Cuando la variable dependiente es dicotómica o binaria, es decir, tiene dos categorías mutuamente excluyentes (éxito/fracaso; sí/no; positivo/negativo, etc.), los modelos de regresión lineal no son el abordaje más adecuado para el análisis. Consideremos un evento de salud que puede ocurrir o no (variable dependiente). Por ejemplo:

- Un paciente hospitalizado muere/no muere antes del alta.
- Un niño nace con/sin una malformación congénita.
- Un sujeto operado se infecta/no se infecta en el postoperatorio.
- Un niño camina/no camina a los 11 meses.
- Una droga mejora/no mejora los síntomas depresivos.

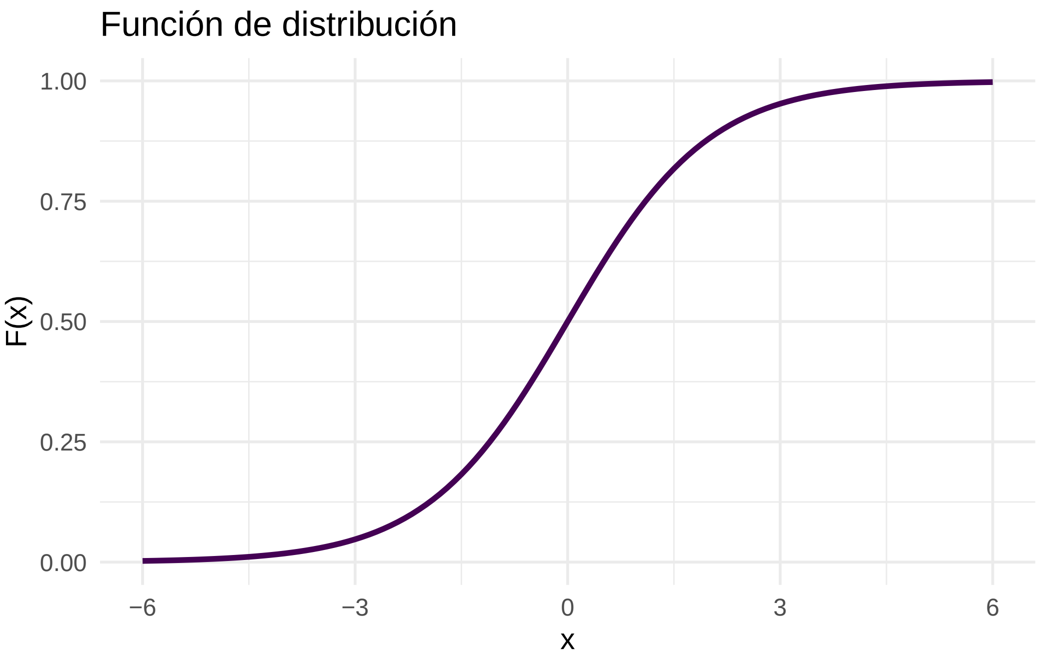
Teniendo en cuenta que nuestra variable respuesta es dicotómica:

$$Y = 1 \rightarrow Si\, el\, hecho\, ocurre$$
$$Y = 0 \rightarrow Si\, el\, hecho\, no\, ocurre$$

Si representáramos los datos con una función lineal, obtendríamos el siguiente gráfico:



Matemáticamente, la función exponencial representa mejor esta relación:



La **regresión logística** se utiliza en los casos en que la variable dependiente es binaria, mientras que las variables independientes pueden ser de cualquier tipo (categóricas, dicotómicas, numéricas discretas o continuas). En vez de la ecuación de la recta, ahora tenemos otra ecuación que expresa la variable respuesta (Y) en función de la/las variables independientes. Esta ecuación, en realidad, expresa la probabilidad de que ocurra un hecho en función de ciertas variables que se presumen relevantes.

La expresión analítica es:

$$P(Y=1)_x = \frac{1}{1+e^{(-\alpha-\beta_1 X_1-\beta_2 X_2-\cdots-\beta_k X_k)}}$$

Para comprender lo que significan los coeficientes β del modelo, vamos a hacer algunas operaciones matemáticas. Comenzaremos por realizar una transformación logística, es decir, dividir ambos miembros de la ecuación por $1-P_{(Y=1)}$:

$$\frac{P(Y=1)_x}{1-P(Y=1)_x} = \frac{\frac{1}{1+e^{(-\alpha-\beta_1 X_1-\beta_2 X_2-\cdots-\beta_k X_k)}}}{1-\frac{1}{1+e^{(-\alpha-\beta_1 X_1-\beta_2 X_2-\cdots-\beta_k X_k)}}}$$

Si ahora aplicamos logaritmo natural (ln) a ambos miembros de la ecuación, y aplicamos propiedades de los logaritmos, nos queda:

$$ln\left[\frac{P_x}{1-P_x}\right] = \alpha + \sum \beta_i x_i$$

Si observamos el término que está entre corchetes, recordaremos que el cociente entre la probabilidad que un suceso ocurra, y la probabilidad de que no ocurra, es lo que conocemos como *Odds*, entonces:

$$ln(Odds) = \alpha + \sum \beta_i x_i$$

Si despejamos *Odds* de la ecuación anterior, podemos concluir entonces que:

$$Odds = e^{(\alpha + \sum \beta_i x_i)}$$

Para comprender mejor cómo se interpretarán los coeficientes en la regresión logística, supongamos que queremos modelar la probabilidad de que un evento ocurra, $P_{(Y=1)}$, en función de una única variable independiente dicotómica, que toma el valor 0 cuando la condición está ausente ($x = 0$) y el valor 1 cuando está presente ($x = 1$). Entonces:

Para $x = 1$

$$Odds_{evento/expuestos} = e^{(\alpha + \beta)}$$

Para $x = 0$

$$Odds_{evento/no\ expuestos} = e^{(\alpha)}$$

Entonces, si queremos calcular el *odds-ratio* (OR):

$$OR = \frac{e^{(\alpha+\beta)}}{e^{\alpha}} = e^{\beta}$$

Por lo tanto:

$$ln\ OR = \beta$$

De esta forma, vemos que β = incremento del logaritmo del OR por cada unidad de incremento de x.

Extendiendo el razonamiento para la regresión logística múltiple, es decir cuando modelamos en función de más de una variable independiente, volvemos a la ecuación:

$$ln(Odds) = \alpha + \sum \beta_i x_i$$

Donde cada β_i: Incremento en log-odds para una unidad de incremento en x_i con todas las otras x_i constantes.

Incluimos esta deducción para facilitar la comprensión del tema, pero para quienes el lenguaje matemático les es adverso, pueden hacer un *"acto de fe"* y quedarse con las conclusiones.

23.2. Modelos lineales generalizados

La regresión logística forma parte de la familia de modelos lineales generalizados (GLM por su nombre en inglés, *Generalized Linear Models*), utilizados para predecir la probabilidad de que ocurra un evento binario (como infectado/no infectado, enfermo/no enfermo, sobreviviente/fallecido, etc.) en función de una o más variables independientes. Los GLM extienden los modelos de regresión lineales al permitir el uso de distribuciones no normales de errores (como binomiales, Poisson, gamma, entre otras) y varianzas no constantes. Estos modelos se caracterizan por una estructura de errores específica y una función de enlace que conecta la variable respuesta con la(s) variable(s) independiente(s).

En el caso de la regresión logística, donde la variable respuesta es binaria (0, 1), la estructura de errores pertenece a la familia de distribución *binomial*. La función de enlace típica para linealizar la relación entre la variable respuesta y la(s) variable(s) independiente(s) es la función *logit*, que es el logaritmo natural del odds-ratio (OR). Esta función de enlace transforma la escala de probabilidades (de 0 a 1) a una escala lineal (de $-\infty$ a $+\infty$), lo que facilita la modelización de la relación entre las variables independientes y la variable respuesta binaria.

23.3. Componentes del modelo logístico

Al igual que en la regresión lineal múltiple (RLM), antes de adentrarnos en el modelado, exploraremos cómo interpretar la salida de R para un modelo de regresión logística obtenida a partir de la función `summary(modelo)`:

```
Call:
glm(formula = y ~ x1 + x2, family = binomial, data = datos)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.3362     0.2816  -1.194   0.2325
x1B           0.4267     0.4012   1.064   0.2875
x1C           0.7016     0.3996   1.756   0.0791 .
x2            1.2669     0.2103   6.023 1.71e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 277.08  on 199  degrees of freedom
Residual deviance: 221.48  on 196  degrees of freedom
AIC: 229.48

Number of Fisher Scoring iterations: 4
```

Los elementos a tener en cuenta son:

- `Call`: Muestra la fórmula del modelo utilizado.

- `Estimate`: Contiene los coeficientes β estimados, incluyendo el intercepto (β_0) y las variables explicativas (β_i). Al aplicar a estos coeficientes la función `exp()`, que representa la función inversa del logaritmo natural, se obtiene el Odds Ratio (OR). Esto implica que los coeficientes de las variables independientes se interpretan como el OR de que ocurra el evento para cada incremento de la variable independiente, manteniendo constantes las demás variables independientes.
- `Std. Error`: Representa el error estándar asociado a cada coeficiente.
- `z value`: Corresponde al estadístico del test de Wald para evaluar la significancia de cada coeficiente.
- `Pr(>|z|)`: Proporciona los *p*-valores asociados al test de Wald, que indican si los coeficientes son significativamente diferentes de cero.

Además, en la parte final de la salida se presentan:

- `Null deviance`: La devianza del modelo nulo, que asume que ninguna de las variables explicativas tiene efecto.
- `Residual deviance`: La devianza del modelo ajustado, que indica cuánto se reduce la devianza al incluir las variables explicativas.
- `AIC`: Una medida de la calidad del modelo, penalizando por la complejidad para favorecer modelos más parsimoniosos.
- `Number of Fisher Scoring iterations`: Número de iteraciones realizadas durante el proceso de estimación de parámetros del modelo.

A continuación, profundizaremos en el significado de algunos de estos conceptos.

23.3.1. Test de Wald

El test de Wald se utiliza para evaluar la significación de una variable dentro del modelo. De manera similar al test F parcial en la regresión lineal múltiple, este test contrasta la hipótesis nula:

$$H_0 : \beta_i = 0$$

Expresamos la prueba en términos del coeficiente β_i porque esta formulación es análoga a la utilizada en regresión lineal. Sin embargo, en regresión logística, los resultados suelen presentarse en términos de *odds-ratio* (OR).

Dado que una pendiente $\beta_i = 0$ implica un OR de 1, la hipótesis nula también puede expresarse como *“la razón de probabilidades es 1”*. Esto significa que la variable explicativa no tiene capacidad predictiva, ya que las probabilidades de los grupos comparados son iguales.

La interpretación sigue criterios comunes en inferencia estadística: valores de $p < 0,05$ sugieren evidencia suficiente para rechazar la hipótesis nula, indicando que la variable contribuye significativamente al modelo ($p < 0,1$ podría considerarse en contextos más flexibles). Es fundamental recordar que, en regresión logística, los coeficientes β_i no se interpretan directamente. Para obtener su efecto sobre la razón de probabilidades, se debe calcular su exponencial exp(β_i).

23.3.2. Máxima verosimilitud

Mientras que en la regresión lineal múltiple (RLM) los coeficientes β se obtenían por el método de los mínimos cuadrados, en la regresión logística (RLOG) se obtienen mediante el método de máxima verosimilitud (ML, por su nombre en inglés, *Maximum Likelihood*). El fundamento de esta técnica radica en utilizar la información disponible de los datos de la muestra para seleccionar el valor del parámetro que maximiza la probabilidad de observar los resultados muestrales. La ML, entonces, se calcula mediante un proceso iterativo.

Por lo tanto, una medida adecuada para evaluar la concordancia del modelo con los datos sería el producto de todas las probabilidades (predichas por el modelo), que los n sujetos de la muestra realmente tengan la condición observada. Es decir, un buen modelo sería el que asigne una probabilidad de 1 ($p = 1$) a cada sujeto que realmente tenga la condición y de 0 ($p = 0$) a cada sujeto libre de ella, correspondiendo a una ML de 1. Por el contrario, un modelo deficiente tendría una verosimilitud cercana a 0. En consecuencia, la proximidad de la verosimilitud a 1 expresa cuán eficiente ha sido el ajuste realizado para modelar la realidad.

23.3.3. *Deviance*

La *deviance* (D), también conocida como devianza o distancia, se define como:

$$D = -2lnV$$

Donde V es la verosimilitud del modelo.

Dado que, como explicamos anteriormente, $V < 1$, su logaritmo siempre será negativo, haciendo que la devianza sea siempre un número positivo. El grado de ajuste de un modelo será mejor cuanto más próxima a 1 es la verosimilitud y, en consecuencia, cuanto más cercana a cero sea la devianza.

Al ajustar el modelo se calculan dos devianzas: la correspondiente al *"modelo nulo"* (D_0), que es aquel en que no se ha incorporado ninguna variable independiente, y la D_f del modelo. La diferencia (o cociente) entre estas dos devianzas mide la "contribución" que hacen las variables incorporadas al modelo:

$$-2lnV_0 - (-2lnV_f)$$

La D_0 es siempre mayor que la de cualquier modelo ampliado. Esto es razonable, ya que el modelo nulo es mucho menos complejo (no incorpora información de variables "explicativas") y, por lo tanto, tendrá una capacidad predictiva inferior.

23.3.4. *Likelihood Ratio Test*

El *Likelihood Ratio Test* (LRT) compara la probabilidad de los datos observados bajo dos modelos: uno que incluye los predictores y otro que no. Para ello, evalúa la diferencia en los residuos entre ambos modelos, lo que equivale a comparar sus respectivas *devianzas* (D).

$$D_0 - D = -2lnV_0 + 2lnV = -2ln(V_0 - V) = -2ln\left(\frac{V_0}{V}\right)$$

Donde $\frac{V_0}{V}$ es el *Likelihood Ratio* o razón de verosimilitud.

El LRT permite evaluar la significancia de la incorporación de predictores al modelo, comparándolo con el modelo nulo (sin predictores).El estadístico de prueba sigue una distribución χ^2 con grados de libertad equivalentes al número de predictores incluidos en el modelo.

En términos prácticos, la *razón de verosimilitud* se obtiene al comparar las *devianzas* de dos modelos, uno con más y otro con menos predictores. Esto permite determinar si la inclusión de ciertas variables mejora significativamente el ajuste del modelo. En la salida de un modelo, los valores de devianza aparecen bajo los títulos `Null deviance` (para el modelo nulo) y `Residual deviance` (para el modelo con predictores).

23.3.5. Criterio de Información de Akaike

El Criterio de Información de Akaike (AIC) es una medida de la calidad relativa de un modelo estadístico, para un conjunto dado de datos. Se define como:

$$AIC = 2k – 2ln(D)$$

Donde,

- k: número de parámetros del modelo.
- D: devianza del modelo.

El AIC proporciona un método para la selección de modelos, donde valores menores de AIC indican un mejor ajuste del modelo a los datos.

Para comparar dos o más modelos de regresión logística, se pueden utilizar las siguientes técnicas:

- **AIC**: El primer término de la ecuación del AIC penaliza por la inclusión de variables en el modelo ($2k$), mientras que el segundo compensa por la bondad de ajuste ($2ln(D)$). Por lo tanto, dado un conjunto de modelos para los datos, el modelo preferido es aquel con el valor mínimo de AIC.

En R, realizamos la comparación con la función `AIC()`:

```
mod2 <- update(modelo, ~ . - x1)

AIC(modelo, mod2)
```

```
       df      AIC
modelo  4 229.4788
mod2    2 228.6598
```

- ***Likelihood Ratio Test:*** Este test permite comparar dos modelos evaluando la significancia de la diferencia de devianzas. El estadístico tiene una distribución χ^2 con grados de libertad iguales a la diferencia en el número de parámetros entre los dos modelos comparados.

La función `anova()` del paquete `stats` permite comparar modelos por si diferencia de varianzas:

```
anova(modelo, mod2, test = "Chisq")
```

```
Analysis of Deviance Table

Model 1: y ~ x1 + x2
Model 2: y ~ x2
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1       196     221.48
2       198     224.66 -2  -3.1811   0.2038
```

También podemos usar la función `test_lrt()` del paquete `performance`:

```
test_lrt(modelo, mod2)
```

```
# Likelihood-Ratio-Test (LRT) for Model Comparison (ML-estimator)

Name   | Model | df | df_diff | Criterion | Chi2 |     p
--------------------------------------------------------
modelo |   glm |  4 |         |    221.48 |      |
mod2   |   glm |  2 |      -2 |    224.66 | 3.18 | 0.204
```

23.3.6. Bondad de ajuste

De forma análoga al Coeficiente de Determinación (R^2) utilizado en regresión lineal, se han desarrollado diversos coeficientes para estimar la proporción de variabilidad explicada por las variables independientes en modelos de regresión logística.

En R, podemos calcular estos coeficientes utilizando el paquete `performance`:

- **Coeficiente de McFadden:** Es una de las medidas más utilizadas y se interpreta de manera similar al R^2 en regresión lineal:

```
r2_mcfadden(modelo)
```

```
# R2 for Generalized Linear Regression
       R2: 0.201
  adj. R2: 0.172
```

- **Coeficiente de Cox y Snell:** Se basa en la razón de verosimilitud y es una generalización del R^2 en modelos lineales:

```
r2_coxsnell(modelo)
```

```
Cox & Snell's R2
       0.2427035
```

- **Coeficiente de Nagelkerke:** El coeficiente de Nagelkerke es una versión corregida del coeficiente de Cox y Snell, cuyo valor máximo es menor a 1 incluso para un modelo perfecto, corrigiendo así la tendencia de subestimación del coeficiente R^2.

```
r2_nagelkerke(modelo)
```

```
Nagelkerke's R2
      0.3237019
```

- **Coeficiente de Tjur:** Calcula el Coeficiente de Discriminación (D) para modelos lineales generalizados con variable respuesta binaria. Es el proporcionado por defecto para modelos de regresión logística en `performance`:

```
r2(modelo)
```

```
# R2 for Logistic Regression
  Tjur's R2: 0.251
```

- **Test de Hosmer-Lemeshow**: Ninguno de los coeficientes anteriores mide directamente la calidad del ajuste del modelo. Para evaluar esto, utilizamos el **test de Hosmer-Lemeshow**, que compara las probabilidades predichas con las observadas.

Este test calcula un estadístico de distribución χ^2 con $n-2$ grados de libertad, utilizando varios grupos basados en los deciles de las probabilidades predichas. En R:

```
performance_hosmer(modelo)
```

```
# Hosmer-Lemeshow Goodness-of-Fit Test

  Chi-squared: 12.091
           df:  8
      p-value:  0.147
```

En un test de bondad de ajuste, la hipótesis nula siempre afirma que el modelo propuesto se ajusta bien a los datos observados. Por lo tanto, un $p > 0,05$ implica que lo observado se ajusta suficientemente bien a lo esperado bajo el modelo.

23.3.7. Capacidad predictiva del modelo

Otro aspecto a evaluar en un modelo de RLOG es su capacidad de *discriminación*, es decir, la habilidad del modelo para distinguir entre individuos en los que ocurre el evento y aquellos en los que no. Una medida común de esta discriminación es el área bajo la **curva ROC** (*Receiver Operating Characteristic*), que se construye utilizando las probabilidades predichas por el modelo.

Para evaluar la efectividad del modelo en la clasificación de observaciones, se puede construir una tabla de clasificación que cruza el verdadero valor de la observación (1 o 0) con la predicción del modelo. Como el modelo de RLOG estimará probabilidades en el rango de 0 a 1, tendremos que elegir un punto de corte en forma arbitraria. Por ejemplo, podríamos decidir que probabilidades estimadas mayores a 0.5 sean indicativas de que el evento ha ocurrido, mientras que probabilidades menores o iguales a 0.5 indiquen que el evento no ha ocurrido.

La capacidad predictiva de un modelo de regresión logística se resume utilizando los conceptos de sensibilidad y especificidad. Quienes trabajen en áreas relacionadas al diagnóstico estarán más familiarizados con estos conceptos.

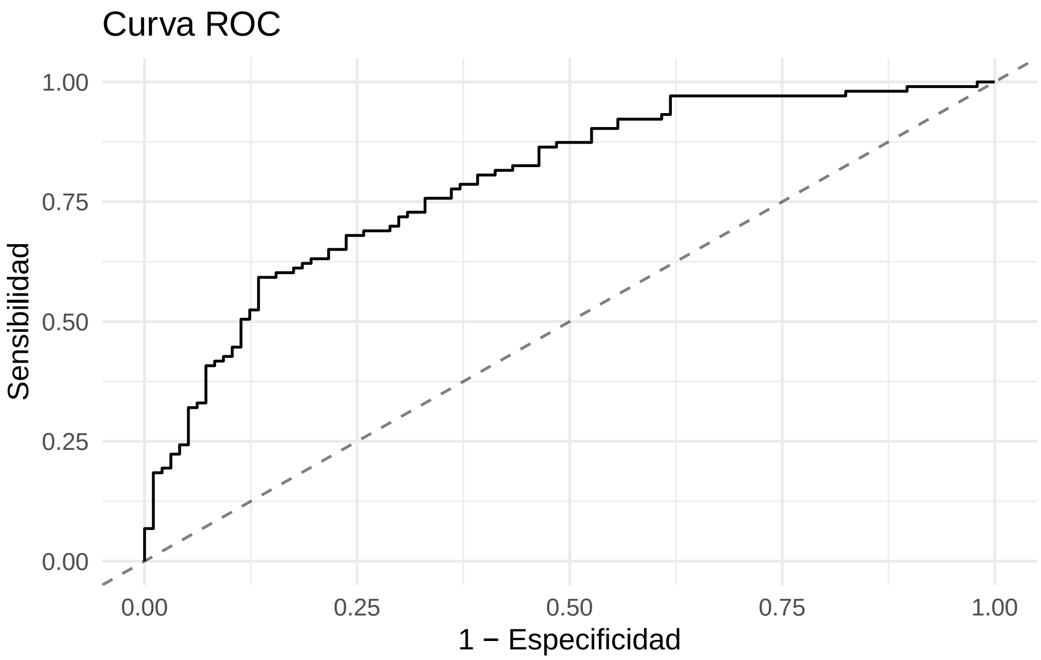
- **Sensibilidad:** Probabilidad de que el modelo prediga correctamente que el evento ha ocurrido cuando *realmente ha ocurrido*.

$$P(\hat{y} = 1 | y = 1)$$

- **Especificidad:** Probabilidad de que el modelo prediga correctamente que el evento no ha ocurrido cuando *realmente no ha ocurrido*.

$$P(\hat{y} = 0 | y = 0)$$

La curva ROC es un gráfico que representa la sensibilidad en función de 1 menos la especificidad. Si vamos modificando los valores del valor de corte y representamos la **sensibilidad** (en el eje Y) frente a **1 – especificidad** (en el eje X) tenemos la curva ROC. Cuanto mayor sea el área bajo esta curva, mejores serán las predicciones del modelo.



Esta curva representa, para todos los pares posibles de individuos formados por uno en el que ocurrió el evento y otro en el que no, la proporción de aquellos para los cuales el modelo predice una mayor probabilidad de haber experimentado el evento.

Para evaluar un modelo de RLOG deberíamos observar:

- **Coeficientes de determinación** : Variabilidad explicada por el modelo.
- **Test de Hosmer-Lemeshow**: Bondad de ajuste, diferencia entre los valores predichos por el modelo y los valores observados en la muestra.
- **AIC**, **LRT** o **ANOVA**: Comparación de modelos.
- **Curva ROC**: Capacidad predictiva (especialmente cuando el propósito es predictivo).
- **Intervalos de confianza**: Exactitud del coeficiente.

23.4. Construcción del modelo en R

El ajuste de un modelo de regresión logística en R se realiza utilizando la función `glm()`, del paquete `stats`:

```
glm(formula, family = binomial(link = "logit"), data)
```

- `formula`: Especifica la relación entre la variable dependiente y las variables independientes en el modelo. Sigue la estructura estándar:

$$variable\_dependiente \sim variable\_indep_1 + variable\_indep_2 + \cdots + variable\_indep_n$$

- `family`: Se refiere a la familia de distribuciones y la función de enlace utilizada para ajustar el modelo. Las opciones comunes incluyen:
  - `gaussian()`: Utilizada para variables dependientes continuas con distribución normal. El enlace predeterminado es `identity`, lo cual es análogo a ajustar un modelo de RLM. Puede emplear también enlaces `log`, e `inverse`.
  - `binomial()`: Utilizada para variables dependientes binarias (0, 1). El enlace predeterminado es `logit`, que es el más común en la regresión logística. También admite enlaces como `probit`, `cauchit`, `log`, y `cloglog`.
  - `poisson()`: Usada para variables dependientes numéricas discretas. El enlace predeterminado es el logaritmo (`log`), y también admite `identity` y `sqrt`.
  - Otras familias como `Gamma()`, `inverse.gaussian()`, `quasi()`, `quasipoisson()`, y `quasibinomial()` para diferentes distribuciones de errores que no abordaremos en el curso.

  Si la función de enlace no se especifica, se utiliza el enlace canónico (predeterminado) para cada familia. Por ejemplo, si omitimos el argumento `link = "logit"`, de todas maneras quedaría definido ese mismo enlace para la familia `binomial`.

- `data`: Nombre del *dataframe* que contiene las variables utilizadas en el modelo.

23.4.1. Gestión de variables dicotómicas

Decíamos que el modelo con enlace *logit* es un modelo de regresión típico:

$$Y = f(X + E)$$

donde la variable respuesta (variable aleatoria Y) es dicotómica o binaria (toma dos valores: 0 y 1), habitualmente sobre si nuestra unidad de análisis tiene una característica (1) o no la tiene (0).

Nuestras variables dicotómicas pueden tener originalmente formatos variados y sus categorías también pueden definirse con etiquetas diferentes. Por ejemplo, podemos tener variables dicotómicas con formato lógico (+/-, `TRUE`/`FALSE`), con formato caracter (Si/No, Vivo/Muerto, etc.) o con formato numérico codificado (0-1, 1-2 o cualquier combinación personalizada de códigos).

En R las variables categóricas que utilizamos como dependientes en estos modelos corresponden convenientemente al tipo de datos `factor`. Recordemos que un factor es, interna y técnicamente, una variable numérica compuesta de enteros sucesivos a partir de 1. Cada entero es un nivel o categoría de la variable y está acompañado de una etiqueta que nos facilita recordar a qué categoría corresponde.

Los modelos binomiales asumen las variables dicotómicas codificándolas como 0 y 1, lo cuál podría ser problemático si, como numéricos, los factores comienzan con 1. Como la estructura de los factores es conocida por las funciones que estiman modelos *logit*, estas funciones convierten internamente la variable categórica/factor en una variable codificada como 0/1, sin que el usuario tenga que hacer nada.

Por lo tanto, no es necesario recodificar la variable o convertirla en *dummy*, esto se procesa de manera transparente para el usuario. Lo único que debemos asegurarnos es que la variable sea factor y que el **nivel de referencia** sea la **ausencia de la característica** (por ejemplo, enfermedad = No).

Debemos usar la función `levels()` de R `base` para consultar los niveles o categorías de un factor e identificar el primer nivel del factor, que será el nivel de referencia en el modelo, es decir, $Y = 0$.

Podemos modificar estos niveles de referencia mediante la función `relevel()` de R `base` o con `fct_rev()` de `tidyverse`, cuando estos se encuentren invertidos.

```
# Factor con dos categorías
factor <- factor(c("1", "0", "1", "1", "0"))

# Cambio nivel de referencia en R base
relevel(factor, ref = "1")
```

```
[1] 1 0 1 1 0
Levels: 1 0
```

```
# Cambio nivel de referencia en tidyverse
fct_rev(factor)
```

```
[1] 1 0 1 1 0
Levels: 1 0
```

Este procesamiento es extensivo a las variables categóricas (dicotómica o politómicas) que se incluyen en los modelos de regresión como explicativas. En `tidyverse`, podemos cambiar los niveles de referencia de un factor con más de dos categorías con la función `fct_relevel()`.

```
# Factor con más de dos categorías
factor <- factor(c("CONF", "DESC", "SOSP", "CONF", "DESC", "PROB"))

# Niveles por defecto
levels(factor)
```

```
[1] "CONF" "DESC" "PROB" "SOSP"
```

```
# Pongo DESC como nivel de referencia
fct_relevel(factor, "DESC", after = 0)
```

```
[1] CONF DESC SOSP CONF DESC PROB
Levels: DESC CONF PROB SOSP
```

23.4.2. Ajuste del modelo

La razón de verosimilitud en un modelo de regresión puede calcularse restando la devianza del modelo con predictores de la del modelo nulo:

```
modelo$null.deviance - modelo$deviance
```

```
[1] 55.60009
```

En este cálculo, estamos comparando la *null deviance* (modelo sin predictores) con la *residual deviance* (modelo con predictores). Alternativamente, la función `anova()` proporciona una tabla detallada que desglosa la contribución de cada variable al modelo junto con sus respectivas devianzas.

```
anova(modelo)
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y

Terms added sequentially (first to last)

     Df Deviance Resid. Df Resid. Dev  Pr(>Chi)    
NULL                   199     277.08              
x1    2    4.080       197     273.00    0.1301    
x2    1   51.521       196     221.48 7.085e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

En un modelo con múltiples predictores, no todas las variables necesariamente aportan información relevante. Identificar y excluir aquellas con menor impacto puede simplificar el modelo sin comprometer su capacidad predictiva. Es fundamental definir qué se entiende por "variable relevante", ya que la importancia de una variable no debe evaluarse únicamente desde el punto de vista estadístico. El marco conceptual es clave en la selección de predictores, especialmente cuando el objetivo del modelo es analítico. La decisión debe basarse tanto en el conocimiento del área de estudio como en la evidencia estadística.

Un error común es suponer que los métodos estadísticos pueden sustituir el conocimiento sustantivo del problema. El abuso de modelos de regresión sin una adecuada justificación teórica puede llevar a asociaciones espurias o a resultados estadísticamente significativos pero clínicamente, biológicamente o socialmente irrelevantes.

Existen diversos métodos estadísticos para la selección de variables en regresión logística. Estos buscan lograr un modelo parsimonioso que se ajuste bien a los datos sin incluir predictores innecesarios. Si bien estos procedimientos son similares a los utilizados en regresión lineal múltiple, algunos criterios de selección específicos varían.

23.5. Ejemplo práctico en R

Utilizaremos una tabla de datos denominada "`caminar.txt`", que contiene observaciones de un estudio de casos y controles donde se evaluó a 800 niños, con el objetivo de identificar qué factores se asociaban al desarrollo de la marcha.

Comencemos cargando en R los paquetes que utilizaremos:

```
# Tablas de coeficientes y regresiones simples
library(gtsummary)

# Chequeo de supuestos
library(easystats)

# Manejo de datos
library(skimr)
library(flextable)
library(tidyverse)
```

Cargamos los datos y exploramos su estructura. Vemos dos tipos de variable: Enteros `<int>` para las numéricas y caracter `<chr>` para las categóricas.

```
# Carga datos
datos <- read.csv2("datos/caminar.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 800
Columns: 6
$ id          <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
$ camina      <int> 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1,~
$ edad_dias   <int> 265, 318, 554, 389, 529, 574, 578, 582, 398, 289, 336, 476,~
$ edad_meses  <int> 8, 10, 18, 12, 17, 19, 19, 19, 13, 9, 11, 15, 16, 9, 9, 18,~
$ pareja_est  <chr> "No", "Si", "No", "Si", "Si", "No", "Si", "Si", "No", "No",~
$ lacto_180   <chr> "Si", "Si", "No", "Si", "Si", "No", "Si", "Si", "Si", "Si",~
```

Las variables a considerar son:

- `camina`: si al momento de la entrevista caminaban (1, 0)
- `edad_meses`: a qué edad habían comenzado a caminar (medida en meses)
- `edad_dias`: a qué edad habían comenzado a caminar (medida en días)
- `pareja_est`: si la madre tenía pareja estable (Si, No)
- `lacto_180`: si había tomado pecho al menos 6 meses (Si, No)

Coercionamos los tipos `character` a `factor`.

```
datos <- datos |>
  mutate(
    camina = factor(camina),
    pareja_est = factor(pareja_est),
    lacto_180 = factor(lacto_180)
  )
```

Como las categorías de las tres variables convertidas son “0” y “1” o “Si” y “No”, y el orden automático del lenguaje R es alfabético, el nivel de referencia será “0” y “No”. Confirmémoslo mediante la visualización de los niveles:

```
levels(datos$camina)
```

```
[1] "0" "1"
```

```
levels(datos$pareja_est)
```

```
[1] "No" "Si"
```

```
levels(datos$lacto_180)
```

```
[1] "No" "Si"
```

Efectuamos una breve descripción univariada usando el paquete `skimr`:

```
datos |>
  # Omitimos identificador de paciente
  select(-id) |>
  skim()
```

Tabla 23.1.: Data summary

Name	select(datos, -id)
Number of rows	800
Number of columns	5
Column type frequency:	
factor	3
numeric	2
Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
camina	0	1	FALSE	2	1: 419, 0: 381
pareja_est	0	1	FALSE	2	No: 467, Si: 333
lacto_180	0	1	FALSE	2	Si: 454, No: 346

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
edad_dias	0	1	424.20	104.30	240	334	424	516.25	600	⍰⍰⍰⍰⍰
edad_meses	0	1	13.67	3.47	8	11	14	17.00	20	⍰⍰⍰⍰⍰

23.5.1. Selección de modelos

A continuación iniciamos un proceso iterativo manual tipo *forward* basado en la selección de modelos propuesto en el capítulo 8 del libro de Silva Ayçaguer (1995).

Paso 1: Se ajustan tantos modelos de regresión logística simple como posibles variables explicativas tengamos. Dos de las variables (`edad_dias` y `edad_meses`) son la misma expresada en unidades diferentes, por lo que intentaremos quedarnos con la que explique más y a su vez sea estadísticamente significativa..

```
# edad en días
mod1 <- glm(camina ~ edad_dias, data = datos, family = binomial)

# edad en meses
mod2 <- glm(camina ~ edad_meses, data = datos, family = binomial)

# pareja estable
mod3 <- glm(camina ~ pareja_est, data = datos, family = binomial)

# lactancia
mod4 <- glm(camina ~ lacto_180, data = datos, family = binomial)
```

Se calcula el *Likelihood Ratio* de cada uno de ellos.

```
mod1$null.deviance - mod1$deviance
```

```
[1] 258.118
```

```
mod2$null.deviance - mod2$deviance
```

```
[1] 258.7028
```

```
mod3$null.deviance - mod3$deviance
```

```
[1] 246.6692
```

```
mod4$null.deviance - mod4$deviance
```

```
[1] 352.0365
```

Se identifica el mayor de estas G^2 (`mod4`) y se evalúa si es significativo

```
test_lrt(mod4)
```

```
# Likelihood-Ratio-Test (LRT) for Model Comparison (ML-estimator)

Name       | Model | df | df_diff | Criterion |    Chi2 |      p
---------------------------------------------------------------
Null model |   glm |  1 |         |   1107.23 |         |
Full model |   glm |  2 |       1 |    755.19 | 352.04 | < .001
```

Paso 2: Como la variable `lacto_180` es significativa, se incorpora como primer variable al modelo y a continuación se generan modelos de dos variables independientes combinando esta variable con cada una de las otras.

```
# mod 4 + edad en días
mod5_1 <- glm(camina ~ lacto_180 + edad_dias, data = datos, family = binomial)

# mod4 + edad en meses
mod5_2 <- glm(camina ~ lacto_180 + edad_meses, data = datos, family = binomial)

# mod4 + pareja estable
mod5_3 <- glm(camina ~ lacto_180 + pareja_est, data = datos, family = binomial)
```

Se identifica la pareja para la cual la deviance ($-2lnV_f$) es menor.

```
mod5_1$deviance
```

```
[1] 592.4274
```

```
mod5_2$deviance
```

```
[1] 593.5655
```

```
mod5_3$deviance
```

```
[1] 614.9587
```

En este caso, el `mod5_1` con las variables `lacto_180` y `edad_dias` tiene una deviance de 592.43, menor a los otros dos modelos.

Se evalúa si el agregado de la variable `edad_dias` es significativa respecto al modelo con la variable `lacto_180` sola.

```
test_lrt(mod4, mod5_1)
```

```
# Likelihood-Ratio-Test (LRT) for Model Comparison (ML-estimator)

Name    | Model | df | df_diff | Criterion |    Chi2 |      p
-----------------------------------------------------------
mod4    |   glm |  2 |         |    755.19 |         |
mod5_1  |   glm |  3 |       1 |    592.43 | 162.77 | < .001
```

Paso 3: Como es significativa, se incorpora al modelo y a continuación se agrega la tercer y última variable posible en el modelo (dado que la variable `edad_meses` es la misma que `edad_dias` que ya está incorporada).

```
mod6 <- glm(
  camina ~ lacto_180 + edad_dias + pareja_est,
  data = datos,
  family = binomial
)
```

Se evalúa la significación del agregado. Si es significativo el modelo quedaría con las tres variables, de lo contrario quedan las dos iniciales.

```
test_lrt(mod5_1, mod6)
```

```
# Likelihood-Ratio-Test (LRT) for Model Comparison (ML-estimator)

Name    | Model | df | df_diff | Criterion |  Chi2 |      p
---------------------------------------------------------
mod5_1  |   glm |  3 |         |    592.43 |       |
mod6    |   glm |  4 |       1 |    497.72 | 94.71 | < .001
```

El proceso iterativo continúa de haber más variables para agregar.

También se pueden utilizar métodos automáticos, al igual que vimos en la regresión lineal múltiple, partiendo por ejemplo de un modelo saturado y aplicando la función `step()`. Recuerden que estos modelos automáticos los utilizamos para comparar los procesos manuales y que se basan en únicamente en iteraciones con procesos matemático-estadísticos sin tener en cuenta la "relevancia" conceptual de las variables y las relaciones entre ellas.

```
modelo_saturado <- glm(camina ~ . - id, data = datos, family = binomial)

modelo_step <- step(modelo_saturado, direction = "both")
```

```
Start:  AIC=507.72
camina ~ (id + edad_dias + edad_meses + pareja_est + lacto_180) - 
    id

             Df Deviance    AIC
```

```
- edad_meses  1   497.72 505.72
- edad_dias   1   498.63 506.63
<none>            497.72 507.72
- pareja_est  1   592.43 600.43
- lacto_180   1   691.11 699.11

Step:  AIC=505.72
camina ~ edad_dias + pareja_est + lacto_180

             Df Deviance    AIC
<none>            497.72 505.72
+ edad_meses  1   497.72 507.72
- pareja_est  1   592.43 598.43
- edad_dias   1   614.96 620.96
- lacto_180   1   691.58 697.58
```

En este caso la selección automática utiliza el AIC que funciona de manera similar a la regresión lineal y el seleccionado termina siendo el que tiene el AIC más bajo.

```
summary(modelo_step)
```

```


Call:
glm(formula = camina ~ edad_dias + pareja_est + lacto_180, family = binomial, 
    data = datos)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)    
(Intercept)  -7.596812   0.619037 -12.272   <2e-16 ***
edad_dias     0.012168   0.001272   9.566   <2e-16 ***
pareja_estSi  2.242393   0.249994   8.970   <2e-16 ***
lacto_180Si   3.007014   0.249097  12.072   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1107.23  on 799  degrees of freedom
Residual deviance:  497.72  on 796  degrees of freedom
AIC: 505.72

Number of Fisher Scoring iterations: 6
```

En este ejemplo el modelo automático es coincidente con el *forward* manual que realizamos.

Aplicando `AIC()` a la serie de modelos que fuimos realizando de forma manual y automática podemos comparar:

```
AIC(mod4)
```

```
[1] 759.1933
```

```
# modelo solo con lacto_180
AIC(mod5_1)
```

```
[1] 598.4274
```

```
# modelo con lacto_180 y edad_dias
AIC(mod6)
```

```
[1] 505.7191
```

```
# modelo con lacto_180, edad_dias y pareja_est
AIC(modelo_step) # modelo automático idem mod6
```

```
[1] 505.7191
```

El modelo final tiene el AIC más bajo (505.7).

Al igual que en RLM el paquete `performance` muestra en una tabla la comparación de índices AIC entre otros, para los modelos definidos.

```
compare_performance(mod4, mod5_1, mod6, metrics = "AIC")
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC (weights)
------------------------------
mod4   |   glm | 759.2 (<.001)
mod5_1 |   glm | 598.4 (<.001)
mod6   |   glm | 505.7 (>.999)
```

Si añadimos el argumento `rank = TRUE` nos muestra los modelos ordenados por AIC:

```
compare_performance(mod4, mod5_1, mod6, metrics = "AIC", rank = TRUE)
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC weights | Performance-Score
------------------------------------------------
mod6   |   glm |        1.00 |           100.00%
mod5_1 |   glm |    7.39e-21 |          7.39e-19%
mod4   |   glm |    9.09e-56 |             0.00%
```

En este caso, observarán que la función `compare_performance()` utiliza el indicador *AIC weights* en lugar del AIC. Los *AIC weights* cuantifican, de forma relativa, la calidad de un modelo dentro de un conjunto de modelos candidatos. Representan Refleja la probabilidad relativa de que un modelo sea el mejor entre un conjunto de modelos candidatos. Su cálculo se basa en la comparación de los valores de AIC de todos los modelos comparados.

Por este motivo, su interpretación es inversa a la del AIC: cuanto mayor es el valor del *AIC weight*, mayor es la probabilidad de que ese modelo sea el más adecuado entre los considerados.

23.5.2. Calidad de ajuste

Existen diversas formas de medir la calidad de ajuste de un modelo de regresión logística. De manera global, esta calidad puede ser evaluada a través de coeficientes de determinación R^2, probabilidades estimadas o test estadísticos.

Los coeficientes de determinación calculan el grado de "*explicación de la variabilidad de la variable de respuesta*" conseguido con el modelo a partir de las variables independientes. Como vimos anteriormente, pueden estimarse con funciones del paquete `performance`:

```
r2_mcfadden(mod6)
```

```
# R2 for Generalized Linear Regression
       R2: 0.550
  adj. R2: 0.543
```

```
r2_coxsnell(mod6)
```

```
Cox & Snell's R2
       0.5332159
```

```
r2_nagelkerke(mod6)
```

```
Nagelkerke's R2
      0.7114904
```

```
r2_tjur(mod6)
```

```
Tjur's R2
0.6177795
```

Estos coeficientes alcanzan el valor 1 cuando el vaticinio es perfecto (esto significa que el modelo atribuye probabilidad 1 a aquellos sujetos de la muestra que efectivamente tuvieron el evento y valores iguales a 0 a quienes no lo tuvieron) y se aproximan a 0 en la medida que las probabilidades atribuidas por el modelo disten más de 1 y 0, respectivamente.

Respecto a test estadísticos, la prueba de razón de verosimilitud (LRT), recientemente utilizada en el proceso de selección de modelos, es una de las que podemos utilizar basada en la devianza.

```
# R base
anova(mod6, test = "Chisq")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: camina

Terms added sequentially (first to last)

           Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                         799    1107.23
lacto_180   1   352.04       798     755.19 < 2.2e-16 ***
edad_dias   1   162.77       797     592.43 < 2.2e-16 ***
pareja_est  1    94.71       796     497.72 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# performance
test_lrt(mod6)
```

```
# Likelihood-Ratio-Test (LRT) for Model Comparison (ML-estimator)

Name       | Model | df | df_diff | Criterion |   Chi2 |      p
---------------------------------------------------------------
Null model |   glm |  1 |         |   1107.23 |        |
Full model |   glm |  4 |       3 |    497.72 | 609.51 | < .001
```

Observamos que las dos funciones son complementarias. Mientras `test_lrt()` nos muestra la razón de verosimilitud entre el modelo completo y el nulo en forma global, la función `anova()` nos detalla en la tabla de análisis de varianzas la *deviance* entre cada paso del modelo, es decir, a partir de incorporar una a una las variables con su respectiva significación.

Entre las pruebas de *bondad de ajuste* basadas en la agrupación de las probabilidades estimadas bajo el modelo de regresión, usaremos el test de Hosmer y Lemeshow. Si bien existen variados paquetes que proveen esta, nosotros utilizaremos la función `performance_hosmer()` del paquete `performance`.

```
performance_hosmer(mod6)
```

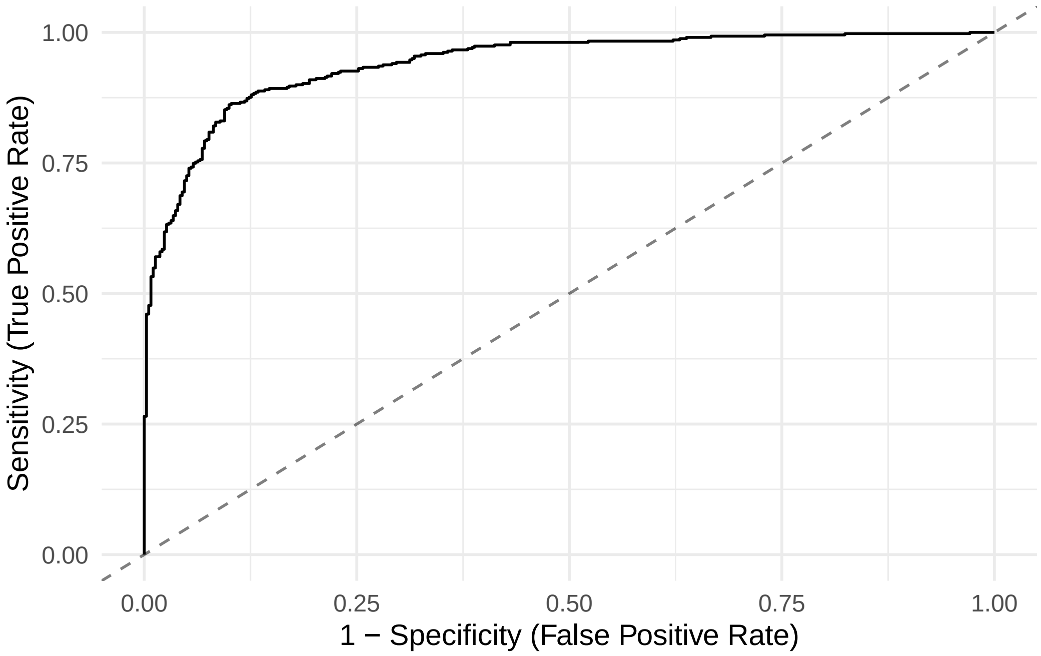
```
# Hosmer-Lemeshow Goodness-of-Fit Test

  Chi-squared: 10.660
           df:  8
      p-value:  0.222
```

El test de bondad de ajuste de Hosmer y Lemeshow nos da un p-valor para el `mod6` de 0.222, por lo que concluimos que el modelo ajusta adecuadamente los datos (no hay diferencia significativa entre los datos observados en relación a los esperados).

23.5.3. Curva ROC

En R podemos graficar la curva ROC, que es una herramienta más específica de modelos predictivos, para comparar diferentes modelos. La función `performance_roc()` del paquete `performance` se aplica de la siguiente manera:

```
performance_roc(mod6) |>

  # Genera gráfico
  plot() +

  # Cambia color de fondo
  theme_minimal()
```



Cuanto mayor es el área bajo la curva, más eficiente es el modelo. Si el modelo tiene capacidad predictiva nula la curva coincide con la diagonal principal del cuadrado, y el área bajo la curva toma su valor mínimo de 0,5. Por el contrario, un modelo perfecto tiene una curva ROC con área 1.

Analíticamente podemos visualizar el valor del área bajo la curva (AUC de *Area under the curve*) e incorporar los intervalos de confianza al 95 % con la función `performance_accuracy()`:

```
performance_accuracy(mod6)
```

```
# Accuracy of Model Predictions

Accuracy (95% CI): 94.00% [93.11%, 95.09%]
Method: Area under Curve
```

Los valores de AUC se pueden interpretar siguiendo el criterio:

Valor	Capacidad predictiva
0.5-0.6	Mala
0.6-0.75	Regular
0.75-0.9	Buena
0.9-0.97	Muy buena
0.97-1	Excelente

Nuestro modelo logra un AUC cercano a 0.94, que está dentro de la categoría muy buena.

23.5.4. Cálculo del OR

Los valores de *odds-ratio* se obtienen como el resultado de calcular el valor del número e elevado a los coeficientes estimados del modelo (exponencial o inversa del logaritmo natural).

El código de R para aplicar esta función al `mod6` (con redondeo de 2 decimales) es:

```
coef(mod6) |>
  exp() |>
  round(2)
```

```
(Intercept)  lacto_180Si    edad_dias pareja_estSi
       0.00        20.23         1.01         9.42
```

Cuando estamos frente a una regresión logística múltiple, como en este caso, nos permite obtener las *odds-ratio* ajustadas por otras variables.

La interpretación sería:

- La probabilidad de caminar de los bebés que fueron amamantados por al menos 6 meses fue 20 veces mayor (OR = 20.23) que la de los bebés que no fueron amamantados ese tiempo, ajustada por las variables edad (en días) y pareja estable.

- La probabilidad de caminar de los bebés cuyas madres tenían pareja estable fue aproximadamente 9 veces mayor (OR = 9.42) que la de los bebés cuyas madres no tenían pareja estable, ajustada por las variables edad (en días) y lactancia a los 6 meses.

- La probabilidad de caminar de los bebés se incrementa un 1 % por cada día más de vida (OR = 1.01), ajustada por las variables lactancia a los 6 meses y pareja estable.

Podemos también, incorporar intervalos de confianza de los OR agregando la función `confint()`.

```
confint(mod6) |>
  exp() |>
  round(2)
```

```
             2.5 % 97.5 %
(Intercept)   0.00   0.00
lacto_180Si  12.60  33.52
edad_dias     1.01   1.01
pareja_estSi  5.84  15.59
```

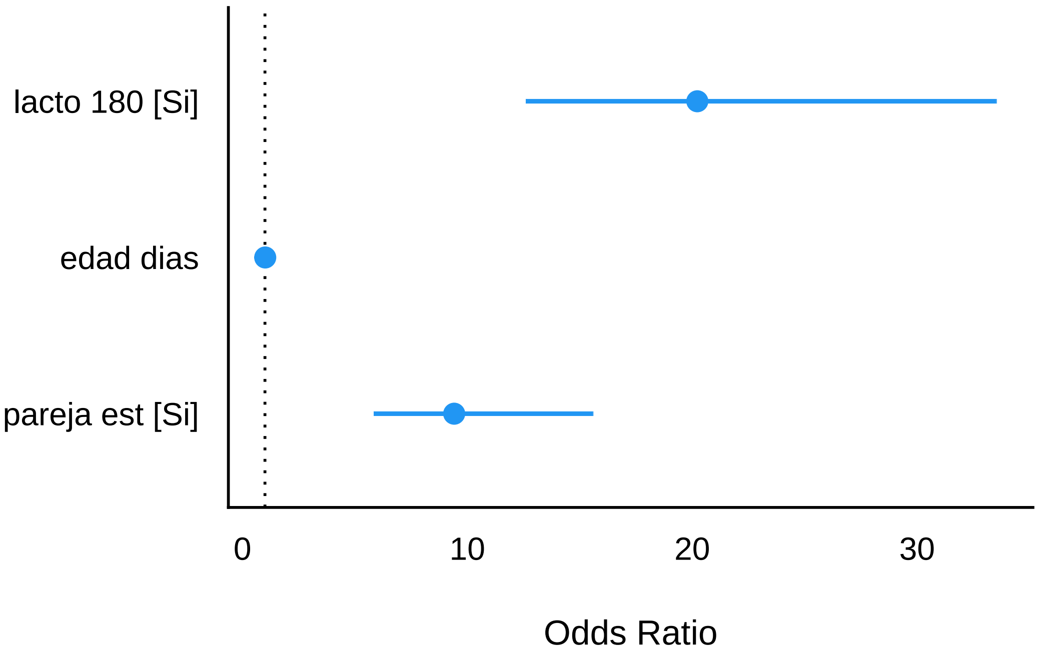
La función `tbl_regression()` del paquete `gtsummary` nos permite generar tablas de los coeficientes exponenciados, su intervalo de confianza y *p*-valor para cada variable del modelo:

```
tbl_regression(mod6, exponentiate = T) |>
  kbl_format()
```

Characteristic	**OR**	**95 % CI**	**p-value**
lacto_180	NA	NA	NA
No	NA	NA	NA
Si	20.2	12.6, 33.5	<0.001
edad_dias	1.01	1.01, 1.01	<0.001
pareja_est	NA	NA	NA
No	NA	NA	NA
Si	9.42	5.84, 15.6	<0.001

Si quisiéramos representar gráficamente el modelo de regresión, podemos usar la función `model_parameters()` de `easystats`:

```
model_parameters(mod6, exponentiate = T) |>
  plot()
```



23.5.5. Residuales

Como en toda regresión, podemos examinar los residuos para asegurarnos que el modelo se ajusta bien a los datos observados.

El principal propósito de examinar los residuales es:

- Detectar los puntos en los que el modelo se ajusta mal.
- Detectar los puntos que ejercen una influencia excesiva sobre el modelo.

Para buscar los casos conflictivos podemos fijarnos en lo siguiente:

- Mirar los residuales estandarizados y asegurarnos que no hay valores atípicos.
- Observar el gráfico *Residuals vs Leverage* incluído en la salida de `plot(modelo)` donde se presentan distancias de Cook.

Los residuales se almacenan dentro del objeto de regresión del modelo bajo el nombre `residuals`. Se puede extraer los residuales estandarizados con la función `rstandar()` y luego clasificar por su valor absoluto y cuantificar. Además, la función `flextable()` del paquete `flextable` (Gohel y Skintzos 2026) nos muestra la salida de manera visualmente atractiva:

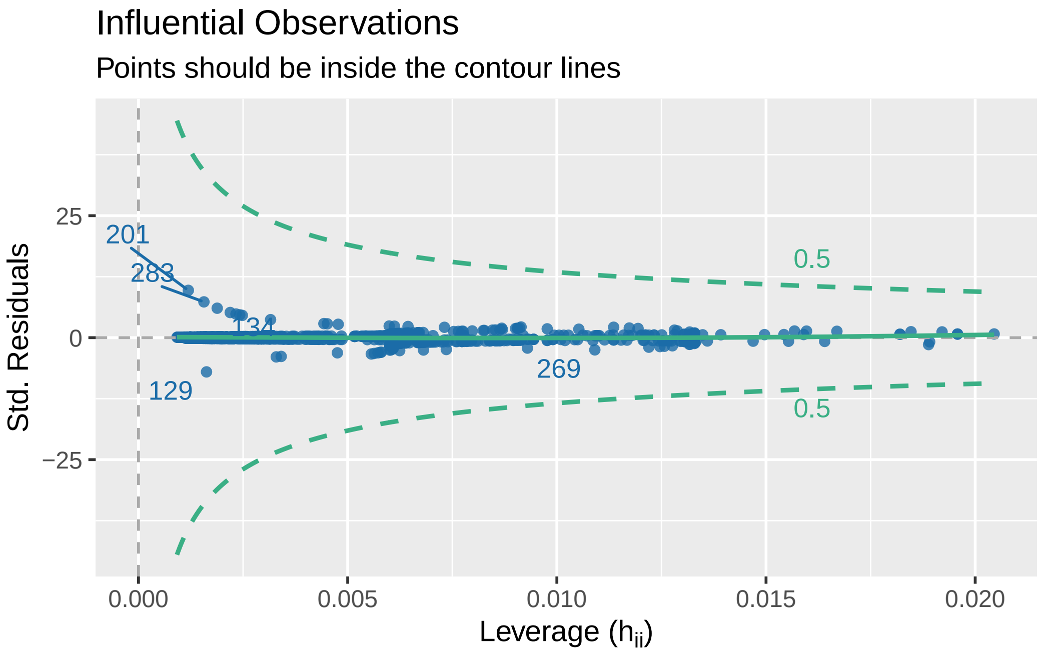
```r
datos$rstandar <- rstandard(mod6)

datos |>
  mutate(
    clasif = case_when(
      between(abs(rstandar), 2, 2.5) ~ "> 2",
      between(abs(rstandar), 2.5, 3) ~ "> 2.5",
      between(abs(rstandar), 3, +Inf) ~ "> 3"
    )
  ) |>
  count(clasif) |>
  mutate(prop = n / sum(n)) |>

  kbl_format()
```

clasif	n	prop
> 2	13	0.01625
> 2.5	8	0.01000
> 3	16	0.02000
NA	763	0.95375

Observamos que las proporciones de valores residuales estandarizados por encima de 2.5 y 3 son muy pequeños. Solo hay un caso mayor de 3 que podríamos revisar.

Podemos obtener el gráfico de las distancias de Cook usando el comando `check_outliers()` de `performance`:

```r
check_outliers(mod6) |>
  plot()
```



De forma similar a la evaluación de residuales de la regresión lineal, estaremos atentos a valores periféricos en la esquina superior e inferior. Esos lugares, fuera de las líneas punteadas, son los lugares donde los puntos influyentes aparecen.

En este ejemplo no se observa ninguno, por lo que en definitiva, no pareciese existir observaciones influyentes que tengan efecto sobre el modelo.

23.5.6. Colinealidad

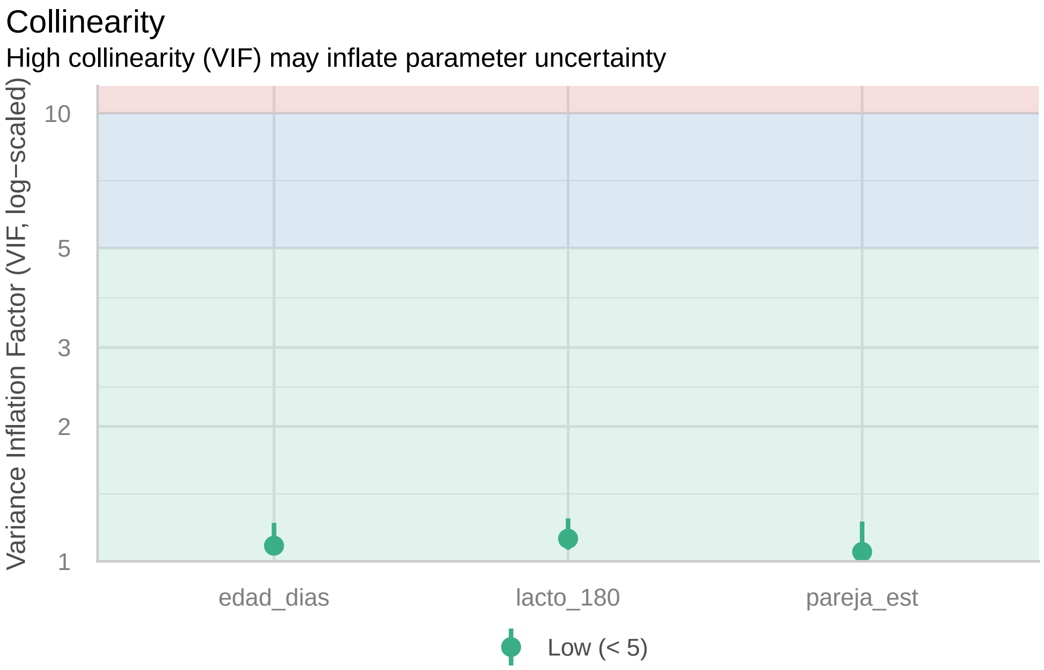
Se dice que existe colinealidad o multicolinealidad cuando dos o más de las covariables del modelo mantienen una relación lineal.

Normalmente, se tolera una multicolinealidad moderada, es decir, una mínima correlación entre covariables. Si esta correlación fuese significativa, su efecto sería el incremento exagerado de los errores estándar y, en ocasiones, del valor estimado para los coeficientes de regresión, lo que hace las estimaciones poco creíbles. Podemos verificar este supuesto con el estadístico VIF (Variance Inflation Factor) utilizando el paquete `performance`:

```r
# Test de colinealidad
check_collinearity(mod6)
```

```
# Check for Multicollinearity

Low Correlation

       Term  VIF   VIF 95% CI adj. VIF Tolerance Tolerance 95% CI
  lacto_180 1.12 [1.06, 1.25]     1.06      0.89     [0.80, 0.94]
  edad_dias 1.08 [1.03, 1.22]     1.04      0.92     [0.82, 0.97]
 pareja_est 1.05 [1.01, 1.23]     1.02      0.95     [0.81, 0.99]
```

```
# Gráfico de colinealidad
check_collinearity(mod6) |>
  plot()
```



Los resultados para cada una de las variables independientes es cercano a 1, por lo que descartamos problemas de multicolinealidad.

23.5.7. Interacción

En este punto aplica el mismo concepto y explicación que la expresada en el material desarrollado en la **Unidad 3**.

Ante la pregunta de la existencia de una variable modificadora de efecto dentro de la regresión múltiple, podemos realizar un análisis comparativo separando por estratos o subgrupos. El objetivo es identificar si la relación de la variable respuesta y una variable independiente cambia de acuerdo al nivel de otra variable independiente.

Tomemos, por ejemplo, las variables `lacto_180` y `pareja_est`. Ambas son estadísticamente significativas dentro del modelo `mod6`. Construyamos dos modelos que contengan `lacto_180` y `edad_dias`, aplicados en dos subgrupos de la tabla `datos`: uno con niños de madres con pareja estable y otro con madres sin pareja estable.

```
# Crear subgrupos
con_pareja <- datos %>% filter(pareja_est == "Si")

sin_pareja <- datos %>% filter(pareja_est == "No")

# Ajustar modelos para cada subgrupo
mod_con_pareja <- glm(
  camina ~ lacto_180 + edad_dias,
  family = binomial(link = "logit"),
  data = con_pareja
)

mod_sin_pareja <- glm(
  camina ~ lacto_180 + edad_dias,
  family = binomial(link = "logit"),
  data = sin_pareja
)
```

Ahora generemos las tablas de coeficientes para cada modelo con la función `tbl_regression()` de `gtsummary`

```
# Comparar coeficientes entre los dos modelos
tab_con_pareja <- tbl_regression(mod_con_pareja, exponentiate = T)

tab_sin_pareja <- tbl_regression(mod_sin_pareja, exponentiate = T)
```

Podemos colocar las tablas una al lado de la otra con la función `tbl_merge()` del mismo paquete:

```
tbl_merge(
  tbls = list(tab_con_pareja, tab_sin_pareja),
  tab_spanner = c("Pareja estable", "Sin pareja estable")
) |>
  kbl_format()
```

Characteristic	**OR**	**95 % CI**	**p-value**	**OR**	**95 % CI**	**p-value**
lacto_180	NA	NA	NA	NA	NA	NA
No	NA	NA	NA	NA	NA	NA
Si	55.1	21.8, 163	<0.001	13.2	7.65, 23.5	<0.001
edad_dias	1.01	1.01, 1.02	<0.001	1.01	1.01, 1.01	<0.001

Al analizar los coeficientes de los modelos ajustados para cada subgrupo, podemos identificar si la relación entre `lacto_180` y `camina`, y entre `edad_dias` y `camina`, varía significativamente según el estado de la variable `pareja_est`. Observamos que los coeficientes de `edad_dias` se mantienen constantes en ambos estratos, mientras que los OR de `lacto_180` son diferentes, esto indica que `pareja_est` actúa como modificadora de efecto.

En R, la forma de incluir y probar un término de interacción dentro de un modelo es mediante el uso del símbolo de multiplicación (`*`) en lugar del `+`. Esto provoca que se tengan en cuenta cada una de las variables individualmente y la interacción entre ellas.

```
mod_interaccion <- glm(
  camina ~ edad_dias + lacto_180 * pareja_est,
  data = datos,
  family = binomial
)
```

Comparo los modelos con y sin interacción usando `performance`:

```
compare_performance(mod6, mod_interaccion, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name            | Model | AIC (weights) | BIC (weights) | Tjur's R2 |  RMSE
----------------------------------------------------------------------------
mod6            |   glm | 505.7 (0.104) | 524.5 (0.548) |     0.618 | 0.309
mod_interaccion |   glm | 501.4 (0.896) | 524.8 (0.452) |     0.622 | 0.307
```

Observamos que el modelo con interacción tiene mejor AIC y R^2 que el `mod6`. Visualicemos ahora sus coeficientes:

```
tbl_regression(mod6, exponentiate = T) |>
  kbl_format()
```

Characteristic	**OR**	**95 % CI**	**p-value**
lacto_180	NA	NA	NA
No	NA	NA	NA
Si	20.2	12.6, 33.5	<0.001
edad_dias	1.01	1.01, 1.01	<0.001
pareja_est	NA	NA	NA
No	NA	NA	NA
Si	9.42	5.84, 15.6	<0.001

El término de interacción es significativo e implica el exceso de la variabilidad de los datos que no puede ser explicada por la suma de las variables consideradas. En este caso la interacción es positiva (existe “sinergia” entre las variables) con un OR de 3.67.

23.5.8. Tamaño de la muestra y número de variables independientes

En el ámbito epidemiológico, es fundamental asegurarse de que el tamaño de la muestra sea adecuado para obtener estimaciones confiables en un modelo de regresión logística. Una regla general para determinar el tamaño de muestra es la fórmula propuesta por Freeman (1987), que establece que el tamaño de muestra debería ser 10 veces el número de variables independientes a estimar, más uno.

$$n = 10 * (k + 1)$$

donde:

- n es el tamaño de la muestra.
- k es el número de variables independientes.

Además, se recomienda que para que la variable dicotómica de respuesta sea fiable, debe haber al menos 10 casos para cada uno de los valores posibles (0 y 1). Esto asegura que las estimaciones sean confiables y que el modelo no esté sesgado.

Para nuestro ejemplo:

- Número de observaciones: 800
- Número de variables independientes en `mod_interaccion`: 4

Aplicando la fórmula de Freeman:

```
n_min <- 10 * (4 + 1)

n_min
```

```
[1] 50
```

Dado que el modelo incluye 4 variables independientes, la fórmula sugiere que se necesita un mínimo de 50 observaciones. En nuestro caso, el tamaño de la muestra es 800, lo cual supera ampliamente el requisito mínimo de 50 observaciones, confirmando que tenemos un tamaño de muestra adecuado para el modelo propuesto.

Verifiquemos ahora si existen al menos 10 casos para cada valor de la variable respuesta:

```
datos |>
  count(camina)
```

```
  camina   n
1      0 381
2      1 419
```

La base de datos tiene 419 casos donde la variable de respuesta es 1 y 381 casos donde es 0. Esto cumple con la recomendación de tener al menos 10 casos para cada valor de la variable dicotómica de respuesta, garantizando que las estimaciones sean confiables.

24. Separación de datos binomiales

24.1. Introducción

En los modelos de regresión logística, se analizan las relaciones entre una variable dependiente de tipo binaria y una o más variables independientes que pueden ser de distintos tipos (numéricas continuas, numéricas discretas, categóricas, binarias, etc.). Sin embargo, en algunas situaciones, la relación entre las variables independientes y la variable respuesta puede derivar en problemas de separación de datos, afectando las estimaciones del modelo.

24.2. Tipos de separación de datos

24.2.1. Separación total

También conocida como separación perfecta o completa, ocurre cuando un predictor o combinación lineal de predictores clasifica perfectamente las observaciones en una de las dos categorías de la variable respuesta. En estos casos, las probabilidades predichas para una de las categorías son 0 o 1, y los coeficientes de regresión para las variables involucradas se vuelven indefinidos. Esto provoca un fallo en la convergencia del modelo, ya que no se pueden calcular los logaritmos para odds ratios (OR) iguales a 0 o a infinito (∞), generando un mensaje de error.

24.2.2. Separación parcial

La separación parcial se presenta cuando un predictor o combinación lineal de predictores predice perfectamente la variable respuesta solo en algunos niveles de las variables independientes. Es decir, separa en gran medida las categorías de la variable respuesta. Esto puede llevar a la estimación de coeficientes y errores estándar extremadamente grandes, con intervalos de confianza muy amplios, lo que hace que las inferencias realizadas sean poco fiables.

Tomemos como ejemplo la base “`covid_cancer.txt`”, que contiene datos de un estudio de casos y controles realizado en la ciudad de Santa Fe, Argentina en pacientes con cáncer hospitalizados por formas severas de COVID-19[1].

Cargamos paquetes necesarios:

```
library(gtsummary)
library(janitor)
library(tidyverse)
```

---

[1] Gastiazoro MP, Cardozo MA, Ricardo T, Ramos JG, Ballina A, Maillo M, et al. Clinical features in cancer patients with COVID-19 in Santa Fe and Buenos Aires, Argentina. J Clin Images Med Case Rep [Internet]. el 14 de febrero de 2023 [citado el 9 de agosto de 2024];4(2). Disponible en: https://jcimcr.org/articles/JCIMCR-v4-2285.html

Cargamos datos:

```
# Carga datos
datos <- read_delim("datos/covid_cancer.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 79
Columns: 13
$ fallecido         <dbl> 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0~
$ sexo              <chr> "M", "M", "F", "F", "F", "M", "F", "M", "F", "M", "M~
$ edad              <dbl> 60, 65, 70, 73, 60, 67, 81, 35, 79, 47, 58, 70, 57, ~
$ comorbilidades    <chr> "Sí", "Sí", "Sí", "Sí", "No", "Sí", "Sí", "No", "Sí"~
$ disnea            <chr> NA, "Sí", "Sí", "No", "No", NA, "No", NA, "Sí", NA, ~
$ neumonia_severa   <chr> "Sí", "Sí", "Sí", "No", "No", "No", "No", "No", "No"~
$ inf_secundaria    <chr> "Sí", NA, "Sí", "No", "No", "No", "Sí", "No", "No", ~
$ complicaciones    <chr> "Sí", NA, "Sí", "No", "No", "No", "Sí", "No", "Sí", ~
$ infil_bilateral   <chr> "Sí", "Sí", "Sí", "No", "No", "No", "No", "No", "No"~
$ asist_resp        <chr> "Sí", "Sí", "Sí", "No", "No", "No", "No", "No", "No"~
$ cancer_tipo       <chr> "Sólido", "Sólido", "Sólido", "Sólido", "Sólido", "S~
$ tr_quimioterapia  <chr> "Sí", "Sí", "Sí", "Sí", "Sí", "Sí", "Sí", "Sí", "Sí"~
$ tr4_quimioterapia <chr> "Sí", "Sí", "Sí", "Sí", "Sí", "Sí", "No", "No", "No"~
```

Las variables de interés incluyen:

- `fallecido`: fallecimiento por COVID-19 (No: 0, Sí: 1)
- `sexo`: sexo biológico del/a paciente (M: masculino, F: femenino)
- `edad`: edad en años al momento de la hospitalización
- `comorbilidades`: presencia de comorbilidades (Sí, No)
- `disnea`: dificultad para respirar (Sí, No)
- `neumonia_severa`: paciente con neumonía severa (Sí, No)
- `inf_secundaria`: presencia de infección secundaria (Sí, No)
- `complicaciones`: complicaciones del COVID-19 (Sí, No)
- `infil_bilateral`: infiltración bilateral en radiografía (Sí, No)
- `asist_resp`: paciente que recibió asistencia respiratoria (Sí, No)
- `cancer_tipo`: tipo de cáncer (Sólido, Hematológico)
- `tr_quimioterapia`: paciente que recibió quimioterapia (Sí, No)
- `tr4_quimioterapia`: paciente que recibió quimioterapia en el último mes (Sí, No)

Nuestra variable dependiente es si el/la paciente falleció a causa del COVID-19. Comenzaremos por ajustar modelos de regresión logística univariados para evaluar qué variables explicativas están asociadas significativamente a la variable dependiente:

```
# Regresiones logísticas simples
datos |>
  tbl_uvregression(
    y = fallecido,
    method = glm,
    method.args = list(family = binomial),
    exponentiate = T
  ) |>
  bold_p()
```

Al ejecutar este código aparecerá el siguiente mensaje de advertencia: `Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred`. Esto indica que todas o alguna de las variables explicativas provoca separación parcial de los datos.

Al observar la tabla generada, notamos que las variables `disnea`, `neumonia_severa`, `inf_secundaria`, `asist_resp` y `tr_quimioterapia` presentan intervalos de confianza excesivamente amplios, lo que sugiere la presencia de separación parcial. Si generamos tablas de 2x2 de cada variable explicativa en función de la variable `fallecido`, podemos visualizar este fenómeno:

```
# disnea
datos |>
  tabyl(disnea, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 disnea     0     1
     No 91.2%  8.8%
     Sí 52.9% 47.1%
```

```
# neumonía severa
datos |>
  tabyl(neumonia_severa, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 neumonia_severa     0     1
              No 90.3%  9.7%
              Sí 52.9% 47.1%
```

```
# infección secundaria
datos |>
  tabyl(inf_secundaria, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 inf_secundaria     0     1
             No 86.2% 13.8%
             Sí 42.9% 57.1%
```

Characteristic	**N**	**OR**	**95 % CI**	**p-value**
sexo	78			
F		—	—	
M		1.95	0.61, 6.56	0.3
edad	74	1.04	0.99, 1.10	0.12
comorbilidades	53			
No		—	—	
Sí		0.93	0.27, 3.30	>0.9
disnea	51			
No		—	—	
Sí		9.19	2.18, 49.2	**0.004**
neumonia_severa	79			
No		—	—	
Sí		8.30	2.37, 31.2	**0.001**
inf_secundaria	36			
No		—	—	
Sí		8.33	1.38, 59.1	**0.023**
complicaciones	36			
No		—	—	
Sí		1,139,380,581	0.00,	>0.9
infil_bilateral	40			
No		—	—	
Sí		1.56	0.42, 5.98	0.5
asist_resp	39			
No		—	—	
Sí		6.00	1.26, 44.4	**0.039**
cancer_tipo	74			
Hematológico		—	—	
Sólido		3,844,194	0.00,	>0.9
tr_quimioterapia	49			
No		—	—	
Sí		5.20	0.85, 101	0.14
tr4_quimioterapia	39			
No		—	—	
Sí		1.40	0.33, 6.48	0.7

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```
# asistencia respiratoria
datos |>
  tabyl(asist_resp, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 asist_resp     0     1
         No 88.9% 11.1%
         Sí 57.1% 42.9%
```

```
# quimioterapia
datos |>
  tabyl(tr_quimioterapia, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 tr_quimioterapia     0     1
               No 92.9%  7.1%
               Sí 71.4% 28.6%
```

Estos análisis muestran que variables como `disnea`, `neumonia_severa`, `inf_secundaria`, `asist_resp` y `tr_quimioterapia` explican casi completamente la mortalidad por COVID-19 severo.

Si evaluamos las variables `complicaciones` y `cancer_tipo`:

```
# complicaciones
datos |>
  tabyl(complicaciones, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
 complicaciones      0     1
             No 100.0%  0.0%
             Sí  42.9% 57.1%
```

```
# tipo de cáncer
datos |>
  tabyl(cancer_tipo, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
  cancer_tipo      0     1
 Hematológico 100.0%  0.0%
       Sólido  80.3% 19.7%
```

Observamos que todos los pacientes sin complicaciones o con cáncer hematológico sobrevivieron, lo cual también indica separación parcial de los datos.

Ahora, ajustemos un modelo para evaluar el efecto de la interacción entre el tipo de cáncer y haber recibido quimioterapia en los 30 días previos, controlando por la presencia de neumonía severa y asistencia respiratoria:

```
fit1 <- glm(
  fallecido ~ cancer_tipo * tr4_quimioterapia + neumonia_severa + asist_resp,
  data = datos,
  family = binomial
)


summary(fit1)
```

```
Call:
glm(formula = fallecido ~ cancer_tipo * tr4_quimioterapia + neumonia_severa +
    asist_resp, family = binomial, data = datos)

Coefficients: (1 not defined because of singularities)
                                       Estimate Std. Error z value Pr(>|z|)
(Intercept)                            -19.4976  2399.5450  -0.008   0.9935
cancer_tipoSólido                       16.7936  2399.5448   0.007   0.9944
tr4_quimioterapiaSí                      0.8431     1.0135   0.832   0.4055
neumonia_severaSí                        2.6769     1.3791   1.941   0.0522 .
asist_respSí                             0.2547     1.3683   0.186   0.8523
cancer_tipoSólido:tr4_quimioterapiaSí        NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 38.024  on 31  degrees of freedom
Residual deviance: 26.599  on 27  degrees of freedom
  (47 observations deleted due to missingness)
AIC: 36.599

Number of Fisher Scoring iterations: 15
```

El resumen del modelo muestra el mensaje: `Coefficients: (1 not defined because of singularities)`, indicando que los errores estándar para el intercepto y `cancer_tipo` son muy grandes. Además, no se muestran los coeficientes ni el *p*-valor para la interacción. Esto se puede visualizar tabulando los coeficientes:

```
tbl_regression(fit1, exponentiate = T)
```

Recibirás advertencias como: `Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred` y `Warning in regularize.values(x, y, ties, missing(ties), na.rm = na.rm) : collapsing to unique 'x' values`, lo que sugiere separación parcial de los datos.

Characteristic	**OR**	**95 % CI**	**p-value**
cancer_tipo			
Hematológico	—	—	
Sólido	19,650,260	0.00,	>0.9
tr4_quimioterapia			
No	—	—	
Sí	2.32	0.33, 21.1	0.4
neumonia_severa			
No	—	—	
Sí	14.5	1.30, 416	0.052
asist_resp			
No	—	—	
Sí	1.29	0.05, 17.7	0.9
cancer_tipo * tr4_quimioterapia			
Sólido * Sí			

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

Para evaluar separación de estos datos, podemos crear una nueva variable que represente el cruce entre tipo de cáncer y quimioterapia usando la función `fct_cross()` de `tidyverse`:

```
datos |>
  mutate(tipo_ca_quim = fct_cross(cancer_tipo, tr4_quimioterapia)) |>
  tabyl(tipo_ca_quim, fallecido, show_na = F) |>
  adorn_percentages() |>
  adorn_pct_formatting()
```

```
    tipo_ca_quim      0     1
 Hematológico:No 100.0%  0.0%
       Sólido:No  76.5% 23.5%
       Sólido:Sí  71.4% 28.6%
```

Este análisis muestra que ninguno de los pacientes con cáncer hematológico recibió quimioterapia en los últimos 30 días, lo que también contribuye a la separación parcial.

Finalmente, para evaluar la separación de datos de manera más formal, utilizamos el paquete `detectseparation` (Kosmidis et al. 2026). Ajustamos nuevamente el modelo con el argumento `method = "detect_separation"`:

```
library(detectseparation)

# Tipo de cáncer
glm(
  fallecido ~ cancer_tipo,
  data = datos,
  family = binomial,
  method = "detect_separation"
)
```

```
Implementation: ROI | Solver: lpsolve
Separation: TRUE
Existence of maximum likelihood estimates
      (Intercept) cancer_tipoSólido
             -Inf               Inf
0: finite value, Inf: infinity, -Inf: -infinity
```

```
# Neumonía severa
glm(
  fallecido ~ neumonia_severa,
  data = datos,
  family = binomial,
  method = "detect_separation"
)
```

```
Implementation: ROI | Solver: lpsolve
Separation: FALSE
Existence of maximum likelihood estimates
      (Intercept) neumonia_severaSí
                0                 0
0: finite value, Inf: infinity, -Inf: -infinity
```

```
# Modelo con interacción
glm(
  fallecido ~ cancer_tipo * tr4_quimioterapia + neumonia_severa + asist_resp,
  data = datos,
  family = binomial,
  method = "detect_separation"
)
```

```
Implementation: ROI | Solver: lpsolve
Separation: TRUE
Existence of maximum likelihood estimates
                          (Intercept)                     cancer_tipoSólido
                                 -Inf                                   Inf
                  tr4_quimioterapiaSí                     neumonia_severaSí
                                    0                                     0
                         asist_respSí cancer_tipoSólido:tr4_quimioterapiaSí
                                    0                                    NA
0: finite value, Inf: infinity, -Inf: -infinity
```

En el caso de separación completa, un nivel de la variable explicativa debería modelar perfectamente los “Sí” y el otro los “No”, y al intentar correr el modelo nos encontraríamos con un error de convergencia que impediría continuar con el análisis.

24.3. Soluciones a la separación de datos

24.3.1. Remuestreo, recolección o submuestreo

Siempre que fuera posible, recolectar más datos puede mitigar la separación total al introducir más variabilidad en los datos. En su defecto, se puede considerar el submuestreo o el sobremuestreo para balancear las clases. Sin embargo, estas técnicas deben aplicarse con cuidado para evitar sesgos.

24.3.2. Eliminación o recategorización de variables

En casos de separación parcial, una opción es simplificar el modelo eliminando las variables causantes de la separación. Si esas variables son esenciales, otra alternativa es agrupar niveles de las variables categóricas para reducir la separación. Esto puede hacer que el modelo sea más robusto al reducir la complejidad y mejorar la estabilidad de las estimaciones.

24.3.3. Uso de modelos alternativos a la regresión logística

Los **modelos de *pseudo-likelihood***, como la regresión logística condicional y la máxima verosimilitud penalizada, permiten ajustar el modelo incluso cuando la separación de datos impide la convergencia con métodos tradicionales. Los **modelos penalizados**, que incluyen las regresiones Ridge, Lasso y Elastic Net, añaden un término de penalización que controla el crecimiento excesivo de los coeficientes en casos de separación parcial, facilitando la convergencia y mejorando la estabilidad del modelo. Por otro lado, los **modelos bayesianos** introducen distribuciones previas sobre los coeficientes, permitiendo una mayor flexibilidad para manejar la separación de datos y ofreciendo estimaciones más estables y confiables. Sin embargo, el ajuste e interpretación de estos modelos es complejo y escapa al alcance de este curso, por lo que no serán abordados en detalle.

25. Regresión logística condicional

25.1. Introducción

En algunos estudios de casos y controles, se utiliza un diseño apareado para controlar la influencia de factores de confusión como la edad, el sexo o el lugar de residencia. La regresión logística condicional (RLC) es una extensión de la regresión logística clásica que incorpora el apareamiento de los datos, estratificando el análisis por grupos definidos por las variables de emparejamiento (**estratos**).

Este modelo permite estimar el efecto de las covariables sobre una variable dependiente binaria mientras controla de manera efectiva los efectos de las variables de emparejamiento o agrupamiento. Es especialmente útil en estudios de casos y controles apareados, investigaciones multicéntricas y estudios observacionales que abarcan múltiples ubicaciones geográficas, proporcionando un enfoque robusto para manejar la heterogeneidad dentro de los datos.

25.2. Modelo logístico condicional

El modelo logístico condicional utiliza una función de verosimilitud condicional que elimina la necesidad de estimar los efectos de los factores de emparejamiento como interceptos adicionales. Este enfoque resulta especialmente útil cuando hay un gran número de grupos o estratos, ya que permite concentrarse exclusivamente en los coeficientes de las covariables de interés.

Se modela la probabilidad de que un evento ocurra en función de las covariables dentro de cada estrato, definido por las variables de emparejamiento. La función de verosimilitud condicional se puede expresar matemáticamente como:

$$logit(p) = \alpha_1 + \alpha_2 z_2 + ... + \alpha_s z_s + \beta_1 x_1 + ... + \beta_p x_p$$

donde:

- $\alpha_1 + \alpha_2 z_2 + ... + \alpha_s z_s$: Coeficientes asociados a las variables de estratificación z_s, que identifican los estratos.
- $z_1, z_2, ..., z_s$: Variables de emparejamiento o estratos.
- $\beta_1, ..., \beta_p$: Efectos de las variables independientes ajustadas por los estratos.
- $x_1, ...x_p$: Variables independientes incluídas en el modelo.

El uso de la verosimilitud condicional simplifica el modelo al evitar la estimación de un gran número de interceptos y reduce el sesgo en situaciones con pocos datos por estrato.

25.2.1. Supuestos de la regresión logística condicional

- **Independencia dentro de los estratos:** Las observaciones son independientes entre los diferentes estratos o grupos definidos por el emparejamiento, pero pueden estar correlacionadas dentro de los mismos.
- **Relación logit-lineal:** Existe una relación lineal entre el logit de la probabilidad del evento y las covariables explicativas.
- **No existencia de confusión residual:** Las variables de emparejamiento deben ser suficientes para controlar por los factores de confusión.
- **Tamaño adecuado del estrato:** Cada estrato debe tener al menos un caso y un control.

25.3. Ejemplo práctico en R

En R podemos ajustar modelos de regresión logística condicional usando la función `clogit()` del paquete `survival` (Therneau 2026).

Cargamos los paquetes necesarios:

```r
# Análisis estadístico
library(gtsummary)
library(survival)

# Chequeo de supuestos
library(easystats)

# Apareamiento de datos
library(MatchIt)

# Manejo y exploración de datos
library(skimr)
library(janitor)
library(tidyverse)
```

Usaremos la base “`higado_graso.txt`” que contiene datos de un estudio de casos y controles apareados por sexo y edad para muertes por hígado graso no alcohólico:

```r
# Carga datos
datos <- read_delim("datos/higado_graso.txt") |>

  # Cambia nivel de referencia de fib4
  mutate(fib4_cat = fct_rev(fib4_cat))

# Explora datos
glimpse(datos)
```

```
Rows: 2,467
Columns: 11
$ id          <dbl> 19, 25, 26, 32, 56, 57, 68, 77, 78, 99, 100, 120, 126, 13~
$ tipo        <dbl> 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, ~
$ edad        <dbl> 35, 39, 55, 45, 51, 81, 51, 55, 32, 33, 50, 82, 56, 69, 7~
$ sexo        <dbl> 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, ~
$ bmi         <dbl> 30.49939, 24.26164, 30.59225, 22.11900, 36.42498, 26.3815~
$ hdl         <dbl> 50.09091, 52.50000, 49.13043, 70.55556, 45.10526, 67.7500~
$ p_sistolica  <dbl> NA, NA, 134.2500, NA, 164.0000, 148.0833, 124.5000, NA, N~
$ p_diastolica <dbl> NA, NA, 134.2500, NA, 164.0000, 148.0833, 124.5000, NA, N~
$ fib4_cat    <fct> bajo riesgo, bajo riesgo, bajo riesgo, bajo riesgo, bajo ~
$ fuma        <dbl> 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, ~
$ status      <dbl> 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, ~
```

El dataset tiene 2467 observaciones y 11 columnas. Las variables de interés son las siguientes:

- `id`: identificador único del paciente.
- `tipo`: Indica si el paciente tiene hígado graso no alcohólico (caso: 1) o no presenta la condición (control: 0).
- `edad`: Edad del paciente al momento de su ingreso al hospital.
- `sexo`: Sexo biológico del paciente (1: masculino, 0: femenino).
- `bmi`: Índice de masa corporal del paciente al momento de la hospitalización.
- `hdl`: Nivel de lipoproteínas de alta densidad (HDL) en sangre.
- `p_sistolica`: Presión sistólica en milímetros de mercurio (mmHg).
- `p_diastolica`: Presión diastólica en milímetros de mercurio (mmHg).
- `fib4_cat`: Índice de fibrosis hepática categorizado (bajo riesgo, alto riesgo).
- `fuma`: Tabaquismo (0: no, 1: sí).
- `status`: Variable dependiente que toma el valor de 0 si la persona sobrevivió y 1 si falleció.

Exploremos más en profundidad los datos usando la función `tabyl()` del paquete `janitor` y la función `skim()` del paquete `skimr`:

```r
# Número de casos y controles
tabyl(datos$tipo) |>
  adorn_pct_formatting()
```

```
 datos$tipo    n percent
          0 1685   68.3%
          1  782   31.7%
```

```r
# Niveles fib4
tabyl(datos$fib4_cat)
```

```
 datos$fib4_cat    n   percent
    bajo riesgo 2109 0.8548845
    alto riesgo  358 0.1451155
```

```
# Explora valores ausentes
skim(datos)
```

Tabla 25.1.: Data summary

Name	datos
Number of rows	2467
Number of columns	11
Column type frequency:	
factor	1
numeric	10
Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
fib4_cat	0	1	FALSE	2	baj: 2109, alt: 358

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
id	0	1.00	8644.27	4998.09	19.00	4486.00	8507.00	12864.00	17563.0	⍰⍰⍰⍰⍰
tipo	0	1.00	0.32	0.47	0.00	0.00	0.00	1.00	1.0	⍰⍰⍰⍰⍰
edad	0	1.00	53.60	14.73	18.00	44.00	53.00	64.00	98.0	⍰⍰⍰⍰⍰
sexo	0	1.00	0.50	0.50	0.00	0.00	0.00	1.00	1.0	⍰⍰⍰⍰⍰
bmi	0	1.00	31.22	7.51	9.21	26.02	30.14	35.04	84.4	⍰⍰⍰⍰⍰
hdl	0	1.00	50.92	14.94	19.83	40.00	48.53	58.50	134.0	⍰⍰⍰⍰⍰
p_sistolica	928	0.62	131.41	17.11	40.00	121.53	132.50	142.67	211.0	⍰⍰⍰⍰⍰
p_diastolica	928	0.62	131.41	17.11	40.00	121.53	132.50	142.67	211.0	⍰⍰⍰⍰⍰
fuma	0	1.00	0.64	0.48	0.00	0.00	1.00	1.00	1.0	⍰⍰⍰⍰⍰
status	0	1.00	0.12	0.32	0.00	0.00	0.00	0.00	1.0	⍰⍰⍰⍰⍰

Las variables `p_sistolica` y `p_diastolica` tienen gran cantidad de valores ausentes, por lo que no serán consideradas para el análisis.

Como no tenemos definido a que caso corresponde cada control, vamos a aparear casos y controles usando la función `matchit()` del paquete `MatchIt` (Ho et al. 2011), especificando que queremos dos controles por caso con el argumento `ratio = 2`:

```
# Aparea casos y controles por sexo y edad
match <- matchit(tipo ~ edad + sexo, data = datos, exact = "sexo", ratio = 2)

# Resumen del apareamiento
summary(match)
```

```
Call:
matchit(formula = tipo ~ edad + sexo, data = datos, exact = "sexo",
    ratio = 2)

Summary of Balance for All Data:
         Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance        0.3268        0.3124          0.2552     1.0589    0.0507
edad           51.0243       54.7923         -0.2550     1.0302    0.0486
sexo            0.4962        0.5003         -0.0083          .    0.0041
         eCDF Max
distance   0.1284
edad       0.1274
sexo       0.0041

Summary of Balance for Matched Data:
         Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance        0.3268        0.3181          0.1549     1.1495    0.0301
edad           51.0243       53.2270         -0.1490     1.1373    0.0288
sexo            0.4962        0.4962          0.0000          .    0.0000
         eCDF Max Std. Pair Dist.
distance   0.0882          0.1578
edad       0.0838          0.1528
sexo       0.0000          0.0000

Sample Sizes:
          Control Treated
All          1685     782
Matched      1564     782
Unmatched     121       0
Discarded       0       0
```

Obtenemos los datos apareados:

```
datos <- match.data(match, drop.unmatched = T)

names(datos)
```

```
 [1] "id"          "tipo"        "edad"        "sexo"        "bmi"
 [6] "hdl"         "p_sistolica" "p_diastolica" "fib4_cat"    "fuma"
[11] "status"      "distance"    "weights"     "subclass"
```

Ajustamos el modelo de regresión logística condicional, usando la variable `subclass` como variable de estratificación:

```
mod_cond <- clogit(
  status ~ bmi + hdl + fuma + fib4_cat + strata(subclass),
  data = datos
)
```

Revisemos la salida del modelo:

```
summary(mod_cond)
```

```
Call:
coxph(formula = Surv(1 + 0 * status, status) ~ bmi + hdl + fuma + 
    fib4_cat + strata(subclass), data = datos, method = "exact")

  n= 2346, number of events= 251 

                         coef exp(coef)  se(coef)      z Pr(>|z|)    
bmi                  0.005507  1.005522  0.012814  0.430    0.667    
hdl                 -0.006502  0.993519  0.006980 -0.931    0.352    
fuma                -0.237017  0.788978  0.186629 -1.270    0.204    
fib4_catalto riesgo  1.072163  2.921692  0.224099  4.784 1.72e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

                    exp(coef) exp(-coef) lower .95 upper .95
bmi                    1.0055     0.9945    0.9806     1.031
hdl                    0.9935     1.0065    0.9800     1.007
fuma                   0.7890     1.2675    0.5473     1.137
fib4_catalto riesgo    2.9217     0.3423    1.8831     4.533

Concordance= 0.625  (se = 0.039 )
Likelihood ratio test= 27.94  on 4 df,   p=1e-05
Wald test            = 25.92  on 4 df,   p=3e-05
Score (logrank) test = 28.23  on 4 df,   p=1e-05
```

Probemos eliminar una de las variables explicativas no significativas a la vez:

```
# (-) Tabaquismo
mod_cond1 <- clogit(
  status ~ bmi + hdl + fib4_cat + strata(subclass),
  data = datos
)

# (-) IMC
mod_cond2 <- clogit(
  status ~ hdl + fuma + fib4_cat + strata(sexo, edad),
  data = datos
)

# (-) HDL
mod_cond3 <- clogit(
  status ~ bmi + fuma + fib4_cat + strata(sexo, edad),
  data = datos
)
```

Comparamos los modelos

```
# Compara modelos
compare_performance(
  mod_cond,
  mod_cond1,
  mod_cond2,
  mod_cond3,
  metrics = "common"
)
```

```
# Comparison of Model Performance Indices

Name      |  Model | AIC (weights) | BIC (weights) | Nagelkerke's R2 |  RMSE
----------------------------------------------------------------------------
mod_cond  | clogit | 393.1 (0.452) | 416.2 (0.044) |           0.073 | 0.230
mod_cond1 | clogit | 392.8 (0.548) | 410.0 (0.956) |           0.069 | 0.230
mod_cond2 | clogit | 921.1 (<.001) | 938.3 (<.001) |           0.055 | 0.276
mod_cond3 | clogit | 923.5 (<.001) | 940.8 (<.001) |           0.052 | 0.276
```

```
# Compara modelos y ordena según performance
compare_performance(
  mod_cond,
  mod_cond1,
  mod_cond2,
  mod_cond3,
  metrics = "common",
  rank = T
)
```

```
# Comparison of Model Performance Indices

Name      |  Model | Nagelkerke's R2 |  RMSE | AIC weights | BIC weights | Performance-
Score
--------------------------------------------------------------------------------------------
mod_cond1 | clogit |           0.069 | 0.230 |       0.548 |       0.956 |            95.02%
mod_cond  | clogit |           0.073 | 0.230 |       0.452 |       0.044 |            71.65%
mod_cond2 | clogit |           0.055 | 0.276 |   1.04e-115 |   1.81e-115 |             3.55%
mod_cond3 | clogit |           0.052 | 0.276 |   3.12e-116 |   5.45e-116 |             0.40%
```

El modelo sin `fuma` tiene mejor *performance* que el modelo saturado:

```r
# Salida modelo condicional 1
summary(mod_cond1)
```

```
Call:
coxph(formula = Surv(1 + 0 * status, status) ~ bmi + hdl + fib4_cat + 
    strata(subclass), data = datos, method = "exact")

  n= 2346, number of events= 251 

                         coef exp(coef)  se(coef)      z Pr(>|z|)    
bmi                  0.004445  1.004445  0.012740  0.349    0.727    
hdl                 -0.006922  0.993102  0.006941 -0.997    0.319    
fib4_catalto riesgo  1.064462  2.899278  0.223595  4.761 1.93e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

                    exp(coef) exp(-coef) lower .95 upper .95
bmi                    1.0045     0.9956    0.9797     1.030
hdl                    0.9931     1.0069    0.9797     1.007
fib4_catalto riesgo    2.8993     0.3449    1.8705     4.494

Concordance= 0.641  (se = 0.038 )
Likelihood ratio test= 26.33  on 3 df,   p=8e-06
Wald test            = 24.56  on 3 df,   p=2e-05
Score (logrank) test = 26.63  on 3 df,   p=7e-06
```

```r
# (-) BMI
mod_cond1a <- clogit(status ~ hdl + fib4_cat + strata(subclass), data = datos)

# (-) HDL
mod_cond1b <- clogit(status ~ bmi + fib4_cat + strata(subclass), data = datos)

# Compara modelos y ordena según performance
compare_performance(
  mod_cond1,
  mod_cond1a,
  mod_cond1b,
  metrics = "common",
  rank = T
)
```

```
# Comparison of Model Performance Indices

Name       |  Model | Nagelkerke's R2 |  RMSE | AIC weights | BIC weights | Performance-
Score
-------------------------------------------------------------------------------------------
mod_cond1a | clogit |           0.069 | 0.230 |       0.492 |       0.601 |             72.00
mod_cond1  | clogit |           0.069 | 0.230 |       0.192 |       0.013 |             50.00
mod_cond1b | clogit |           0.066 | 0.230 |       0.316 |       0.386 |             32.90
```

```r
# Salida modelo condicional 1b
summary(mod_cond1b)
```

```
Call:
coxph(formula = Surv(1 + 0 * status, status) ~ bmi + fib4_cat +
    strata(subclass), data = datos, method = "exact")

  n= 2346, number of events= 251

                        coef exp(coef) se(coef)     z Pr(>|z|)
bmi                 0.007979  1.008011 0.012127 0.658    0.511
fib4_catalto riesgo 1.067510  2.908129 0.223684 4.772 1.82e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

                    exp(coef) exp(-coef) lower .95 upper .95
bmi                     1.008     0.9921    0.9843     1.032
fib4_catalto riesgo     2.908     0.3439    1.8759     4.508

Concordance= 0.601  (se = 0.04 )
Likelihood ratio test= 25.32  on 2 df,   p=3e-06
Wald test            = 23.74  on 2 df,   p=7e-06
Score (logrank) test = 25.68  on 2 df,   p=3e-06
```

```r
# (-) bmi
mod_cond1c <- clogit(
  status ~ fib4_cat +
    strata(subclass),
  data = datos
)

# Compara modelos
compare_performance(mod_cond1b, mod_cond1c, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name       |  Model | AIC (weights) | BIC (weights) | Nagelkerke's R2 |  RMSE
-----------------------------------------------------------------------------
mod_cond1b | clogit | 391.8 (0.313) | 403.3 (0.025) |           0.066 | 0.230
mod_cond1c | clogit | 390.2 (0.687) | 396.0 (0.975) |           0.065 | 0.230
```

Nos quedamos con el modelo que tiene solo `fib4_cat` como variable explicativa:

```r
tbl_regression(mod_cond1c, exponentiate = T)
```

En el modelo final, las personas con alto riesgo de fibrosis hepática presentan 3,7 veces más probabilidades de morir (95 % IC: 2,45-5,57) que aquellas sin la condición.

Characteristic	**OR**	**95 % CI**	**p-value**
fib4_cat			
bajo riesgo	—	—	
alto riesgo	2.94	1.90, 4.55	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

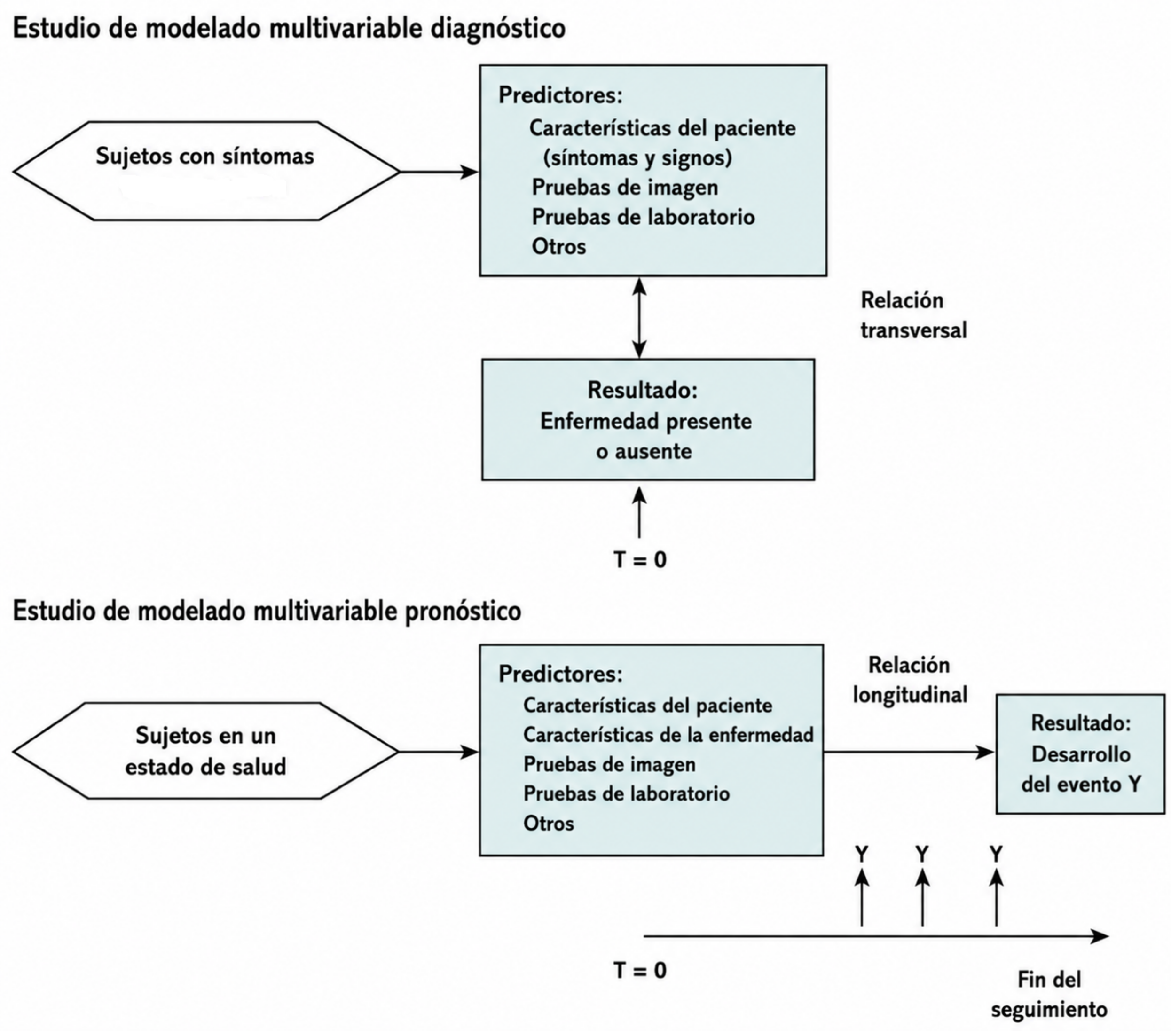
26. Validación de modelos predictivos

26.1. ¿Qué es un modelo predictivo?

Un modelo de predicción es una ecuación matemática que combina información de múltiples predictores o variables observadas o medidas en un individuo para predecir la probabilidad de presencia (diagnóstico) o aparición futura (pronóstico) de un resultado específico. Desde una perspectiva de investigación, el diagnóstico y el pronóstico constituyen un desafío similar: el médico dispone de cierta información y desea saber cómo se relaciona con el verdadero estado del paciente, ya sea que este se pueda conocer en el momento actual (diagnóstico) o solo en algún momento futuro (pronóstico). Para muchas de las aplicaciones médicas, el resultado de interés es binario y la información puede expresarse como predicciones probabilísticas. Los modelos predictivos se emplean en dos contextos o entornos distintos:

- **Entorno pronóstico:** En el cual se desea determinar la probabilidad de que un evento específico ocurrirá en el futuro. Es decir que, estiman la probabilidad de un resultado o evento particular (por ejemplo, mortalidad, aparición o recurrencia de una enfermedad, complicación) dentro de un período determinado en el futuro (horas, días, semanas o años), por ejemplo, para planificar decisiones terapéuticas o de estilo de vida.
- **Entorno diagnóstico:** donde el objetivo se centra en la probabilidad de que cierta condición o enfermedad esté presente o ausente en el momento de la predicción. Como ejemplos podemos considerar un modelo para informar sobre la derivación de pacientes para pruebas adicionales o un modelo para iniciar el tratamiento directamente.

En la Figura 1, adaptada de Collins et al. (2015), se muestra que la principal diferencia entre los modelos predictivos con objetivo pronóstico o diagnóstico es el tiempo para el cual se estiman las probabilidades.


Estudio de modelado multivariable diagnóstico
Sujetos con síntomas
Predictores:
Características del paciente (síntomas y signos)
Pruebas de imagen
Pruebas de laboratorio
Otros
Relación transversal
Resultado:
Enfermedad presente o ausente
T = 0
Estudio de modelado multivariable pronóstico
Sujetos en un estado de salud
Predictores:
Características del paciente
Características de la enfermedad
Pruebas de imagen
Pruebas de laboratorio
Otros
Relación longitudinal
Resultado:
Desarrollo del evento Y
Y
Y
Y
T = 0
Fin del seguimiento

En esta unidad del curso hemos aprendido a construir un modelo de regresión logística. Aprendimos también a evaluar la calidad de dicho modelo. Pero si reflexionamos acerca de la finalidad de un modelo predictivo, podemos pensar que consiste en predecir la variable respuesta en observaciones futuras o en observaciones que el modelo no ha “visto” antes. Consideraremos modelos de predicción que proporcionan predicciones para un resultado dicotómico, ya que son los más relevantes en aplicaciones médicas, por eso desarrollamos este tema en el contexto de la regresión logística. Con estos modelos, buscamos obtener al menos predicciones internamente válidas, es decir, predicciones válidas para sujetos de la población subyacente. Preferiblemente, las predicciones también deben ser generalizables a poblaciones plausiblemente relacionadas. En una estrategia de modelado para datos empíricos, es necesario considerar diversas cuestiones, tanto epidemiológicas como estadísticas. Es por eso que los modelos necesitan ser validados.

26.2. ¿En qué consiste la validación de modelos predictivos?

Cuando hacemos un análisis de residuos, estamos evaluando el error que comete el modelo en predecir observaciones que ya ha “visto”. Pero ¿cómo funcionará con observaciones que nunca “vio”? La validación de un modelo predictivo es el proceso mediante el cual se evalúa qué tan bien funcionará el modelo cuando se aplique a nuevos individuos o nuevas observaciones, distintos de aquellos utilizados para desarrollarlo.

Su objetivo principal es responder la pregunta: ¿El modelo mantiene su capacidad predictiva fuera de la muestra con la que fue construido? La validación permite determinar si el modelo es generalizable o si, por el contrario, está sobreajustado (*overfitting*). Cuando hablamos de validación, debemos considerar dos instancias:

- **Validación externa**: es la validación más estricta, consiste en emplear el modelo predictivo desarrollado en otra muestra de pacientes.
- **Validación interna**: es menos estricta pues usa los datos de los sujetos a partir de los cuales se ha desarrollado el modelo predictivo, pero no necesita del esfuerzo añadido de reclutar nuevos individuos y, por tanto, está al alcance del propio investigador en el momento que desarrolla el modelo.

Para garantizar que los estudios sobre modelos de predicción sean claros, completos y replicables, y que, a su vez, se permita a los lectores evaluar su validez y aplicabilidad clínica, se elaboró la Guía TRIPOD.

26.2.1. Declaración TRIPOD

La declaración TRIPOD **(**Transparent Reporting of multivariable prediction model for Individual Prognosis or Diagnosis) se publicó en 2015 para proporcionar las recomendaciones mínimas de presentación de informes para estudios que desarrollan o evalúan el rendimiento de un modelo de predicción en ciencias biomédicas. Entre otras cosas, contenía dos listas de chequeo, una para el desarrollo de modelos predictivos y otra para validación.

Desde entonces, los avances metodológicos en el campo de la predicción han incluido el uso generalizado de la inteligencia artificial (IA) mediante métodos de aprendizaje automático para desarrollar modelos de predicción.

Por lo tanto, fue necesaria una actualización de la declaración TRIPOD. TRIPOD+IA proporciona una guía armonizada para la presentación de informes sobre estudios de modelos de predicción, independientemente de su naturaleza. En la página web https://www.tripod-statement.org/ podrán encontrar información y guías para profundizar la temática.

Validar un modelo predictivo desarrollado implica evaluar su rendimiento. Para evaluar el rendimiento de un modelo de regresión logística, concretamente, son dos los aspectos principales que se deben valorar: la discriminación y la calibración. Como ya hemos visto, la discriminación de un modelo de regresión logística puede ser evaluada mediante el área bajo la curva ROC (AUC), ya que refleja la probabilidad de que, seleccionados al azar un par de individuos, uno con el evento y otro sin el evento, el modelo asigne la probabilidad más alta al individuo con el evento. Recordamos también que la prueba estadística que evalúa la calibración del modelo es el test de Hosmer-Lemeshow. Tiene el inconveniente, como todos los test frecuentistas, que ante tamaños muestrales grandes por pequeñas diferencias que aparezcan estas resultan estadísticamente significativas.

Al desarrollar un modelo, es evidente que se requiere cuantificar su rendimiento para poder determinar si es adecuado para su propósito o si es mejor que un modelo existente. El rendimiento aparente del modelo en este conjunto de entrenamiento será mejor que el rendimiento en otro conjunto de datos, incluso si el último conjunto de prueba consta de pacientes de la misma población. El optimismo de un modelo es la diferencia entre el desempeño que un modelo muestra en los datos con los que fue desarrollado y el desempeño que realmente tendrá cuando se aplique a datos nuevos de la misma población.

Lo podríamos expresar como:

$$Optimismo = Desempeo - Desempeo\ con\ nuevos\ datos$$

Supongamos un modelo de regresión logística para predecir mortalidad. En la muestra de desarrollo obtiene: $AUC = 0,91$

Al realizar la validación se obtiene: $AUC = 0,84$

Entonces: $Optimismo = 0.91 - 0.84 = 0.07$

Es decir, el modelo había sobreestimado su capacidad discriminativa en 0,07 unidades de AUC.

Validación y optimismo son conceptos que están estrechamente relacionados: el optimismo es la diferencia entre el rendimiento aparente y el esperado en datos nuevos. La validación busca estimar ese rendimiento esperado.

26.3. Descripción de los diferentes métodos de validación interna

Como se exponía anteriormente la validación interna es menos estricta, pues usa los datos de los sujetos a partir de los cuales se ha desarrollado el modelo predictivo. Las técnicas más usadas son:

- *Data splitting* o división de los datos
- *Cross-validation* o validación cruzada. Una extensión de las técnicas de *splitting* que reduce el sesgo y la variabilidad de la estimación del rendimiento.
- *Bootstrap validation*. Este método, como la validación cruzada, utiliza todos los datos empleados en el desarrollo del modelo. Además, nos permite cuantificar el "optimismo" del modelo de predicción final.

26.3.1. Data splitting

El método más simple de validación consiste en dividir de forma aleatoria la muestra original en dos submuestras, una donde se desarrolla el modelo (*training*) y otra donde se valida (*test*). El objetivo es estimar el desempeño predictivo sobre datos no utilizados durante el ajuste, reduciendo el riesgo de sobreajuste (*overfitting*).

La base de datos se divide aleatoriamente en dos subconjuntos independientes: un conjunto de entrenamiento (*training set*), utilizado para desarrollar y ajustar el modelo de regresión logística, y un conjunto de validación o prueba (*test set*), empleado exclusivamente para evaluar su capacidad predictiva. Habitualmente, entre el 70 % y el 80 % de las observaciones se destinan al entrenamiento y el 20 %–30 % restante a la validación, procurando mantener una proporción similar del desenlace en ambos conjuntos mediante un muestreo estratificado cuando la frecuencia del evento es baja. El desempeño del modelo en el conjunto de validación se evalúa mediante medidas de discriminación, calibración y medidas globales de desempeño.

Actualmente, las guías metodológicas para modelos predictivos (como TRIPOD y las recomendaciones de Harrell y Steyerberg) consideran que el *data splitting* es una estrategia menos eficiente que otros métodos de validación interna porque: reduce el tamaño efectivo de la muestra disponible para desarrollar el modelo; aumenta la variabilidad de las estimaciones, especialmente en muestras pequeñas o moderadas; los resultados dependen de una única partición aleatoria de los datos.

Por ello, cuando el tamaño muestral lo permite, suele recomendarse utilizar métodos de remuestreo, como la validación cruzada (*k-fold cross-validation*) o el *bootstrap*, que aprovechan toda la información disponible y proporcionan estimaciones más estables del desempeño del modelo.

En general para muestras grandes (>500–1000 eventos), el método de *data splitting* puede ser adecuado. En muestras pequeñas o moderadas se recomienda preferentemente *bootstrap* o validación cruzada, ya que ofrecen estimaciones más precisas y con menor sesgo del desempeño del modelo.

26.3.2. *Cross-validation* o validación cruzada

Es una extensión de las técnicas de *splitting* que reduce el sesgo y la variabilidad de la estimación del rendimiento.

Se desarrolla el modelo empleando todos los datos disponibles, posteriormente, se segmenta el conjunto de datos en dos grupos de igual tamaño. Se plantea el modelo en uno de ellos (*training*) y se evalúa el rendimiento en el restante (*test*). A continuación, se repite el proceso intercambiando la función de los subconjuntos. El rendimiento del modelo se obtiene como el promedio del rendimiento en cada subconjunto de validación. Este método de validación cruzada tiene su extensión en el **k-folds cross validation**. Consiste en dividir o segmentar el conjunto de datos en k grupos de igual (o similar) tamaño. Y se sigue la misma metodología que se ha explicado en el método de *cross-validation* básico. Se utilizan k-1 subconjuntos para el desarrollo del modelo y el subconjunto restante se utiliza como grupo de validación. Se repite este procedimiento k veces, hasta que cada uno de los subconjuntos ha sido utilizado una vez como grupo de validación. Finalmente, obtenemos la medida de rendimiento como el promedio de las k medidas calculadas.

26.3.3. Bootstrap

Las técnicas de validación *bootstrap* incluyen los siguientes pasos:

1. Desarrollar el modelo predictivo usando todos los datos disponibles y calcular el rendimiento aparente.
2. Generar n muestras bootstrap con reemplazo desde la muestra original y del mismo tamaño que esta.
3. Desarrollar un modelo predictivo en la primera muestra *bootstrap* aplicando las mismas estrategias empleadas en el desarrollo del modelo original.
4. Determinar el rendimiento aparente del modelo en la muestra *bootstrap*, lo que se conoce con el nombre de rendimiento *bootstrap*.
5. Determinar el rendimiento del modelo *bootstrap* en la muestra original, llamado test de rendimiento.
6. Calcular el optimismo como la diferencia entre el rendimiento *bootstrap* (paso 4) y el test de rendimiento (paso 5).
7. Repetir los pasos 4, 5 y 6 en cada una de las n muestras *bootstrap*, al menos 100 muestras.
8. Obtener la estimación del rendimiento ajustada por el optimismo restando al rendimiento aparente (calculado en el paso 1) la media del optimismo obtenido en las n muestras *bootstrap*.

La parte práctica la encontrarán en la video clase correspondiente a validación: allí realizaremos la validación del modelo que obtuvimos para el dataset de Titanic, usando un set de datos de testeo.

Parte V.

Unidad 5

27. Estudios de cohortes

27.1. Introducción

En su forma más básica, un diseño de cohorte implica la selección de un grupo de personas expuestas al factor de estudio y un grupo similar pero no expuesto, considerado como el grupo de referencia. Ambos grupos serán seguidos durante un período determinado y se evaluará si durante el mismo se produjo o no el evento de interés. Al final del seguimiento, se realiza la comparación de ambos grupos para analizar los resultados.

La esencia de los estudios de cohortes radica en el seguimiento de los participantes a lo largo del tiempo. Por tratarse de estudios longitudinales, los mismos podrán ser prospectivos, retrospectivos o mixtos. Además, las cohortes podrán ser fijas o dinámicas. En una cohorte fija, se establece un período de reclutamiento y, una vez finalizado, no se admiten más participantes. En una cohorte dinámica (también llamada cohorte abierta), se permite la entrada y salida de sujetos a lo largo de todo el estudio.

Como recordarán, la selección de los participantes se basa en la exposición, por lo cual la misma debe estar claramente definida. Como dijimos anteriormente, la idea clásica de un estudio de cohorte es comparar dos grupos, expuestos y no expuestos. Sin embargo, existen otras comparaciones posibles:

- **Estudios de comparación con la población general:** El estudio identifica y sigue a una cohorte expuesta, comparando la frecuencia del evento de salud en esta cohorte con la observada en la población general. Para ello, es esencial disponer de registros poblacionales que proporcionen esta información.
- **Comparaciones internas:** La cohorte estudiada incluye tanto personas expuestas como no expuestas, y las comparaciones se realizan dentro de la propia cohorte.

Por otra parte, el evento a observar puede tomar distintas formas: puede tratarse de un evento fijo, eventos múltiples, cambio en una medida de función (la cual se evalúa mediante tasa de cambio) o marcadores intermedios del evento. Estas variaciones también influyen en el abordaje analítico.

Al diseñar un estudio de cohortes, es fundamental definir claramente los siguientes aspectos:

- Criterios de inclusión
- Fechas de entrada y de salida de los participantes
- Estrategia de seguimiento
- Eventos finales de interés
- Definición de la exposición
- Identificación de factores de confusión
- Consideraciones éticas
- Estrategias de análisis
- Potencia estadística

En la siguiente tabla se remarcan ventajas y desventajas de estos estudios:

Ventajas	Desventajas
Más cercanos a un experimento	Alto costo y duración del estudio
Claridad temporal entre exposición y evento	Riesgo de pérdida de participantes
Permiten estudiar varios eventos asociados a la exposición	Necesidad de cohortes grandes
Estimación directa de la incidencia	Dificultad para controlar confusores
Reducción de sesgos de selección y de memoria	Cambios en la exposición durante el estudio
Eficientes para evaluar exposiciones poco frecuentes	No adecuados para enfermedades raras
Permiten fijar criterios de calidad en la medición del evento	

27.1.1. ¿Cómo analizar un estudio de cohortes?

Como recordarán de los cursos de epidemiología previos, una de las formas más simples de análisis en un estudio de cohortes es el cálculo del Riesgo Relativo (RR). Sin embargo, hay aspectos adicionales de este diseño que son importantes de considerar.

El análisis de los estudios de cohortes debe tener en cuenta la variable tiempo. La pérdida de participantes por diversas razones y la diferente duración del seguimiento entre los miembros de la cohorte requieren el uso de técnicas analíticas especiales que permitan aprovechar toda la información disponible. Existen dos estrategias de análisis principales para el estudio de cohortes: el análisis **persona-tiempo**, que desarrollaremos en esta unidad y el **análisis de supervivencia**, que retomaremos en la Unidad 6 cuando veamos estudios experimentales. Ambos métodos requieren información precisa sobre el inicio y final del seguimiento de cada participante, así como su estado al momento de finalizar o salir del estudio (desarrollo o no el evento de interés). Además, es necesario definir una escala temporal adecuada para el análisis.

Las técnicas de análisis **persona-tiempo** utilizan como unidad de análisis estratos o grupos homogéneos en los que se asume una tasa de incidencia uniforme. Las técnicas clásicas de análisis incluyen la comparación de tasas, la estandarización directa y las razones estandarizadas de incidencia o mortalidad (denominadas RIE o SIR, para la incidencia y RME o SMR para la mortalidad). El RIE y RME comparan el número de casos observados con el número esperado, teniendo en cuenta la frecuencia del evento en la población de referencia. La técnica multivariada por excelencia para la modelización de las tasas es la **regresión de Poisson**, que utiliza la distribución de Poisson para modelizar la variabilidad aleatoria del numerador de las tasas.

A modo de resumen, compartimos esta tabla, extraída del libro de Hernández-Ávila (2011):

Aspecto	Análisis de supervivencia	Análisis tiempo-persona
Tamaño de la muestra	Relativamente pequeño	Relativamente grande
Tipo de eventos	Frecuentes	Raros
Escala temporal	Única	Única o múltiple
Tipos de medida	Probabilidad (condicional y acumulada)	Tasa (densidad)
Análisis bivariado	Comparación de curvas de supervivencia	Comparación de tasas
	Log-rank test	Razón de tasas
	Razón de riesgos	REM
Análisis multivariado	Regresión de Cox	Regresión de Poisson

28. Regresión de Poisson

28.1. Introducción

La distribución de Poisson es una distribución de probabilidad que se utiliza para modelar situaciones en las que contamos eventos discretos (número de accidentes, personas que sufren un infarto, visitas a una consulta médica, número de hijos, etc.). Estas situaciones comparten la característica de involucrar números finitos, relativamente pequeños y siempre positivos. Los eventos ocurren dentro de un intervalo definido, que suele ser de tiempo, aunque también puede referirse a otros tipos de intervalos, como el tamaño de la población. En esencia, la distribución de Poisson busca modelar el número de veces en que ocurre un evento durante un intervalo determinado.

La distribución de Poisson toma valores enteros no negativos: 0, 1, 2, 3, 4, etc. Tiene un único parámetro, lambda (λ), que representa tanto la media como la varianza en la distribución. Es decir, que cuanto más grande es el valor esperado, mayor dispersión tienen los valores.

Veamos cómo se aplica la regresión de Poisson en un estudio de cohortes. Supongamos que tenemos un estudio de cohortes clásico con dos grupos de comparación: un grupo expuesto y otro no expuesto. Para cada grupo se dispone el número de eventos (d) y la cantidad de personas-tiempo seguidas (n). Como recordarán, la tasa de incidencia (λ), la podemos calcular como el cociente:

$$\lambda = \frac{d}{n}$$

Se asume que el número de eventos observados en cada grupo sigue una distribución de Poisson, con un valor esperado igual al producto de la tasa de incidencia por las personas-tiempo.

$$E(d) = \lambda * n$$

Así, la probabilidad de observar d eventos se calcula como:

$$P(x = d) = \frac{(\lambda n)^d e^{-\lambda n}}{d}$$

Omitiendo el desarrollo matemático, la regresión de Poisson consiste en establecer un modelo en el que diferentes covariables tienen un efecto lineal sobre el logaritmo de la tasa de incidencia del subgrupo correspondiente. Generalmente, el modelo de Poisson se expresa como:

$$ln\lambda = \alpha + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \qquad (*)$$

En forma equivalente, podemos expresar la tasa de incidencia como:

$$\lambda = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}$$

Siguiendo un razonamiento análogo al de la **regresión logística**, los parámetros del modelo pueden interpretarse como riesgos relativos (cocientes de tasas), representados como potencias de base e. Formalmente, el riesgo relativo entre dos niveles de exposición se expresa como:

$$RR_{x*/x} = e^{\sum_{i}^{k}=1\beta_i(x_i^*-x_i)} = \prod_{i=1}^{k} e^{\beta_i(x_i^*-x_i)}$$

Donde el símbolo $\prod$ (productoria) implica una secuencia de productos.

Volviendo a la ecuación de la tasa de incidencia:

$$\lambda = \frac{d}{n}$$

Si aplicamos logaritmo natural:

$$ln(\lambda) = ln\frac{d}{n} = ln(d) - ln(n)$$

Igualando con la ecuación $(*)$ tenemos:

$$ln\lambda = \alpha + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$
$$ln(d) = ln(n) + \alpha + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Al término $ln(n)$ se lo denomina ***offset***. En general, es un valor que debemos proporcionarle al *software* para ajustar un modelo de Poisson y no se estima a partir de los datos.

Con la ecuación del modelo establecida, el siguiente paso es estimar los coeficientes y evaluar la calidad del ajuste. Los coeficientes se calculan utilizando métodos como la Estimación de Máxima Verosimilitud (MLE). A partir de estos coeficientes, podemos realizar inferencias, como el test de Wald para evaluar la hipótesis nula ($H_0 : \beta_i = 0$), calcular los intervalos de confianza (IC), y obtener los riesgos relativos (RR) con sus respectivos IC. La bondad de ajuste del modelo se evaluará a través de la función Deviance siguiendo el mismo esquema jerárquico de evaluación que en el caso de la regresión logística.

28.2. Supuestos del modelo de Poisson

A modo de resumen: el modelo de Poisson se utiliza para modelar el conteo de eventos que ocurren en intervalos de tiempo, en poblaciones, o incluso en espacios geográficos. La variable respuesta en este modelo toma valores enteros y positivos, y depende de un solo parámetro (λ).

Los principales supuestos del modelo son:

1. **Independencia de las observaciones:** Cada observación debe ser independiente de las demás. Esto significa que la ocurrencia de un evento no debe influir en la probabilidad de que ocurran otros eventos.
2. **Constancia del parámetro λ a lo largo del tiempo:** Para que se cumpla este supuesto, la media y la varianza deben ser iguales (equidispersión). Si λ no es constante, el modelo puede no ser adecuado.
3. **Proporcionalidad de eventos al tamaño del intervalo:** La cantidad de eventos en un intervalo dado debe ser proporcional al tamaño del intervalo. Esto significa que si duplicamos el intervalo, esperaríamos aproximadamente el doble de eventos.

4. **Mutuamente excluyentes en el tiempo:** No pueden ocurrir dos o más eventos en el mismo instante puntual. Cada evento debe ser individual y ocurrir en un momento distinto.

Una característica fundamental de la distribución de Poisson es que la media y la varianza deben ser iguales, lo que se conoce como el supuesto de **equidispersión**. Cuando la varianza observada en los datos excede la varianza esperada bajo este modelo, se enfrenta a un problema de **sobredispersión**. Un método común para detectar sobredispersión es calcular el **coeficiente de variación** (CV), definido como el cociente entre la varianza y la media estimadas. La sobredispersión es frecuente en la práctica y puede afectar la precisión de las estimaciones de los errores estándar de los coeficientes, por lo que es fundamental diagnosticarla y tratarla adecuadamente.

Otra situación en la que el modelo de Poisson puede no ser adecuado es cuando hay un número excesivo de ceros en los datos, es decir, cuando la frecuencia observada de ceros es mayor de lo que predice el modelo. Esto se debe a que el $ln(0)$ no está definido. De manera particular, es posible que el mecanismo aleatorio que dio origen a los datos de conteo muestre una mayor concentración para algún valor específico, que puede ser el cero (como ocurre con algunas variables vinculadas a salud) o cualquier otro valor positivo. Esto implica que dicho valor tiene una mayor probabilidad de ocurrencia que la especificada por la distribución Poisson o cualquier otra distribución.

En el **siguiente capítulo** de esta unidad, exploraremos cómo detectar y manejar tanto la sobredispersión de los datos como el exceso de ceros, para asegurar que el modelo elegido sea el más adecuado para los datos analizados.

28.3. Construcción de un modelo de Poisson en R

La regresión de Poisson está implementada en R como parte de la familia de **Modelos Lineales Generalizados (GLM)**. Esta técnica permite modelar variables de respuesta que representan conteos, asegurando que los valores predichos permanezcan dentro de límites razonables.

Un criterio importante en la selección de la función de enlace dentro de los GLM es garantizar que los valores ajustados del modelo sean coherentes con la naturaleza de la variable respuesta. Para la distribución de Poisson, la función de enlace predeterminada es el **logaritmo** natural. Esta elección asegura que los recuentos ajustados sean siempre mayores o iguales a cero, lo cual es esencial en el contexto de datos de conteo.

En la práctica, esto significa que el modelo ajustado con errores de Poisson y una función de enlace logarítmica linealiza la relación entre la variable respuesta (conteos) y las variables independientes, permitiendo una interpretación clara y coherente de los resultados.

Como es de imaginar utilizaremos la misma función general `glm()`, cambiando los argumentos en familia y enlace. La sintaxis básica de esta función, contenida en el paquete `stats` de R, es:

```
glm(formula, family = poisson(link = "log"), data = datos)
```

Dado que el enlace logarítmico es el predeterminado para la familia de Poisson, podemos omitir su especificación:

```
glm(formula, family = poisson, data = datos)
```

donde:

- `formula`: al igual que en los casos anteriores, describe la fórmula del modelo a ajustar con la estructura:

$$variable\_dependiente \sim variable\_indepen_1 + variable\_indepen_2 + \cdots + variable\_indepen_n$$

- `family`: indica la familia de distribuciones utilizadas y su función de enlace.
- `data`: especifica el nombre de la base de datos (*dataframe*) que contiene las variables del modelo.

Para obtener un resumen de los resultados del modelo se utiliza la función `summary()`:

```
summary(modelo)
```

El resumen del objeto de regresión de Poisson incluye:

- `Call`: fórmula del modelo
- `Deviance Residuals`: muestra la distribución de los residuos (mediana, mínimo, máximo y percentilos 25-75) obtenidos en la última iteración
- `Coefficients`: Incluye los coeficientes del intercepto y de las variables independientes, junto con los errores estándar, el valor z (estadístico de Wald) y el p-valor correspondiente.
- `Dispersion parameter for poisson family taken to be 1`: indica que el modelo asume el supuesto de equidispersión (media igual a varianza).
- `Null deviance`: devianza del modelo nulo, que solo incluye al intercepto.
- `Residual deviance`: devianza del modelo ajustado.
- `AIC`: criterio de información de Akaike.
- `Number of Fisher Scoring iterations`: cantidad de iteraciones realizadas.

El objeto de regresión creado con `glm()` pertenece a las clases `"glm"` y `"lm"` y está compuesto por varios componentes que pueden ser accedidos usando el nombre del modelo seguido del signo $. Algunos de los componentes más relevantes son:

- `modelo$coefficients`: Vector de coeficientes del modelo, también accesible mediante `coef(modelo)`.
- `modelo$residuals`: Vector de residuos obtenidos en la última iteración.
- `modelo$fitted.values`: Valores ajustados medios, obtenidos mediante la transformación de los predictores lineales usando la inversa de la función de enlace.
- `modelo$family`: Familia de distribuciones utilizada en la construcción del modelo.
- `modelo$deviance`: Devianza del modelo ajustado.
- `modelo$aic`: Criterio de información de Akaike (AIC).
- `modelo$null.deviance`: Devianza del modelo nulo.

28.4. Ejemplo práctico en R

Para ilustrar las funciones y la metodología de análisis en R, utilizaremos el archivo de datos "`cohorte_ocupacional.txt`", que contiene resultados de un estudio de cohortes que investiga la asociación entre las muertes respiratorias y la exposición al arsénico en la industria, ajustando por varios otros factores de riesgo.

Comenzaremos activando los paquetes necesarios:

```r
# chequeo de supuestos
library(easystats)

# tablas regresión
library(gtsummary)

# manejo de datos
library(tidyverse)
```

Cargamos la base de datos y exploramos su estructura:

```r
### Carga datos
datos <- read_csv2("datos/cohorte_ocupacional.txt")

### Explora datos
glimpse(datos)
```

```
Rows: 114
Columns: 6
$ muertes      <dbl> 2, 0, 0, 0, 2, 1, 0, 0, 0, 1, 1, 0, 2, 1, 0, 0, 7, 0, 0, ~
$ persona_anio <dbl> 30753, 4858, 4782, 3373, 39816, 6561, 1903, 125, 9368, 19~
$ grupo_edad   <chr> "40-49", "40-49", "40-49", "40-49", "40-49", "40-49", "40~
$ periodo      <chr> "1938-1949", "1938-1949", "1938-1949", "1938-1949", "1938~
$ comienzo     <chr> "1925 y post", "1925 y post", "1925 y post", "1925 y post~
$ arsenico     <chr> "<1 año", "1-4 años", "5-14 años", "15+ años", "<1 año", ~
```

Las variables de estudio son:

- `muertes`: Número de muertes por persona-años (`persona_anio`) en cada categoría. Nuestra variable de interés es la tasa de incidencia de mortalidad.
- `grupo_edad`: Grupo de edad de los sujetos.
- `periodo`: Período de empleo de los sujetos.
- `comienzo`: Año de inicio del empleo.
- `arsenico`: Nivel de exposición al arsénico durante el período de estudio.

Las variables `grupo_edad`, `periodo`, `comienzo` y `arsenico` son de tipo carácter y debemos convertirlas a factor. Podemos hacer esto manualmente para cada variable, o usar el comando `across()` que permite modificar varias columnas a la vez:

```
datos <- datos |>
  mutate(across(.cols = grupo_edad:arsenico, .fns = ~ as.factor(.x)))
```

Revisemos como quedaron los niveles de cada factor:

```
# grupo etario
levels(datos$grupo_edad)
```

```
[1] "40-49" "50-59" "60-69" "70-79"
```

```
# periodo
levels(datos$periodo)
```

```
[1] "1938-1949" "1950-1959" "1960-1969" "1970-1977"
```

```
# comienzo
levels(datos$comienzo)
```

```
[1] "< 1925"      "1925 y post"
```

```
# arsenico
levels(datos$arsenico)
```

```
[1] "<1 año"    "1-4 años"  "15+ años"  "5-14 años"
```

Ahora vamos a explorar los datos de la base organizando la información de persona-años por edad y período:

```
## Creo tabla
datos |>
  count(periodo, grupo_edad, wt = persona_anio) |>
  pivot_wider(names_from = grupo_edad, values_from = n) |>

  # formato de tabla
  tibble()
```

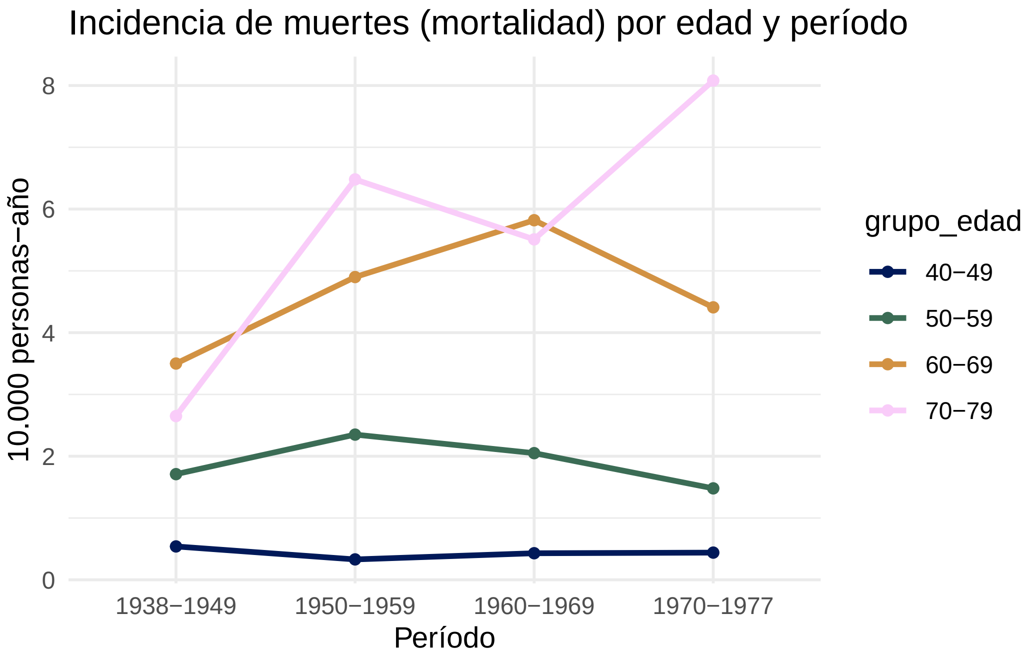
```
# A tibble: 4 x 5
  periodo   `40-49` `50-59` `60-69` `70-79`
  <fct>       <dbl>   <dbl>   <dbl>   <dbl>
1 1938-1949   92171   64211   40056   15072
2 1950-1959  149493  102234   48960   18511
3 1960-1969  161234  136627   75552   27240
4 1970-1977   90730  115040   79366   33410
```

Ahora hacemos lo mismo para el número de muertes y calculamos la incidencia por 10000 personas-año para cada celda:

```r
## Creo objeto para personas-años
personas_anios <- datos |>
  count(periodo, grupo_edad, wt = persona_anio, name = "p.a")

### Creo tabla
datos |>
  count(periodo, grupo_edad, wt = muertes) |>
  left_join(personas_anios) |>
  mutate(incidencia = round(n / p.a * 10000, 2)) |>
  select(-n, -p.a) |>

  tibble()
```

```
# A tibble: 16 x 3
   periodo   grupo_edad incidencia
   <fct>     <fct>           <dbl>
 1 1938-1949 40-49            0.54
 2 1938-1949 50-59            1.71
 3 1938-1949 60-69            3.5
 4 1938-1949 70-79            2.65
 5 1950-1959 40-49            0.33
 6 1950-1959 50-59            2.35
 7 1950-1959 60-69            4.9
 8 1950-1959 70-79            6.48
 9 1960-1969 40-49            0.43
10 1960-1969 50-59            2.05
11 1960-1969 60-69            5.82
12 1960-1969 70-79            5.51
13 1970-1977 40-49            0.44
14 1970-1977 50-59            1.48
15 1970-1977 60-69            4.41
16 1970-1977 70-79            8.08
```

Podemos visualizar la incidencia de muertes ajustadas por periodo y grupo de edad mediante un gráfico. Aunque el código exacto se omite por razones de extensión, los interesados pueden solicitarlo al equipo docente.



En el gráfico anterior, podemos observar que el grupo de mayor edad tiende a tener una incidencia más alta, lo que es consistente con la expectativa de que el riesgo de muerte aumenta con la edad. Sin embargo, para obtener una comprensión más precisa de esta relación, es crucial ajustar por otras covariables. La regresión de Poisson nos permitirá examinar cómo la exposición al arsénico y otros factores afectan la mortalidad, controlando simultáneamente por la edad y otros posibles factores de confusión.

28.4.1. *Offset*

Dado que en este ejemplo nos interesa modelar la tasa de incidencia vamos a necesitar incorporar dentro del modelo un termino de desplazamiento (en inglés *"offset"*).

Creamos el modelo saturado para trazar iteraciones *backward*:

```r
modelo <- glm(
  muertes ~ periodo + grupo_edad + arsenico,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)
```

El término `offset = log(persona_anio)` en el modelo de regresión de Poisson permite que la variable `persona_anio` actúe como el denominador en el cálculo de las tasas de mortalidad. La transformación logarítmica es necesaria porque la función de enlace del modelo de Poisson es el logaritmo.

Para obtener un resumen del modelo ajustado, utilizamos la función `summary()`, que proporciona una visión detallada de los resultados del modelo:

```r
summary(modelo)
```

```
Call:
glm(formula = muertes ~ periodo + grupo_edad + arsenico, family = poisson,
    data = datos, offset = log(persona_anio))

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)       -10.6363     0.2756 -38.594  < 2e-16 ***
periodo1950-1959    0.4087     0.2126   1.923   0.0545 .
periodo1960-1969    0.4413     0.2022   2.183   0.0290 *
periodo1970-1977    0.3181     0.2063   1.542   0.1231
grupo_edad50-59     1.4565     0.2458   5.925 3.12e-09 ***
grupo_edad60-69     2.3527     0.2392   9.837  < 2e-16 ***
grupo_edad70-79     2.6024     0.2578  10.096  < 2e-16 ***
arsenico1-4 años    0.7949     0.1582   5.024 5.06e-07 ***
arsenico15+ años    1.0468     0.1775   5.896 3.72e-09 ***
arsenico5-14 años   0.6070     0.2059   2.948   0.0032 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 376.02  on 113  degrees of freedom
Residual deviance: 116.69  on 104  degrees of freedom
AIC: 355.47

Number of Fisher Scoring iterations: 5
```

Este primer modelo, que incluye todas las covariables, muestra que casi todos los niveles de las variables son significativos respecto a las categorías de referencia.

28.4.2. Bondad de ajuste

Para evaluar la bondad de ajuste del modelo y verificar la presencia de sobredispersión, podemos utilizar la función `check_overdispersion()` del paquete `performance`. Esta función ayuda a identificar si los errores del modelo presentan una dispersión excesiva en comparación con la esperada bajo la distribución de Poisson:

```r
check_overdispersion(modelo)
```

```
# Overdispersion test

       dispersion ratio =   1.116
  Pearson's Chi-Squared = 116.079
                p-value =   0.197
```

El componente de `Pearson's Chi-Squared` se calcula a partir de la devianza del modelo. En las pruebas de bondad de ajuste, buscamos que el valor de *p* sea mayor a 0,05, lo cual indica que el modelo se ajusta adecuadamente a los datos.

28.4.3. Selección de variables explicativas

Para mejorar el modelo anterior y asegurarnos de que incluimos solo las variables significativas, procederemos a eliminar variables una por una en una primera ronda de iteración mediante un proceso *backwards* manual.

```r
# (-) arsénico
mod1 <- glm(
  muertes ~ periodo + grupo_edad,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)

# (-) grupo etario
mod2 <- glm(
  muertes ~ periodo + arsenico,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)

# (-) periodo
mod3 <- glm(
  muertes ~ grupo_edad + arsenico,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)
```

Para comparar los modelos podemos usar la función `compare_perfomance()` del paquete `performance`.

```r
compare_performance(mod1, mod2, mod3, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name | Model | AIC (weights) | BIC (weights) | Nagelkerke's R2 |  RMSE
----------------------------------------------------------------------
mod1 |   glm | 396.6 (<.001) | 415.8 (<.001) |           0.877 | 2.208
mod2 |   glm | 534.7 (<.001) | 553.9 (<.001) |           0.496 | 3.470
mod3 |   glm | 355.0 (>.999) | 374.2 (>.999) |           0.926 | 1.595
```

Buscamos el menor AIC, que en este caso es `mod3` (modelo con variables `grupo_edad` y `arsenico`).

En la segunda fase de iteración partimos de este modelo y quitamos otra variable, lo que signica en nuestro ejemplo construir regresiones simples:

```
# (-) arsénico
mod4.1 <- glm(
  muertes ~ grupo_edad,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)

# (-) grupo etario
mod4.2 <- glm(
  muertes ~ arsenico,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)
```

Volvemos a comparar:

```
compare_performance(mod3, mod4.1, mod4.2, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC (weights) | BIC (weights) | Nagelkerke's R2 |  RMSE
-------------------------------------------------------------------------
mod3   |   glm | 355.0 (>.999) | 374.2 (>.999) |           0.926 | 1.595
mod4.1 |   glm | 394.5 (<.001) | 405.4 (<.001) |           0.871 | 2.181
mod4.2 |   glm | 537.9 (<.001) | 548.8 (<.001) |           0.451 | 3.416
```

El mejor modelo continúa siendo `mod3`, por lo que vamos a probar su bondad de ajuste:

```
check_overdispersion(mod3)
```

```
# Overdispersion test

       dispersion ratio =   1.128
  Pearson's Chi-Squared = 120.713
                p-value =   0.172
```

El *p* valor del test confirma un buen ajuste. Procedemos a chequear significancia de las variables explicativas:

```
summary(mod3)
```

```
Call:
glm(formula = muertes ~ grupo_edad + arsenico, family = poisson,
    data = datos, offset = log(persona_anio))
```

Characteristic	**IRR**	**95 % CI**	**p-value**
grupo_edad			
40-49	—	—	
50-59	4.32	2.72, 7.16	<0.001
60-69	10.5	6.73, 17.2	<0.001
70-79	13.4	8.27, 22.7	<0.001
arsenico			
<1 año	—	—	
1-4 años	2.23	1.63, 3.02	<0.001
15+ años	2.71	1.90, 3.79	<0.001
5-14 años	1.81	1.19, 2.67	0.004

Abbreviations: CI = Confidence Interval, IRR = Incidence Rate Ratio

```
Coefficients:
                    Estimate Std. Error z value Pr(>|z|)    
(Intercept)         -10.2972     0.2237 -46.032  < 2e-16 ***
grupo_edad50-59       1.4624     0.2454   5.960 2.53e-09 ***
grupo_edad60-69       2.3502     0.2380   9.873  < 2e-16 ***
grupo_edad70-79       2.5987     0.2564  10.136  < 2e-16 ***
arsenico1-4 años      0.8041     0.1577   5.099 3.42e-07 ***
arsenico15+ años      0.9976     0.1758   5.673 1.40e-08 ***
arsenico5-14 años     0.5957     0.2059   2.893  0.00381 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 376.02  on 113  degrees of freedom
Residual deviance: 122.25  on 107  degrees of freedom
AIC: 355.04

Number of Fisher Scoring iterations: 5
```

Todos los niveles de las variables son significativos respecto a sus niveles de referencia:

```r
tbl_regression(mod3, exponentiate = T)
```

Traducidos a riesgo, mediante la exponenciación de los coeficientes, observamos que los diferentes categorías de años de exposición al arsénico no varían entre sí. Esto nos hace pensar que puede valer la pena agruparlo en solo dos niveles:

```r
## Recategoriza arsénico
datos <- datos %>%
  mutate(
    arsenico_cat = if_else(
      arsenico == "<1 año",
      "<1 año",
      "1+ años"
    ) |>
      as.factor()
  )

## Modelo con arsénico recategorizado
mod5 <- glm(
  muertes ~ grupo_edad + arsenico_cat,
  offset = log(persona_anio),
  family = poisson,
  data = datos
)

## Compara modelos
compare_performance(mod3, mod5, metrics = "common")
```

```
# Comparison of Model Performance Indices

Name | Model | AIC (weights) | BIC (weights) | Nagelkerke's R2 |  RMSE
-----------------------------------------------------------------------
mod3 |   glm | 355.0 (0.350) | 374.2 (0.034) |           0.926 | 1.595
mod5 |   glm | 353.8 (0.650) | 367.5 (0.966) |           0.924 | 1.613
```

El `mod5` tiene mejor *performance* que el modelo anterior. Chequeamos sobredispersión:

```
check_overdispersion(mod5)
```

```
# Overdispersion test

       dispersion ratio =   1.147
  Pearson's Chi-Squared = 125.038
                p-value =    0.14
```

El ajuste continúa siendo bueno.

Chequeamos significancia:

```
summary(mod5)
```

```


Call:
glm(formula = muertes ~ grupo_edad + arsenico_cat, family = poisson,
    data = datos, offset = log(persona_anio))

Coefficients:
                    Estimate Std. Error z value Pr(>|z|)
(Intercept)         -10.3112     0.2233 -46.169  < 2e-16 ***
```

Characteristic	**IRR**	**95 % CI**	**p-value**
grupo_edad			
40-49	—	—	
50-59	4.35	2.74, 7.21	<0.001
60-69	10.7	6.85, 17.4	<0.001
70-79	13.8	8.51, 23.2	<0.001
arsenico_cat			
<1 año	—	—	
1+ años	2.25	1.77, 2.85	<0.001

Abbreviations: CI = Confidence Interval, IRR = Incidence Rate Ratio

```
grupo_edad50-59      1.4702     0.2453   5.994 2.04e-09 ***
grupo_edad60-69      2.3661     0.2372   9.976  < 2e-16 ***
grupo_edad70-79      2.6238     0.2548  10.297  < 2e-16 ***
arsenico_cat1+ años  0.8109     0.1210   6.699 2.09e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 376.02  on 113  degrees of freedom
Residual deviance: 125.02  on 109  degrees of freedom
AIC: 353.8

Number of Fisher Scoring iterations: 5
```

Las variables incluídas son significativas.

En esta etapa, aceptaríamos el `mod5` como el mejor modelo, ya que tiene el AIC más pequeño entre todos los modelos que hemos probado.

```
tbl_regression(mod5, exponentiate = T)
```

28.4.4. Interpretación de los resultados

Concluimos que las personas expuestas al arsénico durante al menos un año tienen un riesgo 2,25 veces mayor de muerte por enfermedad respiratoria en comparación con aquellas no expuestas. Para entender cómo se obtiene este cálculo, consideremos que en los modelos de Poisson, la estructura de compensación (*offset*) definida como `log(persona_anio)` transforma el resultado en el logaritmo de la densidad de incidencia.

El término `offset = log(persona_anio)` en el modelo de Poisson convierte el recuento de eventos en una densidad de incidencia, ajustada por el tiempo de seguimiento. Esto permite que el modelo estime tasas de incidencia de acuerdo con el tiempo de exposición.

Los coeficientes del modelo de Poisson se interpretan en términos de tasas de incidencia. En particular, el coeficiente asociado a la variable de exposición al arsénico se transforma mediante la exponenciación para obtener el riesgo relativo. Si el coeficiente de la variable `arsenico` es β, el riesgo relativo (RR) se calcula como e^{β}. En nuestro caso, si $e^{\beta} = 2,25$, esto indica que las personas expuestas al arsénico durante al menos un año tienen un riesgo 2,25 veces mayor de muerte por enfermedad respiratoria en comparación con las no expuestas.

28.4.5. Densidad de incidencia

Para calcular la densidad de incidencia esperada en una población específica, utilizamos el modelo ajustado. Por ejemplo, para una población de 100.000 personas de entre 40 y 49 años expuestas al arsénico durante menos de un año, podemos usar el modelo `mod5`. En R, esto se hace creando un nuevo conjunto de datos con las características deseadas y utilizando la función `predict()` para obtener la tasa de incidencia ajustada:

```r
## Genera nuevos datos
newdata <- tibble(
  grupo_edad = "40-49",
  arsenico_cat = "<1 año",
  persona_anio = 100000
)

## Predice datos
predict(object = mod5, newdata = newdata, type = "response")
```

```
       1
3.325736
```

Esta población tendría una densidad de incidencia estimada de 33,26 por 100.000 personas-año.

28.4.6. Riesgo relativo

En un estudio de casos y controles, como los vistos en la **Unidad 4**, el OR se utiliza para comparar la prevalencia de exposición entre casos y controles. En un estudio de cohortes, este valor es igual a la relación entre las probabilidades de contraer una enfermedad entre el grupo expuesto y el no expuesto. La razón de los riesgos para los dos grupos se denomina entonces “razón de riesgo” o **riesgo relativo** (**RR**).

En un estudio de cohorte real, los sujetos no siempre tienen la misma duración de seguimiento. El riesgo relativo ignora la duración del seguimiento. Por tanto, no es una buena medida de comparación del riesgo entre los dos grupos.

28.4.7. Razón de densidad de incidencia

En este ejercicio, todos los sujetos agrupan sus tiempos de seguimiento y este número se denomina “tiempo-persona”, que luego se utiliza como denominador del evento, lo que da como resultado una “densidad de incidencia”.

Comparar la densidad de incidencia entre dos grupos de sujetos por su estado de exposición es más justo que comparar los riesgos crudos. La relación entre las densidades de incidencia de dos grupos se denomina **razón de densidades de incidencia** (**RDI**), que es una forma mejorada de riesgo relativo.

En el `mod5`, para calcular la razón de densidad de incidencia entre los sujetos expuestos al arsénico durante uno o más años frente a los expuestos durante menos de un año, podemos dividir la incidencia entre los primeros por la del segundo grupo.

Characteristic	**IRR**	**95 % CI**	**p-value**
arsenico_cat			
<1 año	—	—	
1+ años	2.25	1.77, 2.85	<0.001

Abbreviations: CI = Confidence Interval, IRR = Incidence Rate Ratio

```r
## Genera nuevos datos
newdata <- tibble(
  grupo_edad = rep("40-49", 2),
  arsenico_cat = c("<1 año", "1+ años"),
  persona_anio = rep(100000, 2)
)

## Densidad de incidencia
di <- predict(object = mod5, newdata = newdata, type = "response")

## Razón densidad de incidencia
rdi.arsenico <- di[2] / di[1]

rdi.arsenico
```

```
       2
2.249864
```

El código anterior comienza agregando una nueva fila al *dataframe* `newdata` que tiene lo mismo que la primera fila, excepto que la variable `arsenico_cat` es **"1+ años"**.

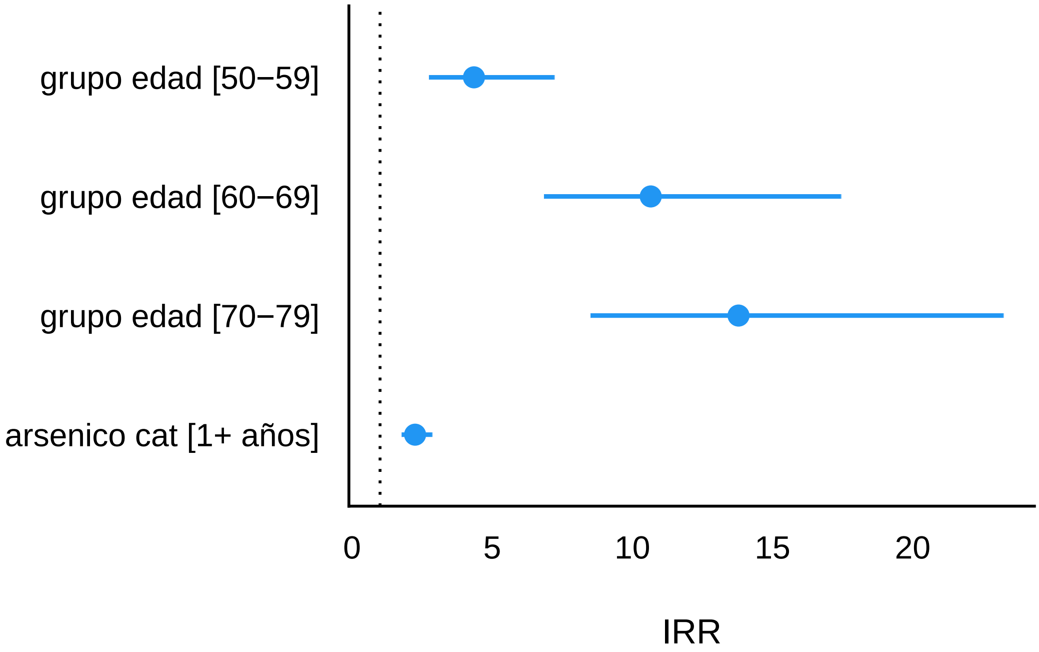
A continuación, se calculan las respuestas o densidades de incidencia de las dos condiciones.

La RDI se obtiene luego de la división de las densidades de incidencia para arsenico_cat = "<1 año" con `arsenico_cat = "1+ años"`. Una forma más corta de obtenerla es mediante el coeficiente de la variable específica `arsenico`, utilizando R `base` o el paquete `gtsummary`.

```r
tbl_regression(mod5, exponentiate = T, include = "arsenico_cat")
```

También podemos mostrar la salida mediante un gráfico usando el ecosistema de paquetes `easystats`, tal como vimos en la **unidad anterior**:

```r
model_parameters(mod5, exponentiate = T) |>
  plot()
```


grupo edad [50−59]
grupo edad [60−69]
grupo edad [70−79]
arsenico cat [1+ años]
0
5
10
15
20
IRR

29. Dispersión de datos

29.1. Introducción

En modelos de regresión Poisson se parte del supuesto de **equidispersión**, lo cual implica que la varianza de la variable dependiente coincide con su media esperada:

$$var(Y) = \mu$$

Para evaluar si un modelo cumple con este supuesto, se suele analizar el cociente entre la devianza residual y los grados de libertad residuales. Si el modelo ajusta adecuadamente con la distribución elegida, este valor debería rondar la unidad.

Sin embargo, en la práctica, esta suposición frecuentemente no se cumple. Cuando la varianza observada supera a la media se habla de **sobredispersión**, mientras que si es menor se denomina **subdispersión**; esta relación se puede expresar formalmente incorporando un parámetro de dispersión (θ).

Adicionalmente, los recuentos pueden contener un exceso de ceros estructurales que el modelo de Poisson tradicional no logra ajustar adecuadamente. Ignorar estos fenómenos subestima los errores estándar y conduce a inferencias incorrectas, por lo que resulta indispensable diagnosticarlos y controlarlos mediante alternativas analíticas robustas.

29.2. Sobredispersión

La sobredispersión es un fenómeno muy común en los datos de conteo que puede ser causado por diversos factores, tales como la heterogeneidad no observada, la correlación entre observaciones o una alta incidencia de valores extremos. Este problema tiende a inflar los errores estándar, lo que conduce sistemáticamente a una subestimación de la significancia estadística. Matemáticamente, la condición de sobredispersión se expresa como:

$$Var(Y) > \mu$$

Para detectar si un modelo tiene sobredispersión, se puede calcular el **ratio de dispersión**, que es el cociente entre la devianza del modelo y los grados de libertad. Un ratio significativamente mayor que 1 indica sobredispersión.

En el entorno de R, la función `check_overdispersion()` del paquete `performance` permite evaluar este cociente de forma directa en un modelo ajustado. Como se vió en el **ejemplo de regresión Poisson**, cuando el cociente del modelo final es cercano a 1, se descarta la sobredispersión y se concluye que la distribución seleccionada ajusta adecuadamente los datos.

29.2.1. Ejemplo en R

Analizaremos un caso práctico basado en los registros de casos de dengue reportados en los departamentos de la Costa Atlántica de la provincia de Buenos Aires (PBA)[1] entre las semanas epidemiológicas 1 y 20 de 2024[2]. Estos registros se encuentran almacenados en el archivo "dengue_costa.txt".

Comenzaremos por cargar los paquetes necesarios para el análisis:

```
# Ajuste de modelos
library(glmmTMB)

# Chequeo de supuestos
library(easystats)
library(DHARMa)

# Coeficientes
library(gtsummary)

# Manejo de datos
library(janitor)
library(tidyverse)
```

Una vez cargadas las librerías, importamos los datos utilizando funciones del ecosistema `tidyverse` y exploramos su estructura general:

```
# Carga datos
datos <- read_delim("datos/dengue_costa.txt")

# Explora datos
glimpse(datos)
```

```
Rows: 1,047
Columns: 6
$ provincia      <chr> "Buenos Aires", "Buenos Aires", "Buenos Aires", "Buenos~
$ departamento   <chr> "BAHIA BLANCA", "BAHIA BLANCA", "BAHIA BLANCA", "BAHIA ~
$ periodo        <dbl> 2024, 2024, 2024, 2024, 2024, 2024, 2024, 2024, 2024, 2~
$ semana_epi     <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4~
$ grupo_edad_cat <chr> "0-9 años", "10-19 años", "20-34 años", "35-65 años", "~
$ casos          <dbl> 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0~
```

Las variables de interés incluidas en la base son las siguientes:

- `semana_epi`: semana epidemiológica en la que se reportaron los casos.
- `grupo_edad_cat`: grupo etario de los pacientes afectados.
- `casos`: número de casos reportados por departamento, semana epidemiológica y grupo etario (variable dependiente).

---

[1] Fuente: https://es.wikipedia.org/wiki/Localidades_balnearias_del_mar_Argentino

[2] Fuente: Vigilancia de las enfermedades por virus del Dengue y Zika. Datos Abiertos del Ministerio de Salud. Disponible en: http://datos.salud.gob.ar/dataset/vigilancia-de-dengue-y-zika

A continuación, ajustamos un modelo de regresión de Poisson incorporando todas las variables explicativas de interés:

```
fit_pois <- glm(
  casos ~ semana_epi + grupo_edad_cat,
  data = datos,
  family = poisson
)

# Salida del modelo
summary(fit_pois)
```

```

Call:
glm(formula = casos ~ semana_epi + grupo_edad_cat, family = poisson,
    data = datos)

Coefficients:
                          Estimate Std. Error z value Pr(>|z|)
(Intercept)              -3.092154   0.279663 -11.057  < 2e-16 ***
semana_epi                0.044249   0.008993   4.920 8.65e-07 ***
grupo_edad_cat10-19 años  1.172838   0.294392   3.984 6.78e-05 ***
grupo_edad_cat20-34 años  2.195273   0.271054   8.099 5.54e-16 ***
grupo_edad_cat35-65 años  2.312218   0.269545   8.578  < 2e-16 ***
grupo_edad_cat65+ años    0.730874   0.314523   2.324   0.0201 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 1436.5  on 1046  degrees of freedom
Residual deviance: 1187.9  on 1041  degrees of freedom
AIC: 1735.1

Number of Fisher Scoring iterations: 6
```

Cabe destacar que el modelo ajustado no incluye un término *offset*, ya que este recurso se utiliza en situaciones donde es necesario ajustar la tasa de eventos en función de una variable de tiempo o de tamaño poblacional. Para los datos del ejemplo, podemos asumir que la población de cada departamento permanece constante durante el período estudiado; además, dado que una misma persona puede contraer la infección por el virus del dengue en más de una ocasión, no se requiere un término de ajuste adicional por sujeto.

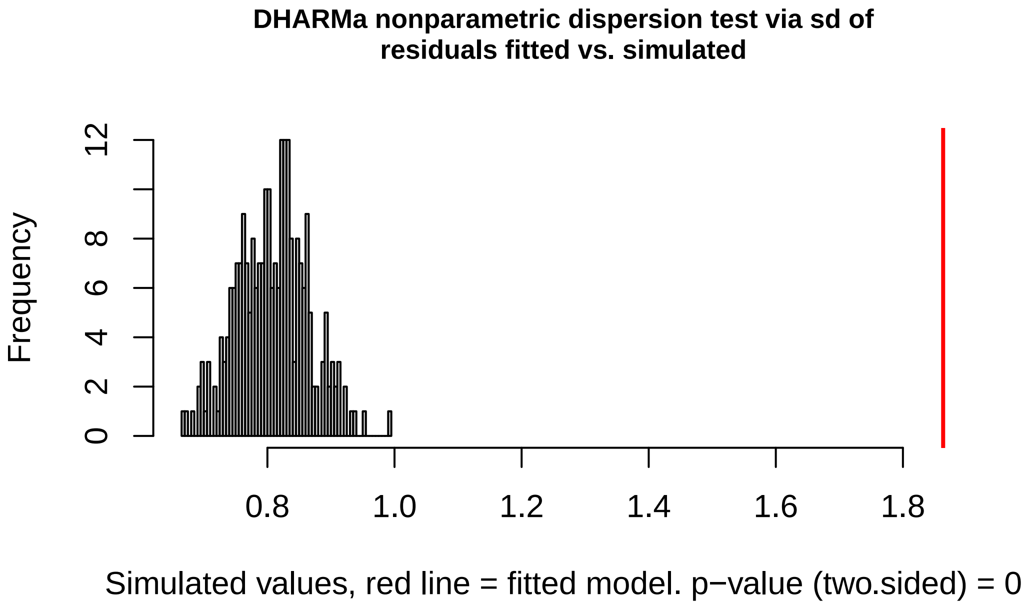
Para finalizar, evaluamos de forma empírica la presencia de sobredispersión :

```
check_overdispersion(fit_pois)
```

```
# Overdispersion test

       dispersion ratio =    1.955
  Pearson's Chi-Squared = 2035.204
                p-value =  < 0.001
```

Otra forma de chequear sobredispersión es utilizando la función `testDispersion()` del paquete DHARMa (Hartig 2026), que compara la varianza de los valores observados con la de residuales simulados por bootstrap:

```
testDispersion(fit_pois)
```



```
	DHARMa nonparametric dispersion test via sd of residuals fitted vs.
	simulated

data:  simulationOutput
dispersion = 2.3072, p-value < 2.2e-16
alternative hypothesis: two.sided
```

El cociente de dispersión alcanza un valor de 1.96. Dado que este valor supera ampliamente la unidad, se confirma la existencia de sobredispersión, lo que evidencia que el modelo de Poisson no resulta adecuado para representar la variabilidad de estos datos.

29.3. Subdispersión

La subdispersión, aunque menos frecuente que la sobredispersión, puede ocurrir en conjuntos de datos de conteo donde las observaciones están fuertemente controladas o sujetas a restricciones estructurales. Este fenómeno provoca que la variabilidad observada sea menor que la esperada bajo el modelo teórico, lo que a su vez puede sesgar los errores estándar y llevar a una sobreestimación de la significancia estadística. Matemáticamente, esta condición se expresa de la siguiente manera:

$$Var(Y) < \mu$$

De manera similar a lo que ocurre con la sobredispersión, para diagnosticar este comportamiento se emplea el *ratio* de dispersión. Un valor de este cociente que resulte significativamente menor que 1 constituye un indicador clave de la presencia de subdispersión en el modelo ajustado.

29.4. Control de sobredispersión y subdispersión

29.4.1. Modelos *quasi*-Poisson

Una forma común de manejar la sobredispersión en los datos de conteo consiste en el uso de modelos *quasi*-Poisson. Estos modelos ajustan los errores estándar de los coeficientes sin modificar la media predicha, incorporando un parámetro adicional que permite que la varianza difiera de la media. Aunque resultan útiles para corregir la subestimación de la variabilidad, presentan importantes limitaciones: no abordan las causas estructurales subyacentes de la sobredispersión, no son adecuados para manejar escenarios con subdispersión o un exceso de ceros, y su estimación de parámetros tiende a ser menos eficiente al asumir que la sobredispersión es constante en toda la muestra. Debido a estas restricciones metodológicas, no profundizaremos en su uso.

29.4.2. Distribución binomial negativa

Dentro del marco de los modelos lineales generalizados (GLM), la distribución binomial negativa representa una de las herramientas más robustas y difundidas para corregir la sobredispersión en datos discretos. Esta parametrización aporta una gran flexibilidad analítica, mejora las inferencias estadísticas mediante estimaciones más precisas de los errores estándar y permite su extensión hacia componentes avanzados como la inflación de ceros.

Dado que la familia binomial negativa no está integrada por defecto en la función `glm()` del paquete `stats`, utilizaremos el paquete `glmmTMB` (McGillycuddy et al. 2025). La función base `glmmTMB()` permite ajustar desde regresiones lineales y modelos lineales generalizados (GLMs) hasta modelos mucho más complejos, lo que lo convierte en una herramienta poderosa para manejar una amplia variedad de estructuras de datos.

A modo de ejemplo, ajustaremos el modelo Poisson anterior con `glmmTMB()`:

```
fit_pois_tmb <- glmmTMB(
  casos ~ semana_epi + grupo_edad_cat,
  data = datos,
  family = poisson
)

# Salida modelo
summary(fit_pois_tmb)
```

```
 Family: poisson  ( log )
Formula:          casos ~ semana_epi + grupo_edad_cat
Data: datos

      AIC       BIC    logLik -2*log(L)  df.resid
   1735.1    1764.8    -861.5    1723.1      1041
```

```
Conditional model:
                          Estimate Std. Error z value Pr(>|z|)
(Intercept)              -3.092153   0.279663 -11.057  < 2e-16 ***
semana_epi                0.044249   0.008993   4.920 8.64e-07 ***
grupo_edad_cat10-19 años  1.172837   0.294392   3.984 6.78e-05 ***
grupo_edad_cat20-34 años  2.195272   0.271054   8.099 5.54e-16 ***
grupo_edad_cat35-65 años  2.312217   0.269545   8.578  < 2e-16 ***
grupo_edad_cat65+ años    0.730874   0.314523   2.324   0.0201 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Al cotejar la salida de glmmTMB con la obtenida previamente mediante el paquete stats, se observa que los coeficientes estimados y los valores de AIC son prácticamente idénticos. A continuación, procedemos a ajustar el modelo de regresión binomial negativa utilizando el argumento `family = nbinom2`, el cual emplea por defecto la función de enlace logarítmica (`link = "log"`):

```
# Modelo binomial negativo
fit_nb <- glmmTMB(
  casos ~ semana_epi + grupo_edad_cat,
  data = datos,
  family = nbinom2
)

# Salida del modelo
summary(fit_nb)
```

```
 Family: nbinom2  ( log )
Formula:          casos ~ semana_epi + grupo_edad_cat
Data: datos

      AIC       BIC    logLik -2*log(L)  df.resid
   1538.4    1573.0    -762.2    1524.4      1040


Dispersion parameter for nbinom2 family (): 0.467

Conditional model:
                         Estimate Std. Error z value Pr(>|z|)
(Intercept)              -3.27950    0.33522  -9.783  < 2e-16 ***
semana_epi                0.06159    0.01570   3.923 8.73e-05 ***
grupo_edad_cat10-19 años  1.16315    0.33067   3.518 0.000435 ***
grupo_edad_cat20-34 años  2.18318    0.30879   7.070 1.55e-12 ***
grupo_edad_cat35-65 años  2.30095    0.30722   7.490 6.90e-14 ***
grupo_edad_cat65+ años    0.71124    0.34949   2.035 0.041842 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Si comparamos los coeficientes de ambos modelos, veremos que presentan trayectorias similares, aunque los intervalos de confianza se ajustan de manera más realista gracias a la flexibilidad de la binomial negativa:

	Poisson				Binomial negativa			
Characteristic	**Group**	**exp(Beta)**	**95 % CI**	**p-value**	**Group**	**exp(Beta)**	**95 % CI**	**p-**
semana_epi	cond	1.05	1.03, 1.06	<0.001	cond	1.06	1.03, 1.10	<
grupo_edad_cat	cond				cond			
grupo_edad_cat10-19 años	cond	3.23	1.81, 5.75	<0.001	cond	3.20	1.67, 6.12	<
grupo_edad_cat20-34 años	cond	8.98	5.28, 15.3	<0.001	cond	8.87	4.85, 16.3	<
grupo_edad_cat35-65 años	cond	10.1	5.95, 17.1	<0.001	cond	9.98	5.47, 18.2	<
grupo_edad_cat65+ años	cond	2.08	1.12, 3.85	0.020	cond	2.04	1.03, 4.04	0

Abbreviation: CI = Confidence Interval

A pesar de sus múltiples ventajas, la regresión binomial negativa comparte la limitación de no ser adecuada para escenarios con subdispersión. Asimismo, su complejidad computacional exige precaución frente a estructuras de datos muy específicas que puedan comprometer la estabilidad numérica de las estimaciones.

La distribución `"nbinom1` del paquete `glmmTMB` tiene la particularidad de que la varianza es un múltiplo de la media:

$$Var(Y) = \mu(1 + \theta)$$

Al igual que los modelos quasi-Poisson no forma parte de la familia exponencial. Sin embargo, a diferencia de aquellos, se fundamenta en una función de verosimilitud completa, lo que permite calcular criterios de información como el AIC y realizar comparaciones de modelos robustas.

29.4.3. Distribución Conway-Maxwell Poisson

Otra alternativa a la regresión de Poisson, que permite controlar tanto sobredispersión como subdispersión y exceso de ceros es el modelo Conway-Maxwell Poisson (COM-Poisson). El mismo ofrece una generalización del modelo Poisson estándar, introduciendo un parámetro adicional que permite ajustar de forma independiente la media y la varianza.

Podemos ajustar estos modelos con la función `glmmTMB()` especificando el argumento `family = compois()`, que tiene como función de enlace por defecto el logaritmo (`link = "log"`):

```r
# Modelo COMPOIS
fit_compois <- glmmTMB(
  casos ~ semana_epi + grupo_edad_cat,
  data = datos,
  family = compois
)

# Salida del modelo
summary(fit_compois)
```

Una desventaja de estos modelos es que son computacionalmente complejos y están implementados en pocos programas estadísticos y paquetes de R. Por otro lado, los parámetros son menos intuitivos de interpretar, y pueden presentar dificultades en el proceso de estimación, especialmente en casos de datos limitados o de baja calidad.

29.5. Exceso de ceros (*zero-inflation*)

El exceso de ceros ocurre cuando se observa una cantidad de ceros en los datos de conteo sustancialmente mayor a la que predecirían teóricamente los modelos de Poisson o binomial negativa. Este fenómeno es muy frecuente en la investigación epidemiológica y sanitaria debido a la propia naturaleza de los registros de vigilancia.

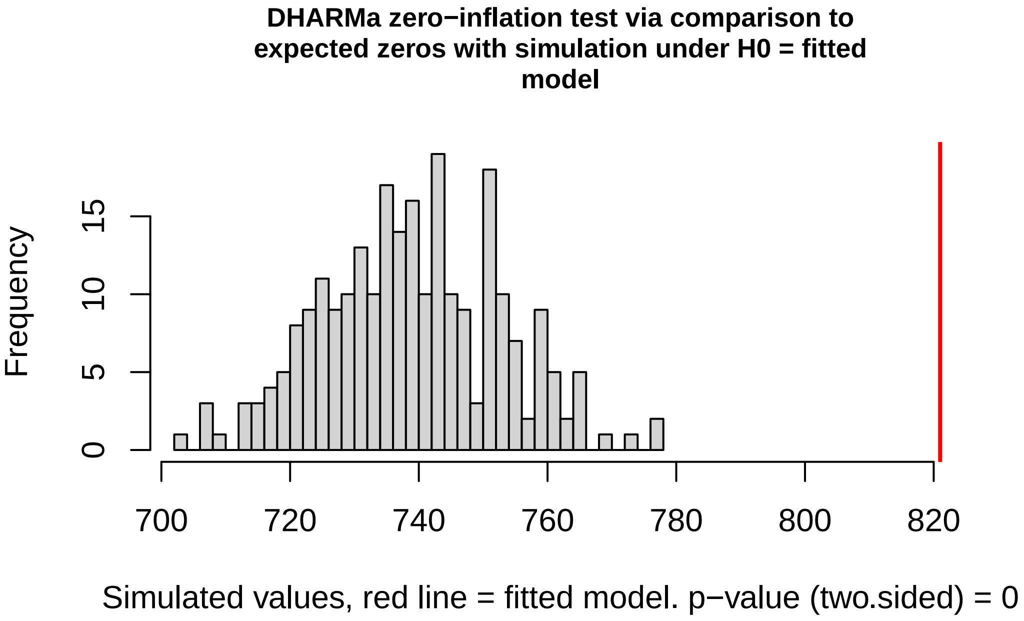
Para identificar un exceso de ceros, se puede utilizar la función `check_zeroinflation()` del paquete `performance`:

```
check_zeroinflation(fit_pois_tmb)
```

```
# Check for zero-inflation

   Observed zeros: 821
  Predicted zeros: 739
            Ratio: 0.90
```

Como alternativa complementaria, la función `testZeroInflation()` del paquete `DHARMa` permite evaluar el mismo supuesto mediante la simulación de residuos normalizados:

```
testZeroInflation(fit_pois_tmb)
```



```
	DHARMa zero-inflation test via comparison to expected zeros with
	simulation under H0 = fitted model

data:  simulationOutput
ratioObsSim = 1.1105, p-value < 2.2e-16
alternative hypothesis: two.sided
```

Cuando los resultados de estas pruebas indican un desajuste significativo, es necesario implementar modelos de regresión con inflación de ceros (*zero-inflated*). Estos modelos asumen que los ceros observados pueden provenir de dos fuentes diferentes:

- Ceros estructurales: generados por una condición intrínseca que impide por completo la ocurrencia del evento
- Ceros muestrales: generados por azar en unidades que, bajo otras circunstancias, habrían tenido la posibilidad de presentar el evento.

Para explorar el origen de este comportamiento en nuestro ejemplo práctico, comenzamos examinando la distribución de los casos agregados por semana epidemiológica:

```r
datos |>
  group_by(semana_epi) |>
  summarise(casos = sum(casos))
```

```
# A tibble: 20 x 2
   semana_epi casos
        <dbl> <dbl>
 1          1     4
 2          2     2
 3          3     4
 4          4     6
 5          5     3
 6          6     6
 7          7     9
 8          8    11
 9          9    20
10         10    48
11         11    51
12         12    59
13         13    63
14         14    49
15         15    31
16         16    11
17         17    12
18         18     9
19         19    11
20         20     1
```

Como se puede observar, se reportaron casos de dengue en todas las semanas epidemiológicas analizadas. A continuación, evaluamos la distribución de los casos agrupados por departamento:

```r
datos |>
  group_by(departamento) |>
  summarise(casos = sum(casos))
```

```
# A tibble: 10 x 2
   departamento       casos
   <chr>               <dbl>
 1 BAHIA BLANCA          117
 2 CORONEL DORREGO         2
 3 GENERAL ALVARADO        6
 4 GENERAL PUEYRREDON     75
 5 LA COSTA              150
 6 LOBERIA                 2
 7 MAR CHIQUITA            8
 8 NECOCHEA               14
 9 PINAMAR                28
10 TRES ARROYOS            8
```

Nuevamente, todos los departamentos de la región reportaron al menos un caso de dengue durante el período de estudio. Para comprender mejor el origen de los ceros, procedemos a cruzar ambas dimensiones espaciales y temporales:

```r
datos |>
  group_by(semana_epi, departamento) |>
  summarise(casos = sum(casos)) |>
  print(n = 20) # Muestra las primeras 20 observaciones
```

```
# A tibble: 200 x 3
# Groups:   semana_epi [20]
   semana_epi departamento       casos
        <dbl> <chr>              <dbl>
 1          1 BAHIA BLANCA           1
 2          1 CORONEL DORREGO        0
 3          1 GENERAL ALVARADO       0
 4          1 GENERAL PUEYRREDON     2
 5          1 LA COSTA               1
 6          1 LOBERIA                0
 7          1 MAR CHIQUITA           0
 8          1 NECOCHEA               0
 9          1 PINAMAR                0
10          1 TRES ARROYOS           0
11          2 BAHIA BLANCA           0
12          2 CORONEL DORREGO        0
13          2 GENERAL ALVARADO       0
14          2 GENERAL PUEYRREDON     1
15          2 LA COSTA               0
16          2 LOBERIA                0
17          2 MAR CHIQUITA           0
18          2 NECOCHEA               0
19          2 PINAMAR                1
20          2 TRES ARROYOS           0
# i 180 more rows
```

Al analizar esta tabla detallada, se advierte que no todos los departamentos reportaron casos en todas las semanas epidemiológicas. Adicionalmente, dado que los datos se encuentran desagregados por grupo etario, es completamente esperable encontrar una alta proporción de combinaciones sin eventos registrados.

Para cuantificar con precisión este fenómeno, generamos una tabla de frecuencias absolutas y relativas para los valores de la variable dependiente:

```
tabyl(datos$casos) |>
  adorn_pct_formatting()
```

```
 datos$casos   n percent
           0 821   78.4%
           1 136   13.0%
           2  44    4.2%
           3  23    2.2%
           4  13    1.2%
           5   5    0.5%
           6   2    0.2%
           7   1    0.1%
           9   1    0.1%
          12   1    0.1%
```

Los resultados indican que un porcentaje elevado de las observaciones (78.4 %) corresponden a recuentos de cero casos, un comportamiento habitual en contextos de vigilancia epidemiológica donde muchas unidades espaciales o temporales no registran eventos activos en períodos específicos.

Para incorporar este comportamiento en el modelado estadístico, podemos emplear la regresión binomial negativa con inflación de ceros mediante la función `glmmTMB()`. Añadiendo el argumento `zi = ~ 1`, especificamos un modelo donde la probabilidad de ceros adicionales se rige por un intercepto único, sin incluir predictores adicionales en la componente de ceros:

```
# Modelo zero-inflated
fit_zi_nb <- glmmTMB(
  casos ~ semana_epi + grupo_edad_cat,
  zi = ~1,
  data = datos,
  family = nbinom2
)

# Salida del modelo
summary(fit_zi_nb)
```

```
 Family: nbinom2  ( log )
Formula:          casos ~ semana_epi + grupo_edad_cat
Zero inflation:         ~1
Data: datos

      AIC       BIC    logLik -2*log(L)  df.resid
   1540.4    1580.0    -762.2    1524.4      1039


Dispersion parameter for nbinom2 family (): 0.467

Conditional model:
```

```
                         Estimate Std. Error z value Pr(>|z|)    
(Intercept)              -3.27950    0.33522  -9.783  < 2e-16 ***
semana_epi                0.06159    0.01570   3.923 8.73e-05 ***
grupo_edad_cat10-19 años  1.16315    0.33067   3.518 0.000435 ***
grupo_edad_cat20-34 años  2.18318    0.30879   7.070 1.55e-12 ***
grupo_edad_cat35-65 años  2.30095    0.30722   7.490 6.90e-14 ***
grupo_edad_cat65+ años    0.71124    0.34949   2.035 0.041842 *  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Zero-inflation model:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   -16.69    3025.54  -0.006    0.996
```

La salida de estos modelos se divide en tres partes que incluyen:

- `Conditional model`: contiene los coeficientes, errores estándar, estadísticos z y valores de significancia correspondientes al modelo de recuento de la familia binomial negativa.
- `Zero-inflation model`: detalla los coeficientes asociados a la probabilidad de observar un cero estructural adicional, incluyendo sus respectivos errores estándar y pruebas de significancia. En este caso, si el intercepto no resulta estadísticamente significativo, sugiere que la probabilidad de ceros adicionales no es uniforme y requiere una parametrización más compleja.
- Estadísticas globales del modelo: incluye los criterios de información (AIC, BIC), la devianza, el logaritmo de verosimilitud y los grados de libertad residuales.

Para flexibilizar el componente de inflación de ceros y capturar de manera más realista la estructura de los datos, por lo general resulta conveniente incorporar las variables independientes también en la fórmula de la componente `zi`:

```
# Modelo zero-inflated
fit_zi_nb2 <- glmmTMB(
  casos ~ semana_epi + grupo_edad_cat,
  zi = ~ semana_epi + grupo_edad_cat,
  data = datos,
  family = nbinom2
)

# Comparar con modelo intercepto
compare_performance(
  fit_nb,
  fit_zi_nb,
  fit_zi_nb2,
  metrics = "common",
  rank = TRUE
)
```

```
# Comparison of Model Performance Indices

Name       |    Model |    R2 | R2 (adj.) |   RMSE | AIC weights | BIC weights | Performance-
Score
```

```
----------------------------------------------------------------------------------------------
fit_zi_nb2 | glmmTMB | 0.124 |     0.119 | 0.912 |       1.000 |       1.000 |           100.0
fit_zi_nb  | glmmTMB | 0.091 |     0.085 | 0.957 |    8.93e-12 |    2.14e-06 |            12.4
fit_nb     | glmmTMB | 0.077 |     0.068 | 0.957 |    2.43e-11 |    6.91e-05 |         1.34e-
03%
```

En este caso, el modelo que incorpora las variables independientes a la fórmula de *zero inflation* tiene mejor *performance* que el modelo que solo incorpora el intercepto y el modelo binomial negativo sin inflación de ceros.

Modelos *hurdle*

Como alternativa , los **modelos *hurdle*** dividen el análisis en dos etapas independientes: un modelo binario para evaluar la probabilidad de que ocurra el evento (cero frente a positivo) y un modelo de conteo truncado en cero para analizar la frecuencia de los casos en las observaciones positivas. No desarrollaremos ejemplos prácticos de esta aproximación por exceder los objetivos y alcances de este curso.

29.6. Resumen

Existen diversas soluciones para tratar la sobredispersión, subdispersión y exceso de ceros en modelos de conteo. En este documento, nos enfocamos en las estrategias más comunes que pueden ser abordadas mediante modelos lineales generalizados utilizando el paquete `glmmTMB`.

Problema	Modelo
Sobredispersión	`glmmTMB(formula, family = nbinom2)`
Subdispersión	`glmmTMB(formula, family = compois)`
Sobredispersión + Exceso de Ceros	`glmmTMB(formula, family = nbinom2, zi = ~1)`

Parte VI.

Unidad 6

30. Estudios Experimentales

30.1. Introducción

En esta última unidad abordaremos los diseños experimentales. En epidemiología, un estudio experimental implica que el investigador manipula activamente las condiciones del estudio con el objetivo de evaluar el efecto de una intervención sobre las observaciones realizadas. La aleatorización constituye una etapa clave, ya que permite asignar la exposición de manera intencional y al azar. En cuanto a su temporalidad, estos estudios son de carácter longitudinal y prospectivo.

Los estudios experimentales presentan una alta capacidad para establecer relaciones causales. De hecho, los meta-análisis de ensayos clínicos aleatorizados constituyen el nivel más alto de evidencia científica. Sin embargo, su implementación suele estar limitada por consideraciones éticas, legales y operativas. Por eso, su aplicación más frecuente se da en la evaluación de fármacos y otras intervenciones sanitarias, especialmente en el contexto de ensayos clínicos. También se utilizan en investigaciones comunitarias orientadas a evaluar la efectividad de programas de salud pública.

Aunque el rasgo distintivo de un estudio experimental es la introducción de una intervención asignada por el investigador, para que se lo considere científicamente válido deben cumplirse las siguientes condiciones:

- La única razón por la que los sujetos reciben la intervención es el cumplimiento del protocolo del estudio.
- El estudio incluye un grupo control que no recibe la intervención bajo análisis.
- La asignación de los sujetos al grupo de intervención se realiza mediante un mecanismo aleatorio.

30.2. Tipos de estudios clínicos experimentales (ECE)

Existen distintas clasificaciones de estudios experimentales según diversos autores, aunque en esencia apuntan a lo mismo. Por ejemplo, Hernández-Ávila (2011) propone la siguiente clasificación:

Tipo de estudio	Objetivos	Duración
Laboratorio	Prueba hipótesis etiológicas y estima respuestas biológicas y/o de comportamiento agudas. Sugiere eficacia de una intervención para modificar factores de riesgo en una población	Horas o días
Ensayo clínico	Prueba hipótesis etiológicas y estima efectos de salud a largo plazo.	

Tipo de estudio	Objetivos	Duración
Prueba eficacia de intervenciones que modifican el estado de salud Sugiere factibilidad de intervenciones	Días hasta años	
Intervención comunitaria	Identifica personas de alto riesgo.	

Prueba eficacia y efectividad de intervenciones clínicas/sociales que modifican el estado de salud dentro de poblaciones particulares Sugiere políticas y programas de salud pública |No menos de 6 meses |

Otras clasificaciones más amplias incluyen estudios donde no se cumplen estrictamente las tres condiciones mencionadas anteriormente, que apuntan a descartar que los efectos observados puedan deberse a factores desconocidos o no controlados. Estos estudios, denominados **cuasi-experimentales**, también implican una intervención, pero carecen del rigor metodológico necesario para asegurar la validez interna. Por lo tanto, presentan limitaciones para atribuir el efecto observado exclusivamente a la intervención. Esta clasificación puede resumirse de la siguiente forma:

Tipo de estudio	Subtipo	Descripción general
Estudios cuasi-experimentales	No controlados	No se incluye grupo de comparación
	Con controles históricos	Comparación con datos del pasado
	Controlados no aleatorizados	Comparación con grupo contemporáneo, pero sin asignación aleatoria
Estudios controlados y aleatorizados	Ensayo de prevención primaria (ensayo de campo)	Evalúan intervenciones preventivas en poblaciones sanas
	Ensayo de intervención comunitaria	Evalúan programas a nivel grupal o comunitario
	Ensayo clínico de grupos paralelos	Cada grupo recibe una intervención distinta de forma simultánea
	Ensayo clínico cruzado	Cada sujeto recibe más de una intervención en distintos períodos
	Ensayo clínico factorial	Evalúa dos o más intervenciones simultáneamente en distintas combinaciones

En esta unidad nos enfocaremos principalmente en los **ensayos clínicos controlados**, pero pueden consultar la bibliografía para explorar los otros diseños. Como en todo estudio epidemiológico, es fundamental adoptar medidas que minimicen el riesgo de sesgos. Habitualmente se consideran 5 principios para evitar sesgos:

- Uso de grupo control
- Aleatorización
- Cegamiento
- Consentimiento antes de aleatorizar
- Análisis por intención de tratar

30.2.1. Uso de grupo control

Al igual que en los estudios de cohortes, los estudios de intervención deben incluir un grupo control con el cual comparar el grupo tratado. Habitualmente se utilizan dos grupos, aunque puede haber más. El grupo control puede recibir un placebo o un tratamiento activo alternativo, como la terapia existente.

30.2.2. Aleatorización

Su objetivo principal es garantizar que las diferencias observadas entre grupos al final del estudio no se deban a factores distintos a la intervención. La aleatorización se refiere a la asignación mediante el azar de las unidades de investigación (individuos) a los distintos tratamientos a evaluar, con la finalidad de compararlos según las variables resultado definidas.

30.2.3. Cegamiento

Es la política de mantener a alguien sin saber qué tratamiento se ha administrado. Existen distintas categorías de cegamiento:

- En estudios **simple ciego**, el sujeto ignora el tratamiento recibido.

- En estudios **doble ciego**, tanto el sujeto como quien evalúa los resultados desconocen la asignación.

- En estudios **triple ciego**, incluso quien analiza los datos (por ejemplo, un/a estadístico/a) también lo desconoce.

El cegamiento reduce el sesgo del observador, aunque no siempre es posible. Un ejemplo claro es cuando se compara un tratamiento con radiación con un tratamiento quirúrgico para el cáncer de mama. Por otro lado, es fundamental que la asignación del tratamiento esté codificada y que alguien pueda descifrar el código en momentos de problemas médicos durante el estudio y cuando se requieran resultados finales.

30.2.4. Consentimiento antes de la aleatorización

Para evitar sesgos en la composición de los grupos, el consentimiento informado y la elegibilidad de los sujetos para cada tratamiento deben verificarse antes de asignar aleatoriamente a los participantes.

30.2.5. Análisis por intención de tratar

Durante el estudio de intervención, los sujetos pueden suspender o modificar su tratamiento asignado por muchas razones, que incluyen enfermedad, toxicidad, migración o simplemente por error. Este enfoque analiza a los sujetos según el tratamiento asignado, independientemente de su adherencia. Esto evita sesgos debidos a interrupciones o cambios de tratamiento, y refleja mejor lo que ocurre en la práctica clínica. No obstante, puede subestimar la diferencia real entre tratamientos. En algunos casos, un análisis ITT completo no es factible.

30.3. Análisis de los estudios experimentales

En un estudio experimental, se realiza un seguimiento de una cohorte. Lo más habitual en ensayos clínicos controlados es medir la incidencia de un evento en ambos grupos durante un período definido. Los eventos se categorizan de manera dicotómica (presencia/ausencia) y se presentan como la proporción de sujetos que presentan el desenlace (por ejemplo, infarto, recurrencia, muerte). Al tratarse de una cohorte, es posible calcular el riesgo relativo (RR), la diferencia de riesgos o la reducción relativa del riesgo. También se utiliza el **número necesario a tratar (NNT)**, que indica cuántos pacientes deben recibir la intervención para evitar un caso adicional.

Una metodología frecuentemente utilizada es el **análisis de supervivencia**, donde el tiempo hasta la ocurrencia del evento cobra un rol central. Como ya se vio en la **unidad anterior**, esta técnica también puede aplicarse a estudios de cohortes. A continuación avanzaremos en su comprensión.

31. Análisis de supervivencia

31.1. Introducción

El estudio de procesos de supervivencia implica el seguimiento de individuos a lo largo del tiempo, pudiéndose presentar diversas situaciones que complican su caracterización. La característica más relevante de este tipo de datos —el tiempo hasta que ocurre un determinado suceso— es que, al finalizar el periodo de observación, probablemente no todos los pacientes habrán experimentado el evento de interés.

Además puede ocurrir que algunos pacientes se hayan perdido durante el seguimiento por causas diversas, imposibilitando conocer su estado final. También es habitual que los pacientes vayan incorporándose durante todo el periodo de observación. En estos casos, quienes ingresan más tarde serán observados durante un tiempo más corto, por lo que su probabilidad de experimentar el suceso será menor.

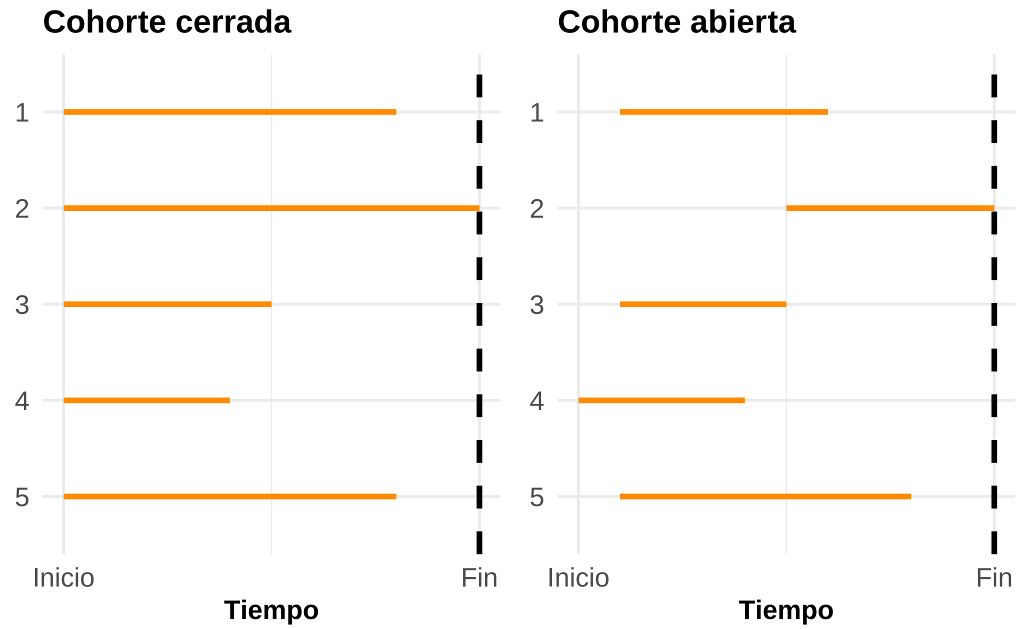
En enfermedades crónicas, tales como el cáncer, la supervivencia se mide como la probabilidad de permanecer con vida durante un determinado intervalo de tiempo. Indicadores como la supervivencia al año o a los cinco años son comúnmente utilizados para caracterizar la gravedad de una enfermedad y establecer pronósticos. Típicamente, el pronóstico del cáncer se valora determinando el porcentaje de pacientes que sobrevive al menos cinco años después del diagnóstico.

El análisis de supervivencia se utiliza cuando el tiempo es el objeto de interés, ya sea como “*tiempo hasta la ocurrencia de un evento*” o como “*riesgo de ocurrencia por unidad de tiempo*”. Este enfoque incorpora el componente dinámico del tiempo, por lo que la variable respuesta es una combinación de dos elementos: el evento (respuesta) y el tiempo hasta que ocurre.

El evento o desenlace de interés es de tipo dicotómico, e indica si el suceso ha ocurrido o no (por ejemplo: muerte, recidiva, aparición de una complicación, etc.). La variable numérica, por su parte, representa el tiempo transcurrido hasta la aparición del evento.

Los paradigmas de diseño de estudio que generan este tipo de información son principalmente los estudios de cohorte y los ensayos clínicos, ya que ambos implican el seguimiento de un grupo de individuos a lo largo del tiempo. Esto permite acceder directamente a la variable respuesta: el tiempo hasta la ocurrencia de un evento.

Recordemos que en ambos diseños, pueden presentarse dos tipos de cohortes: **cerradas** y **abiertas**. Según el tipo, será posible estimar distintas medidas de ocurrencia: incidencia acumulada o densidad de incidencia. También es importante tener presente que, en ambos tipos de cohortes, puede haber pérdida de información por distintas razones, lo que debe ser considerado en el análisis.



A la variable “*tiempo hasta un evento*”, en el contexto del análisis de supervivencia, se la llama habitualmente “**tiempo de supervivencia**”, aunque el evento de interés no sea la muerte. Algunos autores se refieren a los tiempos de supervivencia como “*tiempos de fallo*”. En general, en el lenguaje de supervivencia, el evento de interés, tiene una connotación negativa: muerte, recaída, rechazo a un trasplante, insuficiencia renal, etc.

Comenzaremos intentando comprender las diferencias que este tipo de análisis tiene con otras metodologías, en que se diferencian los datos del resto de los datos y por qué necesitamos mucho soporte estadístico para comprender esta metodología.

31.2. El lenguaje propio del análisis de supervivencia

Una de las características distintivas de los datos de supervivencia es que la variable de interés —el tiempo hasta que ocurre un evento— solo toma **valores positivos** y suele presentar **distribuciones sesgadas**.

Por ejemplo:

- En un estudio sobre recaídas en pacientes de alto riesgo, la mayoría de los eventos pueden ocurrir temprano en el seguimiento.
- En cambio, en un estudio poblacional sobre tiempo hasta la muerte, la mayoría de los eventos pueden ocurrir en etapas más tardías.

Esto nos hace pensar que los procedimientos estadísticos estándar que asumen normalidad de los datos no son apropiados para estos casos. Aunque podríamos recurrir a procedimientos no paramétricos, existe un desafío adicional: la información de seguimiento suele estar incompleta para algunos participantes.

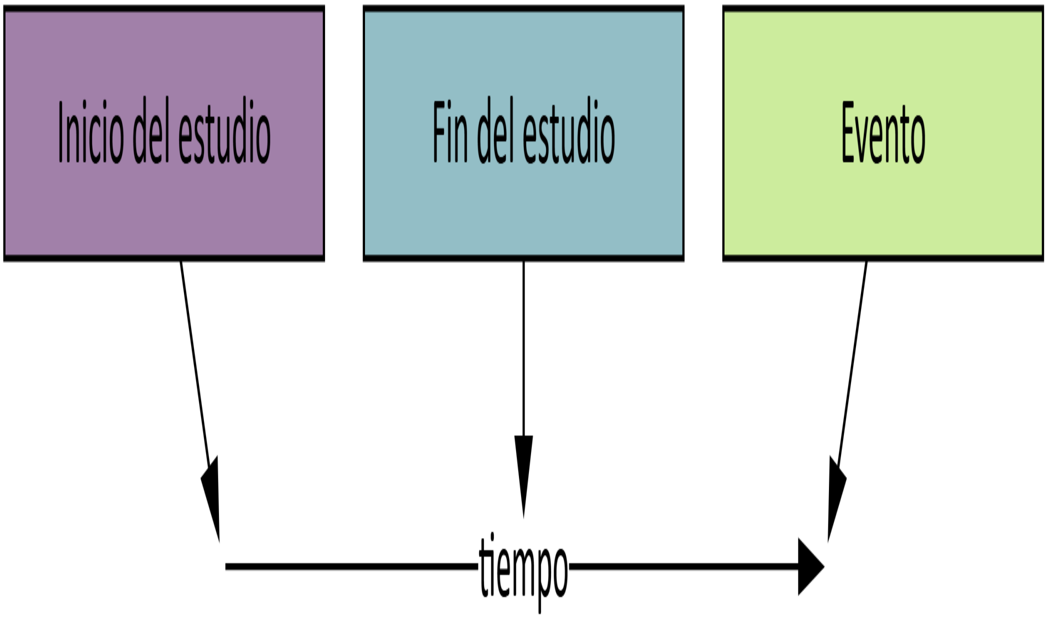
31.2.1. Censura y truncamiento

En estudios con cohortes abiertas, los participantes que ingresan más tarde al estudio tienen un período de seguimiento más corto que quienes ingresan más temprano. Algunos participantes pueden abandonar el estudio antes de finalizar el seguimiento (por mudanza, pérdida de interés, etc.), o incluso fallecer, aunque el evento de interés no sea la muerte.

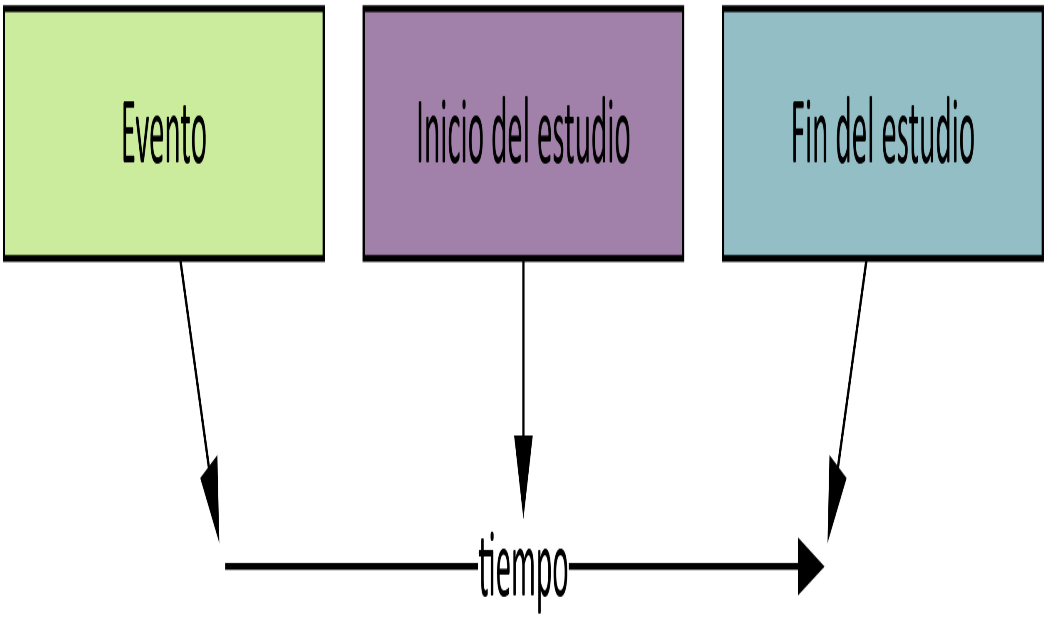
En cada uno de estos casos, tenemos información de seguimiento incompleta. Es posible que el tiempo hasta que se produce el evento del interés no se conozca porque el estudio termina o porque un participante se retira del estudio antes de experimentar el evento. Estos tiempos son llamados **tiempos censurados**.

La **censura**, se refiere a la pérdida de información e indica que no se ha observado el tiempo exacto en que ocurrió el evento de interés. Existen tres tipos:

- **Censura a la derecha**: ocurre cuando el tiempo entre el inicio del estudio y el evento de interés, es mayor que el tiempo de observación.



- **Censura a la izquierda**: ocurre cuando el individuo presenta el evento de interés antes de ingresar al estudio, por lo que se sabe que su tiempo de supervivencia no observado es menor que el tiempo de observación.



Un ejemplo de esta situación puede darse en el caso en que se tiene el interés de determinar el momento (edad) en que el niño aprende a desarrollar cierta habilidad o tarea. Puede darse que algunos niños incluidos en el estudio ya hayan adquirido dicha habilidad previamente al ingreso al estudio.

- **Censura intervalar**: en algunos estudios, la observación de los sujetos no se realiza de manera continua y los períodos de observación pueden ser muy largos entre dos observaciones consecutivas. En esta situación, es común que ocurra el evento de interés entre dos puntos de observación y no se sepa cuando ocurrió exactamente.

El **truncamiento** ocurre cuando ciertos individuos no ingresan al estudio debido a que el evento ya ocurrió fuera del período de observación. Al igual que para la censura, existen truncamientos a la izquierda, a la derecha y por intervalos, siendo los dos primeros los más comunes.

- **Truncamiento a la izquierda**: en un estudio sobre supervivencia en mayores de 60 años, no se incluyen personas que murieron antes de esa edad.

- **Truncamiento a la derecha**: en un estudio sobre infección por VIH, solo se incluyen personas que se infectaron dentro de la ventana del estudio.

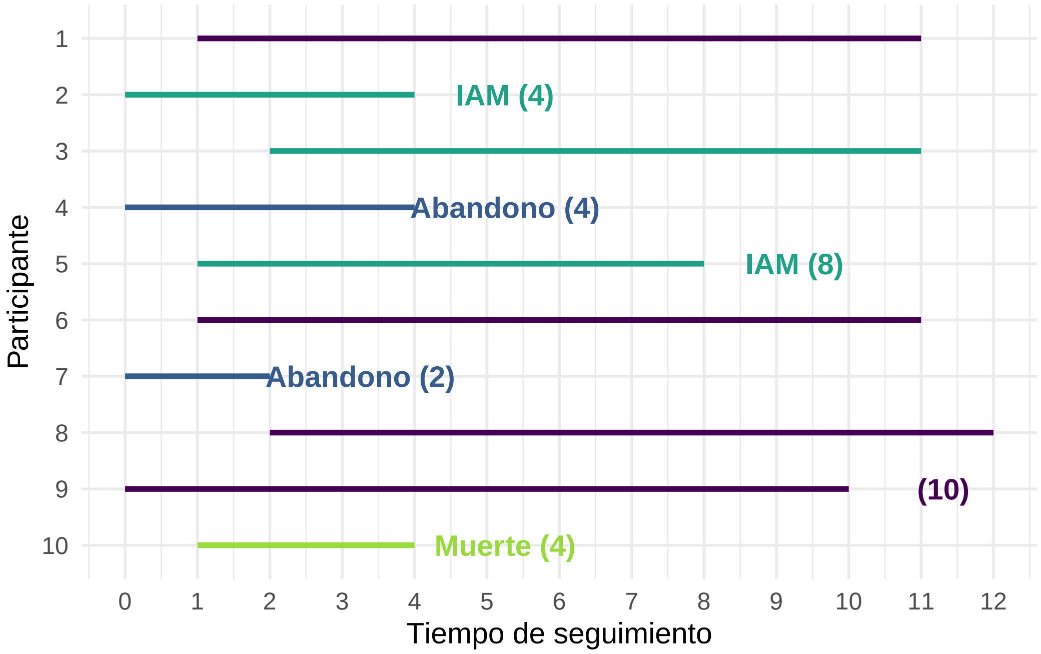
Esta “desinformación” puede traer problemas técnicos importantes. La mayoría de los métodos de análisis de supervivencia suponen que los individuos censurados tienen la misma probabilidad de experimentar el evento que aquellos que permanecieron en el estudio. Si esto es así, decimos que la censura es **“no informativa**” y es lo deseable.

Por ejemplo, si se pierde la información de un individuo por mudanza, no hay razón para pensar que eso está vinculado al evento de interés. Si en cambio, el individuo abandona el estudio por complicaciones médicas, es más probable que esté ligado a nuestro evento de interés. Este último caso, se trata de “**censura informativa**”, lo cual debe evitarse, porque afectará la validez de las estimaciones.

Ejemplo de cohorte abierta

Consideremos un pequeño estudio prospectivo donde se siguen diez participantes para evaluar el desarrollo de infarto agudo de miocardio (IAM) durante un período de 10 años. Los participantes son reclutados para el estudio durante un período de dos años y son seguidos durante un máximo de 10 años.

El siguiente gráfico resume el seguimiento de cada participante, indicando eventos (IAM), abandonos, muertes y finalización del estudio sin evento:



Observando el gráfico, podemos decir que, durante el seguimiento:

- Tres pacientes sufrieron el evento de interés IAM (pacientes 2, 3 y 5)
- Dos pacientes abandonaron el estudio por razones desconocidas (pacientes 4 y 7)
- El paciente 10 falleció
- Cuatro pacientes completaron los 10 años de seguimiento sin sufrir IAM (pacientes 1, 6, 8 y 9)

Basados en estos datos: ¿Cuál es la probabilidad de un paciente de sufrir IAM en 10 años?

Tres de diez participantes sufren IAM en el transcurso del seguimiento, es decir que la probabilidad sería 0,3 (30 %). Pero este valor probablemente sea una subestimación del verdadero porcentaje ya que dos participantes abandonaron el estudio y podrían haber sufrido un infarto, que habría sido observado si hubiésemos podido seguirlos los 10 años completos. Sus tiempos observados fueron **censurados**. Además, un participante muere después de tres años de seguimiento.

¿Debemos incluir estos tres individuos en el análisis?

Si excluimos a los tres participantes censurados, la estimación de la probabilidad de que un individuo sufra un IAM sería 3/7 = 43 %, un valor sensiblemente superior a la estimación inicial de 30 %. Esta diferencia destaca una de las particularidades del análisis de supervivencia: no siempre es posible observar a todos los participantes durante todo el período de seguimiento, lo que hace que estos datos requieran un tratamiento estadístico especial. En nuestro ejemplo:

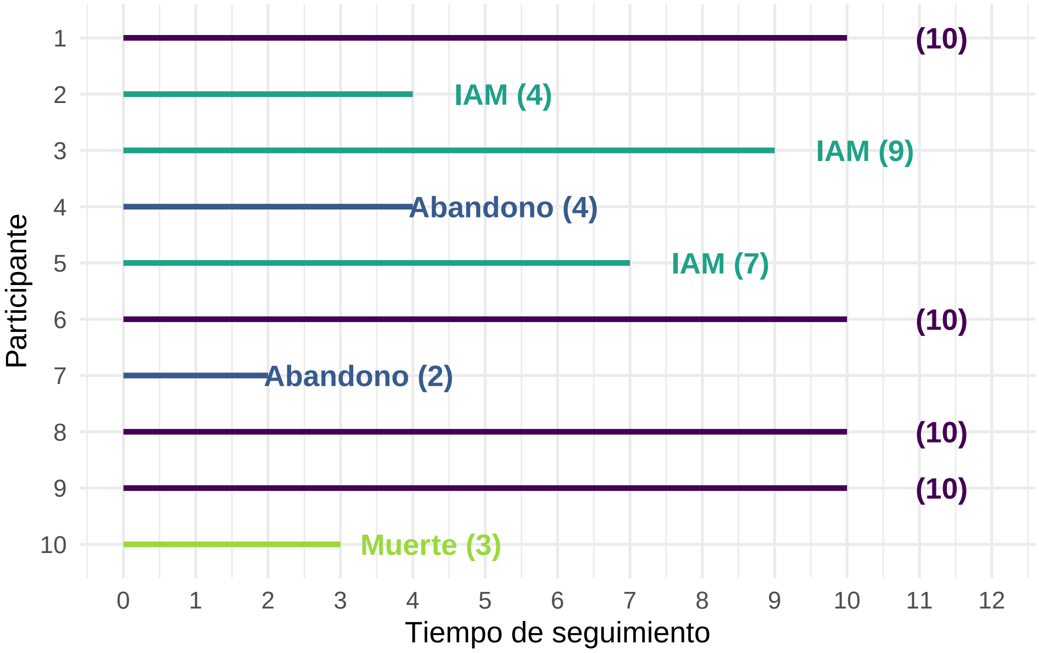
- El participante 4 se observó durante 4 años y durante ese período no tuvo un IAM
- El participante 7 se observó durante 2 años y durante ese periodo no tuvo un IAM

Mientras no hayan experimentado el evento de interés, los individuos en seguimiento aportan información importante. Las técnicas de análisis de supervivencia aprovechan esa información parcial para estimar con mayor precisión la probabilidad de ocurrencia del evento. Muy pronto veremos cómo, pero antes, puntualizaremos un supuesto importante que se hace para hacer un uso adecuado de los datos censurados.

> Específicamente, se supone que la censura es independiente o no relacionada con la probabilidad de desarrollar el evento de interés. Dicho de otra forma, la **censura debe ser no informativa** y asume esencialmente que los participantes cuyos datos son censurados, tendrían la misma distribución de tiempos de supervivencia que si fueran realmente observados.

Ejemplo de cohorte cerrada

Ahora consideremos un estudio con los mismos participantes, pero todos ingresan al mismo tiempo. Aunque los eventos y censuras son los mismos, el momento en que ocurren cambia: los IAM ocurren más temprano, y los abandonos/muertes más tarde:



¿Pueden estas diferencias en las experiencias de los participantes afectar a la estimación de la probabilidad de que un participante sufra un infarto de miocardio en un período de 10 años? En breve contestaremos también esta pregunta.

> En el análisis de supervivencia se analiza no sólo el número de participantes que sufren el evento de interés (indicador dicotómico de estado del evento), sino también los momentos en que se producen los acontecimientos (tiempo de supervivencia).

Si todos los pacientes bajo estudio experimentaran el evento durante el período de observación, sería sencillo calcular la proporción de aquellos que, transcurrido un determinado tiempo desde su inclusión, aún no han presentado el evento: es decir, los “sobrevivientes”. Podríamos entonces representar gráficamente esta proporción en función del tiempo desde la entrada al estudio, y obtendríamos la **función de supervivencia**, denotada como $S(t)$.

Sin embargo, las cosas se complican cuando no todos los pacientes presentan el evento. Por ejemplo, un paciente podría haber ingresado a mitad del período de seguimiento, ser observado durante dos años y, al finalizar el estudio, no haber sufrido el evento.

Como dijimos, en análisis de supervivencia nos interesa el tiempo hasta que ocurre un evento. Este tiempo se conoce como **tiempo de supervivencia**, incluso cuando el evento no sea la muerte. Se trata de una **variable aleatoria continua**, que no puede tomar valores negativos y que rara vez sigue una distribución normal. Ya hemos visto que toda variable aleatoria posee una distribución de probabilidad que describe su comportamiento. Así que, para estimar probabilidades de que el evento ocurra, necesitaremos conocer la distribución de estos tiempos de supervivencia.

> ⚠ Como probablemente habrán advertido, estamos utilizando el lenguaje de la probabilidad, así que si no recuerdan los conceptos de función de densidad y función de probabilidad acumulada para una variable aleatoria continua, es un buen momento para revisarlos.

31.3. Función de Supervivencia $S(t)$

Como mencionamos anteriormente, los tiempos de supervivencia se modelan mediante una **función de densidad de probabilidad**, denotada como $f(t)$, que representa la probabilidad de que el evento ocurra exactamente en el tiempo t. Sin embargo, como se trata de una variable continua, esta probabilidad puntual es técnicamente cero. Por eso, para obtener probabilidades debemos calcular el área bajo la curva de $f(t)$ en un intervalo infinitesimal alrededor de t.

A partir de $f(t)$, podemos definir la **función de distribución acumulada** $F(t)$, que expresa la probabilidad de que el evento ocurra en un tiempo menor o igual a t. Se obtiene calculando la integral de la $f(t)$.

$$F(t) = P[T \leq t] = \int_0^t f(t)dt$$

Por ejemplo, si en un estudio estamos interesados en evaluar la supervivencia de pacientes pos trasplante hepático, necesitaríamos conocer la distribución de estos tiempos de supervivencia. El evento de interés sería “*muerte pos trasplante*” y si quisiéramos conocer la probabilidad de muerte a los dos, tres o cinco años, deberíamos realizar la integral señalada al tiempo deseado.

Si existe una $F(t)$, que nos da la probabilidad de morir a determinado tiempo t, entonces, existirá la función de supervivencia $S(t)$, que mide la probabilidad de sobrevivir más de un tiempo t.

$$S(t) = Pr(T > t)$$

$$F(t) = Pr(T \leq t)$$

$$S(t) = 1 - F(t)$$

Cumple con: $S(0) = 1$ (en el primer momento, todos los pacientes están vivos)

Para $t \to \infty$; $S(t) \to 0$ (en algún momento, lejano, todos moriremos!!!).

Si considero un período de tiempo, entre t_1 y t_2:

$$Si \quad t_1 < t_2;\ entonces \quad S(t_1) > S(t_2)$$

Al considerar la posibilidad de adoptar una distribución teórica para los tiempos de supervivencia, estamos haciendo un *abordaje paramétrico*. Las funciones densidad de probabilidad más utilizadas son: la exponencial, Weibull y lognormal. No profundizaremos en detalles sobre estas aproximaciones, porque no son tan aplicadas en el ámbito del análisis de supervivencia aplicado a salud.

Nos centraremos en los *abordajes no paramétricos*, que parten de la construcción de las llamadas **Tablas de supervivencia**, y tienen como objetivo central la estimación de dos medidas: la probabilidad de morir en un momento dado y la probabilidad de sobrevivir hasta un determinado tiempo. Existen dos métodos: el **método actuarial** y el método de **Kaplan Meier**, que describiremos en el siguiente capítulo.

32. Métodos para el análisis de supervivencia

32.1. Funciones básicas

En el análisis de supervivencia, se busca modelar una variable aleatoria continua y no negativa, denotada por T, que representa el "*tiempo hasta un evento determinado*" para cada individuo del estudio. A continuación, se describen sus cuatro funciones fundamentales.

32.1.1. Función densidad de probabilidad

Como toda variable aleatoria continua, T posee una función de densidad de probabilidad, $f(t)$, que ya presentamos en la primera parte. Esta función puede interpretarse como la probabilidad de que un individuo experimente el evento en un intervalo instantáneo de tiempo. Asociada a ella, también se encuentra la función de distribución acumulada $F(t)$, que representa la probabilidad de que el evento ocurra en un intervalo determinado: [t_1, t_2].

32.1.2. Función de supervivencia

La función de supervivencia, $S(t)$, indica la probabilidad de que un individuo no experimente el evento hasta al menos el tiempo t, es decir, que "sobreviva" hasta ese momento.

32.1.3. Función de riesgo (*hazard*)

En el contexto del análisis de supervivencia, suele mantenerse el término en inglés *hazard*, aunque su traducción aproximada es "riesgo", ya que asume ciertas particularidades. Esta función permite analizar el riesgo instantáneo de que un individuo experimente el evento entre el tiempo t y $t + \varepsilon$, dado que sobrevivió hasta el tiempo t. Por ejemplo, para responder a preguntas como: ¿cuál es el riesgo de un paciente de fallecer después de una cirugía a corazón abierto?, deberemos conocer esta función de riesgo.

Se simboliza como $h(t)$ o $\lambda(t)$ y la expresión es:

$$h(t) = \lim_{n \to \infty} ((t < T < t + \varepsilon)|T > t)/\varepsilon$$

Esta expresión puede interpretarse, aproximadamente, como la probabilidad de morir en el intervalo $(t,\ t + \Delta t]$, dado que se está vivo al tiempo t, o bien de forma general como la probabilidad de presentar el evento en dicho intervalo. Podemos interpretarla como una medida de intensidad de muerte al tiempo t, o una medida de muerte potencial al tiempo t, o en forma general, como una medida de intensidad de eventos a dicho tiempo.

La función de riesgo recibe otros nombres como: tasa de fallo, función o tasa de incidencia, fuerza de mortalidad condicional o simplemente fuerza de mortalidad. Es importante resaltar que a pesar de llevar el nombre de riesgo; $h(t)$ es una tasa y no una probabilidad, es por eso que a veces se prefiere usar el término *hazard*. Su unidad es $tiempo^{-1}$ y puede asumir cualquier valor real mayor que cero.

Una de sus principales utilidades es guiar la elección de un modelo paramétrico adecuado para los datos de supervivencia. En muchos contextos, el investigador tiene una noción previa sobre cómo varía el riesgo a lo largo del tiempo, lo que puede facilitar la elección de una función específica para $h(t)$. Para entender un poco mejor esto último observemos detenidamente la siguiente figura:

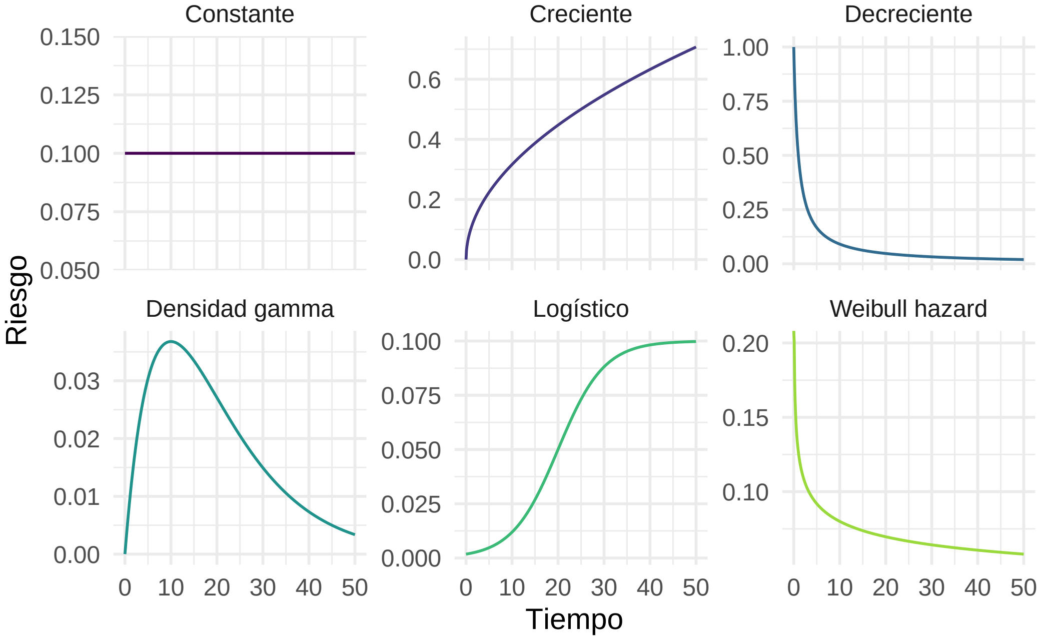


Figura 32.1.: Funciones de riesgo

- El gráfico **A** muestra una función de riesgo constante en el tiempo. Por ejemplo: riesgo de fracturas de escolares de 5to grado durante un año.
- El gráfico **B** muestra una función de riesgo que decrece en el tiempo. Por ejemplo: el riesgo post-quirúrgico, que es mayor en los primeros días post cirugía y luego cae con el tiempo.
- El gráfico **C** muestra una función de riesgo que se incrementa con el tiempo. Por ejemplo: si estudiamos el "tiempo hasta la solidificación de una fractura ósea", en un primer momento el "riesgo de solidificación" de la fractura es nulo, y a medida que transcurre el tiempo se incrementa.
- Los gráficos **D-F** presentan patrones más complejos de riesgo que pueden representar situaciones específicas.

Para cerrar diremos que la función de riesgo marca la dinámica del proceso estudiado, al dar sus valores una adecuada aproximación a la tasa de incidencia del evento de interés.

32.1.4. Función de riesgo acumulado

A partir de la función de riesgo ($h(t)$) es posible calcular la función de riesgo acumulado, que se simboliza como $\Lambda(t)$ o $H(t)$. Esta función mide el riesgo de ocurrencia de un evento hasta un determinado tiempo t. Matemáticamente implica la suma de todos los riesgos en todos los tiempos hasta el tiempo t.

La expresión matemática es:

$$H(t) = \int_0^t h(u)du$$

Por ejemplo, permite responder preguntas como: ¿Cuál es el riesgo acumulado de morir en el primer año luego del diagnóstico de SIDA? ¿Y en los primeros dos años?

Las funciones fundamentales del análisis de supervivencia están estrechamente relacionadas entre sí y son matemáticamente equivalentes. A continuación se resumen sus principales relaciones, omitiremos aquí dichas deducciones, pero hagan un acto de fe y crean en las siguientes equivalencias:

$$S(t) = 1 - F(t)$$

$$h(t) = -\frac{dln(S(t))}{dt}$$

$$h(t) = \frac{f(t)}{S(t)}$$

$$h(t) = \frac{f(t)}{1 - F(t)}$$

$$H(t) = -ln(S(t))$$

$$S(t) = exp(-H(t))$$

Una vez que estimamos una de ellas, será posible obtener cualquiera de las demás.

32.2. Modelos paramétricos

En esencia, cualquier distribución definida sobre los números reales no negativos puede utilizarse para modelar el tiempo hasta la ocurrencia de un evento. Sin embargo, en la práctica, solo un pequeño conjunto de distribuciones es ampliamente utilizado en análisis de supervivencia. Las más frecuentes son la **exponencial**, la **Weibull** y la **lognormal**.

32.2.1. Distribución exponencial

Al asumir que la variable aleatoria T posee una distribución exponencial, su $f(t)$ será:

$$f(t) = \alpha e^{-\alpha t} \quad (\alpha > 0)$$

Utilizando las relaciones entre las funciones básicas de supervivencia se deducen:

$$S(t) = e^{-\alpha t}$$

$$h(t) = \frac{f(t)}{S(t)} = \frac{\alpha e^{-\alpha t}}{e^{-\alpha t}}$$

$$H(t) = -lnS(t) = \alpha t$$

Como podemos ver, al asumir una distribución exponencial para la variable “tiempo hasta que el evento se produzca”, el riesgo es constante en el tiempo y el riesgo acumulado es una función lineal del tiempo:

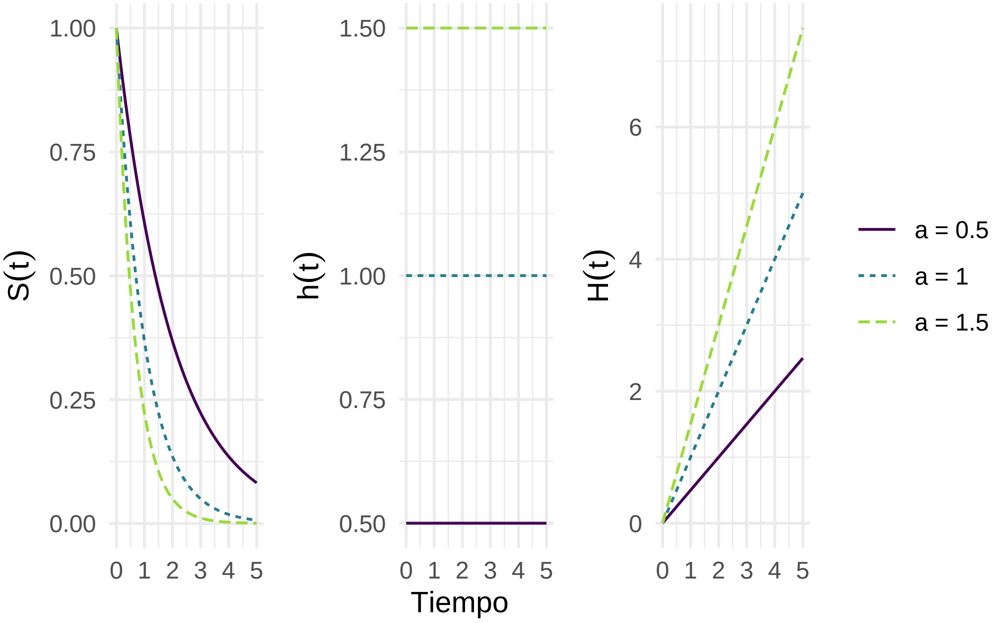


Figura 32.2.: Funciones de supervivencia, riesgo y riesgo acumulado para la distribución exponencial.

Asumir una distribución exponencial para los tiempos de sobrevida es similar a suponer una distribución normal para otras variables: se trata de una elección que facilita el modelado. Sin embargo, en la práctica es una situación muy rara. Como mencionamos anteriormente, una posible excepción sería el riesgo de fractura en escolares de 5º grado a lo largo de un año escolar. Pero no existen demasiadas situaciones en salud donde el riesgo sea constante, en la mayoría de las situaciones el riesgo varía en el tiempo, usualmente incrementándose con la edad del individuo.

Uno de los parámetros clave en el análisis de supervivencia es la **mediana de los tiempos de supervivencia**, es decir, el tiempo en el cual el 50 % de los individuos ha experimentado el evento. Para el caso exponencial:

$$S(t) = e^{-\alpha t} = 0,5$$

$$-\alpha t = ln(0,5)$$

$$\alpha t = ln(2)$$

$$T_{mediano} = \frac{ln(2)}{\alpha}$$

A pesar de su simplicidad, la suposición de un riesgo constante limita seriamente la aplicabilidad del modelo exponencial en contextos de salud.

32.2.2. Distribución Weibull

La distribución Weibull es una generalización de la exponencial, ampliamente utilizada en contextos biomédicos. Su función de densidad es:

$$f(t) = \gamma \alpha^{\gamma} t^{\gamma-1} exp(-(\alpha t)^{\gamma}) \qquad (\alpha > 0, \gamma > 0)$$

A partir de esta expresión, se deducen:

$$S(t) = exp(-(\alpha t)^{\gamma})$$

$$h(t) = \gamma \alpha^{\gamma} t^{\gamma-1}$$

$$H(t) = -lnS(t) = (\alpha t)^{\gamma-1}$$

La distribución Weibull tiene **dos parámetros**:

- α, que regula la escala del modelo
- γ, que controla la forma de la función de riesgo:
  - Si $\gamma < 1$, la función de riesgo es decreciente
  - Si $\gamma > 1$, la función de riesgo es creciente
  - Si $\gamma = 1$, la función de riesgo es constante y equivale a una distribución exponencial

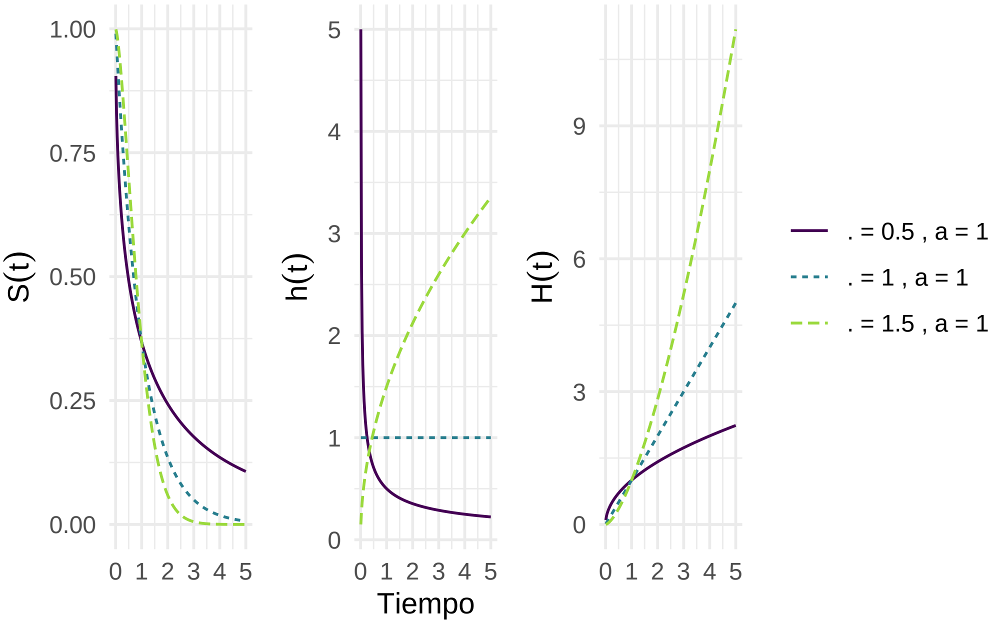


Figura 32.3.: Funciones de supervivencia, riesgo y riesgo acumulado para la distribución Weibull.

Para encontrar el tiempo mediano de supervivencia, procedemos en forma análoga que en la distribución exponencial:

$$S(t) = exp(-(\alpha t)^{\gamma}) = 0,5$$

$$-(\alpha t)^{\gamma} = ln(0,5)$$

$$T_{mediano} = \frac{ln(2)^{1/\gamma}}{\alpha}$$

Dejaremos de lado la distribución lognormal, para pasar ahora a los métodos no paramétricos de estimación.

32.3. Métodos no paramétricos

A continuación, abordaremos dos métodos no paramétricos para estimar las funciones básicas de supervivencia. Ambos se basan en las denominadas *tablas de vida*, que describen el proceso de mortalidad de una cohorte hasta la desaparición del último de sus integrantes bajo la experiencia de mortalidad observada en un período determinado. Estas tablas finalizan con la muerte de todos los sujetos, y su principal diferencia radica en la velocidad con que se alcanza dicho final.

Las tablas de vida se utilizan principalmente en salud pública para medir mortalidad y supervivencia, aunque también son aplicadas en estudios demográficos y actuariales, para analizar longevidad, fertilidad, migraciones, crecimiento poblacional, proyecciones y años de vida sin discapacidad. En resumen, es una presentación tabular del progreso de una cohorte a través del tiempo que transcurre.

Tabla 32.1.: Información inicial para construir la tabla de supervivencia.

Año ingreso	Año	Vivos	Muertes	Pérdidas	Muertos por otras causas	Vivos final
2009	2009	60	21	4	1	18
2009	2010	34	6	1	0	
2009	2010	27	4	0	1	
2009	2011	22	2	1	0	
2009	2011	19	1	0	0	
2010	2011	67	23	3	2	20
2010	2012	39	8	2	1	
2010	2012	28	3	1	0	
2010	2012	24	3	0	1	
2011	2012	53	18	3	0	23
2011	2013	32	5	1	0	
2011	2013	26	2	0	1	
2012	2013	64	22	5	2	25
2012	2013	35	7	2	1	
2013	2013	56	19	2	1	34

Las curvas de supervivencia pueden construirse mediante dos métodos: el método de Kaplan-Meier o el análisis actuarial de Cutler-Ederer. Ambos permiten estimar la supervivencia en presencia de censura. Además, existe un estimador no paramétrico de la función de riesgo acumulado, conocido como *estimador de Nelson-Aalen*, que no será abordado en este curso.

32.3.1. Método actuarial

En este método, los tiempos de supervivencia se agrupan en intervalos, cuya duración puede variar según la frecuencia del evento de interés y puede no ser de la misma longitud. Una limitación de este enfoque es la menor precisión en muestras pequeñas. Sin embargo, en muestras grandes, su influencia sobre las estimaciones es reducida y permite estimar también la función de riesgo. Se asume que las observaciones censuradas se distribuyen homogéneamente dentro de cada intervalo.

Antes de continuar con el ejemplo se sugiere ver la siguiente videoclase:

⍰ Método Actuarial

Consideremos el ejemplo de un estudio de supervivencia pos trasplante hepático en una cohorte de 300 pacientes, seguidos durante cinco años (2009-2014). La siguiente tabla (Tabla 32.1) presenta los datos iniciales:

Para interpretar la información de la tabla, es necesario considerar que la misma presenta la evolución anual de los pacientes que fueron admitidos en el estudio año a año. En 2009 se incorporaron 60 pacientes, de los cuales 21 fallecieron, cuatro se perdieron en el seguimiento (se ignora si sobrevivieron o fallecieron) y uno murió por causas ajenas al trasplante. Estos últimos cinco pacientes corresponden a observaciones *censuradas*. Para el año 2010, 34 pacientes del grupo original y se sumaron 67 nuevos participantes al estudio.

Cada paciente que ingresa durante cada uno de los años del estudio contribuye con información a lo que ocurre durante el primer año de supervivencia:

- Un total de 300 pacientes ingresaron vivos al estudio: $60 + 67 + 53 + 64 + 56 = 300$

Tabla 32.2.: Tabla de supervivencia construida por el método actuarial.

Periodo	Vivos	Muertes	Pérdidas/muertes otras causas	Vivos al final del estudio	q	p	S(t)
1	300	103	23	34	0.379	0.621	0.621
2	140	26	8	25	0.211	0.789	0.490
3	81	9	3	23	0.132	0.868	0.425
4	46	5	2	20	0.143	0.857	0.364
5	19	1	0	18	0.100	0.900	0.368

- Durante el primer año de seguimiento murieron 103 pacientes: $21 + 23 + 18 + 22 + 19 = 103$

Para estimar la probabilidad de morir en un período, se divide el número de muertes en cada período por el número de individuos en riesgo al comienzo del periodo de observación.

Pero; ¿qué hacemos con los sujetos que se perdieron, es decir los individuos censurados? Al clasificar los datos en intervalos de tiempo discretos (como hemos hecho en nuestra tabla), no podremos decir en qué momento se han perdido exactamente: ¿diremos que han sido individuos en riesgo durante todo el año (o período? Lo que el método actuarial asume es una solución “salomónica”: las observaciones censuradas, que se contabilizan como media persona-año:

$$P(morir) = \frac{muertes\ en\ el\ perodo}{indiv.en\ riesgo\ al\ comienzo\ del\ perodo - \frac{indiv.censurados}{2}}$$

Recordemos que los individuos censurados serán quienes están vivos al final del periodo, los que se han perdido y quienes han muerto por causas ajenas al estudio. En nuestra nueva tabla (Tabla 2), esta probabilidad se notará como q_i

Por lo tanto, para el primer periodo:

$$q_i = \frac{103}{300 - \frac{23+34}{2}} = 0,379$$

La probabilidad de supervivencia condicional, p_i, que denota la probabilidad de sobrevivir a un determinado periodo, se calcula como:

$$p_i = 1 - q_i$$

La probabilidad acumulada de supervivencia $S(t)$, que muestra la probabilidad de sobrevivir hasta un tiempo determinado, considerando que se ha sobrevidido a los tiempos anteriores se obtiene multiplicando las probabilidades de supervivencia en los períodos anteriores. Es decir que la probabilidad de sobrevivir al tercer año a partir del trasplante será:

$$S(3) = 0,621 * 0,789 * 0,868 = 0,425\ (42,5\,\%)$$

Esto indica que el 42,5 % de los pacientes sobreviven al menos tres años luego del trasplante:

Puede luego graficarse $S(t)$ vs periodo (Figura 32.4). Sobre el gráfico se pueden ubicar los cuartiles y en particular, la mediana de los tiempos de supervivencia, que indica el momento donde la supervivencia es del 50 %.

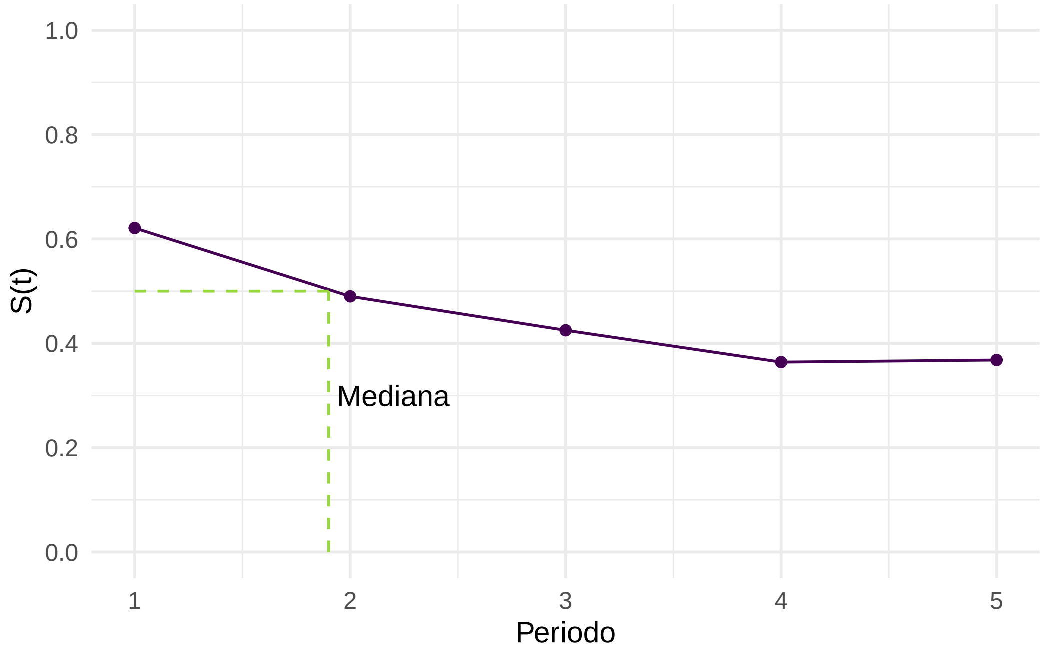


Figura 32.4.: Función de supervivencia estimada a partir de la observación de 300 personas post transplante hepático.

32.3.2. Método de Kaplan-Meier

El método de Kaplan-Meier (KM) utiliza una lógica similar al actuarial, pero permite trabajar con datos individuales y periodos más cortos, siendo particularmente útil cuando el tamaño muestral es reducido. La probabilidad de supervivencia al tiempo t se estima considerando que es independiente de la supervivencia a otros tiempos, entonces la probabilidad de alcanzar el tiempo t es el producto de alcanzar los tiempos anteriores:

$$S_{KM}(t) = \prod_{i:ti<t} (1 - d_i/n_1)$$

donde d_i es el número de eventos y n_i el número de personas expuestas en el tiempo t_i.

La característica distintiva es que la proporción acumulada que sobrevive se calcula para el tiempo de supervivencia individual de cada paciente y no se agrupan los tiempos de supervivencia en intervalos.

Antes de continuar con el ejemplo se sugiere ver la siguiente videoclase:

▯ Método de Kaplan-Meier

Para nuestro ejemplo consideraremos una cohorte cerrada de 10 pacientes con dos años de seguimiento. El evento es muerte por enfermedad cardiovascular (ECV). En la siguiente figura se muestra el esquema del seguimiento, donde “C” indica censura y “M” el evento:

La información de esta figura se resume en la Tabla 32.3. Es importante señalar que la base de datos para análisis de supervivencia debe tener la siguiente estructura: cada fila es un individuo en seguimiento, debe consignarse el tiempo inicial y el tiempo final del seguimiento (en este caso, el tiempo inicial es el mismo para todos porque se trata de una cohorte cerrada, pero esto no es lo más frecuente), el tiempo de seguimiento t (se obtiene de la diferencia tiempo final y tiempo inicial) y la variable `status`, que indica si el paciente sufrió el evento (`status:1`) o es un paciente censurado (`status:0`).

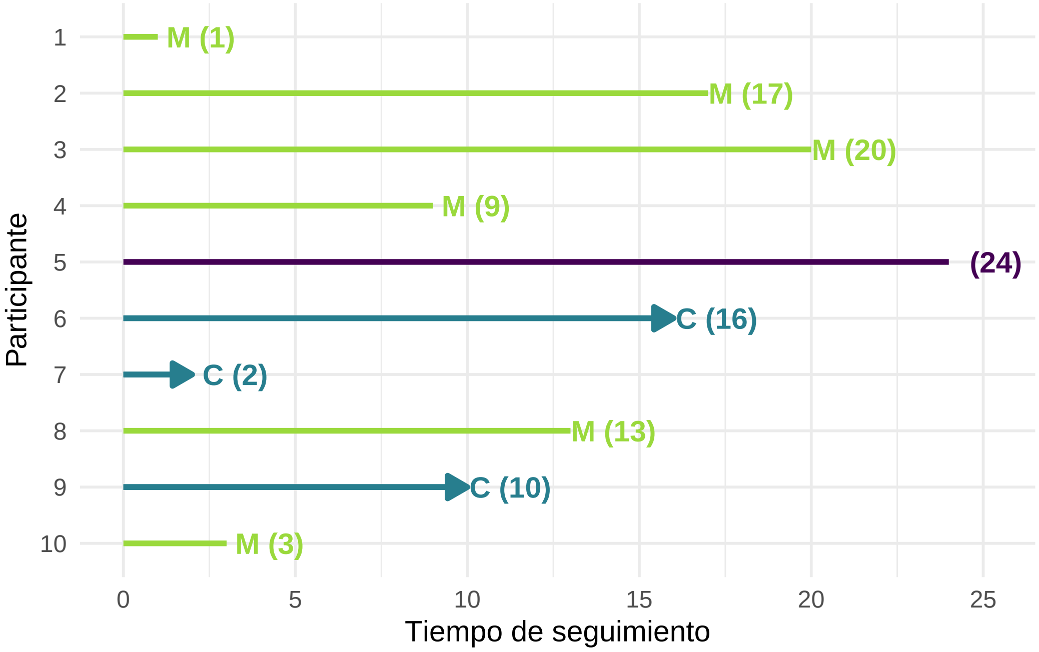


Figura 32.5.: Cohorte cerrada de 10 pacientes en seguimiento.

Tabla 32.3.: Datos de 10 pacientes en seguimiento para ECV.

Paciente	Tiempo inicial	Tiempo final	t (meses)	status
1	0	1	1	1
2	0	17	17	1
3	0	20	20	1
4	0	9	9	1
5	0	24	24	0
6	0	16	16	0
7	0	2	2	0
8	0	13	13	1
9	0	10	10	0
10	0	3	3	1

Tabla 32.4.: Pacientes en seguimientos por ECV, ordenados según tiempo de seguimiento.

Paciente	tiempo	status
1	1	1
7	2	0
10	3	1
4	9	1
9	10	0
8	13	1
6	16	0
2	17	1
3	20	1
5	24	0

Tabla 32.5.: Tabla de supervivencia según KM para pacientes con ECV.

Tiempo	Individuos en riesgo	Eventos	Prob. evento	Prob. supervivencia	S(t)
0	10	0	0.000	1.000	1.000
1	10	1	0.100	0.900	0.900
3	8	1	0.125	0.875	0.787
9	7	1	0.143	0.857	0.675
13	5	1	0.200	0.800	0.540
17	3	1	0.333	0.667	0.360
20	2	1	0.500	0.500	0.180

Para la construcción de la tabla de supervivencia, lo primero que debemos hacer es ordenar los pacientes de acuerdo a su tiempo.

Entonces, la Tabla 32.3, se transforma en la Tabla 32.4:

Comencemos con la construcción de la tabla de supervivencia (Tabla 32.5), teniendo en cuenta que aquí conocemos exactamente el tiempo en el que un individuo estuvo en riesgo de morir por ECV. Esta tabla se construye a partir de los individuos que tuvieron el evento.

Una vez obtenida la $S(t)$, se grafica:

```
NULL
```

Las curvas de supervivencia de Kaplan-Meier son representaciones escalonadas que describen cómo evoluciona la supervivencia acumulada a lo largo del tiempo. Cada descenso en la curva —es decir, cada "escalón"— se produce cuando ocurre un evento (por ejemplo, una muerte), lo que refleja una disminución en la probabilidad de supervivencia acumulada hasta ese instante.

Resumiendo, para construir una curva de supervivencia se deben dar los siguientes pasos:

a. Ordenar los tiempos de supervivencia (o tiempos de observación) en forma ascendente.
b. Construir una tabla que contenga: una columna con los tiempos observados para cada participante (t_i) y una columna con el estado del individuo al final del seguimiento (status).

c. Calcular la probabilidad de supervivencia en cada tiempo (s_i/n_i), es decir, el cociente entre el número de individuos que sobreviven (s_i) y el número de individuos en riesgo de sufrir el evento en ese instante (n_i). Esta columna sólo tendrá valores en los momentos en que ocurre un evento. Es importante señalar que, en el denominador (n_i), se incluyen también los individuos que van a experimentar el evento en ese período, ya que entran al periodo vivos.

d. Obtener la supervivencia acumulada, multiplicando en cada periodo de tiempo los cocientes de supervivencia (s_i/n_i) por los productos acumulados de los tiempos anteriores.

e. Representar gráficamente la curva. La línea comienza en 1 (100 % de supervivencia) y se mantiene constante hasta que ocurre el primer evento, momento en el que desciende un escalón. Este patrón se repite a medida que ocurren nuevos eventos. Por ejemplo, si la supervivencia cae a 0,90 tras el primer evento, ese valor se mantendrá hasta el siguiente.

Algunos investigadores optan por representar también **curvas de incidencia acumulada**, que refleja la probabilidad de experimentar el evento a lo largo del tiempo. Esta también puede calcularse a partir de la tabla de vida generada mediante el método de Kaplan-Meier. Por ejemplo, en la Tabla 32.5, bastaría con usar la columna "Probabilidad del evento" y obtener la probabilidad acumulada multiplicando las probabilidades de distintos tiempos.

Hasta aquí aprendimos a calcular la $S(t)$ a partir de nuestros datos, ya sea por el método actuarial o por el de KM. Todo esto que hicimos "a mano" aquí, las funciones del lenguaje R lo hacen y nos facilitan el trabajo.

Ahora podríamos plantearnos distintos interrogantes, según el estudio, como por ejemplo:

- ¿La curva será una buena estimación tanto para hombres como para mujeres? Es decir: ¿la variable sexo afecta al tiempo de supervivencia?
- O también: ¿el tratamiento afecta el tiempo de supervivencia? Para ello será necesario comparar curvas de supervivencia.

Responder estas preguntas requiere comparar curvas de supervivencia entre grupos, lo cual veremos a continuación.

32.4. Comparación de curvas de supervivencia

Para responder a las preguntas anteriores, la estrategia de KM es estratificar según la variable de interés (sexo en el primer ejemplo, tratamiento en el segundo) y estimar la $S(t)$ para cada estrato. De esta forma podemos comparar las curvas gráficamente.

Por ejemplo, la siguiente figura, nos muestra las curvas de supervivencia para dos grupos de pacientes que recibieron distintos tratamientos (A y B). Si observamos las medianas de los tiempos de supervivencia (líneas punteadas), podemos ver que los pacientes que siguieron el tratamiento B tienen mayores tiempos de supervivencia que los que recibieron el tratamiento A (Mediana tiempos de supervivencia $A \approx 7\ meses$; Mediana tiempos de supervivencia $B \approx 20\ meses$)

Si bien gráficamente podemos tener una idea, lo correcto es hacer una comparación en términos estadísticos. Por lo tanto, existen varios test que nos permiten comparar curvas de supervivencia. Uno de los más usados por los paquetes estadísticos es el log-rank test.

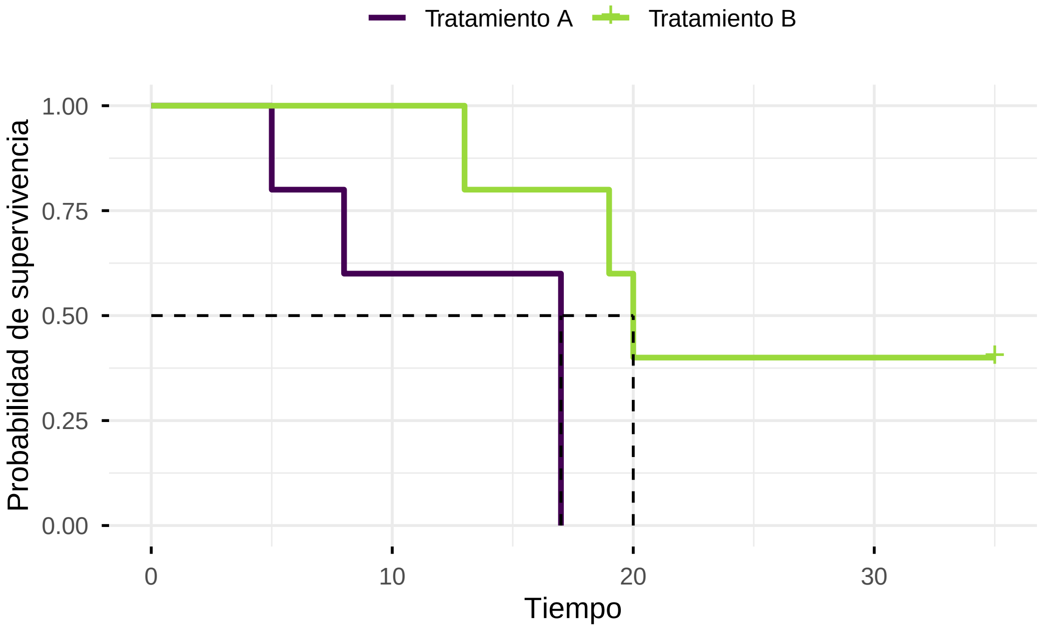


Figura 32.6.: Curvas de KM estratificadas por tipo de tratamiento.

32.4.1. Log-rank test

- La hipótesis nula (H_0) plantea que las supervivencias de los grupos que se comparan (2 o más) son iguales.
- La hipótesis alternativa (H_1) sostiene que al menos uno de los grupos tiene una supervivencia diferente.

El estadístico de contraste utilizado es la **chi-cuadrado** (χ^2) con $k-1$ grados de libertad, donde k representa la cantidad de grupos (es decir, el número de curvas que se comparan). Esta prueba evalúa si la incidencia de eventos es similar en los distintos estratos.

La interpretación del valor p es similar a la de otros tests de hipótesis: típicamente, un valor $p < 0,05$ se considera evidencia suficiente para rechazar la hipótesis nula, indicando diferencias estadísticamente significativas entre los grupos.

En la práctica médica, es frecuente comparar tratamientos o cohortes utilizando como criterio la supervivencia a cinco años. Sin embargo, esta estrategia puede ser engañosa. Es posible que dos curvas muy diferentes reflejen la misma supervivencia en ese único punto temporal. La Figura 32.7 ilustra claramente este problema.

En la Figura 32.7, ambos grupos presentan una supervivencia del 50 % a los 5 años. No obstante, se advierte que el grupo B experimenta una caída más temprana en la probabilidad de supervivencia, lo que indica un peor pronóstico desde los primeros momentos del seguimiento.

Los tests diseñados específicamente para comparar curvas de supervivencia, como el log-rank, consideran todo el trayecto temporal de las curvas, y no únicamente un punto aislado. Esto permite detectar diferencias persistentes en el tiempo.

El test de log-rank (también conocido como test de **Mantel-Haenszel**) es el más adecuado cuando el evento es poco frecuente o cuando las curvas son aproximadamente paralelas.

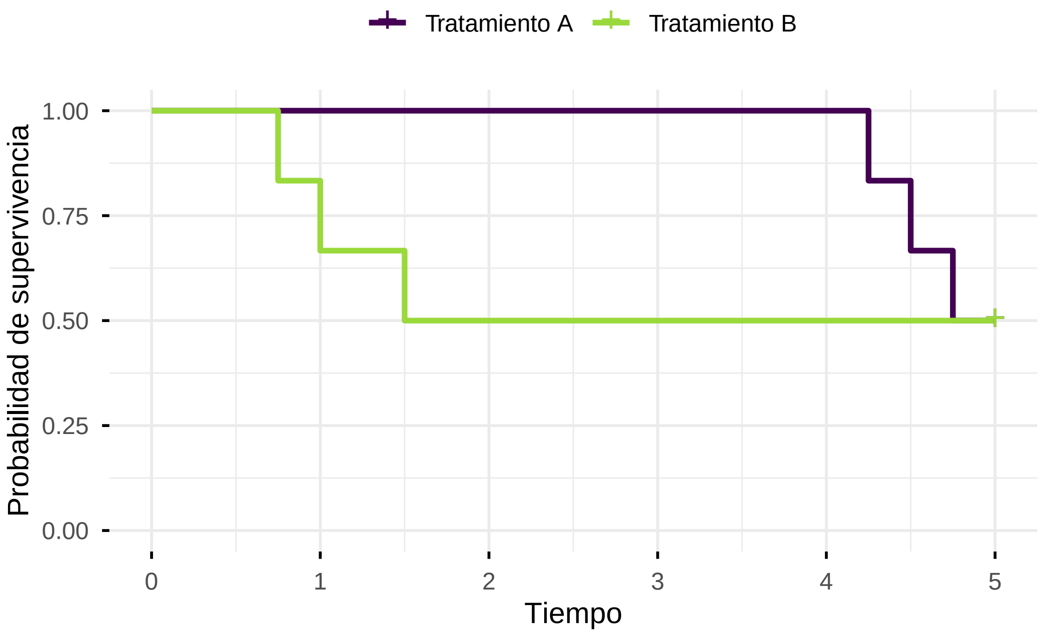


Figura 32.7.: Curvas de supervivencia que exhiben la misma supervivencia a cinco años para diferentes estudios.

Tabla 32.6.: Estimaciones de h(t).

Tiempo	Individuos en riesgo	Eventos	Prob. evento	Prob. supervivencia	lambda(t)
0	10	0	0.000	1.000	0.000
1	10	1	0.100	0.900	0.100
2	8	1	0.125	0.875	0.625
6	7	1	0.143	0.857	0.238

Si las curvas se cruzan en el tiempo —por ejemplo, si un grupo presenta mejor supervivencia inicialmente, pero luego el otro lo supera— conviene emplear el **test de Wilcoxon (o de Breslow)**, que otorga mayor peso a las diferencias tempranas. Un tercer método menos utilizado es el **test de Tarone-Ware**, que aplica una ponderación intermedia donde se le da distinto peso a las diferencias según ocurran más precoz o más tardíamente a lo largo del seguimiento.

32.5. Hazard y Riesgo acumulado

Ya hemos estimado $S(t)$, ahora nos falta abordar las otras funciones básicas de supervivencia

La expresión compleja de $h(t)$, a los efectos de las Tablas de Supervivencia, puede reemplazarse por:

$$\lambda(t) = \frac{Nmero\ de\ \ eventos\ observados\ en\ cada\ intervalo}{personas\ en\ riesgo\ al\ inicio * amplitud\ intervalo}$$

De la misma forma que construimos $S(t)$, podemos estimar $h(t)$ (o un software puede hacerlo por nosotros). La Tabla 32.6, nos muestra las estimaciones para el ejemplo utilizado para KM.

¿Qué expresa $h(t)$ o $\lambda(t)$?

Observemos que la función de riesgo $h(t)$ o $\lambda(t)$ es una medida de la probabilidad de que se produzca el suceso estudiado entre los que quedan sin haber sufrido todavía tal suceso.

Es realmente una función que evalúa, puntualmente, en un período de tiempo determinado, la probabilidad de que un individuo de los que todavía no han sufrido tal suceso lo sufra precisamente en ese período de tiempo. Es, por lo tanto, realmente, una función que mide el riesgo en un período de tiempo concreto.

De forma similar, la expresión para el riesgo acumulado:

$$H(t) = \int_0^t h(u)du$$

puede aproximarse como:

$$\Lambda(t) = \sum \lambda(t) * intervalo$$

No mostramos aquí los cálculos correspondientes para no agobiarlos más, pero pueden intentarlo si están motivados.

32.6. Intervalos de confianza y error estándar para la supervivencia

Si se desea calcular un intervalo de confianza (IC) para la probabilidad de supervivencia estimada en un tiempo determinado, es necesario considerar su **error estándar (SE)**. El mismo se calcula en función de la supervivencia acumulada hasta ese momento, y representa la incertidumbre asociada a la estimación.

Una fórmula común para el error estándar de la estimación de supervivencia acumulada (S_t) en un tiempo dado es el producto de la supervivencia estimada para ese tiempo por la raíz de la suma de los cocientes entre el número de fallecidos en cada momento y el producto de supervivientes y pacientes en riesgo en ese tiempo:

$$SE_{S_t} = S_t \sqrt{\sum \frac{n_i - s_i}{n_i s_i}}$$

Para calcular el 95 % IC, podríamos pensar en usar la fórmula que vimos inicialmente:

$$95\,\%\, IC_{S_t} = S_t \pm 1,96SE \qquad donde\; 1,96\; es\; el\; valor\; Z\; para\; un\; alfa\; bilateral\; del\; 5\,\%$$

Con esta fórmula, en algunos casos se podría obtener, un límite inferior negativo o un límite superior mayor que 1. Ambos resultados carecen de sentido en un contexto de probabilidades.

Para evitar estos problemas, se utiliza una **transformación logarítmica** que estabiliza la varianza. La fórmula ajustada para el error estándar transformado es:

$$SE_t = \sqrt{\frac{1}{(ln[S])^2} \times \sum \frac{n_i - s_i}{n_i s_i}}$$

Y a partir de este SE_t, calcular el intervalo de confianza al 95 %, según la expresión:

$$95\,\%\,IC = SE_t * exp(\pm 1,96 SE_t)$$

Aunque todos estos cálculos pueden realizarse manualmente, **las funciones del lenguaje R ya los incluyen** al utilizar modelos de supervivencia. El propósito aquí es mostrar la lógica matemática detrás de dichas funciones.

32.7. Algunas otras medidas de supervivencia

En la literatura científica, muchas veces nos encontramos con otras medidas de supervivencia, que puntualizaremos a continuación:

- **Supervivencia a 5 años**: es el número de personas aún vivas después de 5 años del diagnóstico.
- **Supervivencia media**: tener en cuenta sólo aquellos individuos de los que se tienen datos completos y se conoce con exactitud su situación clínica. Esta se calcula por:

$$S\ media = \frac{\sum tiempo\ hasta\ alcanzar\ el\ evento}{numero\ total\ de\ individuos\ que\ tuvieron\ el\ evento}$$

- **Tasa de supervivencia**: Se obtiene según la fórmula

$$Tasa\ de\ supervivencia = \frac{nro.\ de\ individuos\ que\ sobreviven\ hasta\ un\ tiempo\ t}{nro.\ total\ de\ individuos}$$

- **Tasa de Incidencia**: Puede calcularse también una tasa de incidencia del evento de interés

$$Tasa\ de\ incidencia = \frac{numero\ de\ eventos}{\sum tiempo - persona\ en\ seguimiento}$$

32.8. Ejemplo práctico en R

A continuación, veremos un ejemplo para aprender a realizar un análisis de supervivencia en R, utilizando datos de pacientes con **leucemia mieloide crónica (LMC)** almacenados en el archivo "`tmo.txt`".

Hasta hace pocos años, el **trasplante de médula ósea (TMO)** a partir de un donante compatible era el único tratamiento disponible para la LMC. Sin embargo, este procedimiento presenta diversos desafíos: una de las complicaciones más frecuentes es la enfermedad injerto contra huésped, que afecta entre el 25 y el 30 % de los pacientes trasplantados. También pueden presentarse efectos adversos vinculados a tratamientos previos como la quimioterapia o la radioterapia. El pronóstico de los pacientes sometidos a TMO se asocia además, con la fase de la enfermedad en el momento del trasplante: aquellos trasplantados durante la fase crónica inicial suelen presentar una mayor probabilidad de supervivencia.

El Centro de Trasplante de Médula Ósea del Instituto Nacional del Cáncer de Brasil, entre 1986 y 1998 realizó 96 trasplantes de médula ósea para tratamiento de LMC. El análisis de los datos de esta cohorte tuvo como objetivo identificar factores que pudieran estar relacionados con el pronóstico, ya sea para orientar intervenciones profilácticas o terapéuticas que mejoren los resultados, o para evaluar la conveniencia de alternativas terapéuticas en subgrupos de alto riesgo.

Existen numerosas librerías para el análisis de supervivencia, nosotros aquí usaremos los paquetes `survival` (Therneau 2026) y `survminer`(Kassambara et al. 2026), este último provee funciones que facilitan el análisis y la visualización de datos de supervivencia. Comenzamos cargando los paquetes:

```
library(survival)
library(survminer)
library(tidyverse)
```

A continuación cargamos los datos y exploramos su estructura:

```
datos <- read_csv2("datos/tmo.txt")
```

Las variables presentes en el conjunto de datos son las siguientes:

- `id`: identificador único del paciente
- `sexo`: sexo biológico (M = masculino, F = femenino)
- `edad`: edad en años al momento del trasplante
- `status`: estado al final del seguimiento (1 = fallecido, 0 = censurado)
- `tiempo`: tiempo hasta la fecha de óbito o censura
- `deag`: presencia de enfermedad injerto-huesped aguda
- `decr`: presencia de enfermedad injerto-huesped crónica
- `fase`: fase de la LMC (aguda, crónica, crisis blástica)

El evento de interés es “*muerte pos-trasplante*”. Comenzaremos por utilizar la función `Surv()`, que toma como argumentos `time` y `event` y devuelve un objeto de supervivencia que se utiliza como insumo para los análisis posteriores. La función `survfit()` toma ese objeto y construye las curvas de supervivencia correspondientes.

Nos centraremos en la construcción de objetos de supervivencia a partir de **datos con censura a la derecha**, ya que es el tipo más común en estudios longitudinales. En este tipo de censura, solo necesitamos dos argumentos para `Surv()`. Para explorar otras formas de censura (por ejemplo, censura por intervalo o truncamiento), se recomienda consultar la documentación del paquete `survival`.

Generamos el objeto de supervivencia usando las variables `tiempo` y `status`:

```
global <- Surv(time = datos$tiempo, event = datos$status)

# Explorar el objeto de supervivencia
global
```

```
 [1] 1000+   39   434    69   672    98  1000+  415   261    65  1000+ 1000+
[13] 1000+  347   453   281  1000+  185  1000+   79  1000+ 1000+  475  1000+
[25]  100   313  1000+  128   427    84   214   200   120    83  1000+  149
[37] 1000+   70  1000+ 1000+   54   101   162   216    63  1000+ 1000+ 1000+
[49] 1000+ 1000+  571+   71  1000+   31   907+  210    32   865+  139   980+
[61]  320+   78    40   753+  846+  774+  522    48   587+   76   468+  434
[73]  452+  487+  586   536+   89+  425   531+  371   489+  445+  243+  362+
[85]  370   263   383+   74   229+  236+  104   110+   54+   54+   32   342+
```

Al llamar el objeto, veremos que algunas observaciones aparecen con el símbolo +, indicando que han sido **censuradas** a la derecha.

La función `survfit()` crea curvas de supervivencia a partir de una fórmula. A continuación, ajustaremos un modelo Kaplan-Meier para estimar la curva de supervivencia global:

```
KM <- survfit(global ~ 1)

# Explorar el modelo de supervivencia
KM
```

```
Call: survfit(formula = global ~ 1)

      n events median 0.95LCL 0.95UCL
[1,] 96     49    453     370      NA
```

Algunos de los argumentos opcionales de la función son los siguientes:

- `conf.int`: permite modificar el nivel de confianza (por ejemplo, `conf.int = 0.90`)
- `conf.type`: define el método de construcción del intervalo:
  - `"log"` (por defecto): usa transformación logarítmica: $g(t) = log(t)$
  - `"log-log"`: usa $g(t) = log(-log(t))$
  - `"plain"`: intervalo lineal sin transformación.

El objeto `KM` contiene información detallada: número de observaciones, cantidad de eventos observados, mediana del tiempo de supervivencia y su intervalo de confianza. Además, al tratarse de un objeto de tipo lista, podemos acceder a información adicional con los siguiente comandos:

```
# devuelve las estimaciones de Kaplan-Meier a cada t_i
KM$surv
```

```
 [1] 0.9895833 0.9687500 0.9583333 0.9479167 0.9375000 0.9270833 0.9164272
 [8] 0.9057711 0.8951149 0.8844588 0.8738027 0.8631466 0.8524904 0.8418343
[15] 0.8311782 0.8205220 0.8098659 0.8098659 0.7990677 0.7882695 0.7774713
[22] 0.7666731 0.7666731 0.7557206 0.7447681 0.7338156 0.7228632 0.7119107
[29] 0.7009582 0.6900057 0.6790533 0.6681008 0.6571483 0.6571483 0.6571483
[36] 0.6571483 0.6456194 0.6340905 0.6225616 0.6110327 0.6110327 0.6110327
[43] 0.5990516 0.5990516 0.5868261 0.5746005 0.5746005 0.5621092 0.5496179
[50] 0.5371266 0.5121440 0.5121440 0.5121440 0.4990121 0.4990121 0.4855252
[57] 0.4855252 0.4855252 0.4712451 0.4712451 0.4712451 0.4712451 0.4555369
[64] 0.4555369 0.4392677 0.4392677 0.4392677 0.4392677 0.4392677 0.4392677
[71] 0.4392677 0.4392677
```

```
# {t_i}
KM$time
```

```
 [1]   31   32   39   40   48   54   63   65   69   70   71   74   76   78   79
[16]   83   84   89   98  100  101  104  110  120  128  139  149  162  185  200
[31]  210  214  216  229  236  243  261  263  281  313  320  342  347  362  370
[46]  371  383  415  425  427  434  445  452  453  468  475  487  489  522  531
[61]  536  571  586  587  672  753  774  846  865  907  980 1000
```

```
# {Y_i}
KM$n.risk
```

```
 [1] 96 95 93 92 91 90 87 86 85 84 83 82 81 80 79 78 77 76 75 74 73 72 71 70 69
[26] 68 67 66 65 64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49 48 47 46 45 44
[51] 43 41 40 39 38 37 36 35 34 33 32 31 30 29 28 27 26 25 24 23 22 21
```

```
# {d_i}
KM$n.event
```

```
 [1] 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 0 0 1 1
[39] 1 1 0 0 1 0 1 1 0 1 1 1 2 0 0 1 0 1 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0
```

```
# error estándar de las estimaciones de  K-M a {t_i}
KM$std.err
```

```
 [1] 0.01047135 0.01833089 0.02128141 0.02392372 0.02635231 0.02862321
 [7] 0.03086977 0.03301123 0.03506847 0.03705709 0.03898920 0.04087437
[13] 0.04272043 0.04453385 0.04632006 0.04808377 0.04982901 0.04982901
[19] 0.05160533 0.05336878 0.05512246 0.05686918 0.05686918 0.05866126
[25] 0.06045058 0.06223956 0.06403050 0.06582557 0.06762684 0.06943634
[31] 0.07125603 0.07308784 0.07493368 0.07493368 0.07493368 0.07493368
[37] 0.07699571 0.07907601 0.08117706 0.08330139 0.08330139 0.08330139
[43] 0.08562288 0.08562288 0.08807070 0.09055225 0.09055225 0.09318156
[49] 0.09585329 0.09857177 0.10416729 0.10416729 0.10416729 0.10735729
[55] 0.10735729 0.11079864 0.11079864 0.11079864 0.11475018 0.11475018
[61] 0.11475018 0.11475018 0.11965379 0.11965379 0.12505911 0.12505911
[67] 0.12505911 0.12505911 0.12505911 0.12505911 0.12505911 0.12505911
```

```
# estimaciones puntuales inferiores (alternativamente, $upper)
KM$lower
```

```
 [1] 0.9694806 0.9345627 0.9191827 0.9044952 0.8903077 0.8765055 0.8626241
 [8] 0.8490226 0.8356578 0.8224974 0.8095165 0.7966952 0.7840176 0.7714705
[15] 0.7590431 0.7467260 0.7345115 0.7345115 0.7221992 0.7099816 0.6978531
[22] 0.6858088 0.6858088 0.6736413 0.6615542 0.6495440 0.6376072 0.6257411
[29] 0.6139430 0.6022106 0.5905417 0.5789346 0.5673875 0.5673875 0.5673875
[36] 0.5673875 0.5551850 0.5430523 0.5309875 0.5189890 0.5189890 0.5189890
[43] 0.5065029 0.5065029 0.4937914 0.4811581 0.4811581 0.4682787 0.4554811
[50] 0.4427639 0.4175656 0.4175656 0.4175656 0.4043229 0.4043229 0.3907508
[57] 0.3907508 0.3907508 0.3763321 0.3763321 0.3763321 0.3763321 0.3603082
[64] 0.3603082 0.3437786 0.3437786 0.3437786 0.3437786 0.3437786 0.3437786
[71] 0.3437786 0.3437786
```

También puede obtenerse un resumen completo mediante:

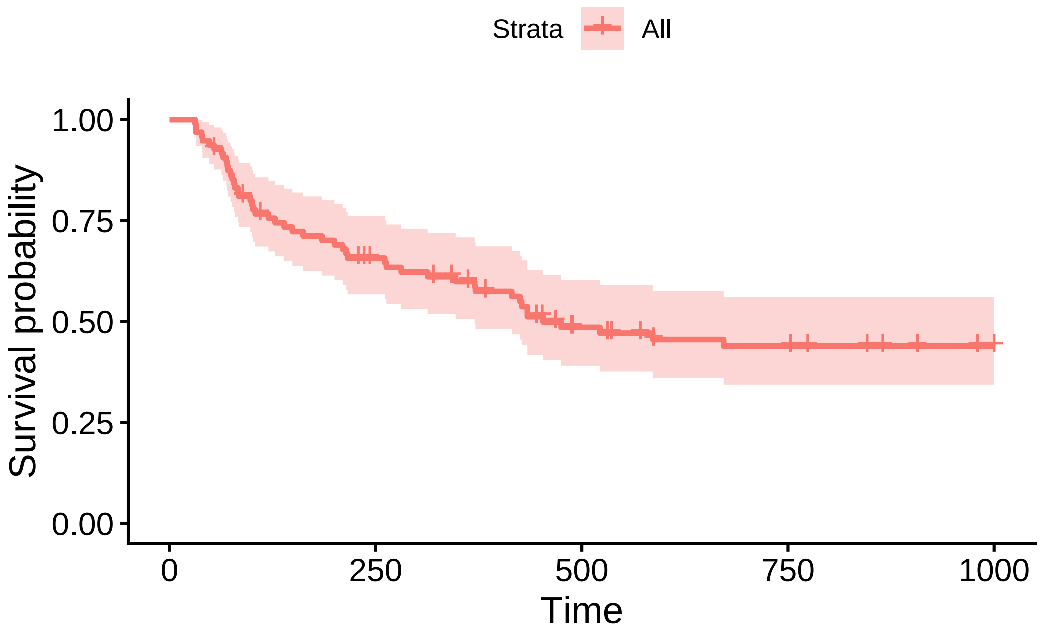
```
summary(KM)
```

```
Call: survfit(formula = global ~ 1)

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
   31     96       1    0.990  0.0104        0.969        1.000
   32     95       2    0.969  0.0178        0.935        1.000
   39     93       1    0.958  0.0204        0.919        0.999
   40     92       1    0.948  0.0227        0.904        0.993
   48     91       1    0.938  0.0247        0.890        0.987
   54     90       1    0.927  0.0265        0.877        0.981
   63     87       1    0.916  0.0283        0.863        0.974
   65     86       1    0.906  0.0299        0.849        0.966
   69     85       1    0.895  0.0314        0.836        0.959
   70     84       1    0.884  0.0328        0.822        0.951
   71     83       1    0.874  0.0341        0.810        0.943
   74     82       1    0.863  0.0353        0.797        0.935
   76     81       1    0.852  0.0364        0.784        0.927
   78     80       1    0.842  0.0375        0.771        0.919
   79     79       1    0.831  0.0385        0.759        0.910
   83     78       1    0.821  0.0395        0.747        0.902
   84     77       1    0.810  0.0404        0.735        0.893
   98     75       1    0.799  0.0412        0.722        0.884
  100     74       1    0.788  0.0421        0.710        0.875
  101     73       1    0.777  0.0429        0.698        0.866
  104     72       1    0.767  0.0436        0.686        0.857
  120     70       1    0.756  0.0443        0.674        0.848
  128     69       1    0.745  0.0450        0.662        0.838
  139     68       1    0.734  0.0457        0.650        0.829
  149     67       1    0.723  0.0463        0.638        0.820
  162     66       1    0.712  0.0469        0.626        0.810
  185     65       1    0.701  0.0474        0.614        0.800
  200     64       1    0.690  0.0479        0.602        0.791
  210     63       1    0.679  0.0484        0.591        0.781
  214     62       1    0.668  0.0488        0.579        0.771
  216     61       1    0.657  0.0492        0.567        0.761
  261     57       1    0.646  0.0497        0.555        0.751
  263     56       1    0.634  0.0501        0.543        0.740
  281     55       1    0.623  0.0505        0.531        0.730
  313     54       1    0.611  0.0509        0.519        0.719
  347     51       1    0.599  0.0513        0.507        0.709
  370     49       1    0.587  0.0517        0.494        0.697
  371     48       1    0.575  0.0520        0.481        0.686
  415     46       1    0.562  0.0524        0.468        0.675
  425     45       1    0.550  0.0527        0.455        0.663
  427     44       1    0.537  0.0529        0.443        0.652
  434     43       2    0.512  0.0533        0.418        0.628
  453     39       1    0.499  0.0536        0.404        0.616
```

```
475     37        1    0.486   0.0538        0.391        0.603
522     34        1    0.471   0.0541        0.376        0.590
586     30        1    0.456   0.0545        0.360        0.576
672     28        1    0.439   0.0549        0.344        0.561
```

Para graficar la función de supervivencia usaremos la función `ggsurvplot()` del paquete `survminer`:

```
ggsurvplot(KM, data = datos)
```



Como se esperaba, la gráfica nos muestra las probabilidades decrecientes de supervivencia a medida que pasa el tiempo. El sombreado alrededor de la curva representa el intervalo de confianza para cada estimación puntual. Más adelante exploraremos cómo personalizar estos gráficos para mejorar su presentación e interpretación.

32.8.1. Comparación de curvas de supervivencia

Intentaremos ahora verificar los factores pronósticos para el tiempo de supervivencia pos-trasplante. Recordemos que el evento a considerar es "*muerte pos-trasplante*". Como herramienta exploratoria, utilizaremos las curvas de supervivencia estimadas por el método de Kaplan Meier según las covariables: `sexo`, Enfermedad injerto crónica (`decr`), Enfermedad injerto aguda (`deag`) y `fase`. Esta parte es equivalente al análisis bivariado que realizamos en regresión lineal y logística. Recordemos que la variable que representa los tiempos de supervivencia es `tiempo` y la que representa el evento o la censura es `status`.

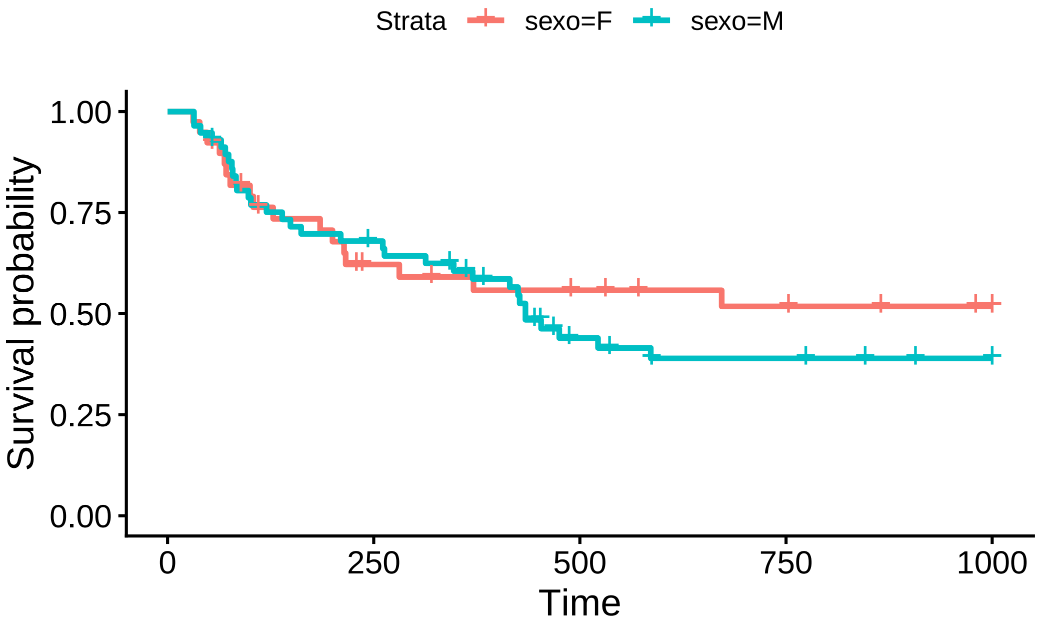
Comenzaremos evaluando las curvas de Kaplan Meier en forma visual, y las completaremos con la información del log-rank test, que nos permite comparar dos funciones de supervivencia.

Definimos la función de supervivencia para `sexo`, usando el objeto de supervivencia creado anteriormente:

```
kmsex <- survfit(global ~ sexo, data = datos)
```

Graficamos la curva de supervivencia:

```
ggsurvplot(kmsex, data = datos)
```



Para comparar ambas funciones de supervivencia se debe ejecutar la función `survdiff()`:

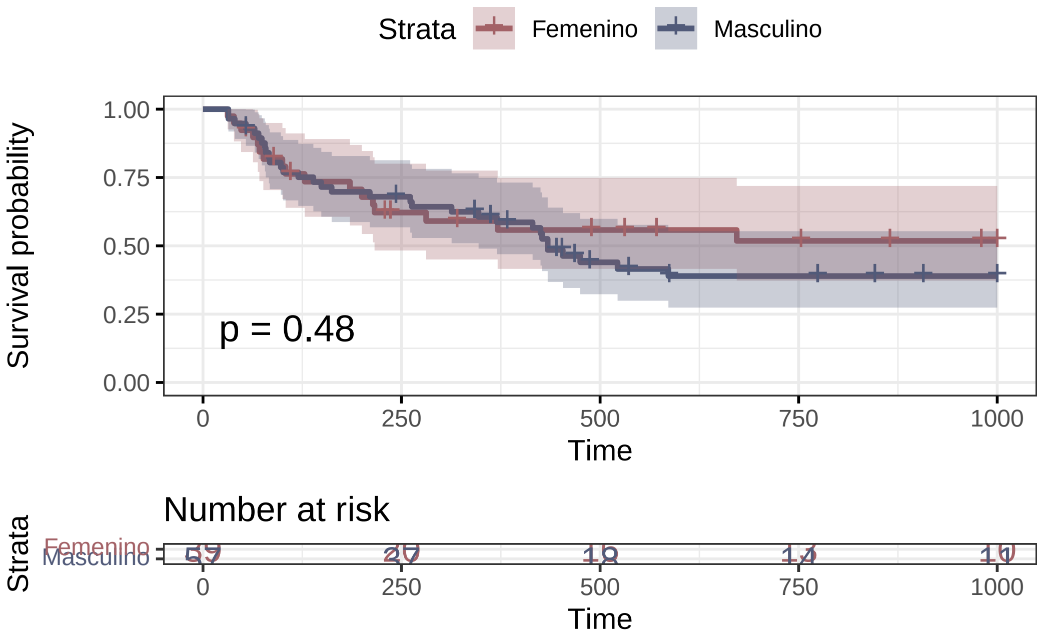
```
logrank <- survdiff(global ~ sexo, data = datos)

logrank
```

```
Call:
survdiff(formula = global ~ sexo, data = datos)

        N Observed Expected (O-E)^2/E (O-E)^2/V
sexo=F 39       17     19.4     0.305     0.508
sexo=M 57       32     29.6     0.201     0.508

 Chisq= 0.5  on 1 degrees of freedom, p= 0.5 
```

Podemos resumir toda la información en un único gráfico, e incluso sumarle información adicional, de acuerdo a cómo personalicemos dicho gráfico:

```
ggsurvplot(
  kmsex,
  data = datos,
  size = 1,                                                              ①
  palette = c("#A36267", "#515A79"),                                   ②
  conf.int = TRUE,                                                     ③
  pval = TRUE,                                                         ④
  risk.table = TRUE,                                                   ⑤
  risk.table.col = "strata",                                           ⑥
  legend.labs = c("Femenino", "Masculino"),                            ⑦
  risk.table.height = 0.25,                                            ⑧
  ggtheme = theme_bw()                                                 ⑨
)
```

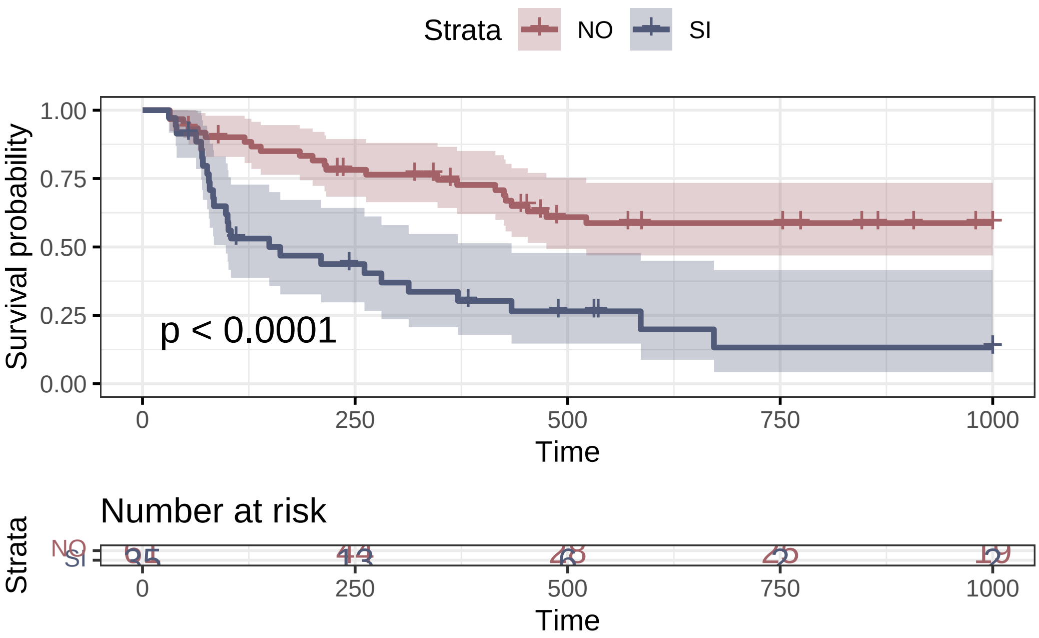
① Cambia el grosor de la línea
② Personaliza las paletas de colores
③ Agrega intervalo de confianza
④ Agrega *p*-valor
⑤ Agrega tabla de riesgo
⑥ Tabla de riesgos con diferente color según grupo
⑦ Cambia etiqueta de la leyenda
⑧ Útil para cambiar cuando se tienen múltiples grupos
⑨ Cambia el tema de `ggplot2`



Hagamos lo mismo para las restantes variables del ejemplo:

Enfermedad injerto aguda:

```
# Función de supervivencia
kmdeag <- survfit(global ~ deag, data = datos)

# Generar curvas de supervivencia
ggsurvplot(
  kmdeag,
  data = datos,
  size = 1,
  palette = c("#A36267", "#515A79"),
  conf.int = TRUE,
  pval = TRUE,
  risk.table = TRUE,
  risk.table.col = "strata",
  legend.labs = c("NO", "SI"),
  risk.table.height = 0.25,
  ggtheme = theme_bw()
)
```

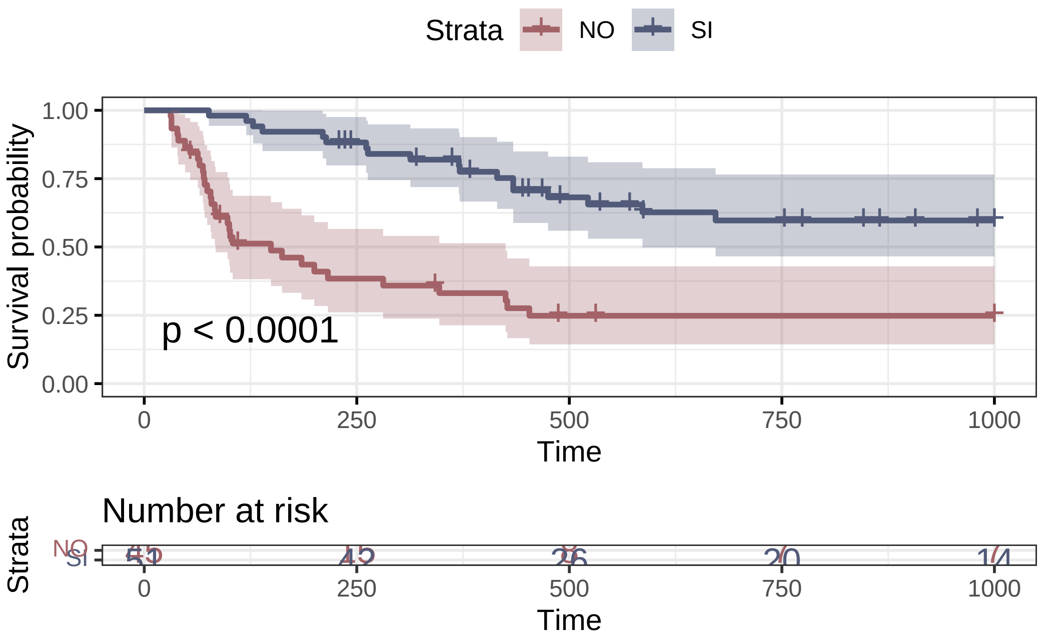


```
# Test de log-rank
logrank <- survdiff(global ~ deag, data = datos)

logrank
```

```
Call:
survdiff(formula = global ~ deag, data = datos)

         N Observed Expected (O-E)^2/E (O-E)^2/V
deag=no 61       23     36.2      4.79      18.8
deag=si 35       26     12.8     13.50      18.8

 Chisq= 18.8  on 1 degrees of freedom, p= 1e-05
```

Enfermedad injerto crónica:

```
# Función de supervivencia
kmdecr <- survfit(global ~ decr, data = datos)

# Generar curvas de supervivencia
ggsurvplot(
  kmdecr,
  data = datos,
  size = 1,
  palette = c("#A36267", "#515A79"),
  conf.int = TRUE,
  pval = TRUE,
  risk.table = TRUE,
  risk.table.col = "strata",
  legend.labs = c("NO", "SI"),
  risk.table.height = 0.25,
  ggtheme = theme_bw()
)
```



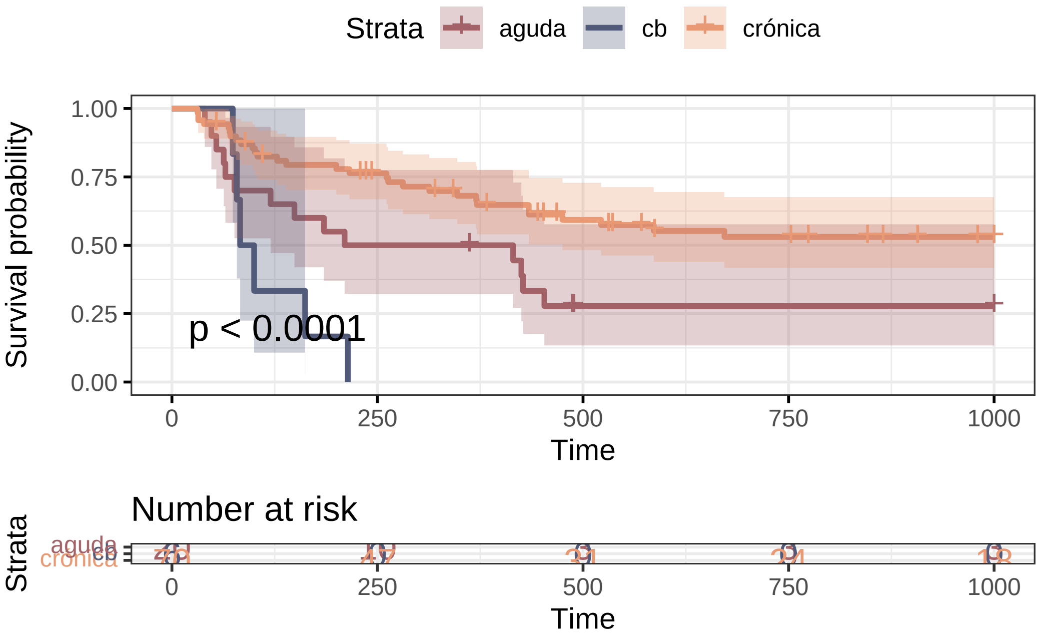
```
# Test de log-rank
logrank <- survdiff(global ~ decr, data = datos)

logrank
```

```
Call:
survdiff(formula = global ~ decr, data = datos)

         N Observed Expected (O-E)^2/E (O-E)^2/V
decr=no 45       31     15.7     14.94      22.6
decr=si 51       18     33.3      7.04      22.6
```

```
 Chisq= 22.6  on 1 degrees of freedom, p= 2e-06
```

Fase de la leucemia:

```
# Función de supervivencia
kmfase <- survfit(global ~ fase, data = datos)

# Generar curvas de supervivencia
ggsurvplot(
  kmfase,
  data = datos,
  size = 1,
  palette = c("#A36267", "#515A79", "#E99973"),
  conf.int = TRUE,
  pval = TRUE,
  risk.table = TRUE,
  risk.table.col = "strata",
  legend.labs = c("aguda", "cb", "crónica"),
  risk.table.height = 0.25,
  ggtheme = theme_bw()
)
```



```
# Test de log-rank
logrank <- survdiff(global ~ fase, data = datos)

logrank
```

```
Call:
```

```
survdiff(formula = global ~ fase, data = datos)

              N Observed Expected (O-E)^2/E (O-E)^2/V
fase=aguda   20       14     8.73      3.19       3.9
fase=cb       6        6     1.50     13.49      14.3
fase=crónica 70       29    38.77      2.46      12.0

 Chisq= 19.7  on 2 degrees of freedom, p= 5e-05
```

¿Qué podemos concluir a partir de las curvas de KM? Observando cada gráfico detenidamente, vemos que:

- Para la variable `sexo`, las curvas se superponen bastante. Es decir que parece razonable suponer que no existe diferencia en la supervivencia pos-trasplante de hombres y mujeres. Dado la forma de las curvas, parecería más correcto emplear el test de Wilcoxon. De todas formas, el valor de ambos test indica lo que ya habíamos predicho observando el gráfico: que no hay diferencia en las curvas de supervivencia entre hombres y mujeres.
- Se observa diferencia en las curvas de supervivencia de los pacientes según hayan padecido o no enfermedad injerto contra huésped aguda (`deag`), con menor supervivencia para quienes la padecieron.
- Se observa diferencia en las curvas de supervivencia de los pacientes según hayan padecido o no enfermedad injerto contra huésped crónica (`decr`), con menor supervivencia para quienes no la padecieron.
- Se observa diferencia en las curvas de supervivencia de los pacientes según la fase de la enfermedad en la que se encontraban al momento del trasplante. Observen que la variable `fase` tiene 3 categorías. El valor del log Rank test nos indica que hay diferencia en la supervivencia de las categorías, pero no nos dice si las 3 son distintas o si sólo una difiere de las otras dos. De la observación del gráfico podemos concluir que los pacientes que sufren crisis blástica (`cb`) tienen una supervivencia menor, de hecho aproximadamente a los 200 días fallecen todos.

Recordamos que KM estratifica por variables cualitativas, por lo tanto `edad` no puede ser evaluada de esta forma. Podría recodificarse si, para el evento en cuestión, el marco teórico señalara alguna edad de corte. Volveremos sobre este ejemplo cuando abordemos el modelo de regresión de Cox.

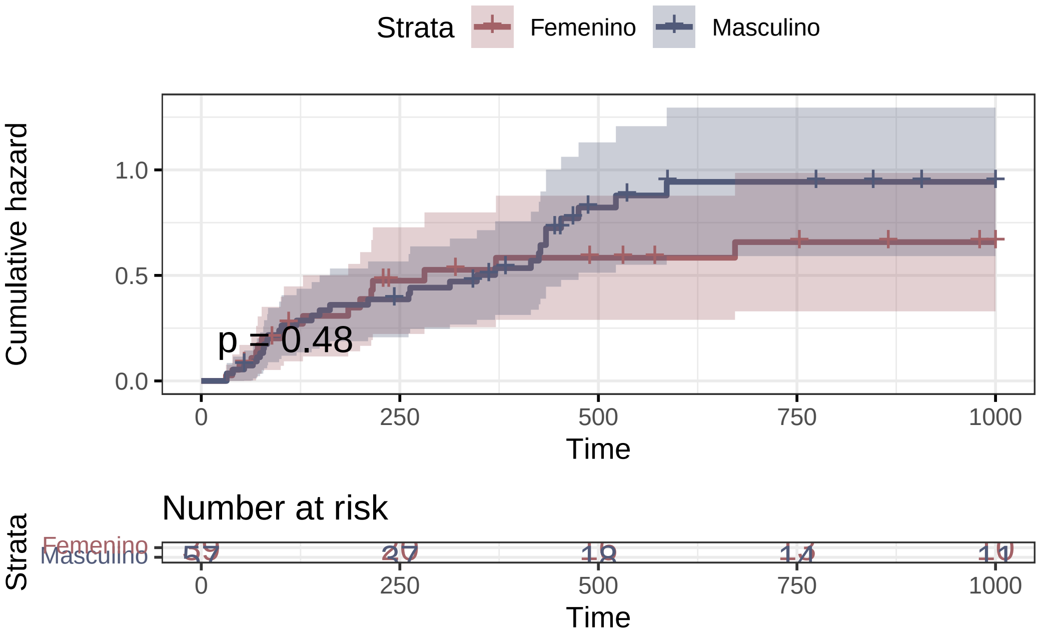
32.8.2. Función de riesgo

En algunas situaciones, dependiendo del evento que estemos considerando, puede resultar más fácil de interpretar la función de riesgo en lugar de la función de supervivencia. En nuestro ejemplo, el evento considerado es el "*tiempo hasta la muerte pos-trasplante*", y en este contexto la función de supervivencia se interpreta fácilmente.

Sin embargo, en otros escenarios —por ejemplo, cuando el evento es la aparición de una infección, una recaída o una complicación— la interpretación de la función de supervivencia puede no ser tan directa. En esos casos, puede ser más útil representar la **función de riesgo acumulado**, que muestra la acumulación del riesgo del evento a lo largo del tiempo.

Una vez definidos los objetos de supervivencia, podemos utilizar la opción `fun = "cumhaz"` dentro de la función `ggsurvplot()` para graficar la función de riesgo acumulado:

```
ggsurvplot(
  kmsex,
  data = datos,
  fun = "cumhaz",
  size = 1,
  palette = c("#A36267", "#515A79"),
  conf.int = TRUE,
  pval = TRUE,
  risk.table = TRUE,
  risk.table.col = "strata",
  legend.labs = c("Femenino", "Masculino"),
  risk.table.height = 0.25,
  ggtheme = theme_bw()
)
```

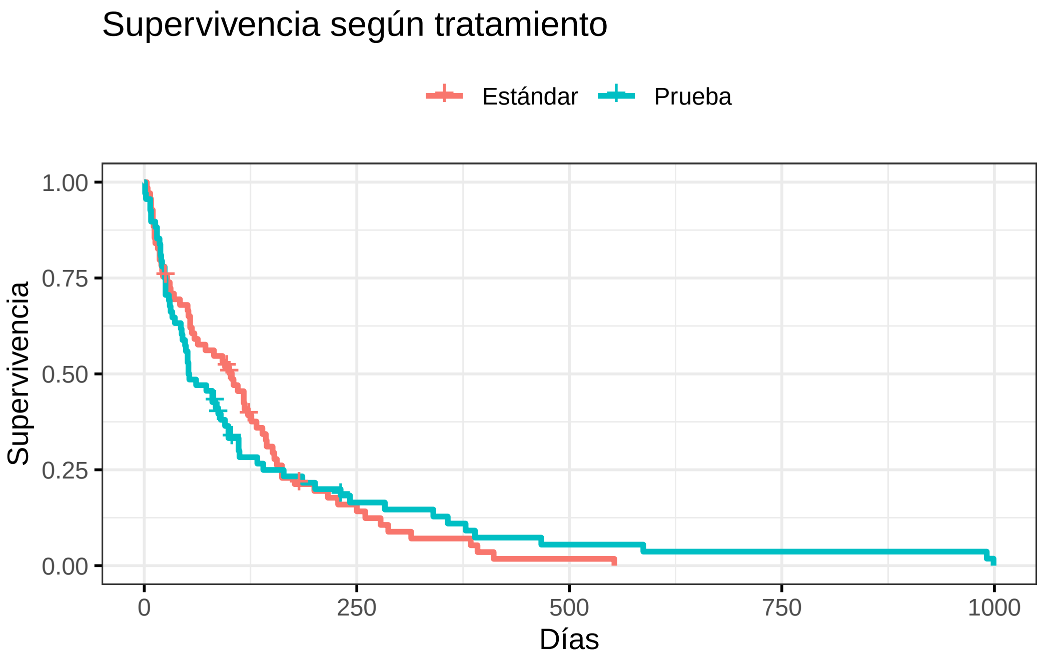


Con el argumento `fun="cumhaz"` le indicamos a `ggsurvplot()` que grafique la función de riesgo acumulado. Este mismo procedimiento puede aplicarse a todos los objetos de supervivencia creados previamente.

33. Regresión de Cox

33.1. Introducción

Hasta el momento hemos visto cómo efectuar estimaciones paramétricas y no paramétricas de la función de supervivencia y de la función de riesgo, que permiten describir la evolución de la "supervivencia" de una población. También hemos visto cómo comparar curvas de supervivencia, al estratificar por las distintas categorías de una variable categórica.



El análisis de supervivencia dispone también de **modelos de regresión**, que permiten identificar y evaluar la relación entre un conjunto de variables explicativas y el tiempo de supervivencia (variable respuesta), por lo que se la considera parte de los modelos lineales generalizados. A continuación, presentaremos uno de los modelos más usados en salud: el modelo de **regresión de Cox**, llamado así en honor a su creador (Cox 1972).

La regresión de Cox, también llamada *modelo de riesgos proporcionales* (*proportional hazards model*) es una técnica muy difundida. Está indicado su uso cuando la variable dependiente esté relacionada con la **supervivencia de un grupo de sujetos** o, en general, con el **tiempo que trascurre hasta que se produce en ellos un suceso o evento**. Como ya dijimos, el evento de interés no tiene por qué ser la muerte, puede ser otro tipo de suceso, por ejemplo, el fallo de una prótesis, la incidencia de una enfermedad o la ocurrencia de una complicación en quien ya tiene una patología de base. Se usa para valorar simultáneamente el efecto independiente de una serie de variables explicativas o factores pronósticos sobre esta supervivencia (es decir, sobre la tasa de mortalidad) o sobre la tasa de ocurrencia de otro fenómeno que vaya ocurriendo tras un periodo de tiempo variable en cada sujeto. Como en cualquier modelo de regresión, las variables explicativas pueden ser tanto cuantitativas como categóricas.

La **regresión de Cox** es la extensión multivariada del **análisis de supervivencia** para evaluar de manera general variables dependientes del tipo “tiempo hasta un suceso o evento” y usa modelos de regresión, próximos al modelo de **regresión logística**. Es una técnica que permite identificar y evaluar la relación entre un conjunto de variables explicativas y la tasa de ocurrencia del suceso de interés.

La mayor utilidad del modelo de regresión de Cox radica en que permite *predecir las probabilidades de supervivencia* (o, en general de permanencia libre del evento) para un determinado sujeto a partir del patrón de valores que presenten sus variables pronósticas.

Al igual que en la regresión logística, en la regresión de Cox también es necesaria una función matemática que transforme el desenlace y permita entender la relación entre las variables por medio de un modelo similar al de la línea recta. Y en esto nos concentraremos a continuación.

33.2. Modelo de Cox

El modelo de Cox es un modelo semi-paramétrico que postula que:

$$h(t,x) = h_o(t) exp(x_1\beta_1 + x_2\beta_2 + \cdots + x_n\beta_n)$$

donde, $h(t,x)$: representa el riesgo de morir al tiempo t de un sujeto que tiene un patrón determinado de las variables x.

$h_0(t)$: representa el riesgo basal dependiente del t.

En este caso, como podemos ver, la función que es “transformada” es la función denominada **Hazard**, que puede simplificarse en términos prácticos como una **medida del riesgo instantáneo de ocurrencia de dicho evento**.

Como podemos ver, este modelo asume un presupuesto importante: las covariables tienen un efecto multiplicativo sobre la función de riesgo. ¿Qué significa esto? Esto significa que la razón entre el riesgo de ocurrencia del evento para dos individuos: individuo i e individuo j, por ejemplo, con covariables:

$xi = (xi_1, xi_2, xi_3, \ldots, xi_n)$ (es el conjunto de covariables del individuo i) $xj = (xj_1, xj_2, xj_3, \ldots, xj_n)$ (es el conjunto de covariables del individuo j) será:

$$\frac{hi(t/xi)}{hj(t/xj)} = \frac{ho(t) exp(xi_1\beta_1 + xi_2\beta_2 + \ldots xi_n\beta_n)}{ho(t) exp(xj_1\beta_1 + xj_2\beta_2 + \ldots xj_n\beta_n)}$$

Entonces:

$$\frac{hi(t/xi)}{hj(t/xj)} = \frac{\cancel{ho(t)} exp(xi_1\beta_1 + xi_2\beta_2 + \ldots xi_n\beta_n)}{\cancel{ho(t)} exp(xj_1\beta_1 + xj_2\beta_2 + \ldots xj_n\beta_n)}$$

O sea que este cociente de riesgos, como podemos ver, es independiente del tiempo. Por este motivo, el modelo de Cox se lo conoce también como **Modelo de Riesgos Proporcionales**. Es un modelo semi-paramétrico, pues no asume ninguna distribución para la función de riesgo basal $ho(t)$, la única asunción es, como ya dijimos antes, que las covariables tienen un efecto multiplicativo sobre la función de riesgo, y esto constituye la parte paramétrica del modelo.

En la regresión de Cox, por lo tanto, el efecto de las variables independientes se presenta como un **Hazard Relativo (HR)**, y expresa la magnitud en la que una variable aumenta o disminuye el riesgo de ocurrencia de un desenlace en el tiempo.

Como en todo modelo, tendremos que estimar los coeficientes (β) y aprender a interpretarlos. A continuación aprenderemos a construir un modelo de Cox, fijaremos criterios para seleccionar el mejor modelo, evaluaremos el ajuste de dicho modelo y veremos algo de análisis de residuos. Allá vamos!

33.2.1. Estimación e interpretación de los coeficientes del modelo de Cox

Los coeficientes del modelo de Cox son estimados por el método de verosimilitud parcial, pero su desarrollo lo omitiremos, dado que los paquetes estadísticos, los calcularán por nosotros. Para comprender su interpretación, consideraremos el modelo más simple, con una única covariable:

$$h(t,x) = ho(t)e^{\beta x}$$

Si se calcula el cociente entre la tasa de riesgo instantáneo para una variable que toma los valores $x = 0$ y $x = 1$:

$$HR = \frac{e^{(\beta*1)}}{e^{(\beta*0)}} = e^{\beta}$$

¿Cómo interpretamos esto? Sé que ya todos tienen mucha experiencia en interpretar correctamente expresiones de este tipo (notarán ciertas similitudes con la regresión logística), pero nunca está de más reforzar.

- Si la variable es *dicotómica*, por ejemplo consideremos *sexo*, donde $x = 1$ corresponde a masculino y $x = 0$ a femenino, diremos que los varones ven incrementado su riesgo en $e\beta$ respecto de las mujeres.
- Si la variable fuera *continua*, como por ejemplo la *edad*, diremos que un incremento de un año en la edad, aumenta el riesgo en $e\beta$. Si calculamos el cambio en h para n variables explicativas, y observamos el coeficiente para cuando cada xi se incrementa en una unidad, manteniendo constante al resto, observaremos algo similar.

Si aplicamos **ln** a la expresión del HR, tendremos que:

$$lnHR = \beta$$

Como acabamos de ver, la interpretación del modelo de Cox no se hace directamente a través de su coeficiente estimado (β) sino del exponencial del coeficiente estimado, $exp(\beta)$.

Para variables dicotómicas $exp(\beta)$ es un estimador de la razón de riesgos (Hazard Ratio) y se interpretará en forma análoga al RR: valores mayores que 1 indican sobreriesgo para el grupo "expuesto", valores menores a 1 indican factor de protección. Para variables continuas, su interpretación es más difícil ya que no es estrictamente un RR, pero lo podemos pensar como algo similar a un RR para un incremento de la variable continua. Para comprender mejor lo que hemos visto hasta aquí y lo que seguiremos viendo, vamos a recurrir a un ejemplo.

33.3. Ejemplo práctico en R

Retomaremos el ejemplo sobre trasplante de médula ósea (TMO), que vimos cuando abordamos la estimación de Kaplan Meier. Recordemos que los datos del archivo "`tmo.txt`" provienen de esa cohorte de 96 pacientes sometidos a trasplante de médula ósea. Las covariables registradas para cada paciente se describen en la siguiente tabla:

Las variables presentes en el conjunto de datos son las siguientes:

- `id`: identificador único del paciente
- `sexo`: sexo biológico (M = masculino, F = femenino)
- `edad`: edad en años al momento del trasplante
- `status`: estado al final del seguimiento (1 = fallecido, 0 = censurado)
- `tiempo`: tiempo hasta la fecha de óbito o censura
- `deag`: presencia de enfermedad injerto-huesped aguda
- `decr`: presencia de enfermedad injerto-huesped crónica
- `fase`: fase de la LMC (aguda, crónica, crisis blástica)

Comenzaremos por activar los paquetes necesarios:

```
library(survival)
library(survminer)
library(easystats)
library(tidyverse)
```

Leemos el archivo de datos

```
# Cargar datos
datos <- read_csv2("datos/tmo.txt")

# Modificar datos
datos <- datos |>
  mutate(across(.cols = c(sexo, deag, decr, fase), .fns = ~ factor(.x)))
```

En general, no se realiza la regresión de Cox con cada una de las variables para decidir cuál incluir en el modelo (puede hacerse para ver el cambio de los coeficientes), sino que *esta decisión se toma en base a las curvas de KM*, considerando las variables en las que se observan diferencias. Como el modelo de Cox, en general se usa con fines predictivos, pueden ser incluidas otras variables que clínicamente se consideren relevantes.

Vamos a considerar un modelo inicial que incluya todas las variables (`m1`) y luego usaremos un procedimiento "hacia atrás" manual. El modelo semiparamétrico de riesgo proporcionales de Cox es realizado con la función `coxph()` del paquete `survival`:

```r
# Ajustar modelo
m1 <- coxph(
  Surv(tiempo, status) ~ fase + decr + deag + sexo + edad,
  data = datos
)

# Salida del modelo
summary(m1)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag + sexo +
    edad, data = datos)

  n= 96, number of events= 49

                 coef exp(coef)  se(coef)      z Pr(>|z|)
fasecb       0.344438  1.411197  0.542408  0.635 0.525417
fasecrónica -0.593973  0.552129  0.371427 -1.599 0.109784
decrsi      -1.061750  0.345850  0.338452 -3.137 0.001706 **
deagsi       1.190381  3.288332  0.327485  3.635 0.000278 ***
sexoM        0.271984  1.312567  0.332115  0.819 0.412816
edad        -0.005019  0.994993  0.014912 -0.337 0.736415
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4112     0.7086    0.4874    4.0859
fasecrónica    0.5521     1.8112    0.2666    1.1434
decrsi         0.3459     2.8914    0.1782    0.6714
deagsi         3.2883     0.3041    1.7307    6.2479
sexoM          1.3126     0.7619    0.6846    2.5166
edad           0.9950     1.0050    0.9663    1.0245

Concordance= 0.768  (se = 0.03 )
Likelihood ratio test= 40.67  on 6 df,   p=3e-07
Wald test            = 38.46  on 6 df,   p=9e-07
Score (logrank) test = 47.62  on 6 df,   p=1e-08
```

- La columna “`coef`” indica los valores de β .
- La columna “`exp (coef)`” indica los valores de HR.
- La columna “`se (coef)`”: desvío estándar de los coeficientes
- `Z` y `Pr (<|Z|)`: estadístico del Test de Wald y su *p*-valor.
- `lower.95` y `upper.95`: 95 % IC para los HR.

En la parte inferior, destacamos el *LR test* del modelo.

Además de lo señalado, se agregan el valor de *Concordancia* así como los *estadísticos de prueba*, *grados de libertad* y *p-valores* de las pruebas de hipótesis de Razón de Verosimilitud, Prueba de Wald y Prueba de Puntajes (*score test*).

Repasemos:

- El **test de Wald** : Testea solo un parámetro, por ejemplo la $Ho : \beta_3 = 0$. El test estadístico es $Z = \frac{\beta}{EE\beta}$ que sigue una distribución aproximadamente $N(0, 1)$. LR y Z son aproximadamente iguales en grandes muestras, pero pueden diferir en muestras pequeñas.
- La **Deviance** puede ser utilizada tanto para obtener una estadística global de ajuste del modelo, como para comparar modelos. Recordemos que: $D = 2(log\ Likelihood\ modelo - log\ Likelihood\ modelo\ nulo)$, donde el Modelo nulo es un modelo teórico sin covariables.

Podemos calcular el **Criterio de Información de Akaike (AIC)** y también el **Criterio de Información Bayesiano (BIC)**. El AIC sirve para comparar la plausibilidad relativa de un conjunto de modelos. Es decir, dado un conjunto de modelos construidos con los mismos datos, el AIC los ordena según su verosimilitud dados los datos con que se construyen. Este criterio tiene en cuenta tanto el ajuste del modelo como su complejidad de acuerdo a la fórmula: $-2 * log(-Likelihood) + k * npar$ donde el primer término mide el ajuste (es la deviance) y el segundo la complejidad ($npar$ es el número de parámetros y $k = 2$).

AIC mide lo lejos que está el modelo de la realidad (de un 'modelo perfecto'), por lo que **cuanto menor es el valor, más plausible es el modelo**. Existen otros criterios parecidos como el BIC (en BIC $k = log(n)$).

AIC no equivale a significación; de hecho este criterio se plantea dentro de una concepción de la estadística distinta –y alternativa– a la fundamentada en testado de hipótesis. El comando es simplemente:

```
# Criterio de información de Akaike
AIC(m1)
```

```
[1] 378.1393
```

```
# Criterio de información Bayesiano
BIC(m1)
```

```
[1] 389.4903
```

El **Likelihood** nos da una evaluación cuantitativa de la compatibilidad de los datos con determinado valor del parámetro.

$$Likelihood\ Ratio = \frac{Likelihood\ del\ modelo}{Mximo\ de\ Likelihood}$$

- El *Likelihood ratio* da una medida más conveniente del soporte de los datos para determinado valor de probabilidades que el likelihood, ya que relativiza los valores de los likelihood al valor del máximo likelihood.
- Cuanto más cerca de 1 está un *Likelihood ratio* significa que ese valor de p está más "sustentado" por los datos.
- El **LR test compara dos modelos anidados**: modelo mayor vs. modelo reducido. La H_0 plantea que algunos parámetros en el modelo completo son igual a 0. Este LR tiene aproximadamente una distribución chi-cuadrado bajo la hipótesis nula.

33.3.1. Analicemos nuestro modelo

Solamente los coeficientes que acompañan a las variables `deag` y `decr` resultaron significativos según el test de Wald. Esto es lo mismo que observamos en las curvas de KM.

Respecto de la variable fase, que tiene 3 categorías, observemos que fue transformada en una variable dummy: el modelo ha tomado como categoría de referencia “aguda” y los valores de HR son en referencia a esta categoría: “crisis blástica” respecto a “aguda” y “crónica” respecto a “aguda”.

La columna `Coefficient` representa los valores de β estimados por verosimilitud parcial. Valores positivos de β indican variables que contribuyen a un incremento de riesgo, en tanto que valores negativos de β indican variables que contribuyen a reducir el riesgo. Si miramos los HR, vemos que `decr` resultó un factor protector importante.

Probaremos un modelo con cuatro variables ¿Cuál saco del modelo: `edad`, `sexo` o `fase`? Probemos distintos modelos:

- **Modelo 2**: `fase + decr + deag + edad` (sacando `sexo`)

```
# Ajustar modelo
m2 <- coxph(
  formula = Surv(tiempo, status) ~ fase + decr + deag + edad,
  data = datos
)

# Salida modelo
summary(m2)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag + edad,
    data = datos)

  n= 96, number of events= 49

                coef exp(coef) se(coef)      z Pr(>|z|)
fasecb       0.34930   1.41808  0.54010  0.647 0.517801
fasecrónica -0.66770   0.51289  0.35813 -1.864 0.062262 .
decrsi      -1.04100   0.35310  0.33283 -3.128 0.001762 **
deagsi       1.15232   3.16554  0.32110  3.589 0.000332 ***
edad        -0.00272   0.99728  0.01452 -0.187 0.851418
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4181     0.7052    0.4920     4.087
fasecrónica    0.5129     1.9498    0.2542     1.035
decrsi         0.3531     2.8321    0.1839     0.678
deagsi         3.1655     0.3159    1.6870     5.940
edad           0.9973     1.0027    0.9693     1.026

Concordance= 0.758  (se = 0.031 )
Likelihood ratio test= 39.98  on 5 df,   p=2e-07
```

```
Wald test            = 38.28  on 5 df,   p=3e-07
Score (logrank) test = 47.16  on 5 df,   p=5e-09
```

```
# AIC
AIC(m2)
```

```
[1] 376.825
```

- **Modelo 3**: `fase + decr + deag + sexo` (sacando edad)

```
m3 <- coxph(
  formula = Surv(tiempo, status) ~ fase + decr + deag + sexo,
  data = datos
)

# Salida modelo
summary(m3)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag + sexo,
    data = datos)

  n= 96, number of events= 49

               coef exp(coef) se(coef)      z Pr(>|z|)
fasecb       0.3378    1.4019   0.5429  0.622 0.533779
fasecrónica -0.5715    0.5647   0.3647 -1.567 0.117090
decrsi      -1.0940    0.3349   0.3246 -3.370 0.000750 ***
deagsi       1.2075    3.3451   0.3240  3.726 0.000194 ***
sexoM        0.2509    1.2852   0.3251  0.772 0.440233
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4019     0.7133    0.4837    4.0631
fasecrónica    0.5647     1.7709    0.2763    1.1540
decrsi         0.3349     2.9863    0.1772    0.6326
deagsi         3.3451     0.2989    1.7725    6.3128
sexoM          1.2852     0.7781    0.6796    2.4306

Concordance= 0.762  (se = 0.03 )
Likelihood ratio test= 40.56  on 5 df,   p=1e-07
Wald test            = 38.19  on 5 df,   p=3e-07
Score (logrank) test = 47.46  on 5 df,   p=5e-09
```

```
# AIC
AIC(m3)
```

```
[1] 376.2529
```

- **Modelo 4**: `sexo + decr + deag + edad` (sacando `fase`)

```
# Ajustar modelo
m4 <- coxph(
  formula = Surv(tiempo, status) ~ decr + deag + sexo + edad,
  data = datos
)

# Salida modelo
summary(m4)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag + sexo + edad,
    data = datos)

  n= 96, number of events= 49

             coef  exp(coef)  se(coef)      z Pr(>|z|)
decrsi -1.3160459  0.2681937  0.3150468 -4.177 2.95e-05 ***
deagsi  1.1988240  3.3162147  0.3084393  3.887 0.000102 ***
sexoM   0.4531812  1.5733092  0.3241990  1.398 0.162158
edad    0.0003653  1.0003654  0.0142724  0.026 0.979578
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2682     3.7286    0.1446    0.4973
deagsi    3.3162     0.3015    1.8118    6.0700
sexoM     1.5733     0.6356    0.8334    2.9701
edad      1.0004     0.9996    0.9728    1.0287

Concordance= 0.732  (se = 0.032 )
Likelihood ratio test= 36.35  on 4 df,   p=2e-07
Wald test            = 32.6  on 4 df,   p=1e-06
Score (logrank) test = 38.43  on 4 df,   p=9e-08
```

```
# AIC
AIC(m4)
```

```
[1] 378.4578
```

De acuerdo al criterio AIC, considerando los modelos construidos, `m3` sería un mejor modelo, dado que tiene un menor AIC, pero se observa que existen términos sin significación en este modelo, según el test de Wald.

Ajustemos ahora un modelo con tres variables, partiendo de `m3`, cuya fórmula era:

```
Surv(tiempo, status) ~ fase + decr + deag + sexo
<environment: 0x00000185dda1ad98>
```

En general, el criterio que se utiliza para “sacar” variables del modelo, es sacar aquellas cuyo β haya resultado no significativo. En el caso de `m3` serían los β correspondientes a las variables `sexo` y `fase`.

- **Modelo 5**: `fase + decr + deag` (sacando `sexo`)

```
# Ajustar modelo
m5 <- coxph(formula = Surv(tiempo, status) ~ fase + decr + deag, data = datos)

# Salida modelo
summary(m5)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag, data = datos)

  n= 96, number of events= 49 

               coef exp(coef) se(coef)      z Pr(>|z|)    
fasecb       0.3449    1.4119   0.5402  0.639 0.523100    
fasecrónica -0.6509    0.5216   0.3467 -1.878 0.060440 .  
decrsi      -1.0597    0.3466   0.3176 -3.336 0.000849 ***
deagsi       1.1633    3.2004   0.3160  3.681 0.000232 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4119     0.7083    0.4898    4.0698
fasecrónica    0.5216     1.9173    0.2644    1.0290
decrsi         0.3466     2.8856    0.1860    0.6458
deagsi         3.2004     0.3125    1.7227    5.9456

Concordance= 0.753  (se = 0.03 )
Likelihood ratio test= 39.95  on 4 df,   p=4e-08
Wald test            = 38.18  on 4 df,   p=1e-07
Score (logrank) test = 47.11  on 4 df,   p=1e-09
```

```
# AIC
AIC(m5)
```

```
[1] 374.8601
```

- **Modelo 6**: `decr + deag + sexo` (sacando `fase`)

```
# Ajustar modelo
m6 <- coxph(formula = Surv(tiempo, status) ~ decr + deag + sexo, data = datos)

# Salida modelo
summary(m6)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag + sexo, data = datos)

  n= 96, number of events= 49 

          coef exp(coef) se(coef)      z Pr(>|z|)    
decrsi -1.3143    0.2687   0.3072 -4.279 1.88e-05 ***
deagsi  1.1977    3.3124   0.3051  3.925 8.66e-05 ***
sexoM   0.4555    1.5769   0.3115  1.462    0.144    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2687     3.7220    0.1472    0.4906
deagsi    3.3124     0.3019    1.8215    6.0236
sexoM     1.5769     0.6341    0.8564    2.9037

Concordance= 0.74  (se = 0.031 )
Likelihood ratio test= 36.35  on 3 df,   p=6e-08
Wald test            = 32.6  on 3 df,   p=4e-07
Score (logrank) test = 38.43  on 3 df,   p=2e-08
```

```r
# AIC
AIC(m6)
```

```
[1] 376.4584
```

El `m5` parece ser un mejor modelo, con un AIC menor (374.8600995).

Ensayemos ahora un modelo con 2 variables, a partir entonces de `m5`:

```
Surv(tiempo, status) ~ fase + decr + deag
<environment: 0x00000185d7833578>
```

Saco `fase`, dado que su coeficiente resultó no significativo.

- **Modelo 7**: `deag + decr`

```r
# Ajustar modelo
m7 <- coxph(formula = Surv(tiempo, status) ~ decr + deag, data = datos)

# Salida modelo
summary(m7)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag, data = datos)

  n= 96, number of events= 49 
```

```
          coef exp(coef) se(coef)      z Pr(>|z|)    
decrsi -1.2532    0.2856   0.3038 -4.125 3.71e-05 ***
deagsi  1.1001    3.0045   0.2959  3.717 0.000201 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2856     3.5015    0.1574     0.518
deagsi    3.0045     0.3328    1.6822     5.366

Concordance= 0.739  (se = 0.03 )
Likelihood ratio test= 34.13  on 2 df,   p=4e-08
Wald test            = 32.9  on 2 df,   p=7e-08
Score (logrank) test = 37.62  on 2 df,   p=7e-09
```

```
# AIC
AIC(m7)
```

```
[1] 376.6813
```

Ahora todas las variables tienen significación y, si bien no ha habido mejora en el AIC, según los valores del test de Wald, parece un mejor modelo. No obstante, podemos comparar los modelos "candidatos".

La comparación entre los modelos se realiza con la función `anova()`, que realiza la prueba de Razón de Verosimilitud entre dos modelos. Los argumentos de la función son los dos modelos ajustados a comparar. La salida de la función es una tabla en donde muestra para los dos modelos el log-verosimilitud, el estadístico `Chisq`, los grados de libertad `Df` y el p-valor `P(>|Chi|)` de la prueba.

Ahora, seleccionamos los dos modelos a comparar en nuestro caso: `m5` y `m7`.

```
anova(m5, m7)
```

```
Analysis of Deviance Table
 Cox model: response is  Surv(tiempo, status)
 Model 1: ~ fase + decr + deag
 Model 2: ~ decr + deag
   loglik  Chisq Df Pr(>|Chi|)  
1 -183.43                       
2 -186.34 5.8212  2    0.05444 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Recordemos que los modelos deben ser anidados para poder hacer ANOVA. En este caso, lo que hace es comparar los log de Likelihood de ambos modelos. Esa diferencia se distribuye como un Chi cuadrado. El valor de *p* se encuentra en el límite de significación (considerando 0,05). Si somos estrictos, el valor de *p* indica que no hay diferencia entre ambos modelos, por lo tanto nos quedaríamos con el de menos variables, de acuerdo al principio de parsimonia.

En nuestro caso, podemos decidir quedarnos con el `m5`, dado que la fase de la enfermedad puede resultar un elemento predictivo importante. Como se indicó anteriormente, la regresión de Cox generalmente tiene fines predictivos, así que podemos ver en la bibliografía, modelos que no siguen todas las “normas” del modelado, porque se hace importante mantener variables como edad o sexo, aún cuando no resulten significativas, a fin de contar con intervalos de predicción por sexo o grupo etario.

Supongamos entonces que decidimos quedarnos con el `m5`, aunque rompamos algunas reglas. Interpretemos ahora los parámetros del modelo:

```
summary(m5)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag, data = datos)

  n= 96, number of events= 49 

               coef exp(coef) se(coef)      z Pr(>|z|)    
fasecb       0.3449    1.4119   0.5402  0.639 0.523100    
fasecrónica -0.6509    0.5216   0.3467 -1.878 0.060440 .  
decrsi      -1.0597    0.3466   0.3176 -3.336 0.000849 ***
deagsi       1.1633    3.2004   0.3160  3.681 0.000232 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4119     0.7083    0.4898    4.0698
fasecrónica    0.5216     1.9173    0.2644    1.0290
decrsi         0.3466     2.8856    0.1860    0.6458
deagsi         3.2004     0.3125    1.7227    5.9456

Concordance= 0.753  (se = 0.03 )
Likelihood ratio test= 39.95  on 4 df,   p=4e-08
Wald test            = 38.18  on 4 df,   p=1e-07
Score (logrank) test = 47.11  on 4 df,   p=1e-09
```

Para la interpretación, nos centramos en la columna `exp(coef)`, que corresponde a los HR. Partiendo de dichos valores, podemos concluir:

- El riesgo de muerte de los pacientes que padecen enfermedad injerto contra huésped aguda es aproximadamente el triple de aquellos que no la han padecido ($HR = 3,2$).
- Los pacientes que hayan padecido enfermedad injerto contra huésped crónica tienen 0,34 veces el riesgo de morir post datos de los que no la tuvieron. Puede decirse también (y quizás quedaría mejor expresado) que tienen un 66 % menos riesgo que los que no tuvieron enfermedad injerto contra huésped crónica ($HR = 0,34$).
- Los pacientes que reciben trasplante en la fase crónica de la enfermedad, tienen un riesgo 48 % menor de morir post datos que aquellos que son trasplantados en la fase aguda (si consideramos un $p < 0,1$, este término tendría significación). Se observó un aumento de riesgo en los que reciben trasplante en la crisis blástica respecto de quienes lo reciben en la aguda, pero este término no resultó significativo.

33.3.2. Algunas alternativas

Como hemos visto para modelados anteriores, podemos construir el modelo de Cox con funciones del paquete `stats` y del paquete `performance` (Lüdecke et al. 2022).

Les recordamos que para la comparación de modelos con la función `compare_performance()` van a observar que se utiliza el indicador **AIC *weights*** en lugar del AIC. El AIC *weights* es un parámetro que cuantifica de manera relativa la calidad de un modelo. Refleja la probabilidad relativa de que un modelo sea el mejor entre un conjunto de modelos candidatos. Su cálculo se basa en la comparación de los valores de AIC de cada modelo considerado.

Por esto, **se interpreta de manera inversa al AIC**: a mayor valor de *AIC weights*, mayor probabilidad de que el modelo sea un mejor candidato en comparación al resto.

Les dejamos a continuación el proceso *backwards* utilizando funciones de estos paquetes.

```
## Selección backwards a partir del modelo saturado
m1 <- coxph(
  Surv(tiempo, status) ~ fase + decr + deag + sexo + edad,
  data = datos
)

drop1(m1)
```

```
Single term deletions

Model:
Surv(tiempo, status) ~ fase + decr + deag + sexo + edad
       Df    AIC
<none>    378.14
fase    2 378.46
decr    1 386.29
deag    1 389.22
sexo    1 376.82
edad    1 376.25
```

```
# (-) sexo
fit_m2 <- update(m1, ~ . - sexo)

# (-) edad
fit_m3 <- update(m1, ~ . - edad)

# (-) fase
fit_m4 <- update(m1, ~ . - fase)

# Salida de los modelos
summary(fit_m2)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag + edad,
    data = datos)

  n= 96, number of events= 49

                coef exp(coef) se(coef)      z Pr(>|z|)
fasecb       0.34930   1.41808  0.54010  0.647 0.517801
fasecrónica -0.66770   0.51289  0.35813 -1.864 0.062262 .
decrsi      -1.04100   0.35310  0.33283 -3.128 0.001762 **
deagsi       1.15232   3.16554  0.32110  3.589 0.000332 ***
edad        -0.00272   0.99728  0.01452 -0.187 0.851418
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4181     0.7052    0.4920     4.087
fasecrónica    0.5129     1.9498    0.2542     1.035
decrsi         0.3531     2.8321    0.1839     0.678
deagsi         3.1655     0.3159    1.6870     5.940
edad           0.9973     1.0027    0.9693     1.026

Concordance= 0.758  (se = 0.031 )
Likelihood ratio test= 39.98  on 5 df,   p=2e-07
Wald test            = 38.28  on 5 df,   p=3e-07
Score (logrank) test = 47.16  on 5 df,   p=5e-09
```

```r
summary(fit_m3)
```

```

Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag + sexo,
    data = datos)

  n= 96, number of events= 49

               coef exp(coef) se(coef)      z Pr(>|z|)
fasecb       0.3378    1.4019   0.5429  0.622 0.533779
fasecrónica -0.5715    0.5647   0.3647 -1.567 0.117090
decrsi      -1.0940    0.3349   0.3246 -3.370 0.000750 ***
deagsi       1.2075    3.3451   0.3240  3.726 0.000194 ***
sexoM        0.2509    1.2852   0.3251  0.772 0.440233
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4019     0.7133    0.4837    4.0631
fasecrónica    0.5647     1.7709    0.2763    1.1540
decrsi         0.3349     2.9863    0.1772    0.6326
deagsi         3.3451     0.2989    1.7725    6.3128
sexoM          1.2852     0.7781    0.6796    2.4306

Concordance= 0.762  (se = 0.03 )
```

```
Likelihood ratio test= 40.56  on 5 df,   p=1e-07
Wald test            = 38.19  on 5 df,   p=3e-07
Score (logrank) test = 47.46  on 5 df,   p=5e-09
```

```r
summary(fit_m4)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag + sexo + edad,
    data = datos)

  n= 96, number of events= 49

             coef  exp(coef)   se(coef)      z Pr(>|z|)
decrsi -1.3160459  0.2681937  0.3150468 -4.177 2.95e-05 ***
deagsi  1.1988240  3.3162147  0.3084393  3.887 0.000102 ***
sexoM   0.4531812  1.5733092  0.3241990  1.398 0.162158
edad    0.0003653  1.0003654  0.0142724  0.026 0.979578
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2682     3.7286    0.1446    0.4973
deagsi    3.3162     0.3015    1.8118    6.0700
sexoM     1.5733     0.6356    0.8334    2.9701
edad      1.0004     0.9996    0.9728    1.0287

Concordance= 0.732  (se = 0.032 )
Likelihood ratio test= 36.35  on 4 df,   p=2e-07
Wald test            = 32.6  on 4 df,   p=1e-06
Score (logrank) test = 38.43  on 4 df,   p=9e-08
```

```r
# Comparación de modelos
compare_performance(m1, fit_m2, fit_m3, fit_m4, metrics = "AIC", rank = T)
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC weights | Performance-Score
------------------------------------------------
fit_m3 | coxph |       0.404 |           100.00%
fit_m2 | coxph |       0.304 |            62.75%
m1     | coxph |       0.157 |             8.58%
fit_m4 | coxph |       0.134 |             0.00%
```

```r
## Selección a partir de fit_m3
drop1(fit_m3)
```

```
Single term deletions
```

```
Model:
Surv(tiempo, status) ~ fase + decr + deag + sexo
       Df    AIC
<none>    376.25
fase    2 376.46
decr    1 386.02
deag    1 387.91
sexo    1 374.86
```

```
# (-) sexo
fit_m5 <- update(fit_m3, ~ . - sexo)

# (-) fase
fit_m6 <- update(fit_m3, ~ . - fase)

# Salida de los modelos
summary(fit_m5)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + decr + deag, data = datos)

  n= 96, number of events= 49 

               coef exp(coef) se(coef)      z Pr(>|z|)    
fasecb       0.3449    1.4119   0.5402  0.639 0.523100    
fasecrónica -0.6509    0.5216   0.3467 -1.878 0.060440 .  
decrsi      -1.0597    0.3466   0.3176 -3.336 0.000849 ***
deagsi       1.1633    3.2004   0.3160  3.681 0.000232 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb         1.4119     0.7083    0.4898    4.0698
fasecrónica    0.5216     1.9173    0.2644    1.0290
decrsi         0.3466     2.8856    0.1860    0.6458
deagsi         3.2004     0.3125    1.7227    5.9456

Concordance= 0.753  (se = 0.03 )
Likelihood ratio test= 39.95  on 4 df,   p=4e-08
Wald test            = 38.18  on 4 df,   p=1e-07
Score (logrank) test = 47.11  on 4 df,   p=1e-09
```

```
summary(fit_m6)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag + sexo, data = datos)

  n= 96, number of events= 49 
```

```
          coef exp(coef) se(coef)      z Pr(>|z|)    
decrsi -1.3143    0.2687   0.3072 -4.279 1.88e-05 ***
deagsi  1.1977    3.3124   0.3051  3.925 8.66e-05 ***
sexoM   0.4555    1.5769   0.3115  1.462    0.144    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2687     3.7220    0.1472    0.4906
deagsi    3.3124     0.3019    1.8215    6.0236
sexoM     1.5769     0.6341    0.8564    2.9037

Concordance= 0.74  (se = 0.031 )
Likelihood ratio test= 36.35  on 3 df,   p=6e-08
Wald test            = 32.6  on 3 df,   p=4e-07
Score (logrank) test = 38.43  on 3 df,   p=2e-08
```

```
# Comparación de modelos
compare_performance(fit_m3, fit_m5, fit_m6, metrics = "AIC", rank = T)
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC weights | Performance-Score
------------------------------------------------
fit_m5 | coxph |       0.513 |           100.00%
fit_m3 | coxph |       0.256 |             8.85%
fit_m6 | coxph |       0.231 |             0.00%
```

```
## Selección a partir de fit_m5
drop1(fit_m5)
```

```
Single term deletions

Model:
Surv(tiempo, status) ~ fase + decr + deag
       Df    AIC
<none>    374.86
fase    2 376.68
decr    1 384.27
deag    1 386.00
```

```
# fit_m5 (-) fase
fit_m7 <- update(fit_m5, ~ . - fase)

summary(fit_m7)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ decr + deag, data = datos)
```

```
  n= 96, number of events= 49

          coef exp(coef) se(coef)      z Pr(>|z|)
decrsi -1.2532    0.2856   0.3038 -4.125 3.71e-05 ***
deagsi  1.1001    3.0045   0.2959  3.717 0.000201 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

       exp(coef) exp(-coef) lower .95 upper .95
decrsi    0.2856     3.5015    0.1574     0.518
deagsi    3.0045     0.3328    1.6822     5.366

Concordance= 0.739  (se = 0.03 )
Likelihood ratio test= 34.13  on 2 df,   p=4e-08
Wald test            = 32.9  on 2 df,   p=7e-08
Score (logrank) test = 37.62  on 2 df,   p=7e-09
```

```r
# Comparación de modelos
compare_performance(fit_m5, fit_m7, metrics = "AIC", rank = T)
```

```
# Comparison of Model Performance Indices

Name   | Model | AIC weights | Performance-Score
------------------------------------------------
fit_m5 | coxph |       0.713 |           100.00%
fit_m7 | coxph |       0.287 |             0.00%
```

Por otro lado, recordemos que R permite también hacer una **selección automática de los modelos**, tanto para supervivencia como para cualquier tipo de modelado. Normalmente la bibliografía no recomienda los procedimientos automáticos, pero tengamos en claro lo que ocurre en cada caso:

- "hacia atrás / adelante" (el valor por defecto): la selección comienza con el modelo completo y elimina los predictores uno a la vez, en cada paso, considerando si el criterio será mejorado agregando de nuevo una variable eliminada en un paso anterior.
- "hacia adelante / hacia atrás", la selección comienza con un modelo que incluye sólo una constante, y añade predictores uno a la vez, en cada paso considerando si el criterio se mejorará mediante la eliminación de una variable agregada anteriormente.
- "atrás" y "adelante" son similares sin la reconsideración en cada paso.

33.3.3. Predicción de la curva de supervivencia

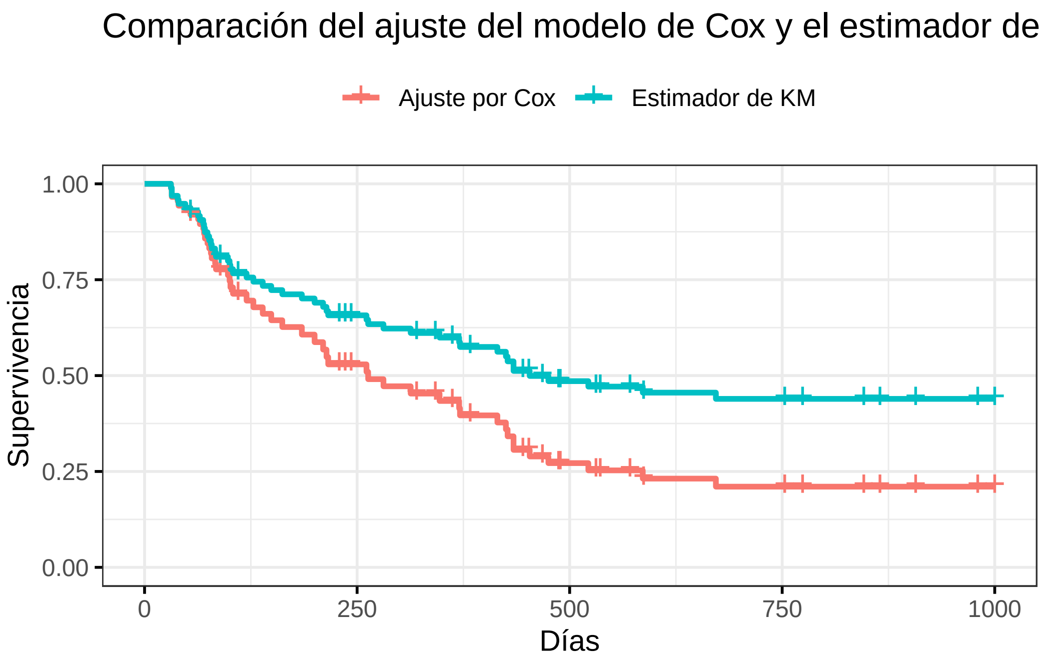
Ahora, estaríamos en condiciones de obtener la función de supervivencia ajustada mediante el modelo de Cox seleccionado. La función `survfit()` del paquete `survival` calcula la función de supervivencia pronosticada para un modelo de riesgos proporcionales de Cox.

Para ello, la función de R es:

```
survfit(m5) |>
  summary()
```

Podríamos comparar esto con lo obtenido en la estimación de Kaplan-Meier (ver módulo de KM). Pero, siempre resulta más claro realizar la comparación en forma gráfica: graficaremos la función de supervivencia obtenida mediante el estimador de Kaplan-Meier y la obtenida mediante el modelo de Cox seleccionado. Para esto, primero tendremos que crear una lista con las funciones de supervivencia requeridas para luego graficarlas utilizando la función `ggsurvplot_combine()` del paquete `survminer` (Kassambara et al. 2026).

```
# Creamos un objeto con la estimación de KM
global <- Surv(datos$tiempo, datos$status)

# Creamos una lista con la función de supervivencia del modelo final
# seleccionado y la función del estimador de KM
fit <- list(modelo = survfit(m5), global = survfit(global ~ 1))

# Generamos las curvas de supervivencia
ggsurvplot_combine(
  fit,
  data = datos,
  ggtheme = theme_bw(),
  title = "Comparación del ajuste del modelo de Cox y el estimador de KM",
  xlab = "Días",
  ylab = "Supervivencia",
  legend.title = "",
  legend.labs = c("Ajuste por Cox", "Estimador de KM")
)
```



Podemos observar que el modelo seleccionado, ajusta bastante bien (sobreestima al principio, subestima al final). Sin embargo, en lo que a modelado respecta, para evaluar la calidad de ajuste contamos con parámetros adicionales.

33.3.4. Calidad de ajuste de un modelo

Hay pocas medidas para evaluar el ajuste de un modelo de supervivencia. Una de las más simples es R^2, cuya expresión es:

$$R^2_{LR} = 1 - \left(\frac{Likelihood(modelo\,nulo)}{Likelihood(modelo\,ajustado)} \right)^{\frac{2}{n}}$$

Su interpretación es similar a la del R^2 de los otros modelos lineales:

R^2 puede ser interpretado como el poder explicativo de las covariables en el tiempo de ocurrencia del evento en estudio.

R^2 tiene valores bajos en los análisis de supervivencia (alrededor de 30 % de poder explicativo), por lo que muchos autores no lo consideran apropiado. De hecho, en las salidas de R de los modelos, observamos que no lo muestra actualmente, siendo que antes sí lo ofrecía, atendiendo a estas y otras cuestiones teóricas. En su lugar, ofrece el valor de **concordancia**. Profundizaremos un poco sobre este concepto, ya que no lo hemos visto en otros modelos.

El uso de la estadística de **concordancia** para los modelos de Cox fue popularizado por Harrel (Harrell et al. 1996), y ahora es la medida más usada como bondad de ajuste en los modelos de supervivencia. Una ventaja de la estadística es que está bien definida, no solo para los modelos de supervivencia, sino también para la regresión lineal logística y ordinaria.

En general, si yi y xi son los valores observados y predichos por el modelo, respectivamente, la concordancia se define como $P(x_i > x_j | y_i > y_j)$, la probabilidad de que la predicción x vaya en la misma dirección que los datos reales y. Un par de las observaciones i, j se consideran concordantes si la predicción y los datos van en la misma dirección. **La concordancia es la fracción de pares concordantes**.

En general, se acepta la siguiente escala:

Concordancia (c)	Poder discriminatorio
0.3 < c < 0.4	Bajo
c = 0.5	Por azar
0.6 < c < 0.7	Común
0.7 < c < 0.8	Muy bueno
0.8 < c < 0.9	Excelente

Si observamos, el valor de concordancia para el modelo `m5` (0.753), lo que indica muy buena concordancia entre lo observado y lo predicho por el modelo.

33.3.5. Análisis de residuos

Como ustedes recordarán, se denomina residuo a la diferencia entre el valor observado y el valor estimado por la ecuación de regresión, es decir a lo que la ecuación de regresión deja sin explicar para cada observación.

La definición de residuo en el análisis de supervivencia no es tan simple y directa como esta definición, dado, que, por ejemplo, un residuo obtenido como respuesta observada menos esperada, no tiene en consideración el tiempo observado de un individuo censurado.

En el análisis de supervivencia, se describen distintos tipos de residuos que sirven para evaluar distintos aspectos del modelo:

Evaluación	Tipo de residuo
Verificar riesgos proporcionales	Residuos de Schoenfeld
Evaluar si es adecuado incluir las variables en forma lineal en el modelo	Residuos Martingale
Detectar residuos grandes (outliers)	Residuos de Deviance
Impacto que tiene cada observación en el modelo, tanto en forma global como por variable	Residuos Score

Y ahora algo más… para covariables con pocas categorías, las curvas de KM estratificadas dan una idea si se cumple el presupuesto de riesgos proporcionales de Cox. Curvas que son razonablemente paralelas a lo largo del tiempo indican proporcionalidad de riesgo entre las categorías y, por el contrario, el entrecruzamiento o una variación en las distancias entre las categorías, podría indicar falta de proporcionalidad.

Si tienen presentes las curvas de KM que construimos para este ejemplo, en todas las curvas que se muestran, parece razonable suponer que se cumple el supuesto de proporcionalidad, aún en el caso de la variable `sexo`, dado que ambas categorías están prácticamente superpuestas, separándose recién al final, cuando quedan pocas observaciones.

A continuación, abordaremos el análisis de residuos, al menos de algunos de ellos. Para el desarrollo de gráficos seguiremos explorando el paquete `survminer`. Les aclaramos, además, que no utilizaremos la función `check_model()` del paquete `performance` para esta ocasión dado que no permite aún el análisis de residuales para modelos de Cox.

Residuos de Schoenfeld

Podríamos preguntarnos si el efecto de cada covariable será el mismo a medida que transcurra el tiempo, porque si no es así, se trataría de covariables tiempo dependientes. Para responder esta cuestión, se calculan los **Residuos de Schoenfeld**.

Se genera un residuo para cada variable y para cada paciente, es decir que si tenemos un modelo de Cox con tres factores pronóstico se calcularán 3 residuos de Schoenfeld por paciente. Estos residuos valen cero para las observaciones incompletas. Es posible modificar estos residuos con el fin de que no valgan cero para las observaciones incompletas, obteniéndose entonces los denominados residuos Schoenfeld corregidos o escalados.

En R, la función `cox.zph()` realiza la prueba de hipótesis de correlación para los residuos de Schoenfeld de cada covariable con alguna transformación del tiempo, los argumentos de la función son:

- `fit`: Un objeto coxph resultado de un ajuste del modelo de regresión de Cox.

- `transform`: Tipo de transformación del tiempo, posibles valores son `"km"`, `"rank"`, `"identity"` o una función.

- `global`: una prueba chi cuadrado global.

La salida de la función es una tabla en donde para cada covariable calcula el coeficiente rho, el estadístico de prueba chisq y su respectivo *p*-valor, además incluye una prueba global. Por ejemplo, para el **m5** o para **m7**, el supuesto de riesgos proporcionales puede ser verificado mediante el contraste de hipótesis generado mediante el comando:

```
cox.zph(m5, transform = "km", global = TRUE)
```

```
        chisq df       p
fase    9.308  2 0.00952
decr    7.834  1 0.00513
deag    0.279  1 0.59742
GLOBAL 18.546  4 0.00097
```

¿Qué significa esta salida?

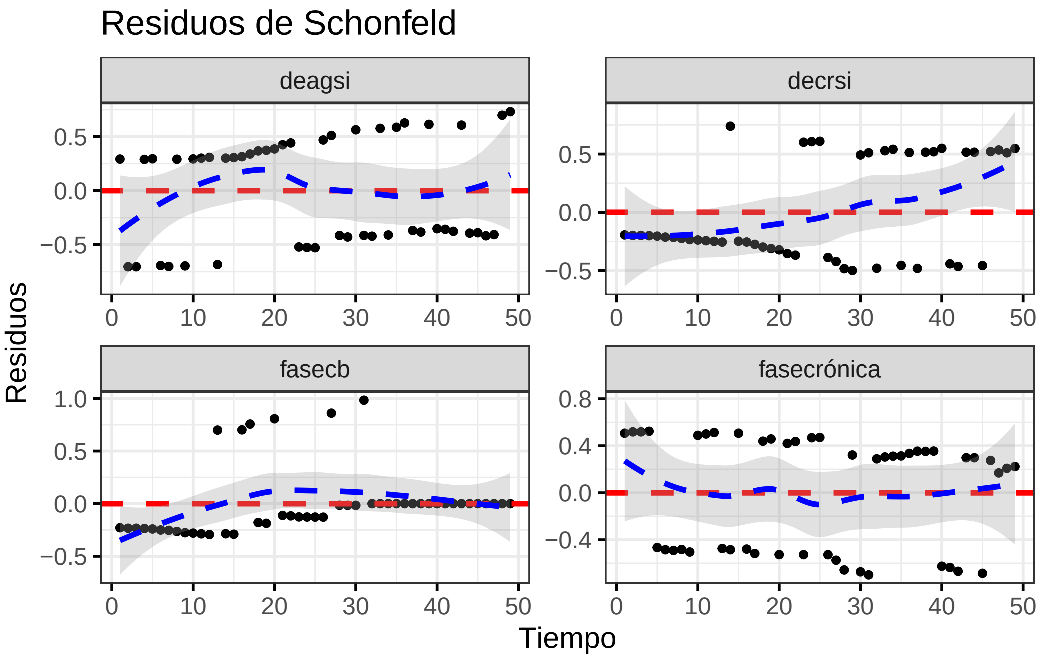
La H_0 que se testea es que la correlación lineal entre el residuo de Schoenfeld y el tiempo de supervivencia es cero.

Esto es equivalente a probar H_0: pendiente igual a cero o $H_0: logHR$ es constante en el tiempo.

De lo expuesto se concluye que existe evidencia significativa al 5 % de que se viole el supuesto de riesgos proporcionales, desde el punto de vista global, y para las covariables `decr` y `fase`.

La validación del supuesto de riesgos proporcionales puede realizarse también en forma gráfica con la función `ggcoxdiagnostics()`:

```
ggcoxdiagnostics(
  m5,
  type = "schoenfeld",
  title = "Residuos de Schonfeld",
  ylab = "Residuos",
  xlab = "Tiempo"
)
```



La interpretación se facilita con el agregado de una línea de tendencia "*lowess*", de esta forma, cuando esta línea se desvía de una línea constante, se sospecha una estructura tiempo dependiente en el efecto de la covariable.

Los gráficos confirman lo que hallamos con el test: los residuos tienen un patrón o estructura con el tiempo. En el caso de la variable `decr`, parece visualizarse una relación lineal, lo mismo en `fase:cb`.

Frente al problema de No proporcionalidad, se deberá evaluar:

- Magnitud
- Puntos influyentes

¿Cuáles son las posibles soluciones? Estratificar por covariable dependiente del tiempo, dividir el eje del tiempo o utilizar otro tipo de modelo.

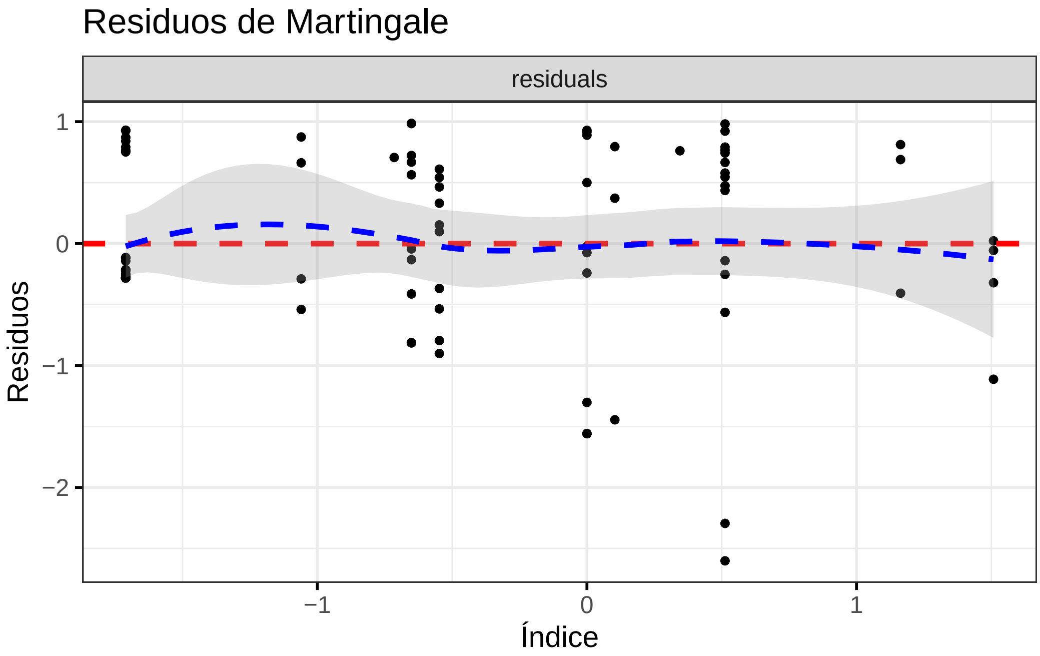
Continuaremos analizando los residuos de este modelo y retomaremos el tema cuando lleguemos a modelos estratificados.

Residuos Martingale

El residuo Martingale de un sujeto es simplemente la diferencia entre el número de acontecimientos observados y el número de acontecimientos esperados según el modelo.

Se grafica: **Mi** vs **índice de individuo**, permitiendo revelar sujetos mal ajustados por el modelo. Nuevamente nos será de utilidad la función `ggcoxdiagnostics()`:

```
ggcoxdiagnostics(
  m5,
  type = "martingale",
  title = "Residuos de Martingale",
  ylab = "Residuos",
  xlab = "Índice"
)
```



Este gráfico sirve para evidenciar posibles valores aberrantes: individuos que demoraron mucho o demasiado poco en tener el evento (como los que señalan las flechas). Se espera que se distribuyan en torno a la línea punteada del 0, sin ningún patrón remanente. Valores de **Mi** mayores que 0, indica que el numero de eventos observados es menor que los estimados por el modelo y viceversa.

33.4. Modelo de Cox estratificado

En algunas situaciones no podemos asumir que el riesgo basal $h_o(t)$ sea el mismo para todos los individuos del estudio. Esto podría ser por la característica propia de la variable (como la edad) o porque el diseño sea estratificado a priori (por ej. estadíos de una enfermedad). En estos casos, los distintos $h_o(t)$ definen los distintos estratos.

Los coeficientes β también se estiman por el método de verosimilitud parcial, pero los individuos en riesgo que participan de la estimación son sólo los del estrato.

El modelo de Cox estratificado constituye una de las maneras de corregir el modelo de Cox cuando no se cumple el supuesto de riesgos proporcionales para alguna de las covariables. En este caso suele correrse el modelo estratificando por la covariable que no cumple con el supuesto de riesgo proporcional. Este procedimiento permite corregir el sesgo en la estimación del parámetro, que puede presentarse cuando se viola el supuesto de riesgo proporcional. Sin embargo, presenta una desventaja y es que no existe ningún β que permita estimar el efecto de la covariable de estratificación.

Continuando con nuestro ejemplo, vimos que la variable decr violaba el supuesto de proporcionalidad. Deberíamos entonces usar un modelo de Cox estratificado, donde decr sea la variable de estratificación.

```
m5st <- coxph(
  formula = Surv(tiempo, status) ~ fase + deag + strata(decr),
  data = datos
)

# Salida del modelo
summary(m5st)
```

```
Call:
coxph(formula = Surv(tiempo, status) ~ fase + deag + strata(decr),
    data = datos)

  n= 96, number of events= 49

                coef exp(coef) se(coef)      z Pr(>|z|)
fasecb       0.09209   1.09647  0.54271  0.170 0.865250
fasecrónica -0.72569   0.48399  0.35111 -2.067 0.038747 *
deagsi       1.12109   3.06819  0.32007  3.503 0.000461 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
fasecb          1.096     0.9120    0.3785    3.1765
fasecrónica     0.484     2.0662    0.2432    0.9632
deagsi          3.068     0.3259    1.6385    5.7454

Concordance= 0.646  (se = 0.041 )
Likelihood ratio test= 17.02  on 3 df,   p=7e-04
Wald test            = 17.36  on 3 df,   p=6e-04
Score (logrank) test = 18.52  on 3 df,   p=3e-04
```

Podemos nuevamente chequear la proporcionalidad en este nuevo modelo:

```
cox.zph(m5st, transform = "km", global = TRUE)
```

```
        chisq df      p
fase   10.848  2 0.0044
deag    0.448  1 0.5031
GLOBAL 11.084  3 0.0113
```

Vemos que el modelo de Cox estratificado tampoco resuelve el problema.

33.5. Modelo de Cox extendido

El modelo de Cox que vimos inicialmente asume que las covariables no varían en el transcurso del tiempo. Pero sabemos que esto no siempre es así.

¿Cómo analizar la supervivencia cuando las covariables cambian con el tiempo, como edad, tratamiento o hábitos alimentarios?

El modelo de Cox extendido aborda esta situación mediante un proceso de conteo. Tiene las mismas premisas del modelo de Cox

$$h(t|x(t)) = h_0(t)exp(x(t)\beta)$$

¿Cuál es la diferencia? La primer diferencia está ya en la base de datos: tendremos más de una fila por individuo, dado que los valores de una variable para el mismo individuo cambian con el correr del tiempo.

33.6. Análisis de supervivencia más complejos

Hasta ahora, vimos modelos de supervivencia adecuados para tratar problemas en los cuales el evento ocurre una única vez. Pero existen situaciones en lo que esto no es así y podríamos estar interesados en eventos que pueden acontecer más de una vez para un mismo individuo, como: infartos, gestaciones, fracturas, internaciones, etc. Así también nos podrían interesar situaciones en las que existen diferentes tipos de eventos derivados de un mismo factor de riesgo en el estudio: efectos adversos por medicamentos, enfermedades oportunistas en SIDA, etc. Este análisis se conoce como *Supervivencia de eventos múltiples*.

El análisis de eventos múltiples, pretende responder a dos preguntas básicas:

1. ¿Cuáles son los factores de riesgo asociados a los tiempos de diferentes eventos en un mismo individuo?
2. ¿Cómo analizar diferentes eventos como desenlace de una misma situación de riesgo?

Las principales características de este tipo de análisis son:

- Tener más de un tiempo de supervivencia por individuo.
- Los individuos pueden mantenerse en el grupo de riesgo aún después de la ocurrencia de algún evento.

Dado que los intervalos de tiempo no son independientes entre sí, la utilización directa del modelo de Cox no sirve. A su vez, no tener independencia entre las observaciones puede generar error en las estimaciones.

Las estrategias para resolver estas situaciones son varias:

- Ajustar un modelo de regresión en el que la variable respuesta es el número de eventos por paciente durante el período de estudio: Poisson
- Utilizar Cox considerando uno de los eventos: tiempo hasta el primer evento
- Utilizar Modelos multinivel
- Utilizar Modelos marginales

A continuación presentaremos una clasificación de eventos múltiples, la cual no es rígida ni definitiva, dado que la evolución de métodos en esta área es grande y van surgiendo nuevos modelos, que se adecúan mejor a las situaciones analizadas.

- **Eventos Competitivos**: sólo es posible observar tiempo hasta un evento primero que impide que otro tenga lugar. Ej; muerte x diferentes causas y un mismo factor de riesgo

- **Eventos Paralelos**: la ocurrencia de un evento no excluye la aparición de otros eventos y no existe orden preferencial (Enfermedades oportunistas, los efectos secundarios, la pérdida de dientes)
- **Eventos Ordenados**: la sucesión de tiempos sigue obligatoriamente un orden. Ej: IAM El riesgo subyacente cambia a medida que el individuo sufre nuevos eventos, es decir, los eventos, además de ordenarse en el tiempo, se ordenan según el riesgo basal.

Como podrán intuir el análisis se complejiza aún más, por lo cual no los abordaremos en este curso. Solamente hemos presentado los lineamientos, para que ustedes sean capaces de reconocer situaciones donde se deben aplicar estas metodologías, con el fin de que no cometan errores metodológicos importantes en el futuro (al menos referidos a estos temas).

33.7. Consideraciones sobre el tamaño de la muestra

Como ya hemos visto en la primera unidad, para determinar el tamaño de muestra necesario para un estudio, tendremos que tener una idea de la magnitud del efecto que deseamos detectar. En el análisis de supervivencia, el efecto es el HR, por ejemplo, a los 5 años, a los 10 años o al intervalo de tiempo que se desee.

Supongamos que deseamos saber si existen diferencias entre dos terapias diferentes A y B utilizadas habitualmente para tratar un determinado tipo de cáncer. Para ello se planea realizar un estudio de supervivencia. *¿Cuántos pacientes deberán estudiarse con cada tratamiento si se desea calcular el riesgo relativo con una precisión del 50 % de su valor real y una seguridad del 95 %?* De experiencias previas, se estima que el valor real del riesgo relativo es aproximadamente igual a 3 y la probabilidad de fallecer entre los pacientes tratados con el tratamiento A de un 20 %.

En este caso, para conocer si el tratamiento A tiene un efecto beneficioso sobre la evolución de los enfermos, podremos utilizar un modelo de regresión de Cox en el que se ajuste por la variable tratamiento, y a partir de cuyos coeficientes podremos estimar el HR asociado a la terapia recibida. Si, con este procedimiento, deseamos calcular el tamaño muestral mínimo necesario para detectar un determinado HR, deberemos conocer:

- Una idea del valor aproximado del riesgo relativo que se desea detectar (HR).
- La proporción de expuestos al factor de estudio (p), es decir, en nuestro caso, la proporción de enfermos habitualmente tratados con la terapia A.
- El porcentaje de observaciones censuradas que se espera en el total de la muestra (%C)
- El nivel de confianza o seguridad con el que se desea trabajar α
- El poder que se quiere para el estudio $1-\beta$

Con estos datos, el cálculo del tamaño muestral puede abordarse mediante la fórmula:

$$n = \frac{(z_{1-\alpha/2} + z_{1+\beta})^2}{(log(HR))^2 * (1- \%C) * (1-p) * p}$$

Ejemplo

Supongamos que el tratamiento (A) suele aplicarse a un 70 % de los pacientes que padecen ese tipo de cáncer, mientras que la otra terapia (B) es recibida sólo por un 30 % de los enfermos. Si el efecto pronóstico del tratamiento recibido va a analizarse de modo univariado, la ecuación puede aplicarse para calcular el número necesario de pacientes a estudiar. Así, para detectar un riesgo relativo de 3, suponiendo un 20 % de censura y trabajando con un $\alpha = 95\,\%$ y un poder del 80 % se tendría:

$$n = \frac{(1.96 + 0.842)^2}{(log3)^2 * (1 - 0.2) * (1 - 0.7) * 0.7} = 47$$

Necesitaríamos 47 pacientes para detectar un HR de 3. A menor HR, mayor tamaño de muestra.

Si ahora deseamos usar un modelo de Cox con más variables, el tratar de ajustar un modelo más complejo con el mismo número de pacientes llevará consigo una pérdida de precisión en la estimación de los coeficientes y, con ello, del HR asociado a cada una de las variables incluidas en el modelo multivariante. En esta situación, es obvio, que se necesita realizar alguna corrección en la ecuación que permita adaptar el tamaño muestral calculado a las variables que se incluirán a posteriori en el modelo.

$$n = \frac{(z_{1-\alpha/2} + z_{1+\beta})^2}{(logHR)^2 * (1 - \%C) * (1 - p) * p} * \frac{1}{1 - \rho^2}$$

El término agregado se lo llama **factor de inflación de la varianza**, donde ρ denota al coeficiente de correlación de Pearson entre el factor en estudio y aquella otra variable que incluiremos en el modelo. En el caso en el que se ajuste por más de otro factor en el modelo, lo más sencillo es considerar como el mayor coeficiente de correlación entre el factor de estudio y todas las variables incluidas. Este coeficiente de correlación, cuanto mayor sea, más incrementará el valor del factor de inflación de la varianza y, por tanto, se incrementará el tamaño de la muestra a estudiar.

Siguiendo con el ejemplo, posiblemente el tratamiento aplicado a cada enfermo dependerá de las características clínicas particulares del mismo, y recibirá una u otra terapia en función, por ejemplo, del estadío del tumor. En términos estadísticos, podrá entonces decirse que el factor tratamiento se encuentra "*correlacionado*" con esta característica. Supongamos, por ejemplo, que la correlación existente entre el nuevo tratamiento y el estadío del tumor es de 0,25. Este dato lo podremos obtener a partir de un estudio piloto o de otros trabajos sobre el tema. En caso de desconocerse, deberemos asumir una correlación suficientemente alta para así asegurar un poder suficiente.

El cálculo del tamaño muestral, por tanto, permite al investigador precisar el número de pacientes a estudiar para detectar como significativos efectos de una magnitud determinada. El no hacerlo, o el no conocer cuántos pacientes necesitamos para detectar un efecto como significativo podría llevarnos a cometer un error de tipo II, es decir, no encontrar diferencias cuando sí las hay.

Almeida Filho, Naomar de, y Maurício Lima Barreto. 2011. *Epidemiologia & Saúde: Fundamentos, Métodos e Aplicações*. Guanabara Koogan.

Bache, Stefan Milton, y Hadley Wickham. 2026. *magrittr: A Forward-Pipe Operator for R*. https://doi.org/10.32614/CRAN.package.magrittr.

Champely, Stephane. 2020. *pwr: Basic Functions for Power Analysis*. https://doi.org/10.32614/CRAN.package.pwr.

Collins, G. S., J. B. Reitsma, D. G. Altman, y K. G. M. Moons. 2015. «Transparent Reporting of a Multivariable Prediction Model for Individual Prognosis or Diagnosis (TRIPOD): The TRIPOD Statement». *BMJ* 350 (jan07 4): g7594-94. https://doi.org/10.1136/bmj.g7594.

Cox, D. R. 1972. «Regression Models and Life-Tables». *Journal of the Royal Statistical Society Series B: Statistical Methodology* 34 (2): 187-202. https://doi.org/10.1111/j.2517-6161.1972.tb00899.x.

Elm, Erik von, Douglas G. Altman, Matthias Egger, Stuart J. Pocock, Peter C. Gøtzsche, y Jan P. Vandenbroucke. 2008. «The Strengthening the Reporting of Observational Studies in Epidemiology (STROBE) Statement: Guidelines for Reporting Observational Studies». *Journal of Clinical Epidemiology* 61 (4): 344-49. https://doi.org/10.1016/j.jclinepi.2007.11.008.

Firke, Sam. 2024. *janitor: Simple Tools for Examining and Cleaning Dirty Data*. https://doi.org/10.32614/CRAN.package.janitor.

Freedman Ellis, Greg, y Ben Schneider. 2026. *srvyr: 'dplyr'-Like Syntax for Summary Statistics of Survey Data*. https://doi.org/10.32614/CRAN.package.srvyr.

Freeman, Daniel H. 1987. *Applied categorical data analysis*. M. Dekker.

Gohel, David, y Panagiotis Skintzos. 2026. *flextable: Functions for Tabular Reporting*. https://doi.org/10.32614/CRAN.package.flextable.

Gordis, Leon. 2017. *Epidemiologia*. Thieme Revinter Publicações LTDA.

Gross, Juergen, y Uwe Ligges. 2015. *nortest: Tests for Normality*. https://doi.org/10.32614/CRAN.package.nortest.

Harrell, Frank E., Kerry L. Lee, y Daniel B. Mark. 1996. «MULTIVARIABLE PROGNOSTIC MODELS: ISSUES IN DEVELOPING MODELS, EVALUATING ASSUMPTIONS AND ADEQUACY, AND MEASURING AND REDUCING ERRORS». *Statistics in Medicine* 15 (4): 361-87. https://doi.org/10.1002/(SICI)1097-0258(19960229)15:4%3C361::AID-SIM168%3E3.0.CO;2-4.

Hartig, Florian. 2026. *DHARMa: Residual Diagnostics for Hierarchical (Multi-Level / Mixed) Regression Models*. https://doi.org/10.32614/CRAN.package.DHARMa.

Hernández-Ávila, Mauricio. 2011. *Epidemiología: diseño y análisis de estudios*. Editorial Médica Panamericana.

Ho, Daniel E., Kosuke Imai, Gary King, y Elizabeth A. Stuart. 2011. «MatchIt: Nonparametric Preprocessing for Parametric Causal Inference». *Journal of Statistical Software* 42 (8): 1-28. https://doi.org/10.18637/jss.v042.i08.

Holmberg, Mathias J., y Lars W. Andersen. 2022. «Collider Bias». *JAMA* 327 (13): 1282-83. https://doi.org/10.1001/jama.2022.1820.

Horikoshi, Masaaki, y Yuan Tang. 2018. *ggfortify: Data Visualization Tools for Statistical Analysis Results*. https://CRAN.R-project.org/package=ggfortify.

Kassambara, Alboukadel. 2026a. *ggpubr: 'ggplot2' Based Publication Ready Plots*. https://CRAN.R-project.org/package=ggpubr.

Kassambara, Alboukadel. 2026b. *rstatix: Pipe-Friendly Framework for Basic Statistical Tests*. https://doi.org/10.32614/CRAN.package.rstatix.

Kassambara, Alboukadel, Marcin Kosinski, y Przemyslaw Biecek. 2026. *survminer: Drawing Survival Curves using 'ggplot2'*. https://doi.org/10.32614/CRAN.package.survminer.

Kim, Jong Hae. 2019. «Multicollinearity and Misleading Statistical Results». *Korean Journal of Anesthesiology* 72 (6): 558-69. https://doi.org/10.4097/kja.19087.

Kleinbaum, David G, Lawrence L Kupper, Keith E Muller, y Azhar Nizam. 1988. *Applied regression analysis and other multivariable methods*. Vol. 601. Duxbury press Belmont, CA.

Komsta, Lukasz, y Frederick Novomestky. 2022. *moments: Moments, Cumulants, Skewness, Kurtosis and Related Tests*. https://doi.org/10.32614/CRAN.package.moments.

Kosmidis, Ioannis, Dirk Schumacher, y Florian Schwendinger. 2026. *detectseparation: Detect and Check for Separation and Infinite Maximum Likelihood Estimates*. https://doi.org/10.32614/CRAN.package.detectseparation.

Lenth, Russell V., y Julia Piaskowski. 2026. *emmeans: Estimated Marginal Means, aka Least-Squares Means*. https://doi.org/10.32614/CRAN.package.emmeans.

Lüdecke, Daniel, Mattan S. Ben-Shachar, Indrajeet Patil, et al. 2022. «easystats: Framework for Easy Statistical Modeling, Visualization, and Reporting». *CRAN*, advance online publication. https://doi.org/10.32614/CRAN.package.easystats.

Lüdecke, Daniel, Indrajeet Patil, Mattan S. Ben-Shachar, Brenton M. Wiernik, Philip Waggoner, y Dominique Makowski. 2021. «see: An R Package for Visualizing Statistical Models». *Journal of Open Source Software* 6 (64): 3393. https://doi.org/10.21105/joss.03393.

Lumley, Thomas. 2010. *Complex Surveys: A Guide to Analysis Using R: A Guide to Analysis Using R*. John Wiley; Sons.

Luque-Fernandez, Miguel Angel, Michael Schomaker, Daniel Redondo-Sanchez, Maria Jose Sanchez Perez, Anand Vaidya, y Mireille E. Schnitzer. 2019. «Educational Note: Paradoxical collider effect in the analysis of non-communicable disease epidemiological data: a reproducible illustration and web application». *International Journal of Epidemiology* 48 (2): 640-53. https://doi.org/10.1093/ije/dyy275.

MacMahon, Brian. 1972. «Concepts of multiple factors». *Multiple factors in the causation of environmentally induced disease. Academic Press, New York*.

Manitz, Juliane, Mark Hempelmann, Goeran Kauermann, et al. 2021. *samplingbook: Survey Sampling Procedures*. https://doi.org/10.32614/CRAN.package.samplingbook.

McGillycuddy, Maeve, David I. Warton, Gordana Popovic, y Benjamin M. Bolker. 2025. «Parsimoniously Fitting Large Multivariate Random Effects in glmmTMB». *Journal of Statistical Software* 112 (1): 1-19. https://doi.org/10.18637/jss.v112.i01.

Meyer, Fanny, y Victor Perrier. 2025. *esquisse: Explore and Visualize Your Data Interactively*. https://doi.org/10.32614/CRAN.package.esquisse.

Müller, Kirill, y Hadley Wickham. 2026. *tibble: Simple Data Frames*. https://doi.org/10.32614/CRAN.package.tibble.

Pedersen, Thomas Lin. 2025. *patchwork: The Composer of Plots*. https://doi.org/10.32614/CRAN.package.patchwork.

Pedersen, Thomas Lin, y Fabio Crameri. 2023. *scico: Colour Palettes Based on the Scientific Colour-Maps*. https://doi.org/10.32614/CRAN.package.scico.

Pena, Edsel A., y Elizabeth H. Slate. 2019. *gvlma: Global Validation of Linear Models Assumptions*. https://doi.org/10.32614/CRAN.package.gvlma.

Ryu, Choonghyun. 2025. *dlookr: Tools for Data Diagnosis, Exploration, Transformation*. https://CRAN.R-project.org/package=dlookr.

Schloerke, Barret, Di Cook, Joseph Larmarange, et al. 2025. *GGally: Extension to 'ggplot2'*. https://doi.org/10.32614/CRAN.package.GGally.

Schulz, K. F, D. G Altman, y D. Moher. 2010. «CONSORT 2010 Statement: Updated Guidelines for Reporting Parallel Group Randomised Trials». *BMJ* 340 (mar23 1): c332-32. https://doi.org/10.1136/bmj.c332.

Silva Ayçaguer, Luis Carlos. 1995. «Excursión a la regresión logística en ciencias de la salud». *Madrid: Díaz de Santos*.

Sjoberg, Daniel D., Karissa Whiting, Michael Curry, Jessica A. Lavery, y Joseph Larmarange. 2021. «Reproducible Summary Tables with the gtsummary Package». *The R Journal* 13: 570-80. https://doi.org/10.32614/RJ-2021-053.

Spina, Alexander, Zhian N. Kamvar, y Dirk Schumacher. 2025. *epikit: Miscellaneous Helper Tools for Epidemiologists*. https://doi.org/10.32614/CRAN.package.epikit.

Stevenson, Mark, y Evan Sergeant. 2026. *epiR: Tools for the Analysis of Epidemiological Data*. https://doi.org/10.32614/CRAN.package.epiR.

Tennekes, Martijn, y Marco J. H. Puts. 2023. «cols4all: a color palette analysis tool». *EuroVis (Short Papers)*. https://doi.org/10.2312/evs.20231040.

Therneau, Terry M. 2026. *A Package for Survival Analysis in R*. https://CRAN.R-project.org/package=survival.

Van Cauwenberghe, Lisbeth, y Colin R. Janssen. 2014. «Microplastics in Bivalves Cultured for Human Consumption». *Environmental Pollution* 193 (octubre): 65-70. https://doi.org/10.1016/j.envpol.2014.06.010.

Waring, Elin, Michael Quinn, Amelia McNamara, Eduardo Arino de la Rubia, Hao Zhu, y Shannon Ellis. 2026. *skimr: Compact and Flexible Summaries of Data*. https://doi.org/10.32614/CRAN.package.skimr.

Wickham, Hadley. 2011. «The Split-Apply-Combine Strategy for Data Analysis». *Journal of Statistical Software* 40 (1): 1-29. https://www.jstatsoft.org/v40/i01/.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.

Wickham, Hadley, Mara Averick, Jennifer Bryan, et al. 2019. «Welcome to the tidyverse». *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, y Jennifer Bryan. 2026. *readxl: Read Excel Files*. https://doi.org/10.32614/CRAN.package.readxl.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, y Davis Vaughan. 2026. *dplyr: A Grammar of Data Manipulation*. https://doi.org/10.32614/CRAN.package.dplyr.

Wickham, Hadley, Jim Hester, y Jennifer Bryan. 2026. *readr: Read Rectangular Text Data*. https://doi.org/10.32614/CRAN.package.readr.

Zeileis, Achim, y Torsten Hothorn. 2002. «Diagnostic Checking in Regression Relationships». *R News* 2 (3): 7-10. https://CRAN.R-project.org/doc/Rnews/.